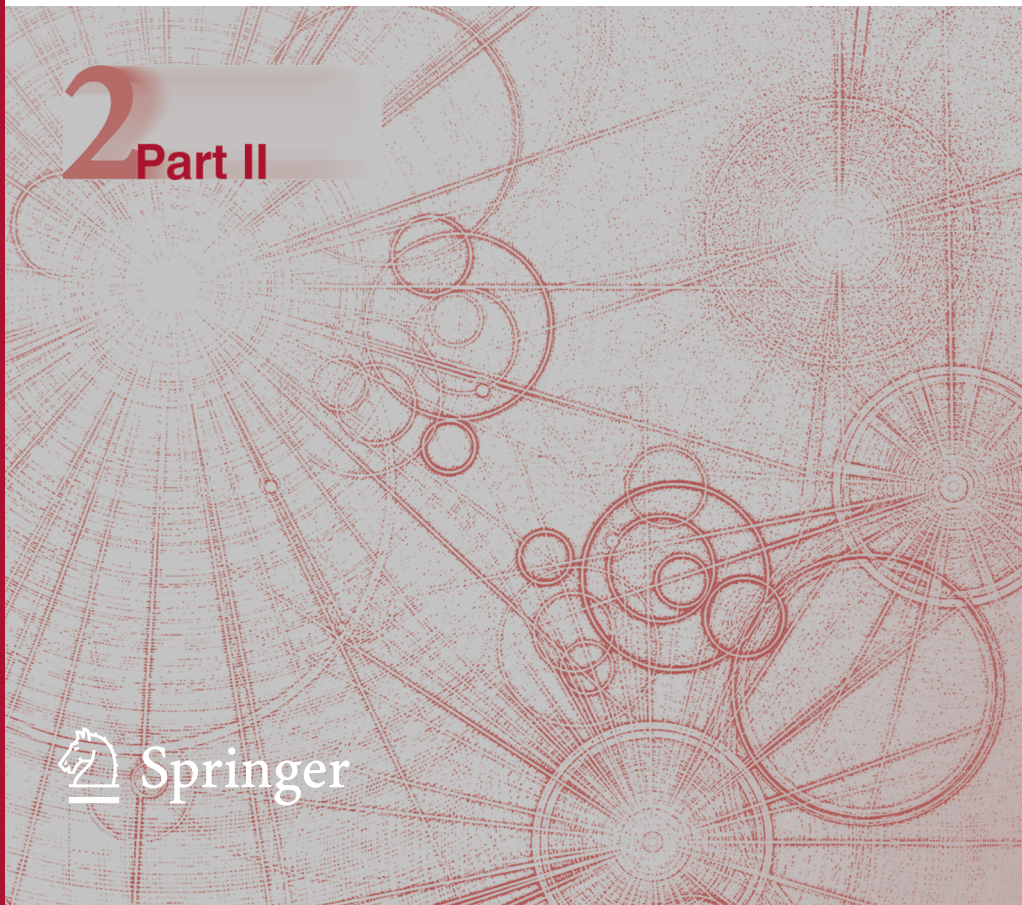
Services Science
LNCS 16321

Marco Aiello · Shuiguang Deng ·
Juan-Manuel Murillo · Ilche Georgievski ·
Boualem Benatallah · Zhongjie Wang (Eds.)

Service-Oriented Computing

23rd International Conference, ICSOC 2025
Shenzhen, China, December 1–4, 2025
Proceedings, Part II



Springer

Lecture Notes in Computer Science **16321**

Founding Editors

Gerhard Goos
Juris Hartmanis

Editorial Board Members

Elisa Bertino, USA
Wen Gao, China
Bernhard Steffen, Germany
Moti Yung, USA

Services Science

Subline of Lecture Notes in Computer Science

Subline Editors-in-Chief

Athman Bouguettaya, *RMIT University, Melbourne, Australia*
Michael P. Papazoglou, *University of Tilburg, The Netherlands*

Subline Editorial Board

Boualem Bentallah
Murthy Devarakonda, USA
Carlo Ghezzi, Italy
Chi-Hung Chi, Tasmania
Hani Jamjoom, USA
Ingolf Krueger, USA
Paul Maglio, USA
Klaus Pohl, Germany
Stefan Tai, Germany
Yuzuru Tanaka, Japan
Christopher Ward, USA

More information about this series at https://link.springer.com/bookseries/558

Marco Aiello · Shuiguang Deng ·
Juan-Manuel Murillo · Ilche Georgievski ·
Boualem Benatallah · Zhongjie Wang
Editors

Service-Oriented Computing

23rd International Conference, ICSOC 2025
Shenzhen, China, December 1–4, 2025
Proceedings, Part II

Springer

Editors
Marco Aiello
University of Stuttgart
Stuttgart, Germany

Juan-Manuel Murillo
University of Extremadura
Badajoz, Spain

Boualem Benatallah
Dublin City University
Dublin, Ireland

Shuiguang Deng
Zhejiang University
Hangzhou, China

Ilche Georgievski
University of Stuttgart
Stuttgart, Germany

Zhongjie Wang
Harbin Institute of Technology
Harbin, China

ISSN 0302-9743 ISSN 1611-3349 (electronic)
Lecture Notes in Computer Science
ISBN 978-981-95-5014-2 ISBN 978-981-95-5015-9 (eBook)
https://doi.org/10.1007/978-981-95-5015-9

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Singapore Pte Ltd. 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

If disposing of this product, please recycle the paper.

Preface

The 23rd International Conference on Service-Oriented Computing (ICSOC 2025) took place from December 1 to 4, 2025, in Shenzhen, China. ICSOC has long been established as the premier international forum for researchers, practitioners, and industry experts in the field of service-oriented computing and related paradigms. This year's edition continued this tradition, while further strengthening the conference's scientific profile and focus.

To sharpen the scope of ICSOC for the 23rd edition, we applied stricter criteria to ensure that submissions were clearly aligned with the field of service-oriented computing. As a result, more than 145 papers, primarily concerned with areas such as federated learning, recommendation systems, operating systems, and networking systems, were desk-rejected. This deliberate effort allowed us to maintain a coherent and strong technical program.

The submission process was organized in four thematic tracks, reflecting the breadth and depth of the SOC community: Service-Oriented Technology Basics and Trends, chaired by Chang-Ai Sun; AI for Services and as-a-Service, chaired by Flavio De Paoli; Novel Service Frameworks for Cloud Continuum and Smart Environments, chaired by Guadalupe Ortiz; and Emerging Technologies, chaired by Schahram Dustdar. Additionally, the early submission track received a set of papers, of which 15 were recommended for revision and resubmission to the regular track.

The regular track received around 400 submissions, of which 219 were selected for review. The review process was conducted under a double-blind policy. Papers that were not anonymous or violated formatting policies, mostly due to length, were also desk-rejected. Each paper received at least two reviews, with the vast majority receiving three. Senior Program Committee members oversaw the process, coordinated reviewer discussions, and provided a meta-review to ensure fairness, consistency, and high-quality feedback.

Following this rigorous evaluation process, 42 full papers were accepted (20% acceptance rate), with an additional 21 (10%) accepted as short papers. This highly selective process yielded a strong and diverse program, representing the very best work in service-oriented computing and establishing ICSOC 2025 as one of the premier events and forums in the field.

A dedicated Artifact Evaluation track was introduced with the goal of enhancing the replicability of the results and reliability of the evaluations presented in the papers. For the first time in ICSOC history, authors of accepted papers were invited to submit companion artifact description papers (e.g., datasets, code, supplementary materials) to undergo reproducibility and verification. The call received 14 artifact submissions. These artifacts were evaluated by the Artifact Evaluation Committee on criteria such as clarity of instructions, executability, provenance of data, and ease of replication. This new track, chaired by Victoria Degeler, underscores our commitment to scientific rigor, openness,

and building a stronger culture of reproducibility in service-oriented computing, in order to enhance the credibility of the community.

We would like to extend our heartfelt gratitude to the track chairs, senior program committee members, and reviewers for their dedication and professionalism in conducting a fair and constructive review process. Our thanks also go to the authors, whose submissions are the lifeblood of the conference, and to all participants for contributing to the continued vitality and excellence of the ICSOC community.

September 2025

Marco Aiello
Shuiguang Deng
Juan-Manuel Murillo
Ilche Georgievski
Boualem Benatallah
Zhongjie Wang

Organization

Honorary General Chair

Xiaofei Xu	Harbin Institute of Technology, China

General Chairs

Boualem Banatallah	Dublin City University, Ireland
Zhongjie Wang	Harbin Institute of Technology, China

Program Committee Chairs

Juan Manuel Murillo	University of Extremadura, Spain
Marco Aiello	University of Stuttgart, Germany
Shuiguang Deng	Zhejiang University, China

Area Chairs

Chang-Ai Sun	University of Science and Technology Beijing, China
Flavio De Paoli	Universita di Milano-Bicocca, Italy
Guadalupe Ortiz	University of Cádiz, Spain
Schahram Dustdar	Vienna University of Technology, Austria

Industry Chairs

Bo Hu	Shenzhen Yihuo Technology Co., Ltd., China
Hakim Hacid	Technology Innovation Institute, UAE
Liang Qiu	Sangfor Co., Ltd., China

Publication Chair

Ilche Georgievski	University of Stuttgart, Germany

Workshop Chairs

Jianwen Su	University of California, Santa Barbara, USA
Sami Yangui	LAAS-CNRS, France
Xiao Xue	Tianjin University, China

Symposium Chairs

Michael Sheng	Macquarie University, Australia
Yingjie Wang	Yantai University, China

Tutorial/Panel Chairs

Fabio Casati	ServiceNow, Switzerland and University of Trento, Italy
Fei Teng	Southwest Jiaotong University, China
Walid Gaaloul	Télécom SudParis, Institut Polytechnique de Paris, France

PhD Symposium Chairs

Cesare Pautasso	Università della Svizzera Italiana, Switzerland
Pengcheng Zhang	Hohai University, China

Demonstrations Chairs

Carlos Canal	University of Málaga, Spain
Jian Wang	Wuhan University, China

Reproducibility Chair

Victoria Degeler	University of Amsterdam, The Netherlands

Publicity, Web and Social Presence Chairs

Buqing Cao	Hunan University of Technology, China
Flavia Monti	Sapienza Università di Roma, Italy
Junna Zhang	Henan Normal University, China
Li Kuang	Central South University, China
Mahi Begoug	University of Quebec, Canada

Web Site Chairs

Shuang Yu	Harbin Institute of Technology, China
Xiao Wang	Harbin Institute of Technology, China

Local Organizing Chairs

Kai Wang	Harbin Institute of Technology, China
Xiang He	Harbin Institute of Technology, China
Yutao Ma	Huazhong Normal University, China

Sponsorship Chair

Zhiying Tu	Harbin Institute of Technology (Weihai), China

Steering Committee Liaison Officer

Liang Zhang	Fudan University, China

Steering Committee

Jian Yang	Macquarie University, Australia
Antonio Ruiz Cortez	University of Seville, Spain

Athman Bouguettaya	University of Sydney, Australia
Boualem Benatallah	Dublin City University, Ireland
Liang Zhang	Fudan University, China
Massimo Mecella	Sapienza Università di Roma, Italy
Winfried Lamersdorf	University of Hamburg, Germany

Senior Program Committee

Antonio Ruiz-Cortés	University of Seville, Spain
Athman Bouguettaya	University of Sydney, Australia
Brahim Medjahed	University of Michigan-Dearborn, USA
Carlos Canal	University of Málaga, Spain
Cesare Pautasso	University of Lugano, Switzerland
Cinzia Cappiello	Politecnico di Milano, Italy
Claus Pahl	Free University of Bozen-Bolzano, Italy
Farouk Toumani	Université Clermont Auvergne, France
Flavio De Paoli	Università di Milano-Bicocca, Italy
François Charoy	Université de Lorraine, France
Jian Yang	Macquarie University, Australia
Jianwei Yin	Zhejiang University, China
Jianwen Su	University of California, Santa Barbara, USA
Khalil Drira	LAAS-CNRS, France
Liang Zhang	Fudan University, China
Lina Yao	CSIRO and University of New South Wales, Australia
Lionel Seinturier	University of Lille, France
Manfred Reichert	University of Ulm, Germany
Manuel Resinas	University of Seville, Spain
Massimo Mecella	Sapienza Università di Roma, Italy
Michael Sheng	Macquarie University, Australia
Mathias Weske	HPI, University of Potsdam, Germany
Mohamed Jmaiel	University of Sfax, Tunisia
Mohand-Saïd Hacid	Université de Lyon 1, France
Naouel Moha	Université du Québec, Canada
Qi Yu	Rochester Institute of Technology, USA
Salima Benbernou	Université Paris Cité, France

Schahram Dustdar	Vienna University of Technology, Austria
Shangguang Wang	Beijing University of Posts and Telecommunications, China
Stefan Tai	TU Berlin, Germany
Walid Gaaloul	Télécom SudParis, France
Xumin Liu	Rochester Institute of Technology, USA

Program Committee

Abdulatif Alabdulatif	Qassim University, Saudi Arabia
Abdullah Alfazi	Prince Sattam bin Abdulaziz University, Saudi Arabia
Abdulwahab Aljubairy	Umm Al-Qura University, Saudi Arabia
Adel N. Toosi	Monash University, Australia
Adnan Mahmood	Macquarie University, Australia
Ahoud Alhazmi	Umm Al-Qura University, Saudi Arabia
Alessandro Aldini	University of Urbino Carlo Bo, Italy
Alex Norta	Tallinn University of Technology, Estonia
Alexander Lazovik	University of Groningen, Netherlands
Alexandre da Silva Veith	Nokia Bell Labs, Belgium
Andrea Delgado	Universidad de la República, Uruguay
Andreas S. Andreou	Cyprus University of Technology, Cyprus
Antonio Brogi	Università di Pisa, Italy
Asma Alkalbani	University of Technology and Applied Sciences, Oman
Azadeh Ghari Neiat	University of Queensland, Australia
Bing Huang	University of Sydney, Australia
Chen Ding	Toronto Metropolitan University, Canada
Chouki Tibermacine	University of Montpellier, France
Christian Zirpins	Karlsruhe University of Applied Sciences, Germany
Christoph Bussler	Robert Bosch LLC, USA
Chung Lawrence	University of Texas at Dallas, USA
Daniele Di Pompeo	University of L'Aquila, Italy
Dong Yuan	University of Sydney, Australia
Ernesto Pimentel	University of Málaga, Spain
Faiza Belala	University of Constantine 2, Algeria
Feifei Chen	Deakin University, Australia
Fuyuki Ishikawa	National Institute of Informatics, Japan
Gangadharan G. R.	IBM, India

Genoveva Vargas-Solar	Laboratoire d'InfoRmatique en Image et Systèmes d'information, France
Georg Grossmann	University of South Australia, Australia
Gianluigi Zavattaro	University of Bologna, Italy
Giovanni Quattrocchi	Politecnico di Milano, Italy
Gregorio Díaz	University of Castilla-La Mancha, Spain
Hai Dong	RMIT University, Australia
Hanchuan Xu	Harbin Institute of Technology, China
Hayet Brabra	Télécom SudParis, France
Ilche Georgievski	University of Stuttgart, Germany
Ismael Bouassida Rodriguez	University of Sfax, Tunisia
Javier Berrocal	University of Extremadura, Spain
Jian Yu	Auckland University of Technology, New Zealand
Jiuyun Xu	China University of Petroleum, China
Joao Eduardo Ferreira	University of São Paulo, Brazil
Jonathan Heiss	TU Berlin, Germany
Jose Garcia-Alonso	University of Extremadura, Spain
José María García	University of Seville, Spain
Juan Boubeta-Puig	University of Cádiz, Spain
Jun Han	Swinburne University of Technology, Australia
Jun Shen	University of Wollongong, Australia
Kais Klai	Sorbonne Paris North University, France
Lars Braubach	City University of Bremen, Germany
Lars Moench	University of Hagen, Germany
Lianyong Qi	Qufu Normal University, China
Luciano Baresi	Politecnico di Milano, Italy
M. Mustafa Rafique	Rochester Institute of Technology, USA
Marco Comuzzi	Ulsan National Institute of Science and Technology, South Korea
Michael Mrissa	InnoRenew CoE, University of Primorska, Slovenia
Michele Tucci	University of L'Aquila, Italy
Mingyi Liu	Harbin Institute of Technology, China
Mohamed Graiet	University of Monastir, Tunisia
Mohamed Sellami	Télécom SudParis, France., France
Mohamed Wiem Mkaouer	University of Michigan-Flint, USA
Monica Vitali	Politecnico di Milano, Italy
Nanjangud Narendra	Ericsson, India
Nawal Guermouche	INSA of Toulouse, LAAS-CNRS, France
Niranjana Deshpande	Rochester Institute of Technology, USA
Nora Faci	Université Lyon 1, France
Onyeka Ezenwoye	Augusta University, USA

Pablo Fernandez	University of Seville, Spain
Philippe Lalanda	University of Grenoble, France
Philippe Massonet	CETIC, France
Philippe Merle	Inria, France
Qiang He	Swinburne University of Technology, Australia
Ricardo Pérez-Castillo	University of Castilla-La Mancha, Spain
Rik Eshuis	Eindhoven University of Technology, Netherlands
Sajib Mistry	Curtin University, Australia
Sami Yangui	LAAS-CNRS, France
Sebastian Werner	TU Berlin, Germany
Sheik Mohammad Mostakim Fattah	Curtin University, Australia, Australia
Shuang Wang	Southeast University, China
Sira Yongchareon	Auckland University of Technology, New Zealand
Slim Kallel	University of Sfax, Tunisia
Stefan Schulte	TU Hamburg, Germany
Stefanie Rinderle-Ma	Technical University of Munich, Germany
Subhash Sagar	UIT University, India
Sumaira Sultan Minhas	University of Manchester, UK
Talal H. Noor	Taibah University, Saudi Arabia
Tetsuya Yoshida	Nara Women's University, Japan
Uwe Breitenbücher	Reutlingen University, Germany
Uwe Zdun	University of Vienna, Austria
Valeria De Castro	Universidad Rey Juan Carlos, Spain
Weiliang Zhao	Macquarie University, Australia
Weiping Li	Peking University, China
Weishi Shi	University of North Texas, USA
Xiang Fu	Hofstra University, USA
Xianzhi Wang	University of Technology Sydney, Australia
Xiao Liu	Deakin University, Australia
Xuyun Zhang	Macquarie University, Australia
Yang Zhang	Macquarie University, Australia
Yanjun Shu	Harbin Institute of Technology, China
Ying Li	Zhejiang University, China
Zhangbing Zhou	China University of Geosciences (Beijing), China
Zheng Song	University of Michigan at Dearborn, USA
Zhiyong Feng	Tianjin University, China
Zhizhong Liu	Henan Polytechnic University, China

Additional Reviewers

Abu-Shaira Mohammad
Akbar Telikani
Anh-Tu Hoang
Anuradha Singh
Asma Alkalbani
Chaimaa Zidane
Charles Liu
Ebaa Alnazer
Fabian Stricker
Fahmida Islam
Fateh Latreche
Felix Schumann
Gaurav Pokharkar
Giuseppe De Palma
Guosheng Li
Hailiang Zhao
Henryk Mustroph
Hosni Ben Douma
Huan Wang
Janik-Vasily Benzin
Jiahui Bai
Jianyi Zhou
Johannes Löbbecke
José Antonio Peregrina Pérez
Juan Luis Herrera
Kamel Boukhelfa
Matteo Trentin
Matthias Ehrendorfer
Michel Kunkler
Mohamed Brahim
Mohamed Lamine Kerdoudi
Mohammad Abu-Shaira
Mona Raoufi
Muhammad Naeem
Muhong Huang
Nan Yang
Okba Tibermacine
Rachida Boucebsi
Robin Pesl
Sergio Laso
Siddhi Baravkar
Summit Shrestha
Sundas Sundas
Wided Meflah
Xin Sun
Xingchen Hu
Xu Bai
Yanli Li
Yunkun Wang
Zhengquan Li
Zhikang Feng
Ziru Niu

Contents

Software as a Services and Cloud Infrastructure

AI-Driven and QoS-Aware Orchestration of Interdependent Services in the Cloud Continuum 3
Henda Sfaxi, Dia Jean Cédric Sanou, Imene Lahyani, Sami Yangui, and Mohamed Jmaiel

Integrating Terms of Service and Service Level Agreements for Automating Cloud Service Management 19
Elena Molino-Peña, José María García, and Antonio Ruiz-Cortés

SDS-Cloud: Secure Deduplication and Sharing Without Third-Party Authorities for Clouds 35
Yuhan Chen, Peng Liu, Lie Wang, Shanyu Gan, and Danping Yang

iSubscription: Bridging the Gap Between Contracts and Runtime Access Control in SaaS 51
Alejandro García-Fernández, José Antonio Parejo, and Antonio Ruiz-Cortés

OSAF: Open Service-Available First Routing Mechanism for Computing Power Network 67
Yunkang Zhang, Qiang Wu, Ran Wang, Jie Haou, Hao Liu, and Yiyun Xu

A-MINT: An LLM Pipeline for Automated Modeling of iPricings from SaaS Pricing Pages 83
Francisco Javier Cavero, José Antonio Parejo, Juan C. Alonso, and Antonio Ruiz-Cortés

LightLoader: Accelerate Python FaaS Cold-Start via Multi-level Source Code Optimization 92
Pengyu Chen, Wei Chen, Guoquan Wu, and Jun Wei

Bridging Service Diversity and CPU Heterogeneity Through Program Similarity-Driven Scheduling 107
Jiayin Luo, Yuxin Ma, Xinkui Zhao, Wei Zhou, and Jianwei Yin

Optimizing QoS Fulfillment of Drone Services 122
Syeda Amna Rizvi and Athman Bouguettaya

DIJS: A Dual Interference-Aware Job Scheduling Framework for Co-located Data Centers 138
Qin Hua, Shiyou Qian, Yufeng Deng, Kaixuan Zhang, Dingyu Yang, Jian Cao, and Guangtao Xue

QONNECT: A QoS-Aware Orchestration System for Distributed Kubernetes Clusters 149
Haci Ismail Aslan, Syed Muhammad Mahmudul Haque, Joel Witzke, and Odej Kao

Privacy, Security and Trust

Towards Trusted Service Monitoring: Verifiable Service Level Agreements 159
Fernando Castillo, Eduardo Brito, Sebastian Werner, Pille Pullonen-Raudvere, and Jonathan Heiss

A Service-Oriented Digital Twin Architecture for Seamless Reliability Verification of IoT Systems 175
Janik-Vasily Benzin, Jürgen Mangler, and Stefanie Rinderle-Ma

Detecting AI-Assisted Tampering in Crowdsourced IoT Service Trust Information 191
Thilina Lokuruge and Athman Bouguettaya

LDP-QWSP: A General Local Differential Privacy Framework for QoS-Based Web Service Prediction 207
Fuchang Luo, Haonan Wu, Shunshun Peng, Quanwang Wu, Hongbing Wang, Mengmeng Yang, and Taolin Guo

Privacy-Preserving Revenue Prediction in Service-Oriented Industrial Supply Chains 216
Xianghui Hu, Kai Di, Zhenyu Wang, Xinran Zhuang, Yichuan Jiang, Hang Liu, and Pan Li

SLIE: A Secure and Lightweight Cryptosystem for Data Sharing in IoT Healthcare Services 225
Ha Xuan Son, Nguyen Quoc Anh, Phat T. Tran-Truong, Le Thanh Tuan, and Pham Thanh Nghiem

Services and Business Processes

P-MDP: A Framework to Optimize NFPs of Business Processes in Uncertain Environments 237
Jun Peng, Jingwei Zhu, Liang Zhang, and Hong-Linh Truong

Business Process Discovery Through Agentic Generative AI 253
Pierre Lindenberg, Indika Kumara, Joshua Owotogbe, Willem-Jan van den Heuvel, and Damian Andrew Tamburri

Usage Control for Process Discovery Through a Trusted Execution Environment 268
Valerio Goretti, Sabrina Kirrane, and Claudio Di Ciccio

GOProQ: A Graphical Query Language for Object-Centric Process Analysis 287
Gyunam Park, Jan Niklas Adams, and Daniel Schuster

BPMN4BC: A Simple BPMN Extension for Blockchain-Enabled Business Process Modeling 306
Hassan Atwi and Cesare Pautasso

ProcNet: Cross-Modal Process Recommendation via LLMs and GCNs 321
Yumeng Jin, Leilei Lin, Siyu Li, Haipeng Wan, Wangyouqi Geng, and Wenlong Chen

Reliable and Configurable Process Simulations via Probabilistic White-Box Models 337
Francesco Vinci, Gyunam Park, Wil M. P. van der Aalst, and Massimiliano de Leoni

MALL: A Mamba-Based Autoencoder Enhanced by an LLM for Multi-Perspective Business Process Anomaly Detection 353
Zhihao Pei and Guiling Li

Quantum Service Computing

AI-Driven Resource Optimization of Quantum Service Computing 365
Javier Romero-Álvarez, Jaime Alvarado-Valiente, Antonio Jesus Sanchez-Gil, Enrique Moguel, and Jose Garcia-Alonso

DeepQUBO: Quantum-Optimized Route Planning for Carpooling Service 380
Zifeng Liu, Yuzhuo Zhao, Xiaofeng Gao, and Kun Tian

Industrial Papers

IoT and LLM Supported Digital Twin Platform: A Case Study on a Net Zero Oriented Building 399
Jiaqi Li, Xingchen Hu, and Jun Shen

Edge-Native Lightweight Model Design and Scheduling for Vehicle Localization Services 415
Jiaqi Chen and Zhicheng Cai

Author Index 425

Software as a Services and Cloud Infrastructure

AI-Driven and QoS-Aware Orchestration of Interdependent Services in the Cloud Continuum

Henda Sfaxi[1], Dia Jean Cédric Sanou[2], Imene Lahyani[1], Sami Yangui[2(✉)], and Mohamed Jmaiel[1]

[1] ReDCAD, ENIS, University of Sfax, 3038 Sfax, Tunisia
[2] LAAS-CNRS, Université de Toulouse, INSA Toulouse, 31400 Toulouse, France
yangui@laas.fr

Abstract. Emerging latency-sensitive and data-intensive applications-such as autonomous unmanned systems, AR/VR, and immersive digital services-demand intelligent orchestration of interdependent services across heterogeneous and geographically distributed cloudâĂŞedge infrastructures. This paper presents a novel AI-driven, QoS-aware orchestration framework designed for the dynamic management of such services within the cloud continuum. The framework introduces a unified modeling approach that captures both service inter-dependencies and shared resource constraints, facilitating SLA- and ALA-driven orchestration. A proactive placement algorithm, enhanced with predictive mechanisms such as workload forecasting and latency trend analysis, ensures compliance with stringent QoS requirements while adapting to the volatility of mobile and heterogeneous environments. The proposed online orchestration system dynamically monitors environmental and workload conditions, enabling real-time service (re)placement to mitigate QoS degradation and optimize resource usage. A proof-of-concept implementation, validated through a wildfire surveillance use case with autonomous drones, demonstrates significant improvements in service availability and responsiveness, advancing the state-of-the-art in continuum computing.

Keywords: Cloud continuum · QoS Management · Service Orchestration · Service Placement

1 Introduction

The proliferation of data-intensive and latency-sensitive applications, including autonomous unmanned systems, augmented and virtual reality (AR/VR), and immersive digital services, has accelerated the evolution of computing infrastructures toward the cloud continuum—a paradigm encompassing heterogeneous, distributed resources spanning cloud, edge, and device layers. These applications inherently depend on interdependent services whose execution demands

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 3–18, 2026.
https://doi.org/10.1007/978-981-95-5015-9_1

not only resource elasticity but also stringent Quality of Service (QoS) guarantees, such as ultra-low latency, high throughput, and uninterrupted availability [13]. The dynamic and volatile nature of continuum environments, driven by fluctuating workloads, resource heterogeneity, and the mobility of both end-users and hosting nodes, renders static or reactive orchestration mechanisms insufficient [3]. Consequently, there is a critical need for orchestration frameworks capable of modeling service interdependencies, predicting environmental changes, and proactively optimizing service placement to ensure adherence to application-level and service-level agreements (ALAs/SLAs).

While recent research has advanced service orchestration and edge-cloud management, most existing approaches either treat services as independent entities, neglecting their structural and resource interdependencies [1,10,14], or rely solely on reactive placement strategies that respond to, rather than anticipate, QoS degradation [7]. Furthermore, mobility-aware orchestration solutions often address node dynamics in isolation, without integrating predictive analytics for workload or mobility trends [4], limiting their capacity to maintain stringent QoS guarantees in volatile environments. This paper addresses these gaps by integrating interdependency modeling, predictive analytics, and proactive decision-making into a unified orchestration framework, enabling resilient and adaptive service management across the cloud continuum.

This paper introduces a novel AI-driven, QoS-aware orchestration framework tailored for the dynamic management of interdependent services across the cloud continuum. The framework first develops a unified model to represent services' structural topologies, functional and non-functional requirements, and shared resource dependencies, with explicit consideration for latency- and bandwidth-critical interactions. Leveraging this model, a QoS-aware placement algorithm is designed to optimize the deployment and migration of individual services and tightly coupled service compositions, ensuring compliance with ALAs and SLAs under dynamic operating conditions. To address the limitations of reactive orchestration, predictive mechanisms, specifically, workload forecasting and network latency variation, are integrated into the orchestration logic. These mechanisms enable the framework to anticipate potential QoS degradation stemming from resource contention or the mobility of drones, vehicles, or nomadic edge servers, thereby facilitating timely, proactive service (re)placement across heterogeneous computing substrates. By employing an online orchestration system, the framework continuously monitors environmental dynamics, dynamically adjusting service placement to optimize resource utilization and uphold stringent QoS guarantees.

The key contributions of this paper are as follows:

1. Propose a unified modeling approach for interdependent services, capturing their topologies, functional and non-functional requirements, and shared resource dependencies, thereby facilitating SLA- and ALA-driven orchestration.

2. Design a QoS-aware placement algorithm for both individual services and service groups, capable of optimizing placement and migration decisions to ensure compliance with application-level and service-level objectives.
3. Incorporate predictive mechanisms, including workload forecasting and network latency variation, into the orchestration logic, enabling proactive service (re)placement to mitigate anticipated QoS degradation.
4. Develop an online orchestration system that continuously monitors and adapts to environmental dynamics, sustaining service availability, and optimizing resource utilization in heterogeneous, mobile, and volatile environments.

The reader should note that a proof-of-concept has been developed and evaluated to implement wildfire surveillance with thermal imaging drones. The results obtained demonstrate that the proposed framework advances the state of the art in intelligent and adaptive service orchestration, demonstrating significant improvements in service availability and responsiveness for next-generation cloud continuum applications.

The rest of this paper is organized as follows: Sect. 2 introduces a motivating use case, that is, wildfire surveillance with thermal imaging drones, and derives the research requirements. Section 3 reviews the related work in the relevant literature in light of the identified requirements. Section 4 describes the proposed architecture that enables the QoS-aware orchestration of interdependent services. Section 5 discusses the associated algorithm for online service (re)placement. Section 6 presents the Proof-of-Concept. Section 7 discusses the validation and the evaluation experiments. Section 8 concludes the paper.

2 Motivating Use Case and Requirements

This section introduces the motivating use case of wildfire surveillance using thermal imaging drones and derives the requirements for an orchestration framework supporting interdependent services.

2.1 Use Case: Wildfire Surveillance Using Thermal Imaging Drones

Drones can be deployed in various contexts to support disaster management, such as detecting and responding to fires. For instance, they can be used in forests to detect wildfires, at industrial sites to identify fire incidents, or on rural highways to monitor fires resulting from accidents.

The use case depicted in Fig. 1 involves a drone patrolling a designated area to detect wildfires and assist authorities in early intervention. The wildfire surveillance application using thermal imaging drones, consists of four interdependent services. The first service (S1) scans the surroundings by capturing a video stream and analyzing it for smoke column patterns. Upon detection, the drone moves closer; service (S2) captures a high-definition image, collects weather data (e.g., wind speed and temperature), and retrieves a smoke sample. Based on this

data, service (S3) performs a more detailed analysis. If a fire is confirmed, the service (S4) transmits all relevant information to the authorities, and the drone can initiate preliminary fire response actions.

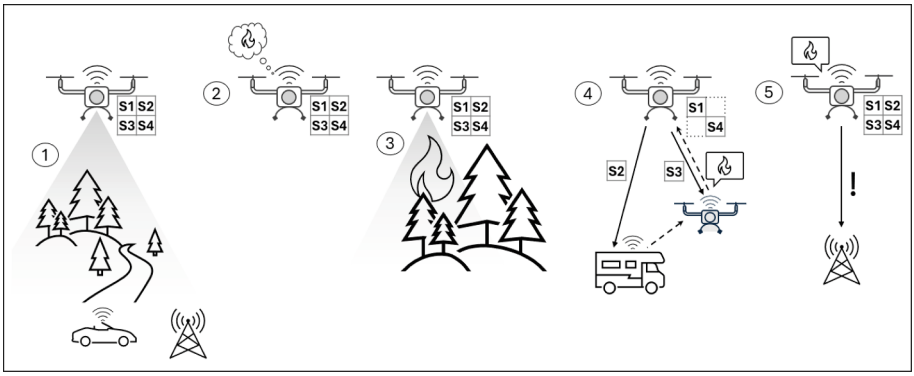


Fig. 1. Wildfire surveillance with thermal imaging drones use case

To support this functionality, the UAV may collaborate with nearby vehicles within the scanning area by offloading certain services—particularly those requiring intensive computation—either when its battery capacity is limited or to collect more accurate data.

Ensuring online service continuity during offloading becomes challenging when services are interdependent, as in the case of S2 and S3, where each has distinct SLA and QoS requirements and depends on the other (S3 uses the output of S2 as its input).

This complexity is further exacerbated by the mobility of drones and potential host nodes (e.g., smart cars or additional UAVs), as well as the dynamic nature of the network, where nodes can join or leave at any time. For instance, if S2 is offloaded to a moving car and S3 to a flying UAV, and these nodes move in opposite directions, communication between S2 and S3 may be disrupted. As a result, meeting ALAs/SLAs and QoS requirements such as latency becomes difficult, potentially causing the wildfire detection application to fail.

2.2 Requirements

To address the limitations of the proposed use case and align with the objectives outlined in the introduction, four key requirements for the orchestration framework are identified:

1. **REQ1: Service Interdependency Modeling**
   A common model is needed to describe service requirements and resource interdependencies post-deployment. In the presented use case, this would enable the drone to offload its interdependent services by specifying only

metadata related to SLAs and ALAs, without having to manage detailed formatting.

2. **REQ2: Proactive QoS-Aware Service (Re)placement**
   An algorithm is necessary to proactively place and trigger service replacement based on predicted QoS degradation. In the proposed use case, QoS degradation may result from the mobility of the drone or the nodes hosting the offloaded services. This proactivity—enabled by QoS degradation prediction—ensures an agile approach that helps drone services meet their requirements while preserving their interdependencies.
3. **REQ3: Service Availability**
   Mechanisms are necessary to ensure service continuity, particularly during offloading, while maintaining QoS and accounting for service interdependencies. For instance, in the presented use case, service continuity may be ensured by migrating the offloaded services to other nodes within the cloud continuum when the offload on the host nodes does no longer meet the SLAs or the QoS requirements.
4. **REQ4: Node Mobility Awareness**
   An adaptive, mobility-aware (re)placement algorithm is needed to account for node movements, especially when services are offloaded to nodes that may leave the coverage area. In the presented use case, the drone is mobile, and potential host nodes (e.g., cars, buses, UAVs) can also move. Services hosted on such nodes may fail to meet latency and other QoS requirements, leading to longer processing times in the best case and disrupting application execution in the worst case.

3 The State of the Art

Recent research has focused on improving service orchestration in distributed cloud-edge environments, primarily targeting QoS compliance and dynamic resource management. Approaches in this space can be broadly categorized into two directions:

- QoS-aware service and task placement strategies seek to optimize placement decisions by leveraging static or real-time QoS metrics. Mobility-aware placement strategies, which account for the movement of edge nodes to ensure service continuity, are encompassed within this category. Nonetheless, these approaches generally lack predictive capabilities to proactively anticipate QoS degradation resulting from mobility.
- Orchestration frameworks oversee the deployment and coordination of services across the edgeâĂŞcloud continuum. This category includes Kubernetes-based extensions that facilitate service placement not only within the cloud but also across the full spectrum of available resources between the cloud and the end user. However, certain frameworks in this domain do not yet incorporate predictive mechanisms capable of simultaneously addressing QoS degradation and mobility challenges.

This paper addresses these limitations by presenting an AI-driven, QoS-aware orchestration solution that unifies these dimensions to enable anticipatory and adaptive service management across the cloud continuum. This Section first describes related work in the relevant literature. Then, it evaluates these works with regard to the requirement identified in Sect. 2.2.

3.1 Related Work

Within this direction, reference [8] integrates heterogeneous resources—clouds, cloudlets, and mobile devices—for interdependent task offloading using a multi-objective Particle Swarm Optimization algorithm, minimizing energy, communication, and computation costs under mobility. Reference [12] proposes a mobility-aware, bus-assisted cloudlet offloading scheme for vehicular networks, modeled as a semi-Markov decision process that reduces computation and communication costs while improving efficiency, though without addressing multi-cloudlet collaboration. Reference [2] introduces a UAVâĂŞedgeâĂŞcloud framework that jointly optimizes latency and energy consumption in a three-tier architecture of UAV swarms, edge servers, and cloud servers, where a Markov-Approximation algorithm improves task placement while reducing overhead. Along similar lines, references [1,10] model tasks and subtasks as DAGs, structuring server resources as a spatial index tree [10] or a B+ tree [1], both enabling parallel scheduling. In [10], execution order follows SubTask Priority (STP), with simultaneous execution of equal-priority subtasks coordinated through game theory; larger subtasks are assigned to MEC servers with lower frequency to save energy, while server energy is predicted via an online Extreme Learning Machine (ELM) model. In [1], task offloading optimization is formulated as an LST-based DDQN problem, supporting parallel execution to minimize latency and energy, with remaining node capacity predicted by an LSTM model. Reference [11] addresses both cross- and intra-application dependencies, representing applications as DAGs and guiding offloading through a predictive model of task completion time. Finally, reference [14] considers service placement and task scheduling between fixed edge servers and end-users, maximizing the hit ratio via DAG-based applications scheduled with a two-level strategy: Bottom Level (BL) ranking within applications and deadline-based ordering across them.

Within the second direction, reference [5] presents Sophos, a network-aware Kubernetes scheduler extension that considers real-time network conditions and application behavior when placing microservice-based applications. However, it overlooks mobility and lacks proactive scheduling mechanisms. Likewise, reference [9] targets critical smart-city applications, extending Kubernetes for dependency-aware scheduling, proactive rescheduling, and runtime monitoring but assumes a static infrastructure. Last but not least, reference [6] introduces ACOA, a distributed QoS-aware orchestration framework modeling inter-component dependencies and enabling per-application scheduler customization. While it supports adaptive deployments, it lacks proactive rescheduling.

3.2 Synthesis

Table 1 depicts the evaluation of the related work with regard to the requirements outlined in Sect. 2.2.

Table 1. Related Work Review. A ✓indicates the criterion is met, while '✕' means it is not addressed.

	REQ1	REQ2	REQ3	REQ4
	Interdependency Modeling	Proactive QoS	Service Availability	Node Mobility
Refs [2,8]	✕	✕	✕	✓
Ref [12]	✕	✕	✓	✓
Refs [14]	✓	✕	✕	✕
Refs [1,10,11]	✓	✓	✕	✕
Ref [5]	✓	✕	✕	✓
Ref [9]	✓	✓	✓	✕
Ref [6]	✓	✕	✓	✓
This paper	✓	✓	✓	✓

Few studies adopt proactive strategies for the placement of interdependent services, as seen in references [1,9–11]. Moreover, only references [6,9,12] explicitly address service availability. In contrast, most works consider service interdependencies within their orchestration models, with the exception of references [2,8,12]. With respect to node mobility—an essential factor in highly dynamic edgeâĂŞcloud environments—references [1,9–11,14] fail to account for it. Finally, the approaches in references [5,6,9] primarily focus on extending Kubernetes as the underlying orchestration engine.

The proposed framework seeks to address these gaps by modeling service interdependencies, handling node mobility, and ensuring service availability through proactive replacement of interdependent services when QoS degradation is predicted.

4 System Design

Figure 2 depicts the high-level architecture that achieves AI-driven and QoS-aware service orchestration.

The proposed architecture supports the dynamic management of interdependent services within the cloud continuum while addressing the requirements identified in Sect. 2.2. It consists of four packages detailed as follows:

1. **SLA Management**: Includes the *SLA Manager* component.
   (a) **SLA Manager**: Collects and formats end-user metrics based on a structured specification defined by the underlying XSD schema, which is designed to be modular and extensible, as illustrated in Fig. 3. This schema defines two core elements:

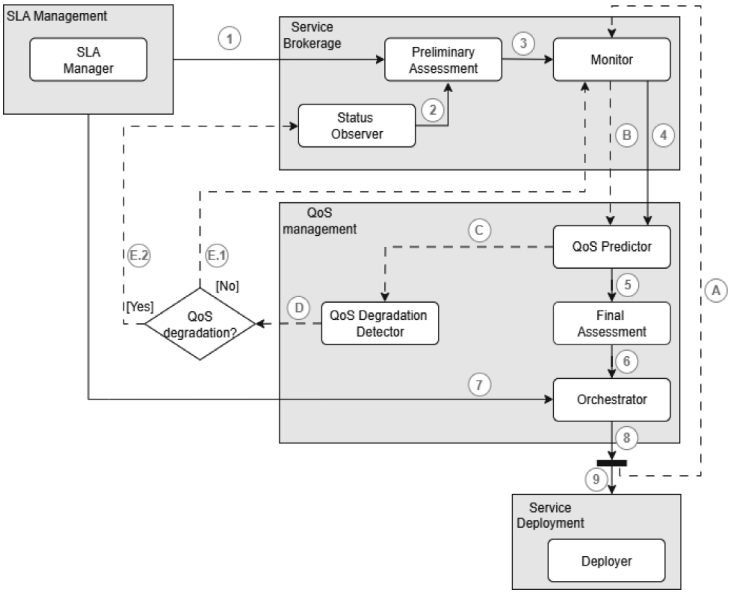


Fig. 2. High-level architecture

- **ALA (Application-Level Agreements)**: Captures measurable application requirements such as end-to-end latency, jitter, availability, packet loss, and session recovery time.
- **Service**: Describes individual services, their resource requirements (CPU, RAM, bandwidth) and dependency relationships with predecessor and successor services.

Based on these definitions, the SLA Manager generates two graphs:

i. A requirements graph incorporating ALAs and SLAs.
ii. A dependency graph capturing data transmission dependencies among user application services.

2. **Service Brokerage**: Comprises three components:
   (a) **Status Observer**: Captures a snapshot of the current network topology, including edge nodes willing to host offloaded services.
   (b) **Preliminary Assessment**: Generates a list of candidate nodes based on the topology snapshot and ALSs/SLAs.
   (c) **Monitor**: Builds a QoS history for the candidate nodes.
      i. In the initial iteration, it forwards candidate data to the *QoS Predictor*.
      ii. In subsequent iterations, it updates the QoS history based on the orchestrator's selected nodes and sends this to the *QoS Predictor*.

3. **QoS Management**: Consists of four components:

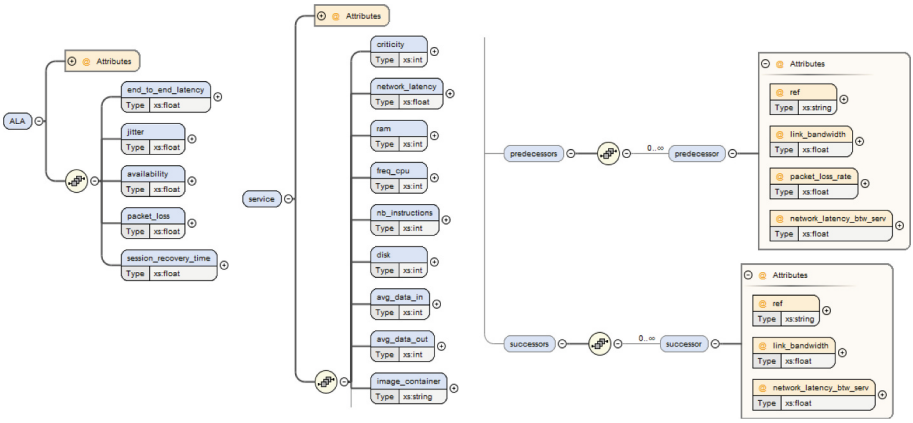


Fig. 3. Graphical view of the XSD for ALAs and services

(a) **QoS Predictor**: Produces QoS predictions based on historical data.
The short-term evolution of critical system parameters including latency, CPU usage, system load, bandwidth, battery level and RAM utilization—is forecast using a machine learning approach.
Due to the heterogeneous units and value ranges of the input parameters, normalization is essential to ensure efficient model training. Accordingly, a linear transformation is performed to scale each feature into the [0, 1] range. Transformer-based architectures are employed due to their proven ability to capture complex temporal dependencies in time series data.
The AI driven -customized model, inspired by the Transformer architecture, is built upon multi-head self-attention blocks, followed by normalization layers, dense layers, and an LSTM block. This hybrid design enables the model to effectively capture complex dependencies between various Quality of Service (QoS). The proposed hybrid architecture, as described in Fig. 4 combines a multi-head attention mechanism with an LSTM layer to capture both contextual and temporal dependencies in QoS time series data. It also incorporates normalization, regularization (Dropout), and projection (Dense) layers to optimize learning and enhance the models generalization capabilities.
The AI-driven QoS predictor successfully predicts the short-term evolution of key QoS parameters, namely CPU usage, RAM utilization, battery level, latency, and bandwidth.

(b) **Final Assessment**: Refines the list of candidates using QoS predictions and the output of the preliminary assessment

(c) **Orchestrator**: Generates the placement plan using the refined list of candidates and the dependency graph produced by the *SLA Manager*.

(d) **QoS Degradation Detector**: identifies QoS degradation based on generated QoS data and predictions. If degradation is detected, a replacement plan is triggered and the process restarts from the *Status Observer*

(arrow E.2 in Fig. 2). Otherwise, the process continues through E.1 to the *Monitor* for ongoing QoS degradation detection.

4. **Service Deployment**: Contains the *Deployer* component.
   (a) **Deployer**: Deploys services on the selected nodes according to the placement plan. The resource allocation and deployment mechanisms are beyond the scope of this paper.

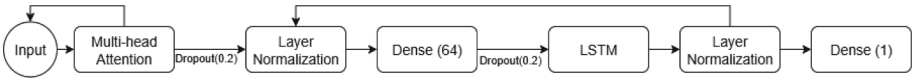


Fig. 4. Customized Model Architecture

5 AI-Driven and QoS-Aware Placement Model

This section presents the model's general algorithm (Sect. 4, Fig. 2), detailed in Algorithm 1 with inline helpers.

Algorithm 1 generates a dependency graph (*dag_dep*) and requirements graph (*dag_req*) from input metrics (lines 2–3). For each time slot t_i in T (line 5), it selects host nodes via CandidateSelection (line 6), deploys the plan (line 7), and monitors QoS with QoSMonitor (line 8). If QoS degradation is detected (`alarm = True`), a re-orchestration loop (line 9) generates a new placement plan (line 11), deploys it (line 12), and monitors QoS again (line 13). This repeats until stability, then the algorithm advances to the next time slot. CandidateSelection obtains the current topology from the *Status Observer* (line 18) and, using the requirements graph (*dag_req*), identifies candidate hosts (*pool_list*) (lines 19–23). It performs a preliminary or refined assessment depending on the existence of a *placement_plan* (lines 20, 22–23). Then, QoS data is collected (line 25), predictions are made (line 26), and the candidate pool is refined (line 27). The *Orchestrator* finally generates the placement plan (line 29). The QoSMonitor updates QoS metrics (line 34), predicts performance (line 35), and detects degradation (line 36), returning `True` early if any node degrades (lines 37–39). Upon degradation, Reorchestrate creates a new dependency graph (line 44) and calls CandidateSelection with it (line 45). The new plan is deployed (line 12) and monitored again (line 13) until stable.

6 Proof-of-Concept

For validation purposes, a running prototype, that implements the wildfire surveillance with thermal imaging drones use case, was developed and evaluated. Figure 5 describes the software architecture of the running prototype.

Algorithm 1. Unified Interdependent Service Orchestration

Require: Performance metrics (ALAs/SLAs), Service set S
Ensure: Continuous service placement with QoS guarantees

$dag_dep \leftarrow$ generateDependencyGraph(metrics) ▷ Build dependency graph
$dag_req \leftarrow$ generateRequirementsGraph(metrics) ▷ Build requirements graph
for each time instant $t_i \in T$ **do**
  $placement_plan \leftarrow$ CandidateSelection($S, dag_dep, dag_req, t_i,$ `null`)
  executePlacementPlan($placement_plan$) ▷ Deployer
  $alarm \leftarrow$ QoSMonitor($placement_plan$)
  while $alarm =$ `True` **do**
    $alarm \leftarrow$ `False`
    $placement_plan \leftarrow$ Reorchestrate($S, dag_req, metrics, t_i, placement_plan$)
    executePlacementPlan($placement_plan$) ▷ Deployer
    $alarm \leftarrow$ QoSMonitor($placement_plan$)
  end while
end for
function CandidateSelection($S, dag_dep, dag_req, t_i, placement_plan$)
  $all_candidates_list \leftarrow \emptyset$
  for each service $s_i \in S$ **do**
    $topology \leftarrow$ getPresenceList(t_i) ▷ Status Observer
    if $placement_plan$ is null **then**
      $pool_list \leftarrow$ poolFilter(dag_req, $topology$) ▷ Preliminary Assessment
    else
      $pool_list \leftarrow$ poolFilter(dag_req, $topology$, $placement_plan$)
    end if
    $qos_history \leftarrow$ getQoSData($pool_list$) ▷ Monitor
    $(pool_list, predictions) \leftarrow$ predictQoS($qos_history$, $pool_list$) ▷ QoS Predictor
    $candidates_list[s_i] \leftarrow$ candidatesFilter($pool_list, predictions$) ▷ Final Assessment
    $all_candidates_list \leftarrow all_candidates_list + candidates_list[s_i]$
    $pool_list \leftarrow pool_list - candidates_list[s_i]$
  end for
  return generatePlacementPlan($\{candidates_list[s_i]\}$, dag_dep) ▷ Orchestrator
end function
function QoSMonitor($placement_plan$)
  $alarm \leftarrow$ `False`
  while $alarm =$ `False` **do**
    for each $(s_i, node_i)$ in $placement_plan$ **do**
      $qos_list \leftarrow$ updateQoS($node_i$) ▷ Monitor
      $prediction \leftarrow$ predict(qos_list) ▷ QoS Predictor
      $alarm \leftarrow$ detectDegradation($prediction$, qos_list) ▷ QoS Degradation Detector
      if $alarm =$ `True` **then**
        return `True` ▷ Exit early if degradation detected on $node_i$
      end if
    end for
  end while
end function
function Reorchestrate($S, dag_req, metrics, t_i, placement_plan$)
  $new_dag_dep \leftarrow$ generateDependencyGraph(metrics, placement_plan)
  return CandidateSelection($S, new_dag_dep, dag_req, t_i, placement_plan$)
end function

Specifically, it implements the services described in Sect. 2.1. The drone is implemented with SunFounder 250 Quadcopter[1] and Raspberry Pi 4 Model B to host and execute services. The latter are implemented as Python RESTful/HTTP services. Service (S1) is deployed on the drone and detects fire using computer vision library called openCV[2]. In case of fire, the drone offloads (S2) and (S3) on neighboring cars implemented SunFounder Picar-X for Raspberry Pi 4[3]. The associated infrastructure is implemented over `drocc`[4], a regional datacenter for research, funded by Occitanie region in France. The different modules (e.g., *Offloading Module*, *SLA Manager*) are implemented as Python RESTful APIs while the *Orchestrator* is implemented with Kubernetes. All are deployed on `drocc`. The considered interdependencies between (S1) → (S2,S3) and between S2 → S3 concerns the network latency that should not exceed 500 ms in total to maintain acceptable QoS. The considered model for QoS degradation prediction are selected offline between the best performing one among a set of pre-trained ESN, GRU, LSTM and RNN models.

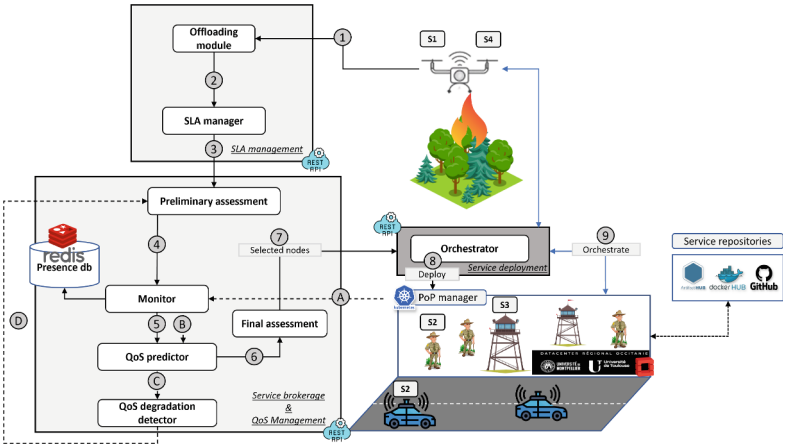


Fig. 5. Software architecture of the wildfire surveillance prototype

The complete source code of the running prototype, as well as, the comprehensive XML descriptor that describes the interdepencies between (S2) and (S3) are available at:[5]. Furthermore, the reader should note that a demo video of the running prototype is available in the same git repository.

Service interdependencies are modeled through a unified JSON descriptor, embedding latency thresholds between tightly coupled services. During the *Preliminary Assessment* (③), this metadata guides the selection of candidate nodes

[1] http://wiki.sunfounder.cc/index.php?title=SunFounder_250.
[2] https://opencv.org/.
[3] https://www.sunfounder.com/products/picar-x?variant=44269165510891.
[4] https://drocc.fr/.
[5] https://github.com/ced-yxos/Code.git.

across the cloud continuum, ensuring latency-aware orchestration and addressing REQ1.

Given the presence of mobile entities, such as drones, connected vehicles and patrol units, within the cloud continuum, the orchestration framework is explicitly designed to be mobility-aware, thus addressing REQ4. The *Monitoring Module* (④) continuously tracks the status and availability of infrastructure nodes using a Redis-based Presence Database. To simulate realistic mobility-induced network behavior, each node is dynamically assigned a latency profile based on empirical measurements from the IMC 2014 dataset "WiFi, LTE, or Both?"[6]. These profiles capture the performance characteristics of Wi-Fi and LTE connections under movement and are updated at runtime using a profiler which through *Linux Traffic Control* command to dynamically apply delay to the nodes to reflect real-world connectivity shifts and potential network degradation. The resulting latency dynamics directly inform the *QoS Predictor* (⑤) and *QoS Degradation Detector* (⑥), which forecast and identify SLA or ALA violations caused by node mobility. Their combined output is used during the *Final Assessment* phase to re-evaluate node suitability and proactively trigger service re-placement when mobility threatens service stability. This mechanism enables the system to continuously adapt to mobility-induced variations in network conditions, thereby fulfilling the requirements of REQ4.

Deployment is managed by the Kubernetes-based *Orchestrator* (⑧), which retrieves containers from trusted repositories and coordinates placement across distributed *Points of Presence (PoPs)*. The *Feedback Loop* (⑨) maintains runtime adaptability, proactively migrating services when nodes exceed latency thresholds or move out of coverage, thereby ensuring real-time resilience and fulfilling REQ4.

7 Validation and Evaluation

The framework's capabilities in proactive QoS-aware placement (REQ2) and service availability (REQ3) are evaluated through an experiment that simulates a dynamic execution environment. A drone offloads tightly coupled services, including a data collection module (S2) and an advanced processing component (S3), to nearby edge nodes while moving through fluctuating network conditions that induce latency variability. Execution times are monitored and compared with latency forecasts from the *QoS Predictor*, which proactively migrates services before SLA violations occur. Furthermore, resilience is further assessed by abruptly disconnecting a service-hosting node to simulate mobility-induced failure.

Figure 6 shows predicted and actual execution times for latency-sensitive services when the system proactively responds to QoS degradation. The top subplot illustrates the raw behavior of the orchestration framework: each latency prediction above the threshold immediately triggers an adaptation. As highlighted in the grey-shaded areas, this produces a dense sequence of adaptations, especially

[6] http://web.mit.edu/cell-vs-wifi/downloads.html.

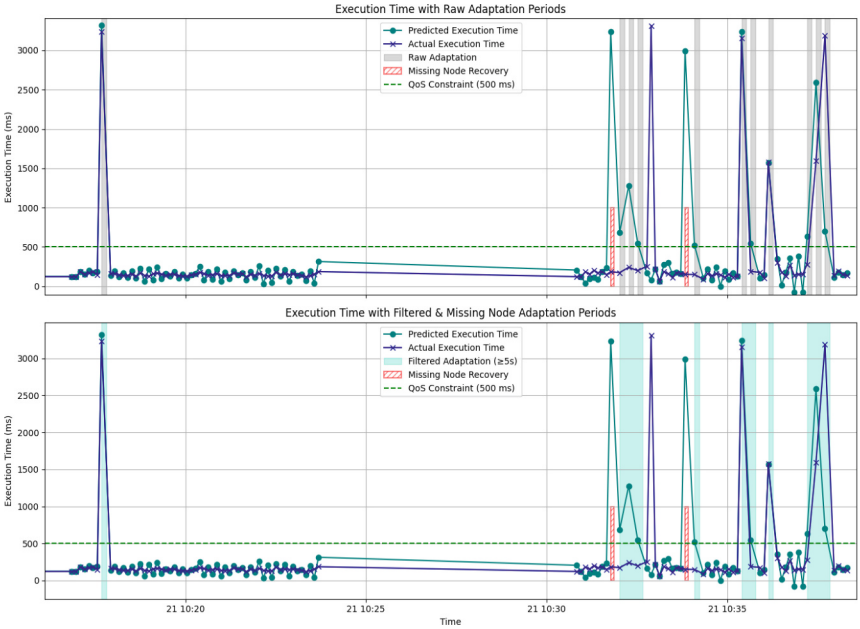


Fig. 6. Evaluation of the proactive and QoS-aware service placement (Color figure online)

in later phases, indicating strong reactivity to predictions. While this prevents SLA violations, it also introduces instability, as consecutive adaptations occur without time for system stabilization.

The bottom subplot applies a filtered view in which only adaptation events separated by at least 5 s are retained, emulating stabilization strategies common in production systems (e.g., cooldown after migration). This yields a smoother adaptation pattern with fewer, better-spaced reconfigurations. Despite fewer adaptations, latency requirements are still satisfied and SLA violations avoided. Thus, the predictive mechanism remains effective, while the cooldown window enhances stability and prevents overreaction to transient conditions.

Overall, the results confirm that the framework can proactively maintain QoS (REQ2). Figure 6 also reports execution times under a simulated node disconnection and recovery. Red-shaded areas mark periods when a hosting node failed or became unreachable, activating the orchestration system's fault-handling mechanisms. Initially, execution times remain below the 500 ms QoS limit (green dashed line), but recovery introduces temporary spikes above this threshold due to migration and redeployment overhead. Despite these transient degradations, the system restores services quickly, with execution times returning to normal shortly after each disconnection. This demonstrates the framework's ability to detect failures in real time and reestablish service availability with minimal downtime, thereby fulfilling REQ3.

8 Conclusion

This paper introduced an AI-driven, QoS-aware orchestration framework for the dynamic deployment and migration of interdependent services across the cloud continuum. By integrating a unified service modeling approach, predictive analytics, and a proactive placement algorithm, the framework ensures sustained QoS compliance and resource efficiency, even in volatile and mobile environments. The proof-of-concept validation, using a wildfire surveillance scenario with autonomous drones, highlights the framework's ability to enhance service continuity and responsiveness while addressing the complex dependencies of distributed applications.

Future research will extend this work by exploring collaborative orchestration across multiple administrative domains, leveraging federated learning and cross-domain policy enforcement to enhance adaptability and interoperability within the cloud continuum. Real-time variations in Service Level Agreements (SLAs) could also be considered to enable more responsive and context-aware decision-making. Additionally, agentic systems could be employed to autonomously generate and adapt placement plans in highly dynamic environments.

References

1. Cai, J., Liu, W., Huang, Z., Yu, F.R.: Task decomposition and hierarchical scheduling for collaborative cloud-edge-end computing. IEEE Trans. Serv. Comput. **17**(6), 4368–4382 (2024)
2. Chen, W., Liu, B., Huang, H., Guo, S., Zheng, Z.: When UAV swarm meets edge-cloud computing: the QoS perspective. IEEE Network **33**(2), 36–43 (2019)
3. Cisneros, J.C., Yangui, S., Hernández, S.E.P., Drira, K.: A survey on distributed NFV multi-domain orchestration from an algorithmic functional perspective. IEEE Commun. Mag. **60**(8), 60–65 (2022)
4. Ma, H., Zhou, Z., Chen, X.: Predictive service placement in mobile edge computing. In: 2019 IEEE/CIC International Conference on Communications in China (ICCC), pp. 792–797 (2019)
5. Marchese, A., Tomarchio, O.: Sophos: a framework for application orchestration in the cloud-to-edge continuum. In: CLOSER, pp. 261–268 (2023)
6. Orive, A., Agirre, A., Truong, H.L., Sarachaga, I., Marcos, M.: Quality of service aware orchestration for cloud-edge continuum applications. Sensors **22**(5), 1755 (2022)
7. Pallewatta, S., Kostakos, V., Buyya, R.: QoS-aware placement of microservices-based IoT applications in fog computing environments. Futur. Gener. Comput. Syst. **131**, 121–136 (2022)
8. Qi, Q., Liao, J., Wang, J., Li, Q., Cao, Y.: Dynamic resource orchestration for multi-task application in heterogeneous mobile cloud computing. In: 2016 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), pp. 221–226. IEEE (2016)
9. Rosmaninho, R., Raposo, D., Rito, P., Sargento, S.: Edge-cloud continuum orchestration of critical services: a smart-city approach. IEEE Trans. Serv. Comput. (2025)

10. Tang, J., Qin, T., Xiang, Y., Zhou, Z., Gu, J.: Optimization search strategy for task offloading from collaborative edge computing. IEEE Trans. Serv. Comput. **16**(3), 2044–2058 (2022)
11. Wang, M., et al.: Dependent task offloading of cross-applications and service caching for end-edge-cloud computing in consumer electronics. IEEE Trans. Consumer Electron. (2025)
12. Wang, Z., Zhong, Z., Ni, M.: A semi-markov decision process-based computation offloading strategy in vehicular networks. In: 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC), pp. 1–6. IEEE (2017)
13. Yangui, S.: Service lifecycle management in the cloud continuum. Ph. D. thesis, INPT Toulouse (2024)
14. Zhang, J., Wang, X., Yuan, P., Dong, H., Zhang, P., Tari, Z.: Dependency-aware task offloading based on application hit ratio. IEEE Trans. Serv. Comput. (2024)

Integrating Terms of Service and Service Level Agreements for Automating Cloud Service Management

Elena Molino-Peña(✉), José María García, and Antonio Ruiz-Cortés

SCORE Lab, I3US Institute, Universidad de Sevilla, Sevilla, Spain
{mmolino,josemgarcia,aruiz}@us.es

Abstract. Customer agreements are fundamental in governing the relationship between service providers and their customers. They are typically composed of interrelated documents that define terms of service, pricing models, and service-level agreements. These documents are managed as separate entities, despite having interdependencies between them. There is a need for a holistic approach to analyse customer agreements as a whole in order to automatically manage and operate services, considering all legal aspects contained in them. In this work, we propose an integrated ontology that provides facilities to semantically describe both terms of service and service-level agreements, enabling operations that not only analyse each part of the customer agreement, but also the interrelated terms, offering a global framework to manage and operate cloud services. We define analysis operations and validate our ontology against cloud service providers, demonstrating their applicability.

Keywords: service management · terms of service · service-level agreements · analysis operations

1 Introduction

The digital transformation of the economy and society is changing how services are provided and data is shared between sectors [7]. As organisations become increasingly interconnected and data-driven, the challenges of managing cloud systems to be secure, efficient, and interoperable are becoming more pronounced, demanding innovative solutions for governance, privacy, and compliance [3].

Typically, providers grant access to functionalities and data through cloud services governed by Customer Agreements (CA), which are legal contracts often composed of additional interrelated documents that describe the Terms of Service (ToS), Service Level Agreements (SLAs), data licences, pricing models, usage, and privacy policies, among others [7,20,21]. All obligations, rights and restrictions, whether related to data access controls or the use of cloud services, must be respected and propagated throughout the digital value chain through these legal contracts, complying with the current European legal framework on

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 19–34, 2026.
https://doi.org/10.1007/978-981-95-5015-9_2

consumer protection and data exchange, as well as contract law and applicable sector-specific regulations [7,27].

These legal documents, typically written in natural language and published online as HTML or PDF files, lack interoperability and cannot be automatically processed [17,23]. This problem is exacerbated by the dematerialisation of contracts and the dispersion of terms across multiple web pages, which make it difficult for average consumers to understand, monitor and enforce contractual terms [18]. Existing approaches in the literature partially address these challenges by focusing on the analysis of certain parts of CA [11,12,17]. However, they do not consider the legal interdependencies that exist between the different documents comprising the CA, possibly rendering some of their results non-compliant with all contracts' terms. For instance, limitations of liability described in the ToS document can restrict compensation offered in the SLA.

In this work, we propose an integrated solution for analysing all terms contained in the different components of a CA by leveraging existing policy ontologies. Building on models such as ODRL and TOSL, our approach enables an interoperable representation of ToS and SLAs, hence enabling analysis operations that consider interrelated terms, providing a holistic framework for the governance of cloud services. In particular, our main contributions are as follows:

- TOSLA, an ontology that extends TOSL to formalise SLA guarantee terms and liabilities, supporting automated reasoning over their structured representations.
- A suite of SPARQL-based analysis operations designed to identify key SLA terms, such as compensations, guaranteed service level values, and exclusions of responsibility; and on the other hand, to analyse aspects that also take ToS into account, such as how the provider can change both ToS and SLAs, or the existence of potentially unfair terms.
- An analysis of three SLAs from commercial SaaS providers demonstrating that TOSLA supports the operationalisation of SLA clauses together with ToS, demonstrating the feasibility and usefulness of our solution.

The remainder of the article is organised as follows. Section 2 discusses the existing approaches for analysing SLAs and the foundational models that our solution relies on. Then, in Sect. 3 we describe our proposed ontology TOSLA, along with our suite of analysis operations in Sect. 4. Section 5 showcases the validation of our proposal, and finally Sect. 6 concludes the article.

2 Background and Related Work

Defined as a W3C Recommendation, the Open Digital Rights Language (ODRL) is an ontology for expressing policies that specify permissions, prohibitions, and obligations [26]. It provides a flexible and interoperable modelling language, defined as the ODRL 2.2 Information Model [15], and a standardised vocabulary, specified as the ODRL 2.2 Vocabulary and Expression [14], to describe the use of digital content and services.

The core of the model describes concepts, entities, and relationships to semantically represent policies that must be composed of a set of rules, each of which expresses whether a given action on a specific asset is permitted, mandatory, or prohibited by a specific responsible party. Rules may also include restrictions such as temporary, space or financial constraints that must be complied with in order to take effect. Furthermore, each type of rule can include associated obligations that must be fulfilled as a condition for the policy to be enforceable, such as a **precondition**, a requirement that must be met before the action is allowed; a **consequence**, a reward for not complying with an obligation; or a **remedy**, an obligation to take corrective action to compensate for failure to comply with a prohibition. Figure 1 shows these interconnected core concepts.

Although the vocabulary provides a solid foundation, offering 72 predefined actions, 34 left operands, and several operators, specific domains often require specialised concepts that go beyond this baseline [14]. As a domain-independent language, ODRL supports extensibility through its profiling mechanism, enabling the incorporation of additional terms tailored to particular areas [19].

2.1 ODRL Profiles

ODRL has emerged as a leading framework for expressing machine-readable normative statements and policies. Even the ODRL terminology has been reused by private initiatives for the development of data spaces for controlled data exchange: *International Data Spaces (IDS)* and *GAIA-X*[1].

Furthermore, ODRL-based profiles[2] have been developed to formalise statements in various domains. These extensions address sector-specific requirements, legal constraints, or technical needs. For instance, *RightsML* has been developed for the management of media rights in the industry, the *Market Data* Profile supports the use of financial data [8,28], and *ODRL-LR* serves the licencing of language resources. Emerging areas such as big data, artificial intelligence and data spaces are addressed by profiles like *Data Usage Control*, *AI Use Policy*, and *ODRL Data Spaces* and *Data Sovereignty*, respectively. Each aligns with EU regulations and the needs of distributed data ecosystems [13,24].

Aligned with our proposal, several extensions focus on modelling ToS, privacy policies, data and cookie policies for access control and compliance assessment of privacy and data protection regulations, such as the GDPR, the Data Governance Act (DGA) and the Unfair Contract Terms Directive (UCTD). *ODRL/L(S)* extends the original ODRL 1.1 by formalising service licences with explicit permissions, guarantees, and limitations, while introducing mechanisms for their automatic composition across interconnected services [10]. *Open Access Control (OAC)* defines access control rules for decentralised environments focused on regulatory compliance [6]. Profiles such as the *Regulatory Compliance* and *Privacy Preferences Ontology Profile (PPOP)* incorporate legal and ethical

[1] https://internationaldataspaces.org/ and https://gaia-x.eu/.
[2] https://www.w3.org/community/odrl/wiki/ODRL_Profiles.

semantics into policy modelling, with the same objective [1,5]. Finally, the *Terms of Service Language (TOSL)* ontology provides a formal model and vocabulary to represent ToS, allowing automatic detection of potentially unfair terms [21].

The TOSL profile is particularly interesting because it introduces new classes for representing liabilities, exclusions, distinctions between the parties (such as the provider and the customer), and dispute resolution methods (such as litigation or arbitration), among other extensions. Regarding the vocabulary, TOSL extends the ODRL core by including actions, asset types, and constraint elements typically used in service environments. Altogether, these enhancements make the profile particularly suitable for modelling SLAs. However, some refinements would be needed to achieve a complete representation of guarantee terms, metrics and limitations of responsibilities commonly found in SLAs.

2.2 Service Level Agreement Models

SLAs regulate service provision by defining a set of guarantees related to specific service properties that the provider is contractually obliged to meet during the contract period [12]. Each guarantee describes the commitments the provider will make to achieve certain Service Level Objectives (SLOs) measured through one or more Service Level Indicators (SLIs). In case of non-compliance, the provider may be liable for compensation (e.g., service credits) [22]. These indicators, defined to measure performance, can be computational, such as availability, and non-computational, such as time to resolve incident tickets.

In addition, the SLA also includes terms for measuring and reporting service breaches, which define how service guarantee breaches are monitored, measured, and reported, as well as who is responsible for these actions. The customer may be required to collect evidence to claim service credits. Finally, the provider's liability exclusions are usually specified. These are situations in which the services are inactive but are not taken into account when calculating the service properties to which the provider is committed, meaning that the provider is exempt from liability [9].

Although previous approaches such as WS-Agreement [2], Linked USDL [12], and Ganapathy et al. [9] attempt to model SLA-related concepts by defining metrics and penalties, they lack sufficient expressiveness to represent various types of clauses defined in CA. In particular, they fail to integrate ToS and SLAs in a unified framework that supports analysis operations.

2.3 Existing Limitations

Although ODRL is a policy language for expressing deontic statements and constraints, it does not inherently address key characteristics of cloud services, such as pricing, SLA, and legal provisions of ToS. For example, SLAs often involve complex concepts such as liability exclusions, indicator-based guarantee obligations, and indemnification clauses, elements that are not explicitly supported by ODRL. Integrating ODRL, TOSL, and the Linked USDL Agreement, we can incorporate the semantics needed to model SLA terms more effectively.

Another challenge is that most approaches that focus on analysing SLAs do not consider other CA components. This results in a fragmented perspective that overlooks critical legal implications normally included in ToS, which are necessary for managing and operating cloud services. Some analysis operations cannot be performed in isolation.

3 Terms of Service Level Agreements (TOSLA)

The main contributions of the Terms of Service Level Agreement (TOSLA) ontology focus on establishing the necessary relationships to reuse existing and well-known ontologies and extend their terminology and instances to accurately model the policies defined by cloud service providers. Since ODRL and its existing profiles do not fully capture the operational semantics required by SLA terms, this extension aims to enable machine-readable SLA between cloud service providers and customers. To this end, it defines permissions, obligations and prohibitions in relation to provider and consumer actions, represents measurable service quality metrics and targets along with their thresholds and conditions, specifies breach management, monitoring, and measurement responsibilities and exclusions and limitations of liability.

The ontology is primarily conceived as a technical data model for researchers, practitioners and developers, enabling automated analysis and serving as a basis for building tools and powering applications to support non-technical stakeholders such as service providers, customers and government agencies. Its potential applications are illustrated through several key use cases: (CU1) ensure interoperability, consistency, and traceability of SLAs across service chains, (CU2) monitoring compliance with SLA commitments at runtime, (CU3) querying obligations, permissions, prohibitions, and restrictions defined in a specific SLA, (CU4) automatically detect internal inconsistencies in SLA terms, such as contradictory metrics or redundant exclusion clauses, (CU5) verifying the alignment of SLA terms with applicable regulations or standards.

3.1 Methodology

In developing TOSLA, we adopted the Open Linking Terms (LOT) methodology [25], which offers a structured approach to ontology engineering. The initial phase, carried out before implementation, focusses on specifying the requirements of the ontology, defining its purpose, and the specific needs it addresses. The aim of TOSLA is to enable the semantic representation of SLAs by reusing and integrating existing ontologies that support the modelling of terms commonly found in policies. This involves identifying relationships among these ontologies and extending them with SLA-specific concepts that cannot be represented otherwise. This approach allows service provider agreements to be fully semantic agreements, which facilitates information retrieval, improves interoperability, and enables automation and reasoning.

LOT emphasises the definition of functional requirements through competency questions (CQ) that align with the goal of the ontology, the vocabulary of the domain and the primary use cases, and also serve as a test suite to evaluate the coverage and adequacy of the ontology. Knowledge acquisition from both ontological and non-ontological sources, such as existing models, vocabularies, relevant standards, and expert knowledge, is central to the design and development of the set of these requirements.

Table 1. SLA Competency Questions

Competency Questions	Domain Concepts
CQ1. Which services are governed by the SLA?	Service, Provider, SLA
CQ2. Which quality of service levels does a service deliver?	QoS, Commitment
CQ3. Which particular properties of a service are guaranteed to have certain values?	SLI, Service Property
CQ4. Which compensations are offered if the guaranteed value of a property is not honored?	Compensation, Penalties
CQ5. Who is the responsible party for enforcing the guaranteed service level values?	Obligation, Provider
CQ6. Who is the responsible party for monitoring and computing the guaranteed values?	Liability, Computation, Evaluation
CQ7. During which period of time a guarantee is offered?	Period Interval
CQ8. How are current values of a service property computed?	Metric Expression

As non-ontological references, we selected NIST Special Publication 500-307 [29], ISO/IEC 19086 [16], and the works of Garcia et al. [12] and Ganapathy et al. [9]. These resources enabled us to discern the typical structure of an SLA and extract its essential vocabulary and entities. Building on this foundation, we then turned to ontological frameworks, such as ODRL, Linked USDL, and TOSL [12,15,21], to represent permissions, obligations, prohibitions, liabilities, and service attributes. Table 1 presents a selection of defined CQs, along with the corresponding derived key concepts. These concepts form the basis for the conceptualisation of the ontology. The result of this phase is the TOSLA Ontology Requirements Specification Document[3] (ORSD) recommended by LOT, which includes the purpose, the scope, intended end users and intended uses of the ontology, and the set of requirements in the form of CQ.

Based on the models and vocabularies of the ODRL [15], TOSL [21] and Linked USDL Agreement [12], as well as the TOSLA ORSD, we conceptualised the TOSLA model using the identified terms (see Figs. 1 and 2). We also formalise CQs as SPARQL queries to retrieve information included in real-world SLA. Additionally, we defined a set of general SPARQL queries to extract the

[3] https://github.com/isa-group/tosla/blob/main/docs/tosla_orsd.md.

obligations, permissions, and prohibitions of the parties involved in the agreement. Although ToS often includes the most unfair terms in an agreement, we reused the Unfair SPARQL queries defined in TOSL [21] to determine whether real-world SLAs also include potentially abusive clauses.

3.2 TOSLA Model

Essentially, a CA comprises a set of terms that define the obligations and rights of the parties, defining actions that may be permitted, mandatory, or prohibited. Service performance expectations and compensation for violation are operational clauses that are typically defined in SLAs. However, the agreement may also include other important provisions, such as complaint procedures, rights to modify services or agreements, limitations of liability, and more. The core concepts for model these terms are presented in Fig. 1.

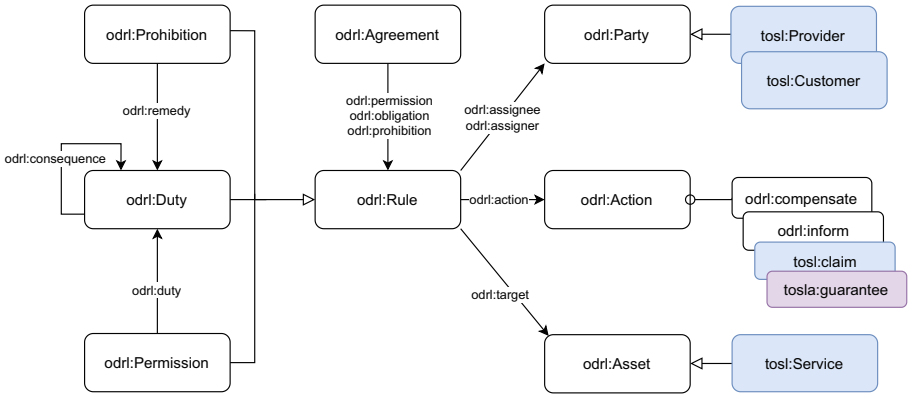


Fig. 1. TOSLA main concepts

As such, a *odrl:Rule* denotes each of these terms defined in the agreement. The entire SLA is defined as a set of instances of rules that may be related to each other as preconditions, consequences, or remedies. Each rule (permission, duty or prohibition) is composed of a responsible *odrl:Party*, an *odrl:Action* and a target *odrl:Asset* to which the action applies, e.g. on the offered service.

In order to model the particular conditions of an SLA, it is necessary to define *odrl:Constraint* within a rule. These constraints serve as prerequisites or preconditions that must be met before the associated action can be executed, thereby regulating when the action becomes active. Additionally, to further detail how an action should be executed, *odrl:Refinement* can be applied. These refinements provide the manner in which the action must be carried out. Whether it involves defining value intervals under different conditions or formulating exclusions through combinations of multiple preconditions, *odrl:LogicalConstraint* can be modelled to express complex constraints.

Both constraints and refinements are specified in the model using a comparison expression, where *odrl:LeftOperand* represents the attribute or property being evaluated, the *odrl:Operator* is the relational operator and the *odrl:RightOperand* is the value to which the left operand is compared (e.g., *leftOperand:payAmount*, *operator:eq*, *rightOperand:100.00*). In an SLA, the left operand usually represents a property of the service, such as availability (a quantitative attribute) or data location (a qualitative attribute), while the valid values in the right operand are bounded by the conditions defined in the policy.

Service properties, particularly quantitative attributes, are often associated with specific measurement methods rather than fixed values. To reflect this, each property is related to a *usdl-agreement:Metric*. A metric must be associated with one or more rules through the service property, which contextualises the metric and defines how it should be interpreted. Each metric must include a clear definition, a formal expression describing its calculation (associated using *usdl-agreement:hasExpression*), and a time interval indicating how often the agreement conditions should be evaluated (via *usdl-agreement:hasMeasuringInterval*), as can be seen in Fig. 2.

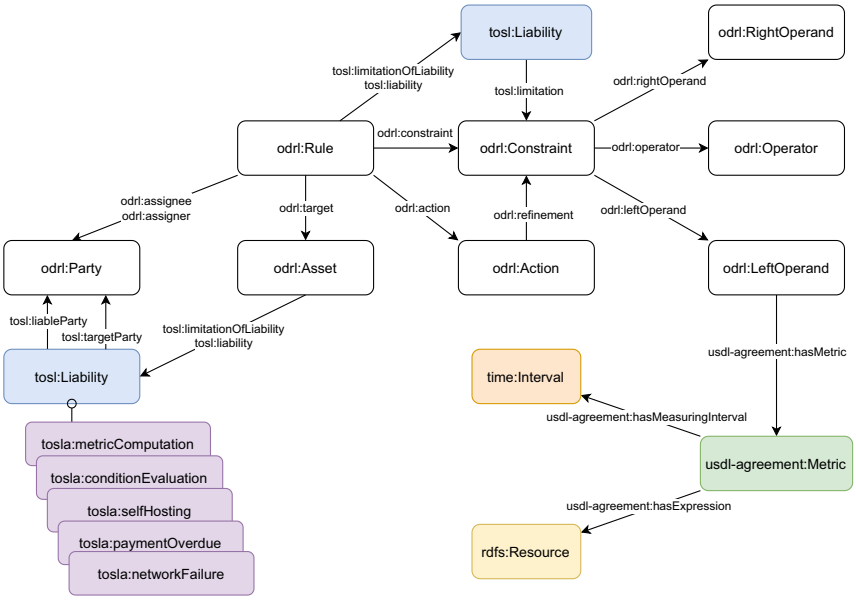


Fig. 2. Agreement constraints and SLA related liabilities

SLAs typically establish both the allocation of responsibilities and the delineation of their scope between the parties involved. Service providers, in particular, often include terms that exclude or limit their liability in specific circumstances that could negatively affect the agreed-upon service quality, such

as scheduled maintenance, overdue payments, or external network failures. An equally important aspect of defining responsibilities is clearly identifying the parties responsible for evaluating, monitoring, and measuring compliance with the stipulated service conditions.

To enable a formal representation of liabilities within a service agreement, a specific *odrl:Policy*, *odrl:Rule*, or *odrl:Asset* can be linked to a liability using *tosl:liability*, or to a limitation or exclusion of liability using *tosl:limitationOfLiability*. These relationships associate a contractual element with a concrete responsibility, represented as an instance of *tosl:Liability*. Each *tosl:Liability* instance captures a specific type of obligation or scenario, such as *tosla:metricComputation*, *tosla:conditionEvaluation*, or *tosla:scheduledMaintenance*, that may arise during the execution of the agreement. These instances also explicitly identify the responsible party (*tosl:liableParty*) and the affected party (*tosl:targetParty*), clarifying who is accountable for fulfilling the responsibility and who is impacted by its execution.

To illustrate how a guarantee-compensation relationship is modelled in practice using TOSLA, consider the following example from Alibaba SLA, formally represented in the Listing 1.1: "*Alibaba guarantees that the monthly uptime percentage for a single instance will be at least 99.975%. Otherwise, a 10% service credit will be granted if the monthly uptime is less than 99.975% but equal to or greater than 99.0%*". Thus, the guarantee term defines the provider's obligation to meet specific performance criteria related to a particular property of the service. Compensation, on the contrary, refers to a secondary obligation where the provider must offer a limited compensation to the customer if certain conditions are met related to the non-fulfilment of agreed service levels.

Listing 1.1. Alibaba instance-level SLA commitment and compensation representation

```
# Asset description
:instanceEcsService a tosl:Service ;
    dcterms:description "Cloud Elastic Compute Service Instance" ;
    dcterms:rightsHolder :alibaba .

# Service property
:monthlyUptimePercentage a odrl:LeftOperand ;
    usdl-agreement:hasMetric :monthlyUptimeMetric .
:monthlyUptimeMetric a usdl-agreement:Metric ;
    qudt:unit <http://qudt.org/vocab/unit#Percent> ;
    usdl-agreement:hasExpression "100 * ((?serviceCycleMinutes - ?downtimeMinutes) / ?
        serviceCycleMinutes)"^^xsd:string ;
    usdl-agreement:hasMeasuringInterval :monthlyInterval .

# Commitment Instance-Level SLA
:uptimeCommitmentInstance a odrl:Duty ;
    odrl:assignee :alibaba ;
    odrl:target :instanceEcsService ;
    odrl:action [
        rdf:value tosla:guarantee;
        odrl:refinement [
            odrl:leftOperand :monthlyUptimePercentage ;
            odrl:operator odrl:gteq ;
            odrl:rightOperand "99.975"^^xsd:decimal ;
        ] ;
        odrl:refinement [
```

```
            odrl:leftOperand odrl:timeInterval;
            odrl:operator odrl:eq ;
            odrl:rightOperand "P30D"^^xsd:duration
        ];
    ];
    tosl:liability [
        a tosl:Liability ;
        dcterms:description "Responsible for tracking and assessing." ;
        rdf:value tosla:conditionEvaluation, tosla:metricComputation ;
        tosl:liableParty :customer
    ] ;
    odrl:consequence :compensationInstance10 .

# Compensation of 10% of the credit for Instance−Level service
:compensationInstance10 a odrl:Duty ;
    odrl:compensatedParty :customer ;
    odrl:assignee :alibaba ;
    odrl:target :instanceEcsService ;
    odrl:action [
        rdf:value odrl:compensate;
        odrl:refinement [
            odrl:leftOperand :serviceCreditPercentage ;
            odrl:operator odrl:eq ;
            odrl:rightOperand "10"^^xsd:decimal ;
        ] ;
    ];
    odrl:constraint [
        a odrl:LogicalConstraint ;
        odrl:and ([
                odrl:leftOperand :monthlyUptimePercentage ;
                odrl:operator odrl:gteq ;
                odrl:rightOperand "99.0"^^xsd:decimal ;
            ][
                odrl:leftOperand :monthlyUptimePercentage ;
                odrl:operator odrl:lt ;
                odrl:rightOperand "99.975"^^xsd:decimal ;
            ])
    ] .
```

4 Analysis Operations

In addition to the TOSLA model we presented before, we aim to provide a comprehensive set of analysis operations to improve the automation degree of SLA-based management, operation, and governance of cloud services. In order to provide a holistic approach to these processes in contrast to state-of-the-art solutions, we introduce in the following analysis operations that not only focus on SLA-specific information, but also consider interrelated information from other parts of the CA, specifically ToS.

When operationalising SLAs, many questions can be directly answered only by analysing the terms contained in the SLA documents. Thus, from the competency questions we discussed in Sect. 3.1, we can directly derive a set of analysis operations that take advantage of the semantic representation of TOSLA. Some of these operations are the following: *Q1.* Given the availability guarantee, what is the maximum continuous service unavailability time before penalties are applied?, *Q2.* What compensation does the client receive when the service has been unavailable for N consecutive minutes? *Q3.* How much continuous service

unavailability time must pass for the client to receive the maximum compensation established in the SLA? To effectively carry out these analyses, different formalisms can be integrated to obtain meaningful results, forming a hybrid solution for service governance. Thus, for operations where temporal relations are present, temporal logic reasoning can be helpful, whereas to identify which party is responsible for metrics measurement, a SPARQL engine can be sufficient.

Even though SLA-specific analysis operations are crucial to manage and operate cloud service ecosystems, ensuring conformance with the actual SLA document, there are several situations where the information contained in SLAs is not sufficient to obtain the appropriate results. In particular, the ToS part of the CA usually contains legal terms, such as liability exemptions, that must be taken into account when answering SLA-related questions. For instance, an operation that requires the set of documents comprising the CA as input is the identification of SLA terms that are invalid, contradictory, or have become ineffective due to the overriding authority of the ToS. Certain operations, although more relevant in some documents than in others, can be applied within the framework of an SLA by interpreting the CA as a set of permissions, obligations and prohibitions, such as the detection of unfair terms.

5 Validation

To validate the effectiveness of TOSLA in modelling and analysing real-world SLAs, we conducted a multi-stage validation process aligned with the requirements identified in Sect. 3. Our dataset consists of three representative SLAs from cloud providers Cedalo[4], Amazon[5], and Alibaba[6] as presented by Ganapathy et al. [9], along with the corresponding Alibaba ToS[7]. These agreements were manually modelled using TOSLA, and the resulting TTL representations have been made available on GitHub[8]. The generated files were validated using a Turtle syntactic validator, a dedicated ODRL validator[9], and an extended version of the TOSL validator, which was enhanced to support TOSLA[10]. These tools employ Shapes Constraint Language (SHACL) to verify semantic constraints and ensure compliance with the ODRL, TOSL, and TOSLA specifications.

Our evaluation approach is based on translating each CQ into a corresponding SPARQL query. These queries collectively form an evaluation framework for the SLAs, which can be executed with respect to their TOSLA representations. Furthermore, to enable a more comprehensive analysis, we consider not only

[4] https://cedalo.com/service-level-agreement/.
[5] https://aws.amazon.com/compute/sla/.
[6] https://www.alibabacloud.com/help/en/legal/latest/elastic-compute-service-service-level-agreement.
[7] https://www.alibabacloud.com/help/en/legal/latest/alibaba-cloud-international-website-product-terms-of-service-v-3-8-0.
[8] https://github.com/isa-group/tosla/tree/main/examples.
[9] https://odrlapi.appspot.com/.
[10] https://github.com/isa-group/tosla/blob/main/validator.

SLAs in isolation, but also the entire CA as a single contractual entity that integrates multiple documents, such as the ToS and the SLAs. This capability highlights one of the main advantages of TOSLA, as it allows agreements to be modelled using the ODRL language, thus enabling a wide range of policies to be represented and multidimensional analysis to be performed.

Three levels of analysis have been conducted to assess the utility of representing CAs as machine-readable TOSLA agreements that enable automation and contract analysis: (1) **Competency Questions Validation** verifying whether SLAs can respond successfully to the eight defined CQs, (2) **Normative Analysis** identifying the obligations, permissions, and prohibitions of the involved parties, and (3) **Fairness Assessment** detecting the presence of potentially unfair clauses within ToS and SLAs using queries proposed in TOSL [21].

As a representative case study, the following sections focus on Alibaba Cloud's CA, combining its ToS with the Elastic Compute Service SLA to demonstrate how CQs can be answered, regulatory elements extracted and potentially unfair terms automatically detected. Some parts of the subsequent analysis are based on the guarantee-compensation relationship modelled in Listing 1.1. The complete experimental results, including the execution of all SPARQL queries on all TOSLA-modelled SLAs, are available in our Python notebook[11].

Competency Questions Validation. Alibaba's SLA defines service guarantees based on monthly uptime percentages, differentiating between instance-level and multi-zone deployments, each with different threshold values across its cloud infrastructure. If these guarantees are not met, Alibaba offers variable service credits, depending on the duration of the downtime. In this context, TOSLA allows you to specify the services governed by the contract (such as line 2 *:instanceEcsService to tosl:Service* in Listing 1.1) and associate them as target assets linked to the guarantee and compensation terms, responding to CQ1.

If we focus on provider commitments, guarantee terms may include a refinement specifying the required quality level at which the asset must be delivered (e.g., *:uptimeCommitmentInstance* in lines 20 and 25 of Listing 1.1). This allows us to obtain SLOs, addressing questions Q2 and Q3 by identifying SLIs and their target values. Each SLI, defined as an odrl:LeftOperand, must be linked to a metric that specifies how the property is measured, often including the expression or formula used for its calculation, information necessary to answer CQ8. In addition, the metric is associated with a measurement interval that indicates how often the property value should be recalculated. This information is also specified in the guarantee as a refinement using odrl:timeInterval, thus addressing CQ7. In the event of non-compliance, the consequence obligation is triggered (*odrl:consequence :compensationInstance10* in line 40 of Listing 1.1), which specifies the exact compensation that the supplier must return to the customer if certain conditions are met (e.g., $99\% \leq \text{uptime} < 99.975\%$, as shown in Listing 1.1), thus answering Q4.

[11] https://github.com/isa-group/tosla/blob/main/bin/TOSLA_Analysis_\penalty-\@Mof_Customer_Agreements.ipynb.

Regarding the party responsible for ensuring compliance and providing compensation, each odrl:Rule is assigned to a party obliged to comply with the rule (*odrl:assignee :alibaba* in Listing 1.1), thus resolving CQ5. However, the responsibility for calculating and monitoring service performance may differ. To specify this responsibility using TOSLA, it is necessary to associate a rule with a defined liability. In this case, *:uptimeCommitmentInstance* is linked to responsibilities such as *tosla:conditionEvaluation* and *tosla:metricComputation*, which are assigned to the party *tosl:liableParty :customer*, answering CQ6.

Normative Analysis. SLAs are typically structured around the obligations of the service provider, including availability levels, response times, and penalties for non-compliance. These provisions focus on the operational and technical responsibilities of the provider. This is also evident in the Alibaba Cloud SLA, which includes nine provider duties but only three customer duties, and grants the provider one permission to unilaterally modify the agreement. The responsibilities are formalised in the TOSLA through permitted, obligatory, or prohibited actions. In this way, it is possible to accurately extract the specific responsibilities assigned to each party under the agreement using queries.

Fairness Assessment. Although the SLA is not usually the component of the CA that contains the largest number of potentially unfair terms, certain types of such terms can be found in the SLAs. These include unilateral modifications to the contract or services, unilateral termination rights, and exclusions of liability for the provider. However, other issues, such as dispute resolution mechanisms, jurisdiction, or applicable law, which may sometimes be considered unfair to the customer, are not typically addressed in the SLA, but rather in the ToS. This highlights the need for a holistic analysis to ensure that both customers and providers are aware of the full range of potentially unfair terms defined in the CA. Running the SPARQL queries proposed in [21] over the Alibaba modelled SLA and ToS, we detected potentially unfair terms related to limitations of liability and unilateral changes in the SLA, as well as choice of law, content removal, contract by use, jurisdiction, termination, and again limitations of liability and unilateral changes in the ToS. Thus, complete analysis reveals the full range of potentially unfair terms present in Alibaba CA. For example, we can see that Alibaba reserves the right to unilaterally modify the terms of both the SLA and ToS without prior notice and that the customer's implied consent constitutes acceptance of the changes, or that dispute resolution for all European customers will be conducted under the laws and courts of England and Wales, rather than the consumer's country, as required by European consumer protection law.

6 Conclusions and Future Work

In this article, we lay out the foundations for a holistic solution to enable a comprehensive representation of CAs that can leverage analysis operations that consider terms of service, service level agreements and other legal clauses in an integrated form. Thus, we propose TOSLA, a semantic framework based on

ODRL and TOSL, to enable a seamless representation of SLAs alongside ToS. TOSLA includes a suite of operations that analyse the information contained in SLAs while also considering interrelated aspects with the rest of the CA. We validate our solution with a set of SLAs from commercial SaaS providers, obtaining valuable insights from the analysis operations evaluated. As future work, we plan to develop a translation system (inspired by [4]) to build a comprehensive dataset that combines SLAs and related ToS. This will allow us to apply more advanced, hybrid reasoning techniques to examine how different terms relate to each other, detect contradictions, compute metrics, among other analysis operations.

Acknowledgments. This publication was supported by the R&D projects PID2021-126227NB-C21, PID2021-126227NB-C22, and PDC2022-133521-I00, funded by MICIU/AEI/10.13039/501100011033/ERDF/EU; TED2021-131023B-C21, funded by MICIU/AEI/10.13039/501100011033/European Union NextGenerationEU/PRTR; and PID2024-155693NB-C44, funded by MICIU/AEI/10.13039/501100011033/FEDER, EU.

References

1. Agarwal, S., Steyskal, S., Antunovic, F., Kirrane, S.: Legislative compliance assessment: framework, model and GDPR instantiation. In: Privacy Technologies and Policy, pp. 131–149. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-02547-2_8
2. Andrieux, A., Czajkowski, K., Dan, A., et al.: Web services agreement specification (WS-Agreement). Global Grid Forum **2** (2004)
3. Camilli, M., Luccioletti, F., Mirandola, R., Scandurra, P.: Integrated QoS- and vulnerability-driven self-adaptation for microservices applications. In: Service-Oriented Computing, pp. 55–71. Springer (2025). https://doi.org/10.1007/978-981-96-0808-9_5
4. Cavero, F.J., Parejo, J.A., Alonso, J.C., Ruiz-Cortés, A.: A-MINT: an LLM pipeline for automated modeling of iPricings from SaaS pricing pages. In: International Conference on Service-Oriented Computing (2025). In press
5. Esteves, B., Asgarinia, H., Penedo, A.C., et al.: Fostering trust with transparency in the data economy era: an integrated ethical, legal, and knowledge engineering approach. In: 1st International Workshop on Data Economy, pp. 57–63 (2022)
6. Esteves, B., Pandit, H.J., Rodríguez-Doncel, V.: ODRL profile for expressing consent through granular access control policies in solid. In: IEEE European Symposium on Security and Privacy (2021)
7. Farrell, E., et al.: European data spaces - scientific insights into data sharing and utilisation at scale. Sci. Analysis Rev. (2023)
8. Fornara, N., Chiappa, A., Colombetti, M.: Using semantic web technologies and production rules for reasoning on obligations and permissions. In: Agreement Technologies, pp. 49–63. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-17294-7_4
9. Ganapathy, D.N., Joshi, K.P.: A semantically rich framework to automate cloud service level agreements. IEEE Trans. Serv. Comput. **16**(1), 53–64 (2023)

10. Gangadharan, G.R., Weiss, M., D'Andrea, V., Iannella, R.: Service license composition and compatibility analysis. In: International Conference on Service-Oriented Computing (ICSOC), pp. 257–269 (2007)
11. García-Fernández, A., Parejo, J.A., Cavero, F.J., Ruiz-Cortés, A.: racing the market: an industry support analysis for pricing-driven devops in SaaS. In: International Conference on Service-Oriented Computing (ICSOC), pp. 260–275 (2025)
12. García, J.M., Fernández, P., Pedrinaci, C., Resinas, M., Cardoso, J., Ruiz-Cortés, A.: Modeling service level agreements with linked USDL agreement. IEEE Trans. Serv. Comput. **10**(1), 52–65 (2017)
13. Golpayegani, D. and Esteves, B. and Pandit, H.J. and Lewis, D.: AIUP: an ODRL profile for expressing AI use policies to support the EU AI act. In: Posters, Demos, Workshops, and Tutorials of the 20th International Conference on Semantic Systems, vol. 3759 (2024)
14. Iannella, R., Steidl, M., Myles, S., Rodríguez-Doncel, V.: ODRL vocabulary expression 2.2. W3c recommendation, W3C (2018). https://www.w3.org/TR/odrl-vocab/. Accessed 13 July 2025
15. Iannella, R., Villata, S.: ODRL information model 2.2. W3c recommendation, W3C (2018). https://www.w3.org/TR/odrl-model/. Accessed 13 July 2025
16. ISO/IEC JTC 1/SC 38: Information technology-cloud computing-service level agreement (SLA) framework-Part 1: Overview and concepts. Tech. Rep. 19086-1:2016(E), ISO/IEC (2016)
17. Lippi, M., et al.: Claudette: an automated detector of potentially unfair clauses in online terms of service. Artif. Intell. Law **27**(2), 117–139 (2019)
18. Loos, M., Luzak, J.: Update the unfair contract terms directive for digital services. Tech. Rep. PE 676.006, Policy Department for Citizens' Rights and Constitutional Affairs, Directorate-General for Internal Policies (2021)
19. Michael Steidl: Odrl v2.2 profile best practices. Community group final report, World Wide Web Consortium (W3C) (2024). https://w3c.github.io/odrl/profile-bp/. Accessed 13 July 2025
20. Michels, J.D., Millard, C., Turton, F.: An analysis of the standard contracts for 40 cloud computing services. Queen Mary School of Law Legal Studies Research Paper No. 334/2020 (06 2020)
21. Molino-Peña, E., Cruz-Lorite, J.M., García, J.M., Ruiz-Cortés, A.: TOSL: an ontology to detect abusive services. In: Proceedings of the Workshop on ODRL and Beyond: Practical Applications and Challenges For Policy-Base Access and Usage Control (OPAL) (2025)
22. Müller, C., Gutierrez, A.M., Fernandez, P., Martín-Díaz, O., Resinas, M., Ruiz-Cortés, A.: Automated validation of compensable SLAs. IEEE Trans. Serv. Comput. **14**(5), 1306–1319 (2021)
23. Obar, J.A., Oeldorf-Hirsch, A.: The biggest lie on the internet: ignoring the privacy policies and terms of service policies of social networking services. Commun. Soc. **23**(1) (2020)
24. Plaza-Ortiz, I., Muñoz-Arcentales, J.A., Salvachúa, J., Aparicio, C., Huecas, G., Barra, E.: Authentication and authorization in Data Spaces: A relationship-based access control approach for policy specification based on ODRL. In: Proceedings of the Workshop on ODRL and Beyond: Practical Applications and Challenges For Policy-Base Access And Usage Control (OPAL) (2025) relationship-based access control approach for policy specification based on ODRL. In: Proceedings of the Workshop on ODRL and Beyond: Practical Applications and Challenges For Policy-Base Access And Usage Control (OPAL) (2025)

25. Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M., García-Castro, R.: Lot: an industrial oriented ontology engineering framework. Eng. Appl. Artif. Intell. **111**, 104755 (2022)
26. Rodríguez-Doncel, V., Roman, N.: Towards conformance in ODRL 3.0. In: Joint Proceedings of the ESWC 2025 Workshops and Tutorials (2025)
27. Slabbinck, W., Rojas Meléndez, J., Esteves, B., Colpaert, P., Verborgh, R.: Interoperable interpretation and evaluation of ODRL policies. In: The Semantic Web (ESWC), pp. 192–209 (2025)
28. Steyskal, S., Kirrane, S.: If you can't enforce it, contract it: enforceability in policy-driven(linked) data markets. In: 11th International Conference on Semantic Systems, pp. 63–66 (2015)
29. de Vaulx, F., Simmon, E., Bohn, R.: Cloud computing service metrics description. Special Publication (NIST SP) 500-307, National Institute of Standards and Technology, Gaithersburg, MD (2018). Accessed 20 June 2025

SDS-Cloud: Secure Deduplication and Sharing Without Third-Party Authorities for Clouds

Yuhan Chen, Peng Liu(✉), Lie Wang, Shanyu Gan, and Danping Yang

College of Computer Science and Engineering/College of Software, Guangxi Normal University, 15 Yucai Road, Guilin 541000, Guangxi, China
liupeng@gxnu.edu.cn
https://www.gxnu.edu.cn/

Abstract. Deduplication and data sharing are two crucial technologies designed to enhance the storage and sharing efficiency of cloud service providers. However, existing encryption-based solutions face critical challenges that hinder their effectiveness. (1) Some existing encryption algorithms introduce randomness, which reduces deduplication efficiency and hampers data sharing. (2) Other algorithms attempt to balance deduplication and data sharing but rely on third-party authority servers. To address these challenges, we introduce the SDS-Cloud scheme, which reduces storage overhead for cloud services and promotes the sharing of data. This scheme operates independently of third-party authority servers for both deduplication and data sharing while defending against various types of attacks. Specifically, deduplication is achieved using an optimized message-locked encryption scheme. Unlike traditional message-locked encryption, which introduces randomness, our approach maintains deduplication efficiency while preventing brute-force dictionary attacks. Proxy re-encryption is employed to manage data access rights, enabling data sharing without granting the cloud service provider access to the actual data. Moreover, security analysis shows that the proposed scheme meets the required security standards. Furthermore, simulation experiments demonstrate that our scheme outperforms existing approaches in both repetitive data verification and permission management efficiency.

Keywords: Cloud computing · Secure deduplication · Data sharing · Proxy re-encryption

1 Introduction

With the advancement of the cloud service, the volume of data held by individuals and institutions is experiencing explosive growth [1]. Due to the rapid development of internet and cloud storage technologies, users are uploading excessive amounts of data to cloud service provide to alleviate storage burdens and facilitate the sharing of data [3]. Because high-quality academic resources, once lost,

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 35–50, 2026.
https://doi.org/10.1007/978-981-95-5015-9_3

are often difficult to recover. Therefore, users frequently back up their data to cloud servers. Armknecht et al. [2] proposed that by implementing deduplication during the backup process, storage efficiency can be improved by 90%–95%, and even in ordinary file domains, deduplication can enhance storage efficiency by more than 50%. Current mainstream deduplication algorithms have significant limitations: they either rely on third-party authoritative institutions to achieve data sharing or focus solely on optimizing the storage of duplicate data. Both approaches fail to adequately ensure the complete control of data owners [4,5] and the autonomous sharing capabilities of their data.

In recent years, Message-Locked Encryption(MLE)-based deduplication methods have attracted significant attention due to their effectiveness in identifying duplicate data [6,7]. MLE [6] is a symmetric encryption scheme that derives the encryption key from the message itself, enabling identical data to produce the same ciphertext and thus allowing the cloud server to eliminate redundant storage. Guo et al. [7] proposed a secure deduplication and ownership management scheme based on broadcast encryption, where users interact only with the cloud server. Their design proves data ownership through proof-of-work to resist brute-force dictionary attacks and manages ownership using a broadcast mechanism. Zhang et al. [8] presented a deduplication sharing scheme that leverages attribute-based encryption, which not only improves storage efficiency through deduplication but also enhances server performance via re-randomization techniques while supporting secure data sharing.

Current deduplication sharing algorithms face two main issues: (1) The use of random numbers in encryption to prevent data inference attacks hinders the cloud serverâĂŹs ability to identify duplicate files, leading to wasted storage. (2) Most algorithms either rely on third-party authorities for data sharing or focus only on eliminating duplicates [10], failing to give data owners full control and autonomous sharing capabilities. In addition, the frequent backup of data increases storage overhead. Our proposed scheme addresses these limitations.

This paper addresses file-level redundancy and backup duplication caused by diverse users, proposing the SDS-Cloud mechanism for deduplication and secure data sharing. SDS-Cloud can also be applied to domains such as healthcare and IIoT, where storage costs and complex access control are crucial. For instance, in urban rail transit corridors, multiple sensors continuously upload large volumes of redundant environmental data, which is accessed by various stakeholders. SDS-Cloud effectively eliminates redundancy while enabling efficient and secure data sharing.

This article provides a more optimized data management solution through the following three contributions.

1. We are optimizing the data encryption and deduplication processes, which significantly reduces data redundancy and thereby greatly enhances the storage efficiency of cloud servers. This design not only inherits the advantages of the proxy re-encryption algorithm but also incorporates targeted optimizations tailored to the characteristics of the data, offering more efficient and secure support for data sharing and storage in the sector.

2. We propose a proxy re-encryption algorithm based on MLE, which innovatively integrates MLE with proxy re-encryption to achieve an efficient data-sharing mechanism that supports deduplication. Unlike existing solutions, our method eliminates the need for reliance on third-party authoritative institutions, thereby promoting storage efficiency while ensuring complete control and autonomous sharing capabilities for data owners. This approach provides a novel solution for secure, decentralized data sharing.
3. We conducted a comprehensive security analysis and performance evaluation of our scheme using real educational resource datasets and compared it with other existing solutions. The results demonstrate that our scheme outperforms others in terms of computational efficiency and security.

The structure of the rest of the paper is outlined as follows: Sect. 2 provides an overview of the related work. Sections 3 detail the system model and details of our scheme. Security analyses and experiments are addressed in Sects. 4 and 5. Finally, Sect. 6 concludes the paper.

2 Related Work

Encrypted deduplication in cloud servers has attracted significant attention from researchers. Bellare et al. [6] introduced message-locked encryption (MLE), deriving encryption keys directly from the message itself. While effective for deduplication, deterministic encryption is vulnerable to brute-force attacks. To mitigate this, Keelveedhi et al. [13] proposed introducing a key server that adds randomness to the key derivation process. However, this approach suffers from single-point failure and is susceptible to Sybil attacks. Non-deterministic encryption schemes [14], which combine random numbers with message-derived keys, offer improved security but limit deduplication efficiency by restricting key distribution to predefined groups. Key server-based deduplication schemes have also been proposed to enhance storage and interaction efficiency [18]. These schemes, however, require frequent interactions with a centralized server, leading to single-point failure risks and significant communication and computation overhead, which constrain their practicality in large-scale or resource-limited environments. Blockchain-based approaches offer an alternative by enabling decentralized deduplication. Aparna et al. [17] suggested comparing file hashes stored on the blockchain to detect duplicates, but this method cannot deduplicate encrypted files and remains vulnerable to inference attacks. Similarly, Ming et al. [15] proposed using smart contracts to compare file tags. Nevertheless, their approach performs modular exponentiation on hash values, which still allows "guess–compute–compare" attacks by cloud service providers. To effectively defend against inference attacks in encrypted deduplication, it is essential to incorporate randomness or key-based blinding into the deduplication process.

While these secure deduplication schemes optimize storage, they do not support data sharing. Sharing encrypted, deduplicated data is more complex due to the challenge of integrating sharing policies across users. In 2016, the REED

scheme [19] was proposed to enable dynamic access control by encrypting random numbers to prevent MLE ciphertext reconstruction. However, this scheme is vulnerable to malicious users who can store specific data blocks and regain access after permission revocation [9]. In 2019, Yuan et al. [16] introduced a deduplication method incorporating random numbers into encryption. While enhancing security, it reduces deduplication efficiency unless a centralized authority coordinates encryption random numbers, which introduces centralization concerns. Using convergent encryption without random numbers, however, exposes the system to attacks like stub retention. Mingyang et al. [1] proposed a method using a third-party authority to encrypt deduplicated data with random numbers, improving deduplication efficiency. However, reliance on a third party introduces single-point failure risks and undermines data owners' control and autonomy in sharing their data. The method proposed by Di et al. [20], which introduces a third party for data deduplication, poses the risk of privacy leakage to the third party and incurs substantial communication overhead.

3 System Model and Proposed Scheme

This section presents the system model of the proposed SDS-Cloud scheme, which is designed for deployment in resource-sharing environments, as illustrated in Fig. 1. The SDS-Cloud scheme involves four entities: the Initial Data Owner (IDO), the Cloud Service Provider (CSP), the Secondary Data Owner (SDO), and the Data Requester (DR).

IDO: The initial uploader. If the CSP confirms the data is new, the user encrypts it with a random key (derived from a random number) and encrypts that key with data-derived info before uploading both to the CSP.

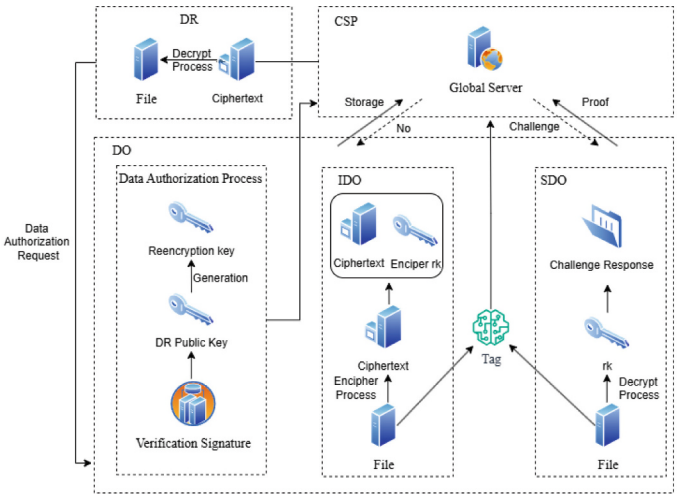


Fig. 1. System Model

SDO: A duplicate uploader. If the CSP detects existing data, the user must complete a challenge to prove ownership. Upon success, they are added to the authorized user list.

CSP: Handles storage, deduplication, access control, and re-encryption for secure sharing.

DR: A requester. With approval from an IDO or SDO, the re-encryption key is sent to the CSP, which generates a shared copy for the DR.

3.1 Overview

SDS-Cloud eliminates data redundancy and supports secure, dynamic operations such as data sharing. It operates in three phases: (1) System Initialization – The DO, DR, and CSP generate key pairs and authenticate to establish secure channels. (2) Storage and Deduplication – The DO encrypts data and generates tags, while the CSP performs deduplication and supports secure sharing via proxy re-encryption. Authorized users can decrypt the data. (3) Data Sharing – Upon request, the DO sends a re-encryption key to the CSP, who re-encrypts data for the DR. This ensures secure, efficient, and owner-controlled data management.

3.2 System Initialize Phase

First, the DO, CSP, and DR generate public parameters, public keys, and private keys using the algorithm predefined by the system. Second, the DO and DR need to perform mutual authentication with the CSP to verify the authenticity of both the users and the CSP. CSP is donate as $C = \{C_1, C_2, ...C_n\}$.DO is donate as $DO = \{DO_1, DO_2, ...DO_n\}$.DR is donate as $DR = \{DR_1, DR_2, ...DR_n\}$ (Table 1).

Table 1. The required symbols in SDS-Cloud are summarized.

Data symbol	Description
PK_x/ SK_x	Public and secret key pair associated with entity x
$ENC(i, j)$	Asymmetric encryption of data j using key i
$SymENC(i, j)$	Symmetric encryption of data j using key i
$SymDEC(i, j)$	Symmetric decryption of ciphertext j using key i
$ReEnc(i, j)$	Re-encryption of data j using re-encryption key i
Sig_i	Digital signature computed over data i
SH	Computation of a short (truncated) hash value
H	Computation of a standard cryptographic hash value
G	One exponentiation operation over the cyclic group G
$ENC(AES/ECC)$	A single AES or ECC encryption operation
$DEC(AES/ECC)$	A single AES or ECC decryption operation

The CSP generates G_1 and G_2 and e, a where G_1 and G_2 are two finite cyclic groups of a large prime P. g is a random generator of G_1, i.e., $g \in G_1$. The bilinear map $e{:}G_1 \times G_1 \rightarrow G_2$ is defined. The CSP selects a random number $r_1 \in Z_p^*$ as the secret key (SK) and computes the public key $PK = g^{r_1}$. The public key PK, along with G_1,G_2, g is published.

If a DO wishes to store data on the CSP or a DR wants to download data from the CSP, they must register with the CSP. The DO or DR first downloads the CSP's public parameters. Then, they select a random number $r_2 \in Z_p^*$ as their private key SK and compute their public key $PK = g^{r_2}$. The DO or DR sends their public key PK and a signature to the CSP. The CSP verifies the correctness of the signature using PK and adds the DO's or DR's PK to the user list.

3.3 Storage and Deduplication Phase

As shown in Fig. 2, the DO generates a tag with a random number for deduplication detection and deletion. Upon receiving the tag, the CSP compares it with existing tags. If no match is found, the DO is considered the IDO and uploads the data. If a match is found, the DO is the SDO, and the CSP challenges the SDO to verify the completeness of the data. Assuming the DO has completed registration and authentication if DO_i wishes to upload a file to C_i, the following steps must be followed:

1. DO generate duplicate verification labels:
   (a) The DO first computes the hash value of the file, denoted as H_{File}, using a hash function. Then, DO selects a random number $r_3 \in Z_p^*$ and calculates the tag for the file as $Tag_{File} = \{g^{r_3}, g^{r_3 * H_{File}}\}$.
   (b) The DO signals the Tag_{File} to generate Sig_T. Subsequently, the DO sends $\{Tag_{File}, Sig_T\}$ to the CSP.
2. CSP Detects duplicate data labels.

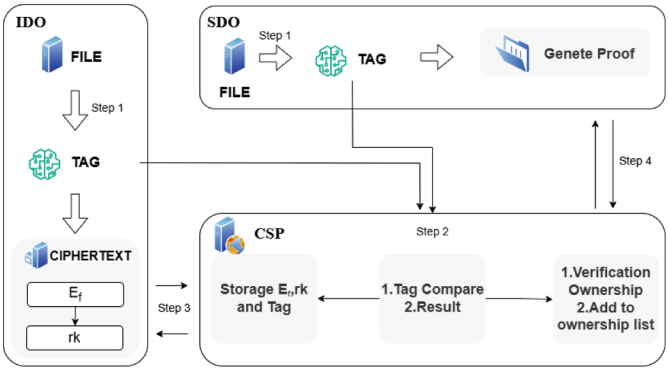


Fig. 2. Process of Tag-assisted storage for deduplication

(a) The CSP needs to verify the correctness of the signature using the verification algorithm: $Ver = Verify(PK_{DO}, Tag_{File}, Sig_T)$. If $Ver = True$ it proves that the data has not been tampered with and can proceed to the comparison phase. Otherwise, the signature is invalid, indicating that the data has been tampered with or the source is untrustworthy.

(b) The CSP performs a tag comparison, specifically comparing $Tag_{File} = \{g^{r_3}, g^{r_3 * H_{File}}\}$ with the stored tag $Tag_i = \{g^{r_i}, g^{r_i * H_i}\}$. The comparison is done by checking whether the $e\{g^{r_3}, g^{r_i * H_i}\} \stackrel{?}{=} e\{g^{r_i}, g^{r_3 * H_{File}}\}$. If the result is No it indicates that the CSP has not stored the corresponding data, and the DO is identified as an IDO who needs to upload the data. Otherwise, the CSP has already stored the data, and the DO is identified as an SDO. The CSP then generates a random number $V_i \in \mathbb{Z}_p^*$ and sends a response to the SDO: $Response = \{PK_{SDO}(V_i), PK_{SDO}(rankey), PK_{SDO}(H_{E_f}), Yes, Sig_{Res}\}$, requesting the SDO to prove possession of the complete data.

3. IDO data upload

(a) The IDO selects a non-repeating random number $r_4 \in \mathbb{Z}_p^*$ and uses the random number r_4 to encrypt the file, obtaining the encrypted file E_f.

$$E_f = SymEnc(r_4, File) \tag{1}$$

(b) The IDO uses H_{File} as the key to perform asymmetric encryption on the random number r_4, resulting in the encrypted random number $rankey$.

$$rankey = Enc(g^{H(File)}, r_4) \tag{2}$$

(c) To securely transmit the data to the CSP and ensure that even if an eavesdropper exists, they cannot obtain the real data information, the IDO adopts an asymmetric encryption method for data transmission. Specifically, the IDO uses the CSP's public key to encrypt the data: $PK_{E_f} = ENC(PK_{CSP}, E_f), PK_{r_k} = ENC(PK_{CSP}, r_k), g^{H(File)} \bmod N$. The IDO will send PK_{E_f}, and $g^{H(File)} \bmod N$, and PK_{r_k} to CSP.

(d) Upon receiving PK_{E_f} and PK_{r_k}, the CSP decrypts them using its private key: PK_{E_f} and PK_{r_k}, then CSP then stores E_f, r_k and Tag_{File}.

4. SDO's challenge response

(a) When the SDO receives the Response, it first verifies Sig_{Res}. Then, the SDO decrypts to obtain V_i, $rankey$, and $H(E_f)$. The SDO can then use r_4 to generate a proxy re-encryption key for data sharing. The SDO also compares $H(E_f)$ from CSP to check if the data is correct and whether the CSP is honestly storing the data.

(b) In response to the challenge, calculate Eq. (3) and output the Proof $= \mu$ as a response to send to the CSP.

$$\mu = (g^{H(File)})^{V_i} \bmod N \tag{3}$$

(c) When the CSP receives the Proof, it verifies it using Eq. (4). If the verification is successful, it indicates that the SDO possesses the complete file.

The CSP will then add the SDOâĂŹs public key (PK) to the ownership list of the file.

$$(g^{H(File)} \bmod N)^{V_i} \bmod N = \mu \tag{4}$$

3.4 Data Share Phase

Either the IDO or the SDO can authorize the DRâĂŹs request; for simplicity, we will refer to them collectively as the IDO in the following discussion. When the IDO receives a sharing request from the DR for file F, if approved, it generates a re-encryption key and sends it to the CSP. The CSP uses this key to re-encrypt *rankey* and sends the data to the DR.

When the IDO receives a data sharing request from the DR, denoted as *Request*, if the IDO is willing to share the data with the DR, the IDO will compute the re-encryption key $rk_{IDO \to DR}$.

The IDO sends $rk_{IDO \to DR}$ and $Sig(rk_{IDO \to DR})$ to the CSP. Upon receiving the data, the CSP first verifies the signature. Once the verification is successful, the CSP generates a re-encrypted copy of data F for sharing. The CSP cannot decrypt data F; it only performs the re-encryption operation. The CSP then sends F_{DR} and $Sig(F_{DR})$ to the DR.

$$F_{DR} = ReENC(rk_{IDO \to DR}, rankey) \tag{5}$$

The DR first verifies the signature. Once the signature verification is successful, the DR decrypts the data to recover the plaintext message m.

$$file = SymDEC(r_4, E_f) \tag{6}$$

4 Threat Model and Informal Security Analysis

4.1 Threat Model

We consider two types of adversaries in SDS-Cloud:

1. Outside Adversary. Outside adversaries are malicious entities attempting unauthorized access to data. They may:
   (a) Impersonate legitimate SDOs or DRs without valid proof of ownership.
   (b) Target specific data or user-uploaded data by eavesdropping on communication channels.
   (c) Perform brute-force or dictionary attacks.
   (d) Access data beyond their granted permissions (privilege escalation).
2. Inside Adversary (Honest-but-Curious CSP). The CSP follows the storage and protocol rules but may attempt to compromise security by:
   (a) Modifying or deleting stored data.
   (b) Using privileges and interaction data to infer or recover users' plaintext data out of curiosity.

4.2 Informal Security Analysis

Note on Proofs. The following proofs provide high-level arguments based on the threat model; they are not formal reductions to standard cryptographic assumptions.

Under this threat model, SDS-Cloud employs cryptographic primitives and protocol mechanisms (e.g., hash functions, discrete logarithm-based verification and proxy re-encryption) to provide confidentiality, integrity, and ownership protection. While formal reduction proofs are not provided, the following high-level arguments hold:

External adversaries cannot easily guess the real data due to randomized tags and ciphertexts, making brute-force or dictionary attacks computationally infeasible.

Ownership verification requires secret information derived from the file itself, preventing external adversaries from forging access.

Any modification or deletion attempts by the CSP will be detected through challenge-response verification, ensuring data integrity.

Theorem 1. *SDS-Cloud ensures data confidentiality against unauthorized users and the CSP, assuming the Discrete Logarithm Problem is hard and the underlying hash functions are one-way.*

Proof. In SDS-Cloud, unauthorized users and the CSP cannot perform brute-force attacks on tags and ciphertexts. For file label comparison, the Equation $e\{g^{r_3}, g^{r_i * H_i}\} \stackrel{?}{=} e\{g^{r_i}, g^{r_3 * H_{File}}\}$ helps the CSP confirm the file's storage without revealing details. For ownership verification, $(g^{H(File)} \bmod N)^{V_i} \bmod N = \mu$ is used. Thanks to the properties of the hash function, this process ensures no extra information about the file or ciphertext is exposed.

Although attackers can intercept data like Tag_{File} and E_f, they cannot extract $H(File)$ to decrypt the file. This is due to the difficulty of solving the Discrete Logarithm Problem, which is computationally hard, especially when N is a large prime. No efficient algorithm exists to solve this problem without additional information.

Theorem 2. *SDS-Cloud can ensure data integrity by Hash function. If the hash function cannot be compromised through collisions, then our solution ensures the integrity of the stored data.*

Proof. 1. CSP Data Integrity: To ensure data integrity, the CSP must not modify or delete the data. The data owmer can challenge the CSP with the Equation $\mu = (g^{E_f})^{V_j} \bmod N$ and verify it with $(g^{E_f} \bmod N)^{V_i} \bmod N = \mu$. The DO only needs to store $(g^{E_f} \bmod N)$, which is small in size, ensuring that the CSP cannot tamper with the data without being caught.

2. SDO Ownership Integrity: A malicious attacker M may intercept $Tag_{File} = \{g^{rj}, g^{rj * H(File)}\}$ and try to gain file ownership. However, without the discrete logarithm of g^{rj}, M cannot retrieve H and thus cannot complete the challenge $\mu = (g^{H(File)})^{V_i} \bmod N$ nor gain access to the file. Even in collusion with the CSP, M cannot brute-force decrypt the ciphertext due to the use of randomization in encryption.

Theorem 3. *SDS-Cloud can protect data from brute-force dictionary attack. Even if attacker is authenticated by the tag.*

Proof. An attacker creates a dictionary $\{d_1, d_2, ..., d_n\}$, generates corresponding tags $\{Tag_1, Tag_2, ..., Tag_n\}$, and repeatedly requests verification from the CSP until passing the preliminary check, receiving a YES and Challenge. After finding the correct tag, the attacker guesses the dataâĂŹs correctness by decrypting, hashing, and validating via the CSP challenge. If successful, the adversary could gain unauthorized access.

However in SDS-Cloud, attackers cannot easily guess real data, as Theorem 1 ensures tags and ciphertexts remain secure. Moreover, by embedding random numbers into both the data and its associated tags, SDS-Cloud can effectively prevent brute-force attacks. This randomness can be retrieved using the original file, enabling legitimate data owners to authorize access while preventing unauthorized inference by adversaries.

Theorem 4. *SDS-Cloud resists collusion attacks under the assumption that the underlying RSA scheme is semantically secure.*

Proof. In the SDS-Cloud, the storage data appear in the CSP in the form of an ciphertexts. So that external attackers must have the private key of the authorization to decrypt the encrypted file. Even if the collusive attackers can obtain encrypted files from CSP and a private key from the user whose permission has been revoked, the collusive attackers still can't decrypt the data. Because the file copy will be updated as the authorization fileâĂŹs user list changes, the file copy will be deleted. Therefore, the private key of the user whose permission has been revoked can no longer decrypt the corresponding data.

4.3 Security Comparison

As shown in Table 2, the security of SDS-Cloud is compared with other related schemes REED [19], TISD [7], ESDS [21]. To resist Brute-Force Dictionary Attacks (BDA), REED and ESDS require key servers for key generation, which introduces a single point of failure.

In both SDS-Cloud and TISD, the SDO must prove ownership of the file to the CSP's file ownership list by performing a proof of work and establishing file ownership. SDS-Cloud also uses encryption with random numbers to prevent BDA and employs bilinear pairing to ensure the consistency of random tags and further protect against BDA. However, TISD only supports data storage and does not allow data sharing.

Table 2. Security Comparison of Different Schemes

	REED	TISD	ESDS	SDS-Cloud
Data Integrity	T	T	T	T
Data Confidentiality	T	T	T	T
BDA for TAG	T	T	T	T
BDA for Ciphertexts	T	T	T	T
Consistency of Tag	T	T	T	T
Decentralization	F	T	F	T
Secure of Share	F	F	T	T

5 Performance Evaluation

We validate SDS-Cloud for deduplication and sharing tasks using the 'Assistments Math 2004-2005 (912 Students)' dataset [22]. This section compares the computational overhead of the DO's TAG generation, the CSP's TAG validation, and file encryption time with those of related schemes, TISD and ESDS. The experiments were conducted on a local cluster composed of a server equipped with an NVIDIA V100 GPU and 125 GB of RAM. All algorithms and experimental procedures were implemented in Python. Each experiment was repeated 100 times, and the average results were reported to reduce the impact of random fluctuations and potential errors. The cryptographic primitives used in SDS-Cloud are implemented by pyecc library and pyCryptodome [23] library.

The computational overhead for the user in the entire process mainly arises during the data upload and sharing stages. For TISD [7], ESDS [21], and SDS-Cloud, there is a difference between the initial file upload and subsequent uploads. The initial upload requires uploading the complete data, whereas subsequent uploads only require ownership verification. Table 3 provides a detailed analysis of the computational time complexity for each scheme.

TISD starts with computing hash values, segment hashes, and random numbers for TAG creation via bilinear mapping. After deduplication: (1) For new data: Generate a random number, AES-encrypt the file, then symmetrically encrypt and hash the random number before uploading to CSP. (2) For existing data: CSP challenges DO to prove ownership. DO calculates the encrypted random number using the file's hash as a private key, performs convergence encryption, and responds with XOR hash values of the random number and ciphertext.

In ESDS, the DO splits the file and interacts with the key server for an encryption key, AES encrypts the file and computes the fileâĂŹs hash as the TAG for deduplication. 1) If the file is not stored, the DO encrypts the convergence key with the master key and uploads the data. 2) If the data is already stored, the DO automatically gains ownership without completing a challenge.

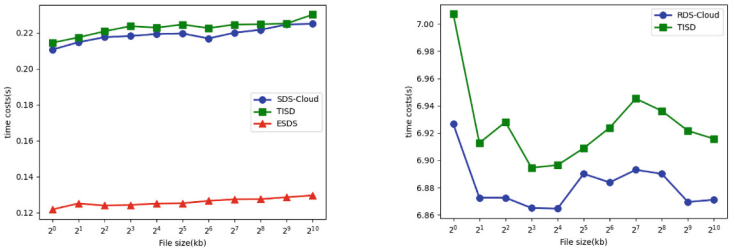
In SDS-Cloud, the TAG computation overhead is $1H + 2G$. The encryption key and random number are selected and encrypted by the IDO, with a time

complexity of $1ENC(ECC)+1SHA+1G+1ENC(AES)$. The time complexity for SDO ownership verification is $1DEC(AES)+1H+1G$.

Table 3. Time Complexity Comparison

	TISD [7]	ESDS [21]	SDS-Cloud
IDO Tag Generation	$1SH+1H+2G$	$nENC(AES)+nH$	$1H+2G$
IDO Enc(File)	$2ENC(AES)+1G+1ENC(ECC)$	$2nH+nG+nENC(AES)$	$1ENC(ECC)+1SHA+1G+1ENC(AES)$
SDO Tag Generation	$1SH+1H+2G$	$nENC(AES)+nH$	$1H+2G$
Response Challenge	$1ENC(AES)+1H+1G+1XOR$	/	$1DEC(AES)+1H+1G$
Authorization	/	$nH+2nDEC(AES)$	$1RE_{key}+1DEC(ECC)+1DEC(AES)$

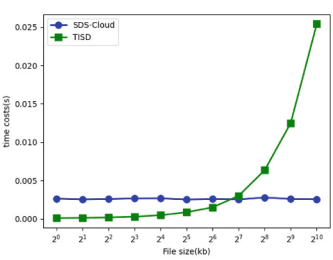
Figure 3(a) shows the computation latency for generating duplicate data detection labels in DO. From Fig. 3(a), it can be seen that TISD has the highest computational load. The computation load of ESDS is smaller than that of both SDS-Cloud and TISD because both SDS-Cloud and TISD require exponentiation operations over G to prevent the exposure of the hash values of the real data. In contrast, ESDS uses symmetric-key encrypted ciphertext hashes as labels, making it more vulnerable to brute-force dictionary attacks. Additionally, since ESDS relies on a key server for encryption, it faces the risk of single-node failures.



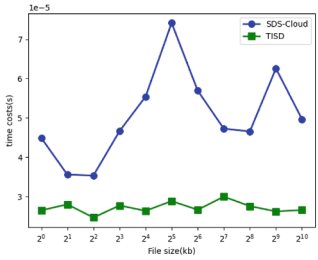
(a) Tag Generation Delay of DO (b) Tag Verification Delay of CSP

Fig. 3. Speedup ratio of SDS-Cloud, TiSD and ESDS against the baseline scheme in Tag generation and verification on education dataset

Figure 3(b) shows the computation latency for CSP to verify duplicate data labels. From Fig. 3(b), it can be seen that the difference between SDS-Cloud and TISD is minimal, but SDS-Cloud has a slightly lower latency than TISD. This is because TISD not only needs to verify the label but also performs verification of the short hash, which increases the computation latency. In contrast, SDS-Cloud only performs label verification. For ESDS, since it only compares the equality of hash values for label verification, its computation latency is too small to be included in the comparison.



(a) Proof of DO challenge generation of SDO

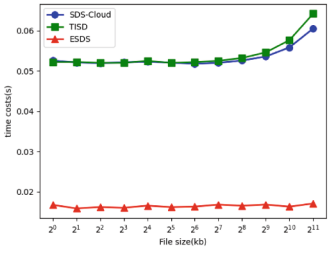


(b) Challenge Validation Delay of CSP

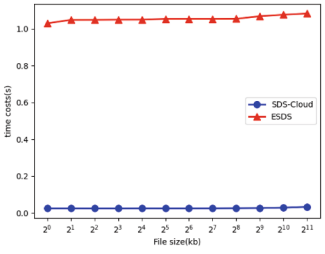
Fig. 4. Comparison between DO's challenge generation and CSP's validation delay

Figure 4(a) compares the challenge generation time of SDS-Cloud and TISD. SDS-Cloud achieves stable performance by computing a single exponentiation in group G using the file's hash, making the delay independent of file size. In contrast, TISD performs full-file re-encryption and hashing, followed by an XOR with a random number, resulting in longer and more variable delays. ESDS lacks a challenge-response phase, relying solely on hash-based ownership verification.

Figure 4(b) compares the proof verification overhead of SDS-Cloud and TISD. SDS-Cloud incurs higher latency due to the use of bilinear pairings, which enhance security against forgery and replay attacks by ensuring only legitimate users can generate valid proofs. In contrast, TISD employs lightweight XOR-based verification, which offers lower latency but weaker security. Thus, SDS-Cloud is better suited for high-integrity scenarios, while TISD favors performance-oriented applications.



(a) File Encryption Delay of IDO



(b) Delay of Data Share

Fig. 5. Performance Analysis of IDO's File Encryption and Data Sharing Latency

Figure 5(a) illustrates the computational latency of IDO data encryption. As shown in Fig. 5(a), the TISD scheme incurs the highest computational delay, while the other two schemes exhibit significantly lower overhead. This is primar-

ily because TISD applies two rounds of symmetric encryption and one ECC-based asymmetric encryption to each file, without considering data sharing. ESDS, on the other hand, employs deterministic encryption, which essentially provides the cloud with a concealed Bloom filter—making it highly vulnerable to privacy probing and dictionary attacks. In contrast, SDS-Cloud uses only one round of symmetric encryption with a random key and one ECC encryption, which not only prevents inference attacks inherent in deterministic encryption but also enables secure data sharing.

Figure 5(b) compares the data sharing latency in SDS-Cloud and ESDS. SDS-Cloud involves re-encryption key generation, re-encryption, and asymmetric decryption, while ESDS performs only two symmetric decryptions over multiple data blocks. Although larger files are split into more blocks, the number of decryption operations remains roughly constant. Due to the short per-block decryption time, the overall latency does not increase linearly with file size, but block-wise processing still causes noticeable growth in computation time.

6 Conclusion

In conclusion, the SDS-Cloud scheme offers a novel solution for efficient storage and sharing of data, addressing the issue of data deduplication in shared environments. Without relying on a third-party authority, we propose a SDS-Cloud, and the ciphertext is resistant to brute-force dictionary. Additionally, we integrate deduplication with proxy re-encryption to improve storage efficiency and facilitate secure data sharing. This solution effectively addresses the challenge of performing deduplication without third-party intervention. Security analysis confirms that the SDS-Cloud scheme ensures both the effectiveness of deduplication and the confidentiality of data sharing. Simulation experiments show that while the computation latency for file sharing is slightly higher, the system is more resilient to attacks and helps prevent data leakage. Overall, the SDS-Cloud scheme demonstrates superior security and performance.

Acknowledgements. The research was supported in part by the National Natural Science Foundation of China (Grant Nos. 62166004, U21A20474), the Guangxi Science and Technology Major Project (Grant No. AA22068070), the Key Lab of Education Blockchain and Intelligent Technology, and the Center for Applied Mathematics of Guangxi.

References

1. Song, M., Hua, Z., Zheng, Y., et al.: Blockchain-based deduplication and integrity auditing over encrypted cloud storage. IEEE Trans. Dependable Secure Comput. **20**(6), 4928–4945 (2023)
2. Armknecht, F., Bohli, J.M., Karame, G.O., et al.: Transparent data deduplication in the cloud. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, pp. 886–900 (2015)

3. Wang, M., Xu, L., Hao, R., et al.: Secure auditing and deduplication with efficient ownership management for cloud storage. J. Syst. Architect. **142**, 102953 (2023)
4. Wenxiu, D., Yan, Z., Deng, R.H.: Privacy-preserving data processing with flexible access control. IEEE Trans. Dependable Secure Comput. **17**(2), 363–376 (2017)
5. Zhang, X., Xu, C., Wang, H., et al.: FS-PEKS: lattice-based forward secure public-key encryption with keyword search for cloud-assisted industrial Internet of Things. IEEE Trans. Dependable Secure Comput. **18**(3), 1019–1032 (2019)
6. Bellare, M., Keelveedhi, S., Ristenpart, T.: Message-locked encryption and secure deduplication. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 296–312. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38348-9_18
7. Guo, C., Wang, L., Tang, X., et al.: Two-party interactive secure deduplication with efficient data ownership management in cloud storage. J. Inform. Secur. Appl. **73**, 103426 (2023)
8. Zhang, T., Wang, C., Chandrasena, M.I.U.: Blockchain-assisted data sharing supports deduplication for cloud storage. Connect. Sci. **35**(1), 2174081 (2023)
9. Qi, S., Wei, W., Wang, J., et al.: Secure data deduplication with dynamic access control for mobile cloud storage. IEEE Trans. Mob. Comput. **23**(4), 2566–2582 (2023)
10. Miao, Y., Gai, K., Zhu, L., et al.: Blockchain-based shared data integrity auditing and deduplication. IEEE Trans. Dependable Secure Comput. **21**(4), 3688–3703 (2023)
11. Benil, T., Jasper, J.: Blockchain based secure medical data outsourcing with data deduplication in cloud environment. Comput. Commun. **209**, 1–13 (2023)
12. Sharma, S., Saini, H.: Fog assisted task allocation and secure deduplication using 2FBO2 and MoWo in cluster-based industrial IoT (IIoT). Comput. Commun. **152**, 187–199 (2020)
13. Keelveedhi, S., Bellare, M., Ristenpart, T.: DupLESS: server-aided encryption for deduplicated storage. In: 22nd USENIX Security Symposium (USENIX Security 13), pp. 179–194 (2013)
14. Tian, G., Hu, Y., Wei, J., et al.: Blockchain-based secure deduplication and shared auditing in decentralized storage. IEEE Trans. Dependable Secure Comput. **19**(6), 3941–3954 (2021)
15. Ming, Y., Wang, C., Liu, H., et al.: Blockchain-enabled efficient dynamic cross-domain deduplication in edge computing. IEEE Internet Things J. **9**(17), 15639–15656 (2022)
16. Yuan, H., Chen, X., Li, J., et al.: Secure cloud data deduplication with efficient re-encryption. IEEE Trans. Serv. Comput. **15**(1), 442–456 (2019)
17. Aparna, R., Bandopadhyay, S., Pandey, S.: BlockDrive: a deduplication framework for cloud using edge-level blockchain. In: international Conference on Communication Information and Computing Technology (ICCICT), vol. 2021, pp. 1–6. IEEE (2021)
18. Li, S., Xu, C., Zhang, Y.: CSED: client-side encrypted deduplication scheme based on proofs of ownership for cloud storage. J. Inform. Secur. Appl. **46**, 250–258 (2019)
19. Qin, C., Li, J., Lee, P.P.C.: The design and implementation of a rekeying-aware encrypted deduplication storage system. ACM Trans. Storage (TOS) **13**(1), 1–30 (2017)
20. Zhang, D., Le, J., Nankun, M., Jiahui, W., Liao, X.: Secure and efficient data deduplication in jointcloud storage. IEEE Trans. Cloud Comput. **11**(1), 156–167 (2023)

21. Wang, Z., Gao, W., Yang, M., et al.: Enabling secure data sharing with data deduplication and sensitive information hiding in cloud-assisted electronic medical systems. Clust. Comput. **26**(6), 3839–3854 (2023)
22. Koedinger, K.R., Baker, R.S.J.d., Cunningham, K., Skogsholm, A., Leber, B., Stamper, J.: A data repository for the EDM community: the PSLC DataShop. In: Romero, C., Ventura, S., Pechenizkiy, M., Baker, R.S.J.d. (Eds.) Handbook of Educational Data Mining. CRC Press, Boca Raton (2010)
23. Eijs, H.: Pycryptodome-the Python cryptography toolkit. https://pypi.org/project/pycryptodome (2019)

iSubscription: Bridging the Gap Between Contracts and Runtime Access Control in SaaS

Alejandro García-Fernández(✉), José Antonio Parejo, and Antonio Ruiz-Cortés

SCORE Lab, I3US Institute, Universidad de Sevilla, Sevilla, Spain
{agarcia29,japarejo,aruiz}@us.es

Abstract. Software as a Service (SaaS) usually provides users with the ability to choose the features and usage limits they need – i.e. a configuration. Once a configuration is selected, it becomes a subscription. Despite enhancing value for customers, this model challenges providers, who must enforce that users operate within subscriptions that may change at any time – either by user choice or provider updates. This makes runtime subscription enforcement a self-adaptation challenge. A promising strategy to address this challenge is pricing-driven feature toggling, currently implemented only by Pricing4SaaS, to the best of our knowledge. This solution relies on two data sources: iPricings – machine-oriented representations of pricings; and subscription states – usage levels' values of usage-limited features. However, Pricing4SaaS delegates subscription state management to the managed service, forcing providers to update subscription logic whenever a new usage limit is added/removed, which increases coupling and hinders scalability. This paper introduces three main contributions to address these limitations. First, we show that usage limits tend to increase as pricings evolve, reinforcing the need to simplify subscription management. Second, we introduce iSubscription, a machine-oriented model of subscription states compatible with iPricings. Third, we introduce SPACE, an independent service for pricing-driven self-adaptation that overcomes all state-of-the-art's limitations.

Keywords: Pricing-driven DevOps · iPricing · Services Operation · SaaS · Self-adaptation

1 Introduction and Motivation

One of the key factors behind the success of Software as a Service (SaaS) lies in giving customers the ability to choose the features and usage limits they need – i.e. a configuration they are willing to pay for on a recurring basis [1]. This choice typically occurs through the SaaS's pricing page, and it is enabled by a mechanism that varies across providers and depends – among other factors – on the size of the set of possible combinations, known as the configuration space [7].

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 51–66, 2026.
https://doi.org/10.1007/978-981-95-5015-9_4

The size of this space is determined by the number of plans and add-ons offered, as well as the constraints among them. As these elements have become more complex and varied, configuration spaces are evolving from offering just a few options to thousands of potential configurations [7]. For instance, Salesforce's pricing in November 2019 supported around 10 different configurations, while its July 2024 allowed for over 12,000 [10].

Each configuration in this space contains a different set of features and usage limits. Once selected by a customer, it becomes a subscription. One of the main challenges in operating a SaaS with a pricing model is to ensure that the system consistently enforces the set of active subscriptions. This is particularly difficult not only due to the vast number of possible configurations but also because any subscription may change at any time – either due to customer decisions (e.g., upgrading or downgrading) or provider-triggered configuration updates (e.g., adding or removing features). In essence, this can be interpreted as a self-adaptation challenge triggered by changes in subscriptions – hereafter referred to as pricing-driven self-adaptation.

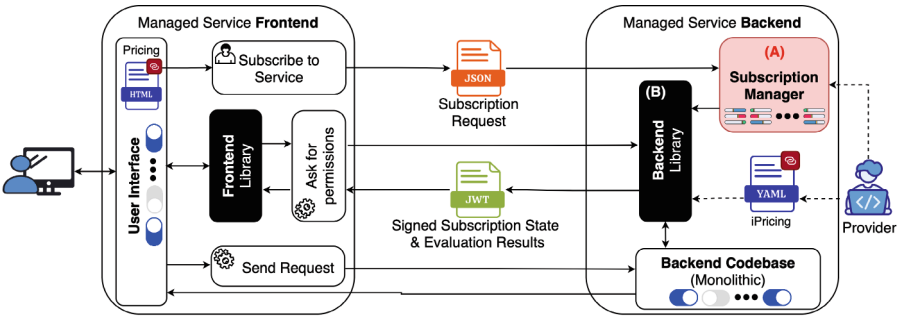


Fig. 1. Self-adaptation by Pricing4SaaS. Its adaptation logic is composed by "frontend" and "backend libraries" along with the "subscription manager".

Among the most promising approaches to address this challenge is pricing-driven feature toggling [7], with Pricing4SaaS standing out as the only solution that enables self-adaptation by modifying a machine-oriented representation of the pricing, so-called iPricing [9]. As shown in Fig. 1, its adaptation logic involves: i) a "subscription manager" – developed and maintained by the provider – which handles "subscription requests" and exposes subscription states – i.e. the usage levels of usage-limited features; ii) a "backend library" (by Pricing4SaaS) that determines at any time whether a user has access to a feature by combining information from the iPricing and the user's subscription state; and iii) a "frontend library" (by Pricing4SaaS) that, based on the information provided by the "backend library", keeps the UI consistent with the subscription.

This architecture provides the core capabilities needed for pricing-driven feature toggling, but it faces two important limitations. First, the complexity of the "subscription manager" scales with the number of usage limits in the pricing, as

each new limit introduces a new usage level that must be integrated into subscription states – requiring additional logic that Pricing4SaaS forces the provider to implement (A). Second, Pricing4SaaS is monolithic, and thus it runs within the same environment as the managed service – i.e. the system integrating it [20] – ultimately hindering scalability and applicability in distributed deployments (B).
To address these limitations, this paper explores the following research questions:

RQ$_1$: *How is the presence of usage limits evolving in pricings?*
RQ$_2$: *What would be the model of a machine-oriented artifact capable of representing subscription states?*
RQ$_3$: *What is the architecture of a solution that decouples all self-adaptation logic from the managed service?*
RQ$_4$: *How does a decoupled pricing-driven self-adaptation solution compare in performance to existing approaches?*

After reviewing the necessary background (Sect. 2), this paper makes three contributions to address the research questions stated above. First, we present an observational study demonstrating that the number of usage limits increases over time (Sect. 3). Second, we introduce the iSubscription metamodel – a machine-oriented representation of subscription states – that interoperates with an extended version of iPricings (Sect. 4). Third, we introduce SPACE, a pricing-driven self-adaptation solution that leverages the new iPricing and iSubscription metamodels to centralize all data required for enforcement, fully decoupling the tool from the managed system (Sect. 5). Then, Sect. 6 compares it against the state of the art (SOTA) in terms of performance. After that, Sect. 7 discusses related work. Finally, Sect. 8 concludes the paper and outlines future research directions.

2 Background

2.1 Pricing and Subscriptions

The pricing is a core component of a SaaS customer agreement (CA) [15] – a provider-issued document, common to all customers. It structures the **features** of a service – i.e. distinctive characteristics whose presence/absence may guide a user's decision towards a particular subscription – and their associated **usage limits** – which impose additional constraints that make access to certain features depend on measurable **usage levels** – into **plans** and/or **add-ons** to control users' access [10].

When the pricing includes plans, users must choose one of them, and may complement it with compatible add-ons. Therefore, this choice determines the set of features and limits a user can access – i.e. a configuration. The set of all valid plan/add-on combinations forms the SaaS **configuration space**, thus the pricing is a compact representation of it. Once the user agrees to pay for a selected configuration on a recurring basis, it becomes a **subscription**. Since

usage limits may condition access depending on current consumption, each subscription has an associated **state** that tracks usage levels' values in real-time.

Figure 2 represents an excerpt of the pricing of Zoom[1], a cloud-based video conferencing service that enables users to conduct virtual meetings.[2] In this example, nine features are managed through plans, two features are associated with add-ons ("translated captions" and "zoom sessions"), and one is governed by both ("zoom webinars"). The pricing also imposes three **usage limits** (two on the "meetings" feature and one on the "cloud storage") meaning that although usage-limited features are available in all plans, their extent of use varies – higher-priced plans offer higher limits.

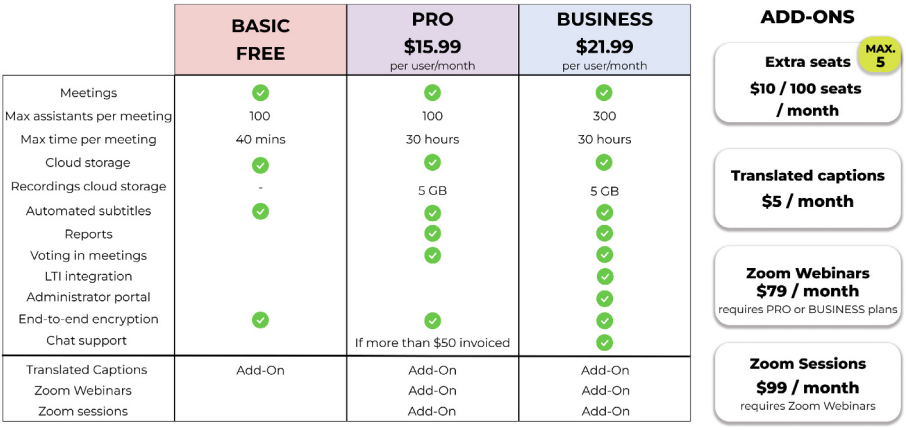


Fig. 2. Excerpt of Zoom's pricing with 12 features, three plans and four add-ons.

As can be noticed, pricings considered in this paper are subscription-based. Naturally, we acknowledge the existence of alternative strategies, such as pay-per-use models or commission-based approaches, but they fall out of scope.

2.2 Feature Flags

Feature Flags —a technique for dynamically enabling or disabling features at runtime without altering the code [6]— offer a promising mechanism for subscription enforcement [7]. In a nutshell, they are boolean expressions whose logic and values can both be assigned or modified at runtime, thus providing dynamic evaluations. In pricing-driven SaaS environments, feature flags[3] that are used

[1] https://zoom.us/pricing, Accessed: July 2025.
[2] Pricing entries that impose or extend limits on meetings are considered usage limits rather than individual features – the overarching usage-limited feature is "meetings".
[3] Feature flags may also be referred to as "feature toggles".

for subscription enforcement have been referred to as pricing-driven feature toggles [7]. By mapping iPricings to these flags, developers can serve all configurations within a single deployment, maximizing the benefits of multi-tenancy [11]. However, this flexibility comes at a cost: each flag increases service variability, complicating testing [17] and contributing to long-term technical debt [19].

Modern feature flag management solutions address such complexity by externalizing the logic governing feature flags from the managed service into external systems, decoupling evaluations into simple method calls (e.g., *isFeatureAvailable(featureId)*) [7]. These tools typically redefine a *feature* as a toggleable unit governed by *rules* (the logic dictating whether the feature is enabled or not) that is evaluated against a given *context* (such as a subscription) representing a user.

However, as demonstrated in [7], current industry solutions struggle to cope with the variability introduced by modern pricings – leading, in some cases, to configuration spaces that become unmanageable. In the same study, the *Pricing4SaaS* library (originally presented in [9]) was identified as the solution that most comprehensively embraces the principles of pricing-driven self-adaptation. Its architecture combines the iPricing metamodel —encoded as Pricing2Yaml [7]— with a custom, ad-hoc subscription module developed by the service provider (see Fig. 1). Together, they determine the access rules that must be satisfied by each user to enable a given feature. Once configured, the component can be invoked by other services within the backend to dynamically evaluate whether a user has access to a specific feature, enforcing or blocking it accordingly. If the provider later modifies the pricing, simply updating the iPricing – and the subscription module, if required – automatically applies the new rules at runtime, without needing to change or redeploy the core service logic.

3 Evolution of Usage Limits in Pricings

Despite their essential role in pricing-driven self-adaptation, prior analyses [7] overlooked usage limits and their link to subscription states. This omission likely stemmed from: i) an early assumption that usage limits played a minor role in pricing-driven self-adaptation; and ii) the absence of modeling standards, as usage limits were often treated as numeric features rather than constraints attached to existing capabilities.

Recent modeling guidelines [16] clarify this distinction. For instance, in Zoom, "Max assistants per meeting" should not be seen as a standalone feature but as a usage limit governing an abstract one: "Meetings". In some cases, such abstract features may not even appear explicitly in the pricing, but can be inferred from its associated constraints. This separation reinforces the notion that features represent capabilities that are either available or not, while usage limits define additional constraints that must be met for a feature to remain effectively available. As a result, the set of features becomes more semantically consistent and compact – e.g., "Meetings" remains a single feature regardless of the number of constraints tied to it – while also making it clearer which usage levels must be tracked in subscription states.

Given this advancement, we revisit more than 240 pricings from [7] hypothesizing that the number of usage limits tends to grow over time. Figure 3 presents the distribution of usage limits per SaaS over the last seven years.[4] The boxplot is enriched with a dashed blue line connecting the yearly means to highlight the overall trend, and medians are marked with green lines. As hypothesized, usage limits show a clear upward trend: the average has grown almost 90%, from 4.15 in 2019 to 7.86 in 2025. Statistical tests confirm the significance of this growth (p-value = 0.03, Mann–Whitney U), with a large effect size (r estimator).

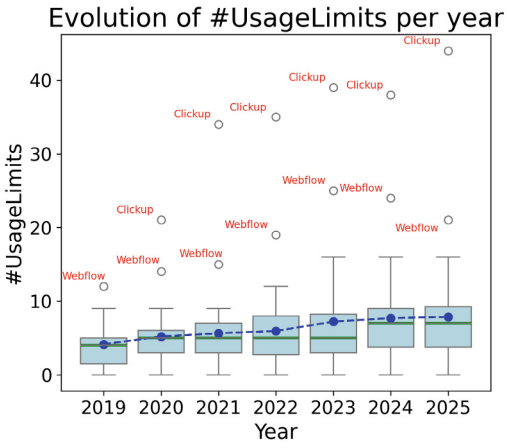


Fig. 3. Usage limits over the last seven years

These results confirm that the burden associated with usage limits in pricing-driven self-adaptation is steadily increasing. For instance, Webflow expanded from 12 usage limits in 2019 to 21 in 2025, while ClickUp, a notable outlier, grew from 1 to 44 over the same period. This trend indicates that subscription states are becoming progressively more complex. Consequently, any self-adaptation approach that delegates subscription state management to the managed service becomes increasingly difficult to maintain over time.

4 Serializing the Subscription State

As a first step towards automating subscription state management and decoupling it from the managed service, we introduce two complementary metamodels:

4.1 Enriched iPricing

This new version of the metamodel proposes three modifications over the version employed in Pricing4SaaS (see Fig. 4). First, usage limits are now classified as either *renewable* – are periodically reset – or *non-renewable* – are static and don't reset–, unifying previously fragmented categories. For example, Zoom defines only non-renewable limits. Second, tracking attributes are introduced to specify how usage should be monitored. Renewable limits introduce a "Period" attribute indicating the reset interval, while non-renewable limits use a "trackable" flag to indicate whether usage levels are needed. In Zoom, "cloud storage" should set *trackable* = *true*, as a usage level (consumed GBs) is needed to

[4] Dataset and code to reproduce the results are available in [8].

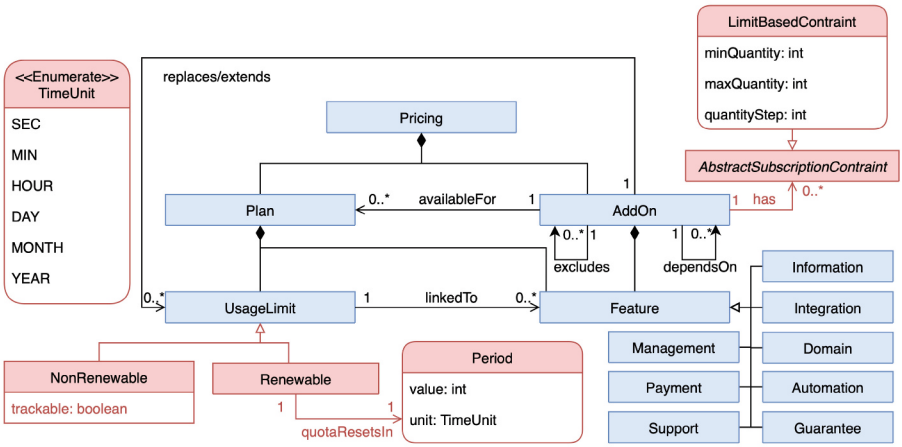


Fig. 4. Enriched version of the iPricing metamodel, highlighting in red the new elements introduced to enhance its integration with iSubscriptions (Color figure online)

determine whether the limit has been surpassed. Finally, a new *subscriptionConstraints* attribute is added to add-ons, specifying conditions that must hold in any subscription that includes them. For example, Zoom's "extra seats" can be subscribed up to 5 times, capping meeting capacity at 800 under the BUSINESS plan.

4.2 The iSubscription

This is a machine-oriented subscription metamodel intended to formalize the representation of user subscriptions and their states, ultimately decoupling subscription management from the managed service. According to Fig. 5, each iSubscription comprises five main components:

- *User Information*, used to associate subscriptions with specific users and to facilitate provider-side tooling development, such as support portals.
- *Billing Period* manages subscription duration by specifying start/end dates, whether it should auto-renew (autoRenew) and the renewal duration (renewalDays). This not only enables automated lifecycle management of subscriptions, but also integration with external payment systems and support of common cancellation scenarios – e.g., canceled subscriptions with access until period end.
- *Usage Level* monitors consumption of trackable usage limits – i.e. those classified as renewable, or as non-renewable with the "trackable" flag activated in the iPricing. It is expected to manage periodic resets of renewable limits. For example, in Zoom, "cloud storage" would have "consumed GBs" associated.
- *Active Subscription*, the core component of iSubscription, captures the user's current plans and add-ons across all services offered by the SaaS. As may

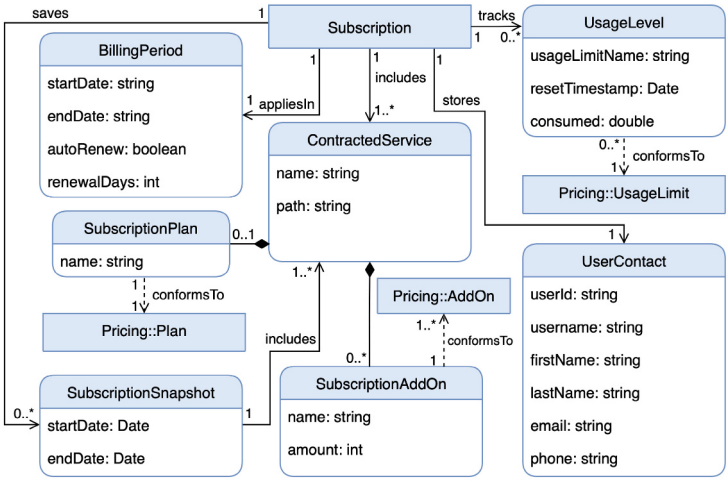


Fig. 5. The iSubscription metamodel

be noticed, this allows to support complex pricing scenarios – such as multi-service platforms with distinct pricings (e.g., GitHub and GitHub Copilot) – by modeling the selection of plans and add-ons for each service independently. For add-ons, it also records the contracted quantity, enabling support for scalable add-ons, for instance, for pay-per-use pricing strategies.

– Lastly, *Subscription Snapshots* retains past subscriptions states with timestamp, enabling subscription rollback and retrospective analysis of user behavior and business trends without additional tooling.

Combined, the enriched iPricing and iSubscription metamodels will offer several advantages over the state of the art. Specifically, they: i) enable the automatic retrieval of user configurations and subscription states using a user identifier (e.g., username); ii) facilitate fully decoupled subscription management from the managed system, which improves scalability; iii) enrich subscription representation capabilities by supporting scalable add-ons, thus enhancing not only runtime adaptation but also expanding the potential of other pricing-driven devops areas, such as iPricings automated analysis; and iv) pave the way towards multi-service pricing-driven systems, allowing unified handling of multiple, independent, or combined pricings (like GitHub and GitHub Copilot).

5 SPACE: A New Way of Pricing-Driven Self-adaptation

Leveraging the new iPricing and iSubscription metamodels introduced in Sect. 4, we created SPACE (Subscription and Pricing Access Control Engine)[5], a pricing-driven self-adaptation solution that addresses the limitations identified in the state-of-the-art (see Sect. 1). Figure 6 depicts the self-adaptation it proposes.

[5] https://github.com/Alex-GF/space.

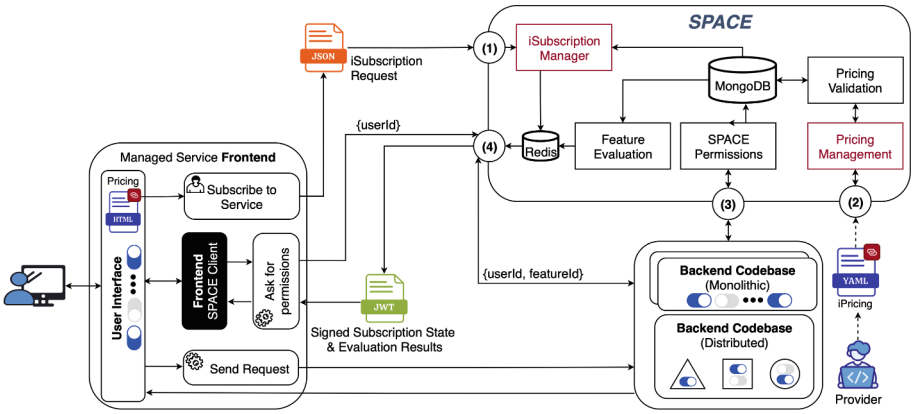


Fig. 6. Self-adaptation by SPACE. (1) Subscription Manager, (2) Pricing Management, (3) Access Control Management, (4) Feature Evaluation interfaces

The first major architectural difference with SOTA lies in the "subscription manager", which is now fully decoupled from the managed service. This allows SPACE to centralize all the data required for pricing-driven self-adaptation – i.e. pricings and subscription states–, which improves performance. The second key difference concerns deployment: SPACE is designed as a standalone, on-premises, API-based component, which enables its integration into any environment and overcomes the monolithic limitations of SOTA. Despite this shift, SPACE retains compatibility with existing toggling workflows by preserving feature flags as the variation points where feature access is evaluated – just as in Pricing4SaaS.

Beyond addressing the architectural limitations of the state of the art, SPACE also introduces several additional advantages: i) it is agnostic to the technology stack of the managed system; ii) it can act as a pricing version control system due to its built-in MongoDB database; and iii) it supports caching mechanisms to accelerate feature evaluations. Moreover, to further streamline integration and minimize boilerplate code, SPACE provides dedicated SDKs – such as *space-react-client*[6] and *space-node-client*[7] – which encapsulate API interactions behind native methods tailored for managed applications.

Regarding its architecture (also represented in Fig. 6), SPACE exposes four interfaces. The **Subscription Manager** interface (1) allows CRUD of user subscription – including retrieval and auto-management of subscription states. The **Pricing Management** interface (2) maintains validated versions of each service's pricing. In large-scale SaaS ecosystems, where multiple services may coexist (e.g., GitHub and GitHub Copilot), distinct pricings per service can be managed through this interface. The **Access Control Management** interface (3) controls access to SPACE's operations, allowing restrictions to be set on which

[6] https://github.com/Alex-GF/space-react-client.
[7] https://github.com/Alex-GF/space-node-client.

actors can perform specific actions. Finally, the **Feature Evaluation** interface (4) supports two complementary uses. By sending just a user ID, it issues a JSON Web Token (JWT) encoding the user's subscription state and a summary of their feature access permissions. These tokens are consumed by SPACE's frontend clients to keep the UI consistent with the subscription. If both a user ID and a feature ID are sent, the interface evaluates whether the user has access to the indicated feature – returning a boolean result without issuing a JWT – nonetheless, this operation is intended for backend clients. No matter the use, evaluations always require information retrieved by interfaces (1) and (2).

To carry the second type of Feature Evaluation interface's evaluations, SPACE follows the algorithm depicted in Fig. 7, which is designed to keep subscription states consistent under concurrency or partial failures. Alongside the user and feature identifiers, the managed service may optionally provide an "expectedConsumption" value (e.g., the size of a video to upload), which enables optimistic updates: usage levels are tentatively increased before processing the request.

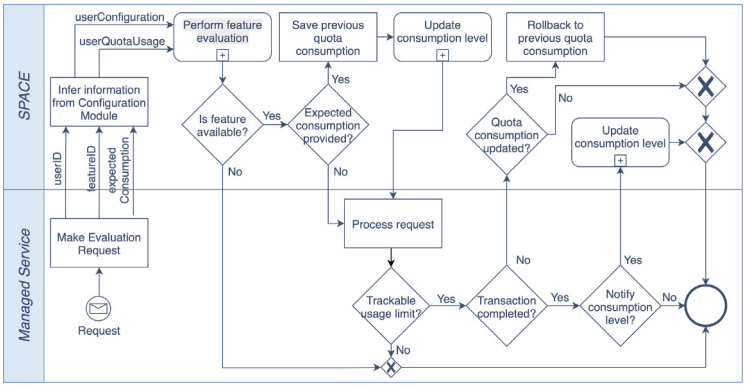


Fig. 7. SPACE's feature evaluation algorithm using iPricing and iSubscription

Regardless of whether this parameter is provided, the algorithm first retrieves the contract assigned to the provided user ID, and infers from it both the user's current configuration and usage levels. At this stage, usage levels tracking renewable limits are reset if needed (e.g., due to billing cycle transitions). Feature access is then evaluated using the same logical criteria as in Pricing4SaaS. If access is denied, the process ends. Otherwise, one of two alternatives is followed:

If expectedConsumption was provided, SPACE temporarily stores the current values of the affected usage levels in memory and applies the expected increase. This snapshot is retained for a short period (e.g. 30–60 s). After that, the managed service may proceed immediately. If the transaction completes, the update is implicitly confirmed; if it fails, a rollback can be requested within the temporary storage window.

If expectedConsumption was not provided, only the access decision is returned. If the request affects usage levels, SPACE must be notified upon completion to apply the corresponding update.

6 Evaluation

6.1 Experimental Validation

To evaluate SPACE's performance, we conducted a comparative analysis against the current state-of-the-art baseline: Pricing4SaaS. The assessment was performed using TomatoMeter[8], a monolithic, client-server demo application that simulates a productivity tool based on the Pomodoro method. While SPACE's design offers architectural advantages over Pricing4SaaS in terms of integration simplicity and suitability for distributed environments, this experiment addresses $\mathbf{RQ}_4$. In this way, TomatoMeter offers the ideal environment, since its monolithic design allows both solutions to be integrated without architectural bias.

We considered five distinct Subject Under Test (SUT) for the evaluation: i) Toma-toMeter integrated with Pricing4SaaS; ii) TomatoMeter integrated with SPACE deployed as a single instance; iii) the same integration with SPACE scaled horizontally to three instances; iv) standalone SPACE with one instance; and v) standalone SPACE with three instances. This design allowed us to assess both the system-wide impact of integrating SPACE and its performance in isolation, including scalability effects. In each scenario, we measured the behavior of the two most frequent operations in pricing-driven self-adaptation: pricing token renewal, which issues tokens encoding the user's permissions to control UI elements rendering on the client side; and feature evaluation, which determines whether access to a specific feature is permitted given a subscription state.

All tests were executed on a Mac Mini equipped with an Apple M4 Pro processor and 24 GB of RAM. For each SUT, we collected four performance metrics: average response time (in ms); peak and average throughput (in requests per second), which are indicative of potential bottlenecks or underutilization; and the success rate, defined as the percentage of virtual users completing all operations without errors. The experiments were executed using the Artillery platform[9], which simulated 5,240 virtual users distributed across three phases: i) a 60 s warm-up, increasing the arrival rate from 5 to 10 user/s; ii) a 60-second ramp-up, escalating from 10 to 50 user/s; and iii) a 30 s spike phase with a constant rate of 100 user/s. Each virtual user executed both evaluated operations 10 times. Table 1 summarizes the results obtained.

The results show that SPACE outperforms Pricing4SaaS in all configurations in terms of success rate, meaning fewer users encountered errors – even in scenarios where SPACE required slightly more time to process requests. As a standalone service, SPACE consistently achieved higher throughput and lower

[8] https://github.com/Alex-GF/TomatoMeter.
[9] https://www.artillery.io.

Table 1. Results of performance comparison between Pricing4SaaS and SPACE

SUT	Renew Token				Evaluate Feature			
	Avg. ms	Avg. req/s	Peak req/s	Success %	Avg. ms	Avg. req/s	Peak req/s	Success %
Pricing4SaaS [9]	99	368	686	82.61%	103	173	651	80.42%
space-only 1-instance	4	421	1011	100%	2	427	1010	100%
space-only 3-instances	2	428	1011	100%	2	431	1018	100%
TomatoMeter 1-instance	1200	343	598	94.3%	929	315	568	82.61%
TomatoMeter 3-instances	228	402	920	100%	26	428	1008	100%

latency, handling a greater number of concurrent users. Thanks to its evaluation cache, response times were up to 50 times faster than those of Pricing4SaaS, emphasizing the efficiency gains of centralizing all subscription enforcement data.

When integrated with TomatoMeter using a single instance, SPACE exhibited slower average response times under heavy load. However, this gap was effectively closed by horizontally scaling SPACE to three instances, which brought feature evaluation latency below 30 ms and brought token renewal latency close to that of Pricing4SaaS, while maintaining a 100% success rate in both operations – even though it was integrated with a single TomatoMeter instance.

Therefore, considering that Pricing4SaaS is inherently unsuitable for distributed environments and would face significant consistency and synchronization challenges in horizontally scaled deployments, scaling SPACE emerges as a more practical and cost-effective solution – even in monolithic contexts. Overall, these results confirm that SPACE not only delivers the expected flexibility and scalability, but also achieves performance that is comparable or superior to the current state-of-the-art under realistic load conditions.

6.2 Threads to Validity

Regarding *Internal Validity*, uncontrolled factors in the experimental setup could affect our observational study. For instance, all tests were conducted on a single hardware platform, which may introduce biases in favor of certain architectures or technology stacks. Additionally, the workload used, while stress-oriented and reproducible, may not fully capture the diversity of operations or traffic seen in production SaaS environments.

Regarding *External validity*, the generalizability of SPACE depends on the expressiveness of the iPricing and iSubscription metamodels. While they cover a wide range of real-world pricing scenarios, they may fall short in modeling complex or unconventional schemes, such as dynamic group pricing or cross-service bundling. Similarly, although SDKs simplify integration, adapting SPACE to legacy or customized systems may require additional engineering effort.

Regarding *Construct validity*, we employed multiple performance metrics — response time, throughput, and success rate—to assess responsiveness and reli-

ability, thereby mitigating mono-method bias. This diversity strengthens confidence in the findings, yet our use of a simplified managed system (TomatoMeter) limits representativeness. Although concurrency and variability were simulated, broader metrics (e.g., memory usage or integration effort) remain unexamined and may constitute an area of future work.

Regarding *Conclusion validity*, the limited test duration and sample size may reduce statistical power, especially for infrequent failure modes. We partially addressed this through repeated trials and complementary metrics, but further longitudinal experiments across varied infrastructures are needed to reinforce the robustness of our conclusions.

7 Related Work

Pricing-driven self-adaptation relates to Software Product Lines (SPLs) in general and Dynamic Software Product Lines (DSPLs) in particular, as well as to Self-Adaptive Systems (SAS).

An adaptive, pricing-driven SaaS can be viewed as a DSPL [12] where the functionality (features) and capacity (usage limits) available to each user are constructed (bound) at runtime. Regarding variability modeling, instead of using Feature Models (FMs) [13], iPricings are employed, from which useful information can be extracted for the DevOps cycle [10] through a catalog of analysis operations [3]. In particular, from a variability modeling perspective, just as a Feature Model compactly expresses the feature configurations of all products in a line, an iPricing also compactly captures all available SaaS configurations. This mirrors the role of FMs in Feature-Oriented Domain Analysis (FODA) [13], where variability is structured around features and represented visually as feature diagrams (FDs). However, FODA is tailored for hierarchical variability, whereas SaaS pricings exhibit a matrix structure [10], grouping features across plans and add-ons. While traditional FMs could, in principle, express these structures, doing so would require numerous cross-tree constraints, leading to overly complex and less maintainable models. This motivates a dedicated alternative which, to the best of our knowledge, has not yet been explored in the literature. The analogy with FODA is straightforward: feature diagrams relate to feature models just as pricing pages relate to iPricings [4].

Finally, pricing-driven self-adaptation complements other runtime enforcement approaches. One such mechanism is API rate limiting [18], which safeguards service availability by restricting request volumes. Although pricing usage limits may look similar, they serve a different purpose: rate limiting focuses on performance and abuse prevention, whereas pricing-driven adaptation enforces functional access, ensuring that users can only use the features included in their active subscriptions. Another mechanism is policy-based authorization frameworks [14], among which CEDAR [5] stands out as a representative example. These frameworks operate at a finer granularity. For instance, while our approach evaluates whether a subscription grants access to the "Cloud Storage" feature in Zoom (see Fig. 2), CEDAR enforces rules over the resources within that feature

(e.g., users may only access the files they have uploaded). Despite this difference in scope, SPACE and CEDAR converge architecturally: they decouple rule specification from access evaluation. CEDAR policies are expressed in a dedicated language and evaluated by an external service (e.g., AWS Verified Permissions [2]), just as SPACE separates pricing information (iPricings/iSubscriptions) from its evaluation engine.

8 Conclusions and Future Work

In this paper, we set out to answer whether the complexity of pricing models affects the number of variables needed to represent subscription states ($\mathbf{RQ}_1$), what kind of machine-oriented artifact could best capture such states ($\mathbf{RQ}_2$), how to architect a solution that decouples the adaptation logic from the managed service ($\mathbf{RQ}_3$), and how such a solution compares in performance to existing pricing-driven self-adaptation state of the art ($\mathbf{RQ}_4$).

For $\mathbf{RQ}_1$, we identified usage limits as the key element linking pricing complexity to subscription states, as a greater number of limits increases the number of variables that must be tracked. We then conducted an observational analysis of over 240 real-world pricings, confirming a steady increase in usage limits over time—nearly 90% growth in the last seven years. For $\mathbf{RQ}_2$, we extended the existing iPricing metamodel and introduced the complementary iSubscription, enabling a standardized representation of all information required for subscription compliance. For $\mathbf{RQ}_3$, we provided SPACE, a prototype that fully decouples pricing-driven self-adaptation from the managed service by externalizing pricing and subscription state management. Achieving this required redesigning the feature access evaluation algorithm. For $\mathbf{RQ}_4$, we evaluated SPACE against Pricing4SaaS. As a standalone service, SPACE achieved over 1000 req/s with 2–4 ms latency and 100% success rate, vs. 80–82% in Pricing4SaaS. When integrated with TomatoMeter – where remote latency applies – a horizontally scaled SPACE matched or exceeded Pricing4SaaS in throughput and reliability, cutting feature evaluation latency by $> 30\times$ and token renewal latency by $> 6\times$ under load.

Future work will focus on three main directions: i) expanding the iSubscription metamodel to support more complex pricing strategies – e.g., multi-user team subscriptions; ii) exploring the applicability of SPACE beyond SaaS deployments; and iii) validating SPACE in real-world, production-grade SaaS platforms to identify security threats and assess its scalability, robustness, and impact.

Acknowledgements. This work has been partially supported by grants PID2021-126227NB-C21, and PID2021-126227NB-C22, funded by MCIN/AEI/10.13039/501100011033/FEDER, EU "ERDF a way of making Europe"; PID2024-155693NB-C44, funded by MICIU/AEI/10.13039/501100011033/FEDER, EU; TED2021-131023B-C21 funded by MCIN/AEI/10.13039/501100011033, EU "NextGenerationEU"/PRTR; and the Horizonte 2027 doctoral training program funded by Junta de Andalucía (Consejería de Universidad, Investigación e Innovación).

References

1. Al-Roomi, M., Al-Ebrahim, S., Buqrais, S., Ahmad, I.: Cloud computing pricing models: a survey. Int. J. Grid Distrib. Comput. **6**(5), 93–106 (2013)
2. AWS Verified Permissions. https://aws.amazon.com/verified-permissions/. Accessed Sep 2025
3. Benavides, D., Segura, S., Ruiz-Cortés, A.: Automated analysis of feature models 20 years later: a literature review. Inf. Syst. **35**(6), 615–636 (2010)
4. Cavero, F.J., Parejo, J.A., Alonso, J.C., Ruiz-Cortés, A.: A-MINT: an LLM pipeline for automated modeling of iPricings from SaaS pricing pages. In: International Conference on Service Oriented Computing (2025)
5. CEDAR. https://docs.cedarpolicy.com. Accessed Sep 2025
6. Fowler, M.: Feature toggles (aka feature flags) (nd). https://martinfowler.com/articles/feature-toggles.html. Accessed Apr 2025
7. García-Fernández, A., Parejo, J.A., Cavero, F.J., Ruiz-Cortés, A.: Racing the market: an industry support analysis for pricing-driven devops in SaaS. In: International Conference on Service-Oriented Computing, pp. 260–275 (2024)
8. García-Fernández, A., Parejo, J.A., Ruiz-Cortés, A.: iSubscription: bridging the gap between contracts and runtime access control in SaaS: the laboratory package (2025). https://doi.org/10.5281/zenodo.16106711
9. García-Fernández, A., Parejo, J.A., Trinidad, P., Ruiz-Cortés, A.: Towards Pricing4SaaS: a framework for pricing-driven feature toggling in SaaS. In: Web Engineering. ICWE, pp. 389–392. Springer (2024). https://doi.org/10.1007/978-3-031-62362-2_30
10. García-Fernández, A., Parejo, J.A., Trinidad, P., Ruiz-Cortés, A.: Automated analysis of pricings in SaaS-based information systems. In: International Conference on Advanced Information Systems Engineering, pp. 223–239 (2025)
11. Ghaddar, A., Tamzalit, D., Assaf, A., Bitar, A.: Variability as a service: outsourcing variability management in multi-tenant SaaS applications. In: Proceeding of the International Conference of Advanced Information Systems Engineering, (CAISE), pp. 175–189 (2012)
12. Hallsteinsen, S., Hinchey, M., Park, S., Schmid, K.: Dynamic software product lines. Computer **41**(4), 93–95 (2008)
13. Kang, K.C., Cohen, S.G., Hess, J.A., Novak, W.E., Peterson, A.S.: Feature-oriented domain analysis (foda) feasibility study. Tech. rep. (1990)
14. Lee, A.J., Winslett, M.: Safety and consistency in policy-based authorization systems. In: Proceedings of the 13th ACM Conference on Computer and Communications Security, pp. 124–133 (2006)
15. Molino-Peña, E., García, J.M., Ruiz-Cortés, A.: Integrating terms of service and service level agreements for automating cloud service management. In: International Conference on Service-Oriented Computing (2025)
16. Good practices when modelling pricings with Pricing2Yaml. https://tinyurl.com/pricing2yaml-good-practices. Accessed May 2025
17. Rahman, M.T., Querel, L.P., Rigby, P.C., Adams, B.: Feature toggles: practitioner practices and a case study. In: Proceedings - 13th Working Conference on Mining Software Repositories, MSR 2016, pp. 201–211 (2016)
18. Serbout, S., El Malki, A., Pautasso, C., Zdun, U.: API rate limit adoption–a pattern collection. In: Proceedings of the 28th European Conference on Pattern Languages of Programs, pp. 1–20 (2023)

19. Tërnava, X., Lesoil, L., Randrianaina, G.A., Khelladi, D.E., Acher, M.: On the interaction of feature toggles. In: Proceedings of the 16th International Working Conference on Variability Modelling of Software-Intensive Systems (2022)
20. Weyns, D., et al.: On patterns for decentralized control in self-adaptive systems. In: Software Engineering for Self-adaptive Systems II: International Seminar, pp. 76–107 (2013)

OSAF: Open Service-Available First Routing Mechanism for Computing Power Network

Yunkang Zhang[1], Qiang Wu[2(✉)], Ran Wang[1], Jie Haou[1], Hao Liu[1], and Yiyun Xu[1]

[1] Faculty of Computing, Nanjing University of Aeronautics and Astronautics, Nanjing, China
{yunkang.zhang,wangran,haojie,liuhaocs,xuyiyun}@nuaa.edu.cn
[2] Zhejiang University, Hangzhou, China
wu.qiang@nuaa.edu.cn

Abstract. The convergence of networking and computing has become a critical architectural paradigm due to the increasing demand for services offering high availability and ultra-low latency, including immersive video and wide-area high-performance computing. However, increasing heterogeneity in computing resources and tighter coupling with algorithms have exacerbated the mismatch between resource supply and service demands. The imperative necessity for service-aware and resource-adaptive routing solutions is underscored by the fact that traditional location-based routing mechanisms neglect service semantics. Against this backdrop, the Computing Power Network (CPN) has emerged, integrating network and computing to enable dynamic, SLA-driven task orchestration. Although substantial progress has been achieved in task scheduling and orchestration in the existing work, the critical issue of service-level routing has not been completely addressed. To bridge this gap, we propose OSAF, a lightweight and backward-compatible service routing mechanism for CPN. OSAF introduced Service Tag (STag) as location-independent semantic identifier, which decouple service requests from host addresses. This enables users to access integrated services without being aware of resource locations. Our design of Service-Aware Routing (SAR) is based on OSAF. This routing algorithm utilizes reinforcement learning and multi-layer graph attention to model fine-grained resource dependencies and adapt to real-time service dynamics. Extensive evaluations of large-scale, dynamic service scenarios under CPN have shown that OSAF outperforms the current routing mechanisms in terms of SLA fulfillment, scalability, and load balancing.

Keywords: Service Tag · Service-Aware Routing · Computing Power Network

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 67–82, 2026.
https://doi.org/10.1007/978-981-95-5015-9_5

1 Introduction

The convergence of networking and computing has become a critical architectural paradigm to address the increasing demand for ultra-low latency, high availability, and dynamic adaptability in wide-area high-performance computing and immersive video interaction services. Nevertheless, high-performance algorithms and computing resources are becoming increasingly diverse, and there is a growing trend of stronger coupling between high-performance algorithms and infrastructure, leading to more pronounced supply-demand mismatches. This challenge is difficult to effectively address due to the complete disregard of service semantics by conventional location-based routing mechanisms. Therefore, the network needs to shift from a passive transmission role to an active service participant to achieve precise resource matching.

As an architectural foundation for the convergence of computing and networking, the Computing Power Network (CPN) has gained attention to facilitate this shift. Defined by ITU-T Recommendation Y.2501 [1], CPN envisions a unified infrastructure that seamlessly integrates communication, computing, storage, and sensing capabilities. Recognized as a key enabler for 6G by the IMT-2030 promotion group [2], and actively pursued by the IETF through the Computing-Aware Traffic Steering (CATS) working group [3], CPN represents a broader movement toward service-centric, resource-aware network architectures—where routing decisions must simultaneously consider network conditions and computing resources.

Despite growing interest, the development of service-level routing mechanisms in CPN remains in its infancy. Prior efforts have explored partial solutions: for instance, DCoflow introduces a deadline-aware coflow scheduler for datacenter networks, but suffers from static scheduling limitations [4]; CMDP-based routing enhances deadline guarantees in wireless mesh networks, but relies on coarse offline policies [5]; and container-level orchestrators such as AiW/COME offer SLA-aware coordination, but lack support for fine-grained, packet-level routing [6]. While these works address aspects of computing-network interaction, none offer a unified, semantically driven routing mechanism capable of dynamically adapting to service requirements and real-time resource fluctuations.

Designing such a mechanism is inherently challenging. First, service-oriented routing differs fundamentally from traditional IP-based routing: whereas traditional IP-based routing relies on fixed, prefix-based addressing, service-aware systems must resolve abstract service intents to a potentially dynamic set of endpoints distributed across the cloud-edge continuum. This invalidates classical location-based semantics. Second, routing in CPN environments must jointly optimize across multiple dimensions—network delay, bandwidth availability, and computational load—which are tightly coupled and often interdependent. These multi-objective tradeoffs significantly increase algorithmic complexity and hinder compatibility with existing infrastructures. Third, the highly dynamic nature of smart environments demands routing mechanisms that are both adaptive and stable in the face of frequent changes in service demand and resource availability (Fig. 1).

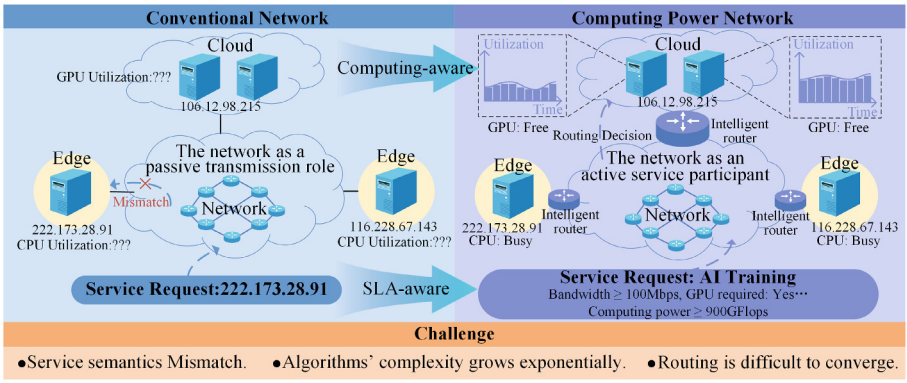


Fig. 1. Comparison between traditional networks and computing power networks, highlighting routing challenges in CPN.

To address these challenges, we propose Open Service-Available First Routing (OSAF), a lightweight and backward-compatible routing mechanism for CPN. OSAF introduces Service Tag (STag) as semantically rich, location-independent identifier that decouple service logic from physical host locations and enable dynamic service resolution. On top of OSAF, we design Service-Aware Routing (SAR), an intelligent routing algorithm that employs multi-layer Graph Attention Networks (GAT) for deep resource-state modeling and a dueling Double Deep Q-Network (DDQN) for stable, adaptive decision-making.

The contributions of this paper are threefold:

- We propose OSAF, a semantic service routing mechanism for CPN that introduces STag as location-independent service identifier. STag enables fine-grained SLA-awareness and robust service continuity across dynamic and heterogeneous environments.
- We design SAR, a deep learning-based routing algorithm that transforms complex multi-resource optimization into a linearized decision process. SAR leverages a multi-layer GAT to capture hierarchical dependencies and a dueling DDQN to improve decision accuracy and learning stability.
- We conduct extensive experiments across large-scale, dynamic service environments. Results show that OSAF achieves superior performance in SLA fulfillment, convergence speed, resource utilization, and load balancing, significantly outperforming existing baselines.

2 Related Work

The convergence of computing and networking has catalyzed the development of CPN. While some initial efforts have been made toward service-aware routing in CPN, existing research remains limited in scope and depth. Tang et al. [7]

proposed an intent-driven CPN framework to improve service granularity and resource coordination, while Xie et al. [8] investigated latency-optimized routing mechanisms that incorporate compute-awareness into decision-making. These efforts underscore the growing need for routing schemes that can dynamically adapt to fluctuating service demands and heterogeneous resource landscapes.

Traditional routing mechanisms, such as OSPF, fall short in such dynamic settings due to their static, location-centric architectures and lack of service-level semantics [9]. To bridge this gap, recent studies have introduced machine learning and reinforcement learning techniques into routing. Deep neural networks have been employed for traffic prediction and task scheduling [10], and Deep Reinforcement Learning (DRL)-based strategies—such as Deep Q-Network(DQN) and actor-critic models—have shown promise in edge computing [11]. However, many of these approaches rely on flat topology representations and struggle to capture long-range dependencies in multi-hop scenarios. GDQR [12] addresses multi-factor optimization, but lacks comprehensive topological awareness. Graph neural networks have been leveraged to improve structural modeling [13], though often at the cost of computational complexity and scalability.

Meanwhile, service-aware and computing-network collaborative routing mechanisms have received increased attention. Zhang et al. [14] proposed a deadline-sensitive routing strategy that jointly optimizes computation and communication. STAR [15] enhances service awareness capabilities through routing segmentation, but is still limited by the inability to perceive computing resources in real-time. CaRCS [16] integrates computing-aware routing and collaborative scheduling, yet its adaptability is limited in highly dynamic or large-scale environments due to delayed state synchronization and coordination overhead.

In contrast to these efforts, our proposed OSAF combines a hierarchical GAT for multi-hop topology modeling with a dual-path decision process that decouple computing and network optimization. Additionally, a dueling DDQN-based feedback loop enhances adaptability to service dynamics. By introducing location-independent STag, OSAF enables semantic-level service identification and SLA-aware routing across distributed, heterogeneous CPN environments.

3 Open Service-Available First Routing Mechanism

3.1 Service Tag

In OSAF, the introduction of STag provides a new interface for IP packet networks that is oriented towards service and computing systems, enabling the network to offer addressing and routing functions based on service semantics. STag, as a bridge between services and resources, serve as an important index for decision-making in SAR and are also a location-independent identifier.

STag in the data plane is a lightweight representation of a set of computing and networking service capabilities aimed at users. The network quality and capabilities associated with STag can be managed and controlled within specific network operation management domains, such as a certain video service with an end-to-end guaranteed bandwidth of 20 Mbit/s or a rendering service with an

end-to-end delay guarantee of 10 ms. Therefore, STag inherently supports the characterization and interface of refined computing network SLA requirements.

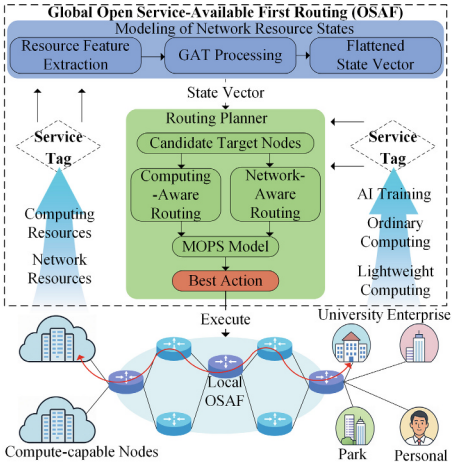


Fig. 2. Operation procedure of OSAF

The control plane, responsible for running OSAF, generates routing and forwarding policies indexed by STag based on the SLA profile of CPN and the resource status of CPN, and issues them to the service gateway to guide service traffic forwarding. Taking the forwarding and routing process of service traffic by the entry service gateway as an example, user-side messages express the intention to access specific computing services in the computing system through STag, as well as the SLA requirements of these services in CPN. STag here does not point to specific hosts, but rather the service gateway selects specific service hosts and network links based on the control plane's cloud-network policy table, thereby achieving both computing power optimization among multiple service instances and path optimization among multiple network paths, providing refined cloud-network policy arrangements for the corresponding services. It can be seen that STag act as resource perception interfaces between the network and computing systems in the east-west direction, and as efficient perception interfaces for customized service SLA requirements between the network and service in the north-south direction (Fig. 2).

It should be noted that STag itself does not need to contain information and parameters of the service SLA requirements; it only needs to serve as a mapping interface between the data plane and the control plane. The semantic of service SLA requirements is maintained and represented by the control plane. The manageable and controllable, lightweight design of service identification addresses security issues without adding extra burden to the hardware processing performance of the service gateway.

3.2 Service-Aware Routing

As the adaptive routing module of OSAF, SAR dynamically plans service delivery paths based on both infrastructure status and SLA requirements.

Modeling of Network Resource States. Our multi-layer multi-head GAT (MMGAT) dynamically captures multi-dimensional resource interdependencies among diverse network nodes. It serves as SAR's semantic abstraction engine, transforming topological data into hierarchical embeddings for routing decisions.

MMGAT dynamically monitors four normalized metrics at each node $V_i \in \mathcal{V}$:

– **Computational resource utilization** (C_i): Quantifies node-level computing utilization to identify processing bottlenecks:

$$C_i = 1 - \frac{C_i^{\text{avail}}}{C_i^{\text{cap}}}, \tag{1}$$

where C_i^{avail} and C_i^{cap} denote the available and total computing resources of node i, respectively.

– **Bandwidth utilization** (B_i): Reflects aggregated link congestion risks across neighboring nodes:

$$B_i = \frac{\sum_{j \in \mathcal{N}(i)} \left(B_{ij}^{\text{cap}} - B_{ij}^{\text{avail}} \right)}{\sum_{j \in \mathcal{N}(i)} B_{ij}^{\text{cap}}}, \tag{2}$$

where $\mathcal{N}(i)$ is the neighbor set of V_i, and B_{ij}^{cap}, B_{ij}^{avail} represent the bandwidth capacity and availability of link (i, j).

– **Delay factor** (D_i): To quantify the composite delay at node i, we consider both the local processing delay and the average transmission delay to its neighbors. The normalized delay factor is defined as:

$$D_i = \frac{D_i^{\text{proc}} + \frac{1}{|N(i)|} \sum_{j \in N(i)} D_{ij}^{\text{trans}} - D_{\min}}{D_{\max} - D_{\min}}, \tag{3}$$

where D_i^{proc} is the processing delay at node i, D_{ij}^{trans} is the transmission delay between node i and its neighbor j, and $N(i)$ denotes the set of neighbors of node i. Here, $D_{\min}$ and $D_{\max}$ represent the minimum and maximum composite delays across all nodes in the network.

– **Load balance degree** (L_i): Measures local resource distribution uniformity to guide global routing:

$$L_i = \frac{\left(\sum_{j \in \mathcal{N}(i)} B_{ij} \right)^2}{|\mathcal{N}(i)| \cdot \sum_{j \in \mathcal{N}(i)} B_{ij}^2}, \tag{4}$$

with B_{ij} derived from Equation (2) as B_{ij} = 1 - $B_{ij}^{\text{avail}}/B_{ij}^{\text{cap}}$.

And the types of service equipment $T_i \in \mathcal{T}$. These metrics are synthesized into an initial feature vector $\mathbf{X}_i = [C_i, B_i, D_i, L_i, T_i]^\top$, which captures node-specific resource states. To model topological dependencies, MMGAT applies a three-layer hierarchical attention structure (Eqs. (5)–(6)), where 1-hop, 2-hop, and 3-hop correlations are progressively aggregated. Compared with conventional flat GAT, this design yields finer-grained representations of computing-network interplay and better reflects semantic service demands.

$$\mathbf{H}_i^{(l)} = \mathcal{F}_{\text{agg}} \left(\bigcup_{k=1}^{K} \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}^{(k)} \mathbf{H}_j^{(l-1)} \right) \right), \tag{5}$$

where multi-head outputs are aggregated via concatenation or averaging, with residuals, dropout ($p = 0.2$), and layer normalization for stability.

The final embeddings are obtained as:

$$\mathbf{s}_t = \mathrm{vec}\Big(\mathcal{F}^{(3)}_{\mathrm{GAT}}\Big(\mathcal{F}^{(2)}_{\mathrm{GAT}}\Big(\mathcal{F}^{(1)}_{\mathrm{GAT}}(\mathbf{X})\Big)\Big)\Big) \in \mathbb{R}^{N\cdot d'}, \tag{6}$$

which jointly encodes node resources and topology. On this basis, SAR integrates dueling DDQN with a multi-objective reward that dynamically balances SLA, utilization, and latency, while STag-aware fusion couples semantic identifiers with learned embeddings. This enables more stable convergence and stronger routing robustness than conventional GAT–DRL methods. MMGAT further avoids explicit adjacency matrices by deriving attention weights from $\mathcal{N}(i)$, improving adaptability to dynamic links.

Routing Planner. Routing Planner (RPL) serves as the decision-making core of SAR, performing service-aware routing by evaluating both computing and network conditions. RPL first associates the service request with its STag $st_i \in \mathcal{ST}$ ($\mathcal{ST}$ is a collection of service types), and retrieves the corresponding SLA file $\Phi(st_i) = (C_i, B_i, D_i, T_i)$, selecting the target nodes based on the service request conditions. The use of STag enables semantic identifiers to be directly integrated with learned embeddings, so that routing decisions account not only for dynamic resource states but also for service-level intents. SLA requirements further guide the multi-objective optimization in SAR, ensuring adaptive trade-offs among latency, resource utilization, and service continuity.

For each candidate node, RPL constructs two routing paths: computing-aware path and network-aware path. These two paths are jointly evaluated using a multi-objective path scoring model. Finally, the path with the highest score and its corresponding node is determined.

Computing-Aware Path Generation. This process leverages a dueling DQN architecture to decouple global resource value estimation from local routing decisions. At each step, the agent selects the next-hop node with the highest Q-value:

$$Q(s,a) = V(s) + \left(A(s,a) - \frac{1}{|A|}\sum_{a'} A(s,a')\right), \tag{7}$$

where $V(s)$ captures global patterns such as underutilized clusters or congested subnets, and $A(s,a)$ focuses on the relative suitability of each neighbor. This design enables the agent to distinguish between globally promising regions and locally optimal hops. For instance, in distributed AI training workloads, RPL prefers routes through nodes with surplus computational capacity, even if intermediate links are moderately loaded.

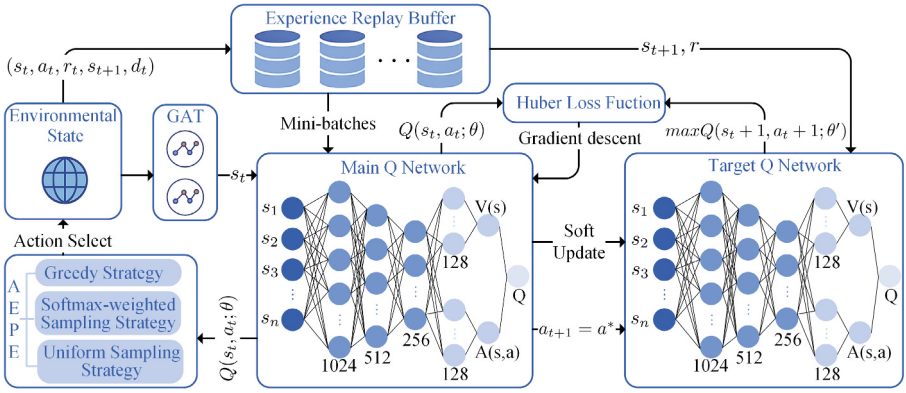


Fig. 3. Overall training process of SAR algorithm

Network-Aware Path Generation. To minimize transmission cost, RPL employs a hybrid delay-bandwidth metric:

$$\mathrm{COST}(u,v) = \alpha \cdot \mathrm{delay}(u,v) + (1-\alpha) \cdot \frac{C_{\mathrm{ref}}}{\mathrm{bandwidth}(u,v)}, \tag{8}$$

where $\alpha \in [0,1]$ is dynamically adjusted based on the service type. Compute-intensive services reduce α to prioritize bandwidth, while latency-sensitive services increase α to emphasize low delay. This allows RPL to flexibly adapt to diverse service demands without relying on static routing policies.

Multi-objective Path Scoring. Once both paths are generated, RPL selects the optimal node-path pair using:

$$\begin{aligned}\mathrm{FinalScore}(P) = \lambda_1 \cdot (w_1 \cdot \mathrm{NetUtil} + w_2 \cdot \mathrm{CompUtil} \\ + w_3 \cdot \mathrm{Delay}) + \lambda_2 \cdot \frac{1}{|P|} \sum Q(s,(u,v)),\end{aligned} \tag{9}$$

where static metrics (e.g., link congestion, node load, latency) are weighted against dynamic Q-values from the learning agent. This unified scoring ensures that RPL selects routing paths that balance performance, resource efficiency, and service-awareness.

Routing Optimization Trainer. Routing Optimization Trainer (ROT) refines routing decisions in SAR through a feedback-driven DRL loop. Unlike static routing strategies, ROT enables continuous policy evolution under dynamic service demands and network conditions. As illustrated in Fig. 3, it integrates DDQN, a multi-objective reward function, and an adaptive exploration strategy.

To balance routing efficiency and fairness, we design a composite reward:

$$r = \omega_1 \cdot \mathcal{O}_1 + \omega_2 \cdot \mathcal{O}_2 + \mathcal{O}_3, \tag{10}$$

where $\mathcal{O}_1$ evaluates path efficiency (e.g., hop count, cost), $\mathcal{O}_2$ quantifies resource utilization (e.g., residual computing power, bandwidth), and $\mathcal{O}_3 = -0.5$ penalizes routing failures. The coefficients ω_1 and ω_2 adaptively shift based on the service semantics (e.g., latency-sensitive services increase ω_1, compute-intensive services increase ω_2), enabling SAR to perform service-specific policy optimization.

ROT adopts DDQN to avoid Q-value overestimation common in vanilla DQN. Specifically, the action is selected using the main network Q_θ, while its value is evaluated by the target network $Q_{\theta'}$:

$$a^* = \arg\max_{a'} Q_\theta(s', a'), \tag{11}$$

$$Q_{\text{target}}^{\text{DDQN}} = r + \gamma \cdot Q_{\theta'}(s', a^*). \tag{12}$$

This dual-network approach improves training stability, especially in large-scale state spaces. Additionally, a Huber loss is applied:

$$L(\theta) = \begin{cases} \frac{1}{2}(Q - Q_{\text{target}})^2, & |Q - Q_{\text{target}}| \leq \delta \\ \delta \cdot (|Q - Q_{\text{target}}| - \frac{\delta}{2}), & \text{otherwise} \end{cases} \tag{13}$$

which smooths out large gradients and enhances robustness against outliers.

To further improve convergence and adaptability, we propose an Adaptive Exploration and Policy Evolution (AEPE) strategy. Exploration is triggered with probability ϵ, which decays exponentially over time:

$$\epsilon \leftarrow \max\left(\epsilon \cdot \text{decay_rate}, \epsilon_{\min}\right). \tag{14}$$

With probability 0.3ϵ, actions are sampled using softmax exploration:

$$P(a) = \frac{\exp(Q(s,a)/T)}{\sum_{a'} \exp(Q(s,a')/T)}, \tag{15}$$

where T controls entropy and encourages policy diversity; the remaining 0.7ϵ enables uniform sampling to escape local optima. When not exploring, ROT selects the greedy action $\arg\max_a Q(s,a)$.

Through this design, ROT ensures that routing policies evolve with the network environment, achieving a balance between performance, adaptability, and service-level fairness.

4 Performance Evaluation

We first train SAR algorithm offline, then evaluate the performance of all algorithms, and finally assess the robustness and efficiency of OSAF in various scenarios.

Table 1. Parameter Settings with Statistical Distributions

Parameter	Values
Link failure probability	$0.005 \sim 0.02$
Link recovery probability	0.015
Node failure probability	$0.001 \sim 0.005$
Node recovery probability	$0.2 \sim 0.4$
Bandwidth fluctuation range	$\mu \pm \sigma = \pm(5\% \sim 20\%)$
Bandwidth fluctuation interval	$\lambda = 0.1 \sim 0.3$/timestep
Computing resource fluctuation range	$\mu \pm \sigma = \pm(10\% \sim 30\%)$
Computing resource fluctuation interval	$\lambda = 0.08 \sim 0.25$/timestep
Latency fluctuation probability	$0.15 \sim 0.35$
Latency increase ratio	$10\% \sim 100\%$
Resource recovery rate	$3\% \sim 6\%$/timestep
Exploration rate decay	0.999
Discount factor	0.95
Training episodes	3000
Replay buffer size	20,000
Batch size	128
Learning rate (DQN/GAT)	2×10^{-5} / 5×10^{-5}

4.1 Experimental Setup

Experiments are conducted in a highly realistic simulation environment using a Python-based tool built on the NetworkX library. We selected 16 types of network topologies from the Internet Topology Zoo dataset for evaluation [17]. To replicate authentic network dynamics, we employ statistical models for failures and fluctuations: link/node failures follow exponential and Bernoulli distributions for sporadic outages, bandwidth/computing variations use normal distributions (magnitude) and Poisson processes (timing), latency spikes adopt Weibull ($k = 1.5$) and power-law ($\alpha = 2.5$) models capturing time-dependent delays and heavy-tailed congestion, and resource recovery follows log-normal distributions reflecting gradual restoration. Detailed statistical parameters are configured in Table 1. This setup facilitates accurate emulation of dynamic network behaviors such as link/node failures, resource fluctuations, and latency changes.

Each node is assigned computing resources based on a hierarchical allocation strategy using centrality metrics (50% degree, 30% betweenness, 20% closeness):

- Top 20% nodes: high-performance, 3500–6000 Gflops.
- Next 30%: medium-performance, 1500–3500 Gflops.
- Remaining 50%: low-performance, 600–1500 Gflops.

Link bandwidth is classified based on node types and importance:

- High–high: 400–700 Mbps.

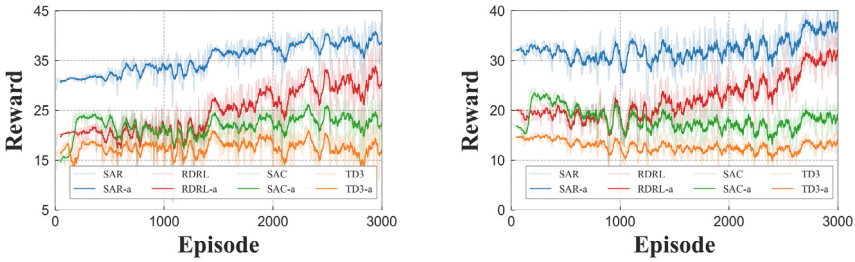


Fig. 4. Convergence performance of the DRL-based algorithms, including RDRL, and DRL. Figures (a), (b) show the learning process under different CPN node scales of (a) 50, (b) 100, respectively.

– High–others: 250–400 Mbps.
– Medium–medium: 80–150 Mbps.
– Medium–low: 40–80 Mbps.
– Low–low: 10–40 Mbps.

Bandwidth is fine-tuned by link importance to ensure critical paths are well-provisioned. And failures are introduced during testing to evaluate robustness.

4.2 Algorithm Training

The following algorithms were used for comparison. Rand DRL (RDRL) randomly selects candidate resources while narrowing the purview of resources within CPN. Twin Delayed Deep Deterministic (TD3) policy gradient algorithm [18] is widely used DRL algorithm without our improvements. Soft Actor-Critic (SAC) algorithm [19] is another traditional DRL algorithm without our improvements. Figure 4 shows the training results of various algorithms across different CPN scales. Each algorithm underwent 3000 training sessions under optimal hyperparameters, with the average reward per 30 steps indicating the convergence trend; smoothed curves are denoted with "−a".

SAR leverages MMGAT with dueling DDQN to achieve fine-grained modeling of resource states and stable reward estimation. This design improves both policy update precision and exploration efficiency, enabling correct exploration and fast learning toward optimal returns. Unlike RDRL, which relies on random exploration and thus converges more slowly, SAR exhibits smoother convergence with reduced variance, demonstrating strong stability against noise. Traditional DRL algorithms such as TD3 and SAC are more prone to instability in large-scale CPNs: TD3 suffers from poor adaptability in large action spaces, while SAC often converges to local optima with oscillatory performance. In contrast, SAR's multi-objective reward and STag-aware fusion provide inherent regularization, preventing reward overestimation and avoiding frequent oscillations, which ensures reliable and robust training dynamics as the network size increases.

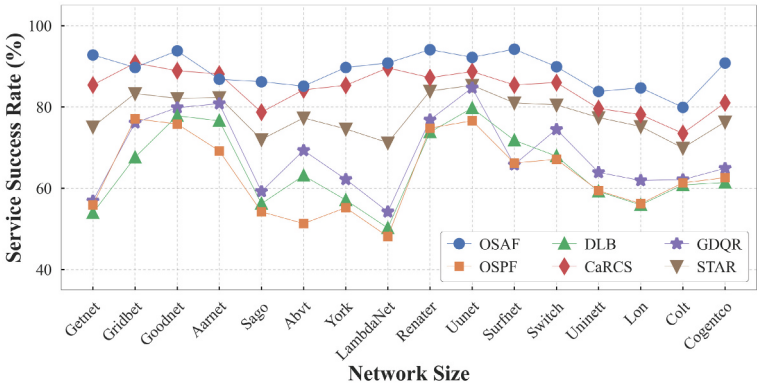


Fig. 5. Service Success Rate Comparison

4.3 Efficiency

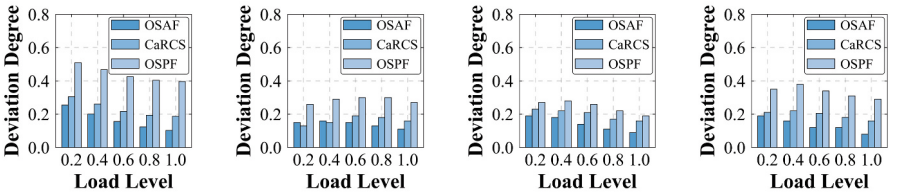
As shown in Fig. 5, we evaluated the Service Success Rate of multiple routing mechanisms on 16 representative CPN topologies ranging from small to large scale.

OSAF achieved consistently high performance, with success rates above 93% in medium-scale networks such as Renater and Surfnet, reaching 94.2% in Switch, and still maintaining 90.8% in the complex Cogentco topology. These results highlight its scalability and robustness, attributed to the semantic abstraction of STag and SAR's dynamic modeling of computing and network states for adaptive multipath routing.

CaRCS [16] was competitive in small and medium topologies (88–90%) but its performance declined to 73.5–81.0% in large-scale networks due to centralized state synchronization overhead. STAR [15], though service-oriented, showed limited adaptability: it reached 83–85% in Renater and Uunet but dropped to 76.3% in Cogentco, reflecting its NFV-centric design. OSPF exhibited the weakest results, barely maintaining 75% in regular structures like Renater and Surfnet and falling to 61–63% in Colt and Cogentco, exposing the limitations of static shortest-path routing in dynamic CPNs.

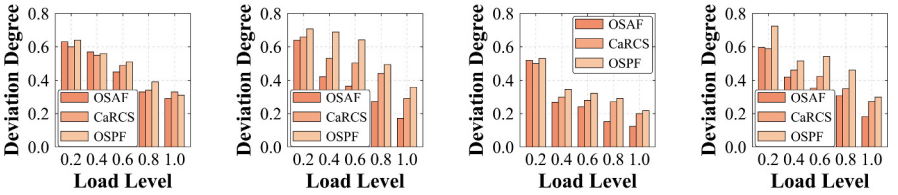
DLB [10] achieved 76–78% in medium networks such as Goodnet and Aarnet by leveraging hash-based load balancing but degraded in complex topologies due to inference overhead. GDQR [12] surpassed 80% in Uunet and Surfnet by capturing global topology through GNNs, yet fell to 64.9% in Cogentco, showing scalability constraints under large action spaces.

In summary, OSAF demonstrates superior adaptability, stability, and scalability across heterogeneous topologies. Its improvements confirm the effectiveness of SLA-aware semantic routing and underline the promise of the "service-computing-network" collaborative paradigm in CPN.



(a) Compute-intensive services (b) Network-intensive services (c) Balanced services (d) Heterogeneous services

Fig. 6. Comparison of Computing Load Deviation.



(a) Compute-intensive services (b) Network-intensive services (c) Balanced services (d) Heterogeneous services

Fig. 7. Comparison of Network Load Deviation.

4.4 Robustness

We demonstrate the robustness of OSAF by comparing its computational and network load deviation. Deviation is the average of the test results for 16 network topologies. Service demand is modeled with three types: compute-intensive services, network-intensive services and balanced services. Computing resources requirements are 4000–5000, 500–1500 and 1000–3000 Gflops, respectively. Bandwidth needs are 10–50, 100–300 and 50–150. Durations vary from 5 to 120 units. Arrival follows a 60% burst (0–0.5 unit interval) and 40% regular (0.5–2 units) pattern. 30% of services have dependencies, with features such as GPU use, data intensity and latency sensitivity. And heterogeneous services are a balanced mix of three types of tasks.

Computational Load Deviation. We evaluate the robustness of OSAF by comparing its computational load deviation with CaRCS and OSPF under different service types and load levels (0.2–1.0), averaging the results across 16 network topologies. As shown in Fig. 6, OSAF consistently achieves lower deviations across all scenarios, indicating its favorable load balancing performance.

In compute-intensive services, OSAF's load deviation decreased from 0.255 to 0.103, outperforming CaRCS (from 0.305 to 0.187) and OSPF (from 0.510 to 0.397). For network-intensive services, OSAF maintains a stable and low

load deviation (0.11–0.16), while CaRCS exhibits noticeable fluctuations. OSPF, which focuses solely on network metrics, shows significantly higher deviation. Similar trends are observed in balanced and heterogeneous service scenarios, where OSAF achieves the lowest deviation at full load (0.09 and 0.08, respectively).

It is worth noting that we observed that under low load and small-scale conditions, CaRCS performed slightly better than OSAF. However, as the network scale and service traffic increase, the performance gap between OSAF and CaRCS widens, highlighting OSAF's superior scalability and adaptability in dynamically high-pressure environments. This is because its centralized coordination becomes a bottleneck under high pressure. These results indicate that OSAF is more effective in achieving fine-grained computational load balancing, especially in high-load and heterogeneous service scenarios in large-scale CPNs.

Network Load Deviation. Figure 7 shows the network load deviation of OSAF, CaRCS, and OSPF under four service types and increasing load levels. The deviation reflects the maximum path utilization gap, serving as an indicator of routing-level load balancing.

In the process of handling network-intensive services, OSAF consistently achieves the lowest load deviation (0.640–0.171) at all load levels. In the other three cases, CaRCS performed better under low load and smaller network scales, but it neglects service semantics, and its performance becomes worse compared to OSAF as the load level increases. However, although OSPF's overall performance in CPN is not as good as that of OSAF and CaRCS, it performs better under specific conditions because it only focuses on network factors. As the load level of OSAF continues to increase, its performance has been improving and maintaining a more stable downward trend, which highlights the importance of focusing on service semantics.

These results indicate OSAF demonstrates improved robustness when subjected to elevated load levels, achieving more balanced and efficient routing decisions in dynamic CPNs.

5 Conclusion and Discussion

In this paper, we presented OSAF, a semantic service routing mechanism for CPN. By introducing location-independent STag, OSAF decouples service requirements from physical infrastructure, enabling adaptive routing and service continuity across heterogeneous environments. We further developed SAR, a DRL-based routing module that jointly optimizes computing and networking resources via multi-layer GAT and dueling DDQN. Extensive evaluations under dynamic large-scale CPNs show that OSAF significantly improves SLA fulfillment, resource utilization, and routing robustness compared with existing approaches.

Beyond performance, OSAF also raises deployment, scalability, and security considerations. The mechanism can be integrated with existing infrastructures through SDN controllers or overlay routing, remaining compatible with

OSPF/BGP. Since SAR training is offline, online inference incurs only limited control-plane overhead, while lightweight STag dissemination mitigates scalability concerns. At the same time, semantic identifiers introduce new security and privacy challenges, which can be addressed through authentication and lightweight encryption. Future work will explore these directions, extend OSAF toward Kubernetes-based orchestration and multi-domain federation, and validate its feasibility through a small-scale testbed prototype.

References

1. International Telecommunication Union (ITU): Computing power network – Framework and architecture. ITU-T Recommendation Y.2501 (2021). https://handle.itu.int/11.1002/1000/14768
2. IMT-2030 (6G) Promotion Group: White Paper on 6G Vision and Candidate Technologies. China Academy of Information and Communications Technology (CAICT) (2021). http://www.caict.ac.cn/english/news/202106/t20210608_378637.html
3. Li, C., et al. (eds.): A framework for computing-aware traffic steering (CATS). IETF Internet-Draft Draft-IETF-CATS-framework-10, Informational (2025). https://datatracker.ietf.org/doc/draft-ietf-cats-framework-10
4. Luu, Q.-T., Brun, O., El-Azouzi, R., De Pellegrini, F., Prabhu, B.J., Richier, C.: DCoflow: deadline-aware scheduling algorithm for coflows in datacenter networks. In: 2022 IFIP Networking Conference, pp. 1–9. IEEE, Catania (2022)
5. Wang, X., Wang, X.: Distributed scheduling for throughput maximization under deadline constraint in wireless mesh networks. arXiv preprint arXiv:2407.10407 (2024)
6. Mampage, A., Karunasekera, S., Buyya, R.: Deadline-aware dynamic resource management in serverless computing environments. In: 2021 IEEE/ACM CCGrid, pp. 483–492. IEEE, Melbourne (2021)
7. Tang, Q., et al.: SIaTS: a service intent-aware task scheduling framework for computing power networks. IEEE Netw. **38**, 233–240 (2024)
8. Xie, R., et al.: Delay-prioritized and reliable task scheduling with long-term load balancing in computing power networks. IEEE Trans. Serv. Comput. **17**, 3359–3372 (2024)
9. Dabaghi, F., Movahedi, Z., Langar, R.: A survey on green routing protocols using sleep-scheduling in wired networks. J. Netw. Comput. Appl. **77**, 106–122 (2017)
10. Zhu, X., Zhang, Q., Cheng, T., Liu, L., Zhou, W., He, J.: DLB: deep learning based load balancing. In: Proceedings of the 2021 IEEE 14th International Conference on Cloud Computing (CLOUD), pp. 1–10. IEEE, Chicago, IL, USA (2021)
11. Jain, P., Singhal, P., Pandey, D., Quattrocchi, G., Vaidhyanathan, K.: POSEIDON: efficient function placement at the edge using deep reinforcement learning. In: Dustdar, S., Aiello, M., Charoy, F., Nardelli, M. (eds.) ICSOC 2024. LNCS, vol. 15404, pp. 21–37. Springer, Cham (2024). https://doi.org/10.1007/978-981-96-0805-8_2
12. Zhao, K., Zhao, Z., Wang, Z., Zhang, H.: Routing algorithm design based on deep reinforcement learning and GNN. In: IEEE INFOCOM 2024 Workshops, pp. 1–6. IEEE, Vancouver (2024)

13. Sawada, K., Kotani, D., Okabe, Y.: Network routing optimization based on machine learning using graph networks robust against topology change. In: 2020 International Conference Information Networking (ICOIN), pp. 608–615. IEEE, Barcelona (2020)
14. Nguyen, N.H., Nguyen, V.-D., Nguyen, A.T., Thieu, N.V., Nguyen, H.N., Chatzinotas, S.: Deadline-aware joint task scheduling and offloading in mobile-edge computing systems. IEEE Internet Things J. **11**, 33282–33295 (2024)
15. Chen, R., Zhao, J.: Scalable and flexible traffic steering for service function chains. IEEE Trans. Netw. Serv. Manage. **19**, 2048–2062 (2022)
16. Feng, L., et al.: CaRCS: joint optimization of computing-aware routing and collaborative scheduling in computing power networks. IEEE Netw. 1 (2025)
17. Knight, S., Nguyen, H.X., Falkner, N., Bowden, R., Roughan, M.: The internet topology zoo. IEEE J. Sel. Areas Commun. **29**, 1765–1775 (2011)
18. Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: Proceedings of the 35th International Conference on Machine Learning (ICML), pp. 1587–1596. Stockholm, Sweden (2018)
19. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Dy, J., Krause, A. (eds.) Proceedings of the 35th International Conference on Machine Learning (ICML), vol. 80, pp. 1861–1870 (2018)

A-MINT: An LLM Pipeline for Automated Modeling of iPricings from SaaS Pricing Pages

Francisco Javier Cavero(✉), José Antonio Parejo, Juan C. Alonso, and Antonio Ruiz-Cortés

SCORE Lab, I3US, Universidad de Sevilla, Seville, Spain
{fcavero,japarejo,javalenzuela,aruiz}@us.es

Abstract. SaaS pricing needs intelligent pricings (iPricings)–machine-readable representations that enable automated validation and analysis. Yet today, pricing is published in heterogeneous, ad-hoc HTML with no widely accepted notation or standard, leaving crucial associations (e.g., plan ↔ add-on ↔ subscription) tied to positional or visual cues and thus ambiguous for machines. We introduce A-MINT (Automated Modeling of iPricings from Natural Text), an end-to-end LLM-powered engine that converts free-form pricing pages into valid iPricings. A-MINT combines HTML preprocessing, LLM-driven extraction, automated constraint validation, and an LLM-based correction loop to resolve structural and linkage errors. A-MINT achieves a mean F_1-score of 70.51% on SPECTRA, a dataset containing 162 real-world iPricings from 30 SaaS providers over six years, showing that iPricings can be modeled at scale despite the absence of a standard representation for pricings on the web.

Keywords: Automated Transformation · SaaS · iPricing · LLM

1 Introduction

The exponential growth in complexity and variability of SaaS pricing exposes fundamental limits in current management and analysis practices [7]. Modern offerings can entail hundreds or thousands of valid subscription combinations with intricate constraints and feature interactions [5], yet pricing is still published in ad-hoc, heterogeneous HTML with *no widely accepted notation or standard*. This lack of semantic structure and standardization ties critical associations (e.g., plan ↔ add-on ↔ subscription) to positional or visual cues, making activities such as configuration management, adaptive decision-making, and competitive benchmarking manual, time-consuming, and error-prone. Recent work has introduced *intelligent pricings (iPricings)*–machine-readable models that support automated validation and analysis [1,7]. However, in the absence of a standard representation for pricing information on the web, the manual translation of free-form SaaS webpages into iPricings remains the main bottleneck to scalability and timely decision-making.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 83–91, 2026.
https://doi.org/10.1007/978-981-95-5015-9_6

We introduce *Automated Modeling of iPricings from Natural Text (A-MINT)*, a fully automated system that transforms HTML pricing pages into validated iPricing models through HTML preprocessing, LLM-based extraction, and automated validation. A-MINT produces accurate, reusable models with minimal manual effort that enable downstream tasks such as subscription recommendation, error detection, benchmarking, and even self-adaptive systems that address the challenge of runtime subscription enforcement [8]. Our contributions are: (i) the A-MINT workflow for scalable, reliable pricing transformation; (ii) an empirical evaluation on a large real-world corpus; (iii) a curated update improving ground-truth quality; (iv) a public release of validated iPricings for reproducibility; and (v) an economic cost analysis highlighting optimization opportunities.

We evaluate A-MINT on the SPECTRA dataset[1] [9], which includes 162 SaaS pricing pages from 30 products over a six-year period. A-MINT achieves a mean F_1-Score of 70.51% without human intervention, significantly reducing the manual effort required to build and maintain valid iPricings. In the process we identified and corrected inconsistencies in the ground truth [4], resulting in the largest update to date (over 25% of iPricings revised, 41/162), and we publicly release 158 validated iPricings generated with A-MINT to facilitate reproducibility and comparative evaluation [2]. We also provide a breakdown of the pipeline's economic cost, highlighting where preprocessing and iterative refinement yield savings and where further optimization is most impactful.

The remainder of this paper is organized as follows. Section 2 introduces the background and related work. Section 3 presents the A-MINT workflow, while Sect. 4 reports its evaluation. Finally, Sect. 5 outlines future directions.

Fig. 1. Buffer's pricing excerpt with 10 features, 3 plans and 2 add-ons

[1] The SaaS Pricing Evaluation Corpus for Temporal Representation & Analysis (SPECTRA) dataset was first introduced in [5].

2 Background and Related Work

Early studies on SaaS pricing [13] highlighted the difficulty of representing multiple subscription options while ensuring scalability and adaptability. As SaaS adoption grows [12], metamodels such as Pricing4SaaS [6] formalize pricing by defining its elements and relationships; in this context, *pricing* is a core component of the customer agreement [14], organizing service *features*–distinctive attributes that influence a user's subscription decision [5]–into *plans* and *add-ons* that govern access to them. An illustrative case is Buffer (see Fig. 1): three plans (Free, Essentials, Team), ten features (three with usage limits) and two plan-specific, mutually exclusive *overage add-ons*–optional upgrades that extend the quota of a feature beyond its base allocation–that increase the number of connected channels. Users typically choose one base plan and can optionally add the associated plan-specific extension, allowing limited personalization without excessive complexity. Machine-processable *iPricings* [1] convert web pricing into analyzable models; Fig. 2 shows part of Buffer's iPricing in Pricing2Yaml [3].

As the number of plans and add-ons grows, managing, validating, and reasoning over all combinations becomes central. The *configuration space* denotes the set of all valid configurations derivable from a pricing [5], where a *configuration* is a distinct, valid combination of plans and/or add-ons; Buffer's pricing yields five valid configurations [3]. The notion of *Pricing-driven Development and Operation of SaaS* [5] links product evolution to pricing; transforming static pages into iPricings enables automated analysis and safer and more efficient releases [7]. Moreover, SaaS architecture and pricing are tightly linked [13]. In fact, software architecture not only determines the feasibility of various pricing strategies but also plays a pivotal role during early-stage product design. Conversely, suboptimal architectures may restrict customization, reduce customers' bargaining

```yaml
syntaxVersion: '2.1'
saasName: Buffer
version: '1.0'
createdAt: '2025-05-06'
url: https://buffer.com/pricing
billing:
  monthly: 1
variables: null
currency: USD
features:
  ideas:
    description: Capture and store.
    valueType: BOOLEAN
    type: DOMAIN
    defaultValue: true
  aiAssistant:
    description: Refine and repurpose.
    valueType: BOOLEAN
    type: AUTOMATION
    automationType: BOT
    defaultValue: true
  reports:
    description: Beautiful reports.
    valueType: BOOLEAN
    type: INFORMATION
    defaultValue: false
  ...
usageLimits:
  ideasLimit:
    description: The number of ideas you can store.
    type: NON_RENEWABLE
    valueType: NUMERIC
    unit: ideas
    linkedFeatures:
    - ideas
    defaultValue: 100
  ...
plans:
  FREE:
    description: Connect up to 3 channels
    price: 0
    unit: month
    features: null
    usageLimits:
      includedChannelsLimit:
        value: 3
  ESSENTIALS:
    description: 1 channel
    price: 6
    unit: month
    usageLimits:
      ideasLimit:
        value: .inf
      tagsLimit:
        value: 250
    features:
      reports:
        value: true
      replyToComments:
        value: true
      ...
addOns:
  ADDITIONAL_ESSENTIALS_CHANNEL:
    description: Add channel to Essentials plan.
    price: 6
    unit: channel/month
    features: null
    usageLimits: null
    availableFor:
    - ESSENTIALS
    excludes: null
    dependsOn: null
    usageLimitsExtensions:
      includedChannelsLimit:
        value: 1
  ADDITIONAL_TEAM_CHANNEL:
    description: Add channel to Team plan.
    price: 12
    unit: channel/month
    features: null
    usageLimits: null
    availableFor:
    - TEAM
    excludes: null
    dependsOn: null
    usageLimitsExtensions:
      includedChannelsLimit:
        value: 1
```

Fig. 2. Excerpt of Buffer's iPricing modeled with Pricing2Yaml syntax [3]

power, and ultimately constrain pricing options. Building on a recent conceptual workflow [1], this paper operationalizes that vision with a complete, end-to-end pipeline evaluated on SPECTRA [9], demonstrating reduced manual effort and error and providing a practical foundation for automating the representation of pricing information in machine-readable formats.

3 Transforming Pricings Into iPricings

Automated Modeling of iPricings from Natural Text (A-MINT)[2]–leverages pre-trained LLMs to transform any SaaS pricing webpage into a valid iPricing model, substantially reducing the manual effort required for pricing formalization.

As Fig. 3 shows, the process starts with scraping and processing the HTML content to isolate relevant details. This content is then interpreted by an LLM, which extracts pricing elements and builds an initial iPricing model. An automated validation and correction loop subsequently ensures the model adheres to the formal specification (in our case, Pricing2Yaml [11]) by repairing both syntactic and semantic issues before returning the final validated model.

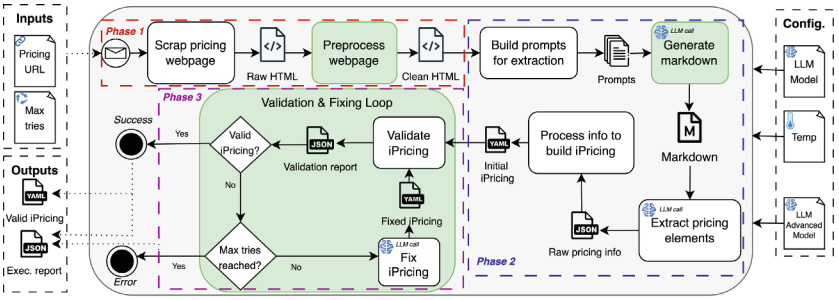


Fig. 3. Workflow of A-MINT

The key innovation of A-MINT lies in this final loop: the *end-to-end automation of both validation and correction*. Unlike previous approaches [1], which required substantial manual curation, A-MINT automatically detects and fixes specification violations and semantic inconsistencies (e.g., invalid configurations). This directly addresses a known limitation in pricing modeling: over 20% of hand-crafted iPricing models in SPECTRA contained structural inconsistencies [7]. A-MINT enables reliable, consistent, largely autonomous extraction, though manual review remains advisable to catch hallucinations and subtle errors.

[2] The full implementation is available as a Python-based API at https://github.com/isa-group/A-MINT-repo. A detailed technical report, including execution details and prompt templates, is available at [2].

The transformation process, illustrated in Fig. 3, is initiated when the user provides a SaaS pricing page URL. Optional parameters include a maximum number of correction attempts, a temperature setting for the LLM, and the specification of both a default and a fallback LLM. If the default LLM produces incoherent output, the system retries with the fallback model.

Phase 1. Once invoked, the system downloads the raw HTML and preprocesses it to discard irrelevant content such as menus, ads, and boilerplate text.

Phase 2. The cleaned HTML is embedded into a structured prompt for the LLM to convert it into simplified markdown, reducing input token length and clarifying subsequent prompts. This involves two distinct prompts: one for markdown generation and another for its validation. Next, a sequence of prompts guides the LLM to extract key elements from the Markdown, focusing on plans, features, usage limits, standard add-ons, and overage add-ons. A final prompt verifies the internal consistency of features and usage limits, helping to mitigate typical hallucinations observed in LLM-based systems [10,15]. The resulting output is a structured JSON, which is normalized and transformed into a valid iPricing.

Phase 3. This model is then validated using the automated operation from [7], which detects both syntactic issues (e.g., mislabeling, non-numeric values) and semantic inconsistencies (e.g., invalid configurations, constraint violations) [16]. If valid, the model is returned as a complete and ready-to-use iPricing. Otherwise, the system generates another prompt, queries the LLM again, and repeats this process up to the maximum of iterations. If no valid model is obtained, the process stops and flags the result for manual review.

4 Evaluation

This section evaluates A-MINT to answer these research questions:

(i) RQ1: *What is the effectiveness of A-MINT in the automated generation of iPricings and how does it compare against alternative approaches?*

(ii) RQ2: *How much does it cost to transform a pricing with A-MINT? Can this economic cost be saved?*

(iii) RQ3: *What are the current limitations of the A-MINT approach in terms of generalizability across pricing patterns, and handling of edge cases?*

4.1 RQ1: Effectiveness of iPricing Generation

We quantify A-MINT's extraction accuracy by encoding each predicted and ground-truth iPricing into a 29-dimensional structural vector capturing the number of plans (free vs. paid), features (8 categories and 10 subcategories [6]), add-ons (those that introduce new features vs. extend existing ones), usage limits (renewable vs. non-renewable [6]), and the configuration-space size [5]. For each pricing page and dimension, true positives are the overlap $\min(\text{pred}, \text{ref})$;

surpluses count as false positives and deficits as false negatives. Summing counts across all 158 pages and dimensions yields overall $\overline{\text{Precision}} = 72.42\%$, $\overline{\text{Recall}} = 77.42\%$, and $\overline{\text{F}_1} = 70.51\%$, indicating strong element coverage with occasional overgeneration or mislabeling–typical LLM issues–while some cases are near-perfect such as Figma 2024 or Slack 2024 (see Table 1). Beyond prior conceptual work such as AI4Pricing, A-MINT implements an end-to-end pipeline that automates both extraction and validation, leading to 98.73% (156/158) of models being successfully validated via the self-correction loop.

RQ_1: How effective is A-MINT in automatic iPricings generation and how does it compare to alternative approaches?

(1) Mean F_1-Score of 70.51% (recall 77.42%, precision 72.42%): broad coverage with controlled hallucinations.

(2) 98.73% iPricings (156 of 158) were validated and refined via the self-correction loop, without manual intervention (advantage over related approaches).

Table 1. Per-year F_1-Score across SaaS providers. Entries with ✗ indicate unavailable ground-truth; ✗ denotes invalid reference data.

SaaS	19	20	21	22	23	24	SaaS	19	20	21	22	23	24
Salesforce	66.8	66.8	67.4	61.5	63.5	12.1	Buffer	59.8	57.6	90.9	86.1	31.4	77.7
GitHub	63.1	5	72.6	79.4	25.4	60.9	Jira	82	81.6	71.2	70.9	71.6	78.6
Postman	✗	81.4	71.5	86.5	7.4	92.4	Notion	✗	✗	77.8	74.8	81.7	82.7
Databox	43.2	35.9	53.6	84.1	41.5	76.8	Figma	83.3	84.8	84.7	88.8	81.6	92
OpenPhone	✗	79.2	86.7	88.3	48.1	64.2	Box	81.7	69	1.7	26.2	80.4	45.7
Wrike	87.6	91	58.7	81.3	86.5	78.5	Canva	74.1	74.9	77.9	75.1	79.9	✗
Tableau	73.5	79.8	65.9	69.6	71.2	74.1	Dropbox	✗	✗	89.4	55.1	88.1	21.5
Evernote	74	76.7	70.8	74.5	70.1	28.8	Zapier	63.1	76.6	✗	75.5	67.7	78.4
Hypercontext	✗	✗	86.1	84.4	83.5	82	Slack	84.6	87.1	✗	✗	84.6	93.3
MailChimp	81.7	76.3	76.1	75.2	82.8	78.5	Pumble	✗	✗	46.8	34.8	80.6	70.9
UserGuiding	✗	65.1	87	79.3	✗	98.8	ClickUp	47.1	69.4	81.2	85.8	52.3	52.6
Crowdcast	✗	82.7	72.8	88.2	87.8	75.1	Planable	63.2	69	71.2	62.3	46.8	56.6
Clockify	70.6	82.5	81.6	80.8	70.1	80.1	Deskera	✗	✗	81.7	63.9	47.8	✗
Microsoft 365	82.5	74.6	86.1	87.9	84.9	81	Overleaf	84.2	80.7	75.5	64.3	69.9	63
Trustmary	✗	✗	40.7	73.5	56	70.4	Quip	69.8	80	77.1	71	60.2	73.9

4.2 RQ2: Economic Cost of Transformation

A-MINT runs on consumer hardware (for our experiments we used MacOS Sequoia 15.5, i9–9880H, 16 GB RAM, 1 TB SSD) without a GPU. In our experiments A-MINT used Gemini 2.5 Flash for 99.3% of calls and Gemini 2.5 Pro as a 0.7% fallback (temperature=0.7; correction loop $\leq$ 50 steps). Costs scale

with LLM usage: across 158 pages the average was $0.22 per page. Lightweight HTML preprocessing cut input by 85.88% (∼13,259,936 tokens), saving $9.77 (∼28%) and reducing total cost from $43.95 to $34.18 by shrinking Phase 1 from $17.83 to $5.67 (Phases 2–3 unchanged at $8.06 and $20.45). The iterative correction cycle dominates spend (59.8%) and is the main optimization target. We used an OpenAI-compatible client for portability; the pipeline is designed to be LLM-agnostic and supports commercial or open-weight deployments (including on-prem). The evaluation of open models is left for future work.

RQ_2: How much does it cost to transform a pricing with A-MINT? Can this economic cost be saved?

(1) The average cost is $0.22 per pricing; $34.18 total (158 pricings).
(2) Preprocessing cuts 85.88% of HTML inputs tokens (∼ 13.26M tokens) and reduces costs 28.58% ($9.77).
(3) The correction cycle is 59.8% of spend (future optimization target); exploring open-source/on-prem LLMs may yield further savings.

4.3 RQ3: Limitations and Generalization Challenges

A qualitative analysis of the evaluation results shows that A-MINT generally performs well across diverse pricing pages, though certain edge cases still pose challenges. While none of the outputs were entirely error-free, most exhibited only minor issues such as hallucinated or missing details. Complex pages caused notable performance drops, and a review of the 53 pages with F_1-Score below 70% identified three main failure causes: LLM-driven errors, interactivity limitations and ambiguity in multi-product pages for improvement. Future work should focus on minimizing hallucinations, improving multi-product support, and developing methods to mimic user interaction with interactive content.

RQ_3: What are the current limitations of A-MINT in terms of generalizability across pricing patterns, and handling of edge cases?

(1) LLM-driven errors: The most frequent issues stemmed from the LLM itself. While often minor, these errors hinder precise extractions. Among the analyzed failures, 18 cases exhibited hallucinated content, 10 had missing information, and 2 were affected by bothreflecting typical LLM limitations [10,15].
(2) Interactivity limitations: No clicks or tab switches, which restricted access to critical pricing details (affected 5 pricing pages). This resulted in incomplete extractions and echoes known challenges [1].
(3) Ambiguity in multi-product pages: In 18 cases, pricing pages described multiple products or offerings. The lack of contextual cues or user intent signals conflated pricings, leading to incorrect iPricings.

5 Conclusions and Future Work

A-MINT addresses the lack of a standardized and machine-readable representation of SaaS pricing by transforming ad-hoc web pages into validated iPricing

models with minimal human effort. It achieves a mean F_1 of 70.51% on the SPECTRA dataset with minimal manual intervention, enabling automation of benchmarking, cost estimation, and subscription optimization that were previously impractical. Moving forward, we aim to drive de facto convergence towards iPricing adoption by stress-testing at larger scale and releasing reproducible artifacts, strengthening the pipeline for dynamic and multi-product pages, assessing open-weight and on-premise LLMs to improve cost-performance, and optimizing the iterative correction loop that dominates runtime and cost.

Acknowledgments. The authors appreciate Ramón Gavira Sánchez for his valuable feedback. This work was partially supported by grants PID2021-126227NB-C21 and PID2021-126227NB-C22 funded by MCIN / AEI / 10.13039 / 501100011033 / FEDER, UE "ERDF"; TED2021-131023B-C21 funded by MCIN / AEI / 10.13039 / 501100011033, EU "NextGenerationEU"/PRTR; PID2024-155693NB-C44 funded by MICIU / AEI / 10.13039/ 501100011033 / FEDER, UE; and by "Smart Computer systems Research and Engineering (SCORE)", funded by the Consejería de Transformación Económica, Industria, Conocimiento y Universidades of Junta de Andalucía under the "Programa de Fortalecimiento de Institutos Universitarios de Investigación de la Universidades Andaluzas, Centros e Infraestructuras para la adquisición del sello 'Severo Ochoa' o 'María de Maeztu"'.

References

1. Cavero, F.J., Alonso, J.C., Ruiz-Cortés, A.: From static to intelligent: evolving SaaS pricing with LLMs. In: Service-Oriented Computing - ICSOC 2024 Workshops. LNCS, vol. 15834, pp. 136–147. Springer, Singapore (2026)
2. Cavero, F.J., Parejo, J.A., Alonso, J.C., Ruiz-Cortés, A.: A-MINT - supplementary material (2025). https://doi.org/10.5281/zenodo.17228340
3. Cavero, F.J., Parejo, J.A., Alonso, J.C., Ruiz-Cortés, A.: Buffer excerpt iPricing (2025), https://sphere.score.us.es/pricings/anonymous/Buffer%20-%20Excerpt?collectionName=ICSOC%202025, Accessed 9 May 2025
4. Cavero, F.J., Parejo, J.A., Alonso, J.C., Ruiz-Cortés, A.: SaaS pricing evaluation corpus for temporal representation & analysis (SPECTRA) (2025). https://doi.org/10.5281/zenodo.17237164
5. García-Fernández, A., Parejo, J.A., Cavero, F.J., Ruiz-Cortés, A.: racing the market: an industry support analysis for pricing-driven DevOps in SaaS. In: Service-Oriented Computing. LNCS, vol. 15405, pp. 260–275. Springer (2025)
6. García-Fernández, A., Parejo, J.A., Ruiz-Cortés, A.: Pricing4SaaS: towards a pricing model to drive the operation of SaaS. In: Intelligent Information Systems. CAiSE. LNBIP, vol. 520, pp. 47–54. Springer, Switzerland (2024)
7. García-Fernández, A., Parejo, J.A., Trinidad, P., Ruiz-Cortés, A.: Automated analysis of pricings on advanced information systems. In: Advanced Information Systems Engineering. CAiSE 2025. LNCS, vol. 15702, pp. 223–239. Springer (2025)
8. García Fernández, A., Parejo, J.A., Ruiz-Cortés, A.: iSubscription: bridging the gap between contracts and runtime access control in SaaS. In: International Conference on Service-Oriented Computing (2025). (In press)

9. García-Fernández, A., Parejo, J.A., Trinidad, P., Ruiz-Cortés, A.: SaaS pricing evaluation corpus for temporal representation & analysis (SPECTRA) (2025). https://doi.org/10.5281/zenodo.17237051
10. Huang, L., et al.: A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions. ACM Trans. Inf. Syst. **43**(2) (2025)
11. ISA Group: Pricing2Yaml. https://pricing4saas-docs.vercel.app/docs/2.0.1/category/pricing2yaml, Accessed 8 May 2025
12. Jiang, Z., Sun, W., Tang, K., Snowdon, J.L., Zhang, X.: A pattern-based design approach for subscription management of software as a service. In: 2009 Congress on Services - I, pp. 678–685 (2009)
13. Laatikainen, G., Ojala, A.: SaaS architecture and pricing models. In: 2014 IEEE International Conference on Services Computing, pp. 597–604 (2014)
14. Molino-Peña, E., García, J.M., Ruiz-Cortés, A.: Integrating terms of service and service level agreements for automating cloud service management. In: International Conference on Service-Oriented Computing (2025). (in press)
15. Rong, D., et al.: LLM enhanced representation for cold start service recommendation. In: Service-Oriented Computing. LNCS, vol. 15404, pp. 153–167. Springer, Singapore (2025)
16. ter Hofstede, A., Proper, H.: How to formalize it?: Formalization principles for information system development methods. Inf. Softw. Technol. **40**(10), 519–540 (1998)

LightLoader: Accelerate Python FaaS Cold-Start via Multi-level Source Code Optimization

Pengyu Chen[1,2], Wei Chen[1,2(✉)], Guoquan Wu[1,2], and Jun Wei[1,2]

[1] University of Chinese Academy of Sciences, Beijing, China
[2] Institute of Software, Chinese Academy of Sciences, Beijing, China
{chenpengyu23,wchen,gqwu,wj}@otcaix.iscas.ac.cn

Abstract. Cold starts are a big challenge in Function-as-a-Service (FaaS). While most solutions focus on optimizing the runtime environment and FaaS scheduling, they often overlook the impact of FaaS implementation on cold-start latency. Dynamic languages like Python, with their extensive dependency imports and dynamic building processes, can particularly suffer from longer startup times.

We propose LightLoader, an approach to accelerating Python-based FaaS (i.e., PyFaaS) cold start by debloating them and optimizing their dynamic build. LightLoader rewrites PyFaaS by converting potentially unused functions to on-demand dynamic loading, thereby reducing loading time. It also delays third-party package loading until its first use instead of importing these all at the beginning. Notably, to ensure that these rewrites do not compromise the original PyFaaS's functionalities, we utilize a large language model (LLM) to test and repair the optimized PyFaaS iteratively. We implemented a prototype on the popular open-source serverless platform OpenFaaS and conducted an extensive evaluation with PyFaaS of various sizes and functionalities. The experimental results show that LightLoader effectively accelerates PyFaaS cold starts, reducing end-to-end latency by 11.44% on average, a 9.32% improvement over the state-of-the-art solution. It also decreases the runtime memory utilization by an average of 9.28% and up to 43.27%. LightLoader is platform-independent and can be a supplement to the optimizations of other layers.

Keywords: FaaS · cold-start · Python · serverless computing

1 Introduction

Function-as-a-service (FaaS) has revolutionized cloud computing by enabling developers to run event-driven services without extensive infrastructure management. However, cold-start, where initial FaaS invocations experience high latency, especially under heavy load [19], remains a significant challenge. Existing solutions address this issue by optimizing the runtime environment and FaaS

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 92–106, 2026.
https://doi.org/10.1007/978-981-95-5015-9_7

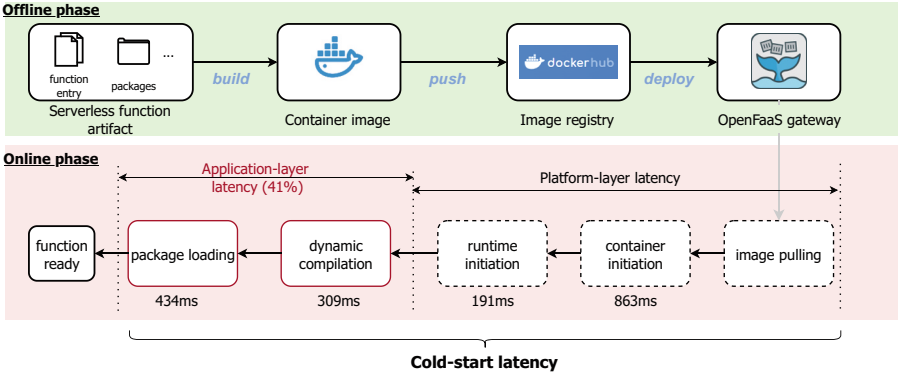


Fig. 1. The end-to-end startup process of a FaaS function.

scheduling, including lightweight sandboxes [9], function orchestration optimization [12,24], keep-alive mechanisms [4,22], and pre-warming techniques [17,19].

These approaches tend to overlook the impact of function implementation on cold-start latency. In particular, FaaS written in dynamic languages can experience longer startup times. For instance, Python is popular for FaaS due to its thriving ecosystem and suitability for tasks such as machine learning and data analysis. Nevertheless, its dynamic nature results in greater cold-start latency compared to statically typed FaaS languages. This is mainly because Python requires interpreter initialization, bytecode compilation, and dependency loading prior to actual execution. Unnecessary dependency imports further worsen this issue, leading to resource inefficiencies and increased delays [16,19].

Only a few studies address the cold-start issue at the function-layer by reducing initialization overhead and optimizing code loading [13] and compilation [11]. FaaSLight [13] uses static analysis to identify and load only essential function code initially, loading optional code on demand. However, imperfect identification leads to frequent on-demand loading and performance penalties. Lazy-import [15] enables deferred module loading, but requires substantial manual code changes, making it difficult to integrate into existing projects.

We designed LightLoader to reduce the cold-start latency of Python-based FaaS (PyFaaS) based on our empirical findings: unused files, functions, and packages are prevalent in PyFaaS and negatively impact cold-start latency and memory utilization. LightLoader employs more accurate function call path analysis to identify functions not invoked by the entry function in a PyFaaS and rewrites these functions to dynamically load on demand, reducing the loading time of unused functions. It also analyzes the PyFaaS abstract syntax tree (AST) to identify the location where a dependent package is first used and relocates the corresponding import statement to that location, rather than importing all dependencies at the beginning. This approach not only reduces the initial loading time but also minimizes memory usage by avoiding unnecessary package initial-

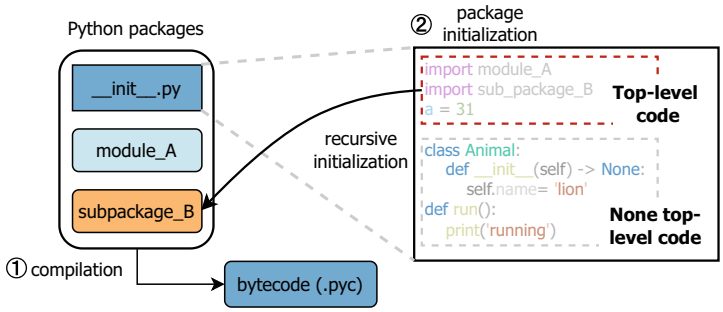


Fig. 2. Two main application-layer causes of PyFaaS cold-start latency.

ization. LightLoader employs LLM-based automated testing and repair to ensure that these rewrites do not compromise the original PyFaaS's functionalities.

We implemented a prototype and conducted extensive evaluations with PyFaaS of various sizes and functionalities. The experimental results show that LightLoader effectively accelerates PyFaaS cold-start, reducing end-to-end latency by 11.44% on average, a 9.32% improvement over the state-of-the-art solution [13]. It also decreases the runtime memory utilization by an average of 9.28% and up to 43.27%.

In summary, this work makes the following contributions.

- A meticulous empirical study of PyFaaS cold-start, revealing that unused files, functions, and packages significantly impact cold-start latency and memory utilization.
- A novel approach to comprehensively accelerating PyFaaS cold-start in multiple aspects from the application layer, requiring little human intervention.
- LightLoader prototype and extensive evaluations, demonstrating its superiority over the state-of-the-art in reducing PyFaaS's end-to-end latency.

2 Background and Motivation

2.1 Cold-Start of a Python Function

Cold-start latency refers to the delay experienced when invoking a PyFaaS after inactivity or for the first time, due to the required initialization steps necessary to prepare the PyFaaS for execution.

Figure 1 illustrates a PyFaaS startup process on OpenFaaS [14]. The total latency comprises two parts: platform-layer tasks, such as image pulling and container initialization, and application-layer tasks, accounting for 41% of the total time, including dynamic bytecode compilation and dependency loading.

Figure 2 illustrates that, in the application layer, PyFaaS cold starts are significantly delayed by (1) the compilation of source code into bytecode before the first execution, and (2) the immediate execution of the top-level code (e.g.,

Table 1. Analysis of PyFaaS redundancies: non-essential file, unused system libraries, unused functions, and unused modules (all sizes measured in MB). Entries with size = 0 and '-' indicate cases that rely solely on built-in packages.

Name (Size)*	U. File(%)	U. Lib(%)	U. Func.(%)	U. Module(%)
RQ (18.14)	4.50	96.60	46.10	34.34
IM (246.58)	13.43	96.10	95.00	77.19
CV (238.13)	3.65	94.10	95.30	82.38
MX (119.39)	33.01	96.60	95.10	80.53
CS (259.08)	21.75	96.00	94.60	69.32
MS (0.52)	37.53	98.30	42.00	40.74
CP (0)	–	98.20	–	–
HT (0.96)	45.89	96.50	42.20	25.93
PG (11.82)	6.74	97.70	55.90	5.63
SL (0)	–	98.30	–	–
TH (20.90)	4.50	98.20	69.30	71.88
UP (0)	–	96.50	–	–
VD (21.00)	4.48	98.20	86.90	89.58
Average	**17.53**	**97.00**	**72.24**	**57.75**

* **PyFaaS Name abbreviations**: RQ (requests), IM (images), CV (cv), MX (matrix), CS (cvs), MS (marshal), CP (compress), HT (html), PG (page), SL (sleep), TH (thumbnailer), UP (uploader), VD (video).

import statements, global variable assignments) upon importing a dependent package or module, potentially triggering recursive initialization and further increasing startup latency if the top-level code imports other dependencies. These factors indicate that application code inefficiencies significantly contribute to cold starts. Notably, some source code and imported packages are not utilized, leading to unnecessary compilation and initialization overhead.

2.2 Empirical Study

We conducted a small-scale empirical study to analyze the unused files, libraries, functions, and modules within 13 representative PyFaaS from popular benchmarks [6,25] and to assess the impact of these inefficiencies on PyFaaS. To perform this analysis, we used *strace* to identify the accessed libraries and *cProfile* to track the usage of functions and modules during execution.

Table 1 presents the results of the empirical study. PyFaaS images are bloated due to non-essential files such as documentation and test suites, which account for an average of 17.53% of the image size. System libraries are also a significant source of redundancy, with over 97% remaining unused at runtime. These factors increase image transfer times.

Profiling PyFaaS revealed that, on average, 72.24% of functions are never called and 57.75% of imported modules are unused during a typical invocation, resulting in unnecessary bytecode compilation and module import overhead. For example, in a typical PyFaaS (i.e., RQ in Table 1), unused modules contribute 44.9% of the total import latency and 15.14% of the memory consumption during the import process.

Key finding: Unused files, functions, and packages are prevalent, resulting in excessive image transfer, unnecessary compilation, and inefficient memory initialization, which ultimately contribute to PyFaaS cold-start latency.

2.3 Motivation

Application-layer redundancies significantly contribute to PyFaaS cold-start latency, which serves as the primary motivation for our research. Existing platform-layer optimizations, such as the keep-alive mechanism and lightweight sandbox, cannot effectively reduce these redundancy-induced cold-start overheads. On the other hand, existing application-layer optimizations are limited. FaaSLight [13] defers loading of optional code based on static analysis, but its imprecision can cause frequent on-demand loading penalties or failures. Lazy-import [15] requires extensive manual code changes, which hinders its adoption in existing FaaS applications.

We are motivated to propose LightLoader to optimize PyFaaS initialization by utilizing more accurate, source-code-level dependency analysis to minimize redundancies at the application layer.

3 LightLoader

LightLoader addresses two fundamental challenges: the **accurate redundancy identification** and the **reliable optimization**. The former identifies what components are unused and the latter ensures that the corresponding code optimization does not alter the original behavior of the PyFaaS.

3.1 Overview

As illustrated in Fig. 3, LightLoader transforms an original PyFaaS into a debloated, optimized image through a workflow based on four core components:
1. The function transformer constructs a more precise function call graph from the original PyFaaS entry function for identifying unused functions. It rewrites them to enable on-demand loading, reducing the initial compilation time and footprint.
2. The package loading optimizer locates import statements and the corresponding modules' first use by analyzing each Python file's AST. It then moves

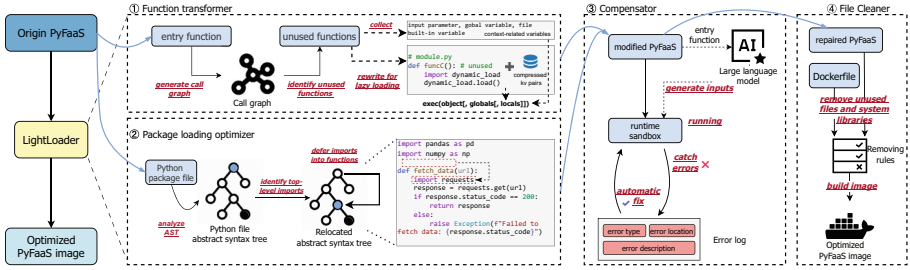


Fig. 3. LightLoader overview

imports from the top of each file to the functions that first use the modules, deferring the import of packages until they are needed.

3. The compensator ensures the correctness of the modified PyFaaS in the prior two stages. It validates the code by generating test inputs using an LLM and executing the PyFaaS in a sandbox. The compensator then fixes encountered runtime errors by analyzing error logs.

4. The file cleaner takes the fixed PyFaaS and the Dockerfile as input and rebuilds a lightweight image by removing non-essential application files and unused system libraries.

3.2 On-Demand Function Loading

The function transformer reduces cold-start latency by identifying and excluding unused functions from the initial compilation based on the rationale that many functions are never called during a typical execution.

LightLoader first identifies unused functions by building a call graph starting from the PyFaaS entry function (e.g., `handler()` in OpenFaaS). It considers reachable functions, including those in imported packages, as used, and flags the rest as unused ones. This call graph is more accurate due to LightLoader's extensive handling of Python's built-in functions and dynamic features like decorators and dynamic typing.

LightLoader then compresses unused functions and their execution contexts (e.g., imported modules, global and local variables) into a key-value database. To handle static analysis limitations, it replaces these code snippets with Python's `exec()` function rather than deleting them directly (as depicted in the *Function transformer* part of Fig. 3). This method enables LightLoader to dynamically retrieve and execute original function code when needed, minimizing initial compilation time and memory footprint.

3.3 Lazy Package Importing

To eliminate the latency caused by Python's eager execution of top-level import statements, the package loading optimizer defers imports from the global scope into the specific functions that first use them, minimizing cold-start overhead.

Specifically, the difficulties lie in accurately identifying the first usage context of each imported module and correctly moving the import statement to the appropriate function scope, without compromising PyFaaS's functionalities.

The package loading optimizer works in three steps (see Fig. 4) based on ASTs of Python source files. (1) First, it extracts top-level import nodes and filters out conditional or *try-catch* imports. This ensures that only the essential imports are retained for further analysis. (2) Then, it scans the AST and applies a set of matching rules to identify where each import is first required, i.e., any classes and functions belonging to the import (known as dependencies) are first used. These rules include *type-annotation matching*, which detects the dependencies that appear in function decorators or type hints; *scope-sensitive matching* captures control-flow-dependent usage of dependencies that are invoked only along specific execution paths, such as inside conditionals or loops; and *attribute chain matching*, which resolves nested access patterns such as `numpy.random.rand()` or `os.path.join()`, where the reference to the top-level module (e.g., `numpy` or `os`) is implicit. (3) Finally, once the first-use location is identified, the package loading optimizer performs a *scope-aware* insertion: using a *Scope Stack* to track the current indentation and nesting context during AST traversal, it accurately moves the import statement to the correct lexical scope, even in the presence of deeply nested control structures or functions. This ensures correctness and minimal overhead by enabling just-in-time loading of dependencies at their precise point of use.

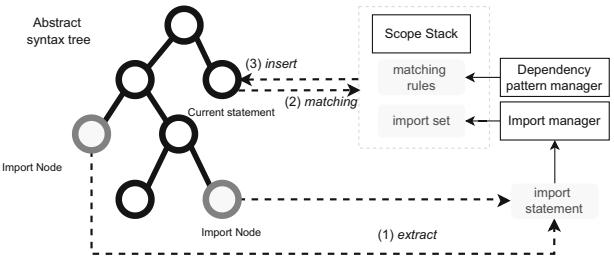


Fig. 4. Lazy package loading.

3.4 Validation and Repair

Validation and repair are crucial for the optimized PyFaaS, as the function transformer and package loading optimizer may introduce runtime errors due to some incomplete (or even incorrect) static analysis. Specifically, incorrect handling of lazy package imports can result in runtime dependency errors.

The compensator works based on two observations. First, most of the runtime errors introduced by our optimization are due to missing `import` statements that were intentionally deferred or removed. Second, Python's runtime

environment produces highly structured and informative error logs. This diagnostic metadata provides a clear, machine-readable link between an error and its root cause. For example, a typical `ImportError` message such as "cannot import name `'port_by_scheme'` from `'urllib3.connectionpool'`" clearly indicates the missing symbol and its module path, enabling deterministic recovery. The log also provides the error trace across files (e.g., `requests`, `urllib3`, and their internal modules), allowing the compensator to precisely locate the fault.

The compensator uses an LLM to generate test cases for validation and detects and repairs errors based on heuristic rules. It executes the optimized PyFaaS in a sandbox, analyzes error logs to pinpoint errors, and resolves them by categorizing exceptions and applying targeted solutions. This "*test-and-repair*" cycle runs iteratively until no errors occur, ensuring optimizations maintain the original PyFaaS's integrity.

The compensator fixes an error in three steps. It first categorizes the exception of errors as the optimizations introduced error types are limited and enumerable. The most common issues are `NameError`, which typically indicates a missing module-level import; `ImportError`, signifying a failure to resolve a submodule; and `AttributeError`, which often arises from complex issues like circular dependencies or partially initialized modules. Errors outside these categories are classified as `OtherError` and handled with a fallback recovery. Next, it extracts critical information from the error log according to the categorized error types. For instance, in the case of an `ImportError`, it retrieves the missing import name (e.g., HTTPException), the source module (e.g., urllib3.connection), and the error file path. Finally, it applies a rule-based fix: for a `NameError`, it injects the correct import statement just before its use; for an `ImportError`, it reconstructs the proper relative import path; and for an `AttributeError`, it restores critical initialization sequences.

3.5 File Cleaning

LightLoader optimizes the PyFaaS image by removing non-essential files, which reduces image size and deployment overhead. This step is crucial for enhancing the efficiency of serverless function deployment, as smaller images lead to faster cold starts and reduced network transfer times.

Based on our empirical study, the file cleaner mainly deletes four categories of non-essential files: (1) **Documentation**, such as `*.md`, `*.rst`, and license files; (2) **Test artifacts**, including unit tests and mock objects; (3) **Development metadata**, like `setup.py`, `requirements.txt`, and `*.pyc` files; and (4) **Redundant system libraries** unused during execution. The file cleaner uses a pattern-matching method to safely remove the first three types of files, as these files are never invoked by PyFaaS at runtime. For system libraries, LightLoader employs `docker-slim` [21], which performs dynamic analysis to retain only those libraries accessed during runs. This process significantly reduces image size while preserving function integrity.

Based on our approach, we prototyped LightLoader in Python, integrating it with the open-source serverless platform OpenFaaS [14]. The implementa-

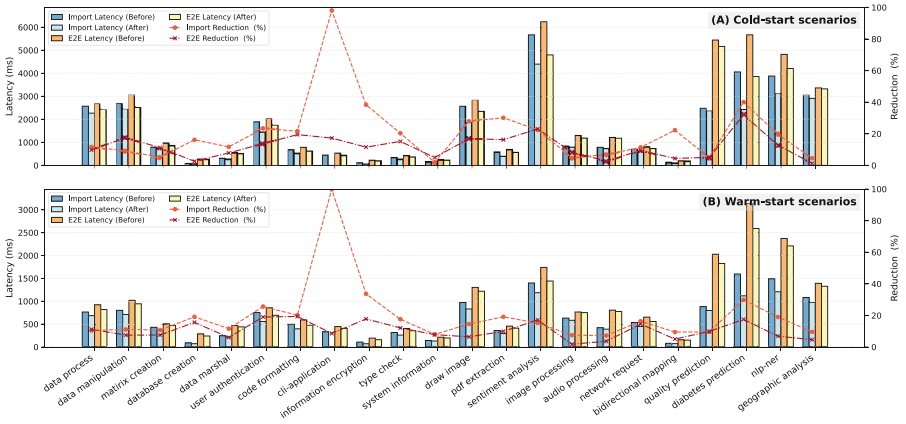


Fig. 5. Performance evaluation of LightLoader optimization on PyFaaS latency in (A) cold start and (B) warm start. Bars represent the latency before and after optimization in terms of milliseconds (ms) for each PyFaaS. The line plots indicate the percentage reduction in latency after optimization.

tion extends JARVIS [10] for call graph construction and employs DeepSeek-V3 [7] for test input generation. We modified the official OpenFaaS Python template to incorporate the lazy loading and dependency repair mechanisms. The complete implementation is publicly available at https://github.com/Bencpy/Light-Loader. Notably, although LightLoader targets Python FaaS, its principles of lazy loading, dependency repair, and test generation are generalizable across languages.

4 Evaluation

4.1 Research Questions

We conducted a comprehensive evaluation of LightLoader by answering the following research questions:

- **RQ1 (Latency):** How well does LightLoader reduce the latency of PyFaaS in cold and warm start scenarios?
- **RQ2 (Memory use):** How well does LightLoader optimize memory usage during the import phase and runtime execution of PyFaaS?
- **RQ3 (Ablation study):** What is the performance contribution of each LightLoader component?

4.2 Experimental Setup

Environment. Our test cluster has a control node (Intel Xeon 16-core, 128 GB RAM) and multiple worker nodes (Raspberry Pi 4B, 4-core, 8 GB RAM). The cluster runs Kubernetes 1.20.15 with containerd 1.6.25 as the container runtime.

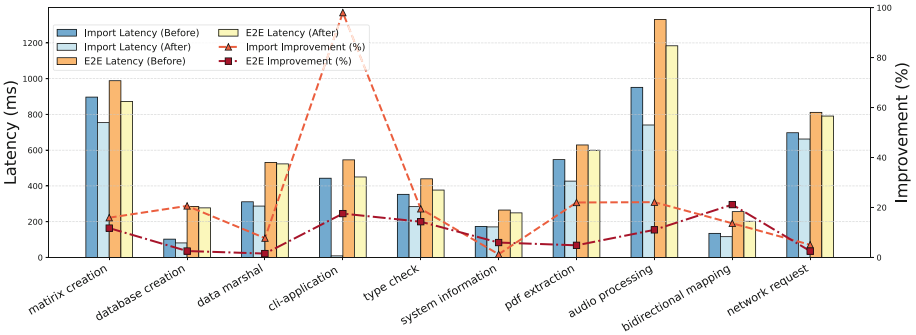


Fig. 6. Latency comparison with FaaSLight. Note that only 10 of 23 PyFaaS are shown due to FaaSLight fails to generate call graphs for the other 13 PyFaaS because of its limitations.

Dataset. We collected 23 PyFaaS from diverse domains like image processing, NLP, and data analysis. These PyFaaS were obtained from benchmark collections [13,23] and augmented with additional PyFaaS collected manually.
Cold/Warm Start Simulation. To simulate cold starts for each invocation through re-compilation, we implemented a hook function to automatically purge compiled Python bytecode after each request completion. For warm starts, bytecode was preserved between invocations.
Baseline. We compare LightLoader against two baselines: (1) the unmodified OpenFaaS Python template, and (2) FaaSLight [13], the state-of-the-art FaaS optimization framework at the application-layer.

4.3 PyFaaS Start Latency Reduction (RQ1)

We evaluate LightLoader's effectiveness in reducing PyFaaS start latency with *import latency* and *end-to-end (E2E) latency*. Import latency is the time to load dependencies; E2E latency is the total time from request to response. Figure 5 shows that LightLoader significantly reduces import latency and improves overall execution time compared to the baseline, in both cold and warm start scenarios.

Overall, LightLoader achieved average 20.86% and 11.44% reduction of import and E2E latencies under cold start, respectively. The optimization benefits are similar in warm start, yielding 19.34% and 10.47% reductions in import and E2E latencies, respectively. These consistent improvements validate our optimization's effectiveness in both cold and warm states. Specifically, 14 out of 22 PyFaaS show moderate *import latency* reductions (5–25%), while five PyFaaS demonstrate exceptional performance improvements over 25%. Notably, the *cli-application* achieved near-complete *import latency* elimination (98.08% reduction) by successfully deferring all initialization operations until their actual invocation points. Further, we observed that PyFaaS with larger original import latencies (*sentiment analysis*: 5670.7 ms, *diabetes prediction*: 4059.9 ms) generally benefit more substantially, suggesting that more complex PyFaaS with

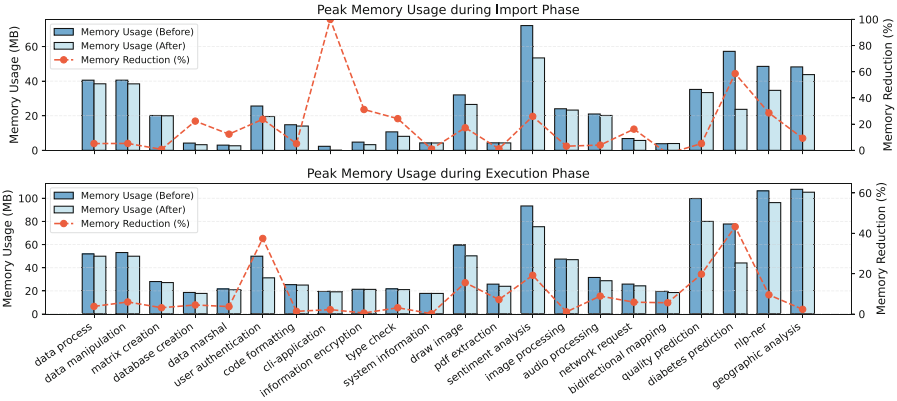


Fig. 7. Memory footprint reduction. The figure shows the impact on memory usage during the import (top) and execution (bottom). The bars represent memory usage before and after optimization, respectively.

heavier initialization burdens represent particularly promising optimization targets.

We also compared LightLoader with the SOTA solution FaaSLight [13]. Since FaaSLight was originally designed for AWS Lambda [2], we implemented necessary adaptations to enable fair comparison within our OpenFaaS environment while maintaining FaaSLight's core optimization mechanisms. Notably, only 10 of the 23 PyFaaS (43.5%) executed successfully with FaaSLight due to its inherent limitations in its CHA approach [5]. This highlights an advantage of LightLoader's more robust optimization strategy.

As Fig. 6 shows, for the subset of executable PyFaaS, LightLoader consistently outperformed FaaSLight, achieving 22.59% and 9.32% reductions in import and E2E latencies during cold-start.

4.4 Memory Usage Optimization (RQ2)

We evaluated LightLoader's memory optimization capabilities by analyzing both import- and runtime-phase memory consumption across our benchmark PyFaaS. The results are presented in Fig. 7, which illustrates the significant impact of LightLoader on memory efficiency. Overall, LightLoader achieved an average 18.07% of reduction in memory consumption during the import phase and a 9.28% decrease in runtime memory footprint.

For import memory, LightLoader demonstrated exceptional effectiveness on machine learning PyFaaS, with *diabetes prediction* achieving a remarkable 58.59% reduction in import memory usage. Similarly, *sentiment analysis* and *nlp-ner* demonstrated substantial improvements of 25.93% and 28.56%, respectively. This is due to ML libraries having extensive auxiliary components that are only selectively used in individual PyFaaS.

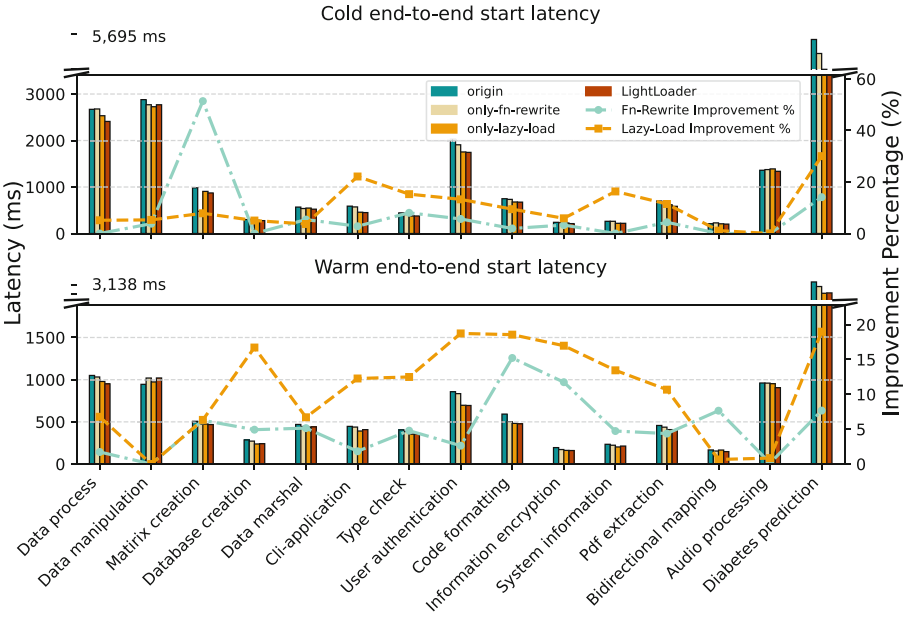


Fig. 8. Impact of component ablation on end-to-end latency.

Runtime memory usage exhibited an average reduction of 9.28%, peaking at 43.27% (*diabetes prediction*). Optimization varied with PyFaaS features and scale. PyFaaS such as *information encryption*, *system information processing*, *code formatting*, and *image processing* exhibited only modest improvements (under 5%), likely because most lazily imported modules are eventually used, limiting potential memory savings.

LightLoader consistently optimizes memory usage for diverse PyFaaS applications, from lightweight utilities to memory-intensive ML applications, in both initialization and runtime.

4.5 Ablation Study (RQ3)

We performed an ablation study to quantify the individual contributions of our core optimization components, i.e., *On-demand Function Loading* and *Lazy Package Importing*. Figure 8 shows the experimental results.

Lazy Package Importing significantly outperformed *On-demand Function Loading* in E2E latency optimization. On-demand Function Loading yielded moderate reductions in cold-start (5.99%) and warm-start (4.73%) latencies. *Lazy Package Importing* achieved substantially greater improvements, decreasing E2E latency by 9.99% during cold starts and 10.48% during warm starts.

This ablation analysis confirms that both components can accelerate PyFaaS's startup, and the *Lazy Package Importing* provides the dominant optimization effect.

5 Discussion

Limitations of Static Analysis. LightLoader relies on static analysis to identify unused functions and dependencies. However, this approach can struggle with dynamic features, leading to potential false positives. To mitigate this, we improve static analysis accuracy by considering built-in Python functions and features like decorators and annotations. In addition, we ensure the correctness of optimizations based on automated testing and repair. This way, LightLoader can effectively suppress the potential false positives and safely optimize PyFaaS.
Incorporation with LLM. LightLoader utilizes an LLM solely for generating test cases, leveraging its ability to produce diverse inputs based on function signatures and contexts. Instead, we repair errors based on heuristic rules without involving the LLM. This way, we can guarantee the correctness of repairs and avoid potential risks of new errors introduced by the LLM. Furthermore, we can eventually revert an erroneous code change if we cannot fix it with any rule.

6 Related Work

Research efforts addressing cold-start latency can be categorized into system- and application-layer optimizations.

System-layer optimizations enhance infrastructure and execution environment. These include: (1) keep-alive mechanisms [4,22] that maintain function instances in memory during idle periods; (2) lightweight sandboxes [1,9] designed to accelerate container initialization; (3) pre-warming strategies [17,19] that proactively initialize function instances; and (4) checkpointing techniques [3,8,20] that rapidly restore executions. Such works are orthogonal to ours and can be effectively complementary.

Application-layer optimizations, focusing on function code or deployment configuration, provide portable solutions with minimal infrastructure changes. Key strategies include: (1) on-demand code loading [13] that selectively loads essential components; (2) selective compilation [11] that uses ahead-of-time compilation for critical functions; and (3) automated function fusion [18] that consolidates fine-grained logic into efficient FaaS functions. These approaches require less modification than system-layered approaches, enabling broader adoption across serverless platforms.

7 Conclusion

We proposed LightLoader, a novel multi-level optimization approach to mitigate cold-start latency of PyFaaS. It systematically analyzes PyFaaS source code to identify unused functions and dependencies, and optimizes the original PyFaaS through on-demand function loading, lazy package importing, and unused file elimination. Our implementation on the OpenFaaS platform, coupled with evaluations using real-world PyFaaS cases, demonstrated that LightLoader significantly reduces cold-start latency and outperforms the state-of-the-art method.

Acknowledgment. This work was supported by the National Key Research and Development Program of China under Grant No. 2024YFB4505902 and the Major Project of ISCAS (ISCAS-ZD-202302).

References

1. Agache, A., et al.: Firecracker: lightweight virtualization for serverless applications. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pp. 419–434 (2020)
2. Amazon Web Services: Serverless Function, faas Serverless - AWS Lambda - AWS, https://aws.amazon.com/lambda/, Accessed 22 July 2025
3. Ao, L., Porter, G., Voelker, G.M.: Faasnap: Faas made fast using snapshot-based vms. In: Proceedings of the Seventeenth European Conference on Computer Systems, pp. 730–746 (2022)
4. Basu Roy, R., Patel, T., Garg, R., Tiwari, D.: Codecrunch: improving serverless performance via function compression and cost-aware warmup location optimization. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 1. pp. 85–101 (2024)
5. Bruce, B.R., Zhang, T., Arora, J., Xu, G.H., Kim, M.: Jshrink: in-depth investigation into debloating modern java applications. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 135–146 (2020)
6. Copik, M., Kwasniewski, G., Besta, M., Podstawski, M., Hoefler, T.: Sebs: a serverless benchmark suite for function-as-a-service computing. In: Proceedings of the 22nd International Middleware Conference, pp. 64–78 (2021)
7. DeepSeek: deepseek-ai/DeepSeek-V3 · Hugging Face (2025), https://huggingface.co/deepseek-ai/DeepSeek-V3, Accessed 22 July 2025
8. Du, D., et al.: Catalyzer: sub-millisecond startup for serverless computing with initialization-less booting. In: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 467–481 (2020)
9. Google: the container security platform - GVisor (2025), https://gvisor.dev/
10. Huang, K., Yan, Y., Chen, B., Tao, Z., Peng, X.: Scalable and precise application-centered call graph construction for python. arXiv preprint arXiv:2305.05949 (2023). (in press)
11. Ivanenko, S., Bruno, R., Stevanovic, J., Veiga, L., Jovanovic, V.: Cloudjit: a just-in-time faas optimizer (work in progress). In: Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes, pp. 12–19 (2023)
12. Liu, D.H., Levy, A., Noghabi, S., Burckhardt, S.: Doing more with less: orchestrating serverless applications without an orchestrator. In: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), pp. 1505–1519 (2023)
13. Liu, X., et al.: Faaslight: general application-level cold-start latency optimization for function-as-a-service in serverless computing. ACM Trans. Softw. Eng. Methodol. **32**(5), 1–29 (2023)
14. Ltd, O.: Openfaas website (2025). https://www.openfaas.com/, Accessed 22 July 2025

15. mnmelo: lazy-import provides a set of functions that load modules, and related attributes, in a lazy fashion, https://pypi.org/project/lazy-import/, Accessed 20 July 2025
16. Oakes, E., et al.: {SOCK}: rapid task provisioning with {Serverless-Optimized} containers. In: 2018 USENIX Annual Technical Conference (USENIX ATC 18), pp. 57–70 (2018)
17. Roy, R.B., Patel, T., Tiwari, D.: Icebreaker: warming serverless functions better with heterogeneity. In: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 753–767 (2022)
18. Schirmer, T., Scheuner, J., Pfandzelter, T., Bermbach, D.: Fusionize++: improving serverless application performance using dynamic task inlining and infrastructure optimization. IEEE Trans. Cloud Comput. (2024)
19. Shahrad, M., et al.: Serverless in the wild: characterizing and optimizing the serverless workload at a large cloud provider. In: 2020 USENIX Annual Technical Conference (USENIX ATC 20), pp. 205–218 (2020)
20. Silva, P., Fireman, D., Pereira, T.E.: Prebaking functions to warm the serverless cold start. In: Proceedings of the 21st International Middleware Conference, pp. 1–13 (2020)
21. Slimtoolkit: Github - slimtoolkit/slim: Slim(toolkit): don't change anything in your container image and minify it by up to 30x (and for compiled languages even more) making it secure too! (free and open source) (2024). https://github.com/slimtoolkit/slim, Accessed 22 July 2025
22. Xu, C., et al.: Faasmem: improving memory efficiency of serverless computing with memory pool architecture. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 3, pp. 331–348 (2024)
23. Yu, H., et al.: Rainbowcake: mitigating cold-starts in serverless with layer-wise container caching and sharing. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 1, pp. 335–350 (2024)
24. Yu, M., Cao, T., Wang, W., Chen, R.: Following the data, not the function: Rethinking function orchestration in serverless computing. In: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), pp. 1489–1504 (2023)
25. Yu, T., et al.: Characterizing serverless platforms with serverlessbench. In: Proceedings of the 11th ACM Symposium on Cloud Computing, pp. 30–44 (2020)

Bridging Service Diversity and CPU Heterogeneity Through Program Similarity-Driven Scheduling

Jiayin Luo[1], Yuxin Ma[1], Xinkui Zhao[1(✉)], Wei Zhou[2], and Jianwei Yin[1]

[1] Zhejiang University, Hangzhou, China
{fro_l,yuxinma,zhaoxinkui}@zju.edu.cn
[2] Chongqing University, Chongqing, China
zjuyjw@zju.edu.cn, zhouwei@cqu.edu.cn

Abstract. As heterogeneous CPU clusters become more prevalent in cloud environments, cloud service providers face the challenge of efficiently scheduling diverse services to harness the potential of CPUs with varying generations and architectures. We present HeteroBridge, a scheduling system that bridges service diversity and CPU heterogeneity through program similarity analysis. To address service diversity, we propose a novel program similarity analysis method that combines embeddings of control flow graphs (CFGs), library function usages, and global features, enabling the system to identify similar programs for resource demands estimation. To address CPU heterogeneity, we introduce the Relative Performance Ratio (RPR) to derive a unified capacity metric across different CPUs. By integrating program similarity analysis and unified capacity modeling, HeteroBridge assigns services to optimal computing nodes. We integrate HeteroBridge with Kubernetes (K8s) and evaluate its performance in heterogeneous environments. Experimental results demonstrate that the Mean Reciprocal Rank (MRR) of program similarity analysis improves by up to 2.37× compared to baseline methods. Simultaneously, service latency is reduced by up to 52% compared to Kubernetes' default scheduler. These findings highlight HeteroBridge's ability to deliver efficient service scheduling in complex cloud environments.

Keywords: Service scheduling · Program similarity analysis · Cloud computing · Heterogeneous CPU clusters

1 Introduction

The rapid growth of cloud computing has driven the adoption of heterogeneous CPU clusters, where data centers integrate CPUs with diverse generations and architectures (e.g., ARM and x86) to optimize the trade-off between performance and cost efficiency [5,9]. This heterogeneity, however, poses critical challenges for efficient service scheduling, particularly in allocating services to optimal nodes

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 107–121, 2026.
https://doi.org/10.1007/978-981-95-5015-9_8

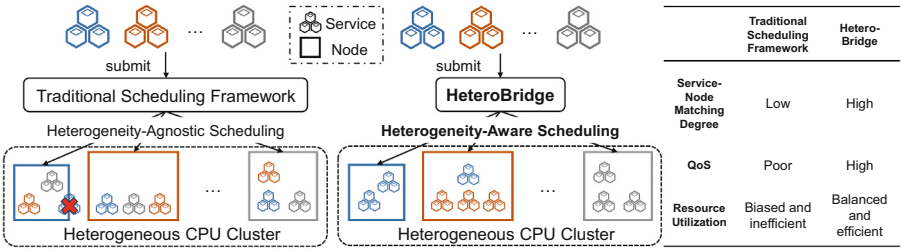


Fig. 1. HeteroBridge's heterogeneity-aware scheduling enhances service performance and resource efficiency in heterogeneous CPU clusters.

to ensure both high-quality service delivery and efficient resource utilization. Current systems, such as Kubernetes[1], rely on user-specified resource demands which require deep workload expertise. In practice, many users lack this expertise, resulting in service-node mismatches that manifest as suboptimal quality of service (QoS) [1]–evident in increased latency and degraded throughput–and inefficient resource utilization, such as resource fragmentation [10].

Existing scheduling systems exhibit two fundamental limitations. First, they treat services as abstract units without accounting for their distinct resource usage patterns [17,18]. For instance, database services demand high I/O throughput while video streaming services prioritize network bandwidth, yet current systems fail to differentiate these characteristics. Second, most approaches neglect CPU heterogeneity, particularly performance disparities stemming from architectural differences (e.g., instruction set extensions) or microarchitectural optimizations (e.g., cache hierarchy) [3]. This dual oversight results in suboptimal service-node mapping, causing (1) compromised QoS due to mismatched execution environments and (2) reduced resource utilization efficiency from underexploited hardware capabilities.

We propose HeteroBridge, a scheduling system that bridges service diversity and CPU heterogeneity through program similarity analysis. Our core insight is that services exhibit distinct and consistent resource usage patterns across different CPUs. By identifying similar programs from history and matching them to their optimal computing nodes, the system enables heterogeneity-aware scheduling without requiring user expertise in workload and hardware characteristics. For similarity analysis, HeteroBridge employs a novel approach that combines multi-dimensional features, including embeddings of CFGs, library function usage, and global features. Specifically, we leverage the Weisfeiler-Lehman (WL) kernel method [16] to generate embeddings efficiently, instead of relying on time-consuming model training [4,6]. To address CPU heterogeneity, we introduce the Relative Performance Ratio (RPR) to derive a unified capacity metric across different CPUs. Integrating program similarity analysis and unified capacity modeling, HeteroBridge achieves efficient and

[1] https://kubernetes.io/.

intelligent service scheduling, ensuring balanced resource utilization and high QoS.

Experimental results demonstrate significant improvements in program similarity analysis accuracy, with the Mean Reciprocal Rank (MRR) improving by up to 2.37 × compared to baselines. Furthermore, service performance is enhanced through intelligent scheduling decisions, achieving service latency reductions of up to 52% under both high-load and low-load scenarios.

To summarize, this paper makes the following contributions:

- We introduce HeteroBridge, a scheduling system that automates service scheduling in heterogeneous CPU clusters via program similarity analysis, hardware abstraction, and real-time scheduling.
- We propose a novel program similarity analysis method, combining embeddings of CFGs, library function usage, and global features to balance between accuracy and efficiency.
- We introduce the RPR to abstract CPU heterogeneity, and develop a real-time scheduling algorithm to assign services to optimal computing nodes.
- We integrate HeteroBridge with Kubernetes. Experimental results demonstrate significant improvements in program similarity analysis accuracy and service performance.

The remainder of this paper is organized as follows: Sect. 2 provides background on service scheduling and program similarity detection; Sect. 3 overviews the architecture of HeteroBridge; Sect. 4 details the key techniques employed in the system; Sect. 5 presents evaluation results; and Sect. 6 concludes the paper.

2 Background and Related Work

2.1 Service Scheduling

Heterogeneous CPU clusters introduce unique challenges for service scheduling due to the coexistence of diverse architectures and varying performance characteristics. Traditional strategies (e.g., first-come-first-served) and heuristic-based approaches [9,25] fail to address resource-performance trade-offs inherent in such environments. Recent advancements have explored energy-aware scheduling [17,18] and machine learning-driven algorithms [2,21,22] to optimize resource allocation. However, most approaches overlook either service diversity [17,18] or CPU heterogeneity [2,22], resulting in suboptimal scheduling decisions that compromise both QoS and resource utilization efficiency.

2.2 Binary Code Similarity Detection (BCSD)

BCSD techniques, which identify structural and behavioral correlations among programs, provide a promising foundation for heterogeneity-aware scheduling. While deep learning-based approaches [4,6,11,12,20] demonstrate strong feature extraction capabilities, their practicality for real-time scheduling remains

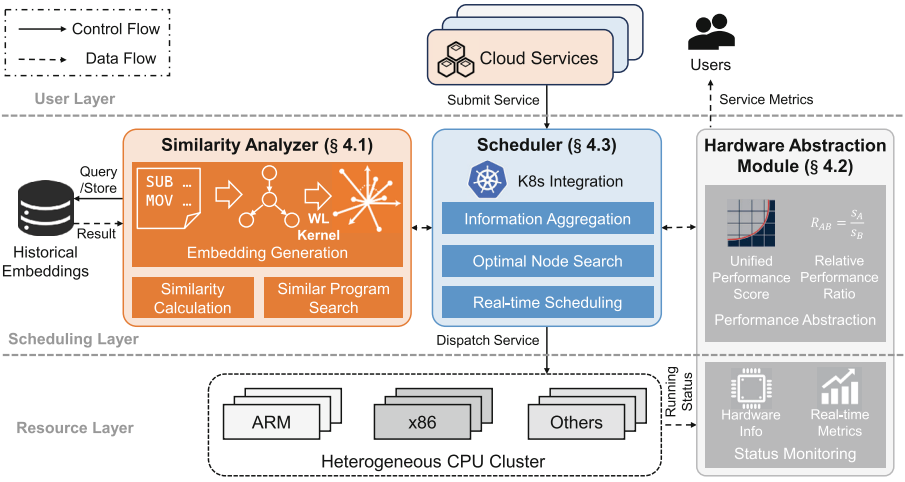


Fig. 2. System architecture of HeteroBridge.

limited due to prohibitive computational costs associated with embedding generation and model training [4,6]. In CFG-based BCSD methods, graph similarity computation serves as a critical enabling component. Classical algorithms such as VF2 [7], GED [24], and Ullmann's algorithm [19] suffer from severe scalability issues, exhibiting exponential or factorial time complexity. In contrast, the WL kernel [16] offers a computationally efficient solution with $O(h \cdot (|V| + |E|))$ complexity, where $|V|$ and $|E|$ represent node and edge counts, and h denotes the iteration depth. By transforming graph comparison into an embedding task, the WL kernel achieves linear scalability for CFGs containing thousands of nodes [15], making it particularly suitable for real-time service scheduling.

3 System Overview

HeteroBridge is designed with a "plug-and-play" computational abstraction philosophy at its core, aiming to simplify the complexity of heterogeneous computing environments.

The overall architecture of HeteroBridge follows a three-layer structure as illustrated in Fig. 2. Users submit diverse services at the *User Layer* without requiring expertise in the underlying hardware and workload characteristics. The *Scheduling Layer* identifies similar programs for resource demands estimation and assigns services to their optimal computing nodes based on the matching degree between services' resource demands and nodes' unified capacity. Specifically, the Scheduling Layer consists of three key components: the *similarity analyzer*, the *hardware abstraction module*, and the *scheduler*. The similarity analyzer extracts CFGs and transforms them into compact embeddings via the WL kernel method. It then calculates similarities among programs based on

Table 1. Instruction categorization based on hardware-specific features.

Category	Description	Example
GP	general purpose instructions	ADD
GP_EXT	extended general purpose instructions	POPCNT
GP_IO	I/O instructions	IN
FPU	floating point unit instructions	FMUL
MMX	multimedia extension instructions	PSUBB
STATE	state control instructions	CLTS
SIMD	single instruction multiple data instructions	PAND
SSE	streaming SIMD extensions	MOVAPS
SCALAR	scalar instructions	ADDSD
CRYPTO	cryptographic hash instructions	SHA256MSG1
AVX	advanced vector extensions	VBLENDPS
AVX512	advanced vector extensions 512	VPADDD
MASK	mask control instructions for vector operations	KADDQ
AMX	advanced matrix extension instructions for AI acceleration	AMX_GEMM

embeddings, global features, and library function usages. Meanwhile, the hardware abstraction module establishes a standardized performance evaluation for heterogeneous CPUs. By integrating historical performance data of similar programs and real-time resource status, the scheduler adaptively selects optimal nodes to balance QoS and resource utilization. The *Resource Layer* manages and monitors nodes with heterogeneous CPUs, and metrics are fed back to the User Layer for observability and the Scheduling Layer for real-time scheduling.

4 Detailed Design and Implementation

4.1 Similarity Analyzer: Service Comparison

Upon receiving the submitted service, the similarity analyzer first disassembles the program into assembly code via `objdump`. It then constructs a CFG whose nodes represent different functions and adds edges by analyzing the relationships of functions. To better capture program characteristics in heterogeneous CPU clusters, instructions are categorized based on hardware-specific features as shown in Table 1. For each node, we gather detailed statistics, including the number of each instruction type, total instruction count, as well as the node's in-degree and out-degree, to form its feature vector, based on which similarities between different CFGs are quantifiable.

After constructing node features $\vec{n} = (n_0, n_1, \cdots, n_{d_n-1})$, the service comparison process comprises four core stages (as shown in Fig. 3): feature aggregation, data normalization, embedding generation, and similarity calculation. These stages work synergistically to capture both structural and functional similarities across services.

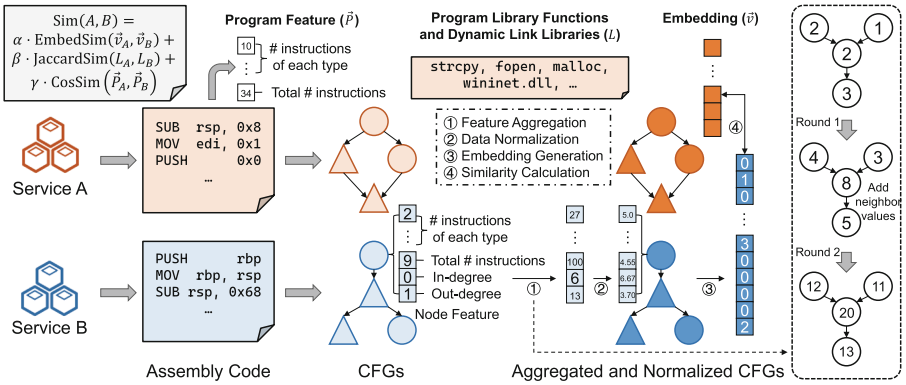


Fig. 3. Service comparison process of the similarity analyzer.

Feature Aggregation. The initial stage employs the WL kernel to iteratively enrich node features by aggregating neighboring structural information. Specifically, as illustrated in the right of Fig. 3, each node undergoes two rounds of aggregation: in the first round, node features are expanded using local neighbor features to capture direct structural dependencies; in the second round, features are further enhanced by incorporating second-degree neighbor information. This hierarchical aggregation ensures that aggregated node features $\vec{n}'$ reflect both local and extended structural characteristics, forming a robust foundation for subsequent processing.

Data Normalization. To address variations in program scale and instruction count, we propose an intermediate representation $\vec{r}$ to normalize aggregated features to values independent of the program scale. For each feature dimension i, we divide the total sum of all nodes' corresponding feature by the current node's feature value, forming a relative proportion $r_i = \sum_{j=0}^{d_n-1} n'_j / n'_i$. Instead of mapping data to fixed intervals (e.g., $[0, 1]$) like traditional normalization methods, the output of our approach spans the range $[1, +\infty)$, emphasizing the relative rarity or significance of node features rather than their absolute values.

The maximum ($\max r_i$) and minimum ($\min r_i$) values of each normalized feature are recorded for subsequent processing.

Embedding Generation. Despite normalization, significant feature range variability persists, as shown in Fig. 4. To address this non-uniformity and map node features into a compact program-level embedding $\vec{v}$ for similarity calculation, we propose a hierarchical interval mapping strategy.

For each normalized feature r_i, partition its value range $[\min r_i, \max r_i]$ into $2^{m_i} - 1$ unit intervals with width $w_i = \frac{\max r_i - \min r_i}{2^{m_i}-1}$, where $m_i > 0$ denotes the number of interval divisions per feature. Interval bounds follow a geometric progression. The i-th interval's value range is $[2^i \cdot w_i, 2^{i+1} \cdot w_i)$. This ensures

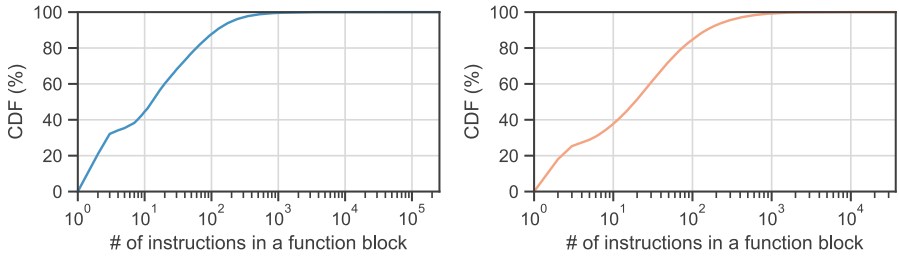


Fig. 4. The distribution of assembly function block lengths in BinKit [8] (left) and Asteria-Pro [23] (right) datasets.

Algorithm 1. Program Embedding Generation

Input: Normalized node feature map N
Output: Program embedding vector $\vec{v}$
Initialize $\vec{v}$ as a zero vector of size d_v
for all key $\in N$.keys **do** ▷ Iterate over each node
  $\vec{r} \leftarrow N[\text{key}]$
  for $i \leftarrow 0$ to $l-1$ **do**
    $\text{dif} \leftarrow \vec{r}[i] - \min r_i$
    $\text{pos} \leftarrow \text{BinSearch}(\text{intervals}, \text{dif})$ ▷ Find interval position via binary search
    $j \leftarrow \text{pos} \times \prod_{k=0}^{i-1} m_k$
    $\vec{v}[j] \leftarrow \vec{v}[j] + 1$ ▷ Increment embedding at corresponding index
  end for
end for

fine-grained resolution for low-magnitude values while coarsening intervals for extreme values, balancing data distribution. The final embedding dimension equals the product of interval counts across features, $d_v = \prod_{i=0}^{l-1} m_i$, where l is the node feature length. We select minimal m_i values preserving critical feature patterns to balance information retention and computational efficiency. This process is formalized in Algorithm 1, which iterates over each CFG node, maps feature values to intervals, and aggregates counts into a program embedding.

Similarity Calculation. The final stage computes program similarity through three complementary metrics: (1) *Embedding Similarity* captures algorithmic structures. Algorithm 2 evaluates embedding alignment by measuring non-zero feature overlap. Value differences are penalized via $\frac{1}{|a-b|+1}$ to prioritize closely matched features. (2) *Library Function Similarity* depicts external dependency profiles by measuring the overlap of shared library functions and system calls (L) between programs using Jaccard similarity. (3) *Global Feature Similarity* models aggregated statistics. It compares overall program features $\vec{P}$ which are consisted of number of total instructions, function calls, and each instruction type using

Algorithm 2. Embedding Similarity Calculation

Input: Embeddings $\vec{v}_A$ (program A) and $\vec{v}_B$ (program B)
Output: Similarity score $\text{EmbedSim}(\vec{v}_A, \vec{v}_B)$

Initialize sum $\leftarrow 0$, intersection $\leftarrow 0$, nz $\leftarrow 0$
for $i \leftarrow 0$ to $d_v - 1$ **do**
  $a \leftarrow \vec{v}_A[i]$, $b \leftarrow \vec{v}_B[i]$
  if $a \neq 0$ **or** $b \neq 0$ **then**
    nz $\leftarrow$ nz $+ 1$
    if $a \neq 0$ **and** $b \neq 0$ **then**
      intersection $\leftarrow$ intersection $+ 1$ ▷ Count intersections
      sum $\leftarrow$ sum $+ \frac{1}{|a-b|+1}$ ▷ Penalize value differences
    end if
  end if
end for
$\text{EmbedSim}(\vec{v}_A, \vec{v}_B) \leftarrow 0.5 \cdot \frac{\text{intersection}+1}{\text{nz}+1} + 0.5 \cdot \frac{\text{sum}+1}{\text{intersection}+1}$

cosine distance. The final similarity score combines these components:

$$\text{Sim}(A, B) = \alpha \cdot \text{EmbedSim}(\vec{v}_A, \vec{v}_B) + \beta \cdot \text{JaccardSim}(L_A, L_B) + \gamma \cdot \text{CosSim}(\vec{P}_A, \vec{P}_B)$$

where $\alpha + \beta + \gamma = 1$. This multifaceted approach captures structural and functional characteristics, ensuring robustness against architectural variations.

4.2 Hardware Abstraction Module: Performance Abstraction

To establish a unified capacity model for heterogeneous CPUs, we leverage the SPEC CPU 2017 benchmark dataset[2], which contains 43 benchmarks organized into four suites and provides comprehensive performance metrics across diverse architectures. Building on this foundation, we develop a performance abstraction method that captures both hardware capabilities and service characteristics.

For each CPU type, we compute a performance score s by averaging normalized SPEC scores across all benchmarks, with weights derived from the similarity between service programs and benchmark workloads. This score serves as a quantified metric of computational capacity tailored to specific service requirements. We further introduce the Relative Performance Ratio (RPR) to model cross-CPU performance disparities. Given a target CPU A and reference CPU B, the RPR $R_{AB} = s_A/s_B$ represents the relative capability of CPU A compared to CPU B for executing the specific service. During scheduling, this ratio enables dynamic capacity calibration: if two nodes with CPU X and reference CPU B each have 4 physical cores available, and their RPR $R_{XB} = 1.5$ for the current workload, the node with CPU X would be credited with 6 equivalent CPU units in the unified capacity model.

This abstraction transforms heterogeneous CPUs into comparable resource units with performance-weighted metrics. By integrating RPR-based capacity

[2] https://www.spec.org/cpu2017/.

modeling into scheduling decisions, HeteroBridge ensures that both CPU heterogeneity and service characteristics are explicitly accounted for, enabling precise service-node matching while improving cluster utilization efficiency.

4.3 Scheduler: Real-Time Scheduling

Leveraging similarity analysis and hardware abstraction, we implement a real-time scheduling algorithm to assign services to optimal computing nodes.

The scheduler continuously collects time-series metrics (e.g., CPU utilization, memory usage, disk I/O, network throughput) via Prometheus[3] to monitor node status and resource consumption of running services. When a new service is submitted, the similarity analyzer identifies the three most similar running services, and the scheduler estimates the resource demands of the submitted service by calculating a weighted average of its similar services' resource usage patterns, with weights proportional to their similarity scores.

To standardize comparisons, we apply Z-score normalization to both the estimated service resource demands $\vec{S}$ and the remaining node resources $\vec{N}$. Notably, the CPU dimensions of $\vec{S}$ and $\vec{N}$ are calibrated using the RPRs introduced in Sect. 4.2, ensuring consistency in heterogeneous environments.

The service-node matching degree is determined through two complementary dimensions. The first dimension, *service scalability*, measures headroom for dynamic scaling by quantifying the minimum ratio of remaining resources to required resources across all dimensions, expressed as $\min\left(\frac{N_i}{S_i+\epsilon}\right)$[4]. These values are normalized to $[0, 1]$ via min-max normalization, denoted as SS. The second dimension, *resource similarity*, accounts for resource efficiency by measuring the alignment between available node resources and service demands. This is calculated as the cosine similarity between $\vec{N}$ and $\vec{S}$, with results RS normalized using the same method as service scalability. The final matching degree is computed as $\theta \cdot \mathrm{SS} + (1-\theta) \cdot \mathrm{RS}$, where $\theta \in [0, 1]$ balances the trade-off between scalability potential and resource alignment. The node with the highest matching degree is selected for service deployment.

By integrating real-time monitoring with program similarity analysis, HeteroBridge achieves efficient service scheduling that simultaneously enhances service performance and resource utilization efficiency.

4.4 Implementation

HeteroBridge is implemented in 4.4K lines of Golang code, leveraging Kubernetes as the orchestration framework for deployment and resource management. During service submission, scheduling decisions are transmitted to K8s via native APIs, enabling seamless integration and scalable execution in heterogeneous environments. For similarity calculation, the parameters α, β, and γ are uniformly set to 1/3. In the service-node matching degree computation, ϵ and θ are configured to 0.01 and 0.5, respectively.

[3] https://prometheus.io/.

[4] ϵ is a small constant to avoid division by zero.

Table 2. HeteroBridge evaluation setup.

Physical machine	(a) Cluster setup				(b) Softwares	
	Virtual machine	CPU cores	RAM (GB)	OS	Software	Version
Intel Core i9 14900K	Master node	4	4	Ubuntu 22.04	Kubernetes	1.31.0
	Worker node A	1	2		MySQL	8.0.35
	Worker node B	2	4		Nginx	1.28
AMD Ryzen9 8945H	Worker node C	2	4		Redis	7.4.1
Intel Xeon Gold 6430	Worker node D	4	8		Stress-ng	0.18

Table 3. Matching results of HeteroBridge versus baselines.

Method	MRR								Precision							
	XO					XC	XA	Average	XO					XC	XA	Average
	O0-O3	O2-O3	O0-Os	O2-Os	Average				O0-O3	O2-O3	Ofast-O3	Ofast-Os	Average			
Safe	0.319	0.774	0.224	0.623	0.485	0.659	0.026	0.390	0.375	0.921	**0.976**	0.856	0.782	0.419	0.104	0.435
Trex	0.287	0.849	0.377	0.663	0.544	0.548	0.514	0.535	0.587	**0.948**	0.958	0.830	0.591	0.786	0.796	0.724
HeteroBridge	**0.907**	**0.957**	**0.930**	**0.926**	**0.930**	**0.928**	**0.914**	**0.924**	**0.866**	0.929	0.961	**0.981**	**0.935**	**0.890**	**0.872**	**0.899**

5 Evaluation

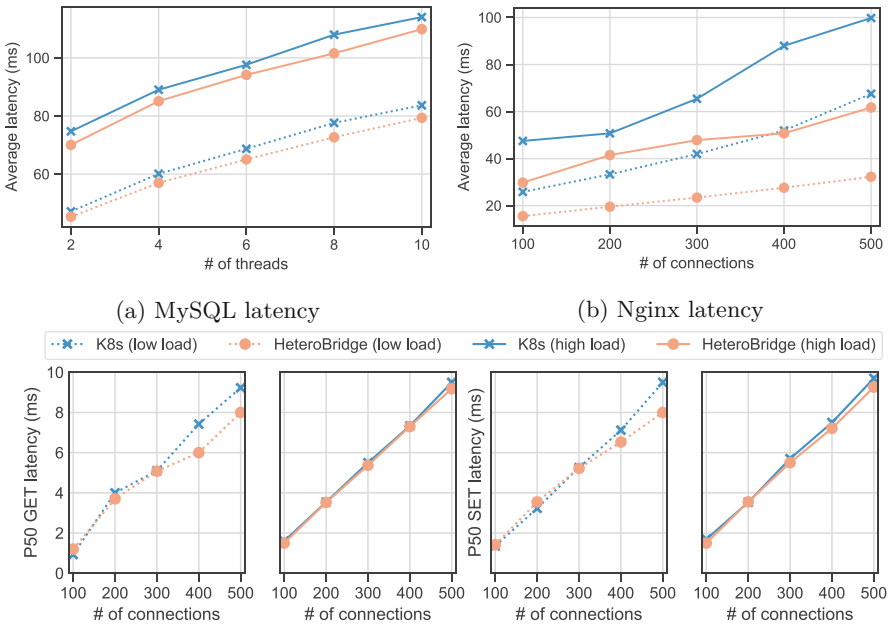
To validate the effectiveness of HeteroBridge, we designed a comprehensive evaluation framework targeting three core objectives: (1) accuracy in program similarity analysis, (2) performance improvement in service scheduling, and (3) scalability for large-scale deployment. As shown in Table 2, experiments were conducted on a Kubernetes cluster comprising one master node and four worker nodes. We deployed nodes with virtual machines (VMs) for flexible resource allocation. Worker nodes were configured with varying CPU types and memory resources (2–4 cores, 2–8 GB RAM) to simulate heterogeneous environments. Stress-ng[5] containers were deployed to simulate real-world workloads.

5.1 Accuracy in Program Similarity Analysis

The first objective was to evaluate HeteroBridge's ability to identify program similarities. The BinKit dataset (371,928 binaries spanning 8 architectures and 23 compilers) was used for evaluation, with a focus on ARM and x86 architecture programs due to their widespread adoption in modern computing systems. We compared it against two baseline methods, Safe [11] and Trex [12], across three critical dimensions: cross-optimizations (XO, e.g., O0-O3), cross-compilers (XC, e.g., Clang vs. GCC), and cross-architectures (XA, e.g., x86 vs. ARM).

As shown in Table 3, HeteroBridge achieved an MRR of 0.930 in cross-optimization tasks, outperforming Trex (0.544) and Safe (0.485) by 71–92%. Precision metrics also showed significant improvements (58% and 19% gains, respectively). In cross-compiler scenarios, HeteroBridge maintained superior performance despite variations in compiler versions and optimization strategies.

[5] https://github.com/ColinIanKing/stress-ng.



(a) MySQL latency (b) Nginx latency

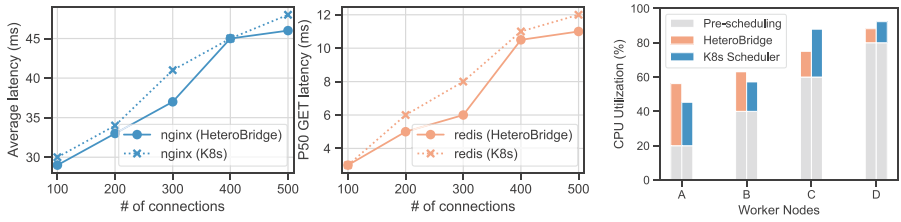
(c) Redis GET (left) and SET (right) latency

Fig. 5. Latency comparisons across MySQL, Nginx, and Redis under different scheduling strategies and load conditions.

Similarly, in cross-architecture comparisons, HeteroBridge's MRR of 0.914 exceeded those of Trex (0.514) and Safe (0.026). On average, HeteroBridge achieved an improvement in MRR by **1.73–2.37**×, with precision enhanced by **1.24–2.07**× across different architectures and optimization levels. These results establish HeteroBridge as a reliable method for capturing low-level patterns that are invariant to compilation settings, thereby forming the foundation for subsequent scheduling optimizations.

5.2 Performance Improvement in Service Scheduling

Building on the accuracy validation, we evaluated how HeteroBridge's program similarity analysis translates into practical performance improvements in service scheduling. Three representative services (MySQL, Nginx, and Redis) were selected for testing. After cluster deployment, services were randomly scheduled to simulate user-submitted workloads, with HeteroBridge compared against Kubernetes' default scheduler under varying load conditions. Workload stress tests were conducted using sysbench, wrk, and redis-benchmark: for MySQL, the sysbench OLTP read-write test was executed at thread counts from 2 to 10 for 1 min, with table sizes set to 1,000,000 (high load) and 500,000 (low load); for Nginx and Redis, wrk and redis-benchmark were configured to handle



(a) Service latencies in heavy-load nodes (b) CPU utilization

Fig. 6. Service latencies and CPU utilization under different node load levels.

100 RPS (high load) and 50 RPS (low load) with concurrent connections ranging from 100 to 500, each running for 5 min. This setup enabled a comprehensive evaluation of HeteroBridge's effectiveness across diverse service types and load scenarios.

As shown in Fig. 5, HeteroBridge consistently outperformed K8s' native scheduler across all tested services. For example, the average latency of MySQL dropped from 114.05 ms to 109.91 ms (Fig. 5A), while Nginx's latency at the highest load (500 connections) improved from 67.54 ms to 32.25 ms, achieving a **52%** reduction (Fig. 5b). Similarly, Redis demonstrated consistent gains in SET and GET operations, particularly under high connection counts (Fig. 5c). These improvements proved HeteroBridge's ability to align program resource profiles with node capabilities, minimizing contention and over-allocation.

To further analyze HeteroBridge's performance, we compared service latencies under different node load levels (high vs. low CPU utilization). Before the experiment, we set the initial resource utilization of each node to range between 20% and 80% (worker node A: 20%; B: 40%; C: 60%; D: 80%) to simulate the real-world distribution of cluster resources.

For nodes under heavy loads (C and D), HeteroBridge exhibited distinct advantages, as illustrated in Fig. 6a. For example, Redis's P50 GET latency decreased by 10.7%, and Nginx latency reduced by 4.1%. These results demonstrate HeteroBridge's capability to minimize bottlenecks in constrained environments through precise resource matching and efficient utilization of limited node capacity. For nodes under low or moderate loads (A and B), HeteroBridge's performance was suboptimal (3.2% latency increase for Redis and 2.4% latency increase for Nginx). While the default scheduler does not strictly follow HeteroBridge's resource proportionality criteria, nodes with abundant idle resources outperformed algorithm-selected nodes. This is attributed to the untapped computational capacity of underutilized nodes, enabling programs to execute more efficiently despite less precise resource matching. However, as shown in Fig. 6b, HeteroBridge achieves more balanced resource utilization across the cluster, mitigating resource fragmentation and idle resource waste.

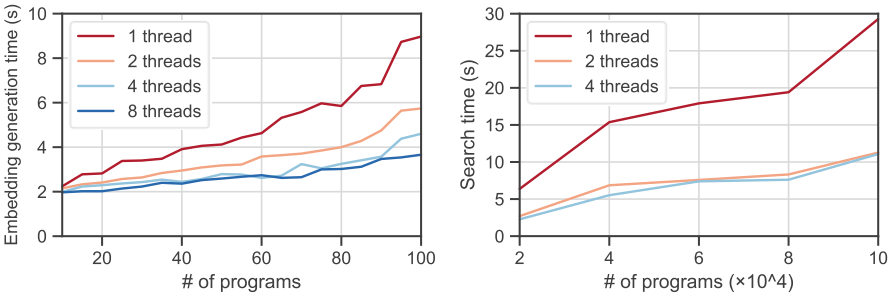


Fig. 7. Time cost of HeteroBridge.

5.3 Scalability for Large-Scale Deployment

To address the third objective, we tested HeteroBridge's computational efficiency in processing large binary datasets.

As shown in Fig. 7, HeteroBridge generated embeddings for **100 binaries in 4 s** using 8 threads, while methods such as VexIR2Vec [20], Safe [11], and Binfinder [13] require hundreds of seconds to process. This efficiency is critical for real-time scheduling, where rapid analysis of program features is required. For search tasks, HeteroBridge employs a parallel design in its search process, allowing simultaneous comparisons of the query object with multiple programs in the library. HeteroBridge achieves a 3× speedup over sequential execution, and it completed queries in a repository of 10^5 programs **within 12 s**. These results confirm that HeteroBridge's design–combining lightweight feature extraction with parallelizable search–enables high-throughput processing, making it suitable for clusters with thousands of nodes. This scalability ensures that the benefits observed in small-scale experiments (e.g., latency reduction in Fig. 5) are maintainable at industrial-scale deployments.

6 Conclusion and Future Work

This paper introduces HeteroBridge, a scheduling system that bridges service diversity and CPU heterogeneity through program similarity analysis. By transforming programs into compact vector representations using the WL kernel method and combining multi-dimensional metrics, our approach achieves high-precision similarity comparisons across different architectures, compilers, and optimization levels (up to 2.37× MRR improvement compared to baselines). HeteroBridge abstracts CPU heterogeneity with the Relative Performance Ratio and develops a real-time scheduling algorithm, enabling efficient service scheduling. Experimental results demonstrate that HeteroBridge significantly reduces service latency in both high-load and low-load scenarios, achieving service latency reductions of up to 52% compared to K8s' default scheduler.

In the future, we aim to enhance scheduling strategies to improve the utilization of idle resources through more adaptive mechanisms. Furthermore, we intend to incorporate predictive models [5,14] to refine resource matching in heterogeneous environments, extending optimization beyond CPUs to include GPUs and NPUs.

Acknowledgments. We thank reviewers for their valuable feedbacks. This work was supported in part by the National Science Foundation of China under Grants (62472375, 62125206 and 62502446), and in part by the Major Program of National Natural Science Foundation of Zhejiang(LD24F020014, LD25F020002), and in part by the Zhejiang Pioneer (Jianbing) Project (2024C01032), in part by the Ningbo Yongjiang Talent Programme(2023A-198-G).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Cai, K., Wu, Q., Zhou, M., Chen, C., Wen, J., Wang, S.: Dynamically scheduling deadline-constrained interleaved workflows on heterogeneous computing systems. IEEE Trans. Serv. Comput. **18**(2), 758–769 (2025)
2. Ding, X., et al.: Towards vm rescheduling optimization through deep reinforcement learning. In: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys 2025) (2025)
3. Gupta, K., Sharma, T.: Changing trends in computer architecture: a comprehensive analysis of arm and x86 processors. Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol. **7** (2021)
4. He, H., et al.: Code is not natural language: unlock the power of Semantics-Oriented graph representation for binary code similarity detection. In: 33rd USENIX Security Symposium (USENIX Security 24) (2024)
5. Hua, Q., Yang, D., Qian, S., Cao, J., Xue, G., Li, M.: Humas: a heterogeneity- and upgrade-aware microservice auto-scaling framework in large-scale data centers. IEEE Trans. Comput. **74**(3) (2025)
6. Jia, A., Fan, M., Xu, X., Jin, W., Wang, H., Liu, T.: Cross-inlining binary function similarity detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024) (2024)
7. Jüttner, A., Madarasi, P.: Vf2++—an improved subgraph isomorphism algorithm. Discrete Appl. Math. **242** (2018)
8. Kim, D., Kim, E., Cha, S.K., Son, S., Kim, Y.: Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned. IEEE Trans. Softw. Eng. **49**(4) (2022)
9. Liu, J., Lai, F., Ding, E., Zhang, Y., Chowdhury, M.: Venn: Resource management for collaborative learning jobs. In: Eighth Conference on Machine Learning and Systems (MLSys 2025) (2025)
10. Lu, C., et al.: Understanding and optimizing workloads for unified resource management in large cloud platforms. In: Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys 2023) (2023)

11. Massarelli, L., Di Luna, G.A., Petroni, F., Querzoni, L., Baldoni, R.: Safe: Self-attentive function embeddings for binary similarity. In: Proceedings of 16th Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA) (2019)
12. Pei, K., Xuan, Z., Yang, J., Jana, S., Ray, B.: Learning approximate execution semantics from traces for binary function similarity. IEEE Trans. Softw. Eng. **49**(4) (2023)
13. Qasem, A., Debbabi, M., Lebel, B., Kassouf, M.: Binary function clone search in the presence of code obfuscation and optimization over multi-cpu architectures. In: Proceedings of the 2023 Acm Asia Conference on Computer and Communications Security (2023)
14. Reidys, B., et al.: Coach: Exploiting temporal patterns for all-resource oversubscription in cloud platforms. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2025), vol. 1 (2025)
15. Schulz, T.H., Horváth, T., Welke, P., Wrobel, S.: A generalized weisfeiler-lehman graph kernel. Mach. Learn. **111**(7) (2022)
16. Shervashidze, N., Schweitzer, P., Van Leeuwen, E.J., Mehlhorn, K., Borgwardt, K.M.: Weisfeiler-lehman graph kernels. J. Mach. Learn. Res. **12**(9) (2011)
17. Stafford, E., Bosque, J.L.: Enhancing heterogeneous cluster efficiency through node-centric scheduling. J. Supercomput. **80**(10) (2024)
18. Sukprasert, T., Souza, A., Bashir, N., Irwin, D., Shenoy, P.: On the limitations of carbon-aware temporal and spatial workload shifting in the cloud. In: Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys 2024) (2024)
19. Ullmann, J.R.: An algorithm for subgraph isomorphism. J. ACM (JACM) **23**(1) (1976)
20. VenkataKeerthy, S., et al.: Vexir2vec: an architecture-neutral embedding framework for binary similarity. ACM Trans. Softw. Eng. Methodol. (2025)
21. Wang, Z., Wang, Y., Deng, J., Zheng, D., Li, A., Ding, Y.: Rap: resource-aware automated gpu sharing for multi-gpu recommendation model training and input preprocessing. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2024), vol. 2 (2024)
22. Wang, Z., Li, P., Liang, C.J.M., Wu, F., Yan, F.Y.: Autothrottle: a practical bi-level approach to resource management for SLO-Targeted microservices. In: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 2024) (2024)
23. Yang, S., et al.: Asteria-pro: enhancing deep learning-based binary code similarity detection by incorporating domain knowledge. ACM Trans. Softw. Eng. Methodol. **33**(1) (2023)
24. Zeng, Z., Tung, A.K., Wang, J., Feng, J., Zhou, L.: Comparing stars: on approximating graph edit distance. Proc. VLDB Endowment **2**(1) (2009)
25. Zhao, B., Xu, W., Liu, S., Tian, Y., Wang, Q., Wu, W.: Training job placement in clusters with statistical in-network aggregation. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2024), vol. 1 (2024)

Optimizing QoS Fulfillment of Drone Services

Syeda Amna Rizvi(✉) and Athman Bouguettaya

The University of Sydney, Sydney, NSW 2000, Australia
{amna.rizvi,athman.bouguettaya}@sydney.edu.au

Abstract. We propose a novel framework for optimizing Quality of Service (QoS) fulfillment to meet drone providers' satisfaction. The proposed framework leverages the provisioning of drone service needs in terms of on-time recharging to maximize providers' satisfaction. A QoS fulfillment impact model is developed to assess the effect of inter-drone interference in meeting desired delivery time and energy consumption targets. A novel interference-aware algorithm is proposed for effective and efficient provisioning of drone service needs to fulfill their desired QoS targets. Experimental results conducted on real-world data validate the efficiency and effectiveness of the proposed approach.

Keywords: Drone Services · Inter-drone Interference · Skyway Network

1 Introduction

Unmanned Aerial Vehicles (UAVs), commonly known as drones, enable rapid provisioning of delivery services where speed and timing are paramount [1]. However, drones have limited battery capacity, which restricts their flight duration when heavier packages are required to be delivered over long distances. Battery swapping techniques are proposed as a solution to facilitate long-distance deliveries [2]. However, battery swapping causes safety concerns and delays due to the need for precise landing and replacement procedures [12]. Therefore, the preferred approach is to recharge at the designated charging stations during the flight. The *skyway network* provides supporting infrastructure for drones to recharge on building rooftops equipped with charging pads [8]. This network consists of *nodes*, which are typically the building's rooftops serving as both the charging stations and delivery destinations. The *skyway segments* are the delimited aerial corridors for drones to transit safely between nodes.

The service paradigm is leveraged to model the services provided by drones through *Drone-as-a-Service (DaaS)* abstraction [17]. A *DaaS* instance refers to the transportation of packages between nodes. We identify two types of constraints that may impact a DaaS instance: *intrinsic* (e.g., battery capacity, payload, flight range) and *extrinsic* (e.g., weather, charging pad availability at nodes). A single *DaaS* instance may not be sufficient to serve a long-distance

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 122–137, 2026.
https://doi.org/10.1007/978-981-95-5015-9_9

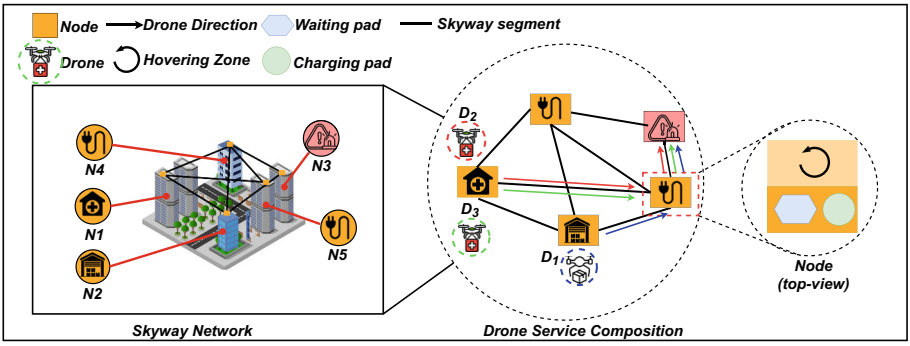
delivery due to the aforementioned constraints [5]. In this respect, *service composition* techniques are employed to integrate multiple DaaS instances [17–19].

A drone-based ecosystem is a multi-stakeholder environment consisting of *service consumers*, *service providers*, and a *super provider*. Service consumers are the end-users who request package delivery services from service providers. Service providers are the drone operators who compose drone delivery paths to meet the consumers' requirements. Super provider serves as a management authority that ensures dynamic provisioning of drone service needs (e.g., recharging) to maximize the fulfillment of their QoS targets (e.g., delivery time, energy consumption). The *satisfaction* of drone providers depends on how well a delivery service meets its predefined QoS targets in a multi-drone skyway network. Most existing work focuses on the individual service provider's perspective, where each provider composes a delivery path *greedily* to meet their own QoS targets. However, these greedy strategies may hinder the fulfillment of other drone providers' QoS requirements, thus limiting equitable access to the shared skyway infrastructure. *To the best of our knowledge, this paper is the first attempt to consider the super provider's perspective to maximize the QoS fulfillment of all the drone providers using the shared skyway network.*

The failure to meet QoS targets mainly occurs due to *inter-drone interference*. We identify that these interferences occur between drones at the *nodes* or *skyway segments*. A *node-level interference* refers to an event when the drone is forced to wait or hover due to the unavailability of the recharging pads. This may lead to non-fulfillment of QoS targets in terms of delivery time due to increased waiting time than initially anticipated by the drone provider. A *segment-level interference* refers to an event where a drone's operation is impacted due to the effect of turbulence caused by nearby drones operating within the same skyway segment. This interference may result in increased battery consumption, affecting the fulfillment of QoS targets such as delivery time, and flight safety [21].

A drone traffic management system is required to dynamically provision the *needs* of drone services to ensure that their QoS targets are met. An example of such a *need* is the availability of a recharging pad upon arrival at a node, with minimal waiting time. A key aspect of this management involves the *detection* and *resolution* of inter-drone interferences. *Detection* refers to the process of identifying interferences that may hinder the fulfillment of QoS targets. *Resolution* refers to the process of mitigating or minimizing the impact of these interferences to ensure service delivery within acceptable QoS bounds. Existing work proposes strategies for resolving *interference* that occurs due to turbulence generated by drones operating in the same segment [16]. Therefore, the focus of this paper is to *maximize the QoS fulfillment by addressing the interferences that may occur between drones at the charging stations (i.e., nodes).*

The efficient provisioning of drone services' charging needs at the nodes is challenging due to the *dynamic* and *constrained* nature of the skyway network. Each node is constrained by a finite number of recharging pads due to the limited rooftop area available for deployment. Furthermore, the environment is highly *dynamic* as the arrival of drones at the nodes depends on the independent sched-



Drone ID	Charging Slot Requirement	Battery Level on Arrival (%)	Battery Consumption due to Hovering (%)	Actual Charging Slot Allocated	Delay in Estimated Delivery Time (min)	Drone Service Status
D_1	10:00-10:20	35%	0%	10:00-10:20	**0**	**Fully Serviced**
D_2	10:05-10:15	15%	0%	10:20-10:30	**15**	**Unserviced**
D_3	10:06-10:10	20%	40%	10:30-10:36	**20**	**Unserviced**

Fig. 1. Motivating Scenario

ules set by drone providers. Moreover, each drone may have varying charging needs based on attributes such as payload weight and flight range. Therefore, the focus of this paper is to optimize the QoS fulfillment of drone services by resolving inter-drone interference occurring at the nodes in a highly constrained skyway network. The main contributions of this paper are as follows:

- We model the impact of interferences on the QoS fulfillment of drone services due to the unavailability of charging pads at the nodes.
- We propose a novel framework for detecting and resolving interferences occurring between drones at the charging stations.
- A QoS-aware interference resolution algorithm that adapts to the dynamic arrivals of drones at a node. The proposed algorithm efficiently provisions the varying charging needs of drone services in a shared skyway network.

2 Motivating Scenario

We consider a shared skyway network where drones from multiple providers compete to access the charging infrastructure to meet their QoS targets. Figure 1 shows the skyway network constructed based on the Line-of-Sight (LoS) skyway segments between nodes. Each *node* serves as a *charging station* for drones. The *charging station* is provisioned with a finite number of charging pads, waiting pads, and a designated *hovering zone*, where drones can wait mid-air if the waiting and charging pads are fully occupied (see Fig. 1). The number of charging and waiting pads at a node is determined based on the capacity of the rooftop area.

Let us assume a scenario where, in response to a localized emergency event, drones are dispatched to deliver urgent medical supplies (e.g., blood units, defibrillators, and trauma kits) to a temporary triage zone established at node $N3$

(see Fig. 1). Drones D_2 and D_3 are dispatched from a medical supply hub located at node $N1$, while drone D_1 takes off from a logistics warehouse at node $N2$. Given the urgency of the situation, all drone providers independently plan the shortest path for the drones to minimize delivery time. These drones are scheduled to recharge at the intermediate node $N3$. *Inter-drone interference* occurs due to the *constrained* availability of the charging and waiting pads at node $N5$. We assume the First-Come-First-Served (FCFS) approach as the strategy for allocating charging pads to drones to ensure fairness. As a result, drone D_2 is forced to wait at the waiting pad due to interference from drone D_1 for $\approx$15 min. Consequently, drone D_3 is forced to hover for $\approx$14 min due to interference from drone D_2. This increased battery consumption due to hovering necessitates the drone to spend more time for successful package delivery to the destination. As a result of these *inter-drone interferences*, the on-time charging requirements of drone services D_2 and D_3 are not fulfilled, leading to unmet QoS targets in terms of delivery time and energy consumption.

While the FCFS approach ensures fairness, it may lead to the non-fulfillment of QoS targets of drone services in a highly *constrained* environment. Therefore, there is a need for an inter-drone interference resolution framework that considers the varying QoS needs of drone services. These QoS needs are critical to meet, especially in the case of time-sensitive deliveries (e.g., medical supplies), where delays can significantly compromise emergency interventions.

3 Related Work

The existing research on drone-based delivery services can be grouped into three categories: (1) Service-based Drone Routing Approaches, (2) Data-driven Drone Routing Approaches, and (3) Unmanned Traffic Management (UTM) Approaches for UAVs in shared urban airspace.

Service-Driven Drone Routing Approaches. Service-driven drone routing approaches leverage the service paradigm to compose an optimal skyway path for package delivery [17]. A Probabilistic Forward Search (PFS) algorithm is proposed to select the skyway segments with favorable wind conditions to reduce delivery delays caused by uncertainty in weather [19]. A constraint-aware deterministic look-ahead algorithm is proposed to reroute the drones to less congested nodes for minimizing the wait due to unavailability of charging pads [18]. Another work proposes an algorithm that ranks multiple candidate paths for drones based on the probability of delays at each node. The aim of the proposed approach is to select a drone service composition plan to minimize delivery time, considering the dynamic congestion conditions at the nodes [20]. An LSTM-based method for composing drone services is proposed while considering congestion levels at the charging stations based on historical data [11].

Data-Driven Drone Routing Approaches. A Heterogeneous Attention-based Deep Reinforcement Learning (HADRL) is proposed to enable route planning for drones to minimize total travel distance. [4]. Another work leverages road traffic to extend drone flight range [9]. The proposed framework allows drones to

recharge at roadside stations or hitchhike on vehicles moving in the same direction. A heuristic-based m-shortest paths approach is proposed to allow drones to recharge at intermediate charging stations [14]. A decentralized framework based on Hedera is proposed for scheduling the charging of drones in a multi-drone network [6]. In this regard, a game-theoretic double auction model is employed to optimize the allocation of drones to the charging stations.

The common denominator in most of the aforementioned approaches is the assumption of instantaneous access to charging pads upon arrival [4,5,14,19]. This assumption is unrealistic, as charging stations are shared among multiple drones using the network. As a result, congestion may occur due to the simultaneous arrival of drones at the same node, leading to delays in the estimated delivery time. Furthermore, most of the existing works consider a simplified charging infrastructure model that serves either a single drone or a fixed number of drones at a time [6,9,11,14,18,20]. These assumptions may lead to underestimation or overestimation of delivery time, given the variations in the number of drones arriving at a node with varying charging needs.

Unmanned Traffic Management Systems (UTMs). The recent proliferation of drones operating in the urban airspace has led to the development of UTMs. The primary goal of UTMs is to ensure a conflict-free delivery route for drones. A *conflict* is defined as an event where two or more drones violate the predefined separation threshold, which may lead to collisions or near-misses. An optimization model is proposed to employ time-based delays to flights for ensuring safer separation between drones [22]. Another work detects conflicts based on the overlapping of 4D operational volumes of drone trajectories [23]. The proposed approach aims to resolve conflicts by introducing ground delays to drone delivery flights. A framework is proposed to detect and resolve spatio-temporal overlap between multiple drone trajectories operating in shared airspace [13]. The proposed approach uses the 2D coordinates along with the timestamps of UAV flight paths for conflict detection.

The common limitation is that the aforementioned UTMs are proposed for point-to-point drone deliveries restricted to a limited geographical area. However, future drone deliveries are expected to cover longer distances, which will require drones to recharge during their delivery flights. This may lead to *interference* between drone services at the charging stations (i.e., nodes) due to the spatio-temporal overlap in their recharging needs. *To the best of our knowledge, there is no previous work done on addressing interferences occurring at the charging stations. This work is, hence, the first attempt to resolve the inter-drone interference considering the varying charging needs of drone services.*

4 Preliminaries

We consider a skyway network S shared among multiple drone providers. The skyway network is modeled as an undirected graph $S = (V, E)$, where V is the set of nodes, and E is the set of skyway segments.

– Each node $v \in V$ represents a building rooftop equipped with charging infrastructure. A node is deployed with m charging pads, n waiting pads, and supports up to k drones in the designated hovering zone. We assume that the total pads $N = \left\lfloor \frac{A_u}{A_r} \right\rfloor$ depend on the usable portion of rooftop area A_u and the area per pad A_r, m and n are computed in terms of N as:

$$m = \lfloor \gamma \cdot N \rfloor, \quad n = N - m \tag{1}$$

where γ is the fixed allocation ratio.
– Each skyway segment $e \in E$ represents a delimited aerial corridor between two nodes. A segment is defined as a 3D volume with length L, width W, and height H to ensure collision-free operation of drones. The width and height of the segment are based on CASA (Civil Aviation Safety Authority) regulatory constraints, and the length L is determined by the Euclidean distance between nodes. A segment e is established only if there exists a Line-of-Sight (LoS) path between the nodes.

Definition 1: Skyway Path Request (SR)**.** Each drone provider submits their skyway path request to the super-provider before takeoff to comply with the safety requirements set by regulatory authorities. Each request is defined as a tuple of $\langle DP_{id}, Drone, NOD, SEG \rangle$, where

– DP_{id} is the unique drone provider identifier who owns the drone D_{id}.
– $Drone$ is defined as a tuple $\langle D_{id}, M, m, V \rangle$ where M is the mass of the drone D_{id} delivering payload of mass m, and V is the speed of drone.
– NOD represents the set of nodes that the drone visits for intermediate recharging. It is represented as a tuple of $\langle N_{\text{src}}, \ldots, N_{\text{dest}} \rangle$ where each $N_i \in NOD$ represents the charging time slot anticipated to be provisioned upon the drone's arrival at the node. Each N_i represents a time interval defined as $< t_{\text{start}}, t_{\text{end}} >$ where t_{start} represents the expected start time of charging, and t_{end} represents the expected end time.
– SEG represents the set of segments that D_{id} traverses to reach the destination node. SEG is defined as a tuple of $< Seg_s, \ldots, Seg_d >$ where each Seg_i is represented as a tuple of $< st_{\text{start}}, st_{\text{end}} >$ where st_{start} and st_{end} are the start and end times of segment traversal, respectively.

Definition 2: QoS Targets (QoS_T). Each skyway path request submitted by drone providers is associated with QoS targets that must be met to ensure the satisfaction of both the drone providers and service consumers. These QoS targets are represented by a tuple of $\langle E_{ex}, DT_{ex} \rangle$, where

– E_{ex} represents energy consumption anticipated to be consumed from the source to the destination node. We compute this as the cumulative sum of energy consumption E over k segments traversed by the drone using Eq. 2 [7].

$$E_{ex} = \sum_{i=1}^{k} E_i = \sum_{i=1}^{k} \left[\frac{D_i}{V_i} \left(\frac{(M+m)V_i}{\frac{3600}{g}(\eta r)} + pw \right) \right] \tag{2}$$

where g is the gravitational acceleration, η is the battery efficiency factor, r is the discharge rate, and pw accounts for power usage of electronics.

– DT_{ex} is the anticipated drone delivery time from the source node N_{src} to the destination node N_{dest}. It is the aggregate of the traversal time (st) across k number of segments and the time spent (nt) at l intermediate nodes for recharging. The anticipated delivery time is computed using Eq. 3:

$$DT_{ex} = \sum_{i=1}^{k} st_i + \sum_{j=1}^{l} nt_j \tag{3}$$

5 QoS Fulfillment Impact Model

In this section, we model the impact of inter-drone interferences on the predefined QoS targets, specifically in terms of anticipated delivery time and battery power consumption. As mentioned above, the satisfaction of drone providers is directly linked to how well a delivery service meets its QoS targets in a shared skyway network. These QoS targets may not be met due to inter-drone interference. We identify that these interferences may occur at the (1) segment-level and/or at the (2) node-level. In this paper, we assume that the impact of segment-level interference is negligible as drones maintain a longitudinal separation of 19000 mm based on the approach proposed in [16]. This approach facilitates interference-free operation of drones by preventing them from entering the region of turbulence of peer drones operating along the same skyway segment. Below, we model the impact of node-level interferences on the QoS fulfillment of drone services.

5.1 Impact of Node-Level Inter-drone Interference

A node-level interference occurs when the limited number of charging pads must accommodate access requests from multiple drones. We formally represent the state of a drone upon arrival at the node based on the availability of charging pads as follows:

$$Drone_{state}(t) = \begin{cases} \text{Recharging}, & \text{if } d_m < m \\ \text{Waiting}, & \text{if } d_m \geq m \text{ and } d_n < n \\ \text{Hovering}, & \text{if } d_m \geq m \text{ and } d_n \geq n \end{cases} \tag{4}$$

where d_m and d_n are the numbers of drones occupying the m charging pads and n waiting pads at time t. The state of a drone is considered to be dynamic and is subject to transition based on the availability of waiting and recharging pads at time t. We define the possible transitions in the drone's state as follows:

$$\begin{aligned} \text{Hovering} &\rightarrow \text{Waiting}, && \text{if } d_n < n \\ \text{Waiting} &\rightarrow \text{Recharging}, && \text{if } d_m < m \\ \text{Hovering} &\rightarrow \text{Recharging}, && \text{if } d_n = 0 \text{ and } d_m < m \end{aligned} \tag{5}$$

The charging need of a drone service is considered to be provisioned without delay if the drone directly enters the *recharging* state upon arrival at the node. However, if the drone enters the *waiting* or *hovering* state, it implies that the charging need may not be provisioned as anticipated. This is because forced *waiting* may lead to an additional delay as drones wait for their turn to occupy the next available charging pad. Similarly, forced hovering may lead to increased energy consumption as the drone remains airborne until the waiting or recharging pad becomes available. Furthermore, this may also extend the drone's recharging time as it regains the battery depleted during hovering. We model the impact of waiting or hovering states on the anticipated delivery time and energy consumption of drones as follows:

Impact on Energy Consumption. We compute the impact on anticipated energy consumption of the drone due to forced hovering at l number of intermediate nodes visited by the drone as follows:

$$E_{\text{ac}} = E_{ex} + \sum_{j=1}^{l} E^{j}_{hover} = E_{ex} + \sum_{j=1}^{l} \left(P^{j}_{hover} \times T^{j}_{hover}\right) \tag{6}$$

where P_{hover} is the drone's power consumption in the hovering zone for time duration T_{hover}.

Impact on Delivery Time. As aforementioned, the unavailability of charging pads may extend the drone's anticipated delivery time. The significance of this delay depends on the duration spent by the drone in *hovering* and *waiting* state before transitioning into the *recharging* state. We use Eq. 7 to compute the actual delivery time of the drone based on the cumulative impact of these delays.

$$DT_{\text{ac}} = DT_{ex} + \sum_{j=1}^{l} T^{j}_{hover} + \sum_{j=1}^{l} RT^{j}_{hover} + \sum_{j=1}^{l} T^{j}_{wait} \tag{7}$$

where T^{j}_{hover} and T^{j}_{wait} are the additional times spent in the *hovering* and *waiting* states at node j, respectively, before accessing the charging pad. We use the charging time model proposed in [16] to compute the time (i.e., RT_{hover}) required by the drone to regain the energy depleted in the hovering zone.

Impact on QoS Target. We quantify the effect of node-level interferences on the fulfillment of drone services' QoS targets. In this regard, we assess deviations from the anticipated delivery time and energy consumption using Eq. 8.

$$QoS_f(SR, QOS_T) = 1 - \left(\alpha \cdot \frac{|E_{\text{ac}} - E_{\text{ex}}|}{E_{\text{ex}}} + \beta \cdot \frac{|DT_{\text{ac}} - DT_{\text{ex}}|}{DT_{\text{ex}}}\right) \tag{8}$$

where α and β are the weights defined by the drone provider based on the sensitivity towards deviation in delivery time and energy consumption, respectively. For instance, in time-sensitive medical drone deliveries, the provider assigns a higher weight to β, meaning the fulfillment score is highly dependent on the

Algorithm 1. Node-level Interference Detection and Resolution

Require: Skyway Requests (SRs), Skyway Network (S), QoS Targets (QoS_T), t
Ensure: Updated QoS_f, Allocation Order (O, v)

Define t as a one-hour time window
Phase 1: Interference Detection
for each node v in S **do**
  Filter all drones requesting recharging at node v during time t
  for each drone d at node v **do**
    Assign initial state to d based on Eq. 4
    Update $DroneState \leftarrow DynamicTransition(d_m, d_n)$
    $T_{wait}, T_{hover} \leftarrow StateDuration(DroneState)$
    Compute RT_{hover} and E_{hover} using Eq. (6) and Eq. (7)
    $E_{ac} \leftarrow ComputeImpactOnEnergy(E_{ex}, E_{hover})$
    $DT_{ac} \leftarrow ComputeImpactOnDT(DT_{ex}, T_{hover}, RT_{hover}, T_{wait})$
    $FlaggedNodes \leftarrow CheckInterference(D, v)$
  end for
end for
Phase 2: Interference Resolution
for each node v in $FlaggedNodes$ **do**
  Generate all possible allocation orders $\mathcal{O}$ of drones at node v
  Evaluate congestion duration $CD(v)$ for each $\mathcal{O}$ using Eq. (10)
  Select $\mathcal{O}^*$ that minimizes $CD(v)$
  Define constraints $C1$ to $C5$ for allocation feasibility
  Update T^d_{start}, T^d_{end} for all d based on selected $\mathcal{O}^*$
  Compute updated QoS_f scores
end for
return $\{T^{d,v}_{start}, QoS^d_f \; \forall d \in D, v \in S\}$, Allocation Order (O, v)

expected delivery time. We categorize the provisioning of drone services' charging needs based on their QoS_f score as follows:

$$\text{ServiceLevel}(QoS_f) = \begin{cases} \text{Fully Serviced}, & \text{if } QoS_f \geq 0.8 \\ \text{Partially Serviced}, & \text{if } 0.5 \leq QoS_f < 0.8 \\ \text{UnServiced}, & \text{if } QoS_f < 0.5 \end{cases} \tag{9}$$

6 QoS-Aware Interference Resolution Framework

We present a node-level interference resolution framework to maximize the QoS fulfillment of drone services. In this regard, the framework aims to *effectively* and *efficiently* provision drone services' charging needs at the shared nodes. The proposed framework consists of two phases: (1) Interference detection, (2) Interference resolution. In what follows, we describe each phase in detail:

6.1 Interference Detection

In this phase, the proposed framework aims to *detect* and compute the *impact* of interference on the fulfillment of QoS targets of drone delivery services. Algo-

rithm 1 provides the general steps of the proposed interference detection approach (Lines 2–13). The input of this phase is the skyway path requests (SRs) and corresponding QoS targets (QoS_T) submitted by the drone providers during time slot t, i.e., one hour. The output is a set of node IDs where inter-drone interferences affect the provisioning of drone services' charging needs.

In the first step, each drone is assigned a state (i.e., recharging, waiting, or hovering) upon arrival at the node using Eq. 4 (Line 5). The $DynamicTransition()$ function dynamically updates the state of each drone based on the availability of charging and waiting pads using Eq. 5 (Line 6). In the next step, the $StateDuration()$ function logs the duration of the drone in waiting state (i.e., T_{wait}) and hovering state (i.e., T_{hover}) (Line 7). The algorithm then computes the impact on anticipated energy consumption and delivery time using Eq. 6 and Eq. 7, respectively (Lines 9–10). Lastly, we flag the nodes where drones experience waiting or hovering delays that lead to violations of QoS targets (Line 11).

6.2 QoS-Aware Interference Resolution

The input of this phase is the set of drone services from the previous phase. The goal of this phase is to maximize the provisioning of drone services' charging needs at the nodes. In this regard, the charging pads, waiting pads, and hovering zone are allocated to drones arriving at a node based on a *heuristic-driven* priority order. This priority order is derived using a heuristic that seeks to minimize the congestion duration at the node. The congestion duration is defined as the cumulative time during which the waiting pads and hovering zone remain occupied. Algorithm 1 provides the details of the proposed approach (Lines 14–22). We formalize the objective function to determine a heuristic-driven priority order that would minimize congestion for optimizing the QoS fulfillment of drone services as follows:

$$\min_{\mathcal{O}} \; CD(v) = \sum_{d \in \mathcal{O}} \left(T^d_{\text{recharge}} + T^d_{\text{wait}} + T^d_{\text{hover}} \right) \tag{10}$$

where $\mathcal{O}$ represents a possible allocation order of drones at node v. T^d_{recharge}, T^d_{wait}, and T^d_{hover} denote the time drone d spends in the recharging pad, waiting pad, and hovering zone, respectively. In the first step, the algorithm enumerates all possible allocation orders $\mathcal{O}$ of drones at the node and computes the corresponding congestion duration $CD(v)$ for each $\mathcal{O}$ (Lines 15–16). The algorithm then selects the optimal allocation order $\mathcal{O}^*$ that yields the minimum congestion duration (Line 17). The proposed heuristic-driven allocation order is subject to the following constraints (Line 18):

- **C1. Charging pad Assignment:** Each charging pad or waiting pad is assigned to only one drone at any given time.
- **C2. Arrival Constraint:** A drone d is not allocated any pad before its arrival time at the node, i.e., $t^{\text{arrival}}_d \leq t^{\text{start}}_d$.
- **C3. Bounded Recharging Duration:** A drone is allocated the recharging pad for the time duration required to complete its recharging need, i.e., $t^{\text{end}}_d - t^{\text{start}}_d = T^d_{\text{recharge}}$

Table 1. Experimental Variables

Parameter	Specification
Drone model	DJI Phantom 3
Drone mass (M)	1216 g
Payload range (m)	500-1100 g
Motor/propeller power transfer efficiency (η)	0.5
Power consumption of electronics (pw)	0.1 kW
Lift-to-drag ratio (r)	3
Allocation ratio (γ)	0.5

- **C4. Access Constraint:** The objective of minimizing congestion at a node may greedily prioritize drones with shorter charging times, pushing drones with longer charging times to the end of the queue. This constraint introduces the urgency score $w_i^{\tau} + \alpha r_i$, where w_i is the waiting time and r_i is the recharge need of drone i. We set $\tau = 1.2$ to moderately amplify waiting time, and $\alpha = 1$ to ensure that drones with long waiting and high recharge demands are also promoted, thereby balancing efficiency and fairness.
- **C5. Capacity Constraint:** The number of drones assigned to the recharging and waiting queues must not exceed the m number of charging and n number of waiting pads at any time t i.e., $\sum_d x_d^{\text{recharge}}(t) \leq m$ and $\sum_d x_d^{\text{wait}}(t) \leq n$.

The selected allocation order $\mathcal{O}^*$ is then used to determine the updated start and end times for each drone's recharging session (Line 19). We then assess the QoS_f score of each drone based on the updated recharging time slots using the model proposed in Sect. 5 (Line 20).

7 Experimental Results and Discussion

7.1 Experimental Setup

Skyway Network Dataset: We use real-world dataset crawled from the OpenStreetMap[1] for the Sydney CBD area. The dataset contains attributes such as building type (e.g., restaurant or accommodation), GPS coordinates, building heights, and rooftop areas. We use this dataset to construct a skyway network of 59 nodes and 1210 LoS-based skyway segments using the algorithm proposed in [8]. Each node is equipped with multiple recharging pads[2] and waiting pads. The number of these pads is determined using Eq. 1, based on an average usable rooftop area of 6.4% as reported in [10].

Drone Delivery Dataset: We specify varying numbers of drone delivery flights with varying payloads and flight ranges executed by multiple drone providers. These flights are based on a real-world baseline scenario where a single provider

[1] https://www.openstreetmap.org/#map=4/-28.15/133.28.
[2] https://www.heishatech.com/for-developers/dpad-c500-drone-charging-pad/.

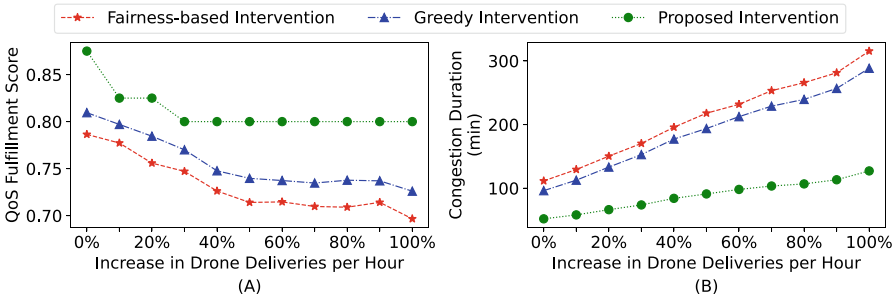


Fig. 2. (A) Average QoS_f Score (B) Average Congestion Duration Across Network

conducts 40 deliveries in an hour [3]. We consider a percentage growth of 10% to 100% over this baseline to evaluate the performance of the network under varying drone traffic loads. Each drone delivery path is computed based on the shortest path algorithm. We compute the charging time of the drone at the intermediate node using the model proposed in [16]. We compute the expected energy consumption and delivery time for each drone delivery path using Eq. 2 and 3, respectively. The rest of the experimental variables are listed in Table 1.

Experimental Methods: We evaluate the performance of our proposed approach against two baseline approaches: (1) *Fairness-based Intervention Approach*, where drones are allocated to recharging pads on First-Come-First-Served (FCFS) policy [24]; (2) *Greedy Intervention Approach*, where drones are assigned to recharging pads based on the shortest recharging time. This approach minimizes immediate congestion for the given time period without look-ahead.

Experimental Metrics: As mentioned earlier, the QoS fulfillment in terms of delivery time and energy consumption is paramount for the satisfaction of drone providers. We evaluate the *effectiveness* of our approach in resolving interferences by computing the *average* QoS_f score of drone delivery services using the network. We compute the QoS_f score of individual drone delivery service using Eq. 8. In addition, we report our proposed approach's *efficiency* in terms of *execution time*. This represents the total computational time the algorithm takes to resolve interference across all N drone delivery services using the network. We report the results of these metrics for varying loads of drone traffic within a shared skyway network. This allows us to evaluate the scalability and robustness of our approach against other approaches under increasing loads of drone traffic in a skyway network. All code, datasets, and instructions required to reproduce the results are available at [15].

7.2 Experimental Results

Effectiveness Results: Figure 2(A) shows effectiveness results in terms of average QoS fulfillment score. We observe that the proposed approach has a significantly higher QoS_f score in comparison to the other two approaches. This is

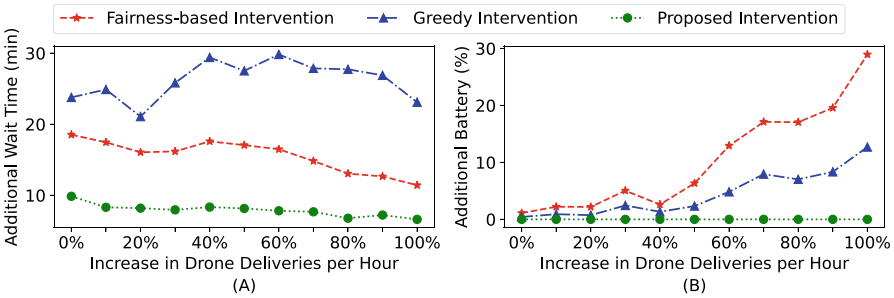


Fig. 3. (A) Average Delay in Delivery Time (B) Average Increase in Battery Usage

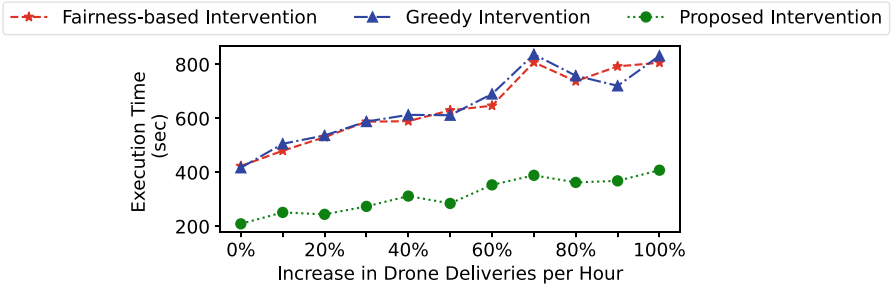


Fig. 4. Execution Times Comparison

because the proposed approach provisions the charging needs of drones arriving at a node based on a heuristic-driven priority order defined in Eq. 10. This priority order ensures that the total congestion duration at a node remains minimal. As a result, drones are serviced with reduced waiting and hovering time. As shown in Fig. 2(B), the average congestion duration of nodes remains ≈63% lower than the fairness-based intervention and ≈57% lower than the greedy intervention approach in comparison to the proposed intervention strategy. We also observe that the *greedy intervention* achieves a slightly lower congestion duration compared to the fairness-based approach. This is because the *fairness-based* approach serves drones based on FCFS policy, assigning equal priority to all drones regardless of their charging needs. In contrast, the greedy intervention approach provisions drones based on the least recharging need, which increases the turnaround time of charging pads and slightly reduces congestion compared to FCFS. The proposed approach provisions the charging needs of drones based on a heuristic-driven priority order. This may lead to drones *waiting* or *hovering* upon arrival at the node as they wait for their turn to access the charging pad. As shown in Fig. 3(A) and 3(B), this leads to a negligible delay of ≈10 min on average in delivery time and nearly 0% additional battery usage on average, respectively, in comparison to the other two approaches. We also observe

in Fig. 2(A), the QoS_f score decreases as the traffic load increases for both the fairness-based and greedy intervention strategies. This is because a higher number of drones compete to access the shared charging infrastructure. As a result, congestion may occur as multiple drones are forced to wait or hover for their turn to access the charging pads. This leads to delivery delays and increased battery consumption than initially anticipated by the providers, resulting in unmet QoS targets. However, with the proposed intervention strategy, the QoS_f score remains stable and higher ≈0.8 even under increasing traffic loads (see Fig. 2(A)). This demonstrates that the proposed approach effectively limits waiting times and prevents excessive battery drain due to congestion in comparison to other approaches.

Efficiency Results: Figure 4 shows the efficiency results in terms of execution time. We observe that the execution time of both the greedy intervention approach and the fairness-based intervention approach is significantly higher compared to the proposed approach. In contrast, the proposed approach reduces the computational time by ≈50% on average. This efficiency is due to the proposed set of heuristics that saves computational time by effectively provisioning the charging needs of drones. The results reveal that the proposed approach outperforms both the fairness-based and greedy intervention approaches in terms of *efficiency* and *effectiveness*.

8 Conclusion

We propose a novel framework to maximize QoS fulfillment for drone services in a shared skyway network. The framework employs a QoS-aware approach for provisioning the charging needs of drone services based on a heuristic-driven priority order. The proposed heuristics aim to minimize the network-wide congestion by effectively allocating drones to the shared charging infrastructure. Experimental results demonstrate that the approach maintains stable QoS fulfillment scores under increasing drone traffic loads while reducing congestion duration by approximately ≈60% and computational overhead by ≈50%. In future work, we will investigate redirection strategies for drone services during periods of peak congestion.

Acknowledgments. This research was partly made possible by DP220101823 and LE220100078 grants from the Australian Research Council. The statements made herein are solely the responsibility of the authors.

Disclosure of Interests. The authors have no competing interests.

References

1. Chiang, W.C., et al.: Impact of drone delivery on sustainability and cost: realizing the uav potential through vehicle routing optimization. Appl. Energy **242**, 1164–1175 (2019)

2. Cokyasar, T., et al.: Designing a drone delivery network with automated battery swapping machines. Comput. Operat. Res. **129**, 105177 (2021)
3. DroneUp: Droneup achieves 500 deliveries per day (2023). https://www.droneup.com/news/droneup-achieves-500-deliveries-per-day
4. Fan, M., et al.: Deep reinforcement learning for uav routing in the presence of multiple charging stations. IEEE Trans. Veh. Technol. **72**(5), 5732–5746 (2022)
5. Hamdi, A., et al.: Drone-as-a-service composition under uncertainty. IEEE Trans. Serv. Comput. **15**(5), 2685–2698 (2021)
6. Hassija, V., Saxena, V., Chamola, V.: Scheduling drone charging for multi-drone network based on consensus time-stamp and game theory. Comput. Commun. **149**, 51–61 (2020)
7. Kornatowski, P.M., et al.: Downside up: rethinking parcel position for aerial delivery. IEEE Robot. Autom. Lett. **5**(3), 4297–4304 (2020)
8. Lee, W., et al.: Package delivery using autonomous drones in skyways. In: Adjunct Proceedings of the 2021 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2021 ACM International Symposium on Wearable Computers, pp. 48–50 (2021)
9. Liu, B., et al.: Novel integrated framework of unmanned aerial vehicle and road traffic for energy-efficient delay-sensitive delivery. IEEE Trans. Intell. Transp. Syst. **23**(8), 10692–10707 (2021)
10. Melius, J., Margolis, R., Ong, S.: Estimating rooftop suitability for pv: a review of methods, patents, and validation techniques (2013)
11. Mezni, H., et al.: Daas composition: enhancing uav delivery services via lstm-based resource prediction and flight patterns mining. Computing **107**(3), 1–52 (2025)
12. Mohsan, S.A.H., et al.: Unmanned aerial vehicles (uavs): practical aspects, applications, open challenges, security issues, and future trends. Intel. Serv. Robot. **16**(1), 109–137 (2023)
13. Papadopoulos, G., et al.: Deep reinforcement learning in service of air traffic controllers to resolve tactical conflicts. Expert Syst. Appli. **236** (2024)
14. Pinto, R., Lagorio, A.: Point-to-point drone-based delivery network design with intermediate charging stations. Trans. Res. Part C: Emerging Technol. **135**, 103506 (2022)
15. Rizvi, S.A., Bouguettaya, A.: Drone-services-qos-optimization. GitHub (2025). https://github.com/AmnaRizvi-S/Drone-Services-QoS-Optimization.git
16. Rizvi, S.A., et al.: Monitoring inter-drone service interference for resilient operations. IEEE Trans. Serv. Comput. **18**(3), 1573–1587 (2025). https://doi.org/10.1109/TSC.2025.3568245
17. Shahzaad, B., et al.: Composing drone-as-a-service (daas) for delivery. In: 2019 IEEE International Conference on Web Services (ICWS), pp. 28–32. IEEE (2019)
18. Shahzaad, B., et al.: Resilient composition of drone services for delivery. Futur. Gener. Comput. Syst. **115**, 335–350 (2021)
19. Shahzaad, B., et al.: Robust composition of drone delivery services under uncertainty. In: 2021 IEEE International Conference on Web Services (ICWS), pp. 675–680. IEEE (2021)
20. Shahzaad, B., Bouguettaya, A.: Top-k dynamic service composition in skyway networks. In: Hacid, H., Kao, O., Mecella, M., Moha, N., Paik, H. (eds.) ICSOC 2021. LNCS, vol. 13121, pp. 479–495. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-91431-8_30
21. Srithar, V., et al.: Impact of spatial proximity on drone services. In: Companion of the 2024 on ACM International Joint Conference on Pervasive and Ubiquitous Computing, pp. 235–238 (2024)

22. Tang, Y., Xu, Y.: Incorporating optimization in strategic conflict resolution for UAS traffic management. IEEE Trans. Intell. Transp. Syst. **24**(11), 12393–12405 (2023)
23. Tang, Y., et al.: Conflict probability based strategic conflict resolution for UAS traffic management. In: 2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC), pp. 1–10. IEEE (2023)
24. Tsai, T.Y., Chung, Y.L., Tsai, Z.: Introduction to packet scheduling algorithms for communication networks. Commun. Netw., 264–287 (2010)

DIJS: A Dual Interference-Aware Job Scheduling Framework for Co-located Data Centers

Qin Hua[1], Shiyou Qian[1(✉)], Yufeng Deng[2], Kaixuan Zhang[1], Dingyu Yang[3], Jian Cao[1], and Guangtao Xue[1]

[1] Shanghai Jiao Tong University, Shanghai, China
{huaqin,qshiyou,zks1anx,cao-jian,gt_xue}@sjtu.edu.cn
[2] Ant Group, Shanghai, China
dengyufeng.dyf@antgroup.com
[3] State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China
yangdingyu@zju.edu.cn

Abstract. To enhance resource utilization in data centers, it is common to co-locate latency-sensitive services and batch jobs on the same host, which leads to batch-to-online (B2O) and intra-batch (B-B) interference due to resource contention. Therefore, it is crucial to schedule jobs effectively to alleviate both types of interference. While most prior studies focus on B2O interference, B-B interference is often overlooked. By analyzing production data, we observe three insights that reveal the relationships between job behaviors and the two types of interference. We propose **DIJS**, a **d**ual **i**nterference-aware **j**ob **s**cheduling framework. DIJS predicts the resource usage patterns and resource sensitivity of new batch jobs. Based on this, it designs two quantizers using reinforcement learning (RL) and attention mechanisms to assess the impact of B2O and B-B interference. Experiments show that DIJS reduces service tail latency by about 41.0% and enhances job efficiency by up to 28.6%.

Keywords: service co-location · data center · interference · QoS · RL

1 Introduction

Cloud-native applications are widely deployed in data centers [1,5]. These applications are typically organized as loosely-coupled **online services**. To enhance resource utilization, co-location mechanisms [2,3] have been proposed. These mechanisms run resource-intensive **batch jobs** on hosts with pre-orchestrated services to make use of idle resources. Batch jobs usually process large volumes of data with relaxed latency restrictions in a best-effort manner.

Although co-location successfully improves resource utilization, it could exacerbate the resource contention among services and jobs for CPU, memory, network, etc. [2,4]. For example, resource contention can arise on LLC, or memory

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 138–148, 2026.
https://doi.org/10.1007/978-981-95-5015-9_10

bandwidth, particularly during peak periods, often leading to a decline in service quality and job efficiency, known as interference [6,8]. We outline four types of interference in co-location situations: 1) **Intra-Online (O-O)**, 2) **Batch-to-Online (B2O)**, 3) **Intra-Batch (B-B)** and 4) **Online-to-Batch (O2B)**.

Reducing interference is crucial to enhance the overall performance of co-located applications. To ensure availability, services are often carefully orchestrated before co-location to avoid O-O interference [1]. Since QoS guarantee is essential for services, it is inevitable that job performance may be compromised by O2B interference. Thus, this study does not focus on O-O and O2B inference. Although prior studies strove to address B2O interference [3,7], job efficiency can be greatly reduced by severe B-B interference as resource-intensive jobs are frequently submitted [8]. This is usually ignored in current studies.

Mitigating B2O and B-B interference presents two challenges. First, the diverse resource characteristics of numerous job types impede proactive, interference-aware scheduling. Second, assessing the impact of B2O and B-B interference caused by new jobs is challenging. The co-location of various jobs with services complicates B2O interference quantification, while complex resource contention interactions among jobs make analyzing B-B interference difficult.

To overcome these issues, we analyze data centers with over 10,000 hosts to identify usage patterns and resource sensitivity in batch jobs. We develop DIJS, a proactive interference-aware job scheduling framework. First, we employ Gaussian mixture models (GMM) and causal inference models to predict resource usage patterns and resource sensitivity for new jobs. Second, we utilize RL models to tackle the B2O interference scoring based on the QoS performance impact caused by new jobs. Third, we design a self-attention mechanism to capture complex interactive B-B interference among jobs and quantify its impact. Ultimately, DIJS can schedule jobs to mitigate both B2O and B-B interference.

The contributions of this work are summarized as follows:

- We propose a machine learning-based predictive resource profiler that accurately forecasts the resource characteristics of new jobs before scheduling.
- We establish two interference quantizers utilizing reinforcement learning and self-attention mechanisms to assess the impact of B2O and B-B interference.
- The experimental results show that DIJS reduces the tail latency of services and the average makespan of jobs by up to 41.0% and 28.6%, respectively.

2 Related Work

Service Resource Allocation. Some studies optimize resource allocation for services to minimize O-O interference. PARTIES [4] introduces a priority-based resource allocation mechanism that dynamically assesses service importance and balances their performance and fairness in co-located environments. Similarly, FIRM [6] utilizes a RL model to dynamically adjust resources, thereby reducing

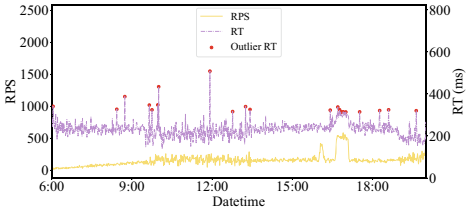


Fig. 1. The distribution of service interference.

O-O interference to ensure service QoS. Meanwhile, Rusty [7] offers an LSTM-based framework to accurately predict performance for co-running applications, allowing proactive interference mitigation.

Job Scheduling. Interference-aware job scheduling effectively addresses job placement to minimize resource interference. Bubble-Up [2] uses bubble scores and sensitivity curves to assess performance interference among services. Paragon [3] identifies similarities between current and historical services, facilitating optimal scheduling in heterogeneous environments. Moreover, PISM [8] is a classification-based job scheduler designed to reduce B2O interference in production clusters.

Our study, unlike previous ones focusing on B2O interferences, also addresses B-B interference to enhance resource utilization in co-located data centers.

3 Analysis and Observations

This study examines the behavior and performance patterns of services and jobs co-located in a cluster with over 10,000 hosts. Each host is equipped with Intel Xeon Platinum 826X CPUs and has a memory capacity of 512 GB.

Observation 1: Different factors influence the various patterns of QoS violations observed in services

Figure 1 illustrates the relationship between request per second (RPS) and response time (RT) for a representative service. In the RT curve, red data points represent outliers with RTs surpassing the day's P95 RT values, termed as "violation points." These points usually belong to two categories. (1) **Concentrated violations** clustered within a short timeframe, typically corresponds to peak workload. In Fig. 1, more than 5 violations are found in 10 min around 5:00 PM. Resource capacity of services should be auto-scaled to address these issues [6]. (2) **Scattered violations** occur sporadically during low loads. These outliers may stem from interference, particularly from resource-intensive jobs. In Fig. 1, scattered violations represent 71.4% of the observed ones. Effective job scheduling to reduce B2O interference is critical.

Observation 2: The resource usage for tasks of the same type typically follows mixed normal distributions

noindent Resource usage patterns vary significantly at the task level in production data centers like Google [5] and Azure [10]. Tasks of the same type typically perform the same logical steps on different data sets. Thus, analyzing these patterns can be achieved by examining various task types. Figure 2 demonstrates that the CPU usage of 8 *Map* tasks in SQL jobs collectively follows a distribution composed of 3 normal distributions.

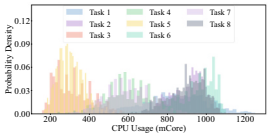


Fig. 2. Distribution of CPU usage among eight Map tasks.

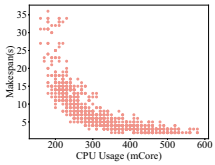


Fig. 3. Relationship between instance CPU usage and makespan.

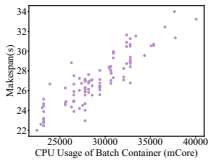


Fig. 4. Batch container CPU usage and makespan of task instances.

Observation 3: As competition for shared resources increases, task efficiency declines significantly

noindent Our analysis reveals that B-B interference impacts task efficiency regarding makespan in two ways. First, task efficiency may drop when resources are scarce, particularly if the dominant resources for tasks are preempted [11]. Figure 3 displays the inverse relationship between CPU usage and makespan for 1,000 instances of a *Map* task. Second, as the resource use of batch containers increases, task interference from B-B escalates. Figure 4 shows the link between the task's average makespan and the average CPU usage of batch containers.

4 System Design

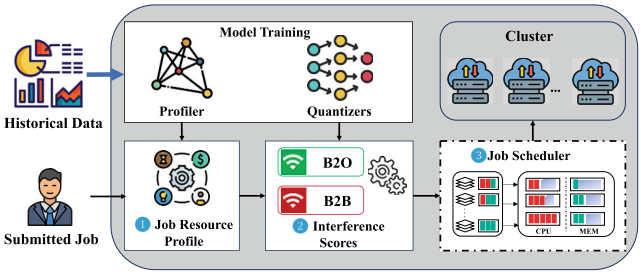


Fig. 5. The structure of DIJS.

4.1 Overview

We introduce DIJS, a dual interference-aware job scheduling framework comprising two main modules, as shown in Fig. 5. First, we establish a predictive profiler to estimate the resource usage of new tasks and identify their sensitive resource type. Second, we propose two quantizers, informed by the profiling results, to assess the impact of B2O and B-B interference from new tasks using the RL model and self-attention mechanism, respectively. Any scheduler can seamlessly integrate with DIJS to identify the optimal hosts for new tasks, effectively reducing overall interference weighted by B2O and B-B factors.

4.2 Job Resource Profiler

Our profiler uses Gaussian mixture models (GMMs) and causal inference to characterize tasks in terms of resource usage and sensitivity. We focus on resources CPU and memory, and predict task makespan to determine their active periods.

Modeling Task Resource Usage Based on Observation 2, the profiler employs a GMM [12] to capture the distributions of resource usage among task instances of each type. For each task type, we utilize historical instance data to train a specific GMM model for each considered metric, which classifies new instances into multiple groups. Each instance i linked to a particular task τ is represented by a vector $\boldsymbol{v}_i$, defined as: $\boldsymbol{v}_i = (st_\tau, ecu_\tau, emu_\tau, N_i^F, B_i, x_i)$

Here, st_τ denotes the time of day when τ is submitted, and ecu_τ and emu_τ represent the average CPU and memory usage anticipated by the task owner. N_i^F and B_i denote the number and byte size of data files processed by instance i, while x_i signifies the actual value of the target metric achieved by instance i. Essentially, DIJS trains GMM models to uncover complex distributions in target metric values based on the metadata from task instances.

We employ the Expectation Maximization (EM) method [13] to iteratively train the GMM parameters, including mean and variance, until the model converges to the optimal parameters. Upon receiving a new task τ, the trained GMM model utilizes its metadata, specifically $(st_\tau, ecu_\tau, emu_\tau, N_\tau^F, B_\tau)$, to determine the sub-distribution that the target metrics of instances in τ follow. Using the fitted distributions, the features of resource usage of τ are estimated as follows:

$$(\hat{\mu}_{cu}, \hat{\mu}_{mu}, \hat{\mu}_{ms}, \hat{\sigma}^2_{cu}, \hat{\sigma}^2_{mu}, \hat{\sigma}^2_{ms}) \tag{1}$$

where $\hat{\mu}_{cu}$, $\hat{\mu}_{mu}$ and $\hat{\mu}_{ms}$ represent the estimated mean value of CPU usage, memory usage, and makespan of instances within τ, respectively. Similarly, $\hat{\sigma}^2_{cu}$, $\hat{\sigma}^2_{mu}$ and $\hat{\sigma}^2_{ms}$ denote the estimated variance of these three metrics, respectively.

Analyzing Task Resource Sensitivity According to Observation 3, DIJS effectively estimates a task's sensitivity to CPU and memory resources.

Causal inference based on the average treatment effect (ATE) [14] is widely used to identify casual links from independent variables to dependent ones. A

confirmed link indicates that changes in independent variables, known as treatments, indeed influence dependent variables in a specific direction to a certain extent. Utilizing ATE values, we quantify resource sensitivity for each task as the differences in impact under heavy and light contention for specific resources.

To tackle this sensitivity profiling issue, we firstly define treatment events. For each task type and each resource type, let $\tilde{X}_i$ and x_i denote the average resource usage of the entire batch container and the individual instance i within the container, respectively (e.g., CPU usage for CPU sensitivity estimation). The performance metric, denoted by y_i, is the makespan of instance i. Ensuring the data partition balance, we use the 60th percentile (P60) of $\tilde{X}_i$ as the threshold ϕ to identify events. Then, we define the metric $treat_i$ for instance i as follows:

$$treat_i = \begin{cases} 1 & \tilde{X}_i > \phi \\ 0 & \text{Otherwise} \end{cases}$$

where $treat_i = 1$ indicates the occurrence of a treatment event. In the same way, we characterize instance i by a joint vector $d_i = (\tilde{X}_i, x_i, y_i, treat_i)$.

Then, we utilize each dataset $\mathbb{D} = \{d_i\}$ to train individual model that quantifies ATE caused by resource contention. Given any specific value x for metric x_i, denoted as $x_i = x$, the normalized ATE value is defined as:

$$ATE = \frac{E\left[\{y_i | x_i = x, treat_i = 1\}\right] - E\left[\{y_j | x_j = x, treat_j = 0\}\right]}{E\left[\{y_j | x_j = x, treat_j = 0\}\right]} \tag{2}$$

where i and j denote two distinct instances, and $E(\cdot)$ refers to the expectation function. DIJS employs the XGBoost model, denoted as XGB, trained by $\mathbb{D}$ to estimate the normalized ATE using the resource usage of specific tasks as input. After analyzing the normalized ATEs from various datasets, we determine the P85 of ATE results as the thresholds ρ_{cpu} and ρ_{mem} to identify tasks with the top 15% ATE effects as ones sensitive to CPU and memory contention. Given the resource usage pattern of a new task τ defined in Eq. (1), we assign it a resource sensitivity label, denoted as rs_τ, based on the following criteria:

$$rs_\tau = \begin{cases} 0 & XGB(\hat{\mu}_{cu}) \leq \rho_{cpu} \ \& \ XGB(\hat{\mu}_{mu}) \leq \rho_{mem} \\ 1 & XGB(\hat{\mu}_{cu}) > \rho_{cpu} \ \& \ XGB(\hat{\mu}_{mu}) \leq \rho_{mem} \\ 2 & XGB(\hat{\mu}_{cu}) \leq \rho_{cpu} \ \& \ XGB(\hat{\mu}_{mu}) > \rho_{mem} \\ 3 & XGB(\hat{\mu}_{cu}) > \rho_{cpu} \ \& \ XGB(\hat{\mu}_{mu}) > \rho_{mem} \end{cases}$$

where labels 0, 1, 2 and 3 represent the task that is: 1) not sensitive on CPU nor memory, 2) sensitive on CPU but not on memory, 3) sensitive on memory but not on CPU, and 4) sensitive on both CPU and memory, respectively.

4.3 Two Interference Quantizers

RL-Based B2O Interference Quantizer By considering host conditions and QoS improvement as state space and rewards for specific scheduling actions of new tasks, we effectively frame the B2O interference scoring issue as a

value-based RL problem [15]. DIJS establishes a neural network to forecast the expected reward from actions, referred to as Q-value. The model is trained via a group of state-action-reward iterations. In each iteration j, a_j denotes the action of assigning a specific task instance to a designated host, while s_j represents the state space, encompassing various elements such as host CPU and memory usage and the RT status of placed services RT_j. Upon executing action a_j, a new state s_{j+1} is established. The resource usage in state s_{j+1} is calculated as the sum of that in s_j and the estimated average resource usage of the new instance predicted by the resource profiler. As for RT_j, our study defines the overall QoS performance of a host as the average RT across all services deployed on the host at state s_j.

We pre-train an efficient multi-layer perceptron (MLP) [12] for each host offline to predict RT_{j+1} online, utilizing the host's state s_{j+1}. Based on the difference between RT_{j+1} and RT_j, the reward r_j is computed as Eq. (3):

$$r_j = \begin{cases} 1 - e^{-\alpha_1 \cdot (1 - \frac{RT_{j+1}}{RT_j})}, & RT_{j+1} \leq \rho_{rt} \\ 1 + \alpha_2 \cdot (1 - \frac{RT_{j+1}}{\rho_{rt}}) - e^{-\alpha_1 \cdot (1 - \frac{RT_{j+1}}{RT_j})}, & RT_{j+1} > \rho_{rt} \end{cases} \tag{3}$$

where ρ_{rt} denotes the threshold of RT, set as the P95 of collected values during nearby time windows. A penalty is imposed on r_j if RT_{j+1} exceeds ρ_{rt}, which emphasizes the long tail issue caused by B2O interference. Following prior work [15], we set $\alpha_1 = 0.5$ and $\alpha_2 = 2$. On this basis, Q-value is estimated to optimize a policy that maximizes the expected rewards related to QoS by scheduling tasks.

Learning Procedure of Q-Value Model. The procedure of quantifying B2O interference consists of two parts: offline training and real-time scoring for a new task instance on each candidate host. Initially, the model is trained using historical data. According to work of DQN [15], we initialize a current network Q with parameters θ and a duplicate known as the target network $Q^{'}$ with parameters $\theta^{'} = \theta$. In each training iteration j, the current network Q generates predicted Q-values, $\hat{q}_j = Q(s_j, a_j, \theta)$, for the current action a_j, while the target network $Q^{'}$ predicts the future reward, $Q'(s_{j+1}, a_{j+1}, \theta^{'})$, which are combined with the obtained reward r_j to estimate the temporal difference (TD) target q_j^{td} as:

$$q_j^{td} = r_j + \gamma Q'(s_{j+1}, a_{j+1}, \theta^{'}) \tag{4}$$

where γ is the discount factor within the range of $[0, 1]$, indicating the weight of future rewards. After each iteration, Q is continuously updated via gradient descent to minimize the loss between $\hat{q}_j$ and q_j^{td}. As for $Q^{'}$ the update of $\theta^{'}$ in is delayed compared to that of θ to reduce TD target fluctuations and occurs every c iterations, contributing to the stabilization of the learning process.

B2O Interference Scoring. After training the current network Q, we use it to score the B2O interference caused by a new task τ on each candidate host. Specifically, for each instance of τ, let $H = \{h_1, \cdots, h_k\}$ denote the candidate

host set, possessing current state s_k of each host $h_k \in H$. The action a is defined as assigning the instance with resource profiling results to a specific host. For any host h_k, we score the B2O interference impact of τ as:

$$Score_k^{b2o} = 1 - Norm(Q(s_k, a, \theta)) \tag{5}$$

where Q-value is normalized into the range of $[0, 1]$. A lower $Score_k^{b2o}$ suggests that scheduling an instance of τ on host h_k results in lighter B2O interference.

B-B Interference Quantizer Based on Attention Mechanism The quantizer includes three components: 1) positional embedding layers encoding types of co-running tasks, 2) a self-attention encoder with masking, and 3) MLP layers for output generation. The performance of a batch container is measured by cycles per instruction (CPI) [3], with higher CPI indicating lower efficiency. For each host at a given time, we construct a vector for each task type that captures CPU usage, memory usage, and instance count. Combining these vectors forms a matrix representing the hostâĂŹs resource usage state.

The model centers on masked self-attention encoder layers, using masking to reduce B-B interference caused by co-running tasks. The attention encoder captures complex resource contention by evaluating the importance of resource status across different task types for each task. The resulting representation is passed to MLP layers to predict the CPI value of the batch container.

A lower CPI indicates less interference within the batch container, so we use predicted CPI to assess B-B interference. When a new task τ arrives at time t, for each candidate host $h_k \in H$ with a CPI value cpi_k^{bef}, we utilize the Transformer model to forecast the CPI value after deploying an instance of τ on h_k, denoted as $\hat{cpi}_k^{aft}$, with the resource usage of the instance estimated by the resource profiler. Then, the B-B interference score of each h_k is defined as:

$$Score_k^{b-b} = Norm\left(\frac{\hat{cpi}_k^{aft} - cpi_k^{bef}}{cpi_k^{bef}}\right) \tag{6}$$

4.4 Batch Job Scheduler

Scheduling a new task τ is divided into sub-schedulings of instances. For each instance, the scheduler determines a candidate host set H based on a specific strategy. Then, DIJS evaluates B2O and B-B interference scores as Eq. (5) and Eq. (6), for each host $h_k \in H$. The weighted interference score is calculated as:

$$Score_k = \beta * Score_k^{b2o} + (1 - \beta) \cdot Score_k^{b-b} \tag{7}$$

where β balances the impact of B2O and B-B interference scores. Each instance of τ is assigned to the host with the lowest weighted interference score.

5 Evaluation

5.1 Setup

Testbed We conduct two sets of experiments to evaluate DIJS. First, we conduct simulations in a simulated cluster including 206 hosts, each with Intel Xeon Platinum 826X (826X) CPUs and 512GB of memory. 1,726 services are orchestrated in this cluster and over 10,000 task instances of 600 types of jobs are submitted for one day. Second, we conduct experiments in a real cluster with 8 hosts, each with 104 logical cores of 826X CPU, and 196GB of memory. The experiment employs a Hotel Reservation benchmark [17] comprising 10 microservices, and submits jobs of SQL processing and machine learning using Spark-Bench.

Baselines We compare DIJS with 7 baselines: 1) Rusty [7], a predictive resource allocator using LSTM; 2) Paragon [3] and 3) Bubble-up [2], which address pressure between services and the entire host to reduce B2O interference; 4) Decima [18], which employs RL-based job scheduler to mitigate B-B interference; 5) Dominant Resource Fairness [11] (DRF); 6) Min-Connection [19] (MC), two scheduling policies aiming at load balancing; and 7) Real Trace, results from our data centers. Moreover, we adjust the weight of the B2O score β in Eq. (7).

Table 1. Service QoS performance.

Method	Mean	P90	P95	P99
Real Trace	45.10	79.08	118.18	339.14
Bubble-up	45.89	82.28	108.32	268.88
Paragon	44.28	75.64	103.45	301.76
Rusty	44.56	78.04	104.92	308.34
Decima	45.09	78.94	104.95	251.84
MC	45.82	80.17	110.16	248.04
DRF	44.36	81.69	109.63	240.40
DIJS (ours, $\beta = 0.8$)	**40.63**	**73.84**	**95.23**	**141.70**
DIJS (ours, $\beta = 0.7$)	41.36	77.42	104.28	190.12
DIJS (ours, $\beta = 0.6$)	43.34	75.44	103.97	195.13

Table 2. Task makespan (ms).

Method	Mean	Max	Variance
Real Trace	66	16,201	63,459
Bubble-up	64	14,427	61,632
Paragon	69	15,856	65,269
Rusty	92	18,769	95,503
Decima	68	14,299	65,390
MC	67	16,508	64,039
DRF	63	14,122	58,831
DIJS (ours, β= 0.8)	56	15,886	55,969
DIJS (ours, β= 0.7)	48	14,219	40,119
DIJS (ours, β = 0.6)	**45**	**13,047**	**34,238**

5.2 Simulations in a Large-Scale Data Center

QoS Performance on Services Table 1 shows the results of the 8 frameworks in terms of RT. These frameworks exhibit similar effectiveness in sustaining stable QoS performance across the majority of hosts, as evidenced by the mean RT values. However, a notable long tail phenomenon for services is observed in real production, where the P95 and P99 RT values are about 2.6× and 8.9× of the mean value. DIJS achieves the state-of-the-art performance in addressing the long tail issue. Paragon and DRF are the best baselines in terms of the P95 and

P99 *RT* values, respectively. DIJS reduces these two values by about 7.9% and 41.0% compared to the two baselines respectively.

Overall Performance on Jobs Table 2 shows the mean and maximum makespan of task instances, achieved by the 8 scheduling frameworks. DIJS demonstrates superior performance in terms of the average job efficiency. Compared to the best baseline DRF, when $\beta = 0.8$, DIJS achieves a reduction of around 11.1% in the mean makespan. Moreover, when we set $\beta = 0.6$, DIJS further reduces the mean and maximum makespan by about 28.6% and 7.6%, respectively compared to DRF. In addition, DIJS excels in maintaining efficiency stability across task instances, as evidenced by that DIJS achieves the lowest variance of makespan.

5.3 Real Experiments in a Small-Scale Cluster

We compare DIJS with 2 baselines: 1) Real Trace, the load-balancing scheduling strategy in our production data centers, and 2) MC [19]. As shown in Table 3, DIJS outperforms the 2 baselines on the performance of both services and jobs. For services, compared to Real Trace, DIJS reduces the mean and P99 *RT* by about 8.4% and 8.1% respectively. As for jobs, although we set the B2O weight β to 0.8 to prioritize service performance, DIJS still reduces the mean and P99 makespan by about 1.8% and 7.8% respectively.

Table 3. Overall performance in the real cluster.

Framework	Online Services				Batch Jobs		
	RT (ms)				Makespan (s)		
	Mean	P50	P95	P99	Mean	P95	P99
Real Trace	6978.13	6705.15	9707.52	11091.97	72.09	141.03	153.82
MC	7136.23	6810.13	9912.12	11543.74	72.51	142.45	153.27
DIJS (ours)	**6386.93**	**6488.06**	**9388.03**	**10182.66**	**70.81**	**138.29**	**141.62**

6 Conclusion

In this paper, we introduce DIJS, a framework for quantifying B2O and B-B interference for new jobs. These interference scores allow DIJS to optimize job scheduling by addressing both interference types simultaneously. Future research will explore the complex relationship between job behaviors and individual service performance, focusing on key job types to enhance interference mitigation.

References

1. Gan, Y., and et al.: Seer: leveraging big data to navigate the complexity of cloud debugging. In: Proc. of the 10th USENIX HotCloud, pp. 1–13 (2018)
2. Mars, J., et al.: Bubble-Up: increasing utilization in modern warehouse scale computers via sensible co-locations. In: Proc. of MICRO, pp. 248–259 (2011)
3. Delimitrou, C., Kozyrakis, C.: Paragon: QoS-aware scheduling for heterogeneous datacenters. In Proc. of ASPLOS, pp. 77–88 (2013)
4. Chen, S., Delimitrou, C., Martínez, J.F. : PARTIES: QoS-aware resource partitioning for multiple interactive services. In: Proc. of ASPLOS, pp. 107–120 (2019)
5. Tirmazi, M., et al.: Borg: The next Generation. In: Proc. of EuroSys, pp.1–14 (2020)
6. Qiu, H., et al.: FIRM: an intelligent fine-grained resource management framework for SLO-oriented microservices. In: Proc. of OSDI, pp. 805–825 (2020)
7. Masouros, D., Xydis, S., Soudris, D.: Rusty: runtime interference-aware predictive monitoring for modern multi-tenant systems. IEEE Trans. Parallel Distributed Syst. **32**(1), 184–198 (2021)
8. Yang, D., et al.: Mitigating interference of microservices with a scoring mechanism in large-scale clusters. J. Supercomput. **81**(1), 1-31 (2025)
9. Luo, S., et al.: Characterizing microservice dependency and performance: alibaba trace analysis. In: Proc. of ACM SoCC, pp. 412–426 (2021)
10. Hadary, O., et al.: Protean: VM allocation service at scale. In: 14th USENIX Symposium on Operating Systems Design and Implementation, pp. 845–861 (2020)
11. Ghodsi, A., et al.: Dominant resource fairness: fair allocation of multiple resource types. In: Proc of 8th USENIX NSDI, pp. 1–14 (2011)
12. Bishop, C.M.: Pattern Recognition and Machine Learning. Springer (2006)
13. Xuan, G., Zhang, W., Chai, P.: EM algorithms of Gaussian mixture model and hidden Markov model. In: Proc. of IEEE ICIP, pp. 145–148 (2001)
14. Pearl, J.: Causality: Models, Reasoning, and Inference. Cambridge University Press (2009)
15. Mnih, V., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
16. Vaswani, A., et al.: Attention is all you need. In: Proc. of NeurIPS, pp. 5998–6008 (2017)
17. Gan, Y., et al.: An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: Proc. of ASPLOS, pp. 3–18 (2019)
18. Mao, H., et al.: Learning scheduling algorithms for data processing clusters. In: Proc. of SIGCOMM, pp. 270–288 (2019)
19. Zhang, M., et al.: A load-balancing algorithm for power internet of things resources based on the improved weighted minimum number of connections. In: Proc. of ICAIS, pp. 563–574 (2021)

QONNECT: A QoS-Aware Orchestration System for Distributed Kubernetes Clusters

Haci Ismail Aslan(✉), Syed Muhammad Mahmudul Haque, Joel Witzke, and Odej Kao

Technische Universität Berlin, Berlin, Germany
{aslan,joel.witzke,odej.kao}@tu-berlin.de, haque@campus.tu-berlin.de

Abstract. Modern applications increasingly span cloud, fog, and edge environments, requiring orchestration systems that adapt to diverse contexts while meeting Quality-of-Service (QoS) goals. Standard Kubernetes schedulers do not account for user-defined objectives such as energy efficiency, cost optimization, and performance, leaving operators to make manual placement decisions. We present QONNECT, a vendor-agnostic orchestration framework that enables declarative, QoS-driven deployment across heterogeneous Kubernetes and K3s clusters. Through a minimal YAML interface, users specify high-level QoS goals, which the system translates into placement and migration actions. Our prototype, evaluated on a federated testbed of nine clusters with the Istio Bookinfo application, demonstrates dynamic microservice placement, automated failover, and QoS-compliant rescheduling without manual intervention. By bridging declarative deployment with operational QoS, QONNECT advances the cloudâĂŞedge continuum toward a unified, self-optimizing platform.

Keywords: Cloud-fog-edge continuum · Orchestration · Self-optimizing clusters · QoS-aware deployment · Microservice placement and migration

1 Introduction

The cloud–edge continuum has emerged to meet diverse latency, privacy, and availability demands of distributed applications [8]. While cloud platforms provide elasticity, scalability, and cost efficiency, their centralized nature cannot satisfy latency-sensitive or intermittently connected workloads [1]. Fog computing brings resources closer to data sources but faces challenges in energy efficiency, security, and fault tolerance [11], whereas edge computing offers real-time responsiveness but is constrained in capacity, reliability, and exposure to security risks [6]. No single paradigm dominates; instead, hybrid approaches coordinating centralized and decentralized resources achieve better flexibility and performance [3].

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 149–156, 2026.
https://doi.org/10.1007/978-981-95-5015-9_11

Cloud-native orchestration tools like Kubernetes (K8s) enable scalable management in clouds [4], and lightweight variants (e.g., k3s, KubeEdge) extend orchestration toward the edge [15]. However, these tools still struggle with distributed, multi-layered applications under diverse QoS requirements [10]. A robust orchestrator must meet seven criteria [13]: (1) unified cross-layer deployment, (2) technology agnosticism, (3) multi-cloud support, (4) dynamic runtime optimization, (5) extensible policy definitions, (6) container and VM compatibility, and (7) production-grade robustness and security. Kubernetes addresses parts of this via autoscaling and extensibility but lacks native QoS-aware scheduling and cross-cluster optimization. Declarative tools such as Helm and Kustomize simplify deployment, yet service placement by latency, cost, or energy remains manual. The default scheduler ignores QoS objectives, leaving administrators to hardcode node assignments and update CI/CD pipelines during deployment or recovery [5].

To address these gaps, we propose QONNECT, an orchestration framework that automates QoS-aware deployment of microservices across cloud, fog, and edge layers. Inspired by Swarmchestrate [7,14] and guided by practical constraints, QONNECT reduces manual intervention while fulfilling orchestrator requirements. Our contributions are:

- **Vendor-agnostic architecture.** Separation of global state (Knowledge Base), distributed decision-making (Raft-replicated RLAs), and local actuation (RAs) keeps the framework independent of provider APIs.
- **QoS-aware scheduler.** A weighted Borda count translates declarative vectors for energy, cost, and performance into placements, requiring minimal YAML additions; the ranking component is replaceable.
- **Fault-tolerant control plane.** RLAs form a Raft quorum, elect new leaders automatically, and cooperate with RAs for transparent workload migration.

2 Background

2.1 Kubernetes and Lightweight Distributions

Kubernetes is the standard platform for orchestrating containerized applications, offering autoscaling, service discovery, and declarative management. For constrained environments, lightweight variants such as *k3s* preserve core primitives while reducing resource footprints, enabling orchestration on fog and edge devices.

2.2 Consensus and Orchestration

Cross-cluster orchestration requires consistent global state and durable decision persistence. Raft [9] provides a practical consensus mechanism to replicate state via the Replicated State Machine (RSM) [2] and coordinate lead agents, ensuring fault-tolerant scheduling and reliable execution across multiple clusters.

2.3 QoS-Aware Scheduling and Borda Count

Traditional Kubernetes schedulers focus on resource availability and node affinity but lack support for QoS-based policies. We adopt a weighted Borda scheme, ranking nodes by attributes (e.g., energy, cost, performance), normalizing scores, and combining them with user-defined weights. This lightweight method enables interpretable, QoS-aware scheduling with acceptable complexity for typical cluster sizes.

3 Architecture

QONNECT is a distributed orchestration framework for QoS-aware deployment across heterogeneous cloud, fog, and edge clusters. Users express high-level QoS goals via declarative policies, while the system autonomously manages placement, migration, and failover. It supports the following functional (FR) and non-functional (NFR) key requirements:

- **Domain Deployment (FR1):** Deploy components to user-specified domains (cloud, fog, edge).
- **QoS-Driven Placement (FR2):** Place and adapt workloads according to energy, cost, or performance goals.
- **Recovery (FR3):** Detect and recover from failures through monitoring and automatic migration.
- **Fault Tolerance (NFR):** Ensure continuous operation by eliminating single points of failure.

3.1 Concept

As shown in Fig. 1, the QONNECT concept is structured around three core components: the *Knowledge Base (KB)*, the *Resource Lead Agent (RLA)*, and the *Resource Agent (RA)*.

Each domain operates its own cluster(s): cloud clusters typically run full Kubernetes, while fog and edge rely on k3s or lightweight alternatives. Worker nodes host applications and are monitored by local RAs, while at least one cloud cluster hosts an RLA for system-wide coordination.

The *Knowledge Base* is a global metadata store for cluster state, resources, workloads, and user QoS requirements. Though logically centralized, it may be backed by any resilient distributed storage (e.g., replicated SQL/NoSQL or blockchain), ensuring consistency and fault tolerance.

Resource Lead Agents handle cross-cluster coordination, scheduling, and consensus. RLAs form a Raft quorum for leader election; the elected leader serves as the orchestration API, evaluating QoS goals (e.g., latency, cost, energy) and selecting suitable clusters for deployment.

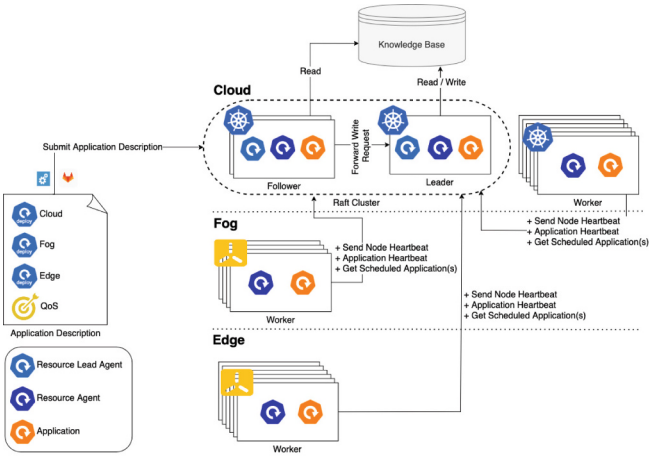


Fig. 1. Architecture overview: component interaction across cloud, fog, and edge domains.

Resource Agents run in every application-hosting cluster. They translate RLA instructions into Kubernetes manifests, apply them locally, and report telemetry (node health, application status, resource use) to the RLA. Stateless and lightweight, RAs ensure fault-tolerant execution and can trigger redeployments when conditions change.

Together, the KB and RLAs provide centralized decision-making, while RAs enforce decisions locally. This hybrid model enables scalable, QoS-aware orchestration across heterogeneous cloud, fog, and edge domains with resilience.

3.2 Implementation

QONNECT implementation follows the principles of *Clean Architecture*, which enforce separation of concerns between domain logic, use cases, and interfaces. This fosters modularity, testability, and portability. We provide an open-source prototype[1] of QONNECT, demonstrating the feasibility of our design. The implementation centers on: a Raft-based control plane, the QoS-aware scheduler, a lightweight REST interface, the Knowledge Base, the Resource Agent (RA), and the Resource Lead Agent (RLA).

Raft. We forked `etcd/contrib/raftexample`, retaining its consensus layer (state machine, WAL, snapshot) and replacing its store/transport with a minimalist HTTP-based REST transport.

Scheduler. The leader RLA periodically (i) places pending services using the Borda scorer and (ii) re-queues stalled ones. The scorer filters ineligible nodes, ranks them by energy, cost, and capacity, aggregates scores by user-defined

[1] https://github.com/dos-group/QONNECT; last accessed: November 8, 2025.

weights, discards below-average nodes, and returns the top-ranked cluster with eligible nodes. This process is simple, interpretable, and ensures QoS-aware placements.

REST API. Portability across domains relies on (i) Ingress naming conventions and (ii) cross-domain placeholders for IP addresses, which RAs resolve locally before manifest application. Applications are described in a thin CRD-style YAML carrying metadata such as name, labels, and QoS targets.

Resource Agent Each RA, built on Clean Architecture principles, orchestrates workloads within its cluster:

- **Registration.** On startup, the RA either loads its ConfigMap (cluster ID, role) or registers with a bootstrap RLA, caching the assigned UUID.
- **Telemetry.** Periodically collects node snapshots and application states, reporting them to the RLA.
- **Configuration sync.** Retrieves cluster ingress mappings and scheduled workloads via REST, updating local ConfigMaps.
- **Deployment.** Instantiates workloads by applying manifests with resolved IPs and scheduler-specified node assignments; updates the applications ConfigMap as ground truth.
- **Heartbeat.** Reports application status as healthy, progressing, or failed; if the RLA responds with withdrawal, the RA cleans up all objects and ConfigMap entries.

RAs are stateless, lightweight, and reactive, ensuring fault tolerance and minimal overhead while bridging global orchestration logic with local Kubernetes execution.

4 Evaluation

4.1 Setup

We evaluated QONNECT using a Kind[2] testbed of nine clusters (three each for cloud, fog, and edge), with each domain containing energy-efficient, cost-efficient, or performance-efficient profiles. Each cluster has one control-plane and two worker nodes. As clusters ran in Docker on a single machine, real telemetry was unavailable; representative energy, cost, and bandwidth values were manually assigned based on public data.

Energy consumption values were derived from Cloud Carbon Footprint coefficients, yielding $E_{\min} \approx 0.0024042$ kWh, $E_{\text{mid}} \approx 0.0025689$ kWh, and $E_{\max} \approx 0.0027335$ kWh for energy, cost, and performance-efficient profiles, respectively. Costs (EUR/h) followed AWS On-Demand pricing, assigned as $C_{\min} = 0.0042$, $C_{\text{mid}} \approx 16.3884$, and $C_{\max} = 32.7726$ EUR/h for cost, energy, and performance-efficient clusters. Bandwidth values were set to 5 Gbps, 52.5

[2] https://kind.sigs.k8s.io; last-accessed: November 8, 2025.

Gbps, and 100 Gbps for cost, energy, and performance-efficient profiles, respectively, following AWS instance specs. The same values were applied to fog and edge clusters for consistency. To simulate energy and cost-efficient characteristics, we reduced worker node resources (5 GiB memory, 4 CPU cores, 2 GiB ephemeral storage) via `kubeadmConfigPatches`.

Test Application. For system evaluation, we adapted the Istio Bookinfo microservices application[3], a multi-service e-commerce system, with minimal configuration modifications and added a Kubernetes Ingress. The Bookinfo application comprises `productpage`, `details`, `reviews`, and `ratings` services. To simulate a cloudâĂŞfogâĂŞedge architecture, we deployed `productpage` on the cloud cluster, `details` and `reviews` on the fog cluster, and `ratings` on the edge cluster. Our deployment retained the overall structure but redistributed services to align with our experimental setup.

4.2 Results

To test the systems requirements, we conducted a series of tests using the developed QONNECT prototype:

1. Deploy the test application, prioritizing performance. Here, QONNECT should schedule the application's components to the 'performance' clusters in each domain.
2. After initial deployment, we change the applications QoS requirement to prioritize energy efficiency instead. The components of the application should be redeployed to the corresponding 'energy' clusters.
3. After initial deployment, we simulate a cluster failing by deleting the RA within the 'performance' cluster of the edge domain. QONNECT should react accordingly be redeploying the ratings component to a different cluster within the edge domain.
4. Test the resilience against lead agent failure by deleting the RLA currently acting as raft leader. In our case that was the RLA of the 'energy' cluster.

Scripts conducting these tests can be found in the same repository together with our QONNECT prototype[4]. The expected output of these tests is also shown in the corresponding artifact paper submitted along with this paper.

5 Related Work

Several orchestration frameworks have emerged to address deployment and QoS challenges across the cloudâĂŞedge continuum. Taherizadeh et al. [12] propose a capillary computing model that migrates Docker-based microservices across

[3] https://istio.io/latest/docs/examples/bookinfo/noistio.svg; last-accessed: November 8, 2025.

[4] https://github.com/dos-group/QONNECT/blob/main/resource-lead-agent/bruno-collection/swarmchestrate.json; last-accessed: November 8, 2025.

Edge, Fog, and Cloud tiers to maintain proximity to mobile clients and balance load. Their vehicle-mounted prototype demonstrates significant improvements in response time and latency variation. MiCADO-Edge [13] orchestrates containers and VMs across heterogeneous infrastructures using TOSCA descriptors, supporting dynamic policy enforcement and multi-platform deployments in domains such as healthcare and video analytics. ACOA [10] introduces a distributed, per-application multi-scheduler architecture built on Kubernetes, enabling QoS-aware placement through customizable scheduling algorithms and consideration of latency, reliability, and resource availability. Nautilus [5] addresses dynamic runtime conditions through RL-based resource management, microservice migration, and latency-aware mapping of service graphs, achieving both QoS guarantees and resource efficiency. Complementing these system-specific efforts, VaÃśo et al. [15] survey the evolution of cloud-native orchestration at the edge, identifying trends in lightweight Kubernetes distributions, container runtimes, and emerging technologies such as WebAssembly-based virtualization.

6 Conclusion and Future Work

This work introduced QONNECT, a QoS-driven orchestrator for heterogeneous Kubernetes and K3s clusters spanning cloud to edge environments. The system enables intent-based deployment by translating high-level QoS vectors into concrete placement and migration decisions, using a modular scheduler architecture and a fault-tolerant Raft-based control plane. Through a federated nine-cluster testbed, QONNECT demonstrated its ability to maintain service-level objectives across dynamic conditions, automatically recovering from node and agent failures while preserving quorum and consistency. By showing reliable, policy-driven orchestration under dynamic conditions, this work takes a step toward invisible infrastructureâĂŤwhere developers express what they need, and the system decides where and how to run it. Future work will focus on scaling to real-world deployments and exploring richer QoS dimensions such as latency and energy efficiency.

Acknowledgments. This paper was partially supported by the Swarmchestrate project of the European Union's Horizon 2023 Research and Innovation programme under grant agreement no. 101135012.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Apostu, A., Puican, F., Ularu, G., Suciu, G., Todoran, G.: Study on advantages and disadvantages of cloud computing - the advantages of telemetry applications in the cloud. In: Applied computer science, pp. 118–123. WSEAS Press, Greece (2013)

2. Beyer, B., Jones, C., Petoff, J., Murphy, N.R.: Site reliability engineering: how Google runs production systems. O'Reilly Media, Inc. (2016)
3. Brogi, A., Forti, S., Ibrahim, A.: How to best deploy your fog applications, probably. In: 2017 IEEE 1st International Conference on Fog and Edge Computing (ICFEC), pp. 105–114 (2017). https://doi.org/10.1109/ICFEC.2017.8
4. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J.: Borg, omega, and kubernetes. Commun. ACM **59**(5), 50–57 (2016). https://doi.org/10.1145/2890784
5. Fu, K., Zhang, W., Chen, Q., Zeng, D., Guo, M.: Adaptive resource efficient microservice deployment in cloud-edge continuum. IEEE Trans. Parallel Distrib. Syst. **33**(8), 1825–1840 (2022). https://doi.org/10.1109/TPDS.2021.3128037
6. Jedidi, A.: Dynamic trust security approach for edge computing-based mobile iot devices using artificial intelligence. Eng. Res. Express **6**(2), 025211 (2024). https://doi.org/10.1088/2631-8695/ad43b5
7. Kiss, T., et al.: Swarmchestrate: Towards a fully decentralised framework for orchestrating applications in the cloud-to-edge continuum. In: Barolli, L. (ed.) Advanced Information Networking and Applications, pp. 89–100. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-57931-8_9
8. Moreschini, S., Pecorelli, F., Li, X., Naz, S., Hästbacka, D., Taibi, D.: Cloud continuum: the definition. IEEE Access **10**, 131876–131886 (2022). https://doi.org/10.1109/ACCESS.2022.3229185
9. Ongaro, D., Ousterhout, J.: In search of an understandable consensus algorithm. In: 2014 USENIX Annual Technical Conference (USENIX ATC 2014), pp. 305–319 (2014)
10. Orive, A., Agirre, A., Truong, H.L., Sarachaga, I., Marcos, M.: Quality of service aware orchestration for cloud–edge continuum applications. Sensors **22**(5) (2022). https://doi.org/10.3390/s22051755
11. Prakash, P., Darshaun, K., Yaazhlene, P., Ganesh, M.V., Vasudha, B.: Fog computing: issues, challenges and future directions. Inter. J. Electr. Comput. Eng. **7**(6), 3669 (2017)
12. Taherizadeh, S., Stankovski, V., Grobelnik, M.: A capillary computing architecture for dynamic internet of things: Orchestration of microservices from edge devices to fog and cloud providers. Sensors **18**(9) (2018). https://doi.org/10.3390/s18092938
13. Ullah, A., Dagdeviren, H., Ariyattu, R.C., DesLauriers, J., Kiss, T., Bowden, J.: Micado-edge: towards an application-level orchestrator for the cloud-to-edge computing continuum. J. Grid Comput. **19**(4), 47 (2021). https://doi.org/10.1007/s10723-021-09589-5
14. Ullah, A., et al.: Towards a decentralised application-centric orchestration framework in the cloud-edge continuum. In: 2025 IEEE 9th International Conference on Fog and Edge Computing (ICFEC), pp. 37–41 (2025). https://doi.org/10.1109/ICFEC65699.2025.00014
15. Vaño, R., Lacalle, I., Sowiński, P., S-Julián, R., Palau, C.E.: Cloud-native workload orchestration at the edge: a deployment review and future directions. Sensors **23**(4) (2023). https://doi.org/10.3390/s23042215

Privacy, Security and Trust

Towards Trusted Service Monitoring: Verifiable Service Level Agreements

Fernando Castillo[1(✉)], Eduardo Brito[2,3(✉)], Sebastian Werner[1(✉)], Pille Pullonen-Raudvere[2(✉)], and Jonathan Heiss[1(✉)]

[1] Information Systems Engineering, Technische Universität Berlin, Berlin, Germany
{fc,sw,jh}@ise.tu-berlin.de
[2] Cybernetica AS, Tallin, Estonia
pille.pullonen-raudvere@cyber.ee
[3] University of Tartu, Tartu, Estonia
eduardo.brito@cyber.ee

Abstract. Service Level Agreement (SLA) monitoring in service-oriented environments suffers from inherent trust conflicts when providers self-report metrics, creating incentives to underreport violations. We introduce a framework for generating verifiable SLA violation claims through trusted hardware monitors and zero-knowledge proofs, establishing cryptographic foundations for genuine trustworthiness in service ecosystems. Our approach starts with machine-readable SLA clauses converted into verifiable predicates and monitored within Trusted Execution Environments. These monitors collect timestamped telemetry, organize measurements into Merkle trees, and produce signed attestations. Zero-knowledge proofs aggregate Service-Level Indicators to evaluate compliance, generating cryptographic proofs verifiable by stakeholders, arbitrators, or insurers in disputes, without accessing underlying data. This ensures three security properties: integrity, authenticity, and validity. Our prototype demonstrates linear scaling up to over 1 million events per hour for measurements with near constant-time proof generation and verification for single violation claims, enabling trustless SLA enforcement through cryptographic guarantees for automated compliance verification in service monitoring.

1 Introduction

Service Level Agreements (SLAs) are widely used to codify non-functional guarantees in cloud and edge systems. Yet, verifying whether these guarantees are upheld remains problematic. SLA monitoring is often under the control of the provider or a third party [12], creating trust asymmetries and limiting transparency [9], especially in adversarial settings where providers may benefit from under-reporting, consumers may seek unjustified compensation, or competitors may launch reputation attacks [19].

This lack of verifiability also undermines broader accountability infrastructures in service ecosystems. In particular, cyber insurance, now a key instrument for managing cyber risk [22], faces significant challenges due to unverifiable

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 159–174, 2026.
https://doi.org/10.1007/978-981-95-5015-9_12

claims of compliance, e.g., malfunctioning of a service or non-compliance of an SLA caused by a data breach. Studies report recurring disputes, delayed reimbursements, and difficulties in underwriting premiums, often due to self-reported or incomplete SLA evidence [2,30]. Despite forecasts estimating the global cyber insurance market to exceed $28 billion by 2026 [30], insurers lack cryptographic assurance over SLA fulfillment, complicating risk pooling and claims processing by needing to spend resources on verifying the authenticity, integrity and validity of the evidence used to generate claims, e.g., unrealized gains due to service unavailability on high spike selling days.

To address this gap, we propose a framework for verifiable SLA compliance that leverages trusted hardware monitors and zero-knowledge proofs. Our approach enables the generation of machine-verifiable, privacy-preserving and tamper-evident SLA claims. This removes undesirable dependencies on third-party SLA monitor providers and enhances automated dispute resolution, addressing emerging requirements in cyber insurance. Our contributions are:

- A trusted monitoring architecture and model that collects Service Level Indicator (SLI) data via active and passive probes inside Trusted Execution Environments (TEEs);
- A method for converting SLA predicates into verifiable Zero Knowledge Virtual Machine (zkVM) programs with publicly auditable semantics;
- A prototype implementation and evaluation showing scalability to over 1M events per hour with near constant-time proof verification of single violation.

The remainder of the paper is organized as follows. Section 2 reviews the background and related work in SLA monitoring, trust models, and verifiable computations. Section 3 defines our monitoring model, its trust assumptions and security properties. Section 4 presents the architecture and verifiability syntax. Section 5 describes our implementation, and Sect. 6 evaluates its performance. Section 7 reflects on implications and future directions. Section 8 concludes.

2 Background and Related Work

This section introduces TEEs for secure SLI measurement and ZKPs for verifiable SLO evaluation, as enablers of integrity, authenticity, and validity. These properties will be formally defined later. It then reviews related work and the state of the art in SLA monitoring. This aligns with emerging paradigms like DevSecOps, VeriDevOps, or TrustOps [5,24], which advocate continuous, evidence-based trust assessment across cloud infrastructures and stakeholders, now being extended to SLA compliance during service delivery.

Trusted Execution Environments. TEEs provide hardware-backed mechanisms to isolate computation and protect integrity in untrusted environments [7,20]. Modern TEEs operate at different levels: Intel TDX and AMD SEV-SNP provide VM-level isolation with encrypted memory, while Intel SGX offers process-level enclaves. TPMs complement these with persistent integrity

guarantees via Platform Configuration Registers (PCRs) [7,20]. Remote attestation enables cryptographic verification of TEE state through reports containing: (i) binary measurements (e.g., MRTD, MRENCLAVE), (i) runtime configuration (e.g., RTMR/PCR values), (iii) ephemeral public keys, and (iv) manufacturer signatures. During execution, TEEs produce supplementary reports with dynamic measurements, enabling continuous integrity verification. They are deployed in the context of cloud platforms [26], trusted compute units [6], confidential containers [4], and trustworthy pre-processing of IoT data [11]. However, integrating them into verifiable SLA monitoring contexts remains an open challenge.

Zero-Knowledge Proofs. ZKPs enable cryptographic verification of statements without revealing underlying data. Zero-knowledge scalable transparent arguments of knowledge (zkSTARKs), a type of ZKP, provide post-quantum secure, transparent proofs, to ensure computational integrity in outsourced computations [29]. A particular application of zkSTARKs are zkVMs, like RISC0 and SP1, which compile programs, for example, to RISC-V bytecode, and generate non-interactive proofs binding: (i) bytecode hash, (ii) public inputs, and (iii) outputs [17]. They provide self-contained cryptographic artifacts that can be verified by any stakeholder without rerunning the computation or accessing raw data [8]. However, proof generation incurs significant overhead (orders of magnitude slower than native execution [11]), prompting optimistic approaches where proofs are generated only for violations, disputes, or audits. This makes cryptographic verification practical in SLA monitoring, where continuous proofs are costly but strong guarantees remain essential.

SLA Monitoring. Service Level Agreements (SLAs) define cloud providers' commitments to performance (e.g., availability, latency) and security (e.g., data protection) [12], based on measurable SLIs (e.g., request latency) and SLO thresholds (e.g., 95% of requests under 300 ms) [21]. SLA monitoring aggregates low-level metrics into compliance checks [23], promoting trust through transparency and predefined escalation paths. In practice, monitoring is typically provider-controlled [27], creating conflicts of interest. Third-party trust delegation [25] can reduce bias but depends on available and impartial arbiters [35]. Probabilistic detection [31] offers another route, though still limited. These shortcomings are critical in domains like cyber insurance, where unverifiable SLA violations impede underwriting, compliance validation, and claims processing [2,30]. Trusted compute units [6] offer verifiable computation, but target composition correctness rather than SLA evaluation. Economic incentives further complicate trust: modern workflow schedulers often maximize profits by tolerating SLA penalties [28,33]. Efforts like rSLA [18], SLA4OAI [10], and blockchain-based models [32] attempt to address these challenges, but issues remain around data provenance [14], measurement integrity [16], and audit-related privacy loss [1].

3 Trustworthy SLA Monitoring Model

To set the scene, we introduce a monitoring model that builds upon established approaches to SLA monitoring including the long standing WSLA [15] and more recent surveys [21]. This model, depicted in Fig. 1, allows us to further refine the problem of trustworthy SLA violation determination.

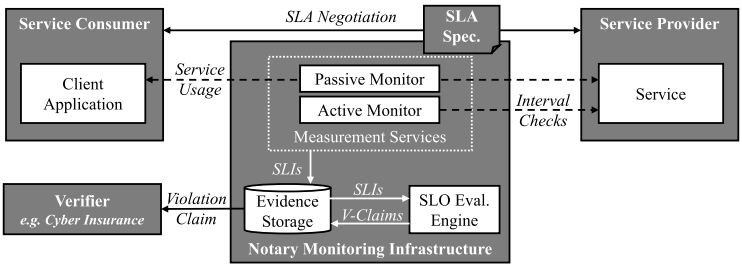


Fig. 1. SLA Monitoring Model based on [15] with Passive and Active Monitor.

3.1 Roles, Components, and Stages

The *Service Provider* delivers *cloud services* under the promise of infrastructure reliability and SLA compliance. The *Service Consumer* utilizes these cloud resources, applications, or functionality and expects the agreed performance standards. Provider and consumer determine their SLA in an *SLA Specification* that defines *Service Level Objectives* (SLOs). *Monitors* (or Measurement Services [15]), either passive or active systems, capture *Service Level Indicator* (SLI) measurements which are persisted in an *Evidence Storage*. An *SLO Evaluation Engine* (SEE) aggregates SLIs into *Violation Claims* to check and disclose the violation of SLOs. To ensure trustworthiness of SLIs and violation claims these components can be deployed on an independent *Monitoring Infrastructured* controlled by a trusted third party (TTP) acting as *Notary*. Further components described in WSLA [15], e.g., SLA Establishment, Deployment, or Management Services, are neglected and not in the scope of this paper. These roles and components interact in different stages throughout the SLA lifecycle:

1. *Negotiation*: SLA is negotiated and signed by the contractual parties.
2. *Deployment*: SLA is validated and distributed to the involved components.
3. *Monitoring*: Monitors collect SLIs, SEEs compute SLA claims.
4. *Evaluation*: SLA claims are validated against SLOs to decide SLA violations.
5. *Termination*: SLA is terminated based on agreed on conditions.

These stages directly derive from [15], except for *four* which we repurposed to explicitly include SLA evaluation thereby neglecting corrective measures.

3.2 Problem Refinement

The trustworthiness of the SLIs and SLA claims are fundamental to guarantee SLA compliant service provisioning and prevent costly disputes between service consumer and provider. In the model presented, consumer and provider fully rely on the trustworthiness of the Notary's independent monitoring infrastructure, expecting monitors, evidence storage, and the SEE to be uncompromisable and functioning correctly. However, trusted third parties like the Notary are often not available in practice or their contractual terms introduces undesirable dependencies and costs. Alternative deployments of such components at the consumer or provider sides, however, lead to information asymmetry that can be exploited by each party. Given an economic relationship, all parties, including the Notary may manipulate the SLIs and/or SLA claims to maximize their utilities. Malicious providers may try to conceal SLA violations, consumers might falsely claim violations, and even the Notary may collude with one of the parties, or sloppily neglect security and safety measures of its infrastructure, allowing for external attacks or failures. These situations may cause unacceptable financial losses. To prevent this, the trustworthiness of the SLIs and SLA claims must be guaranteed in the service ecosystem. We consider this to be achieved if all of the following properties of the involved components are met:

1. *Integrity*: The monitors and SEE collect SLIs and compute SLA claims correctly ("doing the things right").
2. *Authenticity*: The collected SLIs and SLA claims are unmodified and originate from the expected monitors and SEE.
3. *Validity*: The monitors and SEE are running according to their specifications ("doing the right thing").

These properties form design objectives which collectively ensure that evidence of SLA compliance or violation is trustworthy and verifiable by all parties.

4 System Design

This section first provides an overview of our verifiable SLA monitoring system design, then presents the key components in more detail.

4.1 System Overview

We address previous design objectives by a) leveraging TEEs for secure monitoring, b) a blockchain-enabled storage system for secure evidence persistence, and c) ZKPs for verifiable SLA claim validation. The TEE's hardware-secured isolation prevents manipulation attempts from the monitoring infrastructure provider resulting in trustworthy SLI measurements. The ZK-enabled *SLO Evaluation Engine* (SEE) guarantees that the SLIs are correctly checked against the SLOs as determined in the agreed-on SLA. A decentralized storage system realizable through blockchains and content-addressable storage systems, like IPFS, ensures

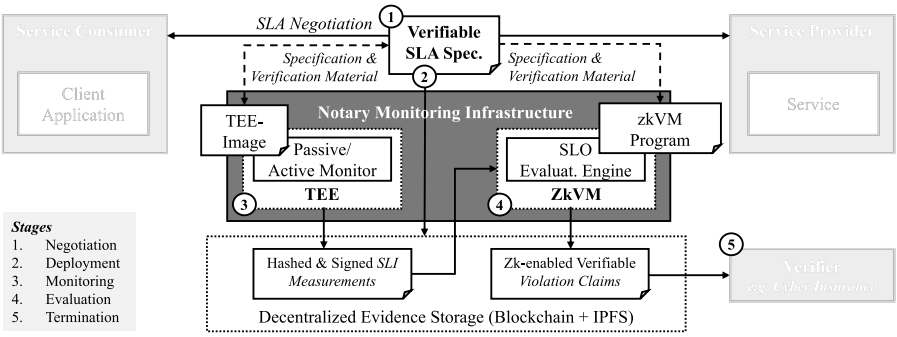


Fig. 2. SLI Measurements flow into verifiable SLA compliance claims.

the tamper-resistance and accessibility of the SLI measurements and violation claims. We describe the interplay and integration of these technologies following the SLA lifecycle stages presented in previous section, as shown in Fig. 2.

1. Negotiation: Consumer, Provider, and Notary negotiate a *Verifiable SLA Specification* (VSLAS) that extends existing standards through the specification of (1) TEE-enabled *Monitors* and (2) ZK-enabled SEE. For (2), the verification key added is cryptographically bound to the implemented SLA terms. For (1), the specification is confirmed and verification material is added during deployment.

2. Deployment: The TEE-enabled monitors and the ZK-enabled SEE program are deployed. For now, we assume their deployment at the Notary's monitoring infrastructure. Once the TEE is booted, an *attestation report* is created and submitted to the decentralized storage system. Contained verification material is added to the VSLAS.

3. Monitoring: The TEE-based monitor runs at the Notary and collects measurements about the service. The measurements are collected in batches. Merkle Tree-based commitments are created over the batches and cryptographically signed before they are persisted in the evidence storage. The monitor is a passive if it is only monitoring traffic as a proxy between the client and the provider, or it is active if it works as a probe doing independent regular checks to the service.

4. Evaluation: Once a violation is detected, the SEE, as the prover, creates a cryptographic proof based on the relevant SLI measurements and the encoded SLA logic. The proof is submitted to the decentralized evidence storage accessible to contractual parties and available for independent verification of validity, integrity, and authenticity using the corresponding verification key

5. Termination: The ZK-enabled Verifiable Violation Claim can be independently verified using the verification material contained in the VSLAS by the Verifier, e.g. Arbitrator or Cyber Insurance.

4.2 Verifiable SLA Specifications and SLO Evaluation

The VSLAS extends existing formats through additional specifications and corresponding verification material for both the TEE-based monitors and the ZK-enabled SEE. While the former guarantee integrity of the monitoring software's execution and authenticity of the measurements, the latter makes sure the SLA Violation Claims are correctly computed based on the right measurements. The VSLAS formalizes SLA predicates as specifications that are translated into provable and publicly auditable ZK-enabled programs, the zkVM bytecode. As depicted in Listing 1.1, our specification extends OpenSLO[1] with named verifiability semantics. The translation from SLA parameters into a deterministic ZK-proving program can be verified before deployment, allowing all parties to audit the exact logic before agreeing to the SLA. This separates what is evaluated (validity) from how it is evaluated (integrity), both verifiably guaranteed.

The *zkVM program* is used by the Verifiable SLO Evaluation Engine (V-SEE) to generate proofs of SLO violations. It takes relevant measurements from the TEE-enabled monitors as inputs and executes two key operations: First, it validates the measurements' authenticity using the verification material of the TEE-enabled monitor. Second, it aggregates the measurements into SLIs and checks them against predefined SLOs, e.g., through range proofs and predefined thresholds. If the measurements' authenticity is proven and the SLIs violate an SLO, an SLA violation is determined. The resulting ZKP can be verified with corresponding verification key that is cryptographically anchored into the SLA program preventing malicious evaluators to impair the computation's integrity.

By encoding SLA compliance as verifiable predicates, the framework supports higher levels of automation. Proof generation can be integrated into orchestration pipelines, producing ZK claims that are independently verifiable, asynchronously by any stakeholder (e.g. automated agents or legal arbitrators), without revealing telemetry or relying on trusted infrastructure. This enables scalable, confidential SLA enforcement while minimizing trust and coordination overhead.

```
## STANDARD USE OMMITED BECAUSE OF SPACE
  # VERIFICATION EXTENSION
  verification:
    monitors:
      - type: tee-passive           # TEE-based monitor type
        location: us-east-1         # Deployment location
        tee-provider: intel-tdx     # Hardware attestation
    evidence:
      storage-type: ipfs-private    # Decentralized storage
      blockchain:
        network: base               # Blockchain for anchoring
        contract: "0x..."           # SLA Evidence Registry contract
    aggregation-engine: risc0       # zkVM for proof generation
```

Listing 1.1. OpenSLO extension specification: Adding verifiability.

[1] https://github.com/OpenSLO/OpenSLO.

4.3 Trusted Monitors and SLIs

The SLA monitors run inside a TEE to protect their computational integrity. Monitors are initialized with specifications for their internal monitoring software co-signed by the provider and consumer. Upon boot, the TEE-based monitor registers its hardware-bound public key with the Evidence Storage and exposes a remote attestation endpoint. The TEE's key certificates and attestation measurement hash (quote) contained in the attestation report enable the verification of the computational integrity of the TEE, i.e., the right monitoring software is running inside the TEE. The TEE's signing key is integrated into the VSLAS of the ZK-enabled SEE to enable proving the authenticity of SLIs collected inside the TEE. If the contained TEE reference corresponds to the one in the SLA, validity is assured. During monitoring, collected measurements, e.g., latency or response code, are signed and sent to storage, forming the leaves of a Merkle tree. The root is signed and checkpointed in the decentralized evidence storage ensuring that all SLIs are accounted for when computing SLOs.

To prevent manipulation of the SLI measurements, storage must be tamper-resistant and equally accessible to all parties. To realize such a decentralized storage systems, we leverage blockchain to store non-disclosing commitments and a content-addressable distributed storage system like IPFS to store the corresponding, cryptographically linked measurements [3]. Signed measurement data is stored off-chain and linked via cryptographic hashes on-chain, in a SLA Evidence Registry smart contract, enabling scalable retention without incurring blockchain overhead. Peer-to-peer replication ensures resilience, while content-addressed identifiers make tampering detectable. This combination of decentralized storage and blockchain anchoring is an established pattern [34] that ensures verifiable, persistent, and trustworthy access to SLA evidence.

5 Technical Realizations

We now detail the system implementation[2] as per the system design in Sect. 4.

5.1 Trusted Monitors Implementation: Active and Passive Monitor

Both monitors rest on a small Monitor Core that executes inside a TEE, like an Intel TDX enclave, deployed in the Notary infrastructure, for instance, with a decentralized TEE provider, to avoid collusion with any party.

At the enclave's first boot, the Core halts until it receives a co-signed JSON configuration produced during SLA negotiation. This document names the target endpoints, the measurement interval Δt, and the schema version. This configuration, co-signed during SLA negotiation, remains immutable throughout the monitor lifecycle. SLA amendments require deploying a new monitor instance with fresh cryptographic keys, ensuring clean security boundaries between different SLA versions. The enclave then produces a remote attestation quote that

[2] Repository: https://github.com/ferjcast/Verifiable-SLAs/.

cryptographically binds the public key to the specific measurements of the initial memory state, the mrTd field of the quote, proving that the key was generated within the authentic monitoring software. This attestation quote, along with the public key and monitor metadata, is submitted to a *SLA Evidence Registry smart contract* through a registration transaction signed by both the service provider and consumer. The Core also exposes an HTTPS */attestation* endpoint that returns the enclave's most recent TEE quote; verifiers can thus confirm the initial memory state and runtime measurements directly with remote attestation.

Failure to publish two consecutive batches raises an on-chain heartbeat alert, signaling that the VM was firewalled or powered down; either party can corroborate this by inspecting the project's audit trail.

Active Monitor. Running on the Core, the monitor acts as a probe and, for example, emits a synthetic GET to */health* endpoint, every Δt seconds. It records latency and status, hashes the sample into an in-memory Merkle tree, and, after a configurable records, e.g. 1024, seals the batch to Evidence Storage and signs the Merkle root.

Passive Monitor. A lightweight Flask proxy, linked against the same Core, sits inline on the service path and derives latency and status directly from production traffic. Since it depends also on the amount of requests made to the service, it can be configured to produce batches in time windows when there are measurements.

5.2 Evidence Storage: Blockchain and IPFS Architecture

The Evidence Storage component implements a hybrid architecture combining a permissioned blockchain network with a private IPFS deployment for decentralized content storage. The *SLA Evidence Registry smart contract* implementation defines three primary data structures. The MonitorRegistry mapping associates TEE-generated blockchain addresses with their attestation quotes and operational status. The EvidenceBatch encodes the Merkle root, submitter address, Unix timestamps for the measurement window, and the batch size. The BatchSequence mapping ensures strict ordering by tracking the expected sequence number for each monitor, preventing evidence omission attacks.

Monitors accumulate CIDs from individual IPFS storage operations into a binary Merkle tree, computing parent nodes as keccak256 (left || right) until reaching the root. This 32-byte root represents a cryptographic commitment of measurements while incurring fixed gas costs per batch submission. Verification of individual measurements requires only log(n) hashes to reconstruct the path from leaf to root, enabling efficient proof generation for specific SLI values. Evidence availability within the IPFS network relies on controlled replication across consortium nodes. The CID-based addressing enables content retrieval from any authorized node in the network, while the blockchain record provides authoritative proof of when evidence was submitted and by which monitor.

As the monitor stores each encrypted measurement in IPFS and collects the resulting CIDs, it maintains an ordered list that preserves the sequence and structure of the measurement batch. Upon reaching the configured batch size of

measurements, the monitor creates a JSON manifest file containing the complete ordered array of CIDs along with metadata including the batch timestamp range, monitor identifier, and schema version. This manifest file is itself stored in the IPFS network and registered in the smart contract as part of the batch, receiving its own CID that serves as a permanent reference to the batch composition.

5.3 Verifiable SLA Specification to ZkVM Bytecode

Our prototype uses RISC0's zkVM, which requires translating SLA specifications into Rust implementations that are then deterministically compiled to RISC-V bytecode. This toolchain provides dual auditability: stakeholders can inspect both the high-level Rust code implementing the SLA logic, and verify its compilation to RISC-V bytecode. The zkSTARK proof system then ensures this exact bytecode was executed during compliance evaluation, creating an auditable chain from specification to execution proof.

While manual specification-to-Rust translation introduces potential for human error, our auditability framework transforms this from a security risk into a process verification step any party can do. Any translation errors become detectable during pre-deployment audit, and the post-deployment cryptographic proofs guarantee execution of the audited logic.

5.4 SLO Evaluation Engine with zkVM-Based Violation Claims

The SLIs evidence evaluation for SLOs is implemented as a service whose sole trusted component is a zkVM. Incoming objects are the batch blobs published to the Evidence Storage by the monitors, and identified by their CIDs, and the accompanying SLI measurements, from the smart contract. The SEE executes four steps:

1. Pre-flight validation: Outside the zkVM, the service (i) fetches the ciphertexts from IPFS, (ii) checks the Ed25519 signature on each measurement, and (iii) recomputes the Merkle root over the telemetry. Only the root, the window parameters $\langle t_{\text{start}}, t_{\text{end}} \rangle$ and the compact metric vector V are passed to the proving environment; in case of giving a wrongful input, the program fails to continue with the next step.

2. Proving inside the zkVM: The compliance predicate, translated from the specification to Rust code, and compiled to use 32-bit RISC-V instruction set, runs in a zkVM STARK proving system, e.g. such as RISC0 or SP1. For the exemplar SLO "95%x of latencies below 300 ms", the circuit consumes the SLI measurements in the time window under evaluation, and outputs the compliance evaluation, the Monitor's signature verification, and the Merkle tree hash match verification in a single bit $\mathsf{ok} \in \{0, 1\}$, for each one of them.

3. Claim emission and storage: The zkVM returns π, ok, and the public inputs $\{\text{root}, t_{\text{start}}, t_{\text{end}}\}$. These values are wrapped into the standardised claim schema of Listing 1.1, yielding a v-claim. The aggregation execution returns the

v-claim and pushes it, together with π, to Evidence Storage; the corresponding CID and Merkle root are anchored on chain exactly as for monitor batches. Because the proof is non-interactive and succinct, any stakeholder can verify the claim without access to telemetry or to the SEE itself.

4. Programmability and upgrade path: A new or updated SLA predicate is compiled to RISC-V byte-code and hot-swapped by redeploying a container image; no parameter change is required on the monitors. The proving key is deterministically derived from the circuit hash and pinned on chain, ensuring that verifiers always load the correct parameters.

6 Evaluation

We evaluate the performance and security properties of our system across three key dimensions: evidence aggregation efficiency, verification cost, and fulfillment of security properties. Experiments simulate TEE-based monitoring and ZKP-based evaluation for a single service.

6.1 Experiment Design

Each monitor was deployed on a Phala Network[3] Confidential VM, as the TEE, instance of type tdx.small with 2 vCPUs and 4 GB of memory, at a cost of 0.138 USD/hr. In particular we deploy the passive and active monitor to generate evidence in batches of 512, 1024, 2048, 4096, and 8192 measurements with varying requests per second (rps), with 32, 64, 128, 256 and 512 rps. Every batch involved evidence generation, IPFS anchoring, blockchain registration, aggregation, and claim verification. We evaluate three proving strategies: (a) **Full Batch Disclosure** [13] – All the measurements are disclosed and rely in the monitor's signature; (b) **Batch-Level Privacy** – ZKP attests "99.9% success rate" without revealing measurements, but disputes require disclosing the entire batch; and (c) **Violation-Level Privacy** – ZKP proves specific violations (e.g., "measurement #5,847 exceeded 100 ms") with Merkle inclusion, requiring disclosure of only those measurements during disputes while preserving privacy of the remaining measurements in the batch. Blockchain anchoring is performed by submitting evidence CIDs and SHA256 commitments to a smart contract.

6.2 Experiment Results

The results are shown in Fig. 3 and 4. Considering blocksize limits, gas costs remained constant at 2825k per deployment and 188k for batch anchoring transaction, as only the Merkle root is registered. Since this cost is independent of batch size, the results confirm the scalability of on-chain anchoring with increasing numbers of evidences.

[3] https://cloud.phala.network.

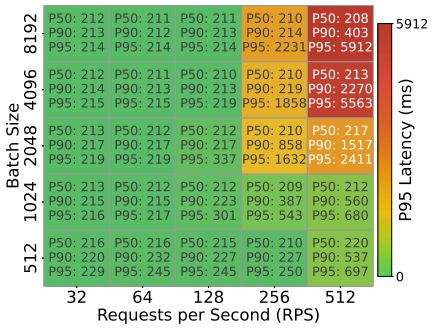


Fig. 3. Stacked Heatmap for P50, P90 and P95 latency for Batch Size vs RPS.

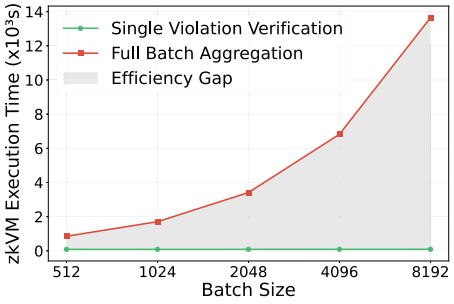


Fig. 4. Verification time vs number of evidences.

zkVM aggregation demonstrates consistent performance for single measurement evaluations, requiring only a signature verification and Merkle root recomputation. However, full batch aggregation exhibits linear scaling behavior, where processing time increases proportionally with the number of measurements in each batch, making larger batches computationally more expensive to evaluate. Our experiments revealed clear performance boundaries: the monitored service maintained stable operation up to 256 RPS with P95 latencies below the 300 ms SLA threshold, but showed initial degradation at 512 RPS. Notably, larger batch sizes (2048, 4096, and 8192) significantly impacted latency at both 256 and 512 RPS, demonstrating that batch size becomes a critical performance factor even before reaching maximum system throughput. In contrast, P50 latencies remained relatively stable across all test cases, indicating that the spike degradation occurs specifically when the Merkle tree is sealed, where the process disproportionately affects tail latencies, leaving median performance unaffected.

7 Discussion and Implications

Our architecture combines TEEs for secure measurement collection with zkVMs for verifiable compliance evaluation, enabling privacy-preserving SLA claims that are independently verifiable and free from reliance on provider-reported metrics or trusted third parties. We examined three verification strategies: Full Batch Disclosure, Batch-Level Privacy, and Violation-Level Privacy, each offering a different trade-off between trust properties and performance. Table 1 summarizes their security, verification, and operational properties. While Full Batch Disclosure is lightweight, it depends on trust in the monitor. In contrast, the zkVM-based strategies improve verifiability without revealing full batches.

Our architecture balances validity, trust, and cost by combining TEEs and zkVMs. TEEs offer efficient, scalable aggregation with strong integrity and authenticity guarantees, but rely on hardware trust. zkVMs remove the hardware trust assumption and ensure cryptographic validity, though at higher proof

Table 1. Comparison of verification strategies and their trade-offs.

Property	Full Batch Disclosure	Batch-Level Privacy	Violation-Level Privacy
Integrity	TEE signatures	TEE + zkVM	TEE + zkVM
Authenticity	TEE attestation	TEE + proof	TEE + proof
Validity	**Manual inspection**	Bytecode reproducibility	Bytecode reproducibility
Verifier Requires	Public key + Complete Batch	ZK verifier + public inputs	ZK verifier + public inputs
Claim Resolution	N/A - already public	Reveal entire batch	Reveal only violations
Best For	Public metrics, transparency	Competitive environments	Sensitive operations

generation cost. Nonetheless, this enables a shift from continuous verification to lightweight, on-demand cryptographic guarantees. The design also supports monitor replication and load balancing for linear scalability.

Direct verification grows linearly with telemetry size (see Fig. 4), making it impractical at scale, but aggregation reduces verifier effort to near-constant cost. While zkVM verification time remains stable, proof generation grows with batch size. To reduce overhead, we adopt an *optimistic mode* where proofs are generated only for violations, disputes, audits, or checkpoints, which, in high-uptime scenarios, can drastically cut costs. SLA terms can set thresholds (e.g., only for violations over $1000), aligning effort with risk. Full Batch Disclosure avoids ZK proof latency but shifts cost to trust. A single monitor key becomes a failure point, requiring audits and legal agreements. These hidden costs often exceed those of cryptographic verification. Using zkSNARKs instead of zkSTARKs could further enable smart contract integration for on-chain SLA enforcement.

Attested TEEs, Merkle commitments, and zkVMs together ensure **integrity** (correct computation), **authenticity** (verified origin), and **validity** (alignment with SLA logic). Merkle trees enable selective audit disclosure without full data exposure. This decentralized auditability reduces operational and legal overhead, resolving the "who monitors the monitors" dilemma through verifiable delegation.

Beyond verification, the system enables SLA-driven automation. Verifiable claims can trigger failovers, renegotiate terms, or automate compensation, supporting SLA-aware orchestration. Even small providers, like micro-clouds or home-labs, can join broader ecosystems by publishing cryptographic proofs. Standardized, machine-verifiable SLA claims could streamline cyber insurance workflows, like underwriting, claims adjudication, and actuarial modeling. However, current insurer processes lack interoperability with cryptographic claims. Future work should explore formal mappings to legal clauses, machine-readable audits, and trusted risk attestations. While broader adoption requires research on secure circuit design, predicate verification, TEE support, operational and setup complexity, and legal recognition of cryptographic evidence. These steps would strengthen the foundation for trustworthy, automated service ecosystems.

8 Conclusion

We introduced a framework for verifiable SLA compliance claims using TEE-based monitors and zkVM-based aggregation. This architecture ensures integrity, authenticity, and validity of monitoring evidence, enabling privacy-preserving, automated evaluation of SLA conditions.

Our evaluation demonstrates that the system scales efficiently, reduces reliance on trusted third parties, and lowers operational costs by minimizing human involvement in verification and dispute resolution. It also enables selective disclosure for audits and supports automated decision-making in decentralized service environments. This approach contributes toward more transparent, resilient, and cost-effective SLA management and trustless service ecosystem. It also lays groundwork for emerging applications in cyber insurance, where verifiable compliance evidence could improve underwriting, claims processing, and legal clarity. Future work includes formal verification of predicates, broader TEE support, and exploring the legal standing of cryptographic compliance claims.

Acknowledgements. Funded by the European Union (EU) (TEADAL, 101070186). Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the EU. Neither the EU nor the granting authority can be held responsible for them. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alzubaidi, A., et al.: A blockchain-based SLA monitoring and compliance assessment for IoT ecosystems. J. Cloud Comput. **12**(1), 50 (2023)
2. Aziz, B., et al.: A systematic literature review of cyber insurance challenges. In: 2020 International Conference on Information Technology Systems and Innovation (ICITSI), pp. 357–363. IEEE (2020)
3. Benisi, N.Z., et al.: Blockchain-based decentralized storage networks: a survey. J. Netw. Comput. Appl. **162**, 102656 (2020)
4. Brasser, F., Jauernig, P., Pustelnik, F., Sadeghi, A.R., Stapf, E.: Trusted container extensions for container-based confidential computing. arXiv preprint arXiv:2205.05747 (2022)
5. Brito, E., Castillo, F., Pullonen-Raudvere, P., Werner, S.: TrustOps: continuously building trustworthy software. In: International Conference on Enterprise Design, Operations, and Computing, pp. 53–67. Springer (2024)
6. Castillo, F., Heiss, J., Werner, S., Tai, S.: Trusted compute units: a framework for chained verifiable computations. In: 2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), pp. 1–9 (2025)
7. Drean, J., et al.: Teaching an old dog new tricks: verifiable FHE using commodity hardware. arXiv preprint arXiv:2412.03550 (2024)
8. Ernstberger, J., et al.: Do you need a zero knowledge proof? Cryptology ePrint Archive (2024)
9. Freitas, A.L., Parlavantzas, N., Pazat, J.L.: Cost reduction through SLA-driven self-management. In: 2011 IEEE Ninth European Conference on Web Services, pp. 117–124. IEEE (2011)

10. Gamez-Diaz, A., Fernandez, P., Ruiz-Cortes, A.: Automating SLA-driven API development with SLA4OAI. In: Yangui, S., Bouassida Rodriguez, I., Drira, K., Tari, Z. (eds.) ICSOC 2019. LNCS, vol. 11895, pp. 20–35. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-33702-5_2
11. Heiss, J., Busse, A., Tai, S.: Trustworthy pre-processing of sensor data in data on-chaining workflows for blockchain-based IoT applications. In: Hacid, H., Kao, O., Mecella, M., Moha, N., Paik, H. (eds.) ICSOC 2021. LNCS, vol. 13121, pp. 133–149. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-91431-8_9
12. Hussain, W., Hussain, F.K., Hussain, O.K.: Maintaining trust in cloud computing through SLA monitoring. In: Loo, C.K., Yap, K.S., Wong, K.W., Beng Jin, A.T., Huang, K. (eds.) ICONIP 2014, Part III. LNCS, vol. 8836, pp. 690–697. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-12643-2_83
13. Kanwal, A., Masood, R., Shibli, M.A., Mumtaz, R.: Taxonomy for trust models in cloud computing. Comput. J. **58**(4), 601–626 (2015)
14. Kapsoulis, N., et al.: Reinforcing SLA consensus on blockchain. Computers **10**(12), 159 (2021)
15. Keller, A., Ludwig, H.: The WSLA framework: specifying and monitoring service level agreements for web services. J. Netw. Syst. Manage. **11**, 57–81 (2003)
16. Khan, K.M., et al.: Blockchain-enabled real-time SLA monitoring for cloud-hosted services. Cluster Comput., 1–23 (2022)
17. Lavin, R., et al.: A survey on the applications of zero-knowledge proofs. arXiv preprint arXiv:2408.00243 (2024)
18. Ludwig, H., et al.: rSLA: monitoring SLAs in dynamic service environments. In: Barros, A., Grigori, D., Narendra, N.C., Dam, H.K. (eds.) ICSOC 2015. LNCS, vol. 9435, pp. 139–153. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48616-0_9
19. Macias, M., Guitart, J.: Analysis of a trust model for SLA negotiation and enforcement in cloud markets. Futur. Gener. Comput. Syst. **55**, 460–472 (2016)
20. Muñoz, A., Ríos, R., Román, R., López, J.: A survey on the (in) security of trusted execution environments. Comput. Secur. **129**, 103180 (2023)
21. Nicolazzo, S., Nocera, A., Pedrycz, W.: Service level agreements and security SLA: a comprehensive survey. arXiv preprint arXiv:2405.00009 (2024)
22. Novo, C.C., et al.: Applications of cyber insurance in smart cities: a systematic literature review. Int. J. Inf. Secur. **24**(2), 93 (2025)
23. Petcu, D., Crăciun, C.: Towards a security SLA-based cloud monitoring service. In: International Conference on Cloud Computing and Services Science, vol. 2, pp. 598–603. SciTePress (2014)
24. Sadovykh, A., et al.: VeriDevOps: automated protection and prevention to meet security requirements in DevOps. In: 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1330–1333. IEEE (2021)
25. Schubert, C., Borkowski, M., Schulte, S.: Trustworthy detection and arbitration of SLA violations in the cloud. In: Kritikos, K., Plebani, P., de Paoli, F. (eds.) ESOCC 2018. LNCS, vol. 11116, pp. 90–104. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-99819-0_7
26. Segarra, C., Feldman-Fitzthum, T., Buono, D., Pietzuch, P.: Serverless confidential containers: challenges and opportunities. In: Proceedings of the 2nd Workshop on SErverless Systems, Applications and MEthodologies, pp. 32–40 (2024)
27. Sghaier, R., et al.: A review on SLA monitoring based on blockchain. In: International Conference on Intelligent Systems Design and Applications. Springer (2023)

28. Shen, Y., Chen, G., Ma, H., Zhang, M.: Cost-aware dynamic cloud workflow scheduling using self-attention and evolutionary reinforcement learning. In: International Conference on Service-Oriented Computing, pp. 3–18. Springer (2024)
29. Thaler, J., et al.: Proofs, arguments, and zero-knowledge. Found. Trends® Priv. Secur. **4**(2–4), 117–660 (2022)
30. Tsohou, A., Diamantopoulou, V., Gritzalis, S., Lambrinoudakis, C.: Cyber insurance: state of the art, trends and future directions. Int. J. Inf. Secur. **22**(3), 737–748 (2023)
31. Wang, X., Yu, R., Yang, D., Gu, H., Li, Z.: VeriEdge: verifying and enforcing service level agreements for pervasive edge computing. In: IEEE INFOCOM 2024-IEEE Conference on Computer Communications, pp. 2149–2158. IEEE (2024)
32. Weerasinghe, N., et al.: Proof-of-monitoring (PoM): a novel consensus mechanism for blockchain-based secure service level agreement management. IEEE Trans. Netw. Serv. Manage. **20**(3), 2783–2803 (2023)
33. Yang, Y., et al.: Dual-tree genetic programming for deadline-constrained dynamic workflow scheduling in cloud. In: International Conference on Service-Oriented Computing, pp. 433–448. Springer (2022)
34. Zhou, H., Ouyang, X., Ren, Z., Su, J., de Laat, C., Zhao, Z.: A blockchain based witness model for trustworthy cloud service level agreement enforcement. In: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, pp. 1567–1575. IEEE (2019)
35. Ziegler, W.: Cloud computing. In: Computational Life Sciences: Data Engineering and Data Mining for Life Sciences, pp. 277–299. Springer (2023)

A Service-Oriented Digital Twin Architecture for Seamless Reliability Verification of IoT Systems

Janik-Vasily Benzin(✉), Jürgen Mangler, and Stefanie Rinderle-Ma

Technical University of Munich, TUM School of Computation, Information and Technology, Garching, Germany
{janik.benzin,juergen.mangler,stefanie.rinderle-ma}@tum.de

Abstract. Internet of Things (IoT) systems are distributed collections of embedded systems that are connected over the Internet or similar networks. A key property of IoT systems is the need for high reliability throughout their lifespan under physical constraints, e.g., material properties, and natural degradation of system components. Simulation and digital twin approaches can quantify statistical assurances at design time and runtime respectively to verify reliability of IoT systems. However, existing approaches lack integration through a multi-paradigm modeling approach for seamless design time and runtime reliability verification, sole use of mature standards and protocols for interoperability and easy extension, and state-of-the-art statistical model checking (SMC) techniques for sound and efficient reliability verification. Consequently, we propose a service-oriented digital twin architecture and implement the architecture in a production-grade, interoperable, and radically distributed scale out process engine that integrates both design and runtime verification in a single model. We demonstrate our architecture and implementation for four IoT systems and report statistical reliability assurances through sound and efficient SMC.

Keywords: Digital twin · Service-Oriented Architecture · Reliability · Execution · Internet of Things · Statistical Model Checking

1 Introduction

Internet of Things (IoT) systems are deployed on heterogeneous IT infrastructures of their application, e.g., healthcare for patient monitoring [12]. The diversity of *embedded systems* [19] that comprise the IoT system can be high and the set of embedded systems in an IoT system can change over time. An embedded system has a discrete control part (microcontroller) and a physical dynamics part (physical device). While embedded systems are characterized by constrained runtime environments and face natural degradation and device failures, they must be reliable over their lifespan to continuously perform their function. For example, a collection of thermostats is reliable, if they maintain the room temperature within a prescribed range over a year with a high probability.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 175–190, 2026.
https://doi.org/10.1007/978-981-95-5015-9_13

Existing approaches to verify reliability through *statistical model checking* (SMC) [5] can be classified along the lifecycle of the IoT system. Design time approaches verify reliability before the IoT system is deployed to production by simulating the system behavior through a system model. The most powerful models are *stochastic hybrid systems* (SHS) [19] that combine stochastic physical dynamics of the physical device with stochastic, discrete control behaviour of the microcontroller. As such, SHS combine *ordinary differential equations* (ODEs) for physical dynamics with state-based models for discrete behaviour. By contrast, runtime approaches verify reliability while the IoT system is running. To that end, runtime approaches use a *digital twin* (DT) [20] of the IoT system, i.e., its dynamic digital representation with the ability to control the IoT system [20]. Because a DT is connected to the real IoT system, runtime approaches do not model the IoT system behaviour, but instead focus on the service design, development, and orchestration to implement and deploy the DT [9,13,27].

Currently, design and runtime approaches are separated since the conceptual models (SHS vs. service orchestration) and technical implementations (monolithic vs. service-oriented) differ significantly. These conceptual and technical differences aggravate model reuse from design time approaches for developing the DT at runtime and make it impossible to use the DT for design time reliability verification. However, a seamless integration of formal design time model and DT bears the great benefit of reduced implementation effort under formal reliability guarantees. Hence, to overcome the challenge of seamless reliability verification and to unlock its potential, we propose **Twin-CPEE**, which is a multi-paradigm DT that combines the *cloud process execution engine* (CPEE) [22] for execution and service orchestration at runtime with SHS for simulation at design time. Our proposal consists of a DT service-oriented architecture and service pattern to implement multi-paradigm DTs [4] that enable seamless reliability verification through SMC. The multi-paradigm DT applies Business Process Modeling and Notation (BPMN)[1] as de facto standard for service orchestration and SHS as the theoretical foundation for simulating both stochastic physical dynamics and discrete control. Twin-CPEE builds on the CPEE due to its use of mature standards [22], its lightweight and flexible deployment to embedded systems [30], and success in industry[2]. Finally, a DT that is developed with Twin-CPEE enables the recording of *IoT-enriched logs* [23] that contain service execution and physical variable information at design time and runtime.

In Sect. 2, we state requirements for a multi-paradigm DT and related work. In Sect. 3, we present the architecture and service pattern of Twin-CPEE. In Sect. 4, we demonstrate Twin-CPEE by verifying reliability of three IoT systems through SMC. In Sect. 5, we conclude and give an outlook.

[1] bpmn.org.

[2] 1.689.504 downloads, see https://rubygems.org/profiles/eTM, accessed 2025-07-21; 16 process orchestrations run operatively at 7 manufacturing companies.

2 Related Work and Requirements

Design time approaches and their formal modelling language to simulate the IoT system stem from different domains, i.e., from formal methods and business process management (BPM). Formal methods approaches (denoted by ^{f} in Table 1) advance the theoretical frontier on model expressiveness and statistical model checking. BPM approaches are denoted by ^{b}. Existing work on extending BPMN to represent IoT systems [12] at design time often lacks formal semantics for reliability verification or focuses on certain aspects such as *complex event processing* [21] without providing a running simulation model. For runtime approaches, we focus on digital twins of IoT systems that are based on some process modeling language to orchestrate the embedded systems. Thus, literature on digital twins of single embedded systems from the engineering domain is out of scope [20]. Additionally, we focus on approaches that produce execution logs in a standard format, e.g., eXtensible Event Stream (XES) or a format of similar quality [16] for subsequent analysis. The resulting set of existing approaches, the requirements set out by existing approaches at design and runtime of the DT, and an assessment of existing approaches along the requirements are depicted in Table 1.

Table 1. Requirements and Existing Approaches.

General				Representation			Architecture		
Lifecycle	Name	Ref.	Year	Cont.	Data	Hybrid	Stoch. (S/D)	SOA	Stand.
Design	IoT-BP-DT^{b}	[28]	2023	✗	✓	✗	✗/✗	✓	✗
	HYPEG^{f}	[26]	2017	✓	✗	✓	✓/✗	✗	✗
	COSMOS^{f}	[3]	2015	✓	✗	✗	✓/✗	✗	✗
	UPPAAL SMC^{f}	[11]	2015	✓	✗	✓	✓/✗	✗	✗
	PRISM^{f}	[18]	2011	✓	✓	✗	✓/✓	✗	✗
	CPNTools^{b}	[17]	2007	✓	✓	✗	✓/✓	✗	✗
Runtime	Smarter	[27]	2025	✓	✓	✗	✗/✗	✓	✓
	FloBP	[13]	2024	✓	✓	✗	✗/✗	✓	✓
	Prowin	[9]	2023	✓	✓	✗	✗/✗	✗	✓
	FAME	[8]	2023	✓	✓	✗	✗/✗	✗	✓
	CPEE	[22]	2022	✓	✓	✗	✗/✗	✓	✓
	IoT-enhanced BP	[29]	2022	✓	✓	✗	✗/✗	✓	✗
	DCH-JBPM	[6]	2019	✓	✗	✗	✗/✗	✗	✗
	makeSense	[25]	2019	✓	✓	✗	✗/✗	✗	✗
	Smart Device	[31]	2018	✓	✓	✗	✗/✗	✓	✓
	ROA	[10]	2015	✓	✗	✗	✗/✗	✓	✗
	ESPT	[1]	2014	✓	✗	✗	✗/✗	✗	✗

In Table 1, representation requirements comprise a *cont*inuous time notion in $\mathbb{R}_{\geqslant 0}$ and *data*flow of physical, continuous and arbitrarily typed variables rep-

resented explicitly, e.g., through a shared variable concept with explicit data manipulation, or not. Moreover, similar to the theory of *hybrid* systems, an approach can either allow simulation of physical processes, e.g., a heating of a kettle, through their corresponding ordinary differential equation systems or not. Likewise, *stoch*astic elements are increasingly added to hybrid systems resulting in stochastic hybrid systems [19], e.g., a sensor variable is assigned a sampled value from a *s*tatic probability distribution to model noisy sensors or from a *d*ynamic probability distribution that changes its shape depending on other variables such as time to model degrading, noisy sensors. Regarding architectural requirements, approaches can benefit from the advantages of service-oriented architectures (*SOA*) or not. Lastly, an approach can either have open interfaces with *stand*ardized protocols or not.

Overall, none of the existing approaches covers both, design and runtime representation for simulation and execution of DTs. This results in the inability to either reliability verification or DT execution. Hence, in the following, we present the multi-paradigm Twin-CPEE approach integrating both, design and runtime representation and implementation of a DT in a seamless manner.

3 Twin-CPEE

We present the service-oriented architecture (cf. Sect. 3.1) and the associated service pattern for implementing multi-paradigm DTs of IoT systems (cf. Sect. 3.2).

3.1 A Service-Oriented Digital Twin Architecture

Due to the heterogeneous IT infrastructures in which IoT systems are deployed, the diverse range of embedded systems, and the dynamic nature of the IoT system, a service-oriented architecture with an orchestrating process engine is well-suited as a DT of an IoT system. The decoupled, modular, and distributed layout of services in the DT architecture of Twin-CPEE is depicted in Fig. 1. On a high-level, the architecture consists of five main groups of (micro-)services (denoted by I-V in Fig. 1). Group I (emphasized in orange in Fig. 1) is the DT service pattern that is presented in detail in the next section. Group II shows the REST operation interface of process instances (as denoted in violet in I) with standard wrapper microservices for connecting communication protocols of embedded systems in more detail than in group I. Group III is the CPEE [22] with its microservices abstracted into a single service. The CPEE supports the representation of the service orchestrations in BPMN that are internally serialized into Refined Process Structure Trees with well-defined execution semantics. Group IV shows the client with two different user interfaces (UIs). Lastly, group V shows supporting microservices that enable offline and online analysis.

As a central, logical container of current sensor and actuator values, the `DT container` in group I is a simple microservice dedicated to only hold the current values. The `DT container` in conjunction with `physical`, `environment`, or `control` process instances is sufficient to model, simulate, and execute an IoT

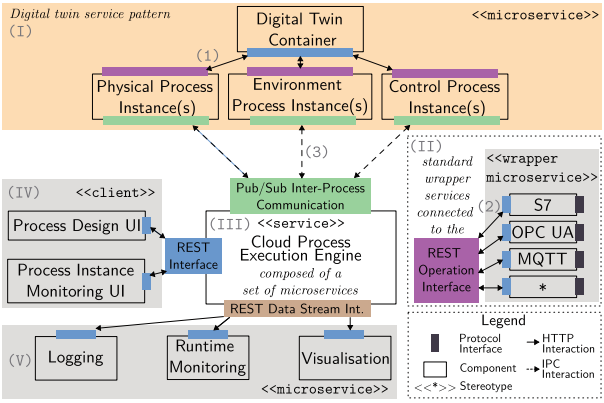


Fig. 1. The service-oriented digital twin architecture of Twin-CPEE.

system and its DT. Independent of the lifecycle, we distinguish different communication pathways that must always be adhered to. Each process instance freely communicates (1) with the `DT container` via REST (cf. group I in Fig. 1), (2) with wrapper microservices (cf. II in Fig. 1) via REST and (3) with the `CPEE` (cf. III in Fig. 1) via *Pub/Sub inter-process communication* (IPC). Via Pathway (1), the `DT container` receives new sensor and actuator values or sends the current values upon request from or to any of the `physical`, `environment`, or `control` process instances respectively. Via pathway (2), a process instance can be connected to embedded systems. To that end, a process instance's operation interface is connected to Twin-CPEE's extensible set of wrapper microservices, which currently wrap the *S7*, *OPC UA* and *MQTT* protocols. Hence, the wrapper microservices allow easy integration with a large set of embedded systems for execution of the DT at runtime through tasks that call the respective wrapper service. The third pathway (3) of exists for highly performant IPC communication between process instances and the CPEE.

Importantly, the architecture prohibits direct communication between process instances for two reasons. First, a single, indirect communication pathway between process instances via the `DT container` is transparent and easily maintainable. Second, any values sent to the `DT container` represent the current state of the IoT system such that monitoring becomes straightforward compared to deciding upon each direct communication between process instances whether this communication describes the IoT system's current state or not.

While a CPEE process model is designed in the `process design UI` (cf. Fig. 1 IV), the UI communicates via REST with the CPEE to change the model. As soon as a CPEE model is instantiated via a REST call to the CPEE's REST interface, the CPEE transpiles the respective CPEE process model into executable Ruby, JavaScript, or platform-dependent bytecode through transpiling into Rust [30] and compiling for the host platform. The executable code is executed in its own OS process of the respective host's runtime environment, where

a host could be a server, e.g., for the `environment` or `control` process instance, or an embedded system, e.g., for the `physical` process instance. Because the Rust compiler targets different host platforms[3], most embedded systems can at least execute their bytecode of a process instance. To coordinate its execution with the CPEE, a process instance instantaneously notifies the process engine about its state changes, e.g., from running to paused, and about new events, e.g., that a task is started or it received new values from the `DT container`. The CPEE sends state changes to the `process instance monitoring UI` (cf. Fig. 1 IV) and the `runtime monitoring` (cf. Fig. 1 V) microservice for instance- and CPEE-level monitoring respectively. It sends events to the `logging` microservice for persistence in standard XES logs (cf. Fig. 1 V) and to the `visualisation` microservice to visually monitor evolution of sensor and actuator values.

Overall, the service-oriented DT architecture of Twin-CPEE consists of a set of microservices in which each microservice corresponds to a clearly delineated functionality required for simulating and executing multi-paradigm DTs of IoT systems. In addition, the architecture defines a set of mainly REST-based communication pathways.

3.2 Multi-paradigm DT Service Pattern for Reliability Verification

In this section, we present the multi-paradigm DT service pattern of Twin-CPEE and implement the service pattern in an example. The multi-paradigm DT service pattern is depicted in Fig. 2, refines how the three different types of process instances are allowed to communicate with the `DT container`, and specifies what process instance can read or write what type of variable via pathway (1) (cf. last section). The pattern realizes a DT of an IoT system with a single embedded system. Since the `physical` and `control` process pattern are independent of the corresponding embedded system, DTs of IoT systems with multiple embedded systems are implemented through applying the `physical` and `control` process pattern for each embedded system respectively. The process that writes a variable determines its namespace: `physical`, `environment`, and `control` processes write `sensor/<var_id>`, `environment/<var_id>`, and `actuator/<var_id>` variables respectively. In the following, we specify read/write operations at design time. We refer to an implementation of the pattern at design time as design time DT. Afterwards, we outline the changes required to move the DT from design to runtime yielding a runtime DT.

Physical Read/Write. The `physical` process can read all three types of variables (cf. "full" denoted in purple in Fig. 2), i.e., sensor variables *sensor*1-*sensorN* and actuator variables *actuator*1-*actuatorN* of a single embedded system as well as environment variables *env*1-*envN* of the IoT system. Hence, all three types of variables with their current values taken together capture the current state of the IoT system. The reading of all three types of variables is executed by the task labelled with "Read" in Fig. 2b. The reading task has CPEE's subscription functionality [22] such that the `DT container` calls the

[3] https://doc.rust-lang.org/nightly/rustc/platform-support.html.

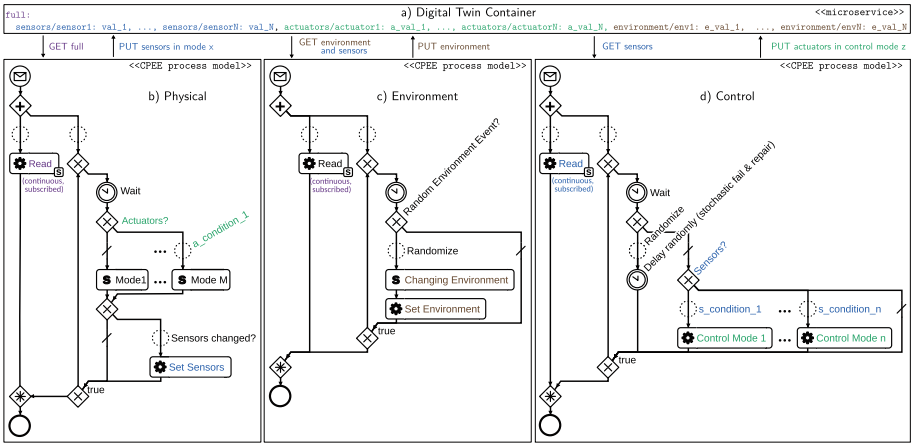


Fig. 2. The multi-paradigm digital twin service pattern of Twin-CPEE.

callback url of the subscribed reading task each time a new value is written to any of the variables to which the reading task is subscribed. Thus, the reading task is running as long as the DT is running.

Due to taking a multi-paradigm approach to combine both SHS and BPMN (cf. Sect. 1), the `physical` process allows to simulate physical dynamics with ODEs. Since embedded systems are *reactive systems* [15] that run indefinitely, an indefinite loop is executed in parallel to the reading task (cf. Fig. 2b) with a certain frequency. Each loop cycle, the `physical` process waits for a defined time, e.g., 1s. Following a *mode-based* specification of SHS [15] that model embedded systems [19], every cycle a decision is taken that depends on the control mode, e.g., a smart light is turned on or an automated motor is set to a discrete acceleration. The control mode is logically expressed as a condition over the current actuator variable values (cf. *a_condition_*1 in Fig. 2b). At design time, each branch of the decision has a script task that is labelled with the corresponding mode, e.g., "Mode 1" in Fig. 2. The mode's respective script task computes a solution to an ODE that specifies the dynamic behaviour of the real-world physical process as captured by the sensors. Similar to SHS, the `physical` process simulates the real-world physical dynamics by computing the ODE solution.

Control Read/Write. The `control` process can read sensor values through a similar reading task as the `physical` process and also has the indefinite cycle of reactive systems (cf. Fig. 2d) for a remote control in IoT systems. Depending on the sensor values (cf. *s_condition_*1), in each cycle a decision is taken that determines the next control mode, which corresponds to new actuator values written to the `DT container` through the corresponding task, e.g., "Control Mode 1". For reliability verification, the `control` process has a stochastic fail and repair, which randomly occurs in every cycle and can be dynamically distributed according to any kind of probability distribution. Upon failure, a random delay models the time of failure and repair. Thus, the `control` process includes randomness

similar to SHS. At runtime, the stochastic fail and repair must be removed from the `control` process, because degradation and failures occur naturally.

Environment Read/Write. The `environment` process can read environment and sensor values and write environment values (cf. Fig. 2c). For modularization, each environment variable that captures physical dynamics, e.g., rain for an irrigation system, should be modeled in a separate `environment` process. Most importantly, the number of `environment` processes is independent of the number of embedded systems in an IoT system. Since the environment of an IoT system exists as long as the IoT system, the indefinite loop is also in the `environment` process. Often, environments are random, e.g., rain, sun, and wind. Hence, a change in environment values randomly occurs with dynamics that are computed by the "Changing environment" script task (possibly with ODEs) and written to the corresponding environment variable through the "Set environment" task.

From Design Time to Runtime. Moving the design time DT implemented with our multi-paradigm service pattern to runtime, computation of solutions to ODEs is not required any more, because the real-world embedded systems and the natural environment are available such that the real physical sensors can be connected. Hence, only the `control` process remains with changed endpoints for the "Read" task and the "Control Mode 1" to "Control Mode n" tasks. Instead of getting sensor values from or sending actuator values to the `DT container`, the "Read" task and the n control model tasks in the `control` process of a runtime DT gets sensor values from and actuate the physical device via the embedded system's API respectively. By executing both the design time and runtime DT on equivalent hosts that are connected through an equivalent network, the simulation of the design DT is realistic and the reliability verified on design time DT is expected to hold at runtime. At the same time, the implementation effort to reuse the model from design time is greatly reduced, as the `control` process can be used almost without any changes except changed endpoints to implement and deploy runtime DT. Moreover, any analysis including reliability verification done using the design time DT can be equally done on the runtime DT without changing the analysis. Hence, our multi-paradigm DT service pattern enables seamless integration of design time and runtime for seamless reliability verification.

Example. To illustrate, we consider a simple IoT system of a water kettle and implement Twin-CPEE's DT service pattern at design time yielding a design time DT (cf. Fig. 3). The IoT system consists of a water kettle that contains water. The water temperature inside the kettle shall be between 80° Celsius (C) and 100 °C. The `DT container` contains one sensor, two actuator, and four environment variables with their respective values. The sensor variable *temperature* is the kettle's water temperature. The two actuator variables *energy* and *heat* show that the water kettle has 4000 W and is turned on. The boolean *heat* actuator variable models the two control modes of the water kettle. Consequently, the `control` process has two tasks: A "On" task that writes *true* to the *heat* variable and a "Off" task that writes *false* to the *heat* variable (highlighted in green in Fig. 3). As the water kettle does not control its energy, *energy* is

not written and not highlighted in green in Fig. 3. The environment variables *ambient_temperature*, *capacity*, and *mass* show that the room, in which the kettle is located, has a temperature of 25 °C and that the water has a capacity of 4186 J/(kg*Celsius) and a mass of 1 kg. All three environment variables are required for computing the solution to Newton's law of heating ODE [14]:

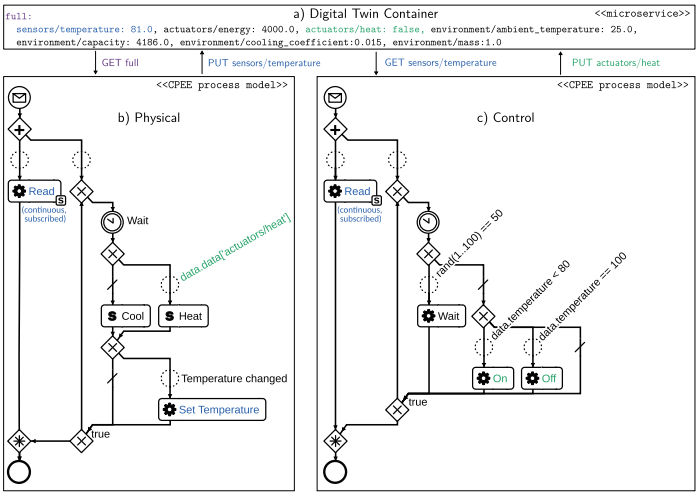


Fig. 3. Twin-CPEE implementation for an example IoT system (Color figure online)

$$temperature_{t_{now}} = min(temperature_{t_{last}} + \frac{energy * (t_{now} - t_{last})}{mass * capacity}, 100) \quad (1)$$

Hence, the current temperature $temperature_{t_{now}}$ changes to the minimum of 100 °C and the last temperature plus a temperature increase, which is proportional to the energy and passed time during heating, because water temperature does not increase above 100 °C. The environment variable $cooling_coefficient$ is required to compute the solution to the ODE for cooling. Hence, the `DT container`'s variables (cf. Fig. 3a) are sufficient to compute solutions to both ODEs that specify water temperature dynamics during heating and cooling. If the water kettle is turned on, the *heat* variable is *true* (cf. *actuators/heat* branch in green in Fig. 3b) and the "Heat" script task in the `physical` process computes the new water temperature with Eq. 1 in every execution. Likewise, if the water kettle is turned off, the "Cool" script task computes the new water temperature with the solution formula to the cooling ODE. In both cases, the water temperature changes and is subsequently written to the `DT container` through the "Set Temperature" task in the `physical` process. Lastly, the control can randomly fail with probability of 1% in each control cycle (cf. Fig. 3c). If the control fails, it is delayed through the "Wait" task to model the time of failure and repair.

Overall, the DT service pattern implementation of the water kettle enables reliability verification of how well the water temperature is kept inside the desired range. Given an implemented DT service pattern, we show how reliability verification through SMC is applied to derive statistical assurances on the sensor variable for four different IoT systems in Sect. 4.

4 Evaluation

We demonstrate the Twin-CPEE architecture and implementation with four IoT systems and verify their reliability through SMC.

4.1 Reliability Verification Through Statistical Model Checking

In this section, we show how SMC together with the multi-paradigm DT service pattern (cf. Sect. 3.2) results in reliability verification of DTs. To reduce the complexity of reliability verification, we focus on bounded reliability that verifies over a bounded time period in which the IoT system operates [5]. The IoT system is *reliable* iff it can be proven that it violates a given boundary condition throughout the time period with probability $p < \theta$. As IoT systems aim to maintain a certain function, e.g., temperature or air quality in a building, the function is formalized through the boundary condition, which expresses the desired function as a logical formula over the sensor variables. For example, "maintain the kettle's water temperature above 76 °C" specifies the function of the water kettle IoT system (cf. Fig. 3). Assuming a short time period of 20s, the boundary condition is:

$$c_h = temperature \geqslant 76 \, \mathcal{U}_{\leqslant 20} \, true \tag{2}$$

The temporal *until* operator is bounded to 20 s and refers to an implicit time $t \in \mathbb{R}_{\geqslant 0}$ with positive, real-valued domain. Hence, we take *metric temporal logic*, which is a continuous time variant of *linear time logic*, to formalize boundary conditions. To verify reliability, the boundary condition must be evaluated on a *run* π of the IoT system. For example, a run of the water kettle IoT system is executing the IoT system for 20s such that c_h is either satisfied by π or not. Runs π can be recorded from a model with formal execution semantics at design time, i.e., from the design time DT in Sect. 3.2, or from the IoT system with sensor recordings at runtime, i.e., from the runtime DT. On the one hand, we can apply *probabilistic model checking* [5] that verifies the boundary condition through exhaustive state-space exploration at design time. On the other hand, we can apply SMC to statistically analyse execution runs of the model or real IoT system both at design time or runtime to statistically verify reliability. Since we aim for seamless reliability verification, we opt for SMC to estimate the expected value of the following probability measure on boundary conditions:

$$P(X = 1), \text{where } X(\pi) = 1, \text{if } \pi \text{ violates the boundary condition, else } 0 \tag{3}$$

Because the random variable maps an execution run to either 0 or 1, each execution run is a Bernoulli-trial and the probability of boundary conditions is binomially-distributed [5].

Depending on the input, SMC techniques are either *fixed-k* or *sequential*. Fixed-k SMC techniques take k execution runs and a significance level $\alpha \in [0, 1]$ as input to empirically estimate the expected value $\hat{p}$ of the probability that the boundary condition holds. Hence, $\hat{p} = \frac{\sum_\pi X(\pi)}{k}$ is the empirical estimation that counts how often a run satisfies the boundary condition in proportion to the total number of runs. Additionally, a confidence interval (CI) I with width $|I|$ around $\hat{p}$ is computed. A SMC technique is *sound* iff the real probability p lies within the estimated CI I of width $|I| \leqslant 2\epsilon$, where $\epsilon \in [0, 1]$ is the error, with probability above $1 - \alpha$ [5]. To illustrate, a typical value for ϵ is 2% and for α is 5%, while k highly depends on the real probability of the IoT system to be reliable. Sequential SMC techniques only take the significance level α and desired error ϵ as input and autonomously compute the necessary number of execution runs k. Lastly, a SMC technique is *inefficient* iff the CI width $|I|$ is either shown to be "unnecessarily" large [5] given k (fixed-k) or the required number of k is "unnecessarily" large (sequential).

Recently, it was shown that many SMC techniques rely on statistical measures that are either unsound, i.e., cannot guarantee statistical assurances for I, or inefficient, i.e., required a larger than necessary k [5,24]. We apply the recent results on soundness and efficiency to quantify statistical assurances for reliability as follows. For the fixed-k setting, we apply the recommended *Clopper-Pearson* interval [7] that is shown to be sound and efficient [5,24] for the probability measure in Eq. 3. For the sequential setting, we apply a similar sequential approach as [5] that precomputes the required k by using the Clopper-Pearson interval. However, instead of precomputing the k by assuming the worst-case probability[4] $p = 0.5$, we assume lower probabilities such as $p = 0.1$, because reliability is typically at least 90% likely. From the proof for soundness of the Clopper-Pearson interval in [24], we conclude that the required k monotonically decreases with lower p. Therefore, the required k we assume is significantly lower than for the worst-case probability.

All in all, applying the two sound and efficient SMC techniques enable reliability verification of IoT systems with statistical assurances on violating boundary conditions. By implementing our multi-paradigm DT service pattern in our service-oriented DT architecture (cf. Fig. 1), reliability verification through SMC is seamless, as the `control` process and, more importantly, its logging does not change (cf. Sect. 3.2). Because the logging service (cf. Sect. 3.1) that logs events while the design time or the runtime DT are executed is the same with the same output, there is no difference between runs from the design time and runtime DT. Moreover, both the design time and runtime DT are implemented with the same architecture, the same pattern, on the same network with equivalent

[4] In [24], it is proven that the width of I is largest iff the real probability p equal 0.5. Hence, applying a sequential approach with $p = 0.5$ results in the required k that is always large enough to yield sound statistical assurances.

infrastructure. Thus, not only are the two DTs conceptually and technically significantly more similar than with other approaches, but also the two sound and efficient SMC techniques can be seamlessly applied on runs from simulation and execution. Altogether, Twin-CPEE enables seamless reliability verification.

4.2 Reliability Verification Results

In this section, we implement three design time DTs for three IoT systems. Additionally, we consider a runtime DT of a fourth IoT system in [2]. We verify reliability of all four IoT systems with the fixed-k and sequential SMC techniques (cf. Sect. 4.1) as implemented in https://github.com/janikbenzin/twin-cpee. The four IoT systems were selected to cover all features of our DT service pattern, while balancing complexity, clarity, and comparability.

IoT Systems and Boundary Conditions. The first IoT system is the water kettle (cf. Fig. 3) with c_h. The second IoT system is a water tank with constant water inflow and a pump that can be turned on to prevent overflow and turned off to prevent drying up. The water tank IoT system is based on a common hybrid systems example from [15]. As both overflow and drying up must be prevented, the boundary condition is $c_w = 2 \leqslant level \leqslant 38\ \mathcal{U}_{\leqslant 20}\ true$, where $level$ is the water level sensor that measures the level inside the tank in litres. The water tank DT does not have a stochastic fail and repair, but instead contains a noisy sensor, i.e., the level sensor is normally-distributed with the actual $level$ as mean and 1l standard deviation. The third IoT system is an agricultural irrigation system that prevents crop on a field from withering. Hence, the boundary condition is $c_i = 0.18 \leqslant moisture\ \mathcal{U}_{\leqslant 24}\ true$, where $moisture$ is the soil moisture sensor that measures in the unit $\%/m^3$. Instead of a fail and repair or a noisy sensor, the irrigation DT has five environment processes that model plant transpiration, humidity, temperature, rain, and the hour of a day. Instead of executing over a whole day, we map 1s of execution time in the DT to 1h of the day through the corresponding environment process, i.e., the boundary condition is bound by a day. The fourth IoT system is an automated cotton candy machine [2]. To produce cotton candy and to prevent it from heating unevenly, the boundary condition on the head temperature (°C) inside the cotton candy machine is $c_c = 40 \leqslant head \leqslant 100\ \mathcal{U}_{\leqslant p}\ true$ with p the actual production time in seconds.

Setup and CPEE Execution Properties. Because all DTs are executed on www.cpee.org and not simulated, the execution time must be at least the time bound for verifying reliability. Hence, each design time DT is executed for the respective time bound: 20s for the water kettle and water tank DT, 24s for the irrigation DT, and between 15s and 1221s for the cotton candy DT. For the water kettle, we vary the initial water temperature starting with either 81 °C, 90 °C, or 99 °C. For the water tank, we vary the initial water level starting with either 11l, 20l, or 29l. Because the irrigation DT includes random rain throughout the day, we do not additionally vary the initial moisture. For each initial parameter, we execute each design time DT $k = 1000$ times (overall 7000 executions and runs π). Due to the manual effort, the runtime DT is executed $k = 175$ times with initial parameters as measured before execution.

Results. We estimate the probability of unreliability p and the CI through SMC (cf. Sect. 4.1). For fixed-k SMC, we take the available runs per DT and initial parameter respectively to empirically estimate $\hat{p}$ with the surrounding CI. In Table 2, the estimated probabilities and CIs are reported. The water kettle DT is not reliable to maintain a water temperature above 76° (cf. c_h) with $\hat{p} = 3\%$. As reliability is concerned with a boundary condition that should not be violated, reliability is derived from the worst-case probability with respect to different initial parameters. Hence, the $\hat{p} = 1.30\%$ and $\hat{p} = 2.30\%$ for the lower and higher initial parameter of 81 °C and 99° respectively are not considered. With at least 95% probability, the true probability p lies in the CI $[3.00\% - 1.11\%, 3.00\% + 1.11\%] = [1.89\%, 4.11\%]$. The water tank DT is not reliable to maintain a level between 2l and 38l with a probability of 4.20%. The true probability lies in the CI $[2.90\%, 5.50\%]$ with at least 95% probability. Next, the irrigation DT shows the best reliability with a probability of 0.40% for violating the boundary condition and a small CI of $[0.00\%, 0.86\%]$. Finally, the cotton candy DT exhibits the worst reliability with a probability of 8.57% for violating the boundary condition and a large CI of $[4.14\%, 13\%]$.

Table 2. Empirically estimated $\hat{p}$, significance level $\alpha = 5\%$.

Setting	k/ϵ	DT	Condition	$init_{low}$	$init_{mid}$	$init_{high}$
Fixed-k	1000	water kettle	c_h	$1.30\% \pm 0.76\%$	$\mathbf{3.00\% \pm 1.11\%}$	$2.30\% \pm 0.98\%$
	1000	water tank	c_w	$\mathbf{4.20\% \pm 1.30\%}$	$0.30\% \pm 0.41\%$	$0.10\% \pm 0.28\%$
	1000	irrigation	c_i	–	$\mathbf{0.40\% \pm 0.46\%}$	–
	175	cotton candy	c_c	–	$\mathbf{8.57\% \pm 4.43\%}$	–
Sequential ϵ	2.5%	water kettle	c_h	$2.19\% \pm 1.27\%$	$\mathbf{5.06\% \pm 1.85\%}$	$3.87\% \pm 1.64\%$
	2.5%	water tank	c_w	$\mathbf{7.08\% \pm 2.15\%}$	$0.51\% \pm 0.68\%$	$0.17\% \pm 0.47\%$
	2.5%	irrigation	c_i	–	$\mathbf{0.67\% \pm 0.77\%}$	–
	2.0%	water kettle	c_h	$1.42\% \pm 0.83\%$	$\mathbf{3.28\% \pm 1.21\%}$	$2.51\% \pm 1.08\%$
	2.0%	water tank	c_w	$\mathbf{4.60\% \pm 1.41\%}$	$0.33\% \pm 0.44\%$	$0.11\% \pm 0.30\%$
	2.0%	irrigation	c_i	–	$\mathbf{0.44\% \pm 0.50\%}$	–

For the sequential SMC, we set two different error values $\epsilon = 2.5\%$ and $\epsilon = 2.0\%$ such that it holds $|I| \leqslant 2\epsilon$ for the CI width $|I|$ (cf. Sect. 4.1). On the assumption of a 10% maximal probability of violating the boundary condition, we compute $k = 593$ and $k = 914$ respectively with the Clopper-Pearson interval. As the cotton candy DT has only 175 runs, we do not estimate probabilities using sequential SMC in this case. In Table 2, the estimated probabilities for unreliability using sequential SMC are reported. The water kettle DT, the water tank DT, and the irrigation DT are not reliable with probability $\hat{p} = 5.06\%$, $\hat{p} = 7.08\%$, and $\hat{p} = 0.67\%$ for $\epsilon = 2.5\%$. Consequently, the respective CIs are never larger than 5%. For $\epsilon = 2.0\%$, the respective probabilities decrease to $\hat{p} = 3.28\%$, $\hat{p} = 4.60\%$, and $\hat{p} = 0.44\%$. By assuming a reliability threshold of

$\theta = 5\%$ (cf. Sect. 4.1), we conclude that the water kettle and irrigation DT are reliable, while the water tank and cotton candy DTs are not reliable. Since design time DT is executed on the same host (www.cpee.org) with the same network as a runtime DT would, the reliability results are expected to also hold for the runtime DT. In addition, implementing the corresponding runtime DT is easy, as only a few minor changes must be applied (cf. Sect. 3.2). Lastly, the design time DT meets the six requirements in Table 1 by taking a multi-paradigm approach to combine BPMN with SHS. Next, we present the recording of IoT-enriched logs through the logging service (cf. Fig. 1).

Recording IoT-Enriched Logs: As the DT architecture in Fig. 1 contains a logging service, both design and runtime DTs implemented with our service pattern enable recording of IoT-enriched logs in the DataStream XES extension [23]. IoT-enriched logs are event logs that not only contain information on what (script) tasks of the DT are executed at what time, but also what variables are read or written. An event recorded from executing the water kettle DT (cf. Fig. 3) could, for example, show that script task "Heat" computed a new temperature value of 85.79 °C by computing the solution of the Newton's law of heating ODE (cf. Sect. 3.2). By taking the whole IoT-enriched log, log analysis techniques from the process mining community can be applied [12].

5 Conclusion, Limitations, and Future Work

Twin-CPEE as a multi-paradigm DT approach seamlessly integrates a formal model (SHS) to simulate DT behaviour at design time and DT implementation to dynamically represent (BPMN) and control IoT systems at runtime (service orchestration and execution). In particular, Twin-CPEE enables seamless reliability verification through SMC that uses the same logging service at design time and runtime. Twin-CPEE has the following limitations. First, the mode-based representation of control can become hard to understand for a large number of modes. Second, the design time DT of Twin-CPEE requires knowledge on the physical dynamics of the IoT system to determine the ODE and is limited to the ODEs fidelity. Third, Twin-CPEE does not consider *partial* differential equations (PDEs) for which no analytical solution can be found. To improve Twin-CPEE, we will investigate modularization of control modes into subprocesses, discovery of ODEs from sensor time series, and include numerical solutions to PDEs.

References

1. Appel, S., Kleber, P., Frischbier, S., Freudenreich, T., Buchmann, A.: Modeling and execution of event stream processing in business processes. Inf. Syst. **46**, 140–156 (2014)
2. Arteaga Garcia, N., Mangler, J.: Cotton Candy XES YAML (Power Consumption, Environment, Temperatures, Metrology) (2025). https://zenodo.org/records/17226615

3. Ballarini, P., Barbot, B., Duflot, M., Haddad, S., Pekergin, N.: HASL: a new approach for performance evaluation and model checking from concepts to experimentation. Perform. Eval. **90**, 53–77 (2015)
4. Barišić, A., et al.: Multi-paradigm modeling for cyber-physical systems: a systematic mapping review. J. Syst. Soft. **183**, 111081 (2022)
5. Budde, C.E., Hartmanns, A., Meggendorfer, T., Weininger, M., Wienhöft, P.: Sound statistical model checking for probabilities and expected rewards. In: TACAS, pp. 167–190. Springer (2025)
6. Cheng, Y., Zhao, S., Cheng, B., Chen, X., Chen, J.: Modeling and deploying IoT-aware business process applications in sensor networks. Sensors **19**(1) (2019)
7. Clopper, C.J., Pearson, E.S.: The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika **26**(4), 404–413 (1934)
8. Corradini, F., Pettinari, S., Re, B., Rossi, L., Tiezzi, F.: A BPMN-driven framework for Multi-Robot System development. Robot. Auton. Syst. **160**, 104322 (2023)
9. Corradini, F., Pettinari, S., Re, B., Rossi, L., Tiezzi, F.: Executable digital process twins: towards the enhancement of process-driven systems. Big Data Cognit. Comput. **7**(3), 139 (2023)
10. Dar, K., Taherkordi, A., Baraki, H., Eliassen, F., Geihs, K.: A resource oriented integration architecture for the Internet of Things: a business process perspective. Pervasive Mob. Comput. **20**, 145–159 (2015)
11. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Poulsen, D.B.: Uppaal SMC tutorial. IJSTTT **17**(4), 397–415 (2015)
12. De Luzi, F., Leotta, F., Marrella, A., Mecella, M.: On the interplay between business process management and internet-of-things. BISE **67**(2), 265–288 (2025)
13. Fedeli, A., Fornari, F., Polini, A., Re, B., Torres, V., Valderas, P.: FloBP: a model-driven approach for developing and executing IoT-enhanced business processes. SoSyM **23**(5), 1217–1246 (2024)
14. Gockenbach, M., Schmidtke, K.: Newton's law of heating and the heat equation. Involve J. Math. **2**(4), 419–437 (2009)
15. Hahn, E.M., Hartmanns, A., Hermanns, H., Katoen, J.P.: A compositional modelling and analysis framework for stochastic hybrid systems. Form. Methods Syst. Des. **43**(2), 191–232 (2013)
16. He, S., He, P., Chen, Z., Yang, T., Su, Y., Lyu, M.R.: A survey on automated log analysis for reliability engineering. ACM CSUR **54**(6), 130:1–130:37 (2021)
17. Jensen, K., Kristensen, L.M., Wells, L.: Coloured petri nets and CPN tools for modelling and validation of concurrent systems. STTT **9**(3), 213–254 (2007)
18. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) CAV 2011. LNCS, vol. 6806, pp. 585–591. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22110-1_47
19. Lavaei, A., Soudjani, S., Abate, A., Zamani, M.: Automated verification and synthesis of stochastic hybrid systems: a survey. Automatica **146**, 110617 (2022)
20. Lehner, D., et al.: Model-driven engineering for digital twins: a systematic mapping study. SoSyM **24**(5), 1339–1377 (2025)
21. Mandal, S., Hewelt, M., Weske, M.: A framework for integrating real-world events and business processes in an IoT environment. In: Panetto, H., et al. (eds.) OTM 2017. LNCS, vol. 10573, pp. 194–212. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-69462-7_13
22. Mangler, J., Rinderle-Ma, S.: Cloud process execution engine: architecture and interfaces. Technical report (2022). http://arxiv.org/abs/2208.12214

23. Mangler, J., et al.: Datastream XES extension: embedding IoT sensor data into extensible event stream logs. Fut. Int. **15**(3), 109 (2023)
24. Meggendorfer, T., Weininger, M., Wienhöft, P.: What are the odds? Improving the foundations of statistical model checking. Technical report arXiv:2404.05424 (2025)
25. Mottola, L., et al.: makeSense: simplifying the integration of wireless sensor networks into business processes. IEEE TSE **45**(6), 576–596 (2019)
26. Pilch, C., Remke, A.: Statistical model checking for hybrid Petri nets with multiple general transitions. In: IEEE/IFIP DSN, pp. 475–486 (2017)
27. Seiger, R., Franceschetti, M., Abbad-Andaloussi, A.: A process to non-invasively augment legacy IoT systems using business processes and microservices. In: ACM IoT, pp. 1–9 (2025)
28. Valderas, P.: Supporting the implementation of digital twins for IoT-enhanced BPs. In: RCIS, pp. 222–238. Springer (2023)
29. Valderas, P., Torres, V., Serral, E.: Modelling and executing IoT-enhanced business processes through BPMN and microservices. J. Syst. Softw. **184**, 111139 (2022)
30. Voigt, D., Podszun, L., Mangler, J., Rinderle-Ma, S.: HiProX: highly efficient process execution on IoT and edge devices. In: BPM: Responsible BPM Forum, Process Technology Forum, Educators Forum, pp. 184–199. Springer (2026)
31. Wehlitz, R., Rößner, I., Franczyk, B.: Integrating smart devices as business process resources – concept and software prototype. In: Braubach, L., et al. (eds.) ICSOC 2017. LNCS, vol. 10797, pp. 252–257. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-91764-1_20

Detecting AI-Assisted Tampering in Crowdsourced IoT Service Trust Information

Thilina Lokuruge(✉) and Athman Bouguettaya

The University of Sydney, Sydney, NSW 2006, Australia
{thilina.thilakarathna,athman.bouguettaya}@sydney.edu.au

Abstract. We propose a distributed trust information management framework for crowdsourced IoT services. The crowdsourced IoT service environment consists of distributed entities that store and manage trust information. Traditional trust management frameworks often assume the trustworthiness of these entities. However, they may tamper with trust data, making the system vulnerable to internal attacks. The rise of AI tools, such as ChatGPT, has further lowered the barrier for adversaries, enabling even low-skilled actors to manipulate trust information with alarming sophistication. AI-assisted tampering could severely compromise trust assessments and mislead IoT service users. To counter this emerging threat, we propose a novel AI-based tampering detection approach, complemented by a heuristic-based approach to identify tampered trust information. We conduct a series of experiments to evaluate the effectiveness of the proposed approaches. The results demonstrate more than 95% accuracy, confirming the effectiveness of our approach.

Keywords: Crowdsourced IoT services · Trust information management · AI-assisted tampering · Tampering detection

1 Introduction

The Internet of Things (IoT) comprises an interconnected system of devices, such as smartphones and wearables. IoT devices are typically capable of sensing, networking, and processing. These capabilities allow IoT devices to collaborate and share resources among each other [3]. For instance, a smartphone (*service provider*) may provide some of its computational resources to a smartwatch (*service consumer*). The smartwatch may use those resources to perform computationally intensive tasks, such as rendering a map. This concept of inter-device resource sharing is commonly known as IoT service crowdsourcing [10]. IoT devices may crowdsource several services, including wireless energy services [2], computational power sharing services [15], and Wi-Fi hotspot services [19].

The crowdsourced IoT environment typically consists of confined areas where IoT service providers and consumers congregate (i.e., *microcells*) [4]. Examples of

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 191–206, 2026.
https://doi.org/10.1007/978-981-95-5015-9_14

microcells include restaurants, cafes, movie theaters, and lecture halls. Service-Oriented Architecture (SOA) is often adopted to enable IoT service crowdsourcing in microcells, particularly in the context of smart cities and campuses [1]. In this context, SOA comprises three main components: service providers, service consumers, and *super-providers* (i.e., the microcell owners who facilitate IoT service crowdsourcing).

IoT devices may misuse crowdsourced IoT services for malicious purposes [7]. For instance, a service consumer might exploit a Wi-Fi hotspot service to launch Denial-of-Service (DoS) attacks. Similarly, a service provider could distribute malware during a computational power-sharing service. Extensive research has proposed Trust Management (TM) as a means to establish and maintain trust in crowdsourced IoT service environment [6,9,17]. These approaches have introduced several robust trust assessment approaches; however, *ensuring the trustworthiness of the trust information remains a significant challenge.* Trust information refers to the data used for trust assessment [7]. For instance, consumer feedback on an IoT service could be considered as trust information [16]. Trust information could be managed in a centralized, semi-centralized, or distributed approach [17]. However, centralized and semi-centralized systems face scalability limitations and are prone to single points of failure [17,25]. We assume that trust information is stored locally within super-provider registries in a distributed manner. This assumption is further justified by privacy and legal constraints.

Most existing approaches assume that super-providers, who store trust information, are inherently trustworthy [7]. Thus, these systems are vulnerable to internal attacks. Malicious or compromised super-providers may tamper with the trust data. For example, a super-provider may inflate trust scores of service providers to attract more consumers. It is highly unlikely that a central authority exists in the crowdsourced IoT environment due to the scale and decentralized nature. Thus, there is no monitoring or auditing mechanisms to ensure the integrity of trust information [7]. Consequently, the trustworthiness of trust data could not be guaranteed.

The emergence of AI tools, particularly large language models (LLMs) (e.g., ChatGPT, DeepSeek, and Gemini), has further intensified the risk of trust data tampering. A malicious super-provider could manipulate trust data with a simple prompt. Traditionally, trust data tampering would require domain knowledge and expertise in relevant tools. However, modern LLMs enable scalable and impactful data manipulation without requiring any understanding of the underlying data [12,21]. Furthermore, prompt engineering techniques may enhance the realism of tampered data, making detection increasingly difficult. To address this issue, we propose an approach to detect AI-assisted tampering by super-providers. Specifically, we employ a transformer-based approach to identify tampered trust information records. The main contributions of this paper are as follows:

– A novel *AI-based Tampering Detection* (AITD) approach to identify AI-assisted tampering in trust data.

– A novel *Heuristic-Based Tampering Detection* (HBTD) approach to detect inconsistencies in trust data.

Note that blockchain-based approaches could offer data integrity [24]. However, they are not well-suited for managing trust in crowdsourced IoT environment due to privacy, scalability, and resource constraints. For example, super-providers may store or share only a subset of trust information due to the privacy requirements of service providers and consumers. Moreover, super-providers may share trust information exclusively with certain microcells based on their preferences. This contrasts with blockchain solutions, where all transactions are shared among all participating nodes [23]. Furthermore, users are identified by their public keys or hashes in blockchain systems [13]. If blockchain is used in a crowdsourced IoT environment, shared trust data may expose sensitive user data, such as location. Additionally, its high computational and connectivity demands exceed the capabilities of many IoT devices [14]. Although encryption could protect data in transit from external attacks, it could not prevent internal tampering by super-providers after decryption. Therefore, we adopt a distributed trust management approach that does not rely on blockchain.

2 Preliminaries

In this section, we first present a motivating scenario to highlight the problem of trust data tampering. We then introduce the notions of IoT services and trust information records to illustrate the concept of trust information management. Finally, we discuss the trust assessment process and the storage of trust information.

2.1 Motivating Scenario

Consider ten microcells participating in an IoT crowdsourced environment, one of which is Joe's coffee shop (M109) (Fig. 1). Suppose a service provider, P1003181, advertises a service within Joe's microcell. Before utilizing the service, a consumer, C00164, requests a trust evaluation for P1003181 from the Trust Management System (TMS) (Fig. 1(C)).

Initially, the TMS retrieves trust data from the local super-provider, i.e., Joe. TMS may collect additional trust information from remote super-providers to enhance the accuracy of the trust assessment (Fig. 1(B)). IoT service providers and consumers are mobile and may crowdsource services across multiple microcells. Thus, trust information about those devices could be obtained from the super-providers of those microcells.

Suppose four remote microcells possess trust records related to provider P1003181 and share them with Joe's microcell. Mary, the owner of a nearby coffee shop, is among these selected remote microcells. Assume that Mary considers Joe a competitor. Mary may deliberately share tampered trust data related to P1003181. Mary may use AI tools such as ChatGPT to tamper with trust

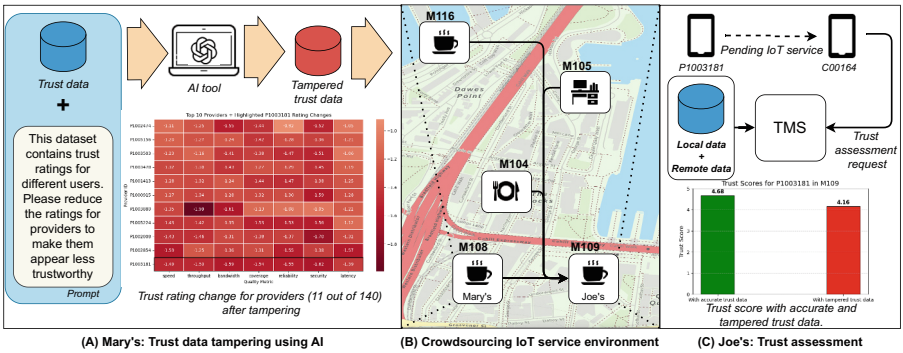


Fig. 1. IoT service crowdsourcing scenario

records (Fig. 1(A)). A simple prompt such as: "This dataset contains trust ratings for different users. Please reduce the ratings for providers to make them appear less trustworthy" could alter trust data and have a considerable impact on trust assessment (Fig. 1(A)). The TMS evaluates P1003181's trustworthiness using local and received trust data (Fig. 1(C)). P1003181 may appear less trustworthy due to the inclusion of tampered data, potentially lose service-providing opportunities, and may ultimately withdraw from offering services in Joe's microcell. In a competitive environment, even a slight reduction in trust score could lead to IoT service rejection.

2.2 Definition 1: IoT Service (S)

We represent a crowdsourced *IoT service* S that is provided by an IoT device, with a tuple of $< SID, m, sp, t, f >$ where,

- SID is a unique service ID.
- m is the microcell where the service S is provided.
- sp is the super-provider that manages the microcell m.
- t is the time interval during which the service S is provided. It is represented as a tuple of the start and end time of S $< ts, te >$.
- f is the function (service type) offered by S (e.g., providing Wi-Fi hotspots).

2.3 Definition 2: Trust Information Record (TIR)

A TIR is a record containing consumer's ratings of an IoT service. Consumer C will rate the IoT service S provided by service provider P after the service usage. A TIR contains identification details about the device and service. The TIR model is represented as a tuple of $< PID, CID, S, A >$ where,

- PID is a unique provider device ID.
- CID is a unique consumer device ID.

– S is the IoT service provided by the service provider.
– A is a set of $\{a_1, a_2, a_3, ..., a_n\}$ where a_i is an attribute of S that influences the trust in an IoT service (e.g., connection security and reliability). Each attribute contains the *level of satisfaction rating* from the service consumer.

For example, let us assume a scenario where a consumer finished consuming a crowdsourced Wi-Fi hotspot service. The TIR might look like Fig. 2(D).

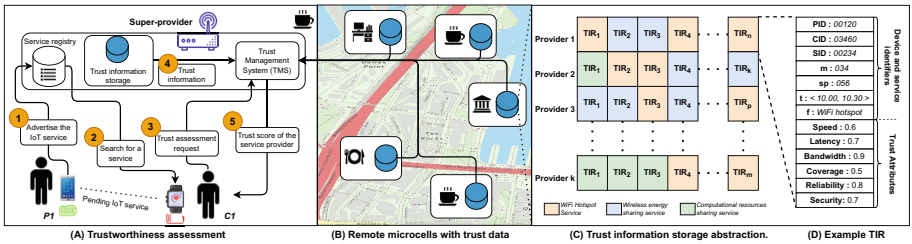


Fig. 2. Trust assessment flow and trust data storage

2.4 Trust Assessment

Service providers advertise their IoT service offerings to the super-provider in the microcell (Fig. 2(A)). Super-provider lists these services in a registry. Service consumers browse this registry to select services that meet their requirements. Before initiating an IoT service transaction, both the consumer and provider must assess each other's trustworthiness. For instance, suppose consumer C1 wishes to access a wireless energy service (S1) provided by P1 (Note: Although this example focuses on wireless energy services, the process applies to any IoT service type). To proceed, C1 requests the TMS to evaluate P1"s trust score[1] [8]. TMS typically use models such as weighted average, usage-based, or multi-perspective approaches [6,9,17]. In this work, we adopt a weighted average model for simplicity. Since a provider may operate across multiple microcells, trust information about P1 may be distributed across several microcells. The TMS first retrieves relevant trust data from the local super-provider and may request additional data from remote microcells to enhance accuracy (Fig. 2(B)). Once trust data is gathered, the system computes P1's trust score and shares it with C1. If the score is satisfactory, C1 proceeds to consume the service. After the service is delivered, C1 provides feedback, which the TMS uses to generate a TIR that is stored by the local super-provider.

[1] For simplicity, we evaluate only the trustworthiness of service providers.

2.5 Trust Information Storage

Several service providers and consumers may crowdsource multiple IoT services within a microcell. Thus, we assume that the trust information storage, i.e., the super-provider registry, contains trust data related to several IoT service providers (Fig. 2(C)). A single service provider may offer multiple IoT services. Therefore, there could be multiple TIRs associated with the provider. These records are organized as a Trust Information Sequence (TIS), defined as a set of $TIR_1, TIR_2, TIR_3, ...TIR_n$, where each TIR_i represents a time-stamped trust record. A TIS thus captures the time-series trust data related to all IoT services offered by a provider within a microcell. A subset of TIRs related to a specific IoT service type (e.g., Wi-Fi hotspot) could be extracted from the TIS. We define this subset as the Service Trust Information Sequence (STIS).

3 Trust Information Tampering

Trust data stored in microcell registries is accessed through four primary operations: create, read, update, and delete TIRs. These operations could be either active or passive. Read is a passive operation that does not alter the data. Active operations (i.e., create, update, and delete) could modify trust data and are essential for maintaining an efficient trust management system. For example, the system may create new TIRs based on consumer feedback, update records to ensure consistency with system standards, or delete outdated records to optimize storage efficiency. However, super-providers may exploit these active operations to tamper with trust information. They may create fake trust records, modify existing records to artificially inflate or reduce trust scores, or delete records to obscure unfavorable historical data. This makes the trust management system vulnerable to internal attacks. In this work, we assume that any active operation initiated by a super-provider is potentially malicious.

Definition of Tampering. We define tampering as any unauthorized and active operation performed by a super-provider on trust information that causes a change (either an increase or decrease) in the trust score of a provider for a given IoT service type. Tampering is identified when two sequential states of STIS, at time $t1$ and $t2$, produce a different output from the trust assessment function F, despite no legitimate system-triggered updates occurring within that time interval. In such a case, the modification must have been introduced by a super-provider, either through the creation, deletion, or update of TIRs. Formally, tampering is defined as follows:

$$\exists STIS(t2) \neq STIS(t1)) \mid F(STIS(t2)) - F(STIS(t1)) \mid > 0$$

Given that,

- There exists at least one $TIR_j \in STIS$ such that one of the following has occurred between time t_1 and t_2:

- $TIR_j(t_2)$ is an updated version of $TIR_j(t_1)$,
- $TIR_j(t_2)$ has been newly added and did not exist at t_1,
- $TIR_j(t_1)$ existed but is missing from $STIS(t_2)$, i.e., it was deleted.

– No legitimate system operations recorded to justify a change to $STIS$.

F represents the trust assessment function, which depends on the selected trust assessment approach for the environment (See Sect. 2.4). In our experiments, we implemented a weighted average model as the trust assessment function.

Motivation for Tampering. Super-providers may manipulate trust information based on two primary motivations: (1) to enhance the reputation of their own microcell, and (2) to tarnish the reputation of competing microcells. A malicious super-provider may carry out various trust-related attacks in an IoT service environment. We identify six common types of attacks [17,26]: (a) Self-Promoting Attacks (SPA), (b) Bad-Mouthing Attacks (BMA), (c) Ballot-Stuffing Attacks (BSA), (d) Whitewashing Attacks (WA), (e) Discriminatory Attacks (DA), and (f) Opportunistic Service Attacks (OSA). SPA and BMA are particularly prominent in microcell-based environment. In SPA, a super-provider manipulates trust records to make IoT services within their own microcell appear more trustworthy, thereby attracting more consumers and enhancing the reputation of their microcell. In BMA, the super-provider intentionally reduces the trust ratings of targeted service providers and shares the tampered data with remote microcells. As a result, these providers appear less trustworthy, reducing their participation in service offerings. Over time, such providers may refrain from offering services and returning to those microcells, resulting in decreased foot traffic in the microcell.

AI-Assisted Tampering. We define AI-assisted tampering as any manipulation of trust data by a super-provider using AI tools. This includes the use of publicly available interfaces such as ChatGPT, as well as the deployment of customized, domain-specific models. Manual or script-based tampering typically requires technical expertise or domain knowledge. However, AI-assisted tampering is significantly more accessible. A user with minimal technical knowledge could upload a trust dataset into an AI interface, enter a basic prompt such as: *Reduce ratings for providers to make them appear less trustworthy*, and receive altered data that could then be integrated into the trust registry (See Sect. 2.1). This ease of use dramatically lowers the barrier to executing impactful and difficult-to-detect tampering.

Figure 3 illustrates the AI-assisted tampering process. The trust data is input into an AI model along with a tampering prompt. The output is a manipulated version of the dataset, customized according to the prompt's instructions. We identify three key factors that could be controlled through the prompt, effectively serving as variables in the tampering process:

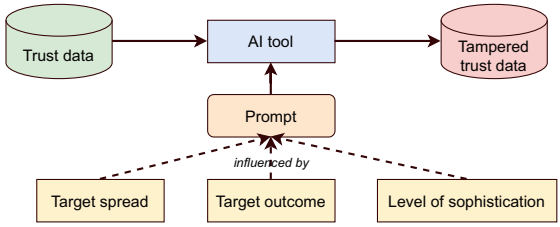


Fig. 3. AI-assisted tampering process

1. **Target Spread** – the number of TIRs that are modified, created, or deleted (e.g., affecting a single provider, a subset of providers, or all providers).
2. **Target Outcome** – the intended direction of manipulation (e.g., increase or decrease in trust ratings).
3. **Level of Sophistication** – the complexity of alterations.

Super-providers could improve the effectiveness of tampering by leveraging prompt engineering techniques. Examples of such techniques include role assignment, few-shot prompting, and chain-of-thought reasoning [22]. These techniques allow super-providers to fine-tune key dimensions of tampering (i.e., target spread, outcome, and sophistication level). Thereby maximizing the impact of tampering while minimizing the likelihood of detection.

4 Proposed Framework

We propose a Trust Information Management Framework (TIMF) to enhance the trustworthiness of trust information used for trust assessment. The framework comprises two core components: (1) an AI-based trust information tampering detection approach (i.e., AITD) and (2) a heuristics-based trust information tampering detection approach (i.e., HBTD) (Fig. 4). We employ both AITD and HBTD to effectively detect tampering in trust information. The AITD approach leverages learned patterns from trained data to identify subtle and complex tampering behaviors. In contrast, HBTD relies on domain knowledge, such as the spread of tampering and log inconsistencies. The intuition behind combining these approaches is that heuristics serve as an explainable post-step for data-driven AI-based tampering detection. These approaches combined offer robust and adaptive tampering detection.

4.1 AI-Based Trust Information Tampering Detection Approach

The AITD approach is designed to learn and differentiate subtle patterns in trust data (i.e., trust attribute ratings) between tampered and untampered records. The neural network architecture used in this approach has two main components: (1) an encoder network and (2) a classifier network. The encoder has two fully connected (linear) layers with ReLU activations and outputs an 8-dimensional

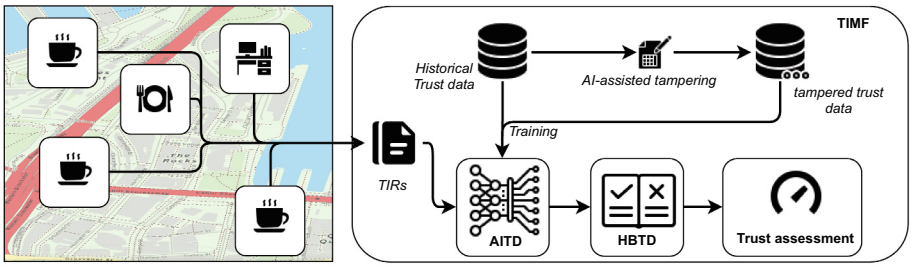


Fig. 4. Trust Information Management Framework

latent representation. The classifier then takes this vector and passes it through another fully connected neural network consisting of a hidden layer with ReLU activation and a final output layer with a sigmoid activation function. This produces a probability score indicating the likelihood that a given record has been tampered with. The model is trained on a labeled dataset containing both known tampered and untampered records. The tampered records were generated by augmenting the trust data using ChatGPT-4o, guided by a variety of prompts. These prompts were designed based on three key dimensions: target spread, target outcome, and level of sophistication (See Sect. 3).

4.2 Heuristics-Based Trust Information Tampering Detection Approach

Algorithm 1. HBTD Approach

Require: $combined_data$ (received + local data) with tam_label_ai, log_data
Ensure: $merged_data$ with final $tampered_label$
1: **for** each microcell mic in $combined_data$ **do**
2: Compute $tampered_ratio(mic)$: proportion of records with $tam_label_ai = 1$
3: Set $hu1 \leftarrow 1$ if $tampered_ratio(mic) > \tau$, else 0 (e.g., $\tau = 0.5$)
4: **end for**
5: Merge $combined_data$ with log_data on (serviceid, timestamp, microcell)
6: Set $hu2 \leftarrow 1$ if record is missing in log_data, else 0
7: Initialize $tampered_label \leftarrow hu1$
8: If $hu1 = 0$ and $hu2 = 1$, then set $tampered_label = 1$
9: **return** $merged_data$ with $tampered_label$

The HBTD approach aims to identify potential missed or incorrect tampering detection by the AITD approach. Algorithm 1 outlines the step-by-step process of this heuristic-based detection. Service provider devices may maintain logs of their service history. In our setting, we assume access to these provider-side logs. The AITD approach classifies trust data as either tampered or untampered. HBTD builds on this by leveraging the AITD output together with the

service logs to further enhance tampering detection. Lines 1–4 implement Heuristic 1: *target spread*, which identifies concentrated tampering within a microcell. A malicious super-provider may manipulate the majority of records to maximize impact. This helps filter out records incorrectly flagged by AITD. Lines 5–6 apply Heuristic 2: *log inconsistencies*, which cross-checks trust data against logs to detect fabricated or deleted records. A record that appears in the trust data but not in the logs could be fabricated. Conversely, missing records that are present only in the logs may indicate deletions. While deletions could be detected through this heuristic, our current focus is on identifying tampered data within the received trust data. Lines 7–8 finalize the labels: Heuristic 1 provides the initial assignment, and Heuristic 2 flags additional inconsistencies among non-tampered records, enhancing overall detection accuracy.

5 Experiments and Results

We evaluate the proposed approaches through an extensive set of experiments. This section first outlines the experimental design, then describes the dataset and settings used, and finally presents the results of the proposed tampering detection methods. The experiments consist of two main parts. First, we analyze a set of prompts to understand their effect on trust data tampering. Thus, demonstrating the potential impact of AI-based manipulation. Second, we assess the effectiveness of the proposed detection methods (AITD and HBTD) using two experimental strategies: (1) We systematically vary the proportion of tampering super-providers from 10% to 90% and evaluate detection performance using standard metrics, including accuracy, precision, and recall. (2) We conduct a study to assess the individual contributions of AITD and HBTD. All experiments were implemented in Python and executed on a MacBook Air equipped with an Apple M2 chip, 8 GB of RAM, and an 8-core CPU.

5.1 Dataset Description

We use a real-world dataset from Airbnb.com[2] to evaluate the proposed framework. The cleaned dataset is available as the Airbnb Listings Dataset[3] and contains information on Airbnb hosts, locations, rating timestamps, and rating details. We extracted Service ID and Provider ID from host-related information. Location and timestamp attributes were utilized to derive spatial-temporal features. We grouped nearby hosts to define microcells based on latitude, longitude, and neighborhood attributes. Table 1 presents the extracted fields, while Table 2 summarizes experimental parameters. A provider may offer services across multiple microcells, leading to the distribution of trust information among them. To mimic this behavior, we identified hosts with multiple Airbnb accommodations as providers operating across different microcells.

[2] https://www.airbnb.com.
[3] http://insideairbnb.com/get-the-data.

Table 1. Feature extraction from the dataset

Field	Mapped field
Host ID	Provider ID
ID	Service ID
Last Rating	Timestamp
Rating scores	Trust Attributes Ratings
Latitude Longitude Neighborhood	Microcell

Table 2. Statistics of the crowdsourced IoT environment

Parameter	Value
Number of trust information records	144741
Number of service providers	3958
Number of microcells	289

We selected the Airbnb Listings dataset for our experiments as it realistically reflects the dynamics of an IoT crowdsourcing environment. Both ecosystems share fundamental similarities, including their architecture, reliance on the sharing economy paradigm, the concept of trust, and their decentralized nature. Both environments involve interactions between service providers (hosts in Airbnb) and consumers (guests in Airbnb). The processes of advertising, searching, and invoking services are inherent to both systems. Moreover, Airbnb and crowdsourced IoT platforms operate within the sharing economy framework, where resource sharing is essential for mutual benefit. In Airbnb, hosts share their homes with travelers, while in crowdsourced IoT, service providers share computational or network resources with consumers within the ecosystem. Additionally, in Airbnb, trust is established through user ratings and reviews, which help both hosts and guests assess reliability before engaging in a transaction. Similarly, in the crowdsourced IoT environment, service providers and consumers rely on trust scores to evaluate IoT service reliability before initiating interactions. Both environments also feature decentralized systems in which entities contribute resources to a broader network of consumers. In Airbnb, accommodation services are distributed across various hosts, whereas in crowdsourced IoT, services such as Wi-Fi hotspots or computational power are shared among multiple providers. These similarities make the Airbnb dataset suitable for modeling and evaluating trust in IoT service crowdsourcing scenarios.

5.2 Framework Evaluation

In experiment 1, we investigate how target spread, target outcome, and level of sophistication influence trust data tampering. To explore this, we designed five tampering prompts: *(a) P1: Randomly increase all trust ratings by 20–50% for all providers (capped at 5), (b) P2: Randomly reduce all trust ratings by 20–50%*

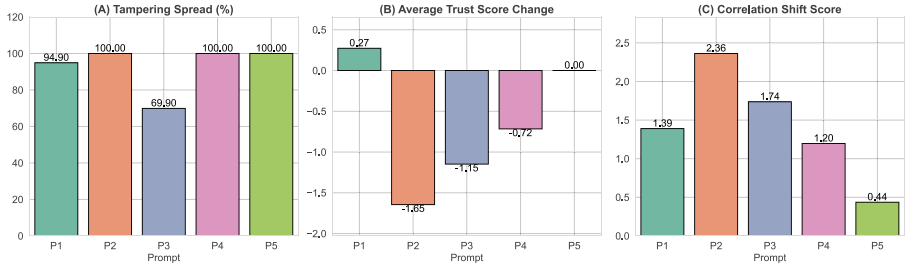


Fig. 5. Effect of prompt on tampering

for all providers, (c) P3: Randomly reduce all trust ratings by 20–50% for 70% of providers, (d) P4: Reduce trust ratings of selected high-impact features (e.g., reliability, security, coverage) by 20–40%, while leaving other features mostly unchanged to maintain partial plausibility, and (e) P5: Reduce trust ratings while preserving feature correlations, making detection more challenging.

P3 shows a tampering spread of 69.9%, indicating that the target spread of tampering could be effectively controlled using the prompt (Fig. 5). P1 results in an increase in trust scores, while P2, P3, and P4 produce negative changes. This validates the ability of prompts to influence the outcome of tampering (i.e., increase vs. decrease in ratings). P5 instructs the model to preserve feature correlations during tampering and achieves the lowest correlation change score (0.436), demonstrating that the level of sophistication in tampering could also be controlled through prompting. Overall, the experiment validates that prompts could effectively manipulate key tampering dimensions (i.e., target spread, target outcome, and level of sophistication).

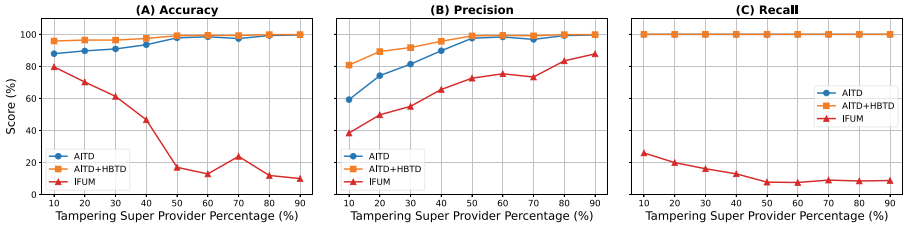


Fig. 6. AITD and HBTD vs IFUM

Experiment 2 demonstrates the effectiveness of the proposed AITD and HBTD approaches. We used a separate portion of the dataset that was not involved in model training for the validation. We filtered nine datasets with 10% to 90% super-providers (i.e., microcells). We kept the complement data separately. We uploaded the datasets to ChatGPT-4o along with the prompt:

Reduce all trust ratings (speed, throughput, bandwidth, coverage, reliability, security, latency) as follows: (1) Decrease the existing ratings by 20–50%. (2) Create new records with the same microcells and providers, assigning new trust ratings that result in lower overall combined scores than the original. We then downloaded the generated datasets, concatenated them with the complementary clean data, and used this combined data in our experiments.

To the best of our knowledge, no existing methods are specifically designed to detect AI-assisted tampering in trust data. The most relevant work we identified is presented in [20], which proposed an unsupervised machine learning approach, Isolation Forest-based Unsupervised Machine Learning (IFUM), for detecting sensor tampering through network traffic patterns. We used this method as a baseline for comparison in our experiments. As shown in Fig. 6, the combined approach (i.e., AITD and HBTD) achieves near-optimal performance, with accuracy and recall close to 100% and precision around 90%. AITD alone also performs well and achieves comparable results. However, the hybrid approach provides a slight improvement in both accuracy and precision. In contrast, IFUM exhibits reduced accuracy and recall as the number of tampering super-providers increases. This is because, as tampered TIRs begin to dominate the data distribution, it becomes more difficult for unsupervised methods to distinguish them as anomalies. Our approach shows slightly lower precision (≈80%) when only 10% of super-providers are tampering. Precision improves as the proportion of tampered records increases. This indicates that our approach mislabels some correct TIRs as tampered (i.e., false positives). However, removing untampered records (usually a few) has minimal impact on trust assessment compared to the risk of retaining tampered records. Note that recall reflects the ability to detect actual tampered records. A near-perfect recall demonstrates that our approach identifies nearly all tampered TIRs. Thus, the filtered trust data remains highly trustworthy for trust assessment. The complete set of experiments is available in the following GitHub repository: https://github.com/Thilina-lokuruge/ai-tampering-detection.git

6 Related Work

Trust Management (TM) systems in IoT typically encompass processes such as data collection, trust evaluation, decision-making, and trust updating. Recent literature emphasizes trust assessment, often using historical records and behavioral patterns to estimate the trustworthiness of service providers. For instance, Chen et al. [11] proposed a TM system within SOA to identify misbehaving IoT entities. SC-Trust [5] evaluates composed services using interaction-based evidence. Other models, such as [8], leverage usage patterns and historical data. TM systems are classified as value-based (e.g., weighted average models) or value-free (e.g., cryptographic-based methods). Value-based models depend on numerical evidence and thresholds, while value-free models emphasize security guarantees. Despite their robustness, these systems often overlook the trustworthiness of trust information [17].

Few approaches have been proposed to preserve the integrity of trust information in crowdsourced IoT service environments [7,18]. In [18], a semi-centralized trust management architecture based on blockchain is proposed. The environment consists of cloud servers and IoT devices exchanging data services. The cloud servers form a decentralized blockchain network, while IoT devices connect to these servers in a centralized manner. However, this approach assumes that the cloud servers are *trustworthy* for managing and assessing trust data. A distributed integrity-preserving framework for storing trust information in the crowdsourced IoT environment is proposed in [7]. This approach uses the mobility patterns of IoT devices to identify a set of microcells and deploy a scoped blockchain for storing trust information. However, it assumes static movement patterns for IoT devices. Tools like GPTZero and ZeroGPT have been developed to detect AI-generated content, primarily in natural language text. However, they do not support structured, tabular data formats such as trust records. Thus, their effectiveness in structured, tabular domains like trust records is unclear.

7 Conclusion

We propose a management framework for trust information in the crowdsourced IoT service environment. We introduce a novel AI-assisted Tampering Detection (AITD) approach to detect tampered records in trust data. In addition, we present a complementary Heuristic-Based Tampering Detection (HBTD) approach designed to enhance overall tampering detection accuracy. Experimental results show that our combined approach achieves an accuracy of around 95%, with a high precision and recall, significantly outperforming existing baselines. Future work will explore strategies for the prevention and resolution of trust information tampering.

Acknowledgments. This research was partly made possible by DP220101823 and LE220100078 grants from the Australian Research Council. The statements made herein are solely the responsibility of the authors.

Disclosure of Interests. Authors have no competing interests.

References

1. Abusafia, A., Bouguettaya, A., Lakhdari, A.: Maximizing consumer satisfaction of IoT energy services. In: Troya, J., Medjahed, B., Piattini, M., Yao, L., Fernández, P., Ruiz-Cortés, A. (eds.) ICSOC 2022. LNCS, vol. 13740, pp. 395–412. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20984-0_28
2. Abusafia, A., Bouguettaya, A., Lakhdari, A.: Efficient provisioning of IoT energy services. In: Gaaloul, W., Sheng, M., Yu, Q., Yangui, S. (eds.) ICSOC 2024. Lecture Notes in Computer Science, vol. 15404, pp. 125–132. Springer, Singapore (2024). https://doi.org/10.1007/978-981-96-0805-8_9
3. Abusafia, A., Bouguettaya, A., Lakhdari, A.: Quality of experience in crowdsourced energy services. IEEE Trans. Serv. Comput. (2025)

4. Abusafia, A., Lakhdari, A., Bouguettaya, A.: Service-based wireless energy crowdsourcing. In: Troya, J., Medjahed, B., Piattini, M., Yao, L., Fernández, P., Ruiz-Cortés, A. (eds.) ICSOC 2022. LNCS, vol. 13740, pp. 653–668. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20984-0_47
5. Adewuyi, A.A., et al.: SC-trust: a dynamic model for trustworthy service composition in the internet of things. IEEE IoT J. **9**(5), 3298–3312 (2021)
6. Bahutair, M., Bougeuttaya, A., Neiat, A.G.: Adaptive trust: usage-based trust in crowdsourced IoT services. In: 2019 IEEE International Conference on Web Services (ICWS), pp. 172–179. IEEE (2019)
7. Bahutair, M., Bouguettaya, A.: Blockchain-based trust information storage in crowdsourced IoT services. In: 2021 IEEE International Conference on Web Services (ICWS), pp. 608–617. IEEE (2021)
8. Bahutair, M., Bouguettaya, A.: An end-to-end trust management framework for crowdsourced IoT services. ACM Trans. Internet Technol. (2023)
9. Bahutair, M., Bouguettaya, A., Neiat, A.G.: Multi-perspective trust management framework for crowdsourced IoT services. IEEE Trans. Serv. Comput. **15**(4), 2396–2409 (2021)
10. Brabham, D.C.: Crowdsourcing as a model for problem solving: an introduction and cases. Convergence **14**(1), 75–90 (2008)
11. Chen, R., Guo, J., Bao, F.: Trust management for SOA-based IoT and its application to service composition. IEEE TSC **9**(3), 482–495 (2014)
12. Das, D., et al.: Under the surface: tracking the artifactuality of LLM-generated data. arXiv preprint arXiv:2401.14698 (2024)
13. Ferdous, M.S., Chowdhury, M.J.M., Hoque, M.A., Colman, A.: Blockchain consensus algorithms: a survey. arXiv preprint arXiv:2001.07091 (2020)
14. Fernández-Caramés, T.M., Fraga-Lamas, P.: A review on the use of blockchain for the internet of things. IEEE Access **6**, 32979–33001 (2018)
15. Habak, K., Ammar, M., Harras, K.A., Zegura, E.: Femto clouds: leveraging mobile devices to provide cloud service at the edge. In: 2015 IEEE 8th International Conference on Cloud Computing, pp. 9–16. IEEE (2015)
16. Jøsang, A., Ismail, R., Boyd, C.: A survey of trust and reputation systems for online service provision. Decis. Support Syst. **43**(2), 618–644 (2007)
17. Liu, Y., Wang, J., Yan, Z., Wan, Z., Jäntti, R.: A survey on blockchain-based trust management for internet of things. IEEE Internet Things J. **10**(7), 5898–5922 (2023)
18. Liu, Y., et al.: A semi-centralized trust management model based on blockchain for data exchange in IoT system. IEEE TSC **16**(2), 858–871 (2022)
19. Neiat, A.G., Bouguettaya, A., Sellis, T., Mistry, S.: Crowdsourced coverage as a service: two-level composition of sensor cloud services. IEEE Trans. Knowl. Data Eng. **29**(7), 1384–1397 (2017)
20. Pathak, A.K., et al.: Anomaly detection using machine learning to discover sensor tampering in IoT systems. In: IEEEICC, pp. 1–6. IEEE (2021)
21. Qian, Y., et al.: UniDM: a unified framework for data manipulation with large language models. Proc. Mach. Learn. Syst. **6**, 465–482 (2024)
22. Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A.: A systematic survey of prompt engineering in large language models: techniques and applications. arXiv preprint arXiv:2402.07927 (2024)
23. Singh, S., Hosen, A.S., Yoon, B.: Blockchain security attacks, challenges, and solutions for the future distributed IoT network. IEEE Access **9**, 13938–13959 (2021)

24. Tripathi, G., Ahad, M.A., Casalino, G.: A comprehensive review of blockchain technology: underlying principles and historical background with future challenges. Decis. Anal. J., 100344 (2023)
25. Wang, J., Yan, Z., Wang, H., Li, T., Pedrycz, W.: A survey on trust models in heterogeneous networks. IEEE Commun. Surv. Tutor. **24**(4), 2127–2162 (2022)
26. Wei, L., Wu, J., Long, C.: Blockchain-enabled trust management in service-oriented internet of things: opportunities and challenges. In: Proceedings of the 2021 3rd International Conference on Blockchain Technology, pp. 90–95 (2021)

LDP-QWSP: A General Local Differential Privacy Framework for QoS-Based Web Service Prediction

Fuchang Luo[1], Haonan Wu[1], Shunshun Peng[1], Quanwang Wu[2], Hongbing Wang[3], Mengmeng Yang[4], and Taolin Guo[1(✉)]

[1] College of Computer and Information Science, Chongqing Normal University, Chongqing 401331, China
tguo@cqnu.edu.cn
[2] College of Computer Science, Chongqing University, Chongqing 400044, China
[3] School of Big Data and Computer Science, Guizhou Normal University, Guiyang 550025, China
[4] Data61 of Commonwealth Scientific and Industrial Research Organization, Clayton, VIC 3168, Australia

Abstract. QoS-based web service prediction is widely used in service computing but raises privacy concerns when collecting user data. Although local differential privacy (LDP) offers strong protection, its direct use in QoS prediction is hindered by attribute heterogeneity, which reduces data utility and prediction accuracy. To address this, we propose LDP-QWSP, an LDP-compliant framework that incorporates attribute-aware normalization and a perturbation mechanism favoring high-utility intervals. Experiments on real-world datasets confirm that LDP-QWSP achieves higher prediction accuracy than existing methods under comparable privacy budgets.

Keywords: QoS prediction · privacy preservation · local differential privacy

1 Introduction

Quality of Service (QoS)-based web service prediction is essential in service-oriented architecture. It aims to forecast the performance of non-functional service attributes, such as response time, availability, reliability, and throughput, through analysis of historical QoS data [8]. By offering these predictive insights, it enables service consumers to make effective decisions regarding service recommendations, load balancing, and fault tolerance. As a result, QoS-based web service prediction is widely used in cloud computing, microservices architectures, and various web service scenarios. However, this prediction relies on the collection and storage of large amounts of user QoS data. The exposure of this data could allow attackers to infer users' underlying application configurations, leading to significant privacy and security concerns [15]. Therefore, protecting the

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 207–215, 2026.
https://doi.org/10.1007/978-981-95-5015-9_15

privacy of user QoS data in web service prediction has garnered considerable research attention [1].

While Local Differential Privacy (LDP) is a state-of-the-art framework for privacy-preserving data collection [7], its application to Quality of Service (QoS)-based web service prediction remains a significant, unaddressed challenge. Existing LDP methods, typically designed for tasks like rating [13] or relationship prediction [10], rely on a privacy model focused on the presence of a single data item, which leads to uniform data perturbation. Such an approach is effective for the simple and uniform data found in those domains (e.g., fixed-scale ratings or binary relationships). However, it is fundamentally ill-suited for the inherent heterogeneity of QoS data, where attributes such as response time and throughput exhibit vastly different scales and distributions. Applying uniform perturbation to heterogeneous QoS data severely undermines the analysis of inter-attribute correlations, a critical factor for accurate modeling, thereby causing a substantial decline in prediction accuracy.

This paper addresses accuracy challenges caused by attribute heterogeneity in QoS-based prediction. We propose LDP-QWSP, a general framework for QoS-based web service prediction under LDP. The framework includes three phases: preprocessing, data perturbation, and prediction. In preprocessing, we use Min–Max normalization to map heterogeneous QoS data into a common interval. During perturbation, we divide the perturbation range into high- and low-functionality intervals and design the mechanism to favor the high-functionality interval. For prediction, we reconstruct QoS data from perturbed values using attribute mappings and employ mainstream prediction methods. Extensive experiments on public datasets show that LDP-QWSP delivers more accurate predictions than conventional LDP-based methods while providing equivalent privacy protection.

The contributions of this paper are summarized as follow. We propose LDP-QWSP, a general framework for preserving local differential privacy in QoS-based web service prediction. We effectively mitigate QoS prediction biases arising from heterogeneous data ranges. We perform extensive experiments on publicly available datasets to evaluate LDP-QWSP against state-of-the-art baselines in terms of privacy preservation and prediction accuracy.

2 Problem Definition

In this section, we formally define the problem addressed in this paper.

2.1 Privacy Definition

Definition 1 *(Local Differential Privacy). A privacy-preserving mechanism $\mathcal{M}$ satisfies ϵ-local differential privacy if, for all possible outputs $\hat{X}_i^j$ and any pair of neighboring QoS data X_i^j and $X_i'^j$ (differing in a single QoS data), the following inequality holds:*

$$e^{-\epsilon} \leq \frac{\Pr(\mathcal{M}(X_i^j) = \hat{X}_i^j)}{\Pr(\mathcal{M}(X_i'^j) = \hat{X}_i^j)} \leq e^{\epsilon}$$

where ϵ is the privacy budget, controlling the privacy-utility trade-off.

2.2 Problem Description

Despite the availability of various LDP mechanisms, their direct application to user QoS data often severely degrades data utility. This is due to the divergent nature of QoS attributes. For example, response time favors low values within a small range, while throughput prefers high values across a larger scale. Applying mechanisms like Laplace noise uniformly distorts these inherent characteristics, ultimately compromising the accuracy of service prediction.

3 Method

In this section, we propose a locally differentially private QoS-based web service prediction framework, named LDP-QWSP. As illustrated in Fig. 1, the proposed framework comprises three phases: preprocessing, data perturbation, and QoS prediction.

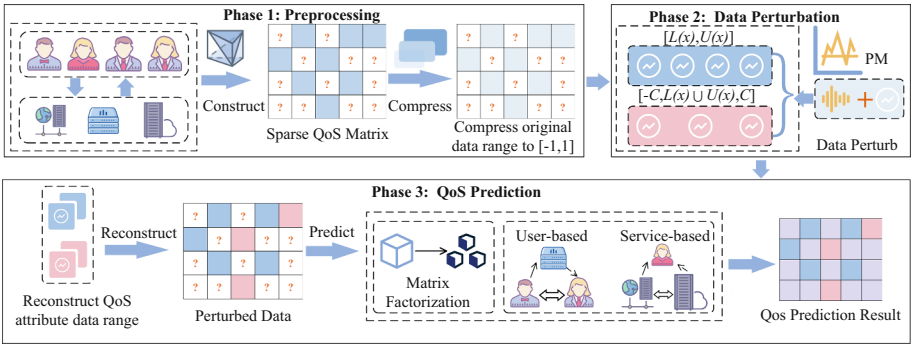


Fig. 1. Overview of the framework.

3.1 Preprocessing

To mitigate prediction bias stemming from attribute heterogeneity, we preprocess the QoS data via normalization. The objective is twofold: (1) to map all attributes to a uniform numerical range, and (2) to preserve the original data distributions. Formally, we apply Min-Max normalization to each attribute $x_i^j(k)$ of a QoS vector $\mathbf{x}_i^j$, transforming it into $y_i^j(k)$ as follows:

$$y_i^j(k) = 2 \cdot \frac{x_i^j(k) - x_{\min}^j(k)}{x_{\max}^j(k) - x_{\min}^j(k)} - 1 \tag{1}$$

where $x^j_{\min}(k)$ and $x^j_{\max}(k)$ represent the global minimum and maximum values for that attribute. This projects all attribute values into the interval $[-1, 1]$.

The normalization preserves ordinal relationships and distribution morphology:

$$\frac{\partial y^j_i(k)}{\partial x^j_i(k)} = \frac{2}{x^j_{\max}(k) - x^j_{\min}(k)} > 0 \tag{2}$$

This property maintains the statistical characteristics essential for downstream prediction tasks.

3.2 Data Perturbation

We define the perturbation output domain as $[-C, C]$, where the boundary parameter C is uniquely determined by the privacy budget ϵ:

$$C = \frac{e^\epsilon + 1}{e^\epsilon - 1} \tag{3}$$

This design ensures dynamic adjustment of the perturbation range: increasing ϵ reduces C, lowering privacy protection while concentrating outputs. To preserve data utility, we partition the perturbation range into high-functionality (H) and low-functionality (L) intervals based on the normalized QoS value $y^j_i(k)$:

$$H = \left[\frac{C+1}{2} y^j_i(k) - \frac{C-1}{2},\ \frac{C+1}{2} y^j_i(k) + \frac{C-1}{2}\right],\ L = [-C, C] \setminus H \tag{4}$$

with the high-functionality interval H maintaining a constant length of $C - 1$ regardless of $y^j_i(k)$. We formulate the perturbation mechanism as a piecewise constant probability density function:

$$z^j_i(k) = \begin{cases} \text{uniform_sample}(H) & \text{with probability } p = \frac{e^\epsilon}{e^\epsilon + 1} \\ \text{uniform_sample}(L) & \text{with probability } 1 - p \end{cases} \tag{5}$$

where uniform_sample$(\cdot)$ denotes uniform random sampling, and p is controlled by ϵ to optimize privacy-utility trade-offs.

3.3 QoS Prediction

Directly using the perturbed QoS data compromises prediction accuracy because privacy-preserving perturbations alter the distribution characteristics of original data. To mitigate this prediction bias, we reconstruct the distribution of perturbed QoS attributes $z^j_i(k)$ to align with the original data characteristics.

First, we apply *proportional scaling* to transform the perturbed values $z^j_i(k) \in [-C, C]$ into the normalized range $[-1, 1]$. Specifically, we have $z_{\text{norm}} = \frac{z^j_i(k)}{C}$. This scaling eliminates the perturbation-specific domain constraints while preserving relative value relationships.

Then, we perform inverse standardization using the original extremum values $x^j_{\max}$ and $x^j_{\min}$ recorded during preprocessing:

$$\hat{x}^j_i(k) = z_{\text{norm}} \cdot (x^j_{\max} - x^j_{\min}) + x^j_{\min} \tag{6}$$

This transformation reconstructs the data's original scale and distribution properties, where $\hat{x}^j_i(k) \in [x^j_{\min}, x^j_{\max}]$ denotes the reconstructed QoS attribute value.

We feed the reconstructed data $\hat{x}^j_i(k)$ into established QoS prediction models, *e.g.*, matrix factorization, neighborhood-based methods. This reconstruction-decoupled approach enables standard predictors to operate on distributionally aligned data while preserving the privacy guarantees of the perturbation phase, effectively bridging the gap between privacy protection and prediction accuracy.

4 Experiments

4.1 Experimental Settings

Datasets. We conduct experimental validation using a publicly available web service QoS dataset [17].

Comparison Methods. To the best of our knowledge, the only method currently available for web service prediction under differential privacy protection is the DPS proposed by Liu *et al.* [12]. This method adds Laplace noise to QoS data. Based on this method, we applied the Laplace [5] and Gaussian [2] to perturb the QoS data in DPS, conducting experimental comparisons with LDP-QWSP.

Evaluation Metrics. We evaluate LDP-QWSP and comparison methods using a range of mainstream QoS prediction algorithms, including UMEAN, IMEAN, UPCC [14], IPCC [16], UIPCC [16], PMF [6], EMF [4], HMF [9], and LoRec [3]. To simulate real-world data sparsity, we randomly split the dataset, using 20% for training and 80% for testing. We apply privacy-preserving perturbation with both LDP-QWSP and comparison methods to the training data, and then feed the processed data into the prediction models. Finally, we employ mean absolute error (MAE) to directly compare the predictions with ground-truth test values, providing a comprehensive assessment of each method's prediction accuracy.

Parameter Settings. We vary the privacy budget from 1 to 5 to assess the prediction accuracy of LDP-QWSP and the comparison methods under different privacy levels. For PMF, EMF, and HMF, we set the initial learning rate to 0.001 and the maximum number of iterations to 300; for IPCC and UPCC, we set the number of similar neighbors $k = 10$, and use $\lambda = 0.9$ as the combination weight in UIPCC. We repeat each experiment 20 times and use the average prediction error to ensure reliability.

4.2 Experimental Results

Tables 1 present the prediction errors achieved by LDP-QWSP and the comparison methods when integrated with various QoS prediction models, evaluated

Table 1. The prediction error of LDP-QWSP and comparison methods.

Method	Prediction methods	QoS attributes									
		Response Time					Throughput				
		1	2	3	4	5	1	2	3	4	5
Gaussian	UMEAN	2.36	1.44	1.18	1.06	1.00	127.67	80.33	66.94	60.79	58.13
	IMEAN	8.46	4.34	2.99	2.33	1.95	448.95	229.19	155.15	119.76	98.11
	UPCC	12.19	6.24	4.28	3.31	2.74	651.24	333.58	229.00	176.73	146.15
	IPCC	16.50	8.31	5.60	4.26	3.47	882.75	444.25	298.40	226.52	183.30
	UIPCC	12.02	6.15	4.22	3.26	2.70	642.49	328.88	225.49	173.78	143.56
	PMF	35.26	17.63	11.81	8.84	7.07	1890.02	948.17	632.26	474.47	379.75
	EMF	34.36	16.89	11.13	8.21	6.46	1860.23	923.17	608.44	451.95	356.05
	HMF	34.57	17.04	11.24	8.35	6.61	1871.84	931.18	618.47	460.93	365.62
	LoRec	14.72	7.40	4.98	3.80	3.09	790.40	397.30	267.73	203.25	164.77
Laplace	UMEAN	1.12	0.94	0.90	0.89	0.88	63.19	55.67	54.57	54.33	54.05
	IMEAN	2.65	1.53	1.18	1.02	0.93	136.94	75.91	56.56	47.25	41.97
	UPCC	3.77	2.11	1.59	1.32	1.16	200.83	112.33	83.55	68.90	59.62
	IPCC	4.87	2.59	1.86	1.50	1.29	258.73	135.74	95.70	76.17	64.62
	UIPCC	3.71	2.07	1.56	1.29	1.13	197.47	109.97	81.56	67.15	58.03
	PMF	10.33	5.15	3.42	2.58	2.09	552.06	275.51	185.49	140.12	112.65
	EMF	9.62	4.58	2.93	2.13	1.68	526.73	253.66	165.07	120.86	95.05
	HMF	9.74	4.68	3.01	2.21	1.74	537.72	262.91	173.54	129.30	102.53
	LoRec	4.64	2.45	1.73	1.38	1.18	249.14	128.95	89.44	70.35	58.98
LDP-QWSP	UMEAN	0.99	0.94	0.90	0.89	0.87	55.63	52.03	50.13	48.78	48.20
	IMEAN	1.37	1.22	1.10	1.02	0.96	75.96	67.29	61.07	56.39	53.37
	UPCC	1.45	1.26	1.12	1.02	0.96	80.26	69.68	61.85	56.35	53.11
	IPCC	1.77	1.48	1.27	1.11	1.01	98.42	83.16	71.15	62.27	56.28
	UIPCC	1.40	1.22	1.09	0.99	0.94	77.54	67.52	60.06	54.83	51.78
	PMF	2.97	2.36	1.89	1.52	1.26	186.87	154.18	127.27	106.96	91.58
	EMF	2.94	2.34	1.86	1.51	1.25	168.17	135.74	109.23	89.10	74.24
	HMF	2.33	1.90	1.77	1.74	1.72	152.96	113.05	94.62	84.09	73.76
	LoRec	1.75	1.54	1.32	1.15	1.04	98.48	85.67	73.89	64.54	58.05

under Response Time (RT) and Throughput (TP) metrics, respectively. Based on these experimental results, we have the following conclusions.

LDP-QWSP Consistently Outperforms Comparison Methods in Prediction Accuracy Across Privacy Levels. We observe that LDP-QWSP consistently achieves substantially lower prediction errors than both the Laplace and Gaussian mechanisms across all privacy budgets (ϵ) and prediction models for the RT and TP attributes. For example, under the UIPCC model with $\epsilon = 1$, LDP-QWSP attains an MAE of 1.40 for RT, while Laplace and Gaussian get significantly higher MAEs of 3.71 and 12.02, respectively. When we increase the privacy budget to $\epsilon = 5$, LDP-QWSP further reduces the MAE to 0.94, outperforming Laplace and Gaussian, which yield MAEs of 1.13 and 2.70, respectively.

Across all examined models, privacy budgets, and error metrics, we find that LDP-QWSP consistently delivers better prediction accuracy. We attribute this improvement to the foundational design of LDP-QWSP, which homogenizes perturbations for heterogeneous QoS attributes, thereby preserving more useful information while adhering to privacy guarantees.

LDP-QWSP Achieves Faster Prediction Error Convergence Than Comparison Methods as the Privacy Budget Increases. As the privacy budget increases, LDP-QWSP consistently converges to lower prediction errors more rapidly than the comparison methods across all prediction models. For example, when predicting the TP attribute using the UIPCC model, increasing the privacy budget ϵ from 1 to 5 results in the MAE of LDP-QWSP decreasing from 77.54 to 51.78. In comparison, the MAEs for the Laplace and Gaussian mechanisms decrease from 197.47 to 58.03 and from 642.49 to 143.56, respectively. These results indicate that although the comparison methods experience a steeper reduction in prediction error as the privacy budget rises, primarily due to their substantially higher initial errors.

However, the rate of error reduction differs significantly between methods. LDP-QWSP demonstrates a more gradual decline in error across prediction models, resulting in the least loss of prediction accuracy. This finding suggests that, compared to the comparison methods, LDP-QWSP converges more quickly and stably to a smaller prediction error as privacy constraints are relaxed.

5 Related Works

This paper focuses on QoS prediction with differential privacy protection, a topic that has received limited attention in the literature. So we summarize existing works related to QoS-specific privacy frameworks. Zhu *et al.* [18] pioneered privacy in QoS recommendations using randomization techniques, though their method lacks formal differential privacy guarantees. Building on this foundation, Liu *et al.* [11] introduced two rigorous differential privacy mechanisms: differential privacy on simple data (DPS) and its improved variant differential privacy on aggregated data (DPA). DPS applies Laplace noise to the entire QoS vector using global sensitivity, yet this often leads to excessive noise. Addressing this limitation, DPA incorporates user grouping to compute local sensitivity for each group and injects noise accordingly, thus improving utility.

6 Conclusion

To overcome the utility degradation caused by heterogeneity in QoS attributes under local differential privacy, this paper proposes LDP-QWSP. The framework incorporates attribute normalization, utility-aware biased perturbation, and data reconstruction to preserve prediction accuracy while guaranteeing ϵ-LDP. Experimental evaluations on public datasets confirm that LDP-QWSP significantly outperforms traditional perturbation mechanisms in both prediction accuracy and convergence speed across diverse service prediction models.

Acknowledgments. This work was partially supported by the National Natural Science Foundation of China (No. U22A2026), the Science and Technology Research Program of Chongqing Municipal Education Commission (No. KJQN202400541), and the Foundation Projection of Chongqing Normal University (Nos. 23XLB026 and 21XLB003).

References

1. Badsha, S., et al.: Privacy preserving location-aware personalized web service recommendations. IEEE Trans. Serv. Comput. **14**(3), 791–804 (2021)
2. Balle, B., Wang, Y.: Improving the gaussian mechanism for differential privacy: analytical calibration and optimal denoising. In: Proceedings of the 35th International Conference on Machine Learning, vol. 80, pp. 403–412 (2018)
3. Chen, X., Zheng, Z., Yu, Q., Lyu, M.R.: Web service recommendation via exploiting location and QoS information. IEEE Trans. Parallel Distributed Syst. **25**(7), 1913–1924 (2014)
4. Chen, X., Du, Y., Tang, G., Chen, F., Luo, Y., Wang, H.: A QoS prediction framework via utility maximization and region-aware matrix factorization. IEEE Trans. Serv. Comput. **18**(2), 557–571 (2025)
5. Costea, S., Tapus, N.: Input validation for the Laplace differential privacy mechanism. In: 20th International Conference on Control Systems and Computer Science, pp. 469–474 (2015)
6. Deng, J., Ran, X., Wang, Y., Zhang, L.Y., Guo, J.: Probabilistic matrix factorization recommendation approach for integrating multiple information sources. IEEE Trans. Syst. Man Cybern. Syst. **53**(10), 6220–6231 (2023)
7. Erlingsson, Ú., Pihur, V., Korolova, A.: RAPPOR: randomized aggregatable privacy-preserving ordinal response. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, pp. 1054–1067 (2014)
8. Ghafouri, S.H., Hashemi, S.M., Hung, P.C.K.: A survey on web service QoS prediction methods. IEEE Trans. Serv. Comput. **15**(4), 2439–2454 (2022)
9. He, P., Zhu, J., Zheng, Z., Xu, J., Lyu, M.R.: Location-based hierarchical matrix factorization for web service recommendation. In: 2014 IEEE International Conference on Web Services, pp. 297–304. IEEE Computer Society (2014)
10. Jiang, L., Yan, Y., Tian, Z., Xiong, Z., Han, Q.: Personalized sampling graph collection with local differential privacy for link prediction. World Wide Web (WWW) **26**(5), 2669–2689 (2023)
11. Liu, A., et al.: Differential private collaborative web services QoS prediction. World Wide Web **22**(6), 2697–2720 (2019)
12. Liu, S., et al.: Privacy-preserving collaborative web services QoS prediction via differential privacy. In: Chen, L., Jensen, C.S., Shahabi, C., Yang, X., Lian, X. (eds.) APWeb-WAIM 2017. LNCS, vol. 10366, pp. 200–214. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-63579-8_16
13. Neera, J., Chen, X., Aslam, N., Wang, K., Shu, Z.: Private and utility enhanced recommendations with local differential privacy and gaussian mixture model. IEEE Trans. Knowl. Data Eng. **35**(4), 4151–4163 (2023)
14. Shao, L., Zhang, J., Wei, Y., Zhao, J., Xie, B., Mei, H.: Personalized QoS prediction for web services via collaborative filtering. In: 2007 IEEE International Conference on Web Services, pp. 439–446 (2007)

15. Zheng, Z., Li, X., Tang, M., Xie, F., Lyu, M.R.: Web service QoS prediction via collaborative filtering: a survey. IEEE Trans. Serv. Comput. **15**(4), 2455–2472 (2022)
16. Zheng, Z., Ma, H., Lyu, M.R., King, I.: QoS-aware web service recommendation by collaborative filtering. IEEE Trans. Serv. Comput. **4**(2), 140–152 (2011)
17. Zheng, Z., Zhang, Y., Lyu, M.R.: Investigating QoS of real-world web services. IEEE Trans. Serv. Comput. **7**(1), 32–39 (2014)
18. Zhu, J., He, P., Zheng, Z., Lyu, M.R.: A privacy-preserving QoS prediction framework for web service recommendation. In: 2015 IEEE International Conference on Web Services, pp. 241–248 (2015)

Privacy-Preserving Revenue Prediction in Service-Oriented Industrial Supply Chains

Xianghui Hu[1], Kai Di[1], Zhenyu Wang[2], Xinran Zhuang[3], Yichuan Jiang[3], Hang Liu[3], and Pan Li[1(✉)]

[1] The School of Automation, Zhejiang Normal University, Hangzhou 311231, China
{dikai,lipan}@seu.edu.cn
[2] The School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China
[3] The School of Computer Science and Engineering, Southeast University, Nanjing 211189, China
{zhuangxinran,yjiang}@seu.edu.cn

Abstract. In service-oriented industrial supply chains, enterprises collaborate to share information while maintaining strict privacy requirements that prohibit raw financial data exchange between corporate groups. We propose a novel Granular Fuzzy Rule-Based Model based on Density Peaks Clustering (DPC-GFRBM) for privacy-preserving revenue prediction in industrial supply chain networks. DPC-GFRBM performs local clustering within each enterprise group to identify revenue patterns, then aggregates these results through collaborative analysis without revealing individual data. A fuzzy rule-based model maps the granulated features to revenue predictions, balancing privacy protection with predictive accuracy. Experimental results on public and real-world enterprise revenue datasets show that DPC-GFRBM consistently outperforms traditional fuzzy rule-based methods, achieving over 20% improvements in the interval prediction balance index, while maintaining low computational overhead.

Keywords: Privacy-Aware Services · Revenue Prediction · Industrial Alliances · Granular Computing · Density Peaks Clustering

1 Introduction

Revenue evaluation enables enterprises to assess profitability and optimize resource allocation, thereby enhancing competitiveness and sustainability [1–3]. Modern manufacturing supply chains have evolved from traditional linear models into service-oriented industrial ecosystems characterized by dynamic enterprise interactions [4,5]. In these ecosystems, enterprises form alliances and partnerships to facilitate collaboration, data exchange, and value co-creation through service-oriented infrastructures.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 216–224, 2026.
https://doi.org/10.1007/978-981-95-5015-9_16

In such distributed service computing environments, comprehensive revenue evaluation faces a critical challenge: how to extract valuable patterns from multiple service participants while maintaining privacy protection and prediction accuracy. While revenue-related features can be shared within alliances under service-level agreements, raw financial data exchange across alliances is restricted due to data sovereignty concerns. This constraint limits traditional machine learning models that rely on centralized data, while federated learning approaches often incur high computational costs.

To address these challenges, we propose a novel service-oriented Granular Fuzzy Rule-Based Model based on Density Peaks Clustering (DPC-GFRBM) for privacy-preserving revenue prediction. Our approach operates through multi-level collaborative modeling that respects service boundaries between enterprise alliances. First, intra-alliance clustering extracts revenue structures within each service group. Then, secondary clustering aggregates local patterns across service domains without direct data exposure. Finally, a fuzzy rule-based model maps service granules to revenue outcomes, ensuring interpretable predictions.

The main contributions of this paper are:

1) We propose a multi-level clustering framework for privacy-preserving collaborative modeling using Density Peaks Clustering within alliances and secondary clustering across alliances.
2) We develop a granular fuzzy rule-based model that transforms clustering results into interval representations, reducing information leakage while retaining essential structures.
3) Extensive experiments show DPC-GFRBM outperforms conventional fuzzy models, achieving over 20% improvement in prediction balance with computational efficiency.

The remainder of this paper is organized as follows: Sect. 2 reviews related work. Section 3 details our proposed DPC-GFRBM algorithm. Section 4 presents experimental results. Section 5 concludes this study.

2 Related Work

In inter-organizational service revenue prediction, models such as multiple regression [6], K-nearest neighbors [7], and exponential GARCH [8] have been adopted to forecast enterprise-level financial outcomes. However, these approaches assume isolated or fully integrated data scenarios, failing to accommodate distributed service computing environments where data resides within separate enterprise alliances governed by strict service-level agreements. This limitation severely restricts their applicability in industrial service ecosystems where data sharing is constrained due to competition and confidentiality concerns.

To address privacy-utility trade-offs, the principle of reasonable granularity [9] provides foundation for privacy-preserving analytics by replacing exact values with interval-based representations. Hierarchical granulation techniques [10] emphasize structural consistency over raw values, while collaborative fuzzy clustering methods [11] discover common patterns from partitioned data sources. Recent efforts combining clustering with fuzzy rule-based modeling [12] define interval bounds using statistical regularities, enhancing interpretability and output granularity.

Nevertheless, existing approaches remain limited in service-oriented industrial settings: they either depend on precise cluster centers (increasing privacy risks), lack mechanisms for inter-alliance coordination (reducing collaborative accuracy), or require excessive computation that hampers scalability in real-world service platforms.

3 Algorithm Design

We propose DPC-GFRBM, a privacy-preserving fuzzy rule model for enterprise revenue prediction across industrial alliances. Unlike traditional fuzzy rule construction that requires sharing raw data, our approach integrates Density Peak Clustering (DPC) with the Justifiable Granularity Principle (JGP) to enable collaborative modeling while maintaining privacy boundaries.

Let $A[1], A[2], ..., A[P]$ represent P alliances, each sharing the same feature space $\mathcal{R}^Z$ and target variable y. Each alliance $A[ii]$ contains a dataset $D[ii]$ with $N[ii]$ enterprise samples.

3.1 Intra-alliance Density Peak Clustering

Each alliance performs local clustering using DPC for its stability and ability to automatically determine cluster centers. The key steps involve:

Distance calculation: $d_{ij} = ||x_i - x_j|| = \sqrt{\sum_{z=1}^{Z}(x_{iz} - x_{jz})^2}$

Local density: $\rho_i = \sum_{j=1}^{N} \exp\left(-\left(\frac{d_{ij}}{d_c}\right)^2\right)$

Minimum distance to higher density: $\delta_i = \min_{j:\rho_j > \rho_i} d_{ij}$ if exists, otherwise $\max_j d_{ij}$

Cluster centers are selected where decision values $\gamma_i = \rho_i \cdot \delta_i > \mu_\gamma + \sigma_\gamma$.

3.2 Privacy-Preserving Granular Representation

To prevent privacy leakage, we transform precise cluster centers into interval-valued representations. For each feature dimension z of cluster center v_k, we optimize:

$$T_z(a, b) = \mathrm{cov}_z(a, b) \cdot \mathrm{sp}_z(a, b) \tag{1}$$

The optimal interval boundaries $[a_z^*, b_z^*] = \arg\max_{a,b} T_z(a, b)$ form the granular prototype:

$$V_k = \{[a_1^*, b_1^*], [a_2^*, b_2^*], ..., [a_Z^*, b_Z^*]\} \tag{2}$$

3.3 Cross-alliance Collaborative Clustering and Revenue Prediction

We perform secondary clustering on granular prototypes using Fuzzy C-Means to generate global prototypes. The objective function minimizes weighted distance between local and global prototypes:

$$J = \sum_{k=1}^{c} \sum_{t=1}^{h} U_{kt}^{m} d(\bar{V}_k, V_t[ii])^2 \tag{3}$$

Global prototypes are updated as:

$$\bar{V}_k = \frac{\sum_{ii=1}^{P} \sum_{t=1}^{h_{ii}} U_{kt}^{m} V_t[ii]}{\sum_{ii=1}^{P} \sum_{t=1}^{h_{ii}} U_{kt}^{m}} \tag{4}$$

For revenue prediction, we optimize revenue intervals $Y_k = [Y_k^L, Y_k^U]$ for each global prototype and construct fuzzy rules. The predicted revenue interval for enterprise x_i is:

$$y_i = \sum_{k=1}^{K} U_{ik} \otimes Y_k = \left[\sum_{k=1}^{K} U_{ik} \cdot Y_k^L, \sum_{k=1}^{K} U_{ik} \cdot Y_k^U \right] \tag{5}$$

Algorithm 1: DPC-GFRBM

Input: Enterprise datasets $D[1], D[2], ..., D[P]$, number of clusters c, fuzzification coefficient m

Output: Predicted revenue intervals $\{y_i\}_{i=1}^{N}$

for $ii = 1, 2, \ldots, P$ **do**
  Apply DPC to dataset $D[ii]$ to obtain local cluster centers;
  Optimize objective function to get interval-valued prototypes $V[ii]$;
end
Aggregate local prototypes and apply FCM to obtain global prototypes $\bar{V}k$;
Calculate membership degrees Uik;
for $k = 1$ *to* c **do**
  Optimize revenue intervals $Y_k = [Y_k^L, Y_k^U]$;
end
for *each enterprise* x_i **do**
  Calculate predicted revenue interval $y_i = \sum_{k=1}^{c} U_{ik} \otimes Y_k$;
end
return $\{y_i\}_{i=1}^{N}$.

4 Experiments

To evaluate the effectiveness of our proposed DPC-GFRBM algorithm for privacy-preserving enterprise revenue prediction, we conducted comprehensive experiments comparing it against state-of-the-art approaches. The source code for the experiments is available at https://github.com/windflowerHu/DPC-GFRBM/.

Table 1. Performance metrics of each algorithm in the public dataset

Dataset	Algorithm	$P = 3$			$c = 6$		
		$c = 2$	$c = 5$	$c = 8$	$P = 2$	$P = 5$	$P = 8$
Airfoil Self Noise	CFC-GFRBM	0.21 ± 0.02	0.23 ± 0.02	0.21 ± 0.02	0.33 ± 0.03	0.26 ± 0.01	0.29 ± 0.01
	ISO-GFRBM	0.25 ± 0.00	0.26 ± 0.00	0.26 ± 0.01	0.25 ± 0.00	0.34 ± 0.00	0.38 ± 0.01
	DBSCAN-GFRBM	0.30 ± 0.00	0.30 ± 0.00	0.30 ± 0.00	0.30 ± 0.00	0.21 ± 0.00	0.16 ± 0.00
	DF-GFRBM	0.30 ± 0.00	$0.30 \pm \pm 0.00$	0.30 ± 0.00	0.30 ± 0.00	0.21 ± 0.00	0.18 ± 0.00
	FCM-GFRBM	0.30 ± 0.00	0.29 ± 0.00	0.27 ± 0.00	0.28 ± 0.00	0.19 ± 0.00	0.15 ± 0.00
	DPC-GFRBM	$\mathbf{0.34 \pm 0.00}$	$\mathbf{0.39 \pm 0.00}$	$\mathbf{0.39 \pm 0.00}$	$\mathbf{0.39 \pm 0.00}$	$\mathbf{0.40 \pm 0.00}$	$\mathbf{0.39 \pm 0.00}$
Parkinsons Updrs	CFC-GFRBM	0.23 ± 0.01	0.22 ± 0.02	0.19 ± 0.01	0.21 ± 0.01	0.28 ± 0.01	0.30 ± 0.01
	ISO-GFRBM	0.28 ± 0.00	0.26 ± 0.00	0.31 ± 0.00	0.30 ± 0.00	0.38 ± 0.00	0.40 ± 0.00
	DBSCAN-GFRBM	0.37 ± 0.00	0.37 ± 0.00	0.37 ± 0.00	0.37 ± 0.00	0.28 ± 0.00	0.22 ± 0.00
	DF-GFRBM	0.37 ± 0.00	0.37 ± 0.00	0.37 ± 0.00	0.37 ± 0.00	0.28 ± 0.00	0.22 ± 0.00
	FCM-GFRBM	0.36 ± 0.00	0.28 ± 0.00	0.20 ± 0.00	0.31 ± 0.00	0.19 ± 0.00	0.14 ± 0.00
	DPC-GFRBM	$\mathbf{0.47 \pm 0.00}$	$\mathbf{0.48 \pm 0.00}$	$\mathbf{0.47 \pm 0.01}$	$\mathbf{0.48 \pm 0.00}$	$\mathbf{0.48 \pm 0.00}$	$\mathbf{0.48 \pm 0.00}$
Compactiv	CFC-GFRBM	0.34 ± 0.01	0.40 ± 0.02	0.36 ± 0.04	0.39 ± 0.01	0.39 ± 0.01	0.43 ± 0.03
	ISO-GFRBM	0.36 ± 0.00	0.36 ± 0.00	0.38 ± 0.02	0.36 ± 0.00	0.42 ± 0.00	0.47 ± 0.00
	DBSCAN-GFRBM	0.25 ± 0.00	0.25 ± 0.00	0.25 ± 0.00	0.25 ± 0.00	0.14 ± 0.00	0.09 ± 0.00
	DF-GFRBM	0.25 ± 0.00	0.25 ± 0.00	0.25 ± 0.00	0.25 ± 0.00	0.14 ± 0.00	0.13 ± 0.00
	FCM-GFRBM	0.27 ± 0.00	0.16 ± 0.00	0.11 ± 0.00	0.13 ± 0.00	0.06 ± 0.00	0.06 ± 0.00
	DPC-GFRBM	$\mathbf{0.43 \pm 0.00}$	$\mathbf{0.51 \pm 0.02}$	$\mathbf{0.52 \pm 0.02}$	$\mathbf{0.50 \pm 0.02}$	$\mathbf{0.41 \pm 0.03}$	$\mathbf{0.49 \pm 0.01}$
Puma32h	CFC-GFRBM	0.27 ± 0.00	0.27 ± 0.00	0.28 ± 0.00	0.27 ± 0.00	0.41 ± 0.00	0.42 ± 0.00
	ISO-GFRBM	0.27 ± 0.00	0.27 ± 0.00	0.27 ± 0.00	0.27 ± 0.00	0.42 ± 0.00	0.42 ± 0.00
	DBSCAN-GFRBM	0.34 ± 0.00	0.34 ± 0.00	0.34 ± 0.00	0.34 ± 0.00	0.25 ± 0.00	0.20 ± 0.00
	DF-GFRBM	0.34 ± 0.00	0.34 ± 0.00	0.34 ± 0.00	0.34 ± 0.00	0.25 ± 0.00	0.20 ± 0.00
	FCM-GFRBM	0.36 ± 0.00	0.36 ± 0.00	0.36 ± 0.00	0.36 ± 0.00	0.27 ± 0.00	0.22 ± 0.00
	DPC-GFRBM	$\mathbf{0.42 \pm 0.01}$	$\mathbf{0.43 \pm 0.00}$	$\mathbf{0.43 \pm 0.01}$	$\mathbf{0.43 \pm 0.00}$	$\mathbf{0.41 \pm 0.01}$	$\mathbf{0.40 \pm 0.02}$

4.1 Experimental Setup

We evaluate our approach on both benchmark regression datasets and enterprise-specific revenue data. For public benchmarks, we selected four diverse datasets from the UCI Machine Learning Repository [13]: Airfoil Self Noise (1,503 samples, 6 features), Parkinsons Updrs (5,875 samples, 22 features), Compactiv (8,192 samples, 22 features), and Puma32h (8,192 samples, 33 features). For realistic revenue prediction scenarios, we used the fortune1000_2024 dataset from Kaggle [14] containing 963 samples and 18 features.

To simulate enterprise alliances, each dataset was randomly split into training (80%) and testing (20%) sets, with training data further divided into $P \in 3, 6, 9$ non-overlapping alliance subsets. Raw data sharing was permitted only within alliances. We compared DPC-GFRBM against five baseline methods: ISO-GFRBM, CFC-GFRBM [15], DBSCAN-GFRBM [16], DF-GFRBM [16], and FCM-GFRBM [16].

We employed two evaluation metrics: (1) Granular Prediction Metric (V) combining coverage and specificity: $V = \text{Coverage} \times \text{Specificity}$, and (2) Computational Efficiency (T) measuring total execution time.

Table 2. Algorithm Running Time of Each Algorithm on Public Datasets

Dataset	Algorithm	$P = 3$			$c = 6$		
		$c = 2$	$c = 5$	$c = 8$	$P = 2$	$P = 5$	$P = 8$
Airfoil Self Noise	CFC-GFRBM	0.68 ± 0.30	1.86 ± 0.18	3.09 ± 0.03	2.26 ± 0.14	1.59 ± 0.13	1.79 ± 0.07
	ISO-GFRBM	0.91 ± 1.43	1.11 ± 0.01	1.74 ± 0.09	1.37 ± 0.06	0.86 ± 0.02	1.14 ± 0.06
	DBSCAN-GFRBM	0.11 ± 0.08	0.10 ± 0.01	0.13 ± 0.01	0.09 ± 0.01	0.13 ± 0.01	0.03 ± 0.00
	DF-GFRBM	0.10 ± 0.09	0.09 ± 0.01	0.13 ± 0.01	0.10 ± 0.01	0.14 ± 0.02	0.13 ± 0.01
	FCM-GFRBM	0.06 ± 0.08	0.05 ± 0.00	0.09 ± 0.00	0.06 ± 0.00	0.08 ± 0.00	0.10 ± 0.00
	DPC-GFRBM	0.11 ± 0.03	0.13 ± 0.02	0.14 ± 0.00	0.13 ± 0.02	0.10 ± 0.01	0.11 ± 0.01
Parkinsons Updrs	CFC-GFRBM	11.93 ± 0.35	27.94 ± 0.68	42.94 ± 0.28	33.85 ± 0.60	22.22 ± 0.10	26.07 ± 0.16
	ISO-GFRBM	6.76 ± 0.10	15.08 ± 0.11	23.34 ± 0.25	17.93 ± 0.19	10.77 ± 0.11	14.27 ± 0.13
	DBSCAN-GFRBM	0.28 ± 0.02	0.49 ± 0.02	0.57 ± 0.02	0.54 ± 0.03	0.34 ± 0.02	0.08 ± 0.01
	DF-GFRBM	0.29 ± 0.03	0.49 ± 0.03	0.57 ± 0.03	0.52 ± 0.02	0.34 ± 0.02	0.26 ± 0.03
	FCM-GFRBM	0.10 ± 0.00	0.22 ± 0.00	0.31 ± 0.00	0.26 ± 0.00	0.33 ± 0.03	0.37 ± 0.01
	DPC-GFRBM	1.05 ± 0.16	1.06 ± 0.01	1.16 ± 0.01	1.09 ± 0.01	1.05 ± 0.02	0.94 ± 0.08
Compactiv	CFC-GFRBM	17.50 ± 0.16	43.59 ± 1.15	66.61 ± 0.69	52.00 ± 1.37	31.06 ± 0.17	35.91 ± 0.21
	ISO-GFRBM	9.87 ± 0.11	21.54 ± 0.19	33.79 ± 0.36	25.88 ± 0.12	14.82 ± 0.17	19.29 ± 0.26
	DBSCAN-GFRBM	0.21 ± 0.02	0.35 ± 0.03	0.38 ± 0.03	0.37 ± 0.04	0.33 ± 0.04	0.08 ± 0.01
	DF-GFRBM	0.21 ± 0.02	0.35 ± 0.03	0.37 ± 0.03	0.37 ± 0.04	0.37 ± 0.04	0.33 ± 0.03
	FCM-GFRBM	0.11 ± 0.03	0.22 ± 0.00	0.29 ± 0.00	0.39 ± 0.00	0.36 ± 0.00	0.40 ± 0.01
	DPC-GFRBM	0.94 ± 0.11	1.02 ± 0.02	1.22 ± 0.08	1.17 ± 0.12	1.29 ± 0.01	0.70 ± 0.01
Puma32h	CFC-GFRBM	14.21 ± 3.67	66.11 ± 1.98	97.51 ± 4.69	78.42 ± 4.78	47.62 ± 0.36	54.88 ± 0.46
	ISO-GFRBM	3.25 ± 0.06	4.57 ± 0.15	6.61 ± 0.35	5.00 ± 0.05	5.15 ± 0.08	7.50 ± 0.20
	DBSCAN-GFRBM	0.16 ± 0.03	0.21 ± 0.02	0.31 ± 0.02	0.25 ± 0.01	0.18 ± 0.02	0.14 ± 0.01
	DF-GFRBM	0.15 ± 0.04	0.19 ± 0.03	0.28 ± 0.02	0.25 ± 0.01	0.17 ± 0.01	0.15 ± 0.01
	FCM-GFRBM	0.09 ± 0.11	0.05 ± 0.00	0.07 ± 0.01	0.06 ± 0.00	0.07 ± 0.00	0.07 ± 0.00
	DPC-GFRBM	19.46 ± 0.79	19.35 ± 0.33	19.81 ± 0.32	19.73 ± 0.37	9.77 ± 0.15	6.79 ± 0.10

4.2 Performance on Public Datasets

Table 1 presents the granular prediction metric (V) results, which evaluate the balance between interval coverage and specificity. Higher V values (range [0,1]) indicate better overall prediction quality. The V metric findings further validate DPC-GFRBM's advantages, with our method achieving the highest scores in nearly all test cases. Across datasets, DPC-GFRBM consistently maintains V values in the 0.4–0.5 range, indicating an optimal balance between ensuring intervals contain true values and maintaining sufficiently narrow widths for meaningful predictions.

While baseline methods exhibit significant V-score fluctuations with changing cluster or alliance counts, DPC-GFRBM delivers stable performance across all configurations. For instance, on the Compactiv dataset, FCM-GFRBM's V score decreases sharply from 0.27 to 0.11 as cluster count increases, while DPC-GFRBM improves slightly from 0.43 to 0.52. This stability represents a crucial advantage in practical applications where optimal parameter configurations may not be known in advance.

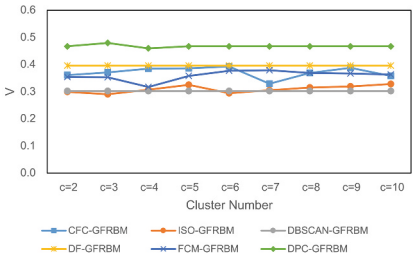


Fig. 1. Performance metric (V) on forturne1000_2024 dataset across different cluster counts

Computational Efficiency Analysis Table 2 presents the running times of all algorithms across datasets, revealing important efficiency tradeoffs. CFC-GFRBM and ISO-GFRBM exhibit the highest computational demands, with execution times increasing substantially as cluster count rises. On the Parkinsons Updrs dataset, CFC-GFRBM's runtime increases from 11.93 to 42.94 s when moving from 2 to 8 clusters due to iterative collaborative optimization.

DPC-GFRBM maintains nearly constant execution times regardless of cluster count (1.05–1.16 s on Parkinsons Updrs), offering substantial efficiency advantages over collaborative baselines while delivering superior accuracy through its hierarchical clustering approach.

4.3 Performance on Enterprise Revenue Datasets

We evaluated DPC-GFRBM on the forturne1000_2024 dataset, where enterprises naturally form alliance groups based on their collaboration patterns. For experimental purposes, companies were organized into three alliances, each with different membership sizes but sharing consistent revenue-related features.

Figure 1 presents performance comparisons across varying numbers of clusters. The granular prediction metric results reveal that while all algorithms maintain relatively stable V values, DPC-GFRBM consistently achieves the highest scores (0.45–0.50), indicating superior balance between interval coverage and specificity. This balance is particularly important in enterprise revenue prediction, where both accuracy and appropriate uncertainty quantification are essential.

4.4 Conclusion

We proposed DPC-GFRBM, a granular fuzzy rule-based model based on density peak clustering, to address enterprise revenue prediction in privacy-constrained, service-oriented industrial ecosystems. By combining two-phase clustering with interval-based representations, the model effectively captures revenue structures while preserving sensitive service data. Experiments on both benchmark and

real-world datasets show that DPC-GFRBM improves overall prediction performance by over 20% compared to existing methods, with stable accuracy and reasonable computational cost across diverse alliance settings. Future research will explore adaptive privacy controls and real-time prediction capabilities.

Acknowledgments. This study was supported by the National Key Research and Development Program of China (No. 2024YFB3309400), the National Natural Science Foundation of China (Nos. 62576088, 62303111, 62476121, 62076060), the Guangxi Science and Technology Major Program, China (No. AA24206003), and the Key Research and Development Program of Guangxi, China (No. AB2410317).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Jia, M.Q.C.: Problems and countermeasures of enterprise financial management in R&D and innovation. Invest. Entrepre. **36**(02), 59–61 (2025)
2. Du, T.T.: Research on income prediction and evaluation methods for photovoltaic power generation enterprises. China Collect. Econ. **24**, 85–8 (2024)
3. Zhou, Y., Jiang, K.: Research on data asset valuation. Invest. Entrepre. **33**(03), 96–8 (2022)
4. Bai, Z., Zhang, C.: Layout and resilience upgrading of China's manufacturing industrial chain under the perspective of global value chain restructuring. Shenzhen Social Sci. **8**(02), 36–48 (2025)
5. Ru, X.W., Zheng, X.R., Wang, Y.Y., et al.: Industrial collaborative agglomeration, knowledge spillover and industrial chain modernization. Mod. Manag. Sci. 93–102 (2024)
6. Pasha, R., Wahba, H., Lasheen, H.Y.: Examining the impact of market uncertainty on earnings forecasts: an empirical study from Egypt. J. Risk Finan. **25**(5), 774–91 (2024)
7. Easton, P.D., Kapons, M.M., Monahan, S.J., et al.: Forecasting earnings using k nearest neighbors. Account. Rev. **99**(3), 115–40 (2024)
8. Kim, J., Schonberger, B., Wasley, C., et al.: Forecasting market volatility: the role of earnings announcements. Account. Rev. **99**(4), 251–79 (2024)
9. Zhu, X.B., Hu, X.C., Yang, L., et al.: A development of fuzzy-rule-based regression models through using decision trees. IEEE Trans. Fuzzy Syst. **32**(5), 2976–86 (2024)
10. Sattler, F., Müller, K.R., Samek, W.: Clustered federated learning: model-agnostic distributed multitask optimization under privacy constraints. IEEE Trans. Neural Netw. Learn. Syst. **32**(8), 3710–22 (2021)
11. Nie, P., Roccotelli, M., Fanti, M.P., et al.: Fuzzy rule-based models for home energy consumption prediction. Energy Rep. **8**(2), 9279–89 (2022)
12. Maciel, L., Ballini, R., Gomide, F., et al.: Forecasting cryptocurrencies prices using data driven level set fuzzy models. Expert Syst. Appl. **210**(6), 118387 (2022)
13. University of California: UC Irvine Machine Learning Repository (2025). https://archive.ics.uci.edu/
14. Kaggle Inc.: Kaggle Datasets (2025). https://www.kaggle.com/

15. Hu, X.C., Shen, Y.H., Pedrycz, W., et al.: Identification of fuzzy rule-based models with collaborative fuzzy clustering. IEEE Trans. Cybern. **52**(7), 6406–19 (2022)
16. Ouyang, T.H., Pedrycz, W., Pizzi, N.J.: Rule-based modeling with DBSCAN-based information granules. IEEE Trans. Cybern. **51**(7), 3653–63 (2021)

SLIE: A Secure and Lightweight Cryptosystem for Data Sharing in IoT Healthcare Services

Ha Xuan Son[1], Nguyen Quoc Anh[2], Phat T. Tran-Truong[3,4(✉)], Le Thanh Tuan[5], and Pham Thanh Nghiem[6]

[1] RMIT University, Ho Chi Minh City, Vietnam
ha.son@rmit.edu.vn
[2] Hitachi Digital Services, Ho Chi Minh City, Vietnam
anh.quocnguyen@hitachids.com
[3] Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam
phatttt@hcmut.edu.vn
[4] Vietnam National University Ho Chi Minh City (VNU-HCM), Ho Chi Minh City, Vietnam
[5] NLS Tech Vietnam, Ho Chi Minh City, Vietnam
tuan@nlstech.net
[6] Polarista A.I, Ho Chi Minh City, Vietnam
nghiem.pham@polarista.vn

Abstract. The Internet of Medical Things (IoMT) has revolutionized healthcare by transforming medical operations into standardized, interoperable services. However, this service-oriented model introduces significant security vulnerabilities in device management and communication, which are especially critical given the sensitivity of medical data. To address these risks, this paper proposes SLIE (Secure and Lightweight Identity Encryption), a novel cryptosystem based on Wildcard Key Derivation Identity-Based Encryption (WKD-IBE). SLIE ensures scalable trust and secure omnidirectional communication through end-to-end encryption, hierarchical access control, and a lightweight key management system designed for resource-constrained devices. It incorporates constant-time operations, memory obfuscation, and expiry-based key revocation to counter side-channel, man-in-the-middle, and unauthorized access attacks, thereby ensuring compliance with standards like HIPAA and GDPR. Evaluations show that SLIE significantly outperforms RSA, with encryption and decryption times of 0.936 ms and 0.217 ms for 1KB of data, an 84.54% improvement in encryption speed, a 99.70% improvement in decryption speed, and an energy efficiency of 0.014 J/KB.

Keywords: WKD-IBE Encryption · IoT Services · Healthcare Services · IoT Security · Data Sharing

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 225–233, 2026.
https://doi.org/10.1007/978-981-95-5015-9_17

1 Introduction

The growing diversity of IoMT devices has enabled novel telehealth and telemedicine business models through the servitization of medical operations within service-oriented architectures. These architectures facilitate remote healthcare delivery by dynamically composing services to maximize value for stakeholders, including providers and patients [1]. By transforming IoT devices into IoT service providers, the IoMT ecosystem enables crowdsourced functionality, where devices not only collect data but also share resources and coordinate actuation across distinguished networks.

Despite these advancements, healthcare's unique security challenges present critical adoption barriers. Medical datasets contain 50% more sensitive information than other sectors, making them high-priority targets for cyberattacks. This is reflected in a 45% increase in ransomware incidents and an average breach cost of $10.93 million [2,3]. While centralized systems like Electronic Health Records (EHRs) and cloud platforms enhance accessibility, their API-dependent architectures introduce vulnerabilities to unauthorized access [4]. Existing cryptographic solutions, though promising, often fail to address the fundamental heterogeneity of IoMT ecosystems, which encompass devices ranging from wearables to implantables with disparate energy, storage, and computational capabilities [5]. This diversity necessitates adaptive service models that holistically address both functional (e.g., data acquisition) and non-functional (e.g., real-time performance, security) requirements.

Current security solutions remain inadequate as they either: (1) address communication security in isolation, or (2) fail to accommodate the heterogeneous nature of IoMT devices. While symmetric encryption (e.g., AES) offers efficiency, it suffers from key distribution vulnerabilities [6]. Emerging solutions based on Attribute-Based Encryption (ABE), Hardware Security Modules, Trusted Execution Environments, and Blockchain provide promising directions for decentralized key management, but require careful adaptation to IoMT's unique constraints of heterogeneous devices and strict regulatory requirements [7]. To overcome these limitations, we propose SLIE (Secure and Lightweight Identity Encryption) - a novel cryptosystem optimized for omnidirectional IoMT service communication. Our proposed cryptosystem makes three fundamental advances in IoMT security:

- **Hierarchical Key Management:** A novel WKD-IBE framework that enables efficient, fine-grained access control across distributed IoMT devices while maintaining minimal computational overhead.
- **Resource-Aware Access Control:** Non-commutative authorization mechanisms specifically optimized for constrained devices, ensuring secure communication without compromising performance.
- **Automated Compliance:** An expiry-based key revocation that enforces HIPAA/GDPR requirements while minimizing management overhead.

Our comprehensive experiments demonstrate SLIE's robust performance across diverse IoMT device capabilities (see Fig. 1), delivering three key secu-

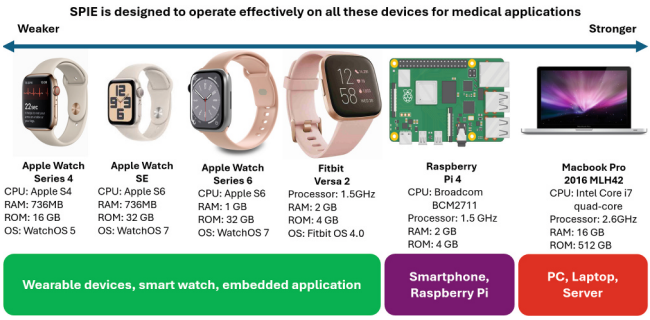


Fig. 1. SLIE compatibility spectrum across healthcare IoT devices

rity advantages: (1) end-to-end encryption with proven resistance against MITM attacks through DBDH-secure cryptographic primitives, (2) comprehensive protection against differential cryptanalysis and chosen-plaintext attacks, and (3) built-in compliance with HIPAA/GDPR regulatory requirements.

2 Literature Review

The sensitive nature of healthcare data makes it a prime target for cyberattacks, with breaches risking patient privacy, safety, and institutional reputation. Securing this data requires efficient, many-to-many, end-to-end encryption across low-power IoT devices. An ideal cryptographic framework must therefore provide robust identity authentication, key management, and access control while remaining adaptable to fast-paced emergency scenarios.

ABE presents a promising solution by integrating access policies into encryption, allowing decryption only when user attributes satisfy predefined criteria [8]. This approach simplifies key management compared to traditional PKI and enables granular access control. For instance, Satheesh and Sree [9] utilized bilinear mapping in ABE for secure decentralized cloud storage, Zhang et al. [10] applied Ciphertext-Policy ABE to streamline healthcare data encryption, and Obiri et al. [11] adopted Key-Policy ABE (KP-ABE) to enable flexible policy adjustments. However, ABE-based systems often rely on computationally intensive bilinear pairing operations, rendering them less suitable for dynamic, resource-constrained IoT environments.

As an alternative, IBE simplifies key distribution by using a user's identity (e.g., an email) as their public key, eliminating the need for pre-shared certificates [12]. Li et al. [13] further enhanced IBE with wildcard key derivation for improved access control and integrated Blockchain for decentralized authorization. While such approaches reduce overhead, they often retain a dependence on a CA, introducing a potential security bottleneck. Conversely, while lightweight cryptographic algorithms are designed for resource-constrained devices, they often lack end-to-end security structures, making them vulnerable to attacks.

Reflecting on these trade-offs, it is clear that the expansion of healthcare IoT demands a cryptographic framework that simultaneously addresses three criti-

cal dimensions for cross-device data sharing: (1) strong security guarantees, (2) efficient resource utilization, and (3) compliance with domain-specific policies.

3 Methodology

3.1 Hierarchical Access Control with Time-Bound Encryption

Our proposed framework integrates lightweight IBE with an efficient revocation mechanism, specifically designed for resource-constrained IoT devices handling sensitive cross-sectional healthcare data. Unlike existing IBE schemes that focus on general access control, SLIE addresses the unique requirements of healthcare hierarchies through role-specific temporal constraints and automatic delegation patterns that align with clinical workflows. Particularly, in Fig. 2:

- Doctor Alice can access `/HC/Data/EHR/full` for entire departmental records
- Nurse Charlie is restricted to specific patient records (`patient_1/record`)
- Third-party providers only receive access to diagnostic outcomes (`diagnosis`)

All data undergoes end-to-end encryption before cloud upload, eliminating plaintext exposure risks. The system implements temporally-graded access, with key validity periods scaled by role sensitivity to balance security and operational flexibility in time-sensitive cases (e.g., stroke, myocardial infarction).

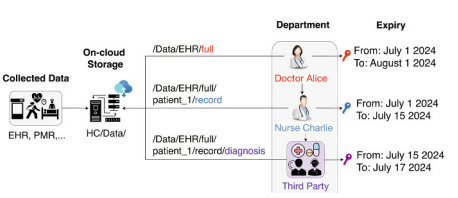


Fig. 2. Hierarchical structure of healthcare communication

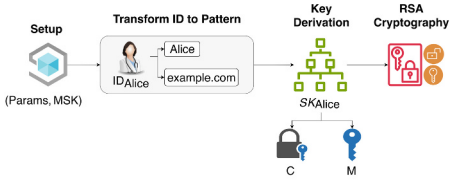


Fig. 3. IBE-based key generation (RSA for double protection is optional)

Keys have a validity period τ that is automatically determined based on healthcare role requirements. This approach addresses a critical limitation in existing IBE schemes, where revocation typically requires maintaining centralized blacklists or complex re-encryption procedures. Our time-bounded approach eliminates these overheads through automatic expiration mechanisms. A key is considered valid or expired based on the current time t (1):

$$\text{Expire}(SK_{\text{Alice}}, t) = \begin{cases} \text{Valid} & \text{if } t \leq \tau \\ \text{Invalid} & \text{if } t > \tau \end{cases} \tag{1}$$

The expiration or validity period (τ) for data access should be set through a risk assessment considering data sensitivity, role changes, regulations (e.g.,

HIPAA, GDPR), and institutional policies, as current regulations lack specific timeframes for healthcare roles. Particularly, in the U.S., with an average hospital stay of 5.5 days [14], nurses providing direct care should have a τ of 6–7 days, doctors expecting follow-ups a τ of up to 21 days, and third-party contractors (e.g., IT, medical services), with less 24/7 monitoring responsibility, a τ of up to 2 days. Family members may receive 1-day portal access. Nevertheless, this highly depends on the aforementioned risk assessment factors. These durations reflect the varying levels of access privilege and security risk associated with different roles. Specifically, the key lifecycle consists of the following steps:

1. **Initial Key Assignment**: Alice receives a private key SK_{Alice} with an expiry time τ_0.
2. **Periodic Key Refresh**: When τ_0 approaches, a new key SK'_{Alice} is issued with an extended expiry $\tau_1 > \tau_0$ (2):

$$\text{Refresh}(SK_{\text{Alice}}, \tau_0) = (SK'_{\text{Alice}}, \tau_1) \tag{2}$$

3. **Revocation by Non-Renewal**: If no new key is issued after expiry, Alice's access is revoked by default.

Access keys for staff are automatically renewed based on active employment and patient responsibilities, as verified by hospital systems. This automated revocation eliminates centralized infrastructure, ensures immediate access termination, and scales with organizational complexity.

3.2 Identity-Based Key Generation

SLIE leverages WKD-IBE [15] for key management (Fig. 3), but extends the standard approach through healthcare-specific pattern designs over $(\mathbb{Z}_p^* \cup \{\perp\})^\ell$. Our contribution lies in the development of patterns that encode clinical hierarchies and role relationships, allowing fine-grained, non-commutative delegation and efficient key derivation across multiple concurrent healthcare departments encoded as disjoint slot partitions. An initial setup entity generates the parameters and a master secret key (MSK) (3). For Alice, with identity ID_{Alice}, a private key is derived (4). Messages are encrypted (5) and decrypted using Alice's private key (6). In addition, keys are preserved on serve database.

$$(\text{Params}, \text{MSK}) = \text{Setup}(1^\kappa) \tag{3}$$

$$SK_{\text{Alice}} = \text{KeyDer}(\text{Params}, \text{MSK}, \text{Pattern}(ID_{\text{Alice}})) \tag{4}$$

$$C = \text{Encrypt}(\text{Params}, \text{Pattern}(ID_{\text{Alice}}), M) \tag{5}$$

$$M = \text{Decrypt}(\text{Params}, SK_{\text{Alice}}, C) \tag{6}$$

Unlike traditional IBE, which uses generic identity strings, SLIE's WKD-IBE framework integrates healthcare workflow patterns directly into the cryptographic structure. This encodes role hierarchy and temporal constraints, enabling automatic policy enforcement without external mechanisms.

4 Experiments and Results

The SLIE evaluation was conducted on the LifeSnaps Fitbit Dataset [16], a longitudinal repository containing over 71 million multi-modal health records from 71 participants collected over four months. The dataset's capture of 35+ clinical biomarkers at varying temporal resolutions (seconds to daily) provides a realistic benchmark for validating the security of healthcare IoT systems.

4.1 Performance on Diverse Resource-Constrained Devices

Table 1 illustrates the compatibility and performance of SLIE across a diverse range of resource-constrained healthcare IoT devices. It demonstrates SLIE's adaptability to real-world healthcare applications by evaluating key operation, such as key generation, encryption, decryption, and delegation, which are averaged over 10 runs to ensure statistical reliability. Resource-rich devices like the MacBook Pro (2016) demonstrate exceptional efficiency, with encryption and decryption times of 0.056 ms and 0.0755 ms, respectively, and a notably low delegation time of 14 ms. In contrast, ultra-low-power devices such as the Apple Watch Series 6 achieve encryption and decryption in 0.1 ms and 0.03 ms, respectively, showcasing SLIE's optimization for constrained environments. The Raspberry Pi 4 strikes a balance, with encryption and decryption times of 0.3 ms and 0.4 ms, suitable for edge computing in IoMT networks.

Table 1. Performance evaluation of SLIE on various devices

Devices	Generate key	Encrypt	Decrypt	Delegate
Apple Watch Series 6	122 ms	0.1 ms	0.03 ms	82 ms
Fitbit Versa 2	182 ms	42 ms	53 ms	83 ms
Raspberry Pi 4	122 ms	0.3 ms	0.4 ms	82 ms
Macbook Pro 2016 MLH42	99 ms	0.056 ms	0.0755 ms	14 ms

Table 2 details the power consumption and resource utilization of SLIE's core operations on representative IoT devices. Encryption and decryption operations exhibit remarkable energy efficiency, consuming 0.014 J and 0.009 J per 1 KB data respectively. These metrics highlight SLIE's suitability for battery-constrained devices, enabling prolonged operation in wearable and implantable medical devices. The delegation operation, critical for secure access control in IoMT systems, consumes 0.052 J/KB. While delegation is more resource-intensive, its moderate power usage (1.45 W) and CPU utilization (35.6%) ensure feasibility on low-power platforms. Memory usage remains modest, ranging from 12.5 MB (decryption) to 22.3 MB (delegation), further supporting SLIE's lightweight design. Collectively, these attributes demonstrate SLIE as an energy-efficient and scalable solution for secure data sharing for IoMT services, balancing performance, power, and security requirements.

Table 2. Power consumption analysis of SLIE operations

Operation	Exec. time (ms)	Power Usage (W)	Mem. Usage (MB)	CPU Util. (%)	Energy Eff. (J/KB)
Encrypt	12	1.20	16.80	27.50	0.014
Decrypt	9	0.95	12.50	19.20	0.009
Delegate	18	1.45	22.30	35.60	0.052

4.2 Scalability and Efficiency in IoT Data Encryption

Table 3 quantifies SLIE's performance across data sizes from 1 kB to 10 MB, averaged over 10 runs. For smaller data sizes (1KB), SLIE achieves encryption and decryption times of 0.936 ms and 0.217 ms, respectively, with minimal CPU usage (0.5% and 0.7%) and RAM consumption (1520 kB and 1300 kB). As data sizes increase, SLIE metrics increase linearly, demonstrating its suitability for resource-constrained IoMT devices, such as wearables and edge nodes.

Table 3. Performance metrics of SLIE under different scenarios

Data Size	Key Creation Time (ms)	Encryption Time (ms)	Encryption CPU Usage (%)	Encryption RAM Usage (KB)	Decryption Time (ms)	Decryption CPU Usage (%)	Decryption RAM Usage (KB)
1 KB	81	0.936	0.5	1520	0.217	0.7	1300
100 KB	81	3.500	1.8	5200	1.184	1.9	4500
500 KB	85	4.769	2.8	9650	1.846	3.0	8800
1 MB	83	12.677	3.2	12000	3.318	4.2	11500
5 MB	95	15.530	4.5	35600	8.993	5.5	30400
10 MB	83	25.036	8.3	55120	15.119	9.2	50800

As benchmarked in Table 4, SLIE demonstrates a compelling balance of performance and security. It drastically outperforms RSA, achieving an 84.54% faster encryption and a 99.70% faster decryption. While its raw speed is lower than the symmetric cipher ChaCha20, SLIE provides essential, built-in security features-such as hierarchical access control–that ChaCha20 lacks. These suggest that SLIE a more optimal solution for IoMT services.

Table 4. Average performance metrics comparison

Algorithm	Key Creation (ms)	Enc (ms)	Dec (ms)	CPU Usage (%)	RAM Usage (KB)
SLIE	82.3	9.73	4.49	3.66	17,535
RSA	121.85	62.93	1476.57	5.86	31,388
ChaCha20	None	3.57	2.81	2.33	14,775

5 Conclusion

In conclusion, the proposed SLIE cryptosystem establishes a robust foundation for secure IoMT services by combining the efficiency of a lightweight WKD-IBE framework with scalable trust management. It achieves a compelling performance profile, surpassing RSA in speed by over 84% for encryption and 99% for decryption while operating with high energy efficiency (0.014 J/KB). Crucially,

SLIE overcomes the key management shortcomings of symmetric systems like ChaCha20 by natively supporting hierarchical access control and automatic key revocation, ensuring both security and regulatory compliance. Looking ahead, our research will evolve to incorporate post-quantum security, optimize for next-generation implantables, and demonstrate real-world interoperability with standards such as FHIR in clinical settings.

6 Code Availability

The source code developed for this study and the scripts used to generate the results can be found at https://github.com/SonHaXuan/SLIE.

Acknowledgement. We acknowledge the support of time and facilities from Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for this study.

References

1. Bouguettaya, A., et al.: An internet of things service roadmap. Commun. ACM **64**(9), 86–95 (2021)
2. The HIPAA Journal. Healthcare ransomware attacks involve 20 percents of stored sensitive data (2024)
3. World Economic Forum. Healthcare pays the highest price of any sector for cyber-attacks that's why cyber resilience is key (2024)
4. Gates, L.G.: Cyber attacks on interoperable electronic health records: A clear and present danger. Missouri Med. **121**(1), 6–9 (2024)
5. Bouguettaya, A., et al.: A service computing manifesto: the next 10 years. Commun. ACM **60**(4), 64–72 (2017)
6. Shankar Lingam, M., Raghavendra, G.S., Kumar, A., Anand, V., Sudhakara, A.M.: Data encryption as security measure in IoT-enabled healthcare. In: Zhang, Y.-D., Senjyu, T., SO–IN, C., Joshi, A. (eds.) Smart Trends in Computing and Communications: Proceedings of SmartCom 2020. SIST, vol. 182, pp. 69–81. Springer, Singapore (2021). https://doi.org/10.1007/978-981-15-5224-3_7
7. Nabeel, M.: The many faces of end-to-end encryption and their security analysis. In: 2017 IEEE International Conference on Edge Computing (EDGE), pp. 252–259 (2017)
8. Li, J., et al.: Attribute based encryption with privacy protection and accountability for cloudiot. IEEE Trans. Cloud Comput. **10**, 762–773 (2022)
9. Satheesh, K.K., Sree, T.K.: AB-DAM: attribute-based data access model in blockchain for healthcare applications. Multimedia Tools Appl. **81**(17), 23567–23588 (2022)
10. Zhang, L., et al.: Hidden ciphertext policy attribute-based encryption with fast decryption for personal health record system. IEEE Access **7**, 33202–33213 (2019)
11. Obiri, I.A., et al.: A fully secure kp-abe scheme on prime-order bilinear groups through selective techniques. Secur. Commun. Netw. **2020**(1), 8869057 (2020)
12. Sun, Y., et al.: Efficient identity-based encryption with revocation for data privacy in internet of things. IEEE IoT-J **9**(4), 2734–2743 (2021)

13. Li, M., et al.: Nacda: naming-based access control and decentralized authorization for secure many-to-many data sharing. Electronics **12**(7), 1651 (2023)
14. Tipton, K., et al.: Interventions to decrease hospital length of stay (2021)
15. Kumar, S., et al.: {JEDI}:{Many-to-Many}{End-to-End} encryption and key delegation for {IoT}. In: 28th USENIX security symposium (USENIX Security 19), pp. 1519–1536 (2019)
16. Yfantidou, S., et al.: Lifesnaps, a 4-month multi-modal dataset capturing unobtrusive snapshots of our lives in the wild. Sci. Data **9**, 663 (2022)

Services and Business Processes

P-MDP: A Framework to Optimize NFPs of Business Processes in Uncertain Environments

Jun Peng[1], Jingwei Zhu[1], Liang Zhang[1(✉)], and Hong-Linh Truong[2]

[1] College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China
jpeng21@m.fudan.edu.cn, {jwzhu20,lzhang}@fudan.edu.cn
[2] Department of Computer Science, Aalto University, Espoo, Finland
linh.truong@aalto.fi

Abstract. In uncertain environments, compliance assurance for business processes faces the intertwined challenges posed by diverse non-functional properties (NFPs) and complex gateways in the models. While solutions like user-defined metrics and constraints empower businesses to act independently, or Markov Decision Process (MDP) enhance statistical algorithm design, they address uncertainties from different views and have not been considered together to tackle a holistic, integrated business and system uncertainties. This paper proposes Process-aware MDP (P-MDP), a framework that unifies the two divergent view to optimize NFPs for business processes operating under uncertainty. We devise a gateway-aware, lazy-evaluation reward mechanism supporting the key stakeholders – business managers – to customize metrics, algorithms, and constraints, and apply them on business processes. Experiments with the WSDREAM benchmark dataset show that P-MDP outperforms the state-of-the-art (SOTA) method constraint-satisfied service composition MDP (CSSC-MDP) at various scales. Moreover, P-MDP demonstrates superior generality and scalability, enabling stakeholders to generate better execution plans for business processes in complex scenarios.

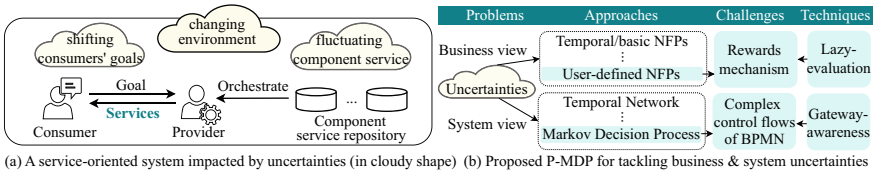
Keywords: Business process · Non-functional properties (NFPs) · Uncertainty · Markov Decision Process (MDP)

1 Introduction

Every application system suffers from uncertainties. Consider a service-oriented system (Fig. 1) where a service provider (*this article's primary focus*) grapples with evolving consumer goals, fluctuating component service supply chains, and a dynamic operating environment to maintain its commitment to consumers. Many efforts have been devoted to addressing challenges stemming from uncertainties [2–4,8–10,12,16]. Among them, a prominent category [2,3,8–10,12] leverages Markov Decision Processes (MDP) and reinforcement learning (RL) algorithms to learn an optimal policy that maximizes expected cumulative reward – various business goals despite systemic uncertainties [11].

However, these MDP-based approaches handle uncertainty only within a narrow scope, yielding two critical limitations:

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 237–252, 2026.
https://doi.org/10.1007/978-981-95-5015-9_18



(a) A service-oriented system impacted by uncertainties (in cloudy shape) (b) Proposed P-MDP for tackling business & system uncertainties

Fig. 1. Uncertainty presents fundamental challenges for service-oriented systems. (a) A typical service-oriented system and diverse uncertainty factors (in cloudy shapes). (b) Available solutions separated from business view and system view and the proposed P-MDP framework unifying the two view (highlighted in blue). (Color figure online)

Unrealistic Assumptions: Studies like [8,10,12] ignore runtime uncertainty (e.g., cost fluctuations) and, to calculate immediate rewards, rely on the assumptions of *variable independence* and *monotonic additive aggregation*, thus only supporting basic (built-in) non-functional properties (NFPs[1]). Others [2,3,9] incorporate runtime uncertainty into MDP by encoding constraint satisfaction levels in the state, but still lack capability of user-defined NFPs.

Isolated Optimization: These approaches model service execution dependencies (functional properties, FPs) in MDP but impose a total order on services *within each path*, thereby ignoring alternative branches/concurrent interleaving. Treating each path in isolation paralyzes the search for a globally optimal policy.

These limitations motivate our core research problem: *optimizing NFPs of business processes in uncertain environments*. Consolidating fragmented services into unified business processes enables globally optimization, while supporting user-defined NFPs expands practical applicability. To address these dual requirements, in our approach in Fig. 1b, we propose a Process-aware Markov Decision Process (P-MDP) framework, with the following contributions:

- *Supporting better business considerations via user-defined NFPs.* By replacing immediate per-timestep reward computation with constraint-oriented lazy-evaluation, P-MDP does not calculate the reward for each constraint until its relevant variables are all instantiated. This eliminates restrictive, unrealistic assumptions of variable independence and monotonicity in prior methods [2,3,8–10,12], naturally supporting user-defined NFPs;
- *Yielding globally optimal policy via comprehensive process structure support.* P-MDP explicitly models complex FPs induced by exclusive/parallel gateways[2] during NFP optimization under uncertainty. Gateway-awareness resolves solution space fragmentation in existing MDP-based methods [2,3,8–10,12], and effectively handles unknown future with multiple selective execution paths and temporal uncertainty in concurrent activities.

For the rest, we model uncertainties and user-defined NFPs in Sect. 2. In Sect. 3, we formulate a P-MDP environment and learn its optimal policy via a

[1] We adopt NFPs as an umbrella term for related concepts in service-oriented computing literature, such as metrics, quality of service (QoS), service level indicators (SLIs), and their associated constraints.

[2] Other gateway types (inclusive/event-based [5] and bounded loops [6]) are reducible to exclusive/parallel equivalents.

tailored deep RL architecture. Section 4 evaluates P-MDP against a SOTA method and its extended capabilities. Finally, we discuss the framework in Sect. 5, summarize related work in Sect. 6, and conclude with future directions in Sect. 7.

2 Modeling Uncertainties and User-Defined NFPs

Before detailing our techniques, we use a running example to explain why we need to optimize NFPs of business processes, and then how we realize it.

Running Example: Consider a service-oriented system for a travel agency in Fig. 2, where the agency (*provider*) integrates third-party airline/rail/hotel component services from various *repositories* to offer the traveler (*consumer*) traveling packages (*services*). For example, `Package 1` prioritizes flights with 3-star hotels, while `Package 2` features 5-star hotels but slower rail transportation. Such a service is further characterized as FPs – activities and control flows on the left, and NFPs – metrics and constraints on the right of Fig. 2, respectively.

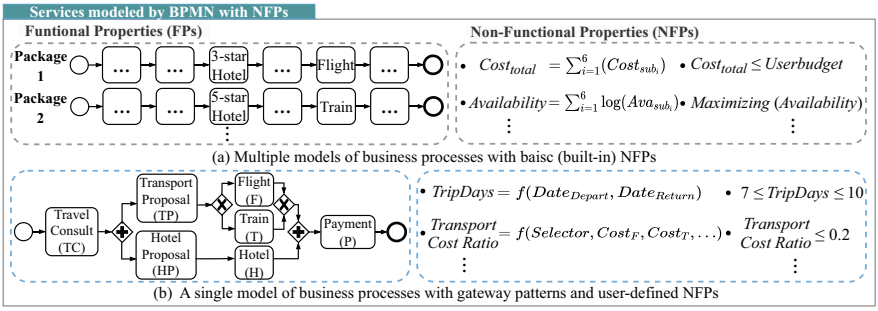


Fig. 2. Modeling a single business process with gateway patterns and user-defined NFPs in complex scenarios: (a) divergent business processes with built-in NFPs and (b) single business process with user-defined NFPs and gateway-awareness.

Required Capabilities of User-Defined NFPs: As shown in Fig. 1, there are multiple sources of uncertainty (depicted in cloud shapes). For example, daily trains and intermittent flights lead to the number of trip days can only be calculated by departure and return dates. This requirement paralyses existing MDP-based approaches [2,3,8–10,12] due to their two unrealistic assumptions. The variable `TripDays` depends on two interrelated variables, neither alone is sufficient to evaluate its reward. In addition, for the travel agency, it is necessary to control the proportion of transportation cost not exceeding a certain ratio, e.g., 20%, of the total cost, to ensure hotel comfort. Obviously, this proportion – influenced by interrelated costs like transportation, hotel, and others – exhibits not only variable dependence (similar to `TripDays`) but also non-monotonic behavior, no longer being simply additive aggregation; for instance, when hotel costs drop faster than transportation costs, the transportation proportion may rise even as its absolute cost decreases.

User-Defined Metrics and Constraints for Uncertainties: In response to the above requirements, we devise a constraint-oriented lazy reward mechanism, which enables modeling NFP parameters as user-defined multivariate functions:

User-defined Metrics as Multivariate Functions *udm* takes the form

$$udm ::= f(x_1, \ldots, x_n) \mid f(x_1, \ldots, x_n; udm_1, \ldots, udm_m) \tag{1}$$

here $x_1, \ldots, x_n$ correspond to factors (variables) across services in the process and might be influenced by runtime uncertainty (e.g., fluctuating ticket prices) from environment and controllable deterministic factors (e.g., designated flight suppliers), together with temporal and logic dependencies induced by FPs. This combination of influencing factors and dependencies forms the configuration that provides the foundation for effective MDP and RL implementation in NFPs optimization; the functor f may serve as either an identity function preserving original variables, or an arbitrary function composed via arithmetic operators – with these operators are extensible (as P-MDP imposes no restrictions on *udm*) to adapt to evolving business needs. For example, the `TransportCostRatio` in Fig. 2b incorporates the variable `Selector` and other *udm*s like $\texttt{Cost}_{\texttt{F}}$.

User-defined Constraints express business constraints as a *udc*,

$$udc ::= udm \bowtie d \mid udc \triangle udc \mid \text{Max}(udm) \mid \text{Min}(udm) \tag{2}$$

here, $\bowtie$ denotes relational operators (e.g., $\leq, \geq, =$), comparing a *udm* to a constant/*udm* d (e.g., `TripDays` ≤ 10); $\triangle$ logical operators (e.g., $\wedge, \vee, \neg$) connecting two *udc*s (e.g., `TripDays` $\leq 10 \wedge$ `TripDays` ≥ 7; and Max(*udm*), Min(*udm*) denote optimization operators targeting *udm* maximization or minimization.

Attaching User-Defined NFPs to a Business Process, Resulting in an Uncertainty-Aware Graph: We transform the business process into a weighted directed graph $G(V, E)$ as shown in Fig. 3. The V encodes the attributes both the FPs and the NFPs, and E represents the probabilities of traditions. More specifically (referring to Table 1),

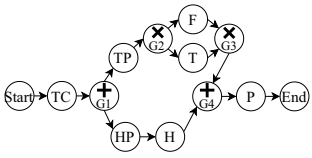


Fig. 3. The Graph Representation of BPMN in Fig. 2b)

Table 1. Some Vertices Attributes in G (Fig. 3)

Vertex	...	G1	F	P
Vertex Type	...	spg	oth	oth
Controllable	...	$\emptyset$	$\{Air_F, ...\}$	...
Uncertain	...	$\emptyset$	$\{Cost_F, ...\}$	...
Stack	...	[Root]	[Root, G1]	[Root]
Vertices on Paths of o_v	...	{{Start, TC}}	{{TP, G2}}	{{..., H,F, ...}, {..., H,T, ...}}

Vertices Set *V*. A $v \in V$ is characterized by the tuple (T_v, C_v, U_v, H_v), where:

– *Vertex Type* T_v is one of six types, i.e., $T_v \in \{$`seg, meg, spg, mpg, ste, oth`$\}$, representing `split/merge exclusive gateways, split/merge parallel gateways, start events`, and `other elements`, respectively;

- *Controllable Variables Set* C_v contains any variable x only if $T_v \in \{\texttt{ste}, \texttt{oth}\}$; else $C_v = \emptyset$. These variables represent control actions executed by the service provider (e.g., the travel agency) to optimize NFPs. For example, selecting a flight supplier (Air_F) with appropriate dates from an airline repository constitutes such an action. While actions C_v directly influence NFPs (e.g., `TripDays`), their effects may be compromised by uncontrollable variables (defined below) within the same vertex or those variables in downstream vertices during process execution.
- *Uncontrollable Variables Set* U_v contains any variable x only if $T_v \in \{\texttt{ste}, \texttt{oth}\}$; else $U_v = \emptyset$. It models runtime uncertainty beyond the provider's control: selecting low-cost carriers does not guarantee lower flight costs ($\texttt{Cost}_\texttt{F}$), as prices remain subject to market dynamics and airline pricing algorithms. Such external fluctuations can weaken controllable decisions' efficacy.
- *The Hierarchy Stack* H_v encodes a vertex's concurrency structure through an ordered sequence where the stack top (rightmost element) corresponds to the innermost concurrent block. Each element records a) the split parallel gateway (`spg`) initiating the concurrent block; b) the vertex's concurrent flow index information (omit in Table 1) within the block; ordering progresses outward along the stack (leftward elements to outer blocks). Further detail information is referred to the supplementary Git[3]

Weighted Directed Arcs Set E. Each directed arc $arc(v_i, v_j, p_{i,j}) \in E$ models a state transition from v_i to v_j (corresponding to current-to-next state transitions in P-MDP) weighted by execution probability $p_{i,j}$. While most arcs have deterministic probability $p_{i,j} = 1$, those originating from `seg` vertices exhibit probabilistic branching: their outgoing arcs' weights sum to 1, enabling multiple potential execution paths. For these `seg`-originating arcs, $p_{i,j}$ may be defined as controllable/uncontrollable variables in v_i's predecessors to support NFP optimization (e.g., path scheduling), or model path uncertainty.

Temporal and Logical Dependencies of Vertices. Sequence flows establish core temporal dependencies among vertices in $G(V, E)$ (i.e., execution order), enabling each vertex v to observe preceding vertices' execution states. Gateways further complicate these dependencies: parallel gateways introduce temporal interleaving, while exclusive gateways add logical branching (introducing logical dependencies). This dual complexity generates diverse states, which in turn influence NFP optimization. The observability set O is structured as follows:

- Each $o_v \in O$ comprises *all feasible execution paths* from the initial vertex of v's concurrent flow (within its innermost concurrent block) to v, this path-based dependency avoids inter-flow variable tracking and enables concurrent hierarchy isolation, thereby simplifying the construction of the P-MDP environment when handling gateway-induced complexity.

[3] Public code and documents: https://github.com/SOARingLab/PMDP-ICSOC2025.

– Each path $\sigma \in o_v$ contains observable variables defined at the vertices along σ. Exclusive gateways introduce path branching, making $|o_v|$ equal to the number of topologically feasible paths.

Vertices without explicit concurrency blocks (e.g., `ste`) are assigned to an implicit root block (single-flow) to ensure uniform hierarchical context. The formal definitions of observability set O are provided in the supplementary Git (See Footnote 3).

3 Optimizing NFPs in the Context of Business Processes

3.1 Establishing P-MDP Environment for RL

We cast NFPs optimization under uncertainty as a sequential decision problem – maximizing expected cumulative reward (e.g., constraint satisfaction [2,3,9], services quality [8,10,12]), and model it as a P-MDP $(\mathcal{H}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, which embeds business-process execution semantics into the environment, enabling us to devise RL algorithms to optimize user-defined NFPs under uncertainty.

Horizon Space of P-MDP. Each P-MDP time-step is defined by the set of currently active vertices in G (e,g., active activities) during the process enactment. Formally, the horizon space is $\mathcal{H} = \{\mathbb{v} \mid \mathbb{v}$ is an active-vertex configuration in $G\}$. A active-vertex set $\mathbb{v}$ may be (i) a singleton $\mathbb{v} = \{v\}$ (sequential step), (ii) a set $\mathbb{v} = \{v_1, \ldots, v_n\}$ (n concurrent flows), (iii) a singleton $\mathbb{v} = \{v\}$ chosen from path branching (exclusive choice), or (iv) any nested combination of these. These sets constitute the P-MDP time-steps, capturing sequence, concurrency, choice and nesting. The construction rules for $\mathcal{H}$ are provided in the supplementary Git (see Footnote 3).

State Space of P-MDP. The state at each time-step is dynamically multidimensional, reflecting the concurrency hierarchy:

– *Single Active Vertex*: When $\mathbb{v} = \{v\}$ (e.g., the process instance is not executing within any concurrent or nested blocks), the state $S_v \in \mathcal{S}_\mathbb{v}$ is 1-dimensional and the state space is defined as follows:

$$\mathcal{S}_\mathbb{v} = \bigcup\nolimits_{\sigma \in o_v} \left(\coprod\nolimits_{x \in \sigma} \mathrm{dom}(x) \right) \tag{3}$$

where $\mathrm{dom}(x)$ denotes x's domain, and $\amalg$ denotes their unordered Cartesian product ($\otimes$). For instance, $\sigma = \{x_1, x_2\}$: x_1 (hotel cost, $\mathrm{dom}(x_1) = \{100, 200\}$) and x_2 (flight cost, $\mathrm{dom}(x_2) = \{300, 400\}$), so $\coprod\limits_{x \in \sigma} \mathrm{dom}(x) = \mathrm{dom}(x_1) \otimes \mathrm{dom}(x_2) = \{(100, 300), \ldots\}$, each pair representing a state.

– *Concurrent or Nested Active Vertices*: For $\mathbb{v} = \{v_1, v_2, \ldots, v_n\}$ with n active vertices, the state is n-dimensional for n concurrent flows. For nested concurrency blocks, the state may involve nested sets of active vertices (e.g., $\mathbb{v} = \{\mathbb{v}_1, \mathbb{v}_2, \ldots, \mathbb{v}_n\}$). The state space is recursively defined as follows:

$$\mathcal{S}^{\mathrm{cl}}_\mathbb{v} = \mathcal{S}_{v_h} \otimes \left(\coprod\nolimits_{\mathbb{v}_i \in \mathbb{v}} \mathcal{S}^*_{\mathbb{v}_i} \right) \tag{4}$$

where v_h is the current concurrency block's split parallel gateway (`spg`) vertex (obtained via GetTop(H_{v_h})); under *concurrent hierarchy isolation*, v_h's state space ($\mathcal{S}_{v_h}$) combines with inner active vertices' state spaces (so $\mathcal{S}_{v_h} \otimes ...$), and $\mathcal{S}^*_{\mathbb{v}_i}$ is $\mathcal{S}_{\mathbb{v}_i}$ or $\mathcal{S}^{\mathrm{d}}_{\mathbb{v}_i}$, based on if $\mathbb{v}_i$ has nested concurrency blocks.

Thus, the state at each time-step is a random variable $S_{\mathbb{v}} \in \mathcal{S}^{\mathrm{d}}_{\mathbb{v}}$, and the overall state space is $\mathcal{S} = \bigcup_{\mathbb{v} \in \mathcal{H}} \mathcal{S}^{\mathrm{d}}_{\mathbb{v}}$.

Action Space of P-MDP. The action space dynamically adjusts its dimensionality (e.g., a sub-action space) at each time-step based on the current state:

- *For 1-dimensional state at a single active vertex* ($\mathbb{v} = \{v\}$): The action space consists of the domains of all controllable variables at v:

$$\mathcal{A}_{\mathbb{v}} = \coprod_{x \in C_v} dom(x) \tag{5}$$

- *For n-dimensional state at concurrent or nested active vertices* ($\mathbb{v} = \{\mathbb{v}_1, \mathbb{v}_2, \ldots, \mathbb{v}_n\}$): The action space is recursively defined as a nested structure:

$$\mathcal{A}^{\mathrm{d}}_{\mathbb{v}} = \coprod_{\mathbb{v}_i \in \mathbb{v}} \mathcal{A}^*_{\mathbb{v}_i} \tag{6}$$

where $\mathcal{A}^*_{\mathbb{v}_i}$ denotes either $\mathcal{A}_{\mathbb{v}_i}$ or $\mathcal{A}^{\mathrm{d}}_{\mathbb{v}_i}$, depending on whether $\mathbb{v}_i$ contains nested concurrency blocks.

Thus, the action at each state is a random variable $A_{\mathbb{v}} \in \mathcal{A}^{\mathrm{d}}_{\mathbb{v}}$, and the overall action space is $\mathcal{A} = \bigcup_{\mathbb{v} \in \mathcal{H}} \mathcal{A}^{\mathrm{d}}_{\mathbb{v}}$.

State Transition Function of P-MDP. The state transition function specifies the probability that action a in state s at the current time-step t will result in state s' at the next time-step $t+1$. This function (*gateway-awareness*) is governed by the directed arcs and vertex types in $G(V, E)$, and the concurrency hierarchy of the active vertices in $\mathbb{v}$:

- *Ordinary Transition*: For a 1-dimensional state with an $arc_{i,j} \in E$ and $T_{v_j} \neq$ `mpg`, indicating routine sequential flow execution, the function is defined:

$$\mathcal{P}(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = p_{i,j} \times \rho \times \prod_{x \in U_{v_i}} x(s') \tag{7}$$

where $S_t \in \mathcal{S}_{\mathbb{v}}$ and $A_t \in \mathcal{A}_{\mathbb{v}}$ represent the current state and action with $\mathbb{v} = \{v_i\}$, $S_{t+1} \in \mathcal{S}_{\mathbb{v}'}$ denotes the next state with $\mathbb{v}' = \{v_j\}$, $p_{i,j}$ is the execution probability of $arc_{i,j}$, and $x(s')$ denotes the probability that x takes a value in s'. For example, in model-based scenarios with explicit distributions, if x has a 40% chance of taking 1, then $x(s') = 0.4$ when $x = 1$ in s'; in model-free scenarios, $x(s')$ is simplified to 1 (as no explicit distribution is modeled), so $\mathcal{P}$ reduces to $p_{i,j} \times \rho$. For ρ, it takes values in $\{0, 1\}$: in concurrent flows, $\rho = 1$ (state transition) or $\rho = 0$ (self-transition); otherwise, ρ is always 1.
- *Divergence Transition*: For a 1-dimensional state with $n \geq 2$ arcs of the same source vertex `spg`, indicating the divergence from v_i into $n \geq 2$ concurrent flows (yielding n-dimensional states), the function is defined:

$$\mathcal{DP}(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = \prod_{v_j \in \mathbb{v}'} \mathcal{P}(s'_j \mid s, a) \tag{8}$$

where $S_t \in \mathcal{S}_{\mathbb{v}}$, $A_t \in \mathcal{A}_{\mathbb{v}}$ with $\mathbb{v} = \{v_i\}$, and $S_{t+1} \in \mathcal{S}^{\mathrm{d}}{}_{\mathbb{v}'}$ consists of states $(s'_1, \ldots, s'_n)$; s'_j denotes the state of concurrent flow $v_j \in \mathbb{v}' = \{v_1, \ldots, v_n\}$.

– *Synchronization Transition*: For a n-dimensional state, if an $arc_{i,j}$ in one of the concurrent flows has its target vertex v_j with $T_{v_j} = \texttt{mpg}$, indicating the flow completion waiting for synchronization, it will perform self-transitions until all flows in the same concurrent block complete. The function is defined:

$$\mathcal{SP}(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = 1 \tag{9}$$

where $S_t \in \mathcal{S}^{\mathrm{d}}_{\mathbb{v}}$, $A_t \in \mathcal{A}^{\mathrm{d}}_{\mathbb{v}}$, $\mathbb{v} = \{v_1, \ldots, v_n\}$, and $S_{t+1} \in \mathcal{S}_{\mathbb{v}'}$ with $\mathbb{v}' = \{v_j\}$.

– *n-dimensional Transition*: For transitions between n-dimensional state, the function is recursively defined and accounts for nested concurrency blocks:

$$\mathcal{P}^{\mathrm{d}}(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = \textstyle\prod_{\mathbb{v}_i \in \mathbb{v}} \mathcal{P}^*(s' \mid s_i, a_i) \tag{10}$$

where $S_t \in \mathcal{S}^{\mathrm{d}}_{\mathbb{v}}$ and $A_t \in \mathcal{A}^{\mathrm{d}}_{\mathbb{v}}$ with $\mathbb{v} = \{\mathbb{v}_1, \cdots, \mathbb{v}_n\}$, $S_{t+1} \in \mathcal{S}^{\mathrm{d}}_{\mathbb{v}'}$ with $\mathbb{v}' = \{\mathbb{v}'_1, \cdots, \mathbb{v}'_n\}$, and $\mathcal{P}^*(s'_i \mid s_i, a_i)$ denotes the transition probability for $\mathbb{v}_i$ that can be either $\mathcal{P}$ for ordinary cases, $\mathcal{DP}$ for divergence, $\mathcal{SP}$ for synchronization, or $\mathcal{P}^{\mathrm{d}}$ for nested multidimensional state depending on $\mathbb{v}_i$.

Thus, the overall transition function $\mathcal{P}^{\mathrm{d}}(s' \mid s, a)$ combines the transition probabilities of its constituent sub-states, ensuring consistency with concurrency structure, gateway-awareness, and the dynamically multidimensional state space.

Reward Function of P-MDP. P-MDP uses constraint-oriented, lazy rewards mechanism once all variables relevant to a constraint (e.g., departure and return dates) are assigned – enabling user-defined NFPs. Each constraint follows three stages: **ini**tial (unassigned), **int**ermediate (assigned), **fin**al (evaluated):

– *Hard vs. Soft Constraints*: Constraints are split into hard (HC) and soft (SC). The normalized reward ($[0, 1]$) is a weighted sum of HC and SC rewards:

$$\mathcal{R}(s, a, s') = w_{HC} \times \mathcal{R}^{HC}(s, a, s') + w_{SC} \times \mathcal{R}^{SC}(s, a, s') \tag{11}$$

– *Hard Constraints*: Defined by relational or logical operators. Then:

$$\mathcal{R}^{HC}(s, a, s') = \textstyle\sum_{hc_i \in HC_{int}} (w_{hc_i} \times check(hc_i) - pt_{hc_i}) \tag{12}$$

where $check(hc_i) \in \{0, 1\}$ indicates satisfaction, $pt_{hc_i} \in \{0, 1\}$ is a penalty.

– *Soft Constraints*: Include relational, logical, or optimization operators. Then:

$$\mathcal{R}^{SC}(s, a, s') = \textstyle\sum_{sc_\in SC_{int}} (w_{sc_i} \times (check^*(sc_i) - pt_{sc_i})) \tag{13}$$

where $check^*(sc_i) \in [0, 1]$ (binary for relational/logical, normalized for optimization), and penalty $pt_{sc_i} \in \{0, 1\}$ only applies to non-optimization types.

This mechanism ensures HC are prioritized: since $\mathcal{R}^{HC} = \sum w_{hc_i} \times check(hc_i) \leq 1$, any unsatisfied hc_i (with $pt_{hc_i} = 1$) forces $\mathcal{R}^{HC} \leq 0$, while $\mathcal{R}^{SC} \in (-\infty, 1]$ depending on satisfaction and penalties. Each hc_i or sc_i carries a user-specified weight (w_{hc_i} or w_{sc_i}), and weights within each class summing to 1. With default $w_{HC} = w_{SC} = 0.5$, the total reward $\mathcal{R} > 0.5$ only if all HC are satisfied – thus, maximizing $\mathcal{R}$ naturally leads P-MDP to fulfill HC before optimizing SC.

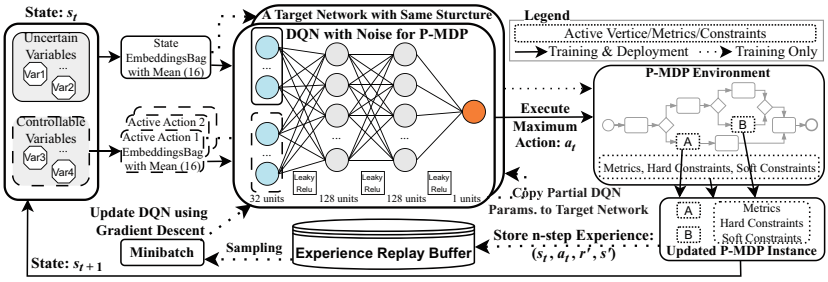


Fig. 4. The integrated architecture of Prioritized Noisy Double Deep Q-Networks with multi-step learning for P-MDP

3.2 Prioritized Noisy Double Deep Q-Networks (DQN) with Multi-Step Learning for P-MDP

We integrate P-MDP with a tailored DQN [11] architecture (Fig. 4) to accommodate two characteristics: (i) The action space grows exponentially with the number of controllable variables. By exploiting their natural distribution across activities, we identify activity-related NFPs, decompose the global action space into per-step sub-spaces, and feed active actions as part of the state to enable scalar outputs for each sub-action; (ii) Delayed-reward from lazy reward mechanism are mitigated via multi-step learning and prioritized experience replay.

P-MDP casts a business process – with its user-defined NFPs – into an interactive RL environment, where an agent learns the optimal policy $\pi^*(s)$ that maximizes the expected return of future discounted rewards $\mathbb{E}[\mathcal{G}_t \mid S_t = s, A_t = \pi^*(s)]$ after taking action $\pi^*(s)$ in state s, where $\mathcal{G}_t \equiv \mathcal{G}_{\tau_{t:T}} = \sum_{k=t}^{T-1} \gamma^{k-t} R_k$, along trajectories $\tau_{t:T} = \{S_t, A_t, R_t, S_{t+1}, \ldots, S_T\}$. DQN, a temporal difference (TD) based algorithms, approximates the optimal action-value function $Q_*(s, a)$, yielding $\pi^*(s) = \arg\max_a Q_*(s, a)$. By the Bellman optimality equation [11],

$$\begin{aligned} Q_*(s,a) &= \max_{\pi} \mathbb{E}\left[R_t + \gamma \mathcal{G}_{t+1} \mid S_t = s, A_t = a\right] \\ &= \mathbb{E}_{S_{t+1} \sim \mathcal{P}^{\text{d}}}[R_t + \gamma \max_{A_{t+1} \in \mathcal{A}_{S_{t+1}}} Q_*(S_{t+1}, A_{t+1})] \approx Q(s,a;\theta) \end{aligned} \tag{14}$$

where R_t is the reward at timestep t, $\gamma \in [0, 1]$ the discount factor, and $Q(s, a; \theta)$ the neural network approximation (DQN) for P-MDP's high-dimensional spaces.

To operationalize this, we employ a noisy double DQN enhanced by multi-step learning and prioritized experience replay (PER) to learn $Q(s, a; \theta)$. The agent interacts with the P-MDP environment to generate 4-tuple experiences $(S_t = s, A_t = a, R_{t:t+n} = \sum_{k=t}^{\min(t+n-1,T-1)} \gamma^{k-t} r_k = r', S_{\min(t+n,T)} = s')$ within $\tau_{t:T}$, which are stored in the PER buffer for subsequent training via Algorithm 1. Key techniques [7], with mathematical details in supplementary[3], are as follows:

- *Multi-step Learning:* uses n-step return r' as the TD target to accelerate delayed reward propagation.
- *Double DQN:* employs online $Q(s, a; \theta)$ and target $Q(s, a; \theta^-)$ networks to reduce bootstrapping bias and stabilize learning.

- *Prioritized Experience Replay:* samples experience 4-tuples (s, a, r', s') using priority weight $\omega \in [0, 1]$ (0 for uniform, 1 for TD-error prioritization), and apply importance weights $\beta \in [0, 1]$ to correct bias. This enables that infrequent but critical delayed-reward experiences are learned more efficiently.
- *Noisy Nets:* integrates learnable noise parameter $\widetilde{\theta} = \theta + \theta_{\text{noisy}}$ for adaptive exploration and exploitation, replacing the manual ϵ-greedy strategy.

Algorithm 1: Prioritized Noisy Double DQN with multi-step learning for P-MDP

Input : Learning rate α, PER params ω and β, Discount factor γ and n in n-step TD
Output: Trained Q for P-MDP

Initialize online Q with $\widetilde{\theta}$, then $\widetilde{\theta}^{-} \leftarrow \widetilde{\theta}$ for target Q
Initialize PER Buffer $B_{PER} \leftarrow \emptyset$

for *each episode* **do**
  Reset P-MDP Environment
  $s \leftarrow s_{initial}$
  while ***not*** *terminal state* s **do**
    /* **Interact with P-MDP** */
    $\mathcal{A}_s^{\text{d}} \leftarrow$ get all active actions of s
    Reset noise of online Q
    $a \leftarrow \arg\max_{a \in \mathcal{A}_s^{\text{d}}} Q(s, a; \widetilde{\theta})$
    Execute a, observe reward r, next state s'
    /* **Update PER Buffer** */
    Compute n-step return r' and store experience (s, a, r', s') in B_{PER}
    /* **Training** */
    if B_{PER} *warm-up* **then**
      Sample a mini-batch from B_{PER} with ω
      Update online Q by minimizing n-step TD error (gradient descent) with β
      Soft-update target Q
    $s \leftarrow s'$

return *Online Q with* $\widetilde{\theta}$*, where params* θ_{noisy} *are disabled*

4 Experiments

To validate P-MDP, we implemented the framework as a Python-based toolset[3] that takes a BPMN model annotated with user-defined NFPs as input, converts it to a graph representation, constructs the corresponding P-MDP environment, and employs RL to optimize NFPs. The RL optimization architecture is in Fig. 4.

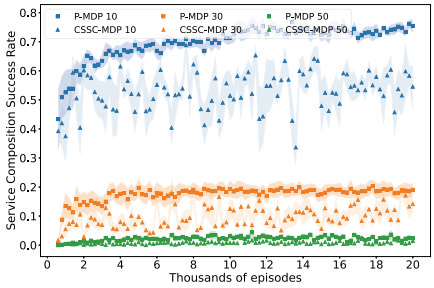
In this section, we presented two complementary experiments: (i) comparing performance with constraint-satisfied service composition MDP (CSSC-MDP), and (ii) evaluating P-MDP's unique modeling capabilities unattainable by prior work. Further details of the experiment can be found in the supplementary. All experiments were carried out on an AMD Ryzen 7 8845H with 32GB RAM and an RTX 4070 laptop GPU.

4.1 Benchmark Comparison

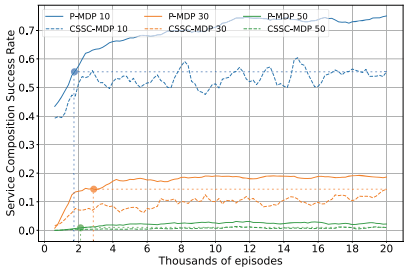
Given CSSC-MDP's superior modeling capability in handling constraints and uncertainty, and its closest alignment with P-MDP, we conduct a comparison by

restricting P-MDP to CSSC-MDP's modeling scope. For fairness, both frameworks employ identical FPs and NFPs configurations. Following CSSC-MDP's experimental setup [9], we utilize the WSDREAM QoS dataset2 [17] that contains 339 users' response time (rt) and throughput (tp) records across 5,825 web services. These services are grouped into 50 abstract services (each with 116 candidates), where each candidate's 339 QoS records simulate runtime uncertainty. We adopt CSSC-MDP's NFPs as constraints: $rt < 22\,\text{s}$ and $tp > 13\,\text{Kbps}$, where $rt = \sum_{i=1}^{m} rt_i$ and $tp = \min_i(tp_i)$ for m sequential abstract services. P-MDP models these two NFPs as soft constraints with $w_{SC} = 1$ during optimization.

For CSSC-MDP, we adopted the recommended hyperparameters: linearly decaying learning rate α (initial 0.5, decreasing by 0.000007 per episode) and ϵ-greedy exploration ($\epsilon = 0.6$). P-MDP employed fixed $\alpha = 0.00375$, default $\omega = 0.5$, exponentially adjusted β (0.1→0.6), and prioritized experience replay (PER) with batch size 16. We set $n = 1$, and discount factor $\gamma = 0.1$ reflecting CSSC-MDP's independent and monotonic metric (rt) and simple threshold filter (tp) – enabling per-step reward shaping without significant future reward impact. PER buffer sizes were scaled at 10K/20K/30K for 10/30/50 – abstract service experiment respective, with replay commencing after 500 warm-up samples. All experiments excuted 20K episodes across three scales (10/30/50 abstract services) using 5 random seeds, pausing every 200 episodes for 500 policy rollouts to evaluate success rates (simultaneously satisfying $rt < 22\,\text{s}$ and $tp > 13\,\text{Kbps}$).



(a) Convergence Comparison: Mean ±1 Standard Error (5 Random Seeds)



(b) Trend Curves with 5-Pt Moving Avg. Smoothing

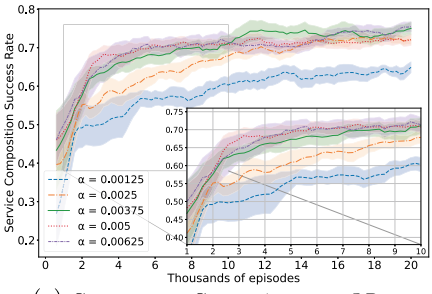
Metric	Scale 10		Scale 30		Scale 50	
	CSSC	P-MDP	CSSC	P-MDP	CSSC	P-MDP
Episode (●)	1.0	**0.088**	1.0	**0.151**	1.0	**0.118**
Time (●)	1.0	**0.840**	1.0	**0.859**	1.0	**0.568**
± SEQ (●)	± 1.0	± **0.853**	± **1.0**	± 1.070	± 1.0	± **0.721**

(c) Comparison of CSSC (CSSC-MDP) and P-MDP across scales in terms of episode consumption ratio (Episode), training time (Time), standard error of policy quality (SEQ). All values are normalized to CSSC-MDP's performance (1.0) as baseline.

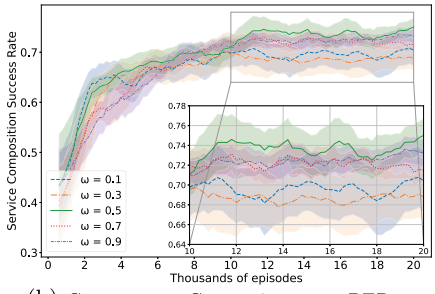
Fig. 5. Convergence behavior and numerical comparisons under varying scales. Metrics marked with represent performance differences in terms of Episode, Time, and Quality under CSSC-MDP's final (20K-episode) policy conditions.

Figure 5a shows that P-MDP consistently outperforms CSSC-MDP in success rates with smaller standard error (shaded regions) and faster convergence – CSSC-MDP starts converging at 20K–50K episodes [9]. Figure 5b depicts convergence dynamics with 5-point moving average smoothing. Figure 5c quantifies P-MDP's superiority: fewer episodes, less training time (up to 43.2% temporal reduction at scale 50 (0.568 vs. 1.0)). P-MDP exhibits lower SEQ overall, though scale 30 shows a marginal degradation (1.070 vs. 1.0).

To analyze P-MDP's hyperparameter sensitivity, we investigated learning rate (α) and PER weight (ω) at scale 10 (Fig. 6a–6b). Figure 6c quantifies impacts via three metrics[4]. Learning rate (α) drives early convergence (1K–10K episodes): Higher α accelerates initial learning, critical for costly episodes (e.g., physical service invocations). PER weight (ω) balances exploration-exploitation: When $\omega < 0.5$, exploration is favored but reduces high TD error sampling, leading to higher late-stage SE (10K–20K). A balanced trade-off occurs at $\omega = 0.5$. For $\omega > 0.5$, policy stability improves (lower SE), with diminishing gains as $\omega \geq 0.7$. For computational scalability (Fig. 6d), training time per episode is: Unaffected by candidate service number (negligible impact). Positively correlated



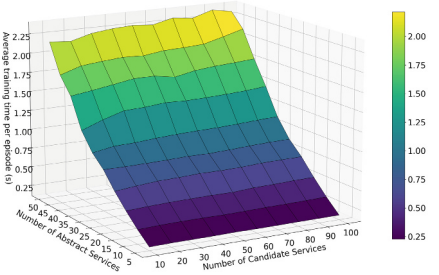
(a) Convergence Comparison over LR α



(b) Convergence Comparison over PER ω

	LR α over 1K to 10K Episodes				
Metric	0.00125	0.0025	0.00375	0.005	0.00625
AUC	0.797	0.895	0.973	**1.0**	0.998
Avg. SR	0.528	0.594	0.645	**0.663**	0.662
Avg. SE	±0.037	±0.030	±0.032	±**0.027**	±0.034
	PER ω over 10K to 20K Episodes				
Metric	0.1	0.3	0.5	0.7	0.9
AUC	0.945	0.932	**1.0**	0.979	0.982
Avg. SR	0.696	0.687	**0.736**	0.721	0.723
Avg. SE	±0.030	±0.029	±0.020	±0.014	±**0.013**

(c) Quantitative Metrics of Convergence Dynamics.



(d) Training Time per Episode

Fig. 6. Experimental Analysis of P-MDP: Parameter Optimization, Convergence Dynamics, and Performance Metrics

[4] AUC (Max-Norm. Area Under the Curve), Avg. SR (Average Service Composition Success Rate), and Avg. SE (Average Standard Error).

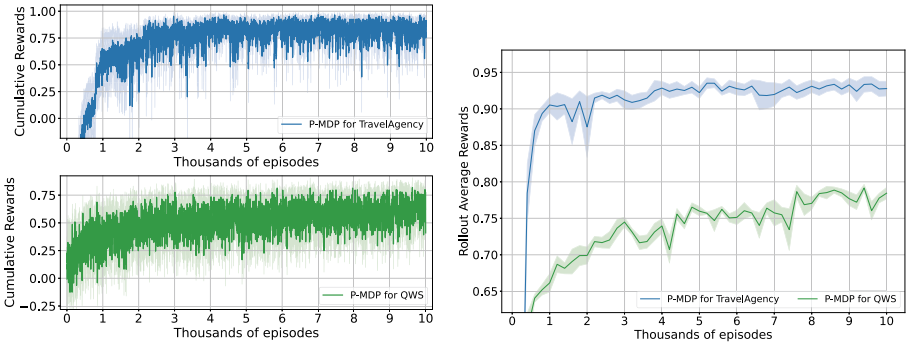
with abstract service number, matching CSSC-MDP, because more abstract services lengthen the process execution chain, thereby increasing training time.

4.2 Framework Features Evaluation

The capability support of user-defined NFPs with uncertainty and business processes in P-MDP is unavailable in prior works. We validate this via experiments on two datasets: the real-world QWS [1] and synthetic TravelAgency dataset (See Footnote 3).

With QWS. For QWS, a 2507-record dataset – normalized to 1 decimal place and spanning 8 basic QoS – was organized into 10 sequential abstract services (10 candidates each). Each candidate has 25 records per metric for uncertainty simulation. Basic metrics use standard aggregations: $\sum$ for RT, LA, CO, BP; $\prod$ for AV, SU, RE; and $\min_i(tp_i)$ for TP. In contrast to traditional MDP-based methods [2,3,8–10,12], P-MDP supports *user-defined metric with complex operators*, such as $LTS = \frac{TP}{\max(1-RT,1-LA)+\epsilon}$ and $SHI = \sqrt{Stability \times Compliance}$ ($Stability = \sqrt[10]{AV} \times \sqrt[10]{SU} \times \sqrt[10]{RE}$, $Compliance = \frac{CO+BP}{2}$), which are unattainable for prior methods, with constraints: $SHI > 0.6$ as hard constraint; $LTS > 2.0$ and maximizing SHI as soft constraints.

With Synthetic Dataset. For TravelAgency, we create a synthetic dataset covering temporal features (weekend premiums, early bird discounts), commercial features (bundled service discounts), and typical features (star-price correlation). The dataset covers a 15-day window, with each day including 20 flight, 40 train, and 60 hotel suppliers. Each supplier provides 100 records for uncertainty simulation: hotel price and star rating; flight and train prices and operational transit time. Figure 2b shows *the typical business process with gateways and some user-defined NFPs*. Additional NFPs are detailed in the supplementary Git (see Footnote 3).



(a) Cumulative Rewards of P-MDP on QWS and TravelAgency Datasets.

(b) Rollout Average Rewards of P-MDP on QWS and TravelAgency Dataset.

Fig. 7. Convergence behavior under varying datasets. Shaded regions show ±1 standard error across 5 random seeds, with curves depicting mean.

Figure 7a shows converges within roughly 2K episodes, with steady improvement afterward. Figure 7b uses the same pause strategy – freezing the policy every 200 episodes for 500 rollouts – to evaluate deployment performance in simulations. Hyperparameters mostly follow Sect. 4.1: $\gamma = 0.9$ and 200 warm-up steps for both. In TravelAgency, $n = 3$, ρ has a 30% self-transition chance (enabling flows concurrency). Soft/hard constraints are set to 0.5 with equal weights.

5 Discussion

Business processes, as a prevalent class of composite services, face significant challenges due to the interplay of diverse NFPs, complex control-flow gateways, and runtime uncertainties.The proposed P-MDP framework addresses this by providing a dynamic decision-making mechanism that enables process instances to adapt to stochastic events while optimizing for user-defined NFPs. P-MDP's principal innovation is its unified approach to managing diverse NFPs and gateway-induced control-flow complexity simultaneously, unlike prior studies addressing these aspects in isolation (Ref. to Sect. 6). Crucially, its MDP-based action–reward structure supports active optimization under uncertainty, moving beyond the analytical nature of formalisms like stochastic Petri nets [15].

Despite these advances, a primary practical challenge remains: the scalability of the action space. Concurrency, in particular, induces a combinatorial, non-decomposable action space, increasing exploration risk as control-flow complexity grows. While our decomposition of activity-related NFPs partially mitigates the action space explosion, it cannot resolve the core issue: the non-decomposable action space induced by concurrency. Emerging uncertainties, such as AI service hallucinations and Industrial IoT dynamics, compound this complexity, making compact action representations a critical future direction.

6 Related Work

To delineate the contributions of our P-MDP framework, we compare it with related approaches along two key dimensions: (1) non-functional properties (NFPs) feature support, and (2) functional properties (FPs) scope coverage. This comparison reveals critical gaps in existing solutions, illustrated in Table 2.

NFPs Dimension. Most related to P-MDP but core capabilities fragmented:

User-Defined metric: MDP-based methods [2,3,8–10,12] are confined by reward mechanisms, supporting only *built-in metrics* without customization. Temporal Network with Uncertainty (TNU)-based methods [4,16] support only simple temporal metrics. Computational Logic (Comp. Logic) methods [6,13,14] predefine complex computation logic for common metrics, coupled with specific optimizers, thus restricting user-defined extensions.

Constraints Operator: Computational Logic methods support full operator sets (Optimization (O), Relational (R), Logical (L)), but cannot cope with runtime

Table 2. Comparison with Existing Solutions. **S.P**: Simple Processes without/partial gateways; **B.P**: Business Processes with Gateways; **Semi**: Predefined NFPs using complex operators (e.g., radical, avg) without user-defined; **O, R, L**: **O**ptimization, **R**elation, and **L**ogic operators for metric and variable levels; $\mathbf{O}_v$, $\mathbf{R}_v$, $\mathbf{L}_v$: Only applicable to variable level; **Extensible**: New operator integration is parsing-layer-only

Solution	FP Scope	User-defined Metric	Constraint Operators	Runtime Uncertainty	Extensible NFP
MDP-based [8,12]	S.P	×	O	×	×
MDP-based [10]	S.P	×	O,R_v, L_v	×	×
MDP-based [2,3,9]	S.P	×	R	✓	×
TNU-based [4,16]	B.P	×	R	✓	×
Comp. Logic [6,13,14]	B.P	Semi	O,R,L	×	×
P-MDP	B.P	✓	O,R,L	✓	✓

uncertainty. MDP-based methods support only O/R operators due to its reward mechanisms, while TNU-based methods are limited to temporal R operators.

Runtime Uncertainty: Computational Logic and some MDP-based [8,10,12] methods completely ignoring runtime dynamics (e.g., cost fluctuations). In contrast, those methods based on CSSC-MDP [2,3,9] address dynamics via state-difference modeling, while TNU-based methods inherit uncertainty capability from TNU.

Extensible NFP: Given their limits on user-defined NFPs, these above-mentioned methods inherently hinder new *udm*/*udc* operator integration.

FPs Dimension. FP scope fragmentation further limits holistic optimization:

Process Structure: TNU and Comp. Logic methods directly model gateway semantics, enabling global NFP optimization over process structures. MDP-based methods, lacking such modeling, struggle to devise a globally optimal policy.

The comparison reveals a fragmentation in existing solutions across NFP expressiveness, FP coverage, and runtime uncertainty handling, motivating a unified framework that can cohesively integrate these three crucial aspects.

7 Conclusion

This paper introduces the P-MDP framework, which integrates lazy-evaluation rewards and gateway-awareness to provide a principled approach for optimizing user-defined NFPs in business processes. The primary challenge emerging from this work is the management of the combinatorial action space induced by concurrency. Developing compact action representations (e.g., hierarchical or factored actions) is therefore not merely a next step, but the critical path toward scaling this approach for complex real-world service ecosystems. The feasibility of this approach was demonstrated through a proof-of-concept prototype, and improving its usability remains a practical next step.

Acknowledgements. This work is partially supported by Projects of International Cooperation and Exchanges NSFC-DFG (Grant No.62061136006).

References

1. Al-Masri, E., et al.: Qos-based discovery and ranking of web services. In: International Conference on Computer Communications and Networks, pp. 529–534. IEEE (2007)
2. Chu, W., et al.: A constraint satisfaction service composition method supporting one to many task pattern. In: International Conference on Services Computing, pp. 382–387. IEEE (2021)
3. Chu, W., et al.: A composite service selection approach based on functionality folding and reinforcement learning. In: International Conference on Service Science, pp. 373–379. Springer, Heidelberg (2023). https://doi.org/10.1007/978-981-99-4402-6_27
4. Combi, C., et al.: Conditional simple temporal networks with uncertainty and resources. J. Artif. Intell. Res. **64**, 931–985 (2019)
5. Dumas, M., et al.: Introduction to business process management. In: Fundamentals of Business Process Management, pp. 1–33 (2018)
6. He, Q., et al.: Quality-aware service selection for service-based systems based on iterative multi-attribute combinatorial auction. IEEE Trans. Softw. Eng. **40**(2), 192–215 (2014)
7. Hessel, M., et al.: Rainbow: combining improvements in deep reinforcement learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)
8. Moustafa, A., Ito, T.: A deep reinforcement learning approach for large-scale service composition. In: Miller, T., et al. (eds.) PRIMA 2018. LNCS (LNAI), vol. 11224, pp. 296–311. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-03098-8_18
9. Ren, L., et al.: A reinforcement learning method for constraint-satisfied services composition. IEEE Trans. Serv. Comput. **13**(5), 786–800 (2020)
10. Ribino, P., et al.: Norm-based reinforcement learning for qos-driven service composition. Inf. Sci. **646**, 119377 (2023)
11. Sutton, R.S., et al.: Reinforcement Learning: An Introduction. MIT press, Cambridge (2018)
12. Wang, H., et al.: Integrating reinforcement learning with multi-agent techniques for adaptive service composition. ACM Trans. Auton. Adapt. Syst. **12**(2), 1–42 (2017)
13. Xi, M., et al.: Quantitative assessment of service pattern: framework, language, and metrics. IEEE Trans. Serv. Comput. **15**(6), 3457–3470 (2021)
14. Xi, M., et al.: Service pattern optimization: focusing on collaboration in service ecosystems. IEEE Trans. Serv. Comput. (2023)
15. Xiong, P., Fan, Y., Zhou, M.: A petri net approach to analysis and composition of web services. IEEE Trans. Syst. Man Cybern.-Part A: Syst. Hum. **40**(2), 376–387 (2010)
16. Zavatteri, M., et al.: Dynamic controllability of temporal networks with instantaneous reaction. Inf. Sci. **613**, 932–952 (2022)
17. Zheng, Z., et al.: Investigating qos of real-world web services. IEEE Trans. Serv. Comput. **7**(1), 32–39 (2012)

Business Process Discovery Through Agentic Generative AI

Pierre Lindenberg[1], Indika Kumara[1,2(✉)], Joshua Owotogbe[1,2], Willem-Jan van den Heuvel[1,2], and Damian Andrew Tamburri[1,3]

[1] Jheronimus Academy of Data Science, Sint Janssingel 92, 5211 DA 's-Hertogenbosch, Netherlands
p.o.lindenberg@student.tue.nl
[2] Tilburg University, Warandelaan 2, 5037 AB Tilburg, Netherlands
{i.p.k.weerasinghadewage,j.s.owotogbe,
w.j.a.m.vdnheuvel}@tilburguniversity.edu
[3] University of Sannio, Piazza Guerrazzi 1, 82100 Benevento, BN, Italy
datamburri@unisannio.it

Abstract. Large Language Models (LLMs) are increasingly used to support intelligent agents in knowledge-intensive tasks. Yet, their application to business process discovery remains underexplored, especially in multi-agent settings requiring coordinated, multi-turn dialogue. This paper introduces an LLM-based Multi-Agent Systems (MAS) framework that uncovers business processes through structured conversational interactions. Using the Gaia methodology, we designed four MAS types (Monolithic, Duo, Manager, and Team) with varying levels of agent role specialization and coordination complexity. A custom, model-agnostic infrastructure is used to support the choreography-based collaboration of LLM-based agents. We evaluated our LLM-based MAS across three realistic business processes and three LLMs (GeminiPro 2.5, Mistral Large 2, DeepSeek-V3). GeminiPro consistently outperformed other models, reaching 100% accuracy in some configuration-process pairs, while Mistral degraded sharply under complex setups. Although modular MAS designs enhanced prompt efficiency, they also introduced coordination overhead and failure risks. These findings demonstrate the feasibility of conversational MAS for process discovery and highlight the importance of aligning MAS system architecture with LLM model capabilities.

Keywords: Large Language Models · Multi-Agent Systems · Business Process Discovery · Conversational Agents · Gaia Methodology

1 Introduction

Business process discovery is a key activity in Business Process Management (BPM), enabling organizations to understand and improve how work is performed [5]. Traditionally, it relies on manual methods such as expert interviews and workshops. While effective at capturing context-specific insights, these methods are resource-intensive, prone to omission of tacit knowledge, and difficult to scale [7,17]. At the same time, organizations today face an increasingly dynamic

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 253–267, 2026.
https://doi.org/10.1007/978-981-95-5015-9_19

and distributed environment shaped by digital transformation and rapid market shifts [13,16]. As processes grow in complexity, manual approaches become more time-consuming and less effective [5]. Much of the relevant process knowledge remains undocumented, residing informally within employees' experience [17].

Meanwhile, advances in artificial intelligence, particularly large language models (LLMs), offer new opportunities to automate knowledge-intensive tasks [8,11]. LLMs support natural language understanding, multi-turn dialogue, and contextual reasoning, enabling systems that interact with users and simulate human-like roles [4,9]. This has led to growing interest in LLM-powered autonomous agents, also known as agentic GenAI [12].

This paper investigates how LLM-based Multi-Agent Systems (MAS) can support interview-driven business process discovery. Prior work has focused on using LLMs to interpret process models or generate them from textual input [1,10,14]. In contrast, we investigate whether distributed, role-specialized agents can autonomously simulate domain experts, extract both explicit and tacit knowledge, and collaboratively construct formal models (BPMN) of business processes from these interactions.

This work makes several research contributions. First, we present the design of an LLM-based MAS where virtual agents role-play human stakeholders in the interview-based process discovery. We apply the Gaia agent-oriented methodology [20] for building our MAS, demonstrating how classical MAS design methods can guide agentic GenAI system development. We also introduced an agent collaboration framework based on the choreography model. Finally, we evaluated four different MAS configurations across three different LLMs and use cases. Results show that LLM-based MAS can effectively support interview-based process discovery; however, the complexity in MAS design can increase failure rates, which in turn reduces the overall performance.

Paper Organization. Sect. 2 presents our MAS design approach. Section 3 describes the implementation of the agent framework. Section 4 presents the empirical study. Section 5 reviews existing studies. Section 6 concludes the paper.

2 Methodology

This work adopts the Gaia methodology [20,22] to guide the structured development of a Multi-Agent System (MAS) for business process discovery. We used Gaia because it is widely used in the classical MAS literature and can support a wide range of MAS architectures. Gaia organizes MAS development into three sequential phases: requirement specification, analysis, and design, each producing formal models that define the system's structure and behavior (see Fig. 1).

2.1 Requirement Specification Phase

The development process begins with thoroughly specifying the system's objectives, boundaries, and context. Our primary goal is to enable efficient and robust

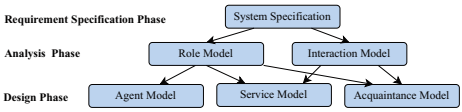


Fig. 1. Overview of the Gaia methodology models and their dependencies [20].

business process discovery by leveraging LLM-powered conversational agents. Hence, the MAS is scoped to encompass all aspects of information gathering, process modeling, and management of agent-to-human interactions. In contrast, the actual execution of business processes remains outside the system's scope. The system operates in a typical organizational environment, engaging with stakeholders such as process managers, process analysts, and process workers. Stakeholder requirements direct all design choices, including reliable knowledge extraction, process transparency, and support for iterative process improvement.

2.2 Analysis Phase

In the Gaia methodology, the analysis phase bridges the gap between high-level requirements and the concrete design of the MAS. It formalizes the organizational structure of the MAS through two core abstractions: the *Role Model* and the *Interaction Model*. They jointly define the system's behavioral logic, specifying how agents coordinate to fulfill the overall objective of process discovery.

Role Model. This defines the internal organization of the MAS by specifying distinct roles, each characterized by explicit behavioral constraints and functional responsibilities. In Gaia, every role is described along four key dimensions. First, *responsibilities* are formalized through *liveness* properties, which describe what the role must eventually achieve, and *safety* properties, which specify conditions that must never be violated. Second, *activities* refer to autonomous operations that the role performs independently, such as asking questions, summarizing findings, or creating process models. Third, *permissions* determine the types of data or resources the role can access, generate, or modify. Finally, *protocols* capture the structured communication interactions that the role engages in with other roles. Using this four-dimensional schema (responsibilities, activities, permissions, and protocols), five core roles are defined in our system, namely *Consulting Manager*, *Process Modeler*, *Interviewer*, *Knowledge Gatherer*, and *Interview Partner*. We identified these roles based on the relevant literature [5] and input from a process mining consultant company in the Netherlands.

The *Consulting Manager* supervises the entire process. It defines the objective of the process discovery effort and validates the final outputs. The responsibilities and constraints of this role are shown in its formal schema (see Fig. 2a). The *Process Modeler* is responsible for generating the BPMN model from synthesized knowledge. It ensures the completeness and validity of the model and reports assumptions or modeling issues (see Fig. 2b). The *Interviewer* executes interviews with *Process Workers* by asking guided questions, collecting narrative

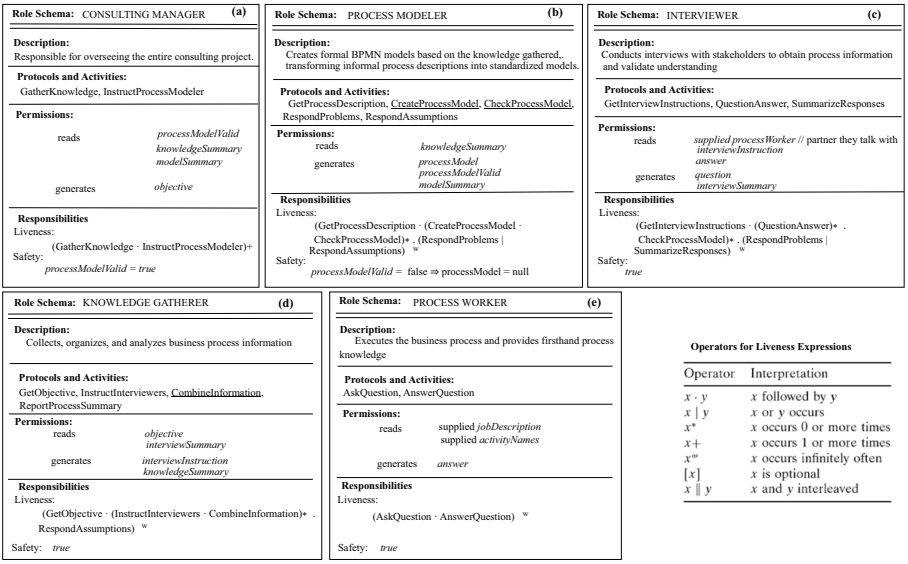


Fig. 2. Role schemas for agents. (a) Consulting Manager, (b) Process Modeler, (c) Interviewer, (d) Knowledge Gatherer, (e) Process Worker (Interview Partner).

responses, and producing summarized reports for the *Knowledge Gatherer* (see Fig. 2c). The *Knowledge Gatherer* coordinates the interview phase by instructing *Interviewers*, combining their summaries, and generating consolidated knowledge. This agent plays a central role in transforming raw narratives into structured process information (see Fig. 2d). The *Interview Partner* represents the external human stakeholder (i.e., *Process Worker*). Although not part of the MAS itself, this role is modeled to reflect its participation in interactions that supply critical domain knowledge (see Fig. 2e).

Interaction Model. This model builds on the communication requirements identified in the role model by defining structured protocols that govern how agents collaborate to achieve shared goals. These protocols specify message sequences, data exchanged, and the participating roles. In our system, four key protocols orchestrate coordination among agents: *QuestionAnswer*, *InstructInterviewers*, *GatherKnowledge*, *InstructProcessModeler*.

The *QuestionAnswer* protocol captures the exchange between an *Interviewer* and a *Process Worker* (an external human). The *Interviewer* poses a domain-specific question based on interview instructions, and the *Process Worker* responds with a first-person, experience-based narrative. This direct interaction forms the core mechanism for extracting accurate, practical knowledge about real-world business processes (Fig. 3a).

The *InstructInterviewers* protocol frames the collaboration between the *Knowledge Gatherer* and *Interviewers*. It begins when the *Knowledge Gatherer* sends interview objectives and instructions to the *Interviewer*. The *Interviewer*

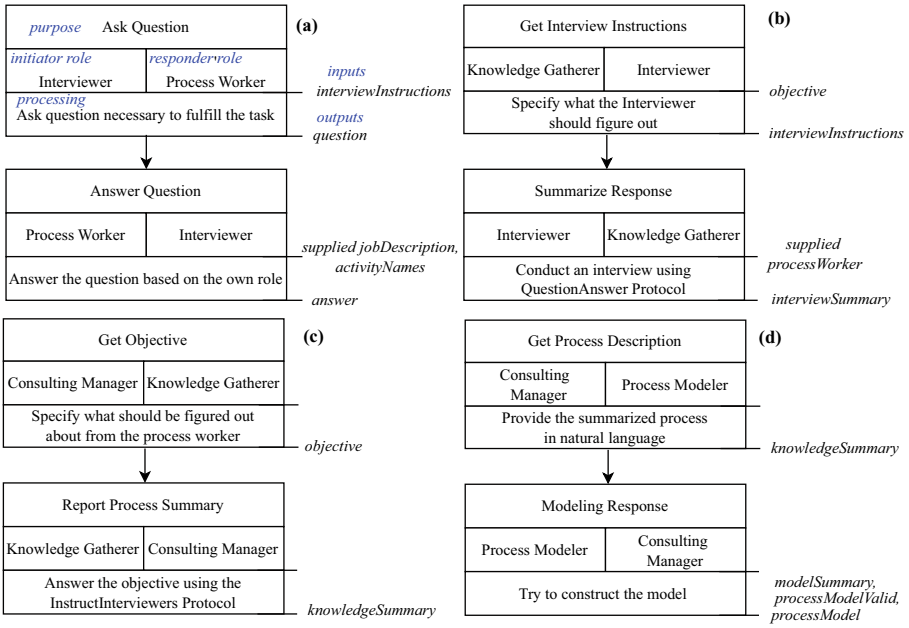


Fig. 3. Protocols for agent-to-agent interactions. (a) QuestionAnswer, (b) InstructInterviewers, (c) GatherKnowledge, (d) InstructProcessModeler.

then conducts one or more *QuestionAnswer* interactions, and, upon completion, summarizes the collected insights and returns them to the *Knowledge Gatherer*. This protocol ensures that interviews are guided, consistent, and yield structured outputs for synthesis and analysis (see Fig. 3b).

The *GatherKnowledge* protocol connects the *Consulting Manager* with the *Knowledge Gatherer*. The *Consulting Manager* initiates it by defining the overall objective of the discovery effort. The *Knowledge Gatherer* then coordinates interviews via the *InstructInterviewers* protocol, aggregates responses, and reports a consolidated process summary to the *Manager*. This interaction supports system-level coordination and completeness (see Fig. 3c).

In the *InstructProcessModeler* protocol, either the *Consulting Manager* or the *Knowledge Gatherer* sends the synthesized process knowledge to the *Process Modeler*. The *Process Modeler* constructs a formal BPMN diagram and returns modeling feedback, which may include assumptions, ambiguities, or problems encountered during the modeling process. This protocol ensures traceability from raw insights to formal representations (see Fig. 3d).

2.3 Design Phase

This phase operationalizes the conceptual structures defined during analysis, laying out the concrete architecture of the MAS. This includes how roles are

realized as agent types, what services agents provide, and how they interact. Gaia structures this phase using three core models: the *Agent Model*, the *Service Model*, and the *Acquaintance Model*. Together, these models transition the system from an abstract specification to an implementable agent society.

The *agent model* defines how the previously specified roles are mapped to concrete agent types. Depending on the system configuration, each agent type corresponds to one or more roles. In simple setups (e.g., Monolithic), a single agent may embody all roles, handling the entire process from interviewing to modeling. In more complex systems (e.g., a Team), each role is instantiated as a distinct agent, allowing specialization and parallelism. The agent model also specifies the number of agents per type and how agent instances relate to one another during execution.

The *service model* defines the services associated with each role. Each agent offers services that encapsulate its operations, derived directly from the protocols and activities specified in the role model. For instance, the *Interviewer* provides services such as *AskQuestion* and *SummarizeResponses*, while the *Process Modeler* exposes services such as *CreateProcessModel* and *CheckProcessModel*. These services define how an agent processes inputs, produces outputs, and interacts with other agents or external actors.

The *acquaintance model* captures the communication topology among agents. It specifies which agents are aware of each other and can exchange messages, without prescribing the low-level mechanisms. For example, in the Team setup, multiple *Interviewers* send their summaries to a single *Knowledge Gatherer*, who then passes the consolidated knowledge to the *Process Modeler*. This model is key for identifying communication bottlenecks, ensuring fault tolerance, and planning for scalability.

2.4 Multi-agent System Configurations

As part of the design phase in Gaia, a key step is mapping roles to agents [20]. This mapping can range from a single agent handling all roles to fully distributed architectures, where each agent specializes in a single role. While recent work has shown that LLM-based agents can perform complex tasks individually [21], coordinated systems of specialized agents can yield better scalability and resilience [19]. We implement four MAS configurations to evaluate these trade-offs: Monolithic, Duo, Manager, and Team, each with increasing modularity and interaction complexity. The *Process Worker* remains an external human actor and is included only in acquaintance models to represent communication pathways.

In the *Monolithic* MAS configuration, all responsibilities, from interviewing process workers to modeling the discovered processes, are handled by a single agent. This design is ideal for simple, linear tasks that require minimal coordination or concurrent operations. Since there is only one agent, no inter-agent communication occurs. Figure 4a shows the single agent encompassing all roles, while Fig. 4b confirms the absence of acquaintance links.

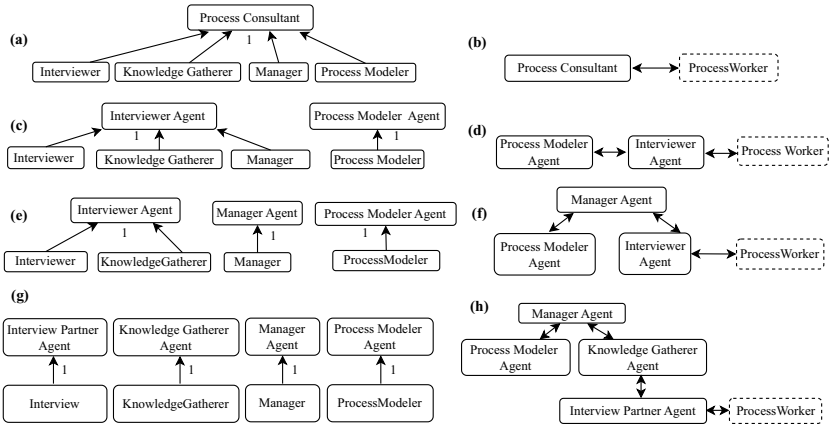


Fig. 4. Agent and acquaintance models across MAS configurations. (a) Monolithic agent model, (b) Monolithic acquaintance model, (c) Duo agent model, (d) Duo acquaintance model, (e) Manager agent model, (f) Manager acquaintance model, (g) Team agent model, and (h) Team acquaintance model.

The *Duo* setup introduces a separation of concerns: one agent (the *Interviewer*) handles knowledge acquisition, and another (the *Process Modeler*) focuses on constructing the BPMN model. These two agents interact via the *InstructProcessModeler* protocol. This division enables a basic form of task delegation and improves maintainability without significant communication overhead (see Figs. 4c and 4d).

The *Manager* setup introduces a third agent, the *Manager*, to coordinate and validate intermediate results. The *Interviewer* focuses on information collection, the *Knowledge Gatherer* compiles findings, and the *Manager* supervises both before delegating to the *Process Modeler*. This configuration supports quality control, protocol compliance, and error resolution. It is particularly beneficial in real-world consulting environments where oversight and validation are crucial. The agent and acquaintance models are shown in Figs. 4e and 4f.

The *Team* setup maximizes modularity and parallelism. Each *Interview Partner* agent is assigned to a specific *Process Worker*, allowing for concurrent interviews. The *Knowledge Gatherer* coordinates the group, consolidates results, and passes them on to the *Process Modeler* via the *Manager*. This architecture enables scalability and fine-grained specialization, particularly in domains with complex or distributed processes. Figures 4g and 4h show the multi-agent structure and communication paths of this most sophisticated setup.

3 Multi-agent Framework Implementation

We developed a lightweight multi-agent framework for conversational LLMs to support autonomous process discovery across multiple stakeholders. While existing orchestration platforms, such as CrewAI and LangGraph, provide useful

abstractions, they lack features essential to our use case, namely decentralized agent autonomy, concurrent multi-user interaction (as in Gaia's choreography model), and fine-grained observability. Our framework addresses these gaps by enabling agents to make independent decisions, select communication partners, and adapt based on their interaction history, without requiring a central controller. This design follows the Gaia methodology [20], which explicitly maps organizational roles to agent behaviors.

Implementing Agents. Each agent consists of a system prompt, a set of tools, and an underlying LLM. These define the agent's behavior, constraints, and tool orchestration. The system prompt specifies goals, context, and limitations. We used zero-shot and few-shot prompting, based on prior work in BPMN model generation [10,23] and LLM role-play [18]. Due to limited space, complete prompt examples are included in the online appendix (see Sect. 4). Tools are callable functions that enable agents to perform tasks beyond those that can be accomplished through language-only operations. The LLM interprets prompts and generates output, with specific LLM models bound via dependency injection. This structure allows flexible model substitution without modifying core logic.

The LLM receives three inputs: the system prompt, the message history (comprising user/model exchanges, tool calls, and results), and a list of tools. Using recent function-calling capabilities of LLMs, tools support structured operations such as API calls, computations, or inter-agent communication. Each tool is defined by a function name, function implementation, description, and parameter specification to guide its selection and use by LLMs.

Collaboration between Agents. Each agent functions as a role-driven module capable of both proactive and reactive messaging, guided by its internal role model. Agent coordination emerges through interaction protocols rather than centralized orchestration, enabling dynamic task routing and asynchronous collaboration. Following the choreography model, agents reason independently, identify eligible peers, and initiate communication based on context. To structure these interactions, we utilize the acquaintance model to define agent connectivity and interaction models to specify the expected message content and structure. These models, along with role models, originate from the MAS design phase (see Sect. 2.3). All four MAS configurations (Monolithic, Duo, Manager, and Team) were implemented using this architecture.

Observability. Robust observability is essential for the framework's usefulness in research and development. To support this, the framework integrates MLflow, an open-source platform for experiment tracking and artifact management. It logs every interaction with the LLM, including token usage, tool invocations, input parameters, and execution errors, whether from tool logic or the LLM API. Using MLflow, agent messages and session data are fully traceable, while generated outputs (e.g., process models) are logged as artifacts linked to each session trace. This tight integration enhances transparency, allowing outcomes to be directly linked to the agent's actions and context, thereby enabling deeper analysis and rigorous evaluation.

4 Evaluation and Results

This section presents the experimental evaluation of the proposed multi-agent systems (MAS) for business process discovery using LLMs. The goal is to assess their effectiveness, efficiency, and robustness under different agent configurations and LLM backends. Specifically, we consider the following research questions:

- **RQ1:** *How does role specialization within an LLM-based MAS influence its ability to discover business processes through conversations with process participants?*
- **RQ2:** *How does the number of agents in an LLM-based MAS affect the efficiency of knowledge gathering and process model construction?*
- **RQ3:** *How robust are different LLM-based MAS designs in handling internal coordination failures?*

Experimental Setup and Design. All experiments were conducted using our agent framework presented in the previous section. This setup allowed for consistent agent interaction patterns across 36 experimental runs, spanning three representative business processes: (1) an online shop process [10], where a customer browses, places an order, and interacts with multiple departments such as payment and shipping; (2) an employee reimbursement process [5], in which staff submit expense claims that are reviewed and approved by finance personnel; and (3) a hotel room service process [10], where a guest places food and drink orders that are fulfilled by various hotel staff operating in parallel. Each scenario was tested across four MAS configurations (Monolithic, Duo, Manager, Team) and three LLMs: GeminiPro 2.5, Mistral Large 2, and DeepSeek-V3. We selected these LLMs because they are from three different continents and include both open-source and proprietary models.

All experiments were recorded using MLflow. We report metrics aligned with our three research questions: BPMN feature identification accuracy for assessing role specialization and process discovery (RQ1), task completion and token efficiency for evaluating performance (RQ2), and failure analysis for measuring robustness under coordination challenges (RQ3). These metrics enable consistent comparison across models and MAS configurations.

Online Appendix. We made the source code of our MAS framework, as well as the experimental data and results, available online[1].

4.1 BPMN Feature Identification Accuracy (RQ1)

To evaluate the MAS's process discovery capability, we measured the accuracy with which each MAS configuration reconstructed the key features of the ground-truth BPMN models. We selected this metric as we aim to assess the degree to which a particular MAS design can achieve its objectives/requirements (compared with evaluating each agent separately). We compared the key BPMN

[1] https://doi.org/10.6084/m9.figshare.29610527.v1.

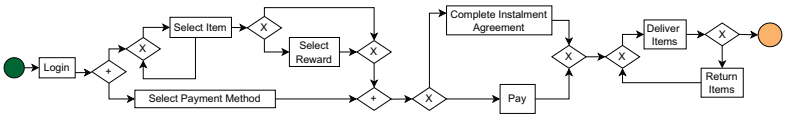


Fig. 5. Reference BPMN model of the Shop process.

Table 1. Detected features in the BPMN model for the Shop process. Monolithic = 1, Duo = 2, Manager = 3, Team = N

	GeminiPro				Mistral				Deepseek			
	1	2	3	N	1	2	3	N	1	2	3	N
Login at first	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓
Select Item exists	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓
Select Item is repeated activity	✓	✓										✓
Paying possible	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓
Two Payment Options exist	✓					✓						✓
Select Payment Method Parallel to Item Selection						✓						✓
Select Reward is optional	✓	✓	✓	✓					✓		✓	✓
Delivery exists	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓
Return exists	✓				✓	✓	✓		✓	✓	✓	✓
Return is optional	✓								✓			
Sum	9	6	5	5	5	7	5	0	7	5	6	9

Table 2. BPMN feature identification accuracy (%) across scenarios: Shop (S), Reimbursement (R), and Hotel (H)

Scenario	GeminiPro			Mistral			Deepseek		
	S	R	H	S	R	H	S	R	H
Monolithic	90%	100%	92%	50%	18%	58%	70%	64%	58%
Duo	60%	91%	75%	70%	82%	58%	50%	55%	67%
Manager	50%	100%	92%	50%	27%	67%	60%	82%	58%
Team	50%	100%	83%	0%	73%	0%	90%	64%	58%

features, such as compulsory activities, decision nodes, and optional activities, to ensure they are identified correctly. For example, the Shop process, shown in Fig. 5, has 10 distinct features. As shown in Table 1, different LLMs and MAS configurations were able to detect some features accurately while missing others. Across the three use cases, we measured the percentages of BPMN features that a given MAS could identify accurately. Table 2 summarizes the results.

GeminiPro consistently achieved the highest accuracy, particularly in the Monolithic and Team setups, exceeding 90% in most cases. DeepSeek was stable with moderate performance, peaking at 90% in the Shop/Team setup. While Mistral exhibited the most significant variability, dropping to 0% in both the Shop and Hotel scenarios under the Team configuration, and as low as 18% in

the Monolithic Reimbursement case, likely due to context fragmentation in more complex agent interactions.

While modular MAS configurations enable better role separation, they do not always lead to improved modeling fidelity; instead, they may introduce coordination overhead, particularly for LLMs with limited context capacity. These findings suggest that role specialization can improve output structure, but its benefits depend heavily on the LLM's ability to handle coordination. While GeminiPro achieved high accuracy in simpler setups, such as Monolithic, and performed well in some use cases (e.g., Reimbursement and Hotel), its performance dropped in more complex configurations, such as Team, during the Shop scenario. This suggests that increasing specialization adds coordination overhead, which only models with strong context retention and reasoning (e.g., GeminiPro) can partially manage. On the other hand, Mistral showed more dramatic drops in Team setups, highlighting its limitations in distributed configurations.

4.2 Task Performance and Resource Efficiency (RQ2)

We analyzed token usage and communication load across MAS types to evaluate system efficiency and scalability.

As shown in Table 3a, GeminiPro consistently delivered strong performance (over 75%) even in complex setups. Here, the performance refers to the percentage of BPMN features that a given MAS can identify accurately or inaccurately (i.e., achieving the main objective of the MAS). DeepSeek was stable and peaked at 70.7% in Team. Mistral performed better in Duo but declined sharply in Team, indicating sensitivity to inter-agent coordination. Task completion rates (Table 3b) reinforce these trends: GeminiPro and DeepSeek consistently completed all tasks (agent tasks such as answering an interview question or generating a process model), whereas Mistral dropped to 33.3% in Duo and Manager and failed in Team, reflecting difficulties handling complex agent workflows. Prompt

Table 3. Summary of MAS Type performance across GeminiPro (GPro), Mistral, and Deepseek (DSeek). Values in "K" denote thousands of tokens.

(a) Average Performance Rate

MAS Type	GPro	Mistral	DSeek
Monolithic	93.9%	42.2%	64.0%
Duo	75.3%	70.1%	57.1%
Manager	80.6%	48.0%	66.7%
Team	77.8%	24.2%	70.7%

(b) Task Completion Rate

MAS Type	GPro	Mistral	DSeek
Monolithic	100%	100%	100%
Duo	100%	33.3%	100%
Manager	100%	33.3%	100%
Team	100%	0%	100%

(c) Average Prompt Tokens

MAS Type	GPro	Mistral	DSeek
Monolithic	89K	163K	40K
Duo	120K	68K	67K
Manager	80K	121K	133K
Team	286K	155K	88K

(d) Average Amount Invocations

MAS Type	GPro	Mistral	DSeek
Monolithic	10.3	20.7	6.3
Duo	27.3	16.3	17.0
Manager	22.0	30.0	30.3
Team	112.3	97.3	57.0

(e) Prompt Tokens per Invocation

MAS Type	GPro	Mistral	DSeek
Monolithic	8.6K	7.9K	6.3K
Duo	4.4K	4.2K	4.0K
Manager	3.6K	4.0K	4.4K
Team	2.5K	1.6K	1.6K

(f) Completion Tokens

MAS Type	GPro	Mistral	DSeek
Monolithic	6.7K	2.3K	0.9K
Duo	8.4K	3.3K	3.4K
Manager	16.1K	6.3K	6.9K
Team	15.0K	6.4K	5.5K

token usage (Table 3c) increased with system complexity, especially for GeminiPro, which scaled from 89K in Monolithic to 286K in Team. Table 3d shows that LLM invocation counts also rose with MAS complexity, peaking at 112 (GeminiPro), 97 (Mistral), and 57 (DeepSeek) in Team setups.

However, Table 3e reveals a key efficiency trend: prompt tokens (the number of tokens in a prompt) per agent/LLM invocation decreased as modularity increased. Specialized agents communicated more concisely, and GeminiPro dropped from 8.6K to 2.5K tokens per call. Completion tokens—the tokens that the LLM generates in response to an input—increased with system complexity, especially for GeminiPro (up to 16.1K in Manager), suggesting more detailed and informative outputs (Table 3f).

Together, these results show that while modular MAS designs incur coordination overhead, capable LLMs can offset this through more focused and richer interactions. This highlights a trade-off: system complexity raises costs, but communicative efficiency and output quality may justify them in scalable, real-world settings. Our results are also partially aligned with Xia et al. [21], who found that single agents can outperform MAS in some software development tasks.

4.3 Failure Analysis and Robustness (RQ3)

To assess the robustness, we employed Cemri et al.'s taxonomy [15] to analyze the frequency and severity of failures under various MAS configurations. It comprises 14 distinct failure modes. The failures were assessed using their proposed LLM-as-a-Judge solution, which takes every individual agent's trace as input.

Table 4a shows that the average failure counts of all models (across three use cases) increased with MAS complexity. It was measured by dividing the total number of failures with a given LLM by 3. GeminiPro saw a moderate rise, peaking at 14.0 failures in the Team setup. DeepSeek, while error-free in Monolithic, showed a sharp jump to 6.33 in Manager and 9.66 in Team. Mistral exhibited more consistent but elevated failures across setups, particularly in Team (12.66). When normalized by agent invocation count (Table 4b), DeepSeek and Mistral showed higher per-interaction failure rates in modular configurations, especially in Duo and Manager. In contrast, GeminiPro maintained relatively stable failure rates across setups, suggesting better robustness under distributed coordination.

Table 5 shows failure types across MAS categories. Monolithic setups were dominated by "Step Repetition" (62.5%), reflecting redundancy from a lack of

Table 4. Summary of MAS type robustness across LLMs and use cases.

(a) Average Failures

MAS Type	GPro	Mistral	DSeek
Monolithic	0.66	1.66	0.0
Duo	0.33	1.0	4.0
Manager	4.0	4.66	6.33
Team	14.0	12.66	9.66

(b) Failures per Agent Invocation

MAS Type	GPro	Mistral	DSeek
Monolithic	5.6	3.2	0.0
Duo	2.4	6.3	8.3
Manager	5.2	7.4	8.4
Team	5.6	8.1	7.7

Table 5. Failure distribution (across three use cases) by MAS type (%)

Failure Type (14 Failure Modes [15])	Duo	Manager	Monolithic	Team
Disobey Task Specification	2.04	8.2	0.0	14.8
Disobey Role Specification	2.04	1.9	0.0	7.4
Step Repetition	13.94	20.4	62.5	20.7
Loss of Conversation History	2.04	0.0	0.0	0.7
Unaware of Termination Conditions	2.04	8.7	12.5	8.0
Conversation Reset	0.0	0.0	0.0	0.2
Fail to Ask for Clarification	14.3	5.4	0.0	2.9
Task Derailment	0.0	1.9	0.0	0.9
Information Withholding	0.0	1.0	0.0	3.5
Ignored Other Agents Input	9.2	11.3	8.33	8.1
Action Reasoning Mismatch	6.8	6.9	8.33	12.4
Premature Termination	0.0	0.0	0.0	3.2
No or Incorrect Verification	0.0	8.9	0.0	4.3
Weak Verification	47.6	25.4	8.33	12.8

role separation. In contrast, Team setups showed higher proportions of failures related to coordination, such as "Disobey Task Specification" (14.8%) and "Action Reasoning Mismatch" (12.4%), indicating challenges in maintaining alignment across specialized roles. Interestingly, "Weak Verification" was most severe in Duo (47.6%) and Manager (25.4%), suggesting that partial modularity may introduce gaps in inter-agent checks without sufficient validation mechanisms.

These results highlight the importance of strong LLM capabilities and structured agent coordination. While modularity improves clarity and parallelism, it demands greater inter-agent safeguards. Robust MAS designs may benefit from verification agents, redundancy checks, or meta-reasoning routines to prevent cascading errors in decentralized agentic workflows [15].

5 Related Work

Recent research has explored the use of LLMs in business process management (BPM). For example, Franzoi et al. [6] demonstrated how LLM-based chat interfaces can retrieve both textual and multimodal business knowledge, improving access to organizational information. Similarly, Berti et al. [2] showed that autonomous LLM agents can analyze process mining artifacts to support process analysis. Closer to our work, Kourani et al. [10] utilized LLMs to generate formal process models from textual descriptions, enabling conversion to BPMN or Petri Nets using tools such as PM4Py [3]. Similarly, Nivon et al. [14] and Ziche and Apruzzese [23] also applied LLMs to translate user-provided requirements into process models, with the latter introducing a chatbot assistant for modelers. However, these approaches rely on existing documentation and do not address the extraction of tacit process knowledge directly from stakeholders. Capturing such knowledge, typically through interviews or workshops, is often labor-intensive and costly.

Our work addresses this gap by developing LLM-based multi-agent systems (MAS) that role-play organizational stakeholders in process discovery scenarios. We further applied the Gaia methodology to develop LLM-based MAS and evaluated four MAS designs across three LLMs in simulated interview settings.

6 Conclusion and Future Work

This paper presented a multi-agent system (MAS) framework, powered by LLMs, for discovering business processes by simulating the process of interviewing human stakeholders to gather business process knowledge to construct process models. Guided by the Gaia MAS design methodology, the LLM-based MAS was designed with explicit roles, responsibilities, and communication protocols to support structured agent interactions. Empirical evaluation across four MAS configurations (Monolithic, Duo, Manager, and Team) and three LLMs (GeminiPro, Mistral, and DeepSeek) revealed that increasing agent specialization and modularity does not continually improve system performance. While modularity reduced prompt length per invocation, it also introduced higher communication overhead and failure rates, especially in distributed setups.

In future research, we aim to extend our framework to support the simultaneous discovery of multiple business processes and introduce agents with domain-specific competencies, such as compliance checking or document parsing. We also plan to standardize our framework by adopting emerging standards, such as the Model Context Protocol and the Agent2Agent Protocol. Finally, we aim to study the impact of advanced prompt strategies and agent coordination safeguards such as inter-agent verification and fault-tolerant messaging.

Acknowledgments. This research has been partially funded by the RAPTURE TKI research project (RT104659-11).

References

1. Berti, A., Kourani, H., van der Aalst, W.M.: Pm-llm-benchmark: evaluating large language models on process mining tasks. In: International Conference on Process Mining, pp. 610–623. Springer, Heidelberg (2024)
2. Berti, A., Maatallah, M., Jessen, U., Sroka, M., Ghannouchi, S.A.: Re-Thinking Process Mining in the AI-Based Agents Era (2024). arXiv:2408.07720
3. Berti, A., van Zelst, S., Schuster, D.: Pm4py: a process mining library for python. Softw. Impacts **17**, 100556 (2023)
4. Deng, Y., Lei, W., Huang, M., Chua, T.S.: Rethinking conversational agents in the era of llms: proactivity, non-collaborativity, and beyond. In: Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, pp. 298–301 (2023)
5. Dumas, M., Rosa, L.M., Mendling, J., Reijers, A.H.: Fundamentals of Business Process Management. Springer, Heidelberg (2018)

6. Franzoi, S., Delwaulle, M., Dyong, J., Schaffner, J., Burger, M., Brocke, J.V.: Using Large Language Models to Generate Process Knowledge from Enterprise Content (2024)
7. Grohs, M., Abb, L., Elsayed, N., Rehse, J.R.: Large language models can accomplish business process management tasks. In: De Weerdt, J., Pufahl, L. (eds.) Business Process Management Workshops, pp. 453–465. Springer, Cham (2024)
8. Guo, T., et al.: Large language model based multi-agents: a survey of progress and challenges. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, pp. 8048–8057. International Joint Conferences on Artificial Intelligence Organization, Jeju (2024)
9. Huang, J., Chang, K.C.C.: Towards Reasoning in Large Language Models: A Survey (2023). arXiv:2212.10403
10. Kourani, H., Berti, A., Schuster, D., van der Aalst, W.M.P.: Process modeling with large language models. In: van der Aa, H., Bork, D., Schmidt, R., Sturm, A. (eds.) Enterprise, Business-Process and Information Systems Modeling, pp. 229–244. Springer, Cham (2024)
11. Liu, Y., et al.: Agent design pattern catalogue: a collection of architectural patterns for foundation model based agents. J. Syst. Softw. **220**, 112278 (2025)
12. Murugesan, S.: The rise of agentic AI: implications, concerns, and the path forward. IEEE Intell. Syst. **40**(2), 8–14 (2025)
13. Muthusamy, V., Isahagian, V., Rizk, Y.: Bringing bpm to the workers: towards worker-centric management of business processes. In: CEUR-WS (2021). iSSN: 1613–0073
14. Nivon, Q., Salaün, G.: Automated generation of bpmn processes from textual requirements. In: Gaaloul, W., Sheng, M., Yu, Q., Yangui, S. (eds.) Service-Oriented Computing, pp. 185–201. Springer, Singapore (2025)
15. Pan, M.Z., et al.: Why do multiagent systems fail? In: ICLR 2025 Workshop on Building Trust in Language Models and Applications (2025)
16. Rosemann, M., Brocke, J.V., Van Looy, A., Santoro, F.: Business process management in the age of AI three essential drifts. Inf. Syst. e-Bus. Manag. (2024)
17. Seghroucheni, O.Z., Achhab, M.A., Lazaar, M.: Systematic review on the conversion of tacit knowledge. In: 2023 7th IEEE Congress on Information Science and Technology (CiSt), pp. 123–128 (2023). iSSN: 2327-1884
18. Shanahan, M., McDonell, K., Reynolds, L.: Role play with large language models. Nature **623**(7987), 493–498 (2023)
19. Tran, K.T., Dao, D., Nguyen, M.D., Pham, Q.V., O'Sullivan, B., Nguyen, H.D.: Multi-agent collaboration mechanisms: a survey of LLMs (2025). arXiv:2501.06322
20. Wooldridge, M., Jennings, N.R., Kinny, D.: The gaia methodology for agent-oriented analysis and design. Auton. Agent. Multi-Agent Syst. **3**(3), 285–312 (2000)
21. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Agentless: demystifying LLM-based Software Engineering Agents (2024). arXiv:2407.01489
22. Zambonelli, F., Jennings, N.R., Wooldridge, M.: Developing multiagent systems: the gaia methodology. ACM Trans. Softw. Eng. Methodol. (TOSEM) **12**(3), 317–370 (2003)
23. Ziche, C., Apruzzese, G.: Llm4pm: a case study on using large language models for process modeling in enterprise organizations. In: International Conference on Business Process Management, pp. 472–483. Springer, Heidelberg (2024)

Usage Control for Process Discovery Through a Trusted Execution Environment

Valerio Goretti[1(✉)], Sabrina Kirrane[2], and Claudio Di Ciccio[3]

[1] Sapienza University of Rome, Rome, Italy
valerio.goretti@uniroma1.it
[2] Vienna University of Economics and Business, Vienna, Austria
sabrina.kirrane@wu.ac.at
[3] Utrecht University, Utrecht, The Netherlands
c.diciccio@uu.nl

Abstract. Process mining provides valuable insights into workflows based on recorded execution data. The registered event logs often contain sensitive information as they may bear personal details about working individuals or reveal organizations' know-how and their operations' history. Therefore, confidentiality and privacy requirements are high priority for process mining. In this paper, we present ProMiSe, a software framework and service that allows users to control information usage for automated process discovery, from input provision, through log processing, to output consultation. Usage control rules are expressed by means of policies. ProMiSe turns them into dedicated applications running within a Trusted Execution Environment (TEE) to enforce those rules. We put our solution to the test with real-world event logs to assess its computational resource consumption and service time.

Keywords: Confidential computing · Confidentiality · Privacy · TEE

1 Introduction

Process mining offers a powerful approach to analyzing workflows by extracting useful information from event data recorded in information systems [2]. It enables organizations to visualize real-world process executions, identify inefficiencies, and discover opportunities for improvement. Process mining is based on data-driven techniques applied to records that potentially exhibit sensitive information, against secrecy and privacy concerns. Thus, organizations may be reluctant to submit their event logs to process mining services.

In order to protect data from unwanted information leakage, pseudonymization and encryption techniques that protect sensitive information are often used [8,9]. Nevertheless, studies have shown that even when data is obfuscated, it is still possible to single out individuals or infer information about the original input [6,19]. Other solutions apply perturbation techniques that alter the original

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 268–286, 2026.
https://doi.org/10.1007/978-981-95-5015-9_20

event log to protect sensitive information [15,17]. These techniques ensure that knowledge about the input can be reconstructed from the output, but the resulting loss of data veracity can compromise the effectiveness of subsequent analyses. More recently, approaches have emerged that leverage hardware-backed solutions to run process mining in computing enclaves [18,21]. Although mitigating the data distortion issue, they focus on the sole data processing phase, disregarding the treatment of input and output.

To overcome the above limitations, we present *P*olicy-aware p*ro*cess *Mi*ning *Se*rvice (*ProMiSe*), a framework that enables organizations to leverage process mining services under data governance. In particular, ProMiSe focuses on supporting process discovery as a subset of the broader process mining domain. ProMiSe is designed to be integrated into the infrastructure of process discovery service providers and serves as a trusted platform for client organizations. ProMiSe enables data owners to define usage policies that control who can access the data, how the data is processed, and what outcome can be inspected, ensuring that process discovery analyses comply with regulatory requirements and organizational constraints. Summarizing our contributions, we: *(i)* introduce a lightweight policy language designed to express constraints to preserve secrecy and confidentiality during the input, processing, and output phases of process discovery; *(ii)* present a framework that interprets and enforces these policies, enabling the secure and compliant execution of process discovery operations on sensitive data; and *(iii)* provide an open-source prototypical implementation of our solution, which we use for an empirical evaluation with data stemming from public benchmarks.

The remainder of the paper is structured as follows. Section 2 provides an overview of related approaches and techniques. Section 3 presents a motivating scenario in the healthcare domain and illustrates the system requirements our solution aims to meet. Section 4 presents the architecture of ProMiSe. An evaluation of our approach is shown in Sect. 5. Section 6 concludes the paper with an outlook for future research endeavors.

2 Background and State of the Art

In this section, we first introduce the background concepts that our solution is built upon, and outline work related to our endeavor.

2.1 Background

Usage control extends traditional access control by enforcing policies not only before but also during and after resource access [3]. Unlike simple authorization models, usage control introduces obligations (actions users must perform) and conditions (environmental constraints), as formalized in the UCON_{ABC} model [22]. UCON_{ABC} supports decision continuity and attribute mutability, enabling dynamic policy enforcement. A policy specifies the rules governing how data can be accessed and used. To enable automated and flexible enforcement,

policies are often represented in RDF, a graph-based format that encodes rules as triples. Usage control frameworks govern the conditions under which data can be accessed and processed. This approach is particularly relevant to process mining, where multiple analytical techniques are applied, and stringent control mechanisms ensure compliance with proper data usage throughout the analysis. **Process mining** is a discipline across data science and process management aimed at analyzing real-world process executions using event logs recorded by information systems [12]. It helps understand how processes are actually performed, verify compliance with expected behavior, assess performance, and compare different execution outcomes. In this paper we will focus on the process discovery task that gives a record of events in input, where each case (process instance) is represented by a trace, i.e., a sequence of timestamped events tied to specific activities, returns a model that identifies the behavior of the process. For example, foundational techniques for process discovery (such as Heuristic Miner and Alpha Miner) use temporal and ordering patterns in the events of the log [13,24]. These logs typically follow the XES (eXtensible Event Stream) standard format [1]. The output is a process model or analysis that reveals behavior, deviations, or inefficiencies. A **Trusted Execution Environment (TEE)** is a secure processor area that protects code and data inside from disclosure or tampering, even by privileged software like the operating system [23]. Frameworks like the Intel Software Guard Extensions (SGX) utilizes *enclaves* to encapsulate the so-called Trusted Applications (TAs) in isolated memory regions and thus securely execute sensitive operations. Data within an enclave is encrypted and inaccessible to external software. Enclaves support sealing, a process that allows data to be securely encrypted and stored outside the enclave, bound to the enclave's identity, ensuring that only the same enclave instance can later decrypt and access the sealed data. Each enclave is uniquely identified by a so-called cryptographic *measurement* computed by the CPU at deployment time [4]. This identity allows external software or services to verify that an enclave runs trusted code on untampered hardware through a mechanism that goes under the name of *remote attestation*. Specifically, the enclave generates a signed remote attestation report that includes its measurement and platform information. This report can be verified by Intel's attestation service [26], establishing trust and enabling secure interaction with sensitive data even in untrusted environments [7]. Since TEEs provide a secure basis for protecting sensitive computation, they are well-suited technologies for enforcing usage control policies that ensure continuous regulation of data access and operations.

2.2 State of the Art

The interest in secrecy and privacy preservation for process mining has been concretized in different research outputs over the last few years. A body of solutions focus on the preprocessing of input data to shield unintentional information revealing. PRIPEL [16] anonymizes contextual information in event logs by applying differential privacy principles. Fahrenkrog et al. [17] later proposed BF-PRETSA, a sanitization method that preserves control-flow semantics while

enforcing k-anonymity and t-closeness, aiming to maintain utility despite structural data modifications. Batista and Solanas present u-PPPM [6], a privacy-preserving process mining technique that addresses privacy threats stemming from distributional and location-based attacks by uniformizing event distributions to limit re-identification risks, while minimizing information loss. Although these techniques enhance privacy by transforming the data, they alter the event log and may compromise the accuracy and interpretability of the discovered models. In contrast, ProMiSe operates on unmodified logs, enforcing compliance through policies that govern data access, processing, and output. Another line of research investigates software and hardware-backed protocols to support cross-organizational process mining while preventing any exposure of the underlying data, revealing only the outcome of the computation. Elkoumy et al. [15] leverage Secure Multi-Party Computation to allow different parties to jointly discover process models while keeping data confidential. Recent work leveraging TEEs for secure process mining [5,18,21] demonstrates how mining can be performed on sensitive data without prior anonymization. However, these solutions focus mainly on secure computation and do not address policy-based control over input and output usage or access rights, thus overlooking end-to-end governance. ProMiSe extends this paradigm by integrating policy enforcement across the full process discovery lifecycle, including input control, algorithm selection, and output restrictions.

3 Motivating Scenario and Requirements

To motivate our research, we introduce a use case scenario inspired by a known case in process mining for healthcare [20]. Hospitals and other entities, such as Italy's National Agency for Regional Health Services (AGENAS), the United States' Joint Commission on Accreditation of Healthcare Organizations (JCAHO), and India's National Accreditation Board for Hospitals & Healthcare Providers (NABH)[1] play a critical role in assessing the quality, safety, and efficiency of healthcare processes. These organizations support continuous improvement through standardized evaluations and data driven performance analysis. In our running example, we consider a hospital specialized in the treatment of sepsis that aims to collaborate with an external provider of process discovery services (henceforth, ProcessLock), to analyze and improve its clinical workflows. While outsourcing to ProcessLock could bring valuable expertise and computational resources, it raises significant concerns regarding the confidentiality and protection of highly sensitive medical data. Medical event logs contain detailed records, including timestamps of clinical activities, laboratory test results, admissions and discharges, diagnostic procedures, and personal information of patients and medical staff. Moreover, ensuring secure and controlled data sharing among trusted stakeholders, such as cooperating hospitals and healthcare quality assurance

[1] AGENAS: www.agenas.gov.it, JCAHO: www.jointcommission.org; NABH: nabh.co. Accessed: December 9, 2025.

Table 1. GDPR-based requirements for a trusted process discovery system

ID	Requirement	GDPR Chapter
R1	**Lawful processing with restricted purposes.** Personal data must be processed lawfully, for defined purposes, and in a transparent manner respecting user consent	2
R2	**Data minimization and retention.** Access to and retention of data must be limited to what is strictly necessary for the defined purpose and duration	3, 8
R3	**Security, protection by design, and accountability.** The system must protect data by default through technical and organizational measures ensuring confidentiality and integrity. It must also enable demonstration of compliance with the GDPR through comprehensive logging, traceability, and auditability mechanisms	4
R4	**Controlled data transfer and jurisdictional awareness**. Personal data must only be transferred to third countries or international organizations if appropriate safeguards are in place or specific derogations apply. The system must support geofencing and jurisdiction based policy enforcement	5
R5	**Facilitate supervisory.** The system must support the export of data protection audit logs and policy definitions in a standardized, human readable format to aid supervisory authority reviews and ensure transparency for compliance inspections	6
R6	**Regulatory cooperation in cross border contexts.** The system must support documentation and traceability mechanisms that facilitate cooperation between supervisory authorities in different countries by ensuring consistent application of GDPR requirements in cross border processing scenarios	7
R7	**Sensitive processing adaptation.** When processing occurs in special contexts such as scientific research, the system must ensure that it is not possible to reconstruct the profiles of data owners	9

bodies like AGENAS, NABH, and JCAHO, is essential to enable cooperative evaluation and continuous clinical improvement.

Table 1 summarizes the set of requirements we derived from the General Data Protection Regulation (GDPR), by addressing all chapters that introduce obligations pertaining to a process mining context. They reflect the core regulatory principles, such as lawful and transparent processing, data minimization, security, and accountability, aimed at ensuring the protection of users' rights and interests, which we aim to fulfill with our solution. Next, we individually contextualize them in our motivating scenario.

When handling sensitive data, such as healthcare information, the risk of misuse is high if processing is allowed without restrictions on its intended purpose. Req. **R1** is needed to prevent uncontrolled or secondary uses of personal data,

ensuring that processing remains tied to the original, legitimate intent defined by the data owner.

Example 1 (Req. R1: Purpose-restricted process discovery in sepsis care). A hospital may share event logs exclusively to analyze delays in antibiotic administration through process discovery. To prevent unintended uses, other analyses are excluded.

Healthcare data often includes large volumes of information, much of which may be irrelevant for a given analysis. Without careful limitation, there is a significant risk of overexposure or unnecessary retention. Req. **R2** addresses the problem of unmotivated access and long-term storage by setting boundaries to what is accessed and for how long.

Example 2 (Req. R2: Minimized and time-bound access to sepsis treatment data). To analyze delays in antibiotic administration, only essential data, such as timestamps, lab results, and drug administration records should be accessed. All other data must be excluded, and retention limited to prevent overexposure and unnecessary storage.

Given the sensitivity of medical logs and the involvement of external service providers, ensuring the confidentiality, integrity, and traceability of data processing is critical. Req. **R3** is essential to prevent unauthorized access and to support accountability through verifiable evidence of who accessed which data and under what conditions.

Example 3. The hospital shares event logs covering a 9 day period with ProcessLock for the analysis of sepsis workflows. Access is restricted to authorized hospital staff and AGENAS. All data interactions are securely logged, enabling both the hospital and AGENAS to verify who accessed the data, when, and for what processing purpose.

When data is shared across borders, differences in legal protections may expose individuals to greater privacy risks. Req. **R4** addresses this by requiring control mechanisms on the location from which data are accessed.

Example 4 (Req. R4: Geographic restriction of data access). The hospital requires that all processing of sepsis event logs by ProcessLock take place within Italy, prohibiting access to user located abroad.

In regulated environments like healthcare, organizations must be able to prove compliance when requested by authorities. Without accessible documentation and records, oversight becomes ineffective. Req. **R5** ensures that policy definitions and audit trails can be reviewed in case of inspections or investigations.

Example 5 (Req. R5: Transparent documentation for regulatory inspection). During an inspection by AGENAS, the hospital exports usage policies and access logs showing how sepsis event data were processed, enabling verification of compliance with national healthcare regulations.

Table 2. Non-functional requirements

ID	Requirement
NR1	**Memory consumption.** The system must optimize the resources available to prevent memory depletion during the execution of tasks [14,18]
NR2	**Runtime overhead.** The system shall ensure that the runtime overhead is minimized compared to baseline execution without security mechanisms [10,25,27]
NR3	**Scalability.** The system shall efficiently handle multiple concurrent clients by maintaining continuous service availability [10,11]

For collaborative analysis, the system should support documentation and evidence that can be shared between supervisors and auditors, to help them resolve any differences in regulatory interpretation. Req. **R6** is necessary to maintain consistency in such settings, preventing conflicts between differing legal expectations.

Example 6 (Req. R6: Facilitating multi-authority cooperation in joint processing scenarios). An Italian hospital participates in a cross-border sepsis research initiative involving hospitals in India. Event logs are shared among participants to enable process discovery across institutions. To ensure consistent regulatory interpretation, the hospital provides all research partners, including oversight bodies such as AGENAS and NABH, with structured, human-readable policy documentation.

Medical information requires special protection due to its sensitivity and potential impact on individuals. In contexts such as research, where data is reused or aggregated, it is essential to implement strict safeguards. Req. R7 specifically addresses the need to prevent re-identification and unintended disclosures while still enabling valuable analysis.

Example 7 (Req. R7: Privacy-preserving use of clinical logs in sepsis research). The hospital collaborates with other institutions to study the impact of variations in antibiotic administration timing on sepsis outcomes. To support this research, process discovery techniques are applied to identify treatment patterns. Access to the data is strictly limited to prevent the exposure of identifying information related to patients or medical staff.

To ensure that the system remains safe and practically usable, we define a set of non-functional requirements based on evaluation practices for systems that leverage TEEs. These requirements are defined from the perspective of the service provider, focusing on aspects that impact system performance, scalability, and robustness. Table 2 shows the non-functional requirements defined for this paper. The requirements have been defined on the basis of commonly adopted assessment criteria reflecting concerns arising in the practical deployment of safe

applications. Due to the limited memory capacity allocated to each TA within TEEs, surpassing this limit may lead to system failures and service disruptions or break confidentiality guarantees due to the use of external memory. It is thus essential that the system be optimized to complete the required operations within available resources. Therefore, the objective of **NR1** is to assess the capability of performing process discovery operations within a TEE by verifying, on real-world logs, whether the required executions can complete successfully without exceeding the memory available to the enclaves. Req. **NR2** focuses on minimizing runtime overhead, as enclaves introduce performance penalties due to restricted system calls, alongside encryption and decryption operations in memory, which are essential to ensure a secure and isolated execution environment [10,25,27]. Since ProMiSe is intended to operate as a continuous service for process discovery, excessive memory usage can threaten its availability. The scalability assessment therefore aims to evaluate the system's ability to maintain stable operation and predictable performance under increasing levels of concurrent use [10,27]. Req. **NR3** addresses the need to ensure that resource consumption, particularly memory, remains within acceptable bounds, while preserving sufficient responsiveness to support realistic multi-user scenarios without compromising overall system reliability.

Example 8 (Reqs. NR1 to NR3: Performance and scalability requirements). The hospital submits sepsis treatment event logs to ProcessLock for process discovery. ProcessLock is required to execute discovery algorithms within the memory constraints of the enclave and within acceptable processing times. Furthermore, as the hospital authorizes multiple external stakeholders to access the results, the system must be scalable to handle concurrent requests without degrading performance or availability.

4 Conceptual Design

In this section, we describe ProMiSe's policies, architecture, and internal data flow.

4.1 Policy

ProMiSe usage policies are modeled using RDF. To align the usage policies with the motivating scenario of the paper, we developed a domain specific policy language tailored to process mining. Our goal is to model the fundamental elements of process models and discovery directly within the policy structure, enabling precise and context-aware policy enforcement.

Figure 1 illustrates the structure of the policies adopted in our approach. Each policy consists of four main elements, which collectively regulate the secure and compliant use of event logs within the trusted environment. (i) The **Object Identification** element defines the data subject to the policy, specifying the event log along with its associated metadata, such as filename and format. (ii) The **Log Usage Rules** element specifies the conditions under which the event

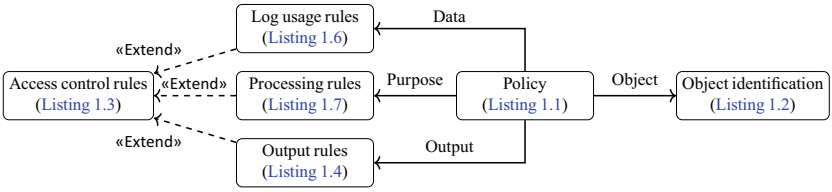


Fig. 1. Policy structure

log may be accessed and used. These rules include constraints such as log expiration, geographic access restrictions, exclusion of sensitive attributes, permitted time windows for analysis, constraints on event inclusion or exclusion, and limits on the number of authorized accesses. (iii) The **Processing Rules** define which process discovery techniques and algorithms are permitted, thereby restricting a third-party platform to execute only authorized forms of analysis within the trusted environment. (iv) The **Output Rules** govern the handling of analysis results, specifying constraints on their accessibility, temporal validity, and geographical availability, thereby ensuring controlled dissemination and use of derived insights. The prefixes specified in the following example apply to all examples presented in this section.

```
@prefix ucon: <http://example.org/ucon#> .
@prefix eventLog: <http://example.org/eventLog#> .
@prefix pmt: <http://example.org/pmt#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix loc: <http://id.loc.gov/vocabulary/countries/> .
ucon:objectId [...];
ucon:logUsageRules [...];
ucon:processingRules [...];
ucon:outputRules [...];
```

Listing 1.1. Example of the policy structure

To identify the object to which the policy applies, the object identification element includes two properties: `fileName`, and `format`, indicating the name and format of the event log file respectively.

```
eventLog:fileName "eventLog.xes"^^xsd:string;
eventLog:format "XES";
```

Listing 1.2. The event log file named “eventLog.xes” is provided in XES format

Users can specify the identities authorized to access each phase, as well as the geographic locations from which access is permitted, within the three elements of the process discovery lifecycle: `logUsageRules`, `processingRules`, and `outputRules`.

```
ucon:allowedLocations loc:it;
ucon:accessControlRules "AGENAS";
```

Listing 1.3. Only AGENAS agents in Italy have the permission to access the event log

The policy also allows users to define expiration times and access limits for both the event log and the output models generated during processing.

This can be achieved by specifying the properties "ucon:expiration" and "ucon:maxAccessCount", as illustrated in the following example.

```
ucon:expiration "2025-12-31T23:59:59Z"^^xsd:dateTime;
ucon:maxAccessCount "9"^^xsd:integer;
```

Listing 1.4. Log expiration set to the end of the year 2025, with at most 9 accesses

To impose temporal constraints on the event log, the policy may include the "allowedTimeRange" property. This property specifies a start and end date, thereby restricting the events considered during processing to those occurring within the defined interval. Additionally, it allows the user to indicate the specific event attribute that contains the timestamp, enabling accurate identification of the temporal dimension within the log.

```
ucon:allowedTimeRange [
    ucon:eventAttribute "time:timestamp";
    ucon:startDate "2020-01-01T00:00:00Z"^^xsd:dateTime;
    ucon:endDate "2025-01-01T00:00:00Z"^^xsd:dateTime ];
```

Listing 1.5. Only events timestamped between 2020 and 2025 can be accessed

The user, defining the "logConstraints" property, in the "logUsageRules" elements, defines conditions based on a specific event attribute (specified by the "ucon:eventAttribute" property) to determine which traces are included in the analysis. It requires that traces contain events with attribute values listed in "mustInclude" property and exclude traces containing events with values listed in "mustExclude" property.

```
ucon:logConstraints [
    ucon:eventAttribute "concept:name";
    ucon:mustInclude ( "ER Triage" );
    ucon:mustExclude ( "Leucocytes" "IV Antibiotics" ) ];
```

Listing 1.6. Include events with value "ER Triage" and exclude those with values "Leucocytes" and "IV Antibiotics."

Finally, within the "processingRules" element, the user can specify the types of operations permitted on the data. This is achieved by defining the "allowedTechnique" property, which contains one or more specifications of mining techniques. Each technique is described using two internal properties: "pmt:techniqueType", indicating the category of the technique, and "pmt:algorithm", specifying the exact algorithm allowed.

```
ucon:allowedTechnique (
    [ pmt:techniqueType pmt:AutomatedDiscovery;
        pmt:algorithm pmt:HeuristicMiner ],
    [ pmt:techniqueType pmt:AutomatedDiscovery;
        pmt:algorithm pmt:AlphaMiner ] )
```

Listing 1.7. Only the HeuristicMiner and AlphaMiner discovery algorithms are allowed

4.2 Architecture

Figure 2 provides an overview of the architecture of ProMiSe. We assume that a user can play two distinct roles: data provider or data consumer. When acting as

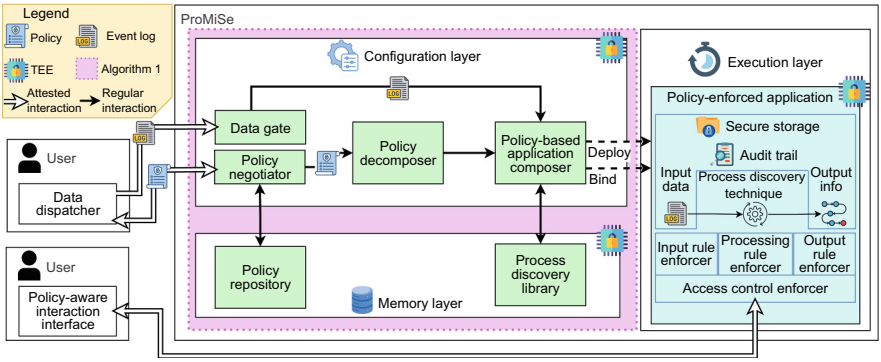


Fig. 2. An overview of the ProMiSe architecture

a data provider, the user initiates the process by requesting the setup of a secure environment for conducting process discovery analysis within the ProMiSe system. To this end, the user submits to ProMiSe both the event log intended for analysis and the corresponding usage policy, which specifies the constraints and conditions under which the data may be accessed and processed. When acting as a data consumer, the user submits a request to ProMiSe to perform an operation on an event log that has already been provided and initialized by a data provider within a secure environment. To initiate such a request, the data consumer must first authenticate. ProMiSe then evaluates whether the requested operation is permitted by verifying compliance with the usage policy associated with the data. If all policy conditions are satisfied, the request is granted and the operation is executed, otherwise, access is denied. ProMiSe is structured into three main layers. The configuration layer is responsible for receiving input data, setting up the execution environment. The memory layer maintains copies of the policies submitted to ProMiSe as well as all known and applicable algorithms relevant to processing the data. The execution layer encompasses the generated execution environment where user data will be preserved, managed and processed. Since these layers handle sensitive user data, they operate entirely within a TEE. Consequently, all operations performed within these layers utilize TEE resources, and all data stored therein are encrypted through TEE's mechanisms (see Sect. 2). The figure depicts two types of arrows: double arrows represent attested interactions, while black arrows indicate regular interactions. Attested interactions involve remote attestation procedures and occur at two critical stages. The first attestation takes place when the data provider submits data to the system, verifying the authenticity and integrity of the system components involved in the interaction, to ensure they have not been compromised. The second attestation occurs when the data consumer requests to access or process the data, and it targets the output produced. This step confirms that the operations were performed by a trusted and unmodified system, thus validating the integrity and trustworthiness of the result.

Algorithm 1: Generation of policy-enforced applications

```
   Input: policyFile (a file with usage policies), and logFile (a file with the event log)
   Output: tApp (a trusted application for policy enforcement)
1  triples ← parseRDF(policyFile);
2  phases ← ⟨"logUsageRules", "processingRules", "outputRules"⟩;
3  foreach phase ∈ phases do
4      rules[phase] ← extractPhase(triples, "ucon:" + phase);
5  policyID ← hash(triples);
6  tApp ← retrieveTA(policyID);
7  if tApp is null then
8      enforcers ← generateEnforcers(rules);
9      tApp ← generateTApp(policyID, enforcers);
10     deployTApp(tApp);
11 bindEventLogToTApp(tApp, logFile);
12 return tApp;
```

ProMiSe receives data from the user acting as a data provider through two components: The *Data Gate*, which processes the submitted data and forwards it to the *policy-based application composer*, and the *Policy Negotiator*, which performs preliminary checks on the policy to ensure it is syntactically correct and compatible with the system. If the check fails, a request for modification is issued to the data provider. Once the policy passes these checks, the *Policy Negotiator* stores a copy in the *Policy Repository* and forwards the policy to the *Policy Decomposer*. The *Policy Repository* leverages sealing within the Trusted Execution Environment, encrypting policies so that they can only be accessed and decrypted by authorized components. The *Policy Decomposer* component is responsible for extracting the relevant information from the submitted policy. Once extracted, this information is passed to the *Policy-based Application Composer*, which is in charge of the generation of the enforcers from the policy and to compose the trusted application responsible for enforcing the policy on the data. This trusted application is the *policy-enforced application*, and it encapsulates both the policy logic and the operational constraints. The *Policy-based Application Composer* receives the decomposed policy and first checks whether its structure matches that of an existing, previously processed policy. If a *policy-enforced application* corresponding to the same policy has already been generated, the *Policy-based Application Composer* reuses the existing instance by binding the new event log to it, avoiding redundant trusted application creation.

Algorithm 1 operates with the components highlighted by a dotted pink area in Fig. 2 and details the procedure for generating the application. This algorithm takes as input the policy and the event log, parses the policy, and identifies the phases for which the data provider can define authorized behaviors. The phases are named `logUsageRules`, `processingRules`, and `outputRules`. Each set of rules is extracted (line 3), and a unique application identifier is computed by hashing the policy structure (line 5). This identifier is used to check whether an application with an identical rule configuration already exists (line 7). If no such application is found, the system generates the necessary enforcers, which are then used to generate the trusted application (line 8). Once built, the appli-

Table 3. Dataset statistics

Name	Activities	Cases	Events		
			Max	Min	Avg.
Sepsis	16	1050	185	3	15
BPIC 2013	7	1487	123	1	9
BPIC 2012	36	13087	75	3	20

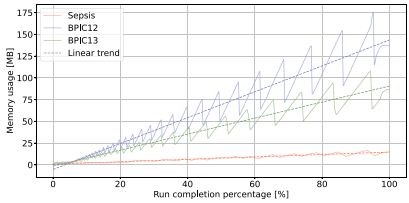


Fig. 3. Memory usage test

cation is deployed (line 9–10) and the event log is stored within its secure storage (line 11). In case a matching application already exists, the system reuses it and associates the new event log with the corresponding trusted application. Otherwise, it proceeds to generate a new *policy-enforced application* that embeds all the constraints specified by the policy and the event log. Generating a new application requires building and deploying it within the TEE to guarantee confidentiality and data integrity. For all the rules regarding which techniques may be applied, the relevant methods are retrieved from the *Process discovery library* and incorporates them into the application. The *Process discovery library* acts as a repository for ProMiSe, providing both domain knowledge and details about available mining techniques. Thus, we have shown how ProMiSe dynamically generates *Policy-enforced application*s that implement the data provider's specified rules. A *Policy-enforced application* includes a *Secure storage*, wherein input and output data are retained in a controlled manner to ensure compliance with the defined policies. In the *Secure storage* is also the audit trail file that each application has to show how the data are used by tracking all operations performed on them. The *Policy-enforced application* enforces rules specified by the data provider to govern the entire lifecycle of the shared data. These rules address the handling of input data, the processing logic applied during computation, and the management of output models generated after execution. At each stage (input, processing, and output), the *Policy-enforced application* applies access control mechanisms, if defined in the policy, to restrict access to authorized entities only.

5 Evaluation

This section presents the evaluation of ProMiSe. We begin with a performance assessment to verify compliance with the non-functional requirements in Table 2 (Sects. 5.1 and 5.2). Then, we show how the system meets the functional requirements in Table 1 by design (Sect. 5.3).

5.1 Implementation and Experimental Setup

We implemented the components running within the TEE in Go using the EGo framework, deploying them within an Intel SGX TEE[2] on a machine equipped

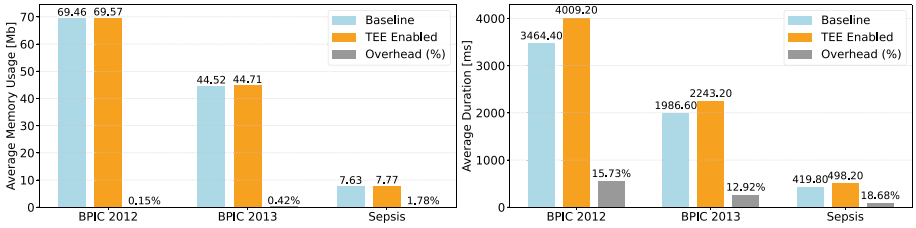
[2] EGo: edgeless.systems/products/ego. Intel SGX: sgx101.gitbook.io/sgx101.

with an Intel Xeon Gold 5415+ CPU. The *Policy-enforced application*, representing the core services offered by ProMiSe, exposes a set of APIs to interact with user-submitted data. Communication with the application occurs via RPC-style calls, enabling remote invocation of only those functionalities explicitly exposed by the application. These reflect the rules defined in the corresponding policy. The remaining components of the system, including user interaction, policy management, and file handling, were developed in Python. The code is available at https://github.com/ValerioGoretti/ProMiSe. We used three real-world event logs summarized in Table 3. We performed performance tests on the execution of the process discovery algorithm, using the Heuristic Miner as a representative case due to its relatively high computational demands compared to other supported techniques. We employed a single policy for all logs, including the rules in Sect. 4.1 and encompassing user authentication, geographical access restrictions, log expiration, maximum access limits, and validation of the permitted processing technique. For this evaluation, a *Policy-enforced application* was generated based on it, and the above event logs were bound to it. We collected test data by issuing requests to the trusted application while triggering all relevant policy enforcement checks. We repeated every experiment five times per configuration.

5.2 Performance

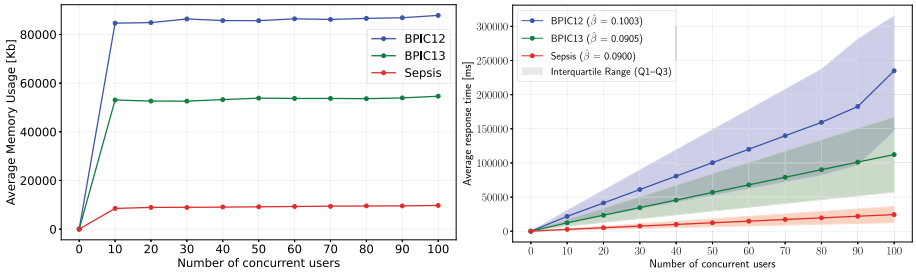
We gauge the performance of our solution in terms of memory consumption (to address **NR1**), runtime overhead (**NR2**), and scalability (**NR3**).

Memory Consumption. Due to the limited memory available to each TA operating within a TEE, the experiment we describe next is of relevance to **NR1**. Figure 3 presents the results of the memory usage test performed during the execution of the process discovery algorithm. The plot illustrates the trend of memory consumption (in MB) as a function of the completion percentage of each test run. For all logs, memory usage exhibits a linear growth trend, with minor oscillations around the regression line that represent the actual consumption profile. The saw-tooth patterns observed in the curves reflect the activity of garbage collection mechanisms, which periodically release accumulated memory during execution. In addition, all the curves exhibit increasingly wide memory fluctuations as the execution progresses. We attribute this behavior to the standard garbage collection policy employed by the Go runtime environment, which is automatically triggered based on an adaptive memory threshold. The execution on the Sepsis log displays consistently low memory usage, remaining below 20 MB and showing a nearly flat growth curve, indicating minimal resource requirements. The execution with BPIC 2013 log demonstrates a moderate and constantly increasing memory footprint, peaking at approximately 90 MB. In contrast, BPIC 2012, the largest and most complex log of the set, exhibits the highest memory demand, reaching up to 150 MB by the end of the execution. Despite this, the processing was successfully completed within the limits of the system used.



(a) Memory overhead test (b) Time overhead test

Fig. 4. Overhead test



(a) Scalability on average memory usage (b) Scalability on average response time

Fig. 5. Scalability tests

Overhead. This subsection assesses the runtime overhead incurred by executing within a TEE. The objective is to demonstrate that the enhanced security guarantees provided by the enclave impose minimal performance degradation relative to an unprotected baseline environment. The overhead evaluation addresses Req. NR2, which concerns the performance impact of executing within a TEE compared to a baseline implementation without the TEE support. Figure 4 summarizes the experiments conducted to measure system overhead in terms of execution time and memory consumption. As depicted in Fig. 4(a), memory usage during TEE execution remains close to that of the baseline, with only a slight increase consistently observed across all logs. Time overheads are minimal, ranging from 0.11 ms for the BPIC 2012 log to 0.19 ms for the Sepsis log, indicating that the performance cost introduced by TEE execution remains under control. Figure 4(b) presents the time overhead observed during the same test runs used to measure memory usage. In this case, the use of the TEE results in approximately a 16 % increase in execution time. This overhead primarily stems from the cryptographic operations performed by the TEE during memory access. Additionally, remote attestation mechanisms contribute to this, as they require the TEE to generate a report based on the output.

Scalability. Since ProMise is intended to operate as a continuous service, we evaluate the scalability of our solution, thus addressing NR3. Our primary objective remains to avoid exceeding TEE memory limits, which can represent a serious threat to service continuity. As stated in Sect. 4, we integrate a queue-based mechanism within the *Policy-enforced application* to counteract possible transient memory overloads by serializing requests.

Figure 5 illustrates the results of the scalability tests. We simulated concurrent invocations of the same Trusted Application by increasing the number of parallel user calls from 10 to 100, by increments of 10. Figure 5(a) illustrates the scalability of the system in terms of average memory usage under concurrent execution. In all the test configurations, the memory consumption presents stable trend. The average memory usage remains approximately 1000 KB for Sepsis, around 5500 KB for BPIC 2013, and slightly above 8000 KB for BPIC 2012, consistently remaining below 9000 KB. These results confirm that the system maintains predictable memory usage even under high concurrency. This queue-based strategy can affect responsiveness in favor of service availability. To quantify this trade-off, we also analyze the response latency induced by multiple concurrent processing requests. Figure 5(b) reports the average waiting time per test. For each curve, we shaded the area between the first and third quartiles using a gradient of the same color, representing the central distribution range of the observed waiting times. The results show a linear increase in waiting time across all logs tested. To quantify the slope of each growth trend, we computed the average $\hat{\beta}$ by fitting a linear regression over response times normalized to the initial value. In this way, each $\hat{\beta}$ expresses the average relative increase (in decimal form) in response time per additional concurrent user. This global trend estimation enables a consistent evaluation and comparison of the scalability behavior across the different logs. The executions on Sepsis and BPIC 2013 exhibit an average relative increase in response time of about 9 % ($\hat{\beta} = 0.09$ and $\hat{\beta} = 0.0905$, respectively) per additional concurrent user, whereas BPIC 2012 shows a slightly higher value of 10.03 % ($\hat{\beta} = 0.1003$). These results confirm that the queue-based implementation, while effective in controlling memory usage, contributes to increased latency under high concurrency, underlining the need for further optimizations to maintain responsiveness in concurrent scenarios.

5.3 Faithfulness

In this section, we discuss how ProMise fulfills the requirements by design, relying on its architecture and the technologies it adopts. As described in the design section, *Policy-enforced application*s are generated based on a submitted policy. Thanks to this close link between policy and the trusted application, the constraints defined by the data owner are enforced, ensuring that only the operations explicitly allowed in the policy can be performed on the data. This design enables compliance with **R1**, since the RPC-based implementation of the generated trusted application exposes to users only the functionalities permitted for that specific event log. Likewise, the system satisfies **R2**, as the policy can specify which data attributes are not accessible and define retention rules such

as expiration and maximum number of accesses. Each time a request is made, these rules are taken into account, along with the user's identity and the geographic location from which the request originates (as required by **R4**) in order to grant or deny access. By enabling control over the entire data processing flow, including input, processing, and output, ProMise also ensures compliance with **R7**, limiting the exposure of sensitive information produced by process discovery techniques. Furthermore, ProMise addresses **R3** through encryption and decryption mechanisms and remote attestation, ensuring high confidentiality on the data and verifiability on the environment that execute and store it. ProMise also facilitates cooperation across jurisdictions by providing, at any time, a structured version of the rules that will be enforced by the application upon data usage requests, in line with **R6**. Finally, thanks to integrated auditing and traceability mechanisms, every operation and request, along with its outcome and all submitted information, is recorded and stored in a retrievable audit trail file, ensuring compliance with **R5**.

6 Conclusion and Future Work

In this paper, we presented ProMiSe, a policy enforcement framework implemented as a service to support process discovery. By enabling data owners to specify fine-grained usage policies, ProMiSe enforces strict control over data access, processing, and output, ensuring adherence to predefined constraints. Our future work will focus on four main directions. First, we aim to verify the extensibility of the defined policy language, evaluating its ability to support additional constraints and evolving requirements over time. Secondly, we plan to integrate automated policy verification and validation mechanisms, ensuring that any data submitted by the data owner complies with the expected constraints and usage rules. We intend to expand the scope of supported process mining tasks, moving beyond process discovery to include additional techniques such as conformance checking, performance mining, and variant analysis, in order to offer broader analytical capabilities. Finally, based on the results of the scalability tests, which revealed increased latency under high concurrency, we will improve request management through two complementary strategies. The first involves implementing a load balancing mechanism across multiple instances of the trusted application, in order to achieve better request distribution, preserve memory isolation, and enhance overall scalability. The second consists in supporting the processing of large log files by dividing them into smaller chunks. This strategy enables the trusted application to handle extensive datasets without exceeding the memory limitations imposed by the secure execution environment.

References

1. IEEE Standard for eXtensible Event Stream (XES) for achieving interoperability in event logs and event streams (2023). https://doi.org/10.1109/IEEESTD.2023.10267858

2. van der Aalst, W.M.P., Carmona, J. (eds.): Process Mining Handbook (2022)
3. Akaichi, I., Kirrane, S.: Usage control specification, enforcement, and robustness: a survey. arXiv preprint arXiv:2203.04800 (2022)
4. Anati, I., Gueron, S., Johnson, S., Scarlata, V.: Innovative technology for cpu based attestation and sealing. In: Proceedings of the 2nd International Workshop on HASP, vol. 13 (2013)
5. Basile, D., Di Ciccio, C.: Secrecy preservation for online process monitoring with trusted execution environment. In: BPM, pp. 235–254 (2025)
6. Batista, E., Solanas, A.: A uniformization-based approach to preserve individuals' privacy during process mining analyses. P2P Network. Appl. **14**(3), 1500–1519 (2021)
7. Birkholz, H., Thaler, D., Richardson, M., et al.: Rfc 9334: remote attestation procedures (rats) architecture (2023)
8. Borovits, N., Bardelloni, G., Tamburri, D.A., van den Heuvel, W.: Anonymization-as-a-service: the service center transcripts industrial case. In: ICSOC, pp. 261–275 (2023)
9. Burattin, A., Conti, M., Turato, D.: Toward an anonymous process mining. In: FiCloud, pp. 58–63 (2015)
10. Chen, L., Li, J., Ma, R., et al.: Enclavecache: a secure and scalable key-value cache in multi-tenant clouds using intel sgx. In: Middleware, pp. 14–27 (2019)
11. Chouksey, A., et al.: Oblix: an efficient oblivious search index. In: IEEE Symposium on Security and Privacy (SP) (2020)
12. De Weerdt, J., Wynn, M.T.: Foundations of process event data. In: van der Aalst and Carmona [2], pp. 193–211 (2022)
13. Dumas, M., La Rosa, M., Mendling, J., Reijers, H.A.: Fundamentals of business process management (2013)
14. El-Hindi, M., Ziegler, T., Heinrich, M., et al.: Benchmarking the second generation of intel sgx hardware. In: Proceedings of the 18th International Workshop on DaMoN, pp. 1–8 (2022)
15. Elkoumy, G., Fahrenkrog-Petersen, S.A., Dumas, M., et al.: Secure multi-party computation for inter-organizational process mining. In: BPMDS, pp. 166–181 (2020)
16. Fahrenkrog-Petersen, S.A., van der Aa, H., Weidlich, M.: PRIPEL: privacy-preserving event log publishing including contextual information. In: BPM 2020 (2020)
17. Fahrenkrog-Petersen, S.A., van der Aa, H., Weidlich, M.: Optimal event log sanitization for privacy-preserving process mining. Data Knowl. Eng. **145**, 102175 (2023)
18. Goretti, V., Basile, D., Barbaro, L., Di Ciccio, C.: Trusted execution environment for decentralized process mining. In: CAiSE, pp. 509–527 (2024)
19. Kirchmann, H., Fahrenkrog-Petersen, S.A., Mannhardt, F., et al.: Control-flow reconstruction attacks on business process models. In: EDOC, pp. 60–77. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-78338-8_4
20. Mannhardt, F.: Sepsis cases - event log (2016). https://doi.org/10.4121/UUID:915D2BFB-7E84-49AD-A286-DC35F063A460
21. Müller, M., Simonet-Boulogne, A., Sengupta, S., Beige, O.: Process mining in trusted execution environments: towards hardware guarantees for trust-aware inter-organizational process analysis. In: ICPM Workshops, pp. 369–381 (2021)
22. Park, J., Sandhu, R.: The UCONABC usage control model. ACM Trans. Inf. Syst. Secur. (TISSEC) **7**(1), 128–174 (2004)

23. Sabt, M., Achemlal, M., Bouabdallah, A.: Trusted execution environment: what it is, and what it is not. In: TrustCom/BigDataSE/ISPA (1), pp. 57–64 (2015)
24. Schönig, S., Di Ciccio, C., Maggi, F.M., Mendling, J.: Discovery of multi-perspective declarative process models. In: ICSOC, pp. 87–103 (2016)
25. Shen, Y., Tian, H., Chen, Y., et al.: Occlum: secure and efficient multitasking inside a single enclave of intel sgx. In: Proceedings of the ASPLOS 25, pp. 955–970 (2020)
26. Wang, J., Hong, Z., Zhang, Y., Jin, Y.: Enabling security-enhanced attestation with intel SGX for remote terminal and IoT. IEEE TCAD **37**(1), 88–96 (2017)
27. Weichbrodt, N., Aublin, P.L., Kapitza, R.: sgx-perf: A performance analysis tool for Intel SGX enclaves. In: Middleware, pp. 201–213 (2018)

GOProQ: A Graphical Query Language for Object-Centric Process Analysis

Gyunam Park[1,2(✉)], Jan Niklas Adams[3], and Daniel Schuster[4]

[1] Eindhoven University of Technology, Eindhoven, The Netherlands
g.park@tue.nl
[2] Fraunhofer Institute for Applied Information Technology (FIT), Sankt Augustin, Germany
[3] Process and Data Science, RWTH Aachen University, Aachen, Germany
[4] Process Intelligence Solutions, Aachen, Germany

Abstract. Service-oriented architectures facilitate distributed business processes where multiple services collaborate to achieve business goals. However, traditional process mining approaches rely on single-case event logs that fail to capture the complex interactions between services and their associated data objects. Object-Centric Event Logs (OCELs) address this challenge by modeling service processes as graphs of interconnected events and objects, providing a foundation for comprehensive service analysis. While existing querying approaches offer capabilities for traditional event data, they lack the expressiveness needed for analyzing object-centric service interactions. This paper introduces GOProQ, a graphical query language specifically designed for querying object-centric service processes. We identify key requirements for service-oriented process querying, define a formal language that addresses these requirements, and implement an interactive tool for service analysts. Our evaluation across multiple real-world service datasets demonstrates GOProQ's expressiveness and performance for service monitoring in complex service-oriented environments.

Keywords: Service-Oriented Process Mining · Service Composition Analysis · Object-Centric Event Logs · Service Monitoring · Service Process Querying

1 Introduction

Process mining provides tools and techniques for discovering, analyzing, and enhancing business processes using event data. Traditionally, process mining relied on sequence-based single-case event logs standardized by the XES format [1]. However, this simplistic representation fails to capture the complexity of modern service-oriented processes where multiple objects (such as orders, items, and deliveries) interact simultaneously within service compositions and orchestrations.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 287–305, 2026.
https://doi.org/10.1007/978-981-95-5015-9_21

To address these limitations, Object-Centric Event Logs (OCELs) have been introduced, representing processes as interconnected graphs of events rather than linear sequences. This approach more naturally reflects the reality of complex service interactions in modern distributed systems.

In service-oriented computing, the ability to query cross-service interactions is essential for monitoring service quality, ensuring compliance with service-level agreements (SLAs), and optimizing service compositions [12,17]. Despite the importance of object-centric service analysis, querying such data remains challenging. Traditional process querying methods cannot adequately handle the multi-object interactions. For example, analyzing "service executions where orders involve items from multiple suppliers, with payments processed before all deliveries complete" requires understanding relationships between orders, items, suppliers, payments, and deliveries simultaneously, something that single-case approaches cannot express naturally.

Existing process querying methods are predominantly designed for traditional flattened event logs [9,11,13] and general process repositories [3,14,15], with limited applicability to service-oriented OCELs. While some approaches use general-purpose query languages like Cypher [7,8], the lack of specialized query frameworks hampers service analysts to extract meaningful insights for service monitoring and optimization.

To fill this gap, we investigate how querying can be adapted to leverage the multi-object structure of OCELs in service-oriented environments. Specifically, we propose GOProQ, a novel graphical query language tailored for object-centric service process data. Graphical query languages align with the structural properties of the data, reducing query complexity and enhancing user experience in service analysis contexts [18].

This paper makes the following contributions:

- We introduce a formal definition of a graphical query language designed for object-centric service event data, addressing the limitations of existing textual query languages for service monitoring and analysis.
- We implement an interactive application that allows service administrators and analysts to define, execute, and visualize graphical queries on OCELs.
- We evaluate the proposed query language using real-world service-oriented datasets, demonstrating its feasibility for analyzing complex service interactions.

The remainder of this paper is structured as follows. Section 2 reviews the state-of-the-art in service process querying and object-centric event data analysis. Next, Sect. 3 introduces preliminaries regarding object-centric event logs and service process executions. Then, Sect. 4 establishes the requirements for an object-centric service process query language, motivating our design choices. Section 5 defines the syntax and semantics of the proposed graphical query language based on these requirements. Section 6 presents a comprehensive evaluation of the language's expressiveness and performance for service analysis, and Sect. 7 concludes the paper.

2 Related Work

In this section, we review existing approaches to process querying [16], with a focus on their applicability to object-centric event data in service-oriented environments.

Service-Oriented Process Querying Languages. Several frameworks and specialized languages have been developed for querying service process data. The Process Instance Query Language (PIQL) [15] provides a comprehensive approach to querying service process instances, offering capabilities for filtering, pattern matching, and aggregation. However, PIQL's design focuses on traditional service process instances, making it less suitable for the graph structure of object-centric service executions that capture complex inter-service interactions. The Data-Aware Process Oriented Query Language (DAPOQ) [14] extends querying capabilities by incorporating data perspectives into process analysis. While DAPOQ enables correlation between service events and data elements, its process model assumes case sequences rather than the interconnected service object relationships found in service-oriented OCELs. BP-SPARQL [3] represents a significant advancement in graph-based service process querying. It utilizes SPARQL to query service process knowledge graphs, enabling complex pattern matching and aggregation across service interactions. While BP-SPARQL shares our approach's graph representation, it was not specifically designed for the unique characteristics of service-oriented OCELs, where multiple object types interact simultaneously within complex service orchestrations and choreographies.

Graph-based Querying Techniques. Graph-based techniques exploit the intuitive visual representation of graphs to model and query service-oriented process data, mirroring the interconnected nature of modern microservice architectures. Ly et al. [13] propose Compliance Rule Graphs (CRGs) based on first-order predicate logic to instantiate and verify service compliance rules. Knuplesch et al. [11] extend CRGs into Extended Compliance Rule Graphs (eCRGs), incorporating additional service process perspectives such as time, data, and resources for comprehensive service monitoring. Hildebrandt et al. [9] introduce Dynamic Condition Response Graphs (DCR Graphs) as a means to capture and enforce service compliance constraints visually. Despite their effectiveness in modeling service compliance, these approaches are tailored to traditional service event logs and fail to account for the complex relationships among different service objects inherent in object-centric service event data, particularly in microservice-based architectures.

Object-Centric Service Process Querying Approaches. The unique challenges of querying object-centric service event data have led to the development of specialized frameworks for service-oriented systems. Esser and Fahland [7,8] introduce event knowledge graphs and implement a querying framework using Neo4j and Cypher to handle object-centric event logs in service environments.

While this approach offers a flexible means of querying using the general-purpose query language, i.e., Cypher, it is not specifically designed for the analysis of object-centric service event data in complex service orchestrations. Such Cypher queries perform well for simple queries but struggle with more complex inter-service queries due to the lack of specialization for object-centric service process analysis and monitoring of service-level agreements.

Model Checking and Temporal Logic Approaches. Model checking approaches have been extensively applied to process verification, with tools like SPIN [10] and NuSMV [4], providing frameworks for verifying temporal properties over finite state machines. These approaches excel at analyzing single execution threads or global system states, where state transitions represent discrete system changes with clear branching choices, as demonstrated in classical applications to communication protocols and concurrent systems [6]. However, the semantic foundations of model checking are fundamentally misaligned with object-centric process querying requirements. While model checking tools like SPIN model system behaviors through state transitions with explicit choices and branching decisions, object-centric process executions represent recorded interactions between multiple service objects with independent lifecycles, where edges capture temporal precedence based on shared objects rather than state transitions. The multi-object nature of service-oriented architectures creates additional complexity that traditional model checking approaches struggle to address efficiently, as evidenced by the state explosion problem when modeling multiple interacting entities [5]. These semantic differences necessitate specialized approaches for object-centric process querying that can natively handle multi-object interactions and actual execution analysis.

3 Preliminaries

In this section, we introduce the core concepts underlying our query language, specifically focusing on Object-Centric Event Logs (OCELs) and the notion of service process executions. We introduce service process executions as the central abstraction for querying object-centric service event data.

Object-Centric Event Logs. Traditional event logs represent service activities from a single-case perspective, typically capturing events related to a single service process instance (or case). In contrast, OCELs record events involving multiple interacting service objects of various types, which better reflect the reality of service orchestrations and choreographies in modern architectures.

Formally, an OCEL is a tuple: $L = (E, O, \pi_{obj}, \pi_{act}, \pi_{time})$, where E is the set of service events; O is the set of service objects, with each object $o \in O$ assigned a type $\pi_{type}(o)$; $\pi_{obj} : E \to \mathcal{P}(O)$ maps each service event to a set of associated service objects; $\pi_{act} : E \to \mathcal{A}$ assigns an activity label to each service event, where $\mathcal{A}$ is the universe of service activities; $\pi_{time} : E \to \mathcal{T}$ maps each service event to a timestamp, enabling service-level agreement monitoring and temporal analysis.

Table 1 shows an event log $L_1 = (E_1, O_1, \pi_{obj}, \pi_{act}, \pi_{time})$ from a service-oriented order fulfillment system where $E_1 = \{e_1, e_2, e_3, \dots\}$ and $O_1 = \{o_1, o_2, i_1, i_2, d_1, c_1, \dots\}$. Each service event $e_i \in E_1$ is associated with a set of service objects through the mapping π_{obj}. For example, $\pi_{obj}(e_1) = \{o_1, i_1, i_2, i_3, c_1\}$ indicates that the service event e_1 involves the order service object o_1, item service objects i_1, i_2, i_3, and customer service object c_1, representing a cross-service interaction typical in microservice architectures. π_{act} assigns service activities to events, e.g., $\pi_{act}(e_1) =$ `Place Order`, while π_{time} assigns timestamps to events, e.g., $\pi_{time}(e_1) = 21.10.\ 02{:}26{:}11$, which is crucial for monitoring service-level agreements and service performance.

Table 1. Example for an object-centric service event log

Event ID	Service Activity	Timestamp	Service Object Types			
			Order	Item	Delivery	Cust.
e_1	Place Order	21.10. 02:26:11	o_1	i_1, i_2, i_3		c_1
e_2	Confirm Order	21.10. 02:26:12	o_1	i_1, i_2, i_3		c_1
e_3	Pick Item	21.10. 02:30:55		i_1		
e_4	Pick Item	21.10. 02:37:50		i_2		
e_5	Pick Item	21.10. 02:57:50		i_3		
e_6	Load Cargo	21.10. 03:02:26		i_1, i_2	d_1	
e_7	Fuel Car	23.10. 14:30:54			d_1	
e_8	Send Invoice	23.10. 16:25:46	o_1			c_1
e_9	Load Cargo	23.10. 17:02:26		i_3	d_2	
e_{10}	Receive Payment	24.10. 12:22:17	o_1			c_1
e_{11}	Receive Payment	24.10. 14:55:44	o_1			c_1
e_{12}	Start Route	24.10. 19:56:03		i_1, i_2	d_1	
e_{13}	Start Route	24.10. 21:54:03		i_1, i_2	d_1	
e_{14}	End Route	26.10. 09:56:03		i_3	d_2	
e_{15}	End Route	26.10. 15:34:03		i_3	d_2	
e_{16}	Place Order	21.10. 11:36:57	o_3	i_4		c_2
...	...	...	...	...	...	...

Given event log $L = (E, O, \pi_{obj}, \pi_{act}, \pi_{time})$, the precedence constraints between service events $\prec_L$ capture the temporal dependencies between the events based on shared service objects, representing service dependencies and interactions. Formally, $\prec_L = \{(e, e') \in E \times E \mid e \neq e', \exists o \in O : o \in \pi_{obj}(e) \wedge o \in \pi_{obj}(e') \wedge \pi_{time}(e) < \pi_{time}(e') \wedge \nexists e'' \in E : o \in \pi_{obj}(e'') \wedge \pi_{time}(e) < \pi_{time}(e'') < \pi_{time}(e')\}$. For instance, since both e_1 (`Place Order`) and e_2 (`Confirm Order`) involve the service object o_1, and $\pi_{time}(e_1) < \pi_{time}(e_2)$ without any event in between, we have $(e_1, e_2) \in \prec_{L_1}$, representing a direct service dependency.

Service Process Executions. The primary goal of our query language is to query service process executions rather than individual service events. A service process execution is a directed graph of events representing the execution of a service-oriented process. The edges describe the precedence constraints between service events given by service objects, capturing service interactions. This model generalizes traditional case-based service execution, which is a sequence of events. A sequence is a special case of a graph, just as a monolithic service is a special case of a microservice architecture.

Formally, a service process execution for a subset of service objects $O' \subseteq O$ is $p = (E', K)$, where (i) $E' \subseteq E$ is the set of service events associated with the service objects in O', i.e., $\forall e \in E'\ [\pi_{obj}(e) \cap O' \neq \emptyset]$ and (ii) $K \subseteq (E' \times E') \cap \prec_L$ is the set of precedence constraints, capturing the temporal order of service events based on shared service objects, representing service dependencies.

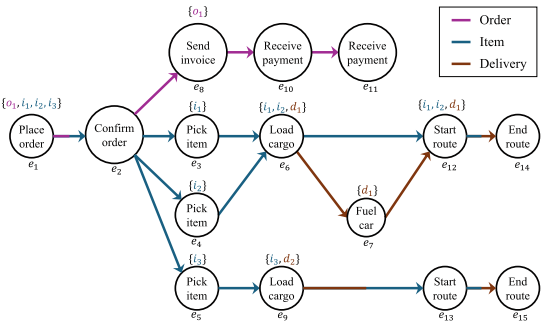


Fig. 1. Service process execution p_1 extracted from L_1 in Table 1, representing cross-service interactions. Note that some service object types/objects are omitted for brevity. c_1 is involved in all the service events.

Figure 1 shows a service process execution $p_1 = (E', K)$ from the service event log L_1 in Table 1. $E' = \{e_1, \ldots, e_{15}\}$ is the set of service events included in the process execution. Each service event in E' is associated with a subset of service objects O', such as orders (o_1), items (i_1, i_2, i_3), deliveries (d_1, d_2), and customers (c_1), representing interactions between different microservices in the system. The edges K define the precedence constraints between service events based on shared service objects and their temporal order, capturing the choreography of the service execution.

Service process executions are derived from object-centric event logs. One possible approach treats the service event log as a bipartite graph where service events and service objects form two distinct node sets, with edges connecting events to their associated objects. Service process executions are then identified by finding connected components in this bipartite graph—events that share service objects are grouped into the same execution, representing cohesive service interactions within a distributed architecture. For instance, in Table 1, service process execution p_1 (shown in Fig. 1) includes events e_1 through e_{15} because

they all share service objects and form a connected component. Conversely, service event e_{16} belongs to a separate process execution as it is associated with a different order service object (o_3) and customer (c_2) that do not interact with the objects in p_1, thus representing an independent service process instance.

4 Requirements for Object-Centric Process Querying

In this section, we establish requirements for a query language specifically designed for object-centric service process data. We analyze the structural characteristics of object-centric service process executions to identify fundamental querying dimensions, then define necessary expressiveness and performance requirements to guide our language design for service-oriented systems.

Service process executions extracted from OCELs have unique structural characteristics that impact querying requirements. As introduced in Sect. 3, a service process execution is represented as a graph, where nodes correspond to service events and edges represent precedence constraints between service events, mirroring the interconnected nature of modern service orchestrations.

The basic principle of querying service-oriented graphs involves interrogating nodes and edges (i.e., structural elements) using their associated labels [2]. In object-centric service process executions, these labels carry rich semantic information:

- Node labels include: (1) service activity names (e.g., `Load cargo`) and (2) service object types (e.g., `Item` and `Delivery`).
- Edge labels capture temporal service relationships: (1) between service activities (e.g., `Pick item` and `Load cargo`) and (2) between service object types (e.g., `Item` and `Delivery`), representing service dependencies.

These structural characteristics give rise to a two-dimensional framework for service process querying, as illustrated in Table 2. The horizontal dimension represents structural elements (nodes and edges), while the vertical dimension represents label types (service activities and service object types). Each cell in this matrix represents a specific query capability that our language must support for effective service monitoring and analysis.

Required Expressiveness for Service-Oriented Querying. Based on the querying dimensions in Table 2 and real-world service analysis needs, we define four key expressiveness requirements:

- **Support for all query dimensions:** The language must enable queries across both structural elements (nodes and edges) and label types (service activities and service object types), covering all cells in the matrix shown in Table 2.
- **Service-level cardinality constraints:** Beyond simple presence queries, the language must support cardinality constraints across all dimensions. This includes: Counting occurrences of service activities (e.g., "service executions

Table 2. Dimensions of querying service process executions. The query language must support all query types represented in this matrix to comprehensively address object-centric service process analysis needs.

Structural Elements	Label Types	
	Service Activity	Service Object Type
Node (Service Event)	Query nodes based on service activity presence, including start/end service activities	Query nodes based on their associated service object types
Edge (Service Precedence Constraint)	Query edges based on temporal relationships between service activities	Query edges based on temporal relationships between associated service object types

with at least three payment activities"); Quantifying service object types (e.g., "orders with more than ten items"); Specifying frequency of temporal service relationships (e.g., "service shipment followed by delivery at least twice for SLA monitoring").
- **Temporal relationship expressiveness:** The language must express both direct temporal succession (directly-follows) and indirect succession (eventually-follows) service relationships. This distinction is crucial for analyzing complex service flows where intermediate service activities may occur between key service steps, particularly in microservice orchestrations.
- **Query composition:** Service-level requirements typically involve combinations of simpler service patterns. The language must support logical operators (AND, OR, NOT) to compose queries across all dimensions into meaningful service patterns for comprehensive service monitoring and optimization.

Performance Considerations for Service Analysis. Object-centric service event logs are typically larger and more complex than traditional service logs, imposing significant performance challenges for service monitoring and analysis. We identify three critical performance requirements:

- **Service data scalability:** The language implementation must scale to handle real-world OCELs with thousands of service process executions, each containing numerous service events and service objects across microservice architectures.
- **Query optimization:** Evaluation strategies must prioritize less computationally expensive operations for efficient service monitoring. For example, filtering by service object type cardinality before checking temporal service patterns can significantly reduce evaluation time, enabling near real-time service analysis.
- **Interactive service response time:** For practical service monitoring, query evaluation should provide results within reasonable timeframes for real-time monitoring needs, depending on service dataset size and query complexity.

5 GOProQ: Graphical Query Language for Object-Centric Service Process Querying

To address the requirements established in Sect. 4, we propose a graphical query language, GOProQ, specifically designed for service-oriented environments. In the following, we introduce the formal syntax and semantics of GoProQ and illustrate its application through concrete service monitoring examples.

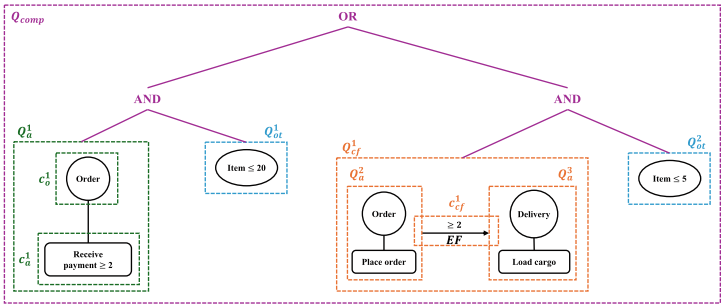


Fig. 2. An example graph query demonstrates the composition of multiple query types to find service process executions with either: (1) an order with fewer than 20 items that have been paid for twice, monitoring potential duplicate payment services, or (2) an order that was loaded into two delivery services when the number of items is fewer than 5, indicating possible inefficient service resource allocation. Querying orders with multiple payment services can reveal duplicate efforts in billing services, while identifying orders loaded into multiple cargo services with few items can highlight optimization opportunities in logistics service planning and service-level agreement compliance.

5.1 GOProQ: Syntax for Service Process Querying

GOProQ is structured around three fundamental query types essential for service analysis: *activity query*, *object type query*, and *control-flow query*.

Activity Query. An activity query targets specific service activities within service process executions, checking for their presence, start, or end. Formally, an activity query is defined as:

$$Q_a = (c_o, c_a), \text{ where}$$

– c_o is the service object type component:

$$c_o = (ot) \quad \text{(single service object type), or}$$
$$c_o = (ot, \odot, n) \quad \text{(service object cardinality)}$$

where $ot \in \mathcal{OT} \cup \{\texttt{ANY}\}$ is a service object type, and $n \in \mathbb{N}_0$ is a count of service objects.

– c_a is the service activity component, which can be:

$$c_a = (l_1) \quad \text{(single service activity)},$$
$$c_a = (\texttt{S}, \{l_1, \ldots, l_n\}) \quad \text{(start service activity)},$$
$$c_a = (\texttt{E}, \{l_1, \ldots, l_n\}) \quad \text{(end service activity)},$$
$$c_a = (\Delta, \{l_1, \ldots, l_n\}) \quad \text{(quantified service activities), or}$$
$$c_a = (l_1, \odot, n) \quad \text{(service activity cardinality)}$$

where $l_1, \ldots, l_n \in \mathcal{A}$ are service activity labels, $\Delta \in \{\texttt{ANY}, \texttt{ALL}\}$ is a service quantifier, $\odot \in \{\geq, \leq, =\}$ is a comparison operator, and $n \in \mathbb{N}_0$ specifies the service activity count for SLA verification.

In Fig. 2, an activity query $Q_a^1 = (c_o^1, c_a^1)$ with $c_o^1 = (\texttt{Order})$ and $c_a^1 = (\texttt{Receive payment}, \geq, 2)$. This checks for the presence of at least 2 occurrences of the payment service activity `Receive payment` for at least one object of type *Order*, helping to identify potential duplicate payment service executions.

Object Type Query. Object type query checks for the existence of service objects of a specified type within the service process execution, enabling monitoring of service resource allocation:

$$Q_{ot} = (c_{ot}), \text{ where}$$

$$c_{ot} = (ot) \quad \text{(simple service object type), or}$$
$$c_{ot} = (ot, \odot, n) \quad \text{(service object type with cardinality)}$$

with $ot \in \mathcal{OT}$ representing service object types, and $n \in \mathbb{N}_0$ representing the service object count for capacity planning.

In Fig. 2, a service object query $Q_{ot}^1 = (\texttt{Item}, \leq, 20)$. This checks whether there are not more than 20 service objects of type `Item` in the service process execution, providing insights for service capacity planning and resource allocation.

Control-Flow Query. A control-flow query Q_{cf} captures temporal and cardinality relationships between service activities in a service process execution, crucial for service orchestration analysis. It is formally defined as:

$$Q_{cf} = (Q_1, Q_2, T, c_{cf}), \text{ where}$$

– Q_1 and Q_2 are activity queries that define the service activities involved in the service control-flow relationship.
– $T \in \{\texttt{DF}, \texttt{EF}\}$ specifies the type of temporal service relationship:
  - `DF` (Directly-Follows): The service activity specified in Q_1 must be directly followed by the service activity specified in Q_2, representing immediate service dependencies.

  - `EF` (Eventually-Follows): The service activity specified in Q_1 must eventually be followed by the service activity specified in Q_2, representing service choreographies where intermediate services may execute.
- c_{cf} is a service constraint component, which can be:
  - $(\odot_1, n_1)$: A service object cardinality constraint, where $\odot_1 \in \{\leq, \geq, =\}$ is a comparison operator, and $n_1 \in \mathbb{N}_0$ denotes the number of service objects in the process execution that satisfy the control-flow relationship.
  - $(\odot_2, n_2)$: A service relationship cardinality constraint, where $\odot_2 \in \{\leq, \geq, =\}$ is a comparison operator, and $n_2 \in \mathbb{N}_0$ specifies the frequency of the temporal relationship for each service object, useful for SLA monitoring.
  - $(\odot_1, n_1, \odot_2, n_2)$: A combined service constraint that specifies both the service object cardinality $(\odot_1, n_1)$ and the service relationship cardinality $(\odot_2, n_2)$.

In Fig. 2, $Q^1_{cf} = (Q^2_a, Q^3_a, \texttt{EF}, c^1_{cf})$ with activity query $Q^2_a = ((\texttt{Order}), (\texttt{Place order}))$, activity query $((\texttt{Delivery}), (\texttt{Load cargo}))$, $c^1_{cf} = (\geq_1, 2)$. This checks if, in the service process execution, the service activity `Place order` eventually follows the service activity `Load cargo` for at least two order-delivery service pairs, helping to identify unusual service orchestration patterns.

Query Composition. To express more complex service requirements, queries can be composed using logical operators such as conjunction (`AND`), disjunction (`OR`), and negation (`NOT`) to create comprehensive service monitoring patterns. For instance, Fig. 2 illustrates a composed query Q_{comp}.

The composed query illustrated in Fig. 2 demonstrates the advantages of our graphical approach over existing query languages for object-centric service analysis. This query involves multiple object types with cardinality constraints, activity frequency requirements, and temporal relationships—patterns that are challenging to express in existing approaches. In BPSPARQL, this query would require complex nested subqueries with multiple GROUP BY clauses for cardinality counting, explicit temporal logic for the eventually-follows relationship, and intricate FILTER expressions to handle multi-object constraints. Using Cypher, the query would need multiple MATCH clauses with separate aggregation steps for each object type, manual temporal relationship handling, and complex WHERE conditions. In contrast, our graphical approach captures the same semantics through intuitive visual composition of four query components (two activity queries, one object type query, and one control-flow query) connected by logical operators.

5.2 GOProQ: Formal Semantics for Service Analysis

The semantics of the query language are defined using the evaluation function $\Phi : \mathbb{U}_Q \times \mathbb{U}_P \rightarrow \{true, false\}$, where $\mathbb{U}_Q$ is the universe of queries and $\mathbb{U}_P$ is the universe of service process executions.

Activity Query Semantics. The evaluation function for an activity query $Q_a = (c_o, c_a)$ is defined as:

$$\Psi(Q_a, p) \iff \Psi_1(c_o, O_p^{ot}) \wedge \Psi_2(c_a, O_p^{ot}), \text{ where}$$

- O_p^{ot} is the set of service objects of type ot in the service process execution p, i.e., $O_p^{ot} = \{o \in O_p \mid \pi_{type}(o) = ot\}$, where O_p is the set of service objects in p.
- $\Psi_1(c_o, O_p^{ot})$ ensures that the specified number of service objects of type ot exist in the service process execution. Formally, given $c_o = (ot, \odot, n)$,

$$\Psi_1(c_o, O_p^{ot}) \iff \left|O_p^{ot}\right| \odot n$$

  If $c_o = (ot)$, we assume $\odot = \geq$ and $n = 1$.
- $\Psi_2(c_a, O_p^{ot})$ verifies whether the specified service activity or set of service activities satisfies the constraints in the trace of each service object $o \in O_p^{ot}$. Formally,

$$\Psi_2(c_a, O_p^{ot}) \iff \forall o \in O_p^{ot}\ [\delta(o, c_a)], \text{ where}$$

  $\delta(o, c_a)$ evaluates the service activity condition for the trace of service object o as:

$$\delta(o, c_a) \Leftrightarrow \begin{cases} \exists e \in E(o)\ [\pi_{act}(e) = l_1], & \text{if } c_a = (l_1) \\ \pi_{trace}(o)(1) \in \{l_1, \ldots, l_n\}, & \text{if } c_a = (\mathtt{S}, \{l_1, \ldots, l_n\}) \\ \pi_{trace}(o)(|\pi_{trace}(o)|) \in \{l_1, \ldots, l_n\}, & \text{if } c_a = (\mathtt{E}, \{l_1, \ldots, l_n\}) \\ \Delta = \mathtt{ANY} \Rightarrow \exists l_i \in \{l_1, \ldots, l_n\}, \exists e \in E(o)\ [\pi_{act}(e) = l_i], & \text{if } c_a = (\Delta, \{l_1, \ldots, l_n\}) \\ \Delta = \mathtt{ALL} \Rightarrow \forall l_i \in \{l_1, \ldots, l_n\}, \exists e \in E(o)\ [\pi_{act}(e) = l_i], & \text{if } c_a = (\Delta, \{l_1, \ldots, l_n\}) \\ |\{e \in E(o) \mid \pi_{act}(e) = l_1\}| \odot n, & \text{if } c_a = (l_1, \odot, n) \end{cases}$$

  where $E(o)$ is the set of service events associated with service object o and $\pi_{trace}(o)$ represents the sequence of service activities associated with a service object o, e.g., $\pi_{trace}(o_1) = \langle \mathtt{Place\ Order}, \ldots, \mathtt{Receive\ Payment} \rangle$.

Consider the activity query $Q_a^1 = (c_o^1, c_a^1)$ in Fig. 2, where $c_o^1 = (\mathtt{Order})$ and $c_a^1 = (\mathtt{Receive\ payment}, \geq, 2)$. This query checks whether there are at least two occurrences of the payment service activity `Receive payment` for service objects of type `Order`, potentially indicating duplicate payment service executions. For the service process execution p_1 in Fig. 1: (i) $O_{p_1}^{\mathtt{Order}} = \{o_1\}$, (ii) $\Psi_1(c_o^1, p_1)$: $\left|\{o_1\}\right| \geq 1$ is true, and (iii) $\Psi_2(c_a^1, p_1)$: For service object o_1, the payment service activity `Receive payment` occurs in service events e_{10} and e_{11}; Thus, $\left|\{e_{10}, e_{11}\}\right| \geq 2$ holds. Hence, Q_a^1 evaluates to *true* for p_1, indicating potential duplicate payment processing.

Object Type Query Semantics. The evaluation function for an object type query $Q_{ot} = (ot, \odot, n)$ checks whether the service process execution p satisfies the service object type and cardinality constraints specified in Q_{ot}, enabling service capacity monitoring. Formally,

$$\Psi(Q_{ot}, p) \iff \left|O_p^{ot}\right| \odot n, \text{ where}$$

O_p^{ot} is the set of service objects of type ot in the service process execution p.

Consider the object type query $Q_{ot}^1 = (\texttt{Item}, \leq, 20)$ in Fig. 2. This query checks whether there are fewer than 20 service objects of type `Item`, important for service capacity planning. For the service process execution p_1 in Fig. 1: (i) $O_p^{\texttt{Item}} = \{i_1, i_2, i_3\}$ and (ii) $\big|\{i_1, i_2, i_3\}\big| \leq 20$. Hence, Q_{ot}^1 evaluates to *true* for p_1, indicating compliance with capacity constraints.

Control-Flow Query Semantics. The evaluation function for a control-flow query $Q_{cf} = (Q_1, Q_2, T, c_{cf})$ evaluates if a service process execution satisfies (i) the activity queries Q_1 and Q_2 and (ii) the temporal service constraint T with the service cardinality constraints c_{cf}, essential for service orchestration monitoring. Formally,

$$\Psi(Q_{cf}, p) \iff \Psi(Q_1, p) \wedge \Psi(Q_2, p) \wedge \Omega(\Lambda(T, p), c_{cf}), \text{ where}$$

– the function $\Lambda(T, p)$ produces a mapping of service object pairs (o_i, o_j) to the corresponding service event pairs (e_i, e_j) that satisfy the temporal service relationship T:

$$\Lambda(T, p) = \{((o_i, o_j), \{(e_i, e_j) \mid e_i \in E(o_i, l_1), e_j \in E(o_j, l_2), \\ \pi_T(e_i, e_j)\}) \mid o_i \in O_{ot_1}, o_j \in O_{ot_2}\}, \text{ where}$$

- l_1 and l_2 are the service activity labels, and ot_1 and ot_2 are the service object types, specified in Q_1 and Q_2,
- O_p^{ot} represents the set of service objects of type ot in the service process execution p,
- $E(o, a)$ is the set of service events associated with service object o and labeled with service activity a, and
- the temporal service condition $\pi_T(e_i, e_j)$ is defined as follows:

$$\pi_T(e_i, e_j) \iff \begin{cases} e_i \prec_L e_j, & \text{if } T = \texttt{DF}, \\ \exists e_1, \ldots, e_n \in E,\ n \geq 2, & \\ \quad \big[e_1 = e_i,\ e_n = e_j,\ \forall k \in [1, n-1] : (e_k, e_{k+1}) \in \prec_L\big], & \text{if } T = \texttt{EF}. \end{cases}$$

– the function Ω evaluates the extracted service event pairs against the specified service constraints c_{cf}:

$$\Omega(\Lambda(T, p), c_{cf}) \iff \begin{cases} \big|\text{dom}(\Lambda(T, p))\big| \ \odot_1 \ n_1, & \text{if } c_{cf} = (\odot_1, n_1), \\ \min\big(\{\big|\Lambda(T, p)((o_i, o_j))\big| \mid \\ (o_i, o_j) \in \text{dom}(\Lambda(T, p))\}\big) \ \odot_2 \ n_2, & \text{if } c_{cf} = (\odot_2, n_2), \\ \Omega(\Lambda(T, p), (\odot_1, n_1)) \wedge \\ \Omega(\Lambda(T, p), (\odot_2, n_2)), & \text{if } c_{cf} = (\odot_1, n_1, \odot_2, n_2). \end{cases}$$

Consider the control-flow query $Q_{cf}^1 = (Q_a^2, Q_a^3, \texttt{DF}, c_{cf}^1)$ in Fig. 2 where $Q_a^2 = ((\texttt{Order}), (\texttt{Place order}))$, $Q_a^3 = ((\texttt{Delivery}), (\texttt{Load cargo}))$, and $c_{cf}^1 = (\geq_1, 2)$. This query checks whether the service activity `Place order` directly

follows the service activity `Load cargo` for at least two order-delivery service pairs, revealing potential service orchestration anomalies. For the service execution p_1 in Fig. 1: (i) $\Psi(Q_a^2, p_1)$: *true*, as service object o_1 performs the service activity `Place order`, (ii) $\Psi(Q_a^3, p_1)$: *true*, as service objects d_1 and d_2 perform the service activity `Load cargo`, and (iii) $\Lambda(\texttt{DF}, p_1)$: Service event pair (e_1, e_6) satisfies the `Directly-Follows` service condition for o_1 and d_1, and similarly for d_2. Since the service cardinality constraint $c_{cf}^1 = (\geq_1, 2)$ is met, Q_{cf}^1 evaluates to *true* for p_1, indicating potential service orchestration patterns requiring optimization.

Query Composition Semantics. Formally, for two queries $Q_1, Q_2 \in \mathbb{U}_Q$, the composed query semantics are defined as:

$$\Psi(Q_{comp}, p) \iff \begin{cases} \Psi(Q_1, p) \wedge \Psi(Q_2, p), & \text{if } Q_{comp} = (Q_1 \wedge Q_2) \\ \Psi(Q_1, p) \vee \Psi(Q_2, p), & \text{if } Q_{comp} = (Q_1 \vee Q_2) \\ \neg\Psi(Q_1, p), & \text{if } Q_{comp} = \neg Q_1 \end{cases}$$

Consider the composed query Q_{comp} in Fig. 2, which combines multiple service patterns to identify either duplicate payment services or inefficient logistics service allocation:

$$Q_{comp} = (Q_a^1 \wedge Q_{ot}^1) \vee (Q_{cf}^1 \wedge Q_{ot}^2)$$

This is evaluated as follows:

$$\Psi(Q_{comp}, p_1) \iff (\Psi(Q_a^1, p_1) \wedge \Psi(Q_{ot}^1, p_1)) \vee (\Psi(Q_{cf}^1, p_1) \wedge \Psi(Q_{ot}^2, p_1))$$

$$\Psi(Q_{comp}, p_1) \iff (true \wedge true) \vee (true \wedge true)$$

$$\Psi(Q_{comp}, p_1) \iff true$$

This evaluation confirms that the service process execution exhibits both potential duplicate payment services and inefficient logistics service resource allocation, highlighting opportunities for service optimization.

6 Evaluation

To evaluate the proposed query language for service-oriented environments, we implemented a graphical user interface tool for querying service-based OCELs. Users can import service OCEL files, compose queries, and visualize the resulting service process executions. The source code and user manual of the tool are publicly available at https://github.com/gyunamister/GOProQ (Fig. 3).

This section presents a comprehensive evaluation of our query language, focusing on how it addresses the service-oriented requirements established in Sect. 4. We evaluate both the expressiveness of the language for service analysis through our implementation in the GOProQ tool and assess its performance feasibility for service monitoring using a series of experimental evaluations across diverse service datasets (Table 3).

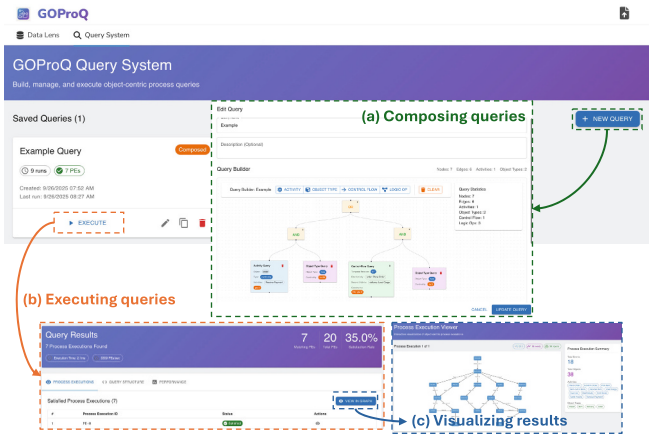


Fig. 3. A screenshot of the tool for object-centric process querying.

Table 3. Overview of service-oriented datasets

Service OCEL	#Service Events	#Service Executions[a]	#Service Objects	Service Object Types
Loan Application (Large)	392,018	31,305	74,139	application, offer
Loan Application (Small)	64,770	7,198	14,396	application, offer
Recruiting[d]	6,980	1,289	15,05	applications, offers
Travel Reimbursement[e]	18,246	375	1,761	Task, permit id
Procure-To-Payment[d]	24,854	16,822	74,489	MATNR, MBLNR
Order-To-Cash[d]	239,391	48	796	item, order, delivery

6.1 Evaluation of Expressiveness Requirements

The GOProQ tool implementation demonstrates that our query language satisfies the expressiveness requirements for service analysis established in Sect. 4. Specifically:

- **Support for all query dimensions:** The tool successfully implements query constructs that cover all cells in the query dimensions matrix (Table 2). It supports activity queries, object type queries, and temporal service relationship queries between both service activities and service object types, providing comprehensive coverage of the defined service dimensions.
- **Service-level cardinality constraints:** The implementation supports service cardinality constraints across all dimensions, allowing users to specify service monitoring conditions such as "service processes with at least three payment service activities" or "orders with more than ten items", enabling effective SLA monitoring.
- **Temporal relationship expressiveness:** Both Directly-Follows (DF) and Eventually-Follows (EF) service relationships are implemented in the tool, enabling service analysts to distinguish between immediate service succession and indirect temporal service relationships in complex microservice orchestrations.

– **Query composition:** The graphical interface of GOProQ allows users to visually compose queries using logical operators (AND, OR, NOT), enabling the construction of complex service patterns that combine multiple simpler service conditions for comprehensive service monitoring and analysis.

6.2 Evaluation of Performance Requirements

Experimental Setup. We conducted experiments using the six service-oriented OCEL datasets. Each dataset represents different service environments, from financial services (Loan Application) to logistics services (Order-To-Cash). The hardware specifications are as follows: Intel Core i5-12600K processor (10 cores, 16 threads, base frequency 3.7 GHz, turbo frequency 4.9 GHz), 32 GB DDR4 RAM (3600 MHz), and Samsung SSD 980 Pro (1 TB, sequential write speed 7,000 MB/s, read speed 5,000 MB/s).

Computation Time of Individual Queries. To evaluate the performance requirements for service monitoring established in Sect. 4, we first assess the computation times of individual query types. Table 4 presents the results for object type, activity, and control-flow queries. Object type queries demonstrate the shortest computation times, as they operate on individual service objects without considering temporal service relationships. Activity queries, such as *Presence*, *Start*, and *End*, exhibit slightly higher computation times but remain computationally efficient, with variations averaging only 1.3%. Control-flow queries, particularly *Eventually-Follows* (EF), have the longest computation times due to the need to explore paths between service events, which is critical for complex service orchestration analysis. In the Travel Reimbursement and Procure-To-Payment datasets, EF queries could not be computed within a reasonable time frame for real-time service monitoring. This limitation is attributed to the presence of service process executions with thousands of nodes and edges (e.g., 8,481 service nodes and 8,611 service dependencies in Travel Reimbursement).

Table 4. Computation time for different query types across datasets

Service Dataset	Service Object Type	Presence	Start/End	DF	EF
Loan Application (Large)	0.2261 s	12.4943 s	12.6322 s	16.6797 s	20.7484 s
Loan Application (Small)	0.0102 s	0.5119 s	0.5023 s	0.7624 s	1.2207 s
Recruiting	0.0077 s	0.0159 s	0.0155 s	0.0384 s	0.0485 s
Travel Reimbursement	0.0023 s	0.1575 s	0.1600 s	0.9523 s	–
Procure-To-Payment	0.0151 s	0.2380 s	0.2304 s	1.5349 s	–
Order-To-Cash	0.0009 s	0.0884 s	0.0918 s	0.0892 s	0.4315 s

These results directly address the service data scalability and service response time requirements identified in Sect. 4. While most queries execute efficiently

across the test datasets, we identify clear performance boundaries, particularly for complex temporal queries on large service process executions, which has implications for real-time service monitoring capabilities.

Reordering Strategy. To address the query optimization requirement from Sect. 4, we developed a reordering strategy based on the insights from individual query evaluations. The strategy prioritizes the evaluation of less computationally intensive queries to filter non-matching service process executions early, thereby reducing the overall computation time for effective service monitoring. The reordering strategy comprises the following steps:

- Evaluate individual queries first:
  - Execute object type queries first, as they are the least computationally expensive for quick service filtering.
  - Execute activity queries next, prioritizing those with fewer associated service objects (e.g., *Item* before *Order*). Queries involving `ANY` are executed last as they are less restrictive in service filtering.
  - Finally, execute control-flow queries, with `DF` queries preceding `EF` queries due to their stricter temporal constraints and lower computational cost.
- Handle composed queries with logical operators:
  - Execute `NOT` queries first to exclude non-matching service executions early.
  - Execute `AND` queries next, as all sub-queries must be satisfied for compliance.
  - Execute `OR` queries last, as they require evaluating all possible sub-queries.

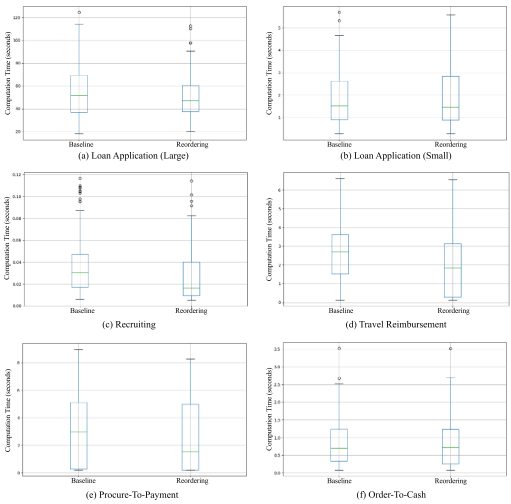


Fig. 4. Query computation times of the baseline and reordering approaches, showing improved efficiency for service monitoring through optimization.

Figure 4 compares the reordering strategy with a baseline approach that evaluates queries in random order. The reordering strategy consistently reduces computation times, highlighting its effectiveness for responsive service monitoring. For instance, the median computation time for the recruiting service dataset is reduced by 57.1%, enabling more responsive service analysis. Using the reordering strategy, the queries on all datasets except Loan Application (Large) take less than 3 s. This demonstrates the practical feasibility of the query language for most real-life service monitoring scenarios, directly addressing the interactive service response time requirement from Sect. 4.

7 Conclusion

In this paper, we introduced GOProQ. Our approach enables expressive queries on activities, object types, and their relationships within service-oriented architectures. We implemented the query language as an interactive tool and evaluated its feasibility and performance for service analysis using six publicly available service OCEL datasets. The results demonstrate the practical usability of the language for service monitoring and orchestration analysis, with computation times that are manageable for most service scenarios.

The approach has limitations in the context of service-oriented computing. First, direct service-to-service relationships not mediated through activities were not explored. While GOProQ supports querying shared service events, user-defined service relationships such as directly-follows or eventually-follows between service objects remain unexplored, which could be valuable for complex microservice dependency analysis. Second, the computation time for complex queries remains a challenge, particularly for large-scale service environments with extensive process executions, where query response times may exceed practical limits for real-time service monitoring. Third, while our graphical approach reduces cognitive load for typical service monitoring queries, very complex multi-layer queries can become cognitively demanding.

References

1. van der Aalst, W.M.P.: Process Mining - Data Science in Action. Springer (2016)
2. Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J.L., Vrgoc, D.: Foundations of modern query languages for graph databases. ACM Comput. Surv. **50**(5), 68:1–68:40 (2017)
3. Beheshti, A., Benatallah, B., Motahari-Nezhad, H.R., Ghodratnama, S., Amouzgar, F.: BP-SPARQL: a query language for summarizing and analyzing big process data. In: Polyvyanyy, A. (ed.) Process Querying Methods, pp. 21–48. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-92875-9_2
4. Cimatti, A., et al.: NuSMV 2: an OpenSource tool for symbolic model checking. In: Brinksma, E., Larsen, K.G. (eds.) CAV 2002. LNCS, vol. 2404, pp. 359–364. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45657-0_29
5. Clarke, E.M.: Model checking - my 27-year quest to overcome the state explosion problem. In: Cervesato, I., Veith, H., Voronkov, A. (eds.) LPAR, vol. 5330, p. 182 (2008)

6. Clarke, E.M., Grumberg, O., Kroening, D., Peled, D.A., Veith, H.: Model checking (2018)
7. Esser, S., Fahland, D.: Multi-dimensional event data in graph databases. J. Data Semant. **10**(1–2), 109–141 (2021)
8. Fahland, D.: Process mining over multiple behavioral dimensions with event knowledge graphs. In: van der Aalst, W.M.P., Carmona, J. (eds.) Process Mining Handbook, vol. 448, pp. 274–319. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-08848-3_9
9. Hildebrandt, T., Mukkamala, R.R., Slaats, T.: Nested dynamic condition response graphs. In: Arbab, F., Sirjani, M. (eds.) FSEN 2011. LNCS, vol. 7141, pp. 343–350. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-29320-7_23
10. Holzmann, G.J.: The SPIN model checker - primer and reference manual (2004)
11. Knuplesch, D., Reichert, M., Kumar, A.: A framework for visually monitoring business process compliance. Inf. Syst. **64**, 381–409 (2017)
12. Lin, J., Chen, P., Zheng, Z.: Microscope: pinpoint performance issues with causal graphs in micro-service environments. In: Pahl, C., Vukovic, M., Yin, J., Yu, Q. (eds.) ICSOC 2018. LNCS, vol. 11236, pp. 3–20. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-03596-9_1
13. Ly, L.T., Rinderle-Ma, S., Dadam, P.: Design and verification of instantiable compliance rule graphs in process-aware information systems. In: Pernici, B. (ed.) CAiSE 2010. LNCS, vol. 6051, pp. 9–23. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-13094-6_3
14. de Murillas, E.G.L., Reijers, H.A., van der Aalst, W.M.P.: Data-aware process oriented query language. In: Polyvyanyy, A. (ed.) Process Querying Methods, pp. 49–83. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-92875-9_3
15. Pérez-Álvarez, J.M., Díaz, A.C., Parody, L., Quintero, A.M.R., Gómez-López, M.T.: Process instance query language and the process querying framework. In: Polyvyanyy, A. (ed.) Process Querying Methods, pp. 85–111. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-92875-9_4
16. Polyvyanyy, A. (ed.): Process Querying Methods. Springer, Cham (2022)
17. Schuster, D., Martini, M., van Zelst, S.J., van der Aalst, W.M.P.: Control-flow-based querying of process executions from partially ordered event data. In: Troya, J., Medjahed, B., Piattini, M., Yao, L., Fernández, P., Ruiz-Cortés, A. (eds.) ICSOC 2022. LNCS, vol. 13740, pp. 19–35. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20984-0_2
18. da Silva, E.A., do Nascimento Fidalgo, R., Ferro, M., Franco, N.: Visual query languages to design complex queries: a systematic literature review. Softw. Syst. Model. **22**(4), 1217–1249 (2023)

BPMN4BC: A Simple BPMN Extension for Blockchain-Enabled Business Process Modeling

Hassan Atwi(✉) and Cesare Pautasso

Software Institute, Università della Svizzera italiana, Lugano, Switzerland
{Hassan.Atwi,Cesare.Pautasso}@usi.ch

Abstract. Blockchain technology has recently gained significant traction to support collaborative processes due to its ability to enable decentralization in environments that do not require a central authority. While there are numerous model-to-code approaches to facilitate the deployment and execution of processes on the blockchain, few have addressed the critical aspect of explicit modeling and designing blockchain-enabled processes. This paper introduces BPMN4BC, an extended set of notations designed to represent blockchain properties in process diagrams. These notations help distinguish on-chain from off-chain logic, preventing their overlap and accurately representing blockchain transactions. BPMN4BC enables business users and domain experts to capture blockchain-specific properties within their collaborative process models explicitly. We also introduce the BPMN4BC compiler that generates and deploys the corresponding Solidity code. To assess the consequences of different modeling choices, we compare the execution cost associated with two alternative deployment strategies, thereby demonstrating how modeling decisions influence blockchain properties such as gas usage.

Keywords: Blockchain · BPMN · Model-to-code · Collaborative processes · Decentralization · Ethereum · Smart contract

1 Introduction

Blockchain technology has gained significant interest in Business Process Management (BPM), particularly with the introduction of smart contracts in blockchain 2.0 [8]. Integrating smart contract technology into the lifecycle of business process management makes an effective tool for implementing and executing collaborative processes which do not rely on a physical central authority as it provides a neutral and logically-centralized place to track the business process execution, mitigating trust issues and enabling seamless collaboration.

The design and modeling phase is a crucial part of the lifecycle of any business process, as it defines compliance rules for participant interactions. Therefore, accuracy in designing the business process is essential for achieving its business goals. Business Process Model and Notation (BPMN [11]) is a rich and

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 306–320, 2026.
https://doi.org/10.1007/978-981-95-5015-9_22

expressive language for modeling and representing conventional collaborative processes. However, the standard BPMN language is not sufficient to accurately capture the unique properties of processes utilizing blockchain technology [12], as it was developed prior to the emergence of blockchain. In particular, distinguishing between on-chain and off-chain activities and delimiting transaction boundaries [6] is challenging for both business users and workflow engines.

This paper introduces BPMN4BC (BPMN for Blockchain), an extension of BPMN designed to incorporate blockchain aspects into collaborative business processes. BPMN4BC serves as a bridge between business practitioners and blockchain experts, enabling them to model and integrate blockchain technology seamlessly. It achieves this by representing blockchain-specific technical details while maintaining BPMN's standard syntax [11]. The BPMN4BC extension is intended to model blockchain logic within collaborative business processes, ensuring a clear distinction between on-chain and off-chain activities. By integrating blockchain logic into process modeling, transaction boundaries in collaborative workflows become more apparent and easier to identify. Additionally, BPMN4BC facilitates the modeling and management of on-chain data flow within business processes. The practicality and expressiveness of the extension are demonstrated through a realistic use case scenario, in which we see how property registration transaction boundaries are modeled. The notation is fully supported by a compiler[1] which generates Solidity code and deploys it to be executed on the Ethereum blockchain network. We include an experimental evaluation of the gas costs incurred during blockchain execution, providing insights into the impact of the contract granularity design decision on the deployment and execution costs.

The rest of this paper is organized as follows: Sect. 2 introduces the concepts of blockchain-based collaborative processes, and discuss the existing related work. Section 3 presents the extended notations to support blockchain modeling in collaborative processes. Section 4 details transaction boundaries and the handling of process variables in the blockchain context. Section 5 presents a use case that illustrates the expressiveness of the extended notation followed by Sects. 6 and 7 on the implementation and evaluation of our model-to-code approach. Finally, Sect. 8 discusses the results before Sect. 9 recaps the paper and outlines possible future research directions.

2 Background, Motivation, and Related Work

2.1 Challenges of Blockchain-Enabled Collaborative Processes

Blockchain technology provides a foundation for collaborative business processes by leveraging its decentralized, tamper-proof ledger [8]. Multiple participants can collaborate in compliance with a predefined process without the need for a central authority. Existing approaches [5,6,14] take a business process model as input and compile it into executable code deployed on the blockchain. Representing blockchain-enabled processes with BPMN is challenging [9]:

[1] https://github.com/Hassan42/BPMN-4-BC.

- *Ambiguity between on-chain and off-chain components of the process.* In collaborative process diagrams, each participant's activities are represented within their own pool, but it is unclear where blockchain-related tasks are placed. In [6], the authors argue that standard collaboration diagrams fail to accurately represent the positioning of local process execution logic and on-chain business rules, which may lead to misuse of blockchain immutability.
- *Lack of visibility of transaction boundaries in the process.* The model does not indicate where each transaction begins or ends. In case of transaction revert, there is no clear indication of how this is handled within the process.
- *Ambiguous on-chain process state management.* Standard BPMN elements do not clearly specify when during the process execution the blockchain state is updated or which participants are responsible for these updates.
- *Limited representation of blockchain configurations.* Standard BPMN lacks specific notations to represent core blockchain elements, such as transactions, smart contracts, or oracles. For instance, processes involving complex configurations like multiple smart contracts deployed across multiple blockchain networks are difficult to represent in a standard process model.

In this paper we address these limitations as they may hinder the accurate design and precise understanding of blockchain-enabled processes and lead to incorrect or inefficient implementations.

2.2 Design Objective and Methodology

This work adopts the design science methodology as outlined by Wieringa [15] to develop a BPMN extension for blockchain-enabled business processes. The objective is to enhance BPMN's integration with blockchain by introducing visually coherent notations that retain BPMN syntax while capturing blockchain-specific semantics such as transaction context, smart contract logic, and event propagation. The design followed an iterative cycle of problem identification, artifact development, and evaluation through modeling scenarios, allowing continuous refinement for semantic clarity. The target audience includes business practitioners integrating blockchain into their processes, requiring modeling support that clearly separates on-chain and off-chain logic and improves blockchain expressiveness.

2.3 Related Work

Existing works focus on modeling blockchain properties within business processes by introducing custom elements dedicated to representing blockchain interactions in their process diagrams. In [3], the authors propose a BPMN 2.0 modeling extension to represent blockchain interactions. The extension models the end user's perspective, such as the sending and receiving of transactions, without providing detailed context about the blockchain's internal operations or the smart contracts' underlying logic. In [4], the blockchain is represented as a dedicated pool, where participants interact with it through message flows.

The blockchain pool contains business rule tasks that are triggered by incoming message events from external participants. In contrast, [13] represents the blockchain as a swimlane within the organization, containing scripts that are executed directly through sequence flows in the process model. [7] introduces a model-driven engineering (MDE) approach for developing blockchain-based business processes, particularly focusing on asset registration. The authors highlight that existing BPMN 2.0 elements cannot accurately represent smart contract properties. To address this limitation, they propose custom elements designed to model the necessary properties of smart contracts within business processes effectively. In [9], the authors explore the representation of blockchain-specific patterns using BPMN and CMMN modeling languages and compare their effectiveness: while BPMN supports most blockchain-specific patterns, it lacks sufficient capabilities to represent tokenization (Fig. 1).

3 Extended Blockchain Notations for Business Processes

The visual design of BPMN4BC focuses on simplicity, utilizing standard BPMN elements uniformly annotated with a blockchain icon. This consistent visual representation maintains familiarity for BPMN users while explicitly depicting the blockchain-related elements within business processes.

To formalize these new modeling constructs, we define the set of BPMN4BC modeling elements, composed of task elements T and event elements E, i.e., $\mathcal{E} = T \cup E$. We classify elements according to their execution context: on-chain elements $\mathcal{E}_{\text{on}} \subset \mathcal{E}$ represent operations executed on the blockchain and include the BPMN4BC on-chain constructs, along with a limited subset of standard BPMN elements such as start events, end events, and script tasks. These standard elements are not novel extensions, but they are included in $\mathcal{E}_{\text{on}}$ to support a complete and executable representation of on-chain process logic. Off-chain elements $\mathcal{E}_{\text{off}} \subset \mathcal{E}$ represent interactions external to the blockchain and include the BPMN4BC off-chain constructs, and all remaining standard BPMN elements.

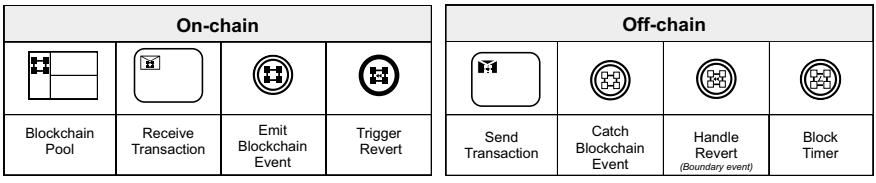


Fig. 1. BPMN for Blockchain Notations.

Blockchain Pool and Smart Contracts. The *blockchain pool* represents a blockchain network on which a process has been deployed. Each *swimlane* within the pool corresponds to a smart contract. This pool ensures a clear separation

between blockchain and off-chain logic. Formally, a blockchain pool is defined as a tuple $B = (\mathcal{S}, T, G, E, F)$, where:

- $\mathcal{S}$ is the set of smart contract swimlanes, which can contain only:
- $\mathcal{E}_{\mathsf{on}} \subseteq T \cup E$, the on-chain task and event elements,
- G is the set of standard BPMN gateways,
- $F \subseteq (T \cup G \cup E)^2$ is the set of sequence flows,

Send and Receive Transaction Tasks. To model blockchain interactions, we refine the standard send/receive tasks into *send transaction* and *receive transaction* tasks. The send transaction task, an off-chain element ($T_s \subseteq \mathcal{E}_{\mathsf{off}}$), constructs and signs a transaction. The receive transaction task, an on-chain element ($T_r \subseteq \mathcal{E}_{\mathsf{on}}$), validates the transaction before executing the logic. These tasks are linked via a message flow, forming an inbound oracle [10]. As blockchains cannot initiate communication, a receive transaction task is the only entry point into a blockchain pool. Consequently, direct message flows from a blockchain pool to any other pool, including other blockchain pools, are not possible. This restriction follows from the fact that only receive transaction tasks are allowed within a blockchain pool, and such tasks cannot be initiated from within the blockchain itself. This reflects the fundamental isolation of blockchain systems, where smart contracts cannot actively send direct messages to external environments but can only react to incoming transactions initiated off chain.

Emit and Catch Blockchain Events. Since blockchains are passive systems, they communicate changes through events. BPMN4BC defines the *emit blockchain event* as an on-chain element ($E_e \subseteq \mathcal{E}_{\mathsf{on}}$) and the *catch blockchain event* as an off-chain element ($E_c \subseteq \mathcal{E}_{\mathsf{off}}$). These events are loosely coupled and follow a publish-subscribe model. When a blockchain process emits an event, it is published to the network and can be observed by any number of off-chain participants. There is no direct or synchronous communication between the blockchain and the listeners. Multiple off-chain processes may subscribe to and react to the same event independently.

Trigger and Handle Revert Events. To capture transaction rollbacks, BPMN4BC introduces the *trigger revert end event* ($E_v \subseteq \mathcal{E}_{\mathsf{on}}$) and the *handle revert boundary event* ($E_h \subseteq \mathcal{E}_{\mathsf{off}}$). Triggering a revert event aborts execution within the blockchain pool and rolls back the blockchain state to the point before the transaction was executed. The boundary event can be attached to a send transaction task to define off-chain compensation.

Block Timer Event. Because blockchain time advances via block production rather than real-time clocks, BPMN4BC introduces the *block timer event*, which delays execution until a specified number of blocks have passed. This event is exclusively off-chain ($E_t \subseteq \mathcal{E}_{\mathsf{off}}$) and is useful for modeling confirmation waits or timeout conditions that align with blockchain behavior.

4 Execution Semantics

4.1 Transaction Execution Context

In BPMN4BC, blockchain execution is structured around transactions. A transaction begins when a smart contract receives a signed message from an external participant, modeled by a *receive transaction task* $t_{\text{start}} \in T_r$. From that point, the process executes in the context of a blockchain transaction, traversing a finite sequence of elements and potentially modifying the blockchain state. The transaction terminates upon reaching a designated exit point:

- a receive transaction task T_r,
- a standard end event or a revert end event, denoted $E_{\text{end}} \subseteq E$,
- an AND gateway, denoted $G_{\wedge} \subseteq G$,
- an event-based gateway, denoted $G_{\text{event}} \subseteq G$.

In the case of an AND merge (join) gateway, if all parallel branches have been synchronized, i.e., the gateway is enabled, the transaction continues beyond the gateway. Otherwise, the transaction terminates at the unsynchronized join. Formally, we define a transaction as a tuple:

$$\tau = (t_{\text{start}}, \sigma, \pi, \sigma', n_{\text{end}})$$

where:

- $t_{\text{start}} \in T_r$ is the initiating receive transaction task,
- $n_{\text{end}} \in T_r \cup E_v \cup E_{\text{end}} \cup G_{\wedge} \cup G_{\text{event}}$ is the terminating node,
- $\sigma, \sigma' \in \Sigma$ are the blockchain states, before and after execution,
- $\pi = \langle n_1, \ldots, n_k \rangle \in (T \cup G \cup E)^*$ is the execution path such that:

$$(t_{\text{start}}, n_1),\ (n_1, n_2),\ \ldots,\ (n_{k-1}, n_k),\ (n_k, n_{\text{end}}) \in F$$

- Executing π yields a state transition $(\sigma, \pi, \sigma') \in \Sigma \times (T \cup G \cup E)^* \times \Sigma$,

The definition of a blockchain transaction in BPMN4BC enables reasoning about runtime behaviors. We highlight three key semantics that distinguish blockchain-aware execution from standard BPMN.

Deferred Event Emission. Unlike traditional BPMN signal events, blockchain events emitted during transaction execution are not dispatched immediately. Instead, they are deferred until the transaction concludes successfully. During the execution path π of some transaction τ, all *emit blockchain event* elements encountered are queued. These events remain pending and are collectively published only after the transaction is mined and finalized, that is, when $n_{\text{end}} \neq E_v$. This deferred emission model preserves atomicity and ensures that no partial effects are externally observable before the transaction is confirmed. Off-chain participants listening for blockchain events (via *catch blockchain event*s) can only react once the entire transaction has completed successfully. Since blockchain events follow a publish-subscribe mechanism, all queued events are broadcast to the network upon commit and may be independently consumed by any number of off-chain processes. Process modelers must account for this asynchronous behavior, which prevents real-time interaction during transaction execution.

Transaction Revert and Compensation Handling. If an error occurs during execution that leads to a *trigger revert end event*, the transaction is reverted. In such cases, the blockchain state remains unchanged ($\sigma' = \sigma$), and control is rolled back to the transaction's entry point $t_{\text{start}} \in T_r$. Additionally, the originating *send transaction task* is notified of the failure via a corresponding *handle revert boundary event* in the off-chain process. Reverts may be explicitly modeled using a *trigger revert end event*, or they may arise implicitly from any uncaught or undefined error condition within the transaction scope. By default, any runtime error not explicitly handled in the process model will raise a revert event and cause a rollback. This reinforces the transactional nature of blockchain execution: compensation is not incremental or partial, but rather an all-or-nothing commitment model that the process modeler must be aware of when designing blockchain-aware workflows.

Parallel Execution and Gateway Semantics. It is valid for a split AND gateway to branch into multiple *receive transaction tasks*, as this reflects the decentralized nature of transaction initiation in blockchain-based processes. Each branch corresponds to a potential transaction initiated by a separate participant. Since blockchain consensus protocols are inherently non-deterministic in ordering, the order in which these transactions are received and mined is determined by the network's miners and not by the process model [1]. As a result, the branches are executed in an arbitrary order, and each transaction proceeds independently from its corresponding *receive transaction task*. It is important to note that only one transaction executes at a time due to the sequential nature of blockchain transaction processing. However, within the execution path π of any transaction τ, split AND gateways are not permitted if they lead to more than one outgoing sequence flow that targets elements in $\mathcal{E}_{\mathsf{on}} \setminus T_r$. Formally, let $g \in G_{\wedge}^{\text{split}}$ be a split AND gateway, and let $\text{out}(g) \subseteq T \cup E$ be the set of its immediate successors. Then the following must hold:

$$\text{if } g \in \pi, \text{ then } |\text{out}(g) \cap (\mathcal{E}_{\mathsf{on}} \setminus T_r)| \leq 1$$

This constraint ensures that transactions do not diverge into parallel branches, preserving deterministic and atomic execution, as blockchain runtimes do not support concurrent on-chain execution within a single transaction.

4.2 Process Variables

In BPMN4BC, process variables represent data relevant to the execution and state of a blockchain-aware business process. These variables can be initialized by either a *receive transaction task* ($t \in T_r$) or a *script task* during the execution of a transaction. Based on its lifetime and storage semantics, a process variable is classified as either a *memory variable* or a *persistent variable*.

Persistent Variables. are stored on the blockchain and remain available after the transaction in which they were created. A persistent variable can be written

from the smart contract swimlane S where it was defined and is readable from any swimlane in the blockchain pool. This reflects typical access control in smart contract design. Persistent variables are ideal for long-lived, verifiable data such as ownership records or stateful contract variables.

Memory Variables. are volatile and exist only for the duration of the transaction execution path π in which they are initialized. Once the transaction terminates, regardless of success or revert, these variables are discarded, and their values are no longer accessible. Because they are not stored on-chain, memory variables are cost-efficient and suitable for computations that do not require persistence. Any swimlane participating in the transaction may read or write memory variables during the transaction's execution.

5 Use Case Scenario

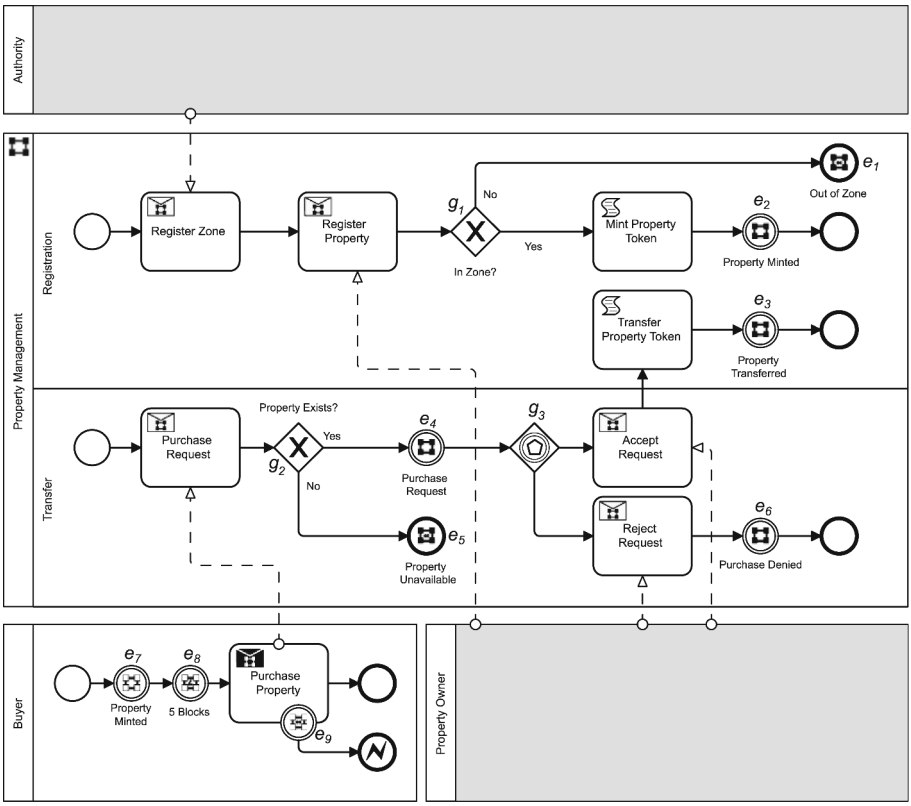


Fig. 2. Blockchain-enabled property registration process use case.

This section presents a blockchain-based property registration use case modeled with a BPMN4BC collaboration diagram (Fig. 2) illustrating the scenario with one pool for the blockchain network and three for the participants' local off-chain processes. The process involves registering and transferring property ownership through blockchain interaction. We show transaction boundaries and process variable management using an execution log (Table 1).

In the blockchain pool the business concerns are separated into two swimlanes, each corresponding to a smart contract: one for registration and the other for property transfer. In the registration smart contract, the authority submits a transaction to define the permissible area for property registration by specifying a zone. Following the zone registration, the property owner submits a transaction to register their property. At this point, an XOR gateway (g_1) represents the two verification outcomes: whether or not the property falls within the correct zone. If the property is within the zone, it is registered. Otherwise, the transaction is reverted, and the process state is rolled back to the point before the property registration transaction started. In the transfer smart contract, the buyer submits a transaction specifying the property to be purchased. The XOR gateway (g_2) checks the property's availability. If the requested property is available, the process waits for the original owner to accept or reject the request. Upon acceptance, the ownership of the property is transferred to the buyer, otherwise the purchase is denied.

In the buyer pool, off-chain elements are used to describe the buyer's local process. The process listens for the "property minted" event (e_7) emitted by the registration contract. Upon successful property registration, the buyer process waits for five blocks (e_8) to pass before sending a purchase request. A handle revert boundary event (e_9) is triggered if the transaction is reverted, such as when the property is unavailable.

The use case process diagram (Fig. 2) reveals five transaction entry points and six transaction exit points. One such entry point is the purchase request task, which begins a transaction when invoked by the buyer. From this point, the transaction proceeds along a control-flow path π until reaching an exit point. Due to the XOR gateway g_2, the transaction has two alternative execution paths. If the property is available, the transaction follows the path $\pi = \langle purchase_request, g_2, e_4, g_3 \rangle$, reaching the event-based gateway g_3. If the property is unavailable, the transaction follows $\pi = \langle purchase_request, g_2, e_5 \rangle$, terminating at the trigger revert end event e_5, thereby reverting the transaction.

Table 1 presents a sample execution log for the use case process. The log includes five transactions, detailing the specific tasks performed during each transaction execution and the associated process variables. In the first transaction, the authority submits a transaction to define a zone as the permissible area for property registration. The zone data (`zone`) is stored in the registration smart contract, making it persistent. In the second transaction, the property owner attempts to register their property. The property data (`req_zone, prop_id, prop_owner`) is exported to the memory to verify its compliance with the zone restrictions before permanently storing it. Since the zone submitted

by the property owner does not match the zone defined by the authority, the transaction is reverted. In the third transaction, the property owner submits a compliant registration zone, thus the property is successfully registered by exporting the property data in the storage of the registration contract. Upon registering the property on the blockchain, the buyer attempts to purchase the registered property by specifying the property id (`req_prop_id`) in memory. In the final transaction, the property owner confirms the purchase and provides the address of the new owner (`new_owner`). This address is copied from memory to the registration contract's storage, updating the owner variable (`prop_owner`) to record the transfer of property ownership permanently.

Table 1. Execution log of the property registration process.

Transaction	Task	Memory	Persistent On-Chain Storage	
			Registration Contract	Transfer Contract
Tx1	Register Zone	zone: 5	zone: 5	∅
Tx2 (Reverted)	Register Property	req_zone: 2 prop_id: 1 prop_owner: 0x3dF...	zone: 5	∅
Tx3	Register Property	req_zone: 5 prop_id: 1 prop_owner: 0x3dF...	zone: 5	∅
	Mint Property	req_zone: 5 prop_id: 1 prop_owner: 0x3dF...	zone: 5 prop_id: 1 prop_owner: 0x3dF...	∅
Tx4	Purchase Request	req_prop_id: 1	zone: 5 prop_id: 1 prop_owner: 0x3dF...	∅
Tx5	Accept Request	new_owner: 0x1F7...	zone: 5 prop_id: 1 prop_owner: 0x3dF...	∅
	Transfer Property	new_owner: 0x1F7...	zone: 5 prop_id: 1 prop_owner: 0x1F7...	∅

6 Model-Driven Code Generation

In this section, we describe our model-to-code tool that compiles BPMN4BC models into Solidity code. We only consider the on-chain notations as input as the off-chain notations are beyond the scope of this paper. In Fig. 3, we present the meta model for the on-chain blockchain notations and their corresponding mappings to Solidity elements. A blockchain pool serves as the primary container for all blockchain notations and represents a unique blockchain network. Within

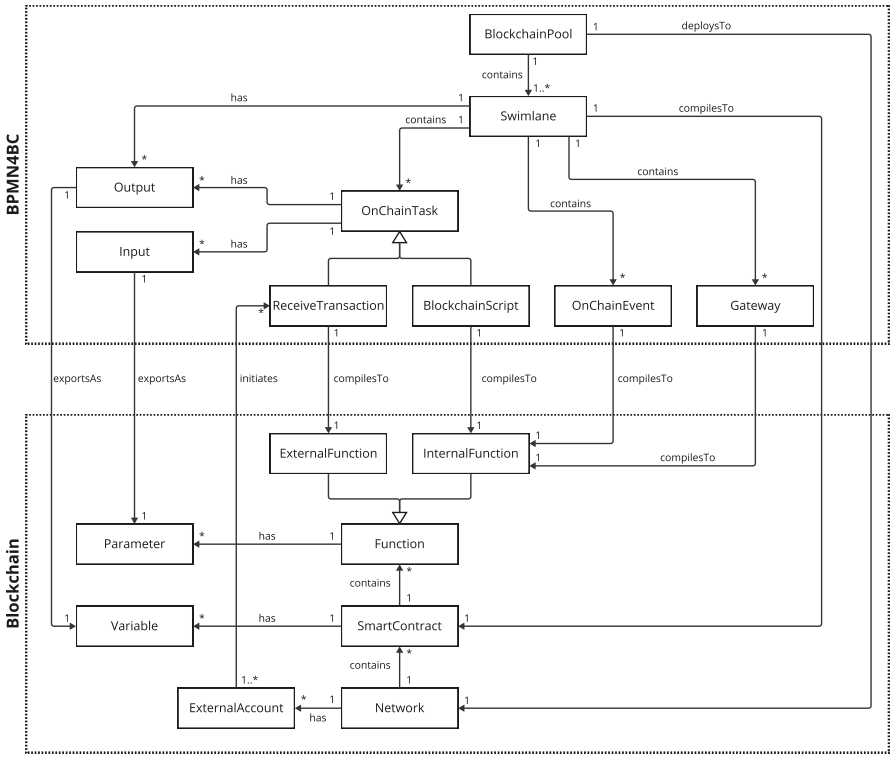


Fig. 3. Metamodel for BPMN4BC to code transformation.

a blockchain pool, multiple swimlanes can exist, each corresponding to a smart contract. A swimlane encapsulate the logic and functionality of their respective smart contract. On-chain tasks (e.g., receive transaction, script tasks), on-chain events (e.g., blockchain event, trigger revert), and gateways are compiled into Solidity functions. The receive transaction task is specifically compiled as an external function to allow transactions from external accounts. In contrast, other notations are compiled as internal functions, which are executed within the context of a transaction. On-chain tasks have input and output sets attached to them, representing the process variables defined by the user during the modeling phase. The input set consists of memory process variables that exist only within the transaction boundary in which the associated task is executed. Once the transaction completes, these variables are lost. In contrast, the output set consists of storage process variables that persist beyond the transaction execution, ensuring they remain available even after the transaction has completed. Additionally, swimlanes have an output set attached to them, allowing them to export storage process variables at deployment time. These variables are stored within the smart contract and remain available throughout the contract's lifecycle, enabling persistent data.

Listing 4 presents an excerpt of the compiled Solidity code generated from the blockchain-enabled process. This excerpt includes a part of the compiled code of the registration swimlane (Fig. 2). The registration swimlane is mapped directly to a smart contract (Lines 141). The `localStorage` struct (Line 12) encapsulates all process variables, which are readable across the network but can only be modified within the host smart contract's execution context. The Register Property task is compiled into the Solidity function `Register_Property` (Lines 1535). The function parameters capture inputs provided by the property owner, while the `isAccessible` modifier ensures only authorized participants can execute it by verifying the caller's address. The XOR gateway (g_1) is implemented as the function `XOR_Gateway_G1`, which compares the provided zone (`req_zone`) against the stored zone in `localStorage`. If the zones match, the `mint_property` function is triggered to register the property (Line 27), and an event signals its successful completion (Line 28). If the zones do not match, the transaction is reverted with a custom error (Line 30). Finally, the state of the task is updated to mark it as completed (Line 34) (Fig. 4).

7 Cost Evaluation

To conduct the evaluation, we host a test Ethereum network using Ganache[2]. We assess the gas costs under two configurations: one-contract and two-contract. In the two-contract configuration, the process diagram is deployed as-is (Fig. 2), where each swimlane corresponds to a separate smart contract: registration and transfer. In the one-contract configuration, we merge the two swimlanes into one, resulting in a single smart contract. We deploy the generated contracts independently and simulate transactions submitted by participants. Transactions are submitted sequentially, following the order of execution outlined in the execution log (Table 1). We measure the gas usage for the contract deployment and for every transaction execution in both configurations.

Figure 5 compares gas usage for contract deployment transactions across both configurations. The gas usage to deploy contracts in the two-contract setting is higher than the one in the one-contract configuration. This is due to the presence of fixed deployment fees [16] and redundant code duplicated in each smart contract responsible for resource-assignment variables and functions.

Figure 6 shows that task invocation transactions have relatively similar gas usage across the two configurations. The slight difference observed in Tx2 and Tx3, where the one-contract configuration incurs slightly higher gas usage, is attributed to the additional data processing required in a single contract. This includes operations such as traversing the state mapping (Listing 4, Line 3). However, the"Accept Request" transaction (Tx5) significantly differs. In the two-contract configuration, the transaction requires additional gas due to cross-contract interactions, whereas in the one-contract configuration, the call remains local within a single contract.

[2] https://archive.trufflesuite.com/ganache/.

```
contract Property_Contract {
    // Mapping to track the state of activities
    mapping(string => State) public state;

    // Struct to store property details
    struct LocalStorage {
        string property_owner;  // Owner of the property
        uint   property_id;     // Unique property identifier
        uint   zone;            // Zone where the property is located
    }

    LocalStorage private localStorage;

    // External function to register a property
    function Register_Property(
        uint   req_zone,
        string memory prop_owner,
        uint   prop_id
    )
        public isAccessible("Activity_11aq4w4")
    {
        // Call XOR_Gateway_G1 to determine the branch
        uint256 branch = XOR_Gateway_G1(req_zone);

        // Check the branch and execute actions accordingly
        if ((branch & (1 << 0)) != 0) {
            mint_property(prop_owner, prop_id); // Mint property token
            emit_Property_Mint();               // Emit event for property minting
        } else if ((branch & (1 << 1)) != 0) {
            revert("Zone not open for registration.");
        }

        // Mark the task as completed
        state["Register_Property"] = State.Completed;
    }

    // XOR gateway internal function
    function XOR_Gateway_G1(uint req_zone) public returns (uint256) {
        require(state["Register_Property"] == State.Open);
        return ((localStorage.zone == req_zone) ? (1 << 0) : 0) |
               (1 << 1);  // Bitwise OR to return the appropriate branch
    }
    ...
}
```

Fig. 4. Solidity code for the property register task in the property contract

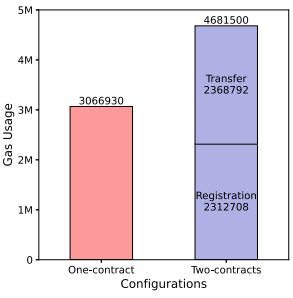


Fig. 5. Gas usage for contract deployment.

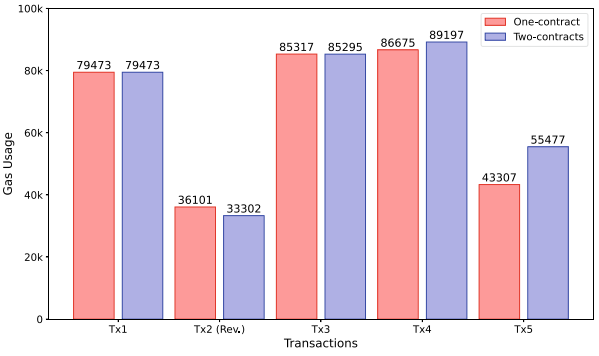


Fig. 6. Gas usage for transaction invocation.

8 Discussion

BPMN for blockchain enables users to integrate blockchain-related aspects into collaborative processes while clearly separating on-chain and off-chain logic. This distinction avoids ambiguity and misuse, e.g., recording private data on-chain and violating confidentiality, or executing on-chain tasks off-chain and losing the immutability and transparency of the process execution log. Clear boundaries thus prevent errors and ensure correct application of blockchain features.

Additionally, it becomes possible to identify transaction boundaries, allowing practitioners to specify the logic and the flow of each transaction. This helps optimize logic complexity by rearranging steps more efficiently, such as placing validation checks, e.g., XOR gateways, immediately after a transaction's entry point. This arrangement reduces gas fees by skipping the remaining logic in the case of invalid transaction data input. Breaking down transactions into multiple steps, which can be visualized on the diagram, paves the way towards a practical approach to studying blockchain business process optimization.

Furthermore, the extended notations offer a clearer definition of blockchain logic within the process diagram, enabling deeper analysis to identify reusable patterns [2] for specific use cases. As seen in the evaluation, the two different configurations for contract deployment yield different results in terms of gas usage, with a trade-off between modularity and cost. Deploying all logic within a single smart contract may reduce deployment costs but sacrifice modularity—if changes to the process are required, the entire contract must be redeployed.

9 Conclusion and Future Work

This paper introduces BPMN4BC for explicitly modeling blockchain technology in collaborative process diagrams. Each extension is discussed in terms of its usage and demonstrated through a blockchain-based property management use case. The model is translated into Solidity and executed on an Ethereum test network to evaluate gas usage under two contract deployment configurations. Future work could explore additional deployment strategies, assessing trade-offs at both technical and business levels. Research into reusable modeling patterns may support modularity for recurring scenarios. Furthermore, a usability study involving BPMN and blockchain experts is needed to evaluate the visual clarity and expressiveness of BPMN4BC, identifying opportunities for refinement.

Acknowledgement. This work is supported by the Swiss National Science Foundation (SNSF) funded project "Flexible Choreographies in Multi-chain Environments" (196958).

References

1. Atwi, H., Lichtenstein, T., Pautasso, C., Weske, M.: Transparent transaction ordering in blockchain-based collaborative processes. In: Di Ciccio, C., et al. (eds.) BPM 2024. LNBIP, vol. 527, pp. 25–39. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70445-1_2
2. Atwi, H., Pautasso, C.: Gas management patterns in blockchain-enabled process execution. Springer, Cham (2025, to appear). https://dlt4bpm.github.io/assets/papers/BPM_2025_paper_348.pdf
3. Falazi, G., Hahn, M., Breitenbücher, U., Leymann, F.: Modeling and execution of blockchain-aware business processes. SICS Softw.-Intensive Cyber-Phys. Syst. 1–12 (2019). https://doi.org/10.1007/s00450-019-00399-5
4. Haarmann, S., Batoulis, K., Nikaj, A., Weske, M.: DMN decision execution on the ethereum blockchain. In: Krogstie, J., Reijers, H.A. (eds.) CAiSE 2018. LNCS, vol. 10816, pp. 327–341. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-91563-0_20
5. Ladleif, J., Weske, M., Weber, I.: Modeling and enforcing blockchain-based choreographies. In: Hildebrandt, T., van Dongen, B.F., Röglinger, M., Mendling, J. (eds.) BPM 2019. LNCS, vol. 11675, pp. 69–85. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26619-6_7
6. López-Pintado, O., García-Bañuelos, L., Dumas, M., Weber, I., Ponomarev, A.: Caterpillar: a business process execution engine on the ethereum blockchain. Softw.: Pract. Exp. **49**(7), 1162–1193 (2019)
7. Lu, Q., et al.: Integrated model-driven engineering of blockchain applications for business processes and asset management. Softw.: Pract. Exp. **51**(5), 1059–1079 (2021)
8. Mendling, J., et al.: Blockchains for business process management-challenges and opportunities. ACM Trans. Manage. Inf. Syst. (TMIS) **9**(1), 1–16 (2018)
9. Milani, F., Garcia-Banuelos, L., Filipova, S., Markovska, M.: Modelling blockchain-based business processes: a comparative analysis of BPMN vs CMMN. Bus. Process. Manag. J. **27**(2), 638–657 (2021)
10. Mühlberger, R., et al.: Foundational oracle patterns: connecting blockchain to the off-chain world. In: Asatiani, A., et al. (eds.) BPM 2020. LNBIP, vol. 393, pp. 35–51. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58779-6_3
11. OMG: Business process model and notation specification version 2.0 (2011)
12. Taherdoost, H., Madanchian, M.: Blockchain and business process management (BPM) synergy: a comparative analysis of modeling approaches. Information **15**(1), 9 (2023)
13. Tilbury, J.L., de la Rey, E., van der Schyff, K.: Business process models of blockchain and South African real estate transactions. In: 2019 International Conference on Advances in Big Data, Computing and Data Communication Systems (icABCD), pp. 1–7. IEEE (2019)
14. Weber, I., Xu, X., Riveret, R., Governatori, G., Ponomarev, A., Mendling, J.: Untrusted business process monitoring and execution using blockchain. In: La Rosa, M., Loos, P., Pastor, O. (eds.) BPM 2016. LNCS, vol. 9850, pp. 329–347. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-45348-4_19
15. Wieringa, R.: Design Science Methodology for Information Systems and Software Engineering (01 2014)
16. Wood, G., et al.: Ethereum: a secure decentralised generalised transaction ledger. Ethereum Proj. Yellow Pap. **151**(2014), 1–32 (2014)

ProcNet: Cross-Modal Process Recommendation via LLMs and GCNs

Yumeng Jin[1], Leilei Lin[2(✉)], Siyu Li[2], Haipeng Wan[3], Wangyouqi Geng[1], and Wenlong Chen[1]

[1] Information Engineering College, Capital Normal University, Beijing, China
jinyumeng2012@163.com, Geng_wyq@163.com, chenwenlong@cnu.edu.cn
[2] School of Management, Capital Normal University, Beijing, China
leilei_lin@cnu.edu.cn, siyu_li3164@qq.com
[3] College of Education, Capital Normal University, Beijing, China
dnvhp@cnu.edu.cn

Abstract. Process models represent valuable experiential knowledge accumulated by enterprises throughout their development. The reuse of such knowledge can save managerial time and enhance the efficiency of process designers. Traditional process recommendation methods mostly focus on matching between process models themselves while neglecting the cross-modal matching problem between process texts and process models. Furthermore, enterprises accumulate process models in diverse forms, yet existing methods can only recommend a single form of process model (e.g., BPMN or Petri nets) and are unable to recommend multiple forms simultaneously. To address these challenges, this paper proposes a novel **process** recommendation **network**, referred to as **ProcNet**, which integrates large language models (LLMs) and graph convolutional networks (GCNs) for process recommendation. Specifically, (1) ProcNet uses LLMs to extract process activities and their relationships from requirement texts, constructs a directed graph, and semantically aligns it with the process activities in the database. Meanwhile, the process models in the database are also transformed into directed graphs; (2) ProcNet employs GCNs to obtain high-dimensional feature representations of each directed graph; (3) ProcNet calculates the top-k process models in the database most similar to the requirements using Euclidean distance and recommends them to the user. Extensive experiments demonstrate that ProcNet outperforms existing baselines, achieving remarkable results in recall, precision, F1-score, and mean average precision (MAP).

Keywords: Process model · Large language models · Directed graphs · Graph convolutional networks · Process text

1 Introduction

A process model typically refers to a formal representation of a business or operational process, often visually depicted to illustrate the sequence of activities, decision points, and flow relations within the process [1]. In the course

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 321–336, 2026.
https://doi.org/10.1007/978-981-95-5015-9_23

of providing high-quality services to users, enterprises accumulate a substantial number of process models [2,3]. The reuse of these models benefits diverse stakeholders. For instance, managers from different domains can quickly gain an understanding of relevant business content by sharing process models, while process designers can save considerable time by reusing or making minor modifications to existing models. The objective of process recommendation techniques is to recommend existing process models to users, enabling their reuse to reduce costs and enhance efficiency [4,5]. Early recommendation efforts predominantly focused on unimodal recommendation, which matches existing process models based on user-provided process models [6–8]. However, with the rapid advancement of natural language processing technologies, scenarios in which business process requirements are described in natural language have gained attention. Consequently, in recent years, some studies have begun to explore cross-modal recommendation approaches, which recommend existing process models through process description text input by users [9–11].

Cross-modal recommendation has achieved notable progress in fields such as multimedia [12] and natural language processing [13]. However, several limitations remain in the domain of process recommendation [9–11,14,15]: (1) It fails to accurately capture the relationships between activities described in process texts, such as exclusive and parallel relations. As we known, one of the most critical aspects of a process model is the execution relationships among activities. Inaccurate extraction of these relationships from textual descriptions leads to suboptimal recommendation performance. (2) It cannot simultaneously recommend process models in multiple formats, such as BPMN and Petri nets. If only a single format of process model can be recommended, the value of historical data (i.e., existing process models) cannot be fully realized. As illustrated in Fig. 1, this paper proposes a novel **process** recommendation **network**, named **ProcNet**, which can accurately extract process activities and their interrelations from user-provided process description texts. Consequently, ProcNet recommends the top-k process models encompassing multiple forms to users. Specifically, the main contributions are summarized as follows:

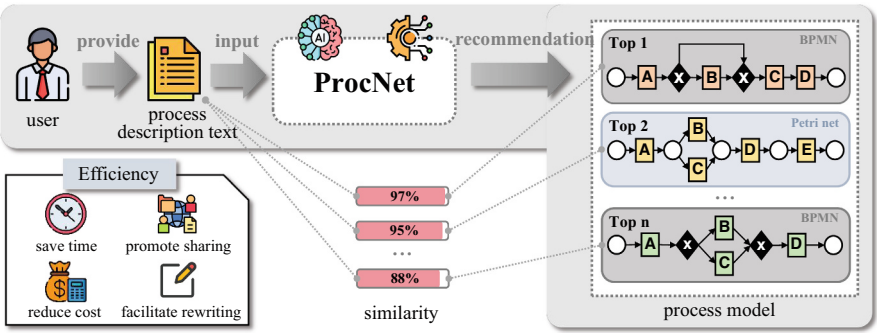


Fig. 1. Illustration of cross-modal process recommendation.

- We leverage large language models (LLMs) to parse the process description texts to identify process activities and their relationships. To improve parsing accuracy, inspired by the Chain of Thought (CoT) approach [16], we employ multi-turn dialogues with the LLMs to progressively complete the parsing task. Furthermore, to ensure robustness, semantic alignment is used to match activity names.
- We represent the features of both the process description texts and the process models by constructing directed graphs. Prior studies have demonstrated that directed graphs effectively represent processes [17]. Accordingly, ProcNet utilizes directed graphs to model different forms of process models and texts, ensuring accuracy in cross-modal recommendation.
- We embed these directed graphs using graph convolutional networks (GCNs) and efficiently perform process recommendation by computing Euclidean distances between high-dimensional embeddings. Extensive experiments show that ProcNet significantly outperforms existing methods in terms of precision, recall, F1-score, and mean average precision (MAP).

The paper is organized as follows: Sect. 2 reviews the currently process recommendation techniques, and summarize the characteristics of different techniques. Section 3 details the new process recommendation approach ProcNet. Section 4 evaluates ProcNet experimentally and compares it with the baselines. Finally, Sect. 5 summarizes our works and discusses potential future research directions.

2 Related Work

Business process recommendation techniques are increasingly being adopted as a core means to enhance modeling efficiency and have gradually given rise to two major research paradigms: unimodal recommendation and cross-modal process recommendation. The former focuses on the similarity matching between process models, while the latter is committed to realizing the cross-modal mapping from natural language descriptions to process models.

Unimodal recommendation mainly focus on recommending existing process models based on the structural or semantic similarity of the input process fragments. The methods based on structural patterns emphasize graph topology matching. Li et al. [4] utilized minimal DFS for efficient graph distance calculation to enable real-time node recommendation. Deng et al. [7] introduced a two-stage system mining node relationship patterns offline and recommending via pattern matching during modeling. Wang et al. [18] proposed an algorithm based on independent paths, achieving efficient similarity calculation via TPST and TAR. Li et al. [19] proposed a metadata-based method (MFI-5), defining a business process description framework and creating process vectors and similarity matrices using metadata features for recommendation. Trabelsi et al. [20] explored the use of decision trees trained on historical modeling data to recommend the next suitable task in BPMN base process models. In addition, Sola et al. [6] proposed a rule-driven method providing explanatory recommendations through abstraction and rule learning. In the methods based on semantic

similarity, Sola et al. [8] proposed new rules to address label matching issues and developed a semantic similarity extension. In addition, they [21] proposed a T5-based sequence-to-sequence framework integrating node sequences and label semantics. Wang et al. [22] presented a context-aware system, extracting semantics via paths and P, Q-grams, and combining Word2vec embeddings for recommendation. However, unimodal recommendation methods rely on process models as their input, which necessitates that users have both process modeling expertise and thorough knowledge of the business context. Moreover, in many practical situations, process models are not available in advance, and only textual descriptions of business process requirements exist. This limitation constrains the applicability of unimodal recommendation methods.

With the increasing demand for natural language interaction, text-based process recommendation are gradually becoming a research hotspot. Existing work mainly focuses on extracting process semantics from unstructured text and realizing cross-modal recommendations. Halioui et al. [14] proposed an ontology and word sense disambiguation-based extraction framework. Khider et al. [23,24] utilized social media data to build interest profiles for process recommendation based on Model-Driven Engineering (MDE). Wen et al. [11] constructed a Heterogeneous Information Network with clustering for scientific workflow recommendation, though text served mainly as auxiliary labels. Yu et al. [25] proposed the WFER method, applying graph embeddings for semantic-based process similarity computation. Diao et al. [15] proposed a Social-Aware Process Knowledge Graph, integrating social factors for efficient process fragment discovery. Gu et al. [10] proposed a Scientific WOrkflow Retrieval by learning Textual and Structural semantics (SWORTS), which fused textual and structural semantics for a two-stage retrieval, but failed to model the dynamic logic between activities. Additionally, they [9] proposed PGM, leveraging LLMs to parse user queries and generate structured workflow specifications, achieving recommendation through plan, generate and match. This method significantly outperformed traditional text matching in complex semantic understanding but did not explore the unified representation of multiple process formats.

In summary, unimodal recommendation methods are limited in scenarios where process requirements are described using textual descriptions. Existing cross-modal process recommendation approaches fail to accurately capture the complex relationships among activities from text, resulting in low recommendation accuracy. Furthermore, some methods are restricted to recommending only a single form of process model, whereas process models in practice exhibit diverse forms. Therefore, this paper proposes a novel process recommendation network with LLMs and GCNs. Table 1 highlights the differences between our method and other methods.

3 Methodology

In this section, we introduce a novel framework, ProcNet, which integrates LLMs and GCNs for process recommendation. Figure 2 shows an overview of

Table 1. Evaluation capabilities of different approaches.

Literature	Strategy	Cross-Model	Characteristics		Multi-form
			Structure	Semantics	
[6]	Multi-template Rule	✘	✓	✘	✘
[8]	Semantics-aware Rule	✘	✘	✓	✘
[9]	LLMs	✓	✘	✓	✘
[10]	TF-IDF and hierarchical neural matching	✓	✓	✓	✘
[11]	HIN metapath similarity with DPC clustering	✓	✓	✓	✘
[20]	Decision Tree	✘	✓	✘	✘
[21]	Pre-trained Language Model	✘	✘	✓	✘
[22]	Word2vec	✘	✓	✓	✘
[25]	SDNE graph embedding	✓	✓	✘	✘
This paper	LLMs and GCNs	✓	✓	✓	✓

our proposed method: upon receiving a user query (i.e., process text), ProcNet first utilizes LLMs to extract process activities, order relationships, and parallel relationships for directed graph construction, followed by semantic alignment. Additionally, heterogeneous processes in the process database are transformed into directed graphs. Subsequently, GCNs are employed for feature embedding, enabling each node in the directed graph to integrate information from its neighboring nodes into high-dimensional vectors. Finally, Euclidean distance is calculated to compare the distances between directed graphs, and the top-k process models with different forms are recommended to the user.

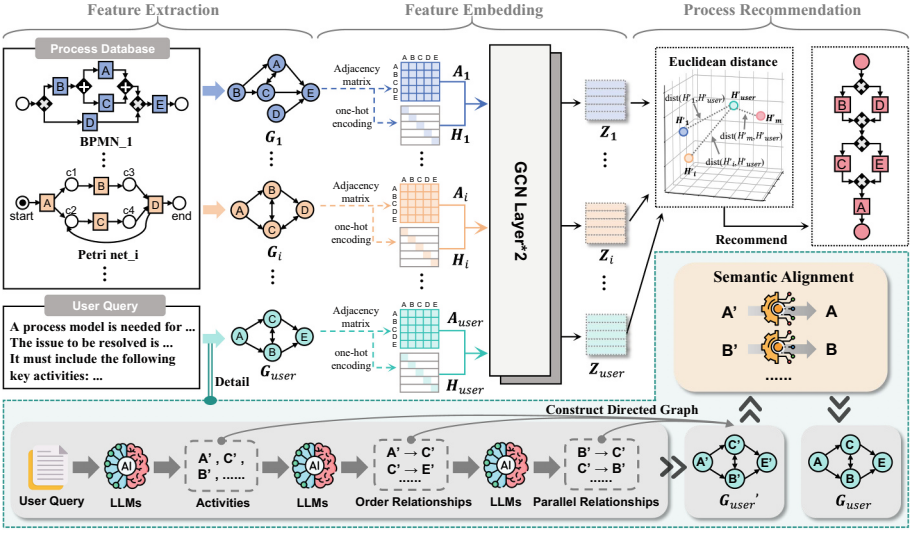


Fig. 2. Overview of our technique named ProcNet for process recommendation.

3.1 Feature Extraction

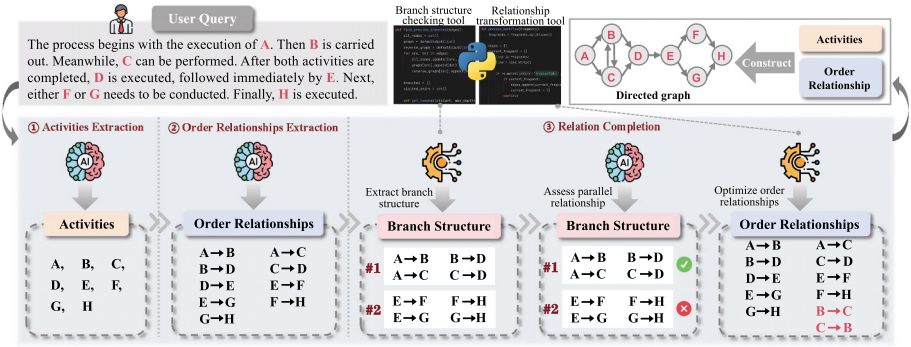


Fig. 3. An example of transforming a user query into a directed graph.

The first phase of the ProcNet is feature extraction for both the user query and the process models in the process database, that is, to uniformly transform them into directed graphs. As shown in Fig. 3, we use the LLMs to transform user query into directed graph. Although the LLMs demonstrate powerful semantic understanding capabilities, they still face significant challenges in logical reasoning between activities, especially non-sequential activities such as parallel and exclusive relationships. Therefore, we adopt a multi-turn LLMs dialogue strategy to gradually transform text into a directed graph, with the following steps: **1) Activities Extraction.** Based on the process description text provided by the user, the LLMs extracts all activities that can serve as nodes in the directed graph, such as the 'Activities' in the Fig. 3. **2) Order Relationships Extraction**. The LLMs extracts the order relationships between activities based on the process description text. The 'Order Relationships' in Fig. 3, where 'A→B' indicates a directly follows relation between activity A and activity B. **3) Relation Completion.** Since the LLMs have difficulty understanding non-sequential activity relationships, especially parallel activity relationships, we introduce two self-developed external tools to assist the LLMs in extracting parallel relationships. Specifically, we first use a branch structure checking tool to identify branching structures in the order relationships generated in step 2. As shown in Fig. 3, the order relationships 'A→B', 'B→D', 'A→C', and 'C→D' form a branching structure because they have a common starting node A and a common ending node D, consisting of two branches. For example, two branching structures are identified in Fig. 3 (i.e., #1 and #2). Next, the LLMs determine whether the two branching structures have parallel relationships based on process text. Then, an relationship transformation tool is used to optimize 'Order Relationships' based on parallel relationships. Finally, we construct a directed graph based on 'Acitvities' and 'Order relationships'.

In order to better leverage the capabilities of LLMs, we adopted the following strategies: **1) Few-shot.** This method involves providing LLMs with examples of inputs and expected outputs to guide them in solving tasks [26]. Therefore, we provided LLMs with examples of extracting activities or order relationships. **2) Chain-of-Thought.** The concept of Chain-of-Thought (CoT) was proposed to enhance the ability of large language models to handle more complex tasks, based on the principle of 'thinking step by step' [16,27]. Inspired by this concept, we also incorporated the CoT method into our prompting strategy to strengthen the extraction capabilities of LLMs. **3) Iterative Reasoning.** During the feature extraction phase, LLMs are prone to feature omission. So, we employ iterative reasoning to improve the accuracy of extracting activities and order relationships, with a default of two iterations.

To improve the matching accuracy between the directed graphs extracted by LLMs and the process models stored in the database, it is necessary to perform semantic alignment between the activity names in the directed graphs and those in the database. To achieve this goal, we employ a Transformer-based model [28] to obtain semantic vectors for all activity names. Let $L^{llm} = \left\{a_1^{llm}, a_2^{llm}, \ldots, a_m^{llm}\right\}$ denote the set of activities extracted by the LLMs, and $L^{std} = \left\{a_1^{std}, a_2^{std}, \ldots, a_n^{std}\right\}$ denote the set of activities in the database. Subsequently, cosine similarity is calculated to evaluate the similarity in the semantic vector space between L^{llm} and L^{std}:

$$sim\left(a_i^{llm}, a_j^{std}\right) = \frac{v_i^{llm} \cdot v_j^{std}}{\left\|v_i^{llm}\right\| \left\|v_j^{std}\right\|} \cdot \tag{1}$$

where v_i^{llm} represents the semantic vector of activity a_i^{llm} in L^{llm}, and v_j^{std} represents the semantic vector of activity a_j^{std} in L^{std}. Furthermore, for each activity a_i^{llm} in L^{llm}, its semantically aligned target standard activity $a_{aligned}^{std}$ is determined by Eq. (2):

$$a_{aligned}^{std}\left(a_i^{llm}\right) = \underset{a_j^{std} \in L^{std}}{\operatorname{argmax}} \left(sim\left(a_i^{llm}, a_j^{std}\right)\right) \cdot \tag{2}$$

After completing semantic alignment, all activities in L^{llm} are replaced with their corresponding target standard activities in L^{std}.

Next, we will perform feature extraction on multi-form process models in the database and transform them into directed graphs. As illustrated in Fig. 4, the BPMN and Petri net are first decomposed into multiple block structures. The red boxes represent parallel block structures, the orange boxes represent sequential block structures, and the purple boxes represent exclusive block structures. Multiple block structures are connected through sequential relationships. For parallel block structures, all activities within the block exhibit mutual directly follows relation. For exclusive block structures, no directly follows relation exists between any activities within the block structure. Finally, edges are introduced to connect activities between block structures, thus completing the conversion into directed graphs.

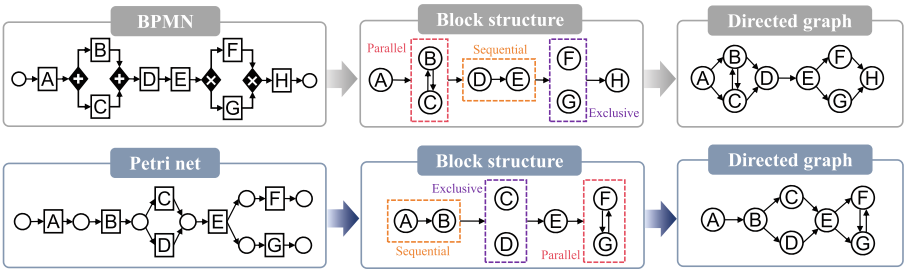


Fig. 4. The process of converting BPMN and Petri net into directed graphs.

3.2 Feature Embedding

After transforming user query and process models into directed graphs, it is necessary to extract feature embeddings of them. Classic graph traversal algorithms, such as depth-first search and DeepWalk [29], have significant limitations in efficiency and accuracy when comparing the structural differences of directed graphs. First, these traversal algorithms are inefficient in dealing with complex structures and cannot handle large-scale graph data. Second, they are also inaccurate in precision, as they can only compare local features between two directed graphs and fail to capture the differences between them from the overall topological structure. In contrast, Graph Convolutional Networks (GCNs) [30] overcome these limitations through a hierarchical propagation mechanism, which iteratively aggregates neighborhood information, enabling GCNs to capture key features of the global graph structure while preserving local neighborhood information [31,32]. Based on this advantage, ProcNet employs multi-layer GCNs to extract feature embeddings of directed graphs, with the propagation rule defined as follows:

$$H^{(l+1)} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right). \tag{3}$$

where $\widetilde{A} = A + I_N$ is the adjacency matrix of the directed graph with added self-connections, I_N is the identity matrix, and $\widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}$ is the diagonal degree matrix. $W^{(1)}$ is a layer-specific trainable weight matrix, $\sigma(\cdot)$ denotes the activation function, typically ReLU. $H^{(1)} \in \mathbb{Z}^{N\times D}$ is the activation matrix for the l-th layer, with $H^{(0)} = X$, and X represents the initial feature matrix of the directed graph.

It is important to note that during the convolution operation, the weight matrix $W^{(1)}$ is randomly initialized, which can significantly interfere with the reliability of graph similarity calculations. Therefore, ProcNet introduces a siamese mechanism [33] to construct two GCN models sharing identical weights. Specifically, let the weights of GCN models M and N be $W^M = \langle w_1^M, w_2^M, \ldots, w_m^M \rangle$ and $W^N = \langle w_1^N, w_2^N, \ldots, w_n^N \rangle$ respectively. The siamese mechanism is structured as follows:

$$\begin{cases} W \equiv N \\ m \equiv n \\ w_i^M \equiv w_i^N, \forall i \in [1, 2, \ldots, m] \end{cases} . \tag{4}$$

Before the graph convolution, each node in the directed graph is assigned a high-dimensional vector v based on one-hot encoding, where the dimension q equals the total number of nodes across both graphs, i.e., $One_Hot(v) \in \mathbb{R}^q$. Therefore, the initial feature matrix for the graph is $X = \{v_1, v_2, \ldots, v_k\} = H^{(0)}$. Subsequently, a two-layer GCN model performs forward propagation as follows:

$$Z = f(X, A) = \text{sofmax}\left(\widehat{A}\,\text{ReLU}\left(\widehat{A}XW^{(0)}\right)W^{(1)}\right). \tag{5}$$

Here, $\widehat{A} = \widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}$ denotes the normalization preprocessing of the adjacency matrix, and $W^{(0)} \in \mathbb{R}^q$ is the input-to-hidden weight matrix. After two layers of convolution, the GCNs achieves hierarchical learning of graph structural information, generating the directed graph feature embedding vector Z.

3.3 Process Recommendation

After feature embedding is complete, each directed graph has been transformed into a high-dimensional vector Z, which integrate local neighborhood details and global topological information. To quantify the difference between the user query graph G_u and each process model graph G_d in the process database, we introduce Euclidean distance as the core metric:

$$d(G_u, G_d) = \sqrt{\sum_{i=1}^{q}(Z_{u,i} - Z_{d,i})^2}. \tag{6}$$

Among them, Z_u and Z_d represent the graph-level embedding vectors of the user query graph and a database process model, respectively. The Euclidean distance calculation assigns a distance value to each process model in the database relative to the user query. A lower distance indicates higher similarity to the user query. Subsequently, we sort all process models in the database in ascending order based on these distance values. The top-k process models with the smallest distances are selected as recommendation candidates. Here, k is a configurable parameter controlling the number of the activities in recommedation list.

4 Experiments

In this section, we evaluate the performance of the ProcNet framework. The large language models used in our experiments are DeepSeek-R1, and all code is publicly available[1]. In terms of software and hardware, our test environment

[1] https://anonymous.4open.science/r/ProcNet-EA01.

consists of a computer equipped with an Intel i5-12500H@2.50 GHz processor and 16 GB of RAM (64-bit), running the Windows 11 operating system, with Python 3.9 as the programming environment.

4.1 Datasets

We select ProDrift[2] as the dataset because the process models within it encompass four types of control-flow structures: sequence, exclusive, loop, and parallel. ProDrift describes a business process for loan application, which includes a basic BPMN model along with 23 modified variants. To expand the dataset, we devised six modification rules (as shown in Table 2) to adjust the basic BPMN model. Additionally, we manually construct a basic Petri net equivalent to the basic BPMN model and applied the same six modification rules to extend the Petri net models. Among the six modification rules, the rule for modify gateway type involves changing the original parallel relationship to an exclusive one (and vice versa). The composite modification rule refers to the simultaneous application of multiple modification rules to the basic model. In total, we construct 392 BPMN models and 338 Petri nets, resulting in 730 process models that are stored in the process database as experimental data.

Table 2. Quantity statistics of transformations applied to basic BPMN and Petri net.

Number	Modification rule	Process model	
		BPMN	Petri net
1	Modify activity name	75	61
2	Add/Delete activity	78	72
3	Modify gateway type	12	13
4	Add/Delete gateway	15	19
5	Modify activity order	62	29
6	Composite modification	150	144

4.2 Baselines and Metrics

Baselines: We compare ProcNet with the following four state-of-the-art baselines: 1) SWORTS [10]. This method employs a two-stage framework that captures both textual and structural semantics of processes through path embedding and matching prediction models, enabling process recommendation. 2) CSBR [34]. This method constructs a service package repository based on composite semantic mining, learning functional semantics from Mashup collaborations, and employs a greedy algorithm for recommendation. 3) HDSWR [11]. This

[2] https://data.4tu.nl/articles/dataset/Business_Process_Drift/12712436.

method leverages heterogeneous information networks to integrate multi-source attributes, combining meta-path similarity computation and Density Peak Clustering to achieve recommendations. 4) LLMs. This method utilizes DeepSeek-R1 to perform end-to-end semantic parsing and process recommendation.

Metrics: We select 30 process models from the process database as the standard process models and convert them into process description texts to serve as user queries for the experiment. Therefore, for each standard process model, there is a corresponding matching process description text. Additionally, we select process models that are similar in both structure and semantics from the dataset, and construct a ground truth list PM_t sorted in descending order of similarity. To evaluate the performance of ProcNet in process recommendation, we employ four metrics: precision, recall, F1-score, and MAP. Among them, MAP is a metric that takes into account both relevance and ranking order. The evaluation metrics are defined by the following formulas:

$$Precision = \frac{|PM_r \cap PM_t|}{|PM_r|}, \tag{7}$$

$$Recall = \frac{|PM_r \cap PM_t|}{|PM_t|}, \tag{8}$$

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}, \tag{9}$$

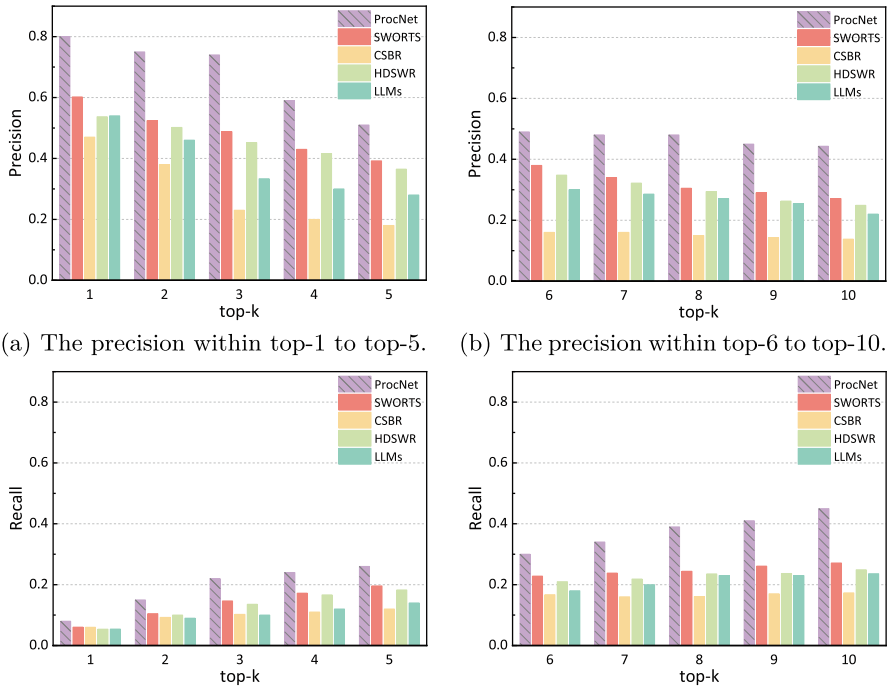
$$MAP = \frac{1}{Q} \sum_{q=1}^{Q} \frac{1}{|PM_t|} \sum_{i=1}^{|PM_r|} \left(\frac{rel_i}{i} \right). \tag{10}$$

Here, PM_r represents the list of process models recommended by ProcNet, Q denotes the number of user queries in the experiment, i is the ordinal position in PM_r, and rel_i indicates the number of processes in the top i processes of PM_r that belong to the ground truth list.

4.3 Comparing with the Baselines

We compare the performance of ProcNet with four state-of-the-art baselines. In the experiment, we set the length of PM_t to 10.

As shown in Fig. 5(a) and 5(b), the precision of ProcNet consistently maintains the highest value. In the Top-1, ProcNet achieves a precision of 0.8, significantly outperforming SWORTS, CSBR, HDSWR, and LLMs. As the number of recommendations increases, the precision of ProcNet gradually decreases. There is a relatively substantial decline in precision at Top-4, but it stabilizes in subsequent recommendations, eventually stabilizing at 0.45. This indicates that ProcNet can maintain high precision while increasing the number of recommendations, demonstrating good scalability and stability. In contrast, the precision of SWORTS, HDSWR, and LLMs is far inferior to that of ProcNet. Although



(a) The precision within top-1 to top-5.

(b) The precision within top-6 to top-10.

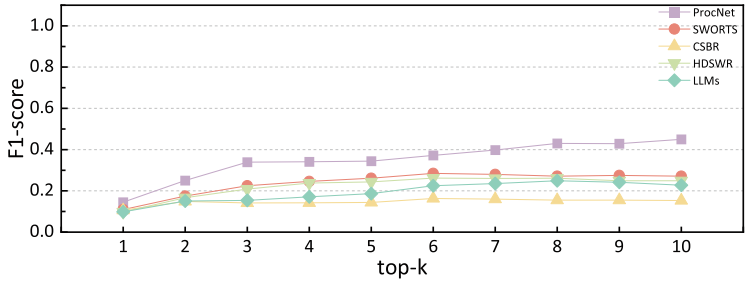
(c) The recall within top-1 to top-5.

(d) The recall within top-6 to top-10.

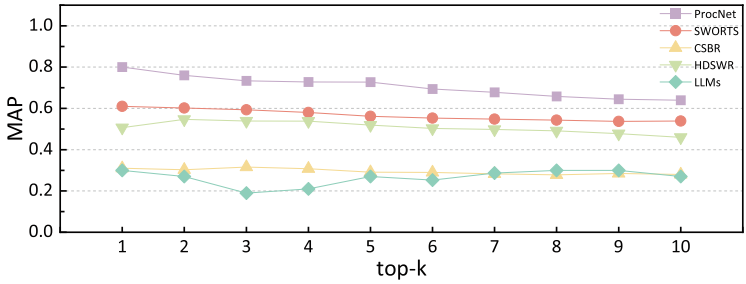
Fig. 5. Comparison of ProcNet with four baselines on precision and recall.

CSBR tends to stabilize after Top-3, its precision is too low. At Top-10, its precision drops to 0.14, indicating a severe loss of precision. While LLMs perform reasonably well at Top-1 and Top-2, the quality of their recommendations rapidly declines as Top-k increases, suggesting that relying solely on language models is insufficient to capture the semantic relationships at the process structure level. As depicted in Fig. 5(c) and 5(d), the recall of ProcNet steadily increases with the increase of Top-k, growing from 0.08 at Top-1 to 0.45 at Top-10. In comparison, although the recall of SWORTS and HDSWR also rises with Top-k, the overall are relatively low, reaching only 0.27 and 0.25 at Top-10, respectively, with a noticeably slower growth rate than ProcNet. The recall growth of CSBR is the slowest, while LLMs, despite a significant increase after Top-5, still have a limited overall improvement, with a recall of only 0.24 at Top-10.

Figure 6(a) illustrates the variation in F1-score. ProcNet consistently maintains the highest comprehensive performance within the Top-1 to Top-10. At Top-1, the F1-score of ProcNet is 0.15, already surpassing all baselines. As the number of recommendations increases, its F1-score grows steadily, reaching 0.45 at Top-10, significantly higher than the other baselines. In contrast, the final F1-scores of the baselines are all below 0.28, with CSBR and LLMs showing an increase of less than 50%. This indicates that ProcNet has a unique advantage



(a) The F1-score values within the top-10 recommendation list.



(b) The MAP values within the top-10 recommendation list.

Fig. 6. Comparison of ProcNet with four baselines on F1-score and MAP.

in balancing precision and recall, enabling it to continuously output high-quality recommendations. In terms of the MAP for measuring ranking quality, as shown in Fig. 6(b), the advantage of ProcNet is even more pronounced. At Top-1, its MAP has already reached 0.8, while the best baseline, SWORTS, is only 0.61, and LLMs are as low as 0.3. As the number of recommendations increases, ProcNet's MAP declines gently to 0.64 at Top-10, consistently about 18% higher than SWORTS. Additionally, as the recommendations grows, the MAP of LLMs fluctuates within the range of 0.18-0.3, while CSBR stagnates around 0.28.

In summary, the ProcNet significantly outperforms four baselines across the metrics of precision, recall, F1-score, and MAP. Especially when increasing the number of process recommendations, it maintains excellent recommendation accuracy and stability, and demonstrates significant advantages in the ranking of recommendation results. Additionally, to verify the scalability of ProcNet, we replace DeepSeek-R1 with ChatGPT-4 for process recommendation. The experiments demonstrated that the recommendation performances of the two base large language models are largely comparable. However, in terms of Top-10 recommendation results, the F1-score of ChatGPT-4 is 0.43 and the MAP is 0.61, slightly lower than those of DeepSeek-R1. Therefore, the default base large language model in the ProcNet framework remains DeepSeek-R1.

5 Conclusion and Future Work

Process modeling is a prominent topic in the field of service computing, and process recommendation techniques not only assist business practitioners in rapidly constructing models but also facilitate knowledge sharing. This paper addresses the issue of low accuracy in cross-modal recommendation by proposing a novel process recommendation framework, ProcNet, which integrates large language models (LLMs) and graph convolutional networks (GCNs). ProcNet employs a multi-turn dialogue mechanism to fully leverage the capabilities of LLMs, enabling precise extraction of activities and their relationships from user query (i.e., process texts). Simultaneously, ProcNet enhances LLMs' ability to capture parallel relationships within process texts and perform semantic alignment between activities by utilizing external tools. Furthermore, to tackle the challenge of recommending process models in multiple forms (such as BPMN and Petri nets), ProcNet converts different forms of process models into directed graphs and uses GCNs to obtain high-dimensional vector representations for each directed graph. Finally, it recommends the Top-k process models to users through distance-based calculations. We conduct experiments comparing ProcNet with four baselines, and the results demonstrate that ProcNet outperforms existing approaches in terms of precision, recall, F1-score, and MAP.

In future work, we plan to extend this research to accommodate a wider variety of modalities, such as business process video, and to support more modeling elements, including message flows and data flows.

Acknowledgments. The work was supported by Beijing Natural Science Foundation (No. QY25395), the Program of National Natural Science Foundation of China (No. 62477034).

References

1. Weske, M.: Business Process Management: Concepts, Languages, Architectures. Springer, Heidelberg (2007)
2. Gao, X.: Towards the next generation intelligent bpm – in the era of big data. In: Daniel, F., Wang, J., Weber, B. (eds.) BPM 2013. LNCS, vol. 8094, pp. 4–9. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40176-3_2
3. Reijers, H.A., Mendling, J.: A study into the factors that influence the understandability of business process models. IEEE Trans. Syst. Man Cybern.-Part A: Syst. Hum. **41**(3), 449–462 (2010)
4. Li, Y., et al.: An efficient recommendation method for improving business process modeling. IEEE Trans. Industr. Inf. **10**(1), 502–513 (2013)
5. Zhang, J., Liu, Q., Xu, K.: FlowRecommender: a workflow recommendation technique for process provenance, pp. 55–61. Australian Computer Society, Inc. (2009)
6. Sola, D., Meilicke, C., van der Aa, H., Stuckenschmidt, H.: A rule-based recommendation approach for business process modeling. In: La Rosa, M., Sadiq, S., Teniente, E. (eds.) CAiSE 2021. LNCS, vol. 12751, pp. 328–343. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79382-1_20

7. Deng, S., et al.: A recommendation system to facilitate business process modeling. IEEE Trans. Cybern. **47**(6), 1380–1394 (2016)
8. Sola, D., van der Aa, H., Meilicke, C., Stuckenschmidt, H.: Exploiting label semantics for rule-based activity recommendation in business process modeling. Inf. Syst. **108**, 102049 (2022)
9. Gu, Y., Cao, J., Guo, Y., Qian, S., Guan, W.: Plan, generate and match: scientific workflow recommendation with large language models. In: Monti, F., Rinderle-Ma, S., Ruiz Cortés, A., Zheng, Z., Mecella, M. (eds.) ICSOC 2023. LNCS, vol. 14419, pp. 86–102. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-48421-6_7
10. Gu, Y., Cao, J., Qian, S., Guan, W.: SWORTS: a scientific workflow retrieval approach by learning textual and structural semantics. IEEE Trans. Serv. Comput. **16**(6), 4205–4219 (2023)
11. Wen, Y., Hou, J., Yuan, Z., Zhou, D.: Heterogeneous information network-based scientific workflow recommendation for complex applications. Complexity **2020**(1), 4129063 (2020)
12. Wu, X., Huang, A., Yang, H., He, H., Tai, Y., Zhang, W.: Towards bridging the cross-modal semantic gap for multi-modal recommendation. arXiv preprint arXiv:2407.05420 (2024)
13. Liu, Z., Zhang, H., Dong, K., Fang, Y.: Collaborative cross-modal fusion with large language model for recommendation. In: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, pp. 1565–1574 (2024)
14. Halioui, A., Valtchev, P., Diallo, A.B.: Bioinformatic workflow extraction from scientific texts based on word sense disambiguation. IEEE/ACM Trans. Comput. Biol. Bioinf. **15**(6), 1979–1990 (2018)
15. Diao, J., Zhou, Z., Xue, X., Zhao, D., Chen, S.: Bioinformatic workflow fragment discovery leveraging the social-aware knowledge graph. Front. Genet. **13**, 941996 (2022)
16. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. Adv. Neural Inf. Process. Syst. **35**, 24 824–24 837 (2022)
17. Lin, L., et al.: Process drift detection in event logs with graph convolutional networks. In: Wang, X., et al. (eds.) DASFAA 2023. LNCS, vol. 13946, pp. 380–396. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30678-5_29
18. Wang, J., Tan, D., Cao, B., Fan, J., Deep, S.: Independent path-based process recommendation algorithm for improving biomedical process modelling. Electron. Lett. **56**(11), 531–533 (2020)
19. Li, Z., Wu, J., Zhang, X., He, J., Chen, P., He, K.: Using metadata for recommending business process. J. Supercomput. **76**, 3729–3748 (2020)
20. Trabelsi, F.Z., Khtira, A., El Asri, B.: Towards an approach of recommendation in business processes using decision trees. In: 2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC), pp. 341–347. IEEE (2021)
21. Sola, D., van der Aa, H., Meilicke, C., Stuckenschmidt, H.: Activity recommendation for business process modeling with pre-trained language models. In: Pesquita, C., et al. (eds.) ESWC 2023. LNCS, vol. 13870, pp. 316–334. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-33455-9_19
22. Wang, J., Gao, S., Tang, Z., Tan, D., Cao, B., Fan, J.: A context-aware recommendation system for improving manufacturing process modeling. J. Intell. Manuf. **34**(3), 1347–1368 (2023)
23. Khider, H., Hammoudi, S., Benna, A., Meziane, A.: Social business process model recommender: an MDE approach. In: 2018 Fifth International Conference on Social Networks Analysis, Management and Security, pp. 106–113. IEEE (2018)

24. Khider, H., Hammoudi, S., Meziane, A.: Business process model recommendation as a transformation process in MDE: conceptualization and first experiments. In: MODELSWARD, pp. 65–75 (2020)
25. Yu, X., Wu, W., Liao, X.: Workflow recommendation based on graph embedding. In: 2020 IEEE World Congress on Services, pp. 89–94. IEEE (2020)
26. Brown, T., et al.: Language models are few-shot learners. Adv. Neural. Inf. Process. Syst. **33**, 1877–1901 (2020)
27. Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y.: Large language models are zero-shot reasoners. Adv. Neural. Inf. Process. Syst. **35**, 22 199–22 213 (2022)
28. Nikolaev, D., Padó, S.: Representation biases in sentence transformers. arXiv preprint arXiv:2301.13039 (2023)
29. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710 (2014)
30. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 (2016)
31. Cao, Y., Lin, L., Di, Y., Li, X., Chen, W.: MDD: process drift detection in event logs integrating multiple perspectives. In: 2024 IEEE International Conference on Web Services (ICWS), pp. 1125–1134. IEEE (2024)
32. Lin, L., Cao, Y., Chen, W., Zong, Z., Qian, C., Wen, L.: Multi-view trace clustering based on graph convolutional networks in process mining. IEEE Trans. Serv. Comput. **01**, 1–12 (2025)
33. Mueller, J., Thyagarajan, A.: Siamese recurrent architectures for learning sentence similarity. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30, no. 1, pp. 2786–2792 (2016)
34. Gu, Q., Cao, J., Liu, Y.: CSBR: a compositional semantics-based service bundle recommendation approach for mashup development. IEEE Trans. Serv. Comput. **15**(6), 3170–3183 (2021)

Reliable and Configurable Process Simulations via Probabilistic White-Box Models

Francesco Vinci[1(✉)], Gyunam Park[2,3], Wil M. P. van der Aalst[2,4], and Massimiliano de Leoni[1]

[1] University of Padua, Padua, Italy
francesco.vinci.1@phd.unipd.it, deleoni@math.unipd.it
[2] Fraunhofer Institute for Applied Information Technology (FIT), Sankt Augustin, Germany
g.park@tue.nl
[3] Eindhoven University of Technology, Eindhoven, The Netherlands
[4] Process and Data Science, RWTH Aachen University, Aachen, Germany
wvdaalst@pads.rwth-aachen.de

Abstract. Business process simulation (BPS) has emerged as a crucial tool that offers a risk-free virtual environment to analyze, test, and optimize complex service compositions and orchestrations in cloud and edge computing environments. BPS enables the evaluation of alternative scenarios (a.k.a. "what-if" scenarios) by capturing the dynamic behavior of service interactions, including control-flow, task durations, and resource utilization. Several techniques to discover BPS models employ black-box predictors to characterize the run-time simulation aspects. The downside of these approaches is that the rules guiding the predictors cannot be provided in an intelligible form. This means that service architects and process analysts cannot modify or explore the decision-making logic that drives service orchestration and composition in these simulations. Moreover, these models are often deterministic and thus unable to capture uncertainty, which is essential to realistically simulate dynamic environments. This paper presents white-box predictors based on probabilistic decision trees, which are intelligible and easy to configure. The experiments show that white-box predictors can improve the simulation accuracy and variability, while being naturally intelligible.

Keywords: Business Process Simulation · Service Composition · Machine Learning · Explainable AI · Probabilistic Decision Trees

1 Introduction

Service-oriented business processes have become the backbone of modern enterprise architectures, where complex workflows are executed through the composition and orchestration of distributed services [11]. In cloud and edge computing environments, these processes typically coordinate multiple external services, microservices, and APIs to deliver end-to-end business functionality. The dynamic nature of such architectures poses significant challenges in understanding and predicting process behavior [21].

Business process simulation (BPS) has emerged as a key tool in service-oriented computing, offering a risk-free virtual environment to analyze, test, and improve service

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 337–352, 2026.
https://doi.org/10.1007/978-981-95-5015-9_24

compositions and orchestrations [4,19]. In the first phase, BPS allows the identification of potential delays (e.g., slow services or bottlenecks) and the validation of process design. In the second phase, analysts can define *what-if* scenarios by adjusting resource allocations, modifying SLAs, or replacing human tasks with automated services [10].

The starting point in service-oriented BPS is a process model that describes service composition and orchestration logic, extended with an accurate run-time characterization of different service perspectives, primarily data flow, guards, resource allocation, and temporal behavior [20]. However, traditional approaches often fail to reflect real-world behavior. Current state-of-the-art works show that integrating deep learning models into BPS can lead to more realistic run-time characterization [7]. However, deep learning models are black-box: the rules guiding these predictors are not explicitly provided as analytic expressions, thus not being intelligible. As such, it becomes impossible to alter these rules to define *what-if* scenarios and test the consequences on process performance via BPS. Moreover, since deep learning models usually require a large amount of data, deep learning-based BPS models cannot be constructed when the real process data are limited or missing. In addition, these models are often deterministic, which reduces their ability to generalize across different scenarios and limits their applicability in dynamic or uncertain environments.

This paper addresses these limitations by proposing a simulation approach specifically designed for service-oriented business processes, where the run-time characterization is based on probabilistic decision tree models. These white-box models essentially consist of decision rules that service architects and process analysts can inspect and alter to test *what-if* scenarios involving different service configurations, SLA modifications, or infrastructure changes. We introduce **Decision-Aware Business Process Simulation** models, which are basically process models extended with the run-time characterization of BPS based on decision trees (cf. Sect. 3). Our technique discovers decision trees from service execution data, using machine learning to capture service selection rules and dependencies across various features (e.g., service attributes, timing, workload) (cf. Sect. 3.3). We also present a method for interacting with these decision trees to define *what-if* scenarios for service-oriented processes, enabling service architects to evaluate the impact of service changes or new service compositions (cf. Sect. 3.4).

We evaluated our approach in five case studies to assess its accuracy in reproducing the behavior of the real process (cf. Sect. 4). The results demonstrate that our Decision-Aware Business Process Simulation (DBPS) approach consistently outperforms the state-of-the-art white-box simulation method, namely Simod [9] and achieves competitive accuracy compared to black-box approaches [5,17] while offering superior interpretability for service management decisions. Additionally, our method demonstrates greater realism and variability in simulation outcomes, making it particularly suitable for dynamic service-oriented environments that demand transparent, configurable, and realistic behavioral modeling for effective service orchestration and composition management.

2 Related Work

The work by Rozinat et al. [20] is one of the first combining process mining techniques to discover multiple perspectives of a process (control-flow, attributes, service perfor-

Table 1. Comparison of various simulation approaches across different perspectives (Control-Flow, Temporal, and Resource). A ○ symbol indicates that the perspective is modeled using a white-box approach, while ● denotes a black-box approach. $\mathbf{P}$ and $\mathbf{D}$ indicate if the models are probabilistic or deterministic, respectively. A gray box indicates the use of machine/deep learning techniques for the underlying rules. The last column indicates whether the simulation model is centered around the control-flow perspective, particularly relevant for service-oriented processes.

Approach	Process Perspective			Control-Flow Centric
	Control-Flow	Temporal	Resource	
Rozinat et al. [20]	○$_{\mathbf{D}}$	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	✓
Simod [9]	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	✓
DSim [5]	○$_{\mathbf{P}}$	●$_{\mathbf{D}}$	○$_{\mathbf{P}}$	✓
RIMS [17]	○$_{\mathbf{P}}$	●$_{\mathbf{D}}$	○$_{\mathbf{P}}$	✓
Agent Simulator [12]	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	
Our	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	○$_{\mathbf{P}}$	✓

mance, and resource aspects) and integrating them into complete simulation models. In this work, the authors propose the use of decision tree models as decision guards in the control-flow perspective.

The increasing availability of data and the advancement of new machine and deep learning techniques led to the integration of these into traditional process simulation methods. Camargo et al. [7] investigated the benefits of employing deep learning techniques compared to the traditional methods, referred to as "data-driven". The authors found that deep learning techniques produce more accurate simulations, although their technique does not enable *what-if* analyses.

Camargo et al. [5] and Meneghello et al. [17] propose two hybrid approaches, respectively named **DSim** and **RIMS**, where a process model is discovered to model the process' control-flow perspective, which is extended with deep learning models for the run-time characterization of the time perspectives. However, as mentioned in Sect. 1, the use of black-box models makes it impossible to conduct specific *what-if* analyses. Additionally, deep learning models are usually "data greedy", making these approaches less suitable in case of scarcity of process data for one or more perspectives, or even impossible when no process data is available for different perspectives (e.g. event logs without resource information). Moreover, these models are inherently deterministic, always producing the same output for a given input. This lack of variability introduces a limitation in service-oriented business process simulation, where incorporating noise and randomness is essential to generate realistic and insightful simulation results. As a consequence, their deterministic nature reduces their ability to generalize across different scenarios, limiting their applicability in dynamic or uncertain service environments.

Some body of research exists that focuses on defining white-box models to characterize the run-time BPS aspects. In [14], the authors have proposed to train logistic regression models to define data-dependent probabilistic guards, but they do not consider the perspectives on resource and time. The same limitation is also shared with RIMS as well as with **Simod** [9], another BPS approach that focuses on discovering and

modelling data-dependent branching probabilities using rule-aware models. Recently, Kirchdorfer et al. [12] have proposed **Agent Simulator**, an agent-based approach for discovering BPS from an event log, providing high interpretability and adaptability. This approach reverts the center of simulation attention from the control-flow to the resource perspective, using resource models as backbone for BPS models: it follows that can be challenging to perform *what-if* analyses heavily focused on the control-flow perspective. In a context of service-oriented business process, approaches that are not control-flow centric, such as Agent Simulator, focus primarily on individual services. As a result, it becomes more difficult to directly experiment with alternative ways of orchestrating a set of services to execute business processes. This orchestration is, in fact, the main focus of the control-flow perspective.

Table 1 provides a comparative summary of state-of-the-art process simulation approaches across the key process perspectives (control-flow, temporal, resource), highlighting the nature of the modeling techniques employed and whether or not they are centered on the control-flow, which is sensible for service-oriented business process simulation. The last row refers to the approach proposed in this paper, which - along with Simod - is the only approach that employs probabilistic methods to model the runtime characterization of the process' perspectives on control-flow, time and resource. As mentioned, a probabilistic characterization allows simulations to be more accurate and more generalized, while white-box methods are preferable over black-box because the former makes it extremely easier to configure process simulation models for alternative *what-if* analyses. Among the white-box, probabilistic approaches, Simod uses simple distributions and probabilities to characterize the temporal and resource perspective, which naturally do not provide outcomes that are as accurate as those provided by machine learning approaches. In fact, our approach focuses on stochastic decision trees, highly intuitive and easily configurable. In summary, Table 1 allows one to conclude that our approach is more accurately capture real-world uncertainties, incorporating noise and stochasticity for greater realism, while easily enabling what-if analyses. The experiments presented in Sect. 4 provide evidence supporting these claims.

Note that every approach discussed so far, including those reported Table 1 and ours, uses methods that capture correlations. These correlations are not used directly for root-cause analysis and process improvements but are merely used to guide the simulation, which is conversely used for root-cause analysis and improvement. There is a large body of research on causal reasoning methods, such as by Alaee et al. [2], which certainly provides more rigorous methods for causal inference but, on the other hand, do not easily enable *what-if* analyses.

3 Decision-Aware Business Process Simulation

In this paper, we introduce Decision-Aware Business Process Simulation models (DBPS), where decision models govern control-flow and temporal simulation parameters based on feature values of simulation instances. These decision models operate within defined feature spaces, allowing users to interact with and modify service rules to perform *what-if* analyses.

We first introduce some preliminary concepts in Sect. 3.1. In Sect. 3.2, we formally define DBPS, while Sect. 3.3 presents a technique to discover them from an input event

log $\mathcal{L}$. Finally, Sect. 3.4 illustrates how users can leverage these simulation models for interactive *what-if* analysis.

3.1 Preliminaries

In this section, we introduce key preliminary concepts essential for discussing our proposal. First, basic process mining definitions are presented, including events, event logs, and Petri net models [1]. Finally, we explore (probabilistic) decision tree models and their application in service-oriented BPS settings.

Process Data. Event logs are collections of traces [1], each recording a process execution (a.k.a. case) as a temporally ordered sequence of events [1]. Each event carries a data payload. Let $\mathcal{I}$, $\mathcal{A}$, and $\mathcal{R}$ denote the sets of all possible case identifiers, activities, and resources, respectively. For an event e, we have $case(e) \in \mathcal{I}$, $act(e) \in \mathcal{A}$, $res(e) \in \mathcal{R}$, and $time(e) \in \mathbb{N}$, corresponding to the case identifier, executed activity, involved resource, and timestamp, respectively. The latter induces the ordering within traces. Moreover, $life(e) \in \{start, complete\}$ indicates whether e marks the start or completion of the activity. Finally, let $\mathcal{V}$ and $\mathcal{U}$ denote the sets of attribute names and values; then $attr(e) : \mathcal{V} \to \mathcal{U}$ maps each $v \in \mathcal{V}$ to its value $attr(e)(v) \in \mathcal{U}$, inherited from the trace to which e belongs.

Event timestamps define key elements to model in a process simulator. The **inter-arrival times** are the times between the arrival of two consecutive cases; the **waiting time** of an event is the time between the start of that event and the time is enabled to start; the service **execution time** of an event is the time needed for executing an activity.

Process Models. In this paper, we use Petri nets to model and represent the activity control-flow of service processes with a set of recorded traces $\mathcal{L}$ [1]. We opted for Petri nets because they have a simple yet formal syntax and semantics for defining the control-flow stochasticity. However, the discussion remains valid even using other process model notations, such as process trees or BPMN models. A **Petri net** is a tuple $(P, T, F, \mathcal{A}, \lambda)$, where P is a finite set of places, T is a finite set of transitions, $F \subseteq (P \times T) \cup (T \times P)$ is a set of arcs, and $\lambda : T \to \mathcal{A} \cup \{\tau\}$ is a labelling function where τ denotes the label of invisible transitions. A Petri net marking is a function $m\colon P \to \mathbb{N}$, that assigns to each place the number of tokens it contains, representing the current state of the net.

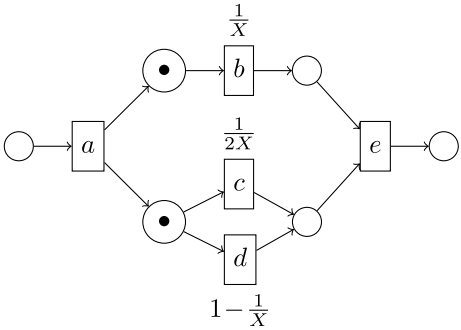


Fig. 1. Example of a Stochastic Labelled Petri net. The black dots (•) represent the current marking. Transitions of the enabled transitions are annotated with weights depending on the variable X.

A transition $t \in T$ is said to be enabled in a marking m if each of its input places contains at least one token. When an enabled transition fires, it consumes one token from each of its input places and produces one token in each of its output places, resulting in a new marking that reflects the updated state of the net.

For modeling the inherent stochasticity of the process simulations, we used the concept of stochastic Petri nets following the procedure described in [14], where each transition $t \in T$ is associated with a weight $\theta_t(X) \geq 0$ dependent on some values X. Specifically, the probability of firing a transition $t \in T$ at a certain marking m is defined as

$$P(t, m) = \begin{cases} \frac{\theta_t(X)}{\sum_{t' \in E(m)} \theta_{t'}(X)} & \text{if } t \in E(m) \\ 0 & \text{otherwise} \end{cases}$$

where $E(m) \subseteq T$ is the set of transitions enabled at m.

Example 1 (Stochastic Labelled Petri Net). Let N be a Petri net as shown in Fig. 1, with the current marking m highlighted with black dots. The set of enabled transition is then $E(m) = \{b, c, d\}$. The probability of firing the transition b is computed as $P(b, m) = \frac{\frac{1}{X}}{\frac{1}{X} + \frac{1}{2X} + 1 - \frac{1}{X}} = \frac{2}{1+2X}$.

Probabilistic Decision Trees. In this paper, we leverage **probabilistic decision tree** models for their interpretability, explainability, and ease of user interaction. Their white-box, interpretable nature makes them very suitable to enable *what if* analyses, where deep learning models fail (cf. Sect. 2). Other white-box models, such as logistic regression, rely on counterintuitive coefficient, making them less practical for domain analysts who aim to define *what-if* analyses. Probabilistic decision trees make predictions by hierarchically splitting the feature space into distinct regions based on specific decision rules. In these models, internal nodes and arcs define the decision criteria based on features in $\mathcal{F}$, while leaf nodes represent probability distributions. The use of probabilistic distributions make them be different from traditional decision tree, where leafs are associated to single static values.

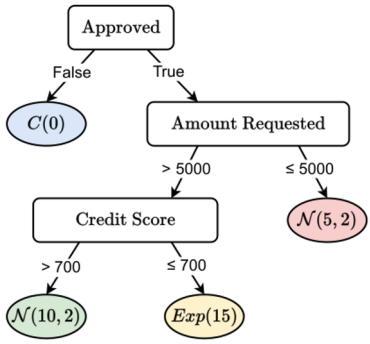


Fig. 2. Example of a stochastic decision tree that models the execution time of the activity *Close Application* in a credit loan process. $C(t)$ indicates a constant distribution with value t, $Exp(t)$ the exponential distribution with mean t, $\mathcal{N}(\mu, \sigma)$ the normal distribution with mean μ and standard deviation σ. Numbers at leaf values are in minutes.

In the BPS domain, stochasticity plays a crucial role. Traditional deterministic decision tree models may be inadequate, as they always produce the same output without accounting for noises or variability. Through stochastic decision tree, the final output is then generated by sampling from the distribution associated to the interested leaf.

Example 2. In a loan application process, service times of certain activities may depend on rules based on previous activities and/or on process attributes. Figure 2 illustrates an example of a decision tree modeling the execution time of the closure application activity. If the loan has not been approved the application closes immediately. Otherwise, for requested amounts below 5000, the duration follows a normal distribution with mean 5 and standard deviation 2. For higher amounts, service time increases and depends on the applicant's credit score. For example, for higher requested amount from customers with a low score the closure time follows an Exponential distribution with average 15.

Given a set of features $\mathcal{F}$, a deterministic decision tree is defined as a function $\theta \in (\mathcal{F} \to \mathbb{R}) \to \mathbb{R}$, where, for a given input $x \in (\mathcal{F} \to \mathbb{R})$, that assigns a numerical value to each feature, $\theta(x)$ represents the decision tree prediction. A probabilistic decision tree is a function $\theta_{\mathcal{P}} \in (\mathcal{F} \to \mathbb{R}) \to \mathcal{P}$, where $\mathcal{P}$ denotes a set of possible probability distributions. For a given input $x \in (\mathcal{F} \to \mathbb{R})$, the output $\theta_{\mathcal{P}}(x)$ represents a probability distribution over possible predictions. Hereafter, $\Theta^{\mathcal{F}} = (\mathcal{F} \to \mathbb{R}) \to \mathbb{R}$ and $\Theta^{\mathcal{F}}_{\mathcal{P}} = (\mathcal{F} \to \mathbb{R}) \to \mathcal{P}$ indicate the set of all possible deterministic and stochastic decision trees built using features $\mathcal{F}$, respectively.

3.2 Definition

Business process simulation models are typically defined by combining a process model with a set of simulation parameters. Indeed, process models offer a clear and understandable structure for representing the activity control-flow of a process.

We define a **Decision-Aware Business Process Simulation** (DBPS) model as a tuple (N, D, P) where:

- $N = (P, T, F, \mathcal{A}, \lambda)$ is a Petri net describing the activity control-flow.
- D represents descriptive parameters, including resource calendars, the set of possible resources $\mathcal{R}$, and distributions of trace service attributes.
- $P = (p^{CF}, p^{ET}, p^{WT}, \theta^{AT})$ is the tuple of predictive parameters:
  - $p^{CF}\colon T \to \Theta^{\mathcal{F}_{CF}}$ assigns to each transition a decision tree that determines the **transition weights** (cf. Sect. 3.1) based on the set of control-flow-related features $\mathcal{F}_{CF}$. These features include trace history (i.e., how many times an activity occurred in past executions of the same trace) and trace attributes, following approaches such as those in [14], where past executions and trace-level context influence branching decisions.
  - $p^{ET}\colon \mathcal{A} \to \Theta^{\mathcal{F}_{ET}}_{\mathcal{P}}$ maps each activity to a probabilistic decision tree that predicts its service **execution time**. The set of features $\mathcal{F}_{ET}$ includes the resource involved in the event, the trace history, and the trace attributes. This is consistent with state-of-the-art works where execution duration is shown to depend on who performs the task [16];
  - $p^{WT}\colon \mathcal{R} \to \Theta^{\mathcal{F}_{WT}}_{\mathcal{P}}$ returns, for each resource $r \in \mathcal{R}$, a probabilistic decision tree that predicts **waiting times** of events performed by r. The features in $\mathcal{F}_{WT}$ incorporate the resource workload (i.e. the current amount of events performed by the resource r), trace history and attributes. Waiting time is thus defined as resource-dependent, coherently with classical process simulator approaches, where waiting times are driven by resource availabilities and queues;

– $\theta^{AT} \in \Theta_{\mathcal{P}}^{\mathcal{F}_{AT}}$ is a probabilistic decision tree that predicts **inter-arrival times** of new process instances. The feature set $\mathcal{F}_{AT}$ includes global context features - specifically weekday and hour of the day - since arrival times are externally driven and must be modeled independently of the internal process execution.

Once a DBPS model is defined, it can be used to simulate the process. The predictive models are then integrated at run-time, following the approach of [17]. This enables the generation of event logs that contain a specified number of traces, starting from a given initial timestamp.

3.3 Discovery

In this section, we present a methodology for discovering Decision-Aware Business Process Simulation models from an input event log $\mathcal{L}$. The objective is to identify service rule patterns within the event log data for building decision tree models.

The Petri net model $N = (P, T, F, \mathcal{A}, \lambda)$ can be obtained using process discovery algorithms such as Inductive Miner [13], while the set of descriptive parameters D can be obtained using several techniques proposed in the literature, e.g. [15].

The event log $\mathcal{L}$ is then aligned with the control-flow model N, resulting in an event log $\mathcal{L}_N$ compliant with N. This alignment procedure serves as a basis for the construction of the training datasets required by predictive models, namely, the decision trees, used in the DBPS. The predictive parameters are then discovered as follows.

Transition Weights. For each transition $t \in T$ a training dataset is constructed following a procedure similar to that proposed in [14], but using decision trees instead of logistic regression models. Specifically, the training dataset is generated by collecting feature values of $\mathcal{F}_{CF}$ (i.e., trace history and trace attributes), whenever t is enabled in the aligned event log $\mathcal{L}_N$. The goal is to predict if t will be fired or not. A decision tree is then trained as a binary classifier. Finally the leaf values are computed as the proportion of correctly classified samples in each leaf, representing the probabilities of firing the transition t when enabled. Each decision tree defines the transition weight function $p^{CF}(t) \in \Theta^{\mathcal{F}_{CF}}$ for each $t \in T$.

Execution Times. For each activity $a \in \mathcal{A}$, we observe the events $e \in \mathcal{L}$ such that $act(e) = a$, and build a training set of feature values of $\mathcal{F}_{ET}$, which includes resource $res(e)$, trace history and trace attributes $attr(e)$ with their values, aiming to predict the execution time of e. A decision tree is then built for each activity $a \in \mathcal{A}$. Finally, for each leaf, a best fitting distribution is determined from a set of predefined ones by filtering the data based on the corresponding decision rules. The distributions are discovered following the procedure in [6], which aims to minimize the Wasserstein distance. The set of predefined distributions $\mathcal{P}$ include: *Normal*, *Exponential*, *LogNormal*, *Uniform*, and *Constant*. This results in probabilistic decision trees (cf. Sect. 3.1) defining $p^{ET}(a) \in \Theta_{\mathcal{P}}^{\mathcal{F}_{ET}}$ for each activity $a \in \mathcal{A}$.

Waiting Times. For each resource $r \in \mathcal{R}$ we observed the events $e \in \mathcal{L}_N$ such that $res(e) = r$ and build a training set of feature values in $\mathcal{F}_{WT}$ (i.e., resource workload, trace history and trace attributes) with the goal of predicting the waiting time. Specifically, the waiting time can be computed from the aligned event log by looking

at the time between the start timestamp of the event e and the time the correspondent transition, in the aligned event log, is enabled. This results in a probabilistic decision tree $p^{WT}(r) \in \Theta_{\mathcal{P}}^{\mathcal{F}_{WT}}$ for each resource $r \in \mathcal{R}$ that models the waiting time distribution for each leaf.

Inter-Arrival Times. A training dataset is built as follows: for each trace $\sigma \in \mathcal{L}$, the feature values of $\mathcal{F}_{AT}$ (i.e., the current weekday and hour) are retrieved in order to predict the arrival time of σ. A decision tree is then trained to estimate the arrival time from these feature values. The same procedure as for execution and waiting time is then followed to build the probabilistic decision tree $\theta^{AT} \in \Theta_{\mathcal{P}}^{\mathcal{F}_{AT}}$.

3.4 User Interaction for What-If Analysis

In service-oriented business process simulation, explainability is the key. The user aims to interact with the simulator to investigate several *what-if* scenarios, enabling them to evaluate the potential impacts of changes without putting them in production.

Since DBPSs are defined via (probabilistic) decision trees, such analyses are based on their modifications, which are of three types:

1. **Change leaves values**, namely distribution parameters in a certain leaf are altered.
2. **Remove subtrees**, namely a subtree is replaced by a single leaf.
3. **Add subtrees**, namely a composite tree replaces a single leaf.

Note that these interactions can also be combined, and the user can also define new decision tree models (e.g. when including more resources or new activities in the process models). When including new resources the user must also model the waiting time for them, which may depend on some features in $\mathcal{F}_{WT}$. Alternatively, users can also reuse existing decision trees from other resources to mimic the same behaviors. Also, when including a new activity, i.e., new transitions in the process model with a new activity label, the user can similarly introduce new decision trees for each new transition defining the probability of firing that transition when enabled.

Example 3. Figure 3 illustrates an example of interacting with the decision tree from Example example:dt to perform a *what-if* analysis. In this new scenario, the service execution time of *Closure* is no longer dependent on the *Credit Score* for requested amounts exceeding 5000. Additionally, for requested amounts below 2000, the service time is reduced. This adjustment allows evaluating how the process performance is affected by these changes.

4 Experiments

This section presents the experiments conducted to evaluate the performance of our approach compared with the state-of-the-art works. The goal is to show the ability of DBPS in producing accurate and realistic simulations from various perspectives.

Accuracy evaluation focuses on computing distances between the distributions observed in the simulated event logs and the real ones [8]. We compared our results

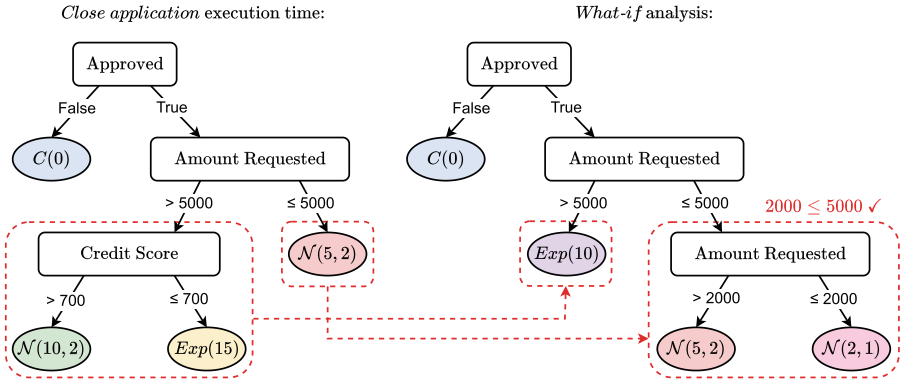


Fig. 3. User interaction example on the decision tree illustrated in Fig. 2. $C(t)$ indicates a constant distribution with value t, $Exp(t)$ the exponential distribution with mean t, $\mathcal{N}(\mu, \sigma)$ the normal distribution with mean μ and standard deviation σ. Rules are modified by removing a decision tree and adding a new one in a different branch. The new rule satisfies the constraint of the parent node ($2000 < 5000$).

with those obtained by state-of-the-art methods (cf. Sect. 2), showing that our approach outperforms the white-box simulation model Simod [9], and often achieves competitive or superior performance compared to the black-box methods [5, 17].

Beyond the overall simulation accuracy, our method also demonstrates high precision in reproducing decision behavior, thanks to explicit modeling of decision rules. Furthermore, our simulations exhibit greater generalization and variability, key indicators of realistic behavior, as evidenced by the entropy analysis.

4.1 Experimental Setup

The simulator has been implemented as a Python package: the user can manually define each element of the DBPS or can discover one by giving an input event log and/or a Petri net model.[1]

We temporally split each event log into train and test sets with respectively 80% and 20% of the total traces. The train set is used to discover the simulation parameters as described in Sect. 3.3, while the test set is employed for the final evaluation.

To ensure a fair evaluation of the different methods, we used the same control-flow model as the backbone of the simulation models of the different methods. Specifically, we employed the models used in [5] and [17].

Decision trees have been discovered using the CART algorithm. To maximize accuracy, we performed a 3-fold cross-validation with hyperparameter optimization. Specifically, we defined the hyperparameter search space for the maximum tree depth by varying its values in the range 1 to 5. In fact, the more terminal nodes and the deeper the tree, the more difficult it becomes to understand the decision rules of a tree [18].

[1] https://github.com/franvinci/prosit.

4.2 Processes and Event Logs

To assess the accuracy of our approach, we discovered the DBPS models (cf. Sect. 3.3) using available event logs from five different business processes:

- Purchase to Pay (P2P), which registers executions of a purchasing process.[2]
- CVS retail pharmacy (CVS), which focuses on a retail pharmacy process.[3]
- Academic Credentials Recognition (ACR), which records the executions process at the University of Los Andes in Colombia (see footnote 3).
- BPIC12W, which captures the execution of the subprocess for the workflow-relevant activities, namely those starting with W, of a loan application process run by a Dutch financial institution in 2011.[4]
- BPIC17, which refers to the same sub-process as BPC12W, but related to the executions in 2016, when the process underwent some significant changes.[5]

The use cases include two synthetic and three real-life event logs. The BPIC12W and BPIC17W event logs also contain trace attributes that are used in the simulations.

4.3 Accuracy Evaluation

The goal of the accuracy evaluation is to assess how realistic the simulation conducted with our method is and to compare it with Simod [9], which also uses white-box models, as well as with RIMS [17] and DeepSimulator (DSim) [5], which are hybrid BPS methods that leverage deep learning techniques.

Since Agent Simulator [12] does not have an explicit control-flow model, this method cannot be compared against the others, which conversely have the control-flow model as backbone. After all, Agent Simulator has a distinct goal, focusing on resources as the main process perspective, whereas the others highlight the control-flow perspective.

To assess the accuracy of our proposed method, we conducted simulations and computed distances between the event logs obtained by the different simulation methods and the original ones, which were used to build the BPS models. We used three key distance metrics introduced by Chapela-Campa et al. [8], each targeting a specific perspective of process behavior that aligns with the focus of our approach: **N-Gram Distance** (NGD), with $N = 3$ for measuring the control-flow quality; **Cycle Time Distribution distance** (CTD) to measure the ability of the model for capturing the times of process instances; **Case Arrival Rate distance** (CAR) to computes the difference in the temporal distribution of case arrivals throughout the process timeline.

For each case study, we ran five simulations to obtain event logs containing the same number of traces as the real ones, and computed the average distances between the obtained event logs. Table 2 presents the results of simulation distances across five case studies, comparing our approach (DBPS) with both white-box (Simod [9]) and

[2] https://fluxicon.com/academic/material.
[3] https://zenodo.org/records/4699983.
[4] https://doi.org/10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f.
[5] https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b.

Table 2. Simulation distances results. For each case study, correspondent metric results are reported for each method: Simod [9], our approach DBPS, DSim [5], and RIMS [17]. The best results are highlighted in bold. Cells highlighted in green indicate best results among the white-box methods (i.e. Simod and DBPS). Last rows, denoted with *Avg.*, indicate average results for each metric and method across the case studies.

Case Study	Metric	White-Box		Black-Box	
		Simod [9]	DBPS	DSim [5]	RIMS [17]
P2P	NGD	0.58	**0.35**	0.60	0.59
	CTD	453.62	**432.12**	584.30	578.33
	CAR	731.43	**669.17**	770.54	773.02
ACR	NGD	0.53	**0.23**	0.25	**0.23**
	CTD	716.05	190.08	70.19	**46.28**
	CAR	250.33	**180.24**	236.81	233.66
CVS	NGD	0.58	**0.12**	0.34	0.38
	CTD	269.93	59.90	**52.43**	99.25
	CAR	**5.39**	17.78	20.37	20.26
BPIC12W	NGD	0.79	**0.35**	0.66	0.63
	CTD	191.92	**32.08**	153.37	92.28
	CAR	**28.74**	79.71	97.49	92.74
BPIC17W	NGD	0.81	**0.32**	0.58	0.58
	CTD	148.79	54.97	114.31	**36.53**
	CAR	145.96	53.72	**24.33**	82.29
Avg.	NGD	0.66	**0.28**	0.49	0.48
	CTD	356.06	**153.83**	194.92	170.53
	CAR	232.37	**200.12**	229.91	240.39

black-box (DSim [5], RIMS [17]) methods. Across nearly all metrics and datasets, DBPS consistently outperforms Simod, except for two metrics, establishing it as the most effective white-box method. Notably, our method even matches or comes close to the performance of state-of-the-art black-box methods RIMS and DSim.

Although both Simod [9] and RIMS [17] incorporate decision models within the control-flow, our approach demonstrates superior effectiveness. This can be attributed to two key factors: (1) unlike Simod and RIMS, which rely solely on event attributes, our method also leverages process history as a predictive feature; and (2) we model transition behavior using weighted probabilities on transitions, allowing us to capture priority in concurrent executions (see Example 1), while Simod and RIMS apply decision trees only at XOR gateways.

To further evaluate the accuracy of our method in modeling decision rules, we analyzed the behavior of simulated and real event data at the level of decision tree leaves. For time-related decision trees (i.e., arrival time, execution time, and waiting time; see Sect. 3), we used the Wasserstein distance to compare distributions (respectively indi-

Table 3. Rule-based distance results. Each row reports a case study. Results are grouped by metric (Rule-CFD, Rule-ATD, Rule-ETD, Rule-WTD), with Simod [9] and our approach (DBPS). The best results are highlighted in bold. Averages are reported in the last row.

Case Study	Rule-CFD		Rule-ATD		Rule-ETD		Rule-WTD	
	Simod [9]	DBPS	Simod [9]	DBPS	Simod [9]	DBPS	Simod [9]	DBPS
P2P	0.16	**0.05**	**768.31**	786.06	128.82	**24.76**	**1525.20**	1706.52
ACR	0.28	**0.13**	136.77	**81.43**	**6.22**	10.54	**364.92**	742.67
CVS	0.24	**0.01**	**0.37**	0.60	3.23	**0.70**	1222.63	**515.57**
BPIC12W	0.26	**0.08**	5.26	**2.26**	10.35	**3.57**	471.85	**224.02**
BPIC17W	0.23	**0.08**	2.35	**1.91**	11.71	**8.95**	**827.22**	873.44
Avg.	0.23	**0.07**	182.61	**174.45**	32.07	**9.70**	882.37	**812.44**

cated with Rule-ATD, Rule-ETD, Rule-WTD). For decision trees modeling transition weights, we computed the absolute difference in transition firing frequencies (Rule-CFD). We compared our approach with Simod [9], as DSim [5] and RIMS [17] either do not produce resource information - used as features in some decision trees - or lack the necessary trace attribute (cf. Sect. 3.2). Table 3 summarizes the results across the four rule-based metrics. Our approach (DBPS) outperforms Simod [9] in the vast majority of cases, achieving the lowest average distances in all four metrics. In particular, DBPS shows substantial improvements in Rule-CFD and Rule-ETD, indicating more accurate modeling of control-flow decisions and execution time distributions. These results confirm the enhanced precision of our simulations in replicating the decision logic within the process.

Table 4. Average entropy of execution time distributions. For each case study, correspondent results are reported for each method: Simod [9], our approach DBPS, DSim [5], RIMS [17], and our deterministic approach DBPS-det. Best results are highlighted in bold. Red background is used to indicate the worst results. Last row, denoted with *Avg.*, indicate average results for each metric and method across the case studies.

Case Study	**Probabilistic**		**(Partly) Deterministic**		
	Simod [9]	DBPS	DSim [5]	RIMS [17]	DBPS-det
P2P	1.33	**1.38**	1.16	1.20	0.56
ACR	1.01	**1.15**	0.74	0.59	0.85
CVS	**0.64**	0.36	0.57	0.57	0.08
BPIC12W	1.65	**2.25**	0.56	0.56	0.60
BPIC17W	1.97	**2.21**	0.61	0.61	0.76
Avg.	1.32	**1.47**	0.73	0.71	0.57

We attribute the superior and competitive performance of our method compared to black-box ones to its inherently probabilistic nature. While RIMS [17] and DSim [5]

rely on deterministic deep learning models to generate execution times—often resulting in rigid, overfitted behavior with limited temporal variability—our method leverages probabilistic decision trees to effectively capture uncertainty and reflect the natural variability observed in real-world processes. To further support this claim, we computed the entropy of execution time distributions across all activities on the simulated event logs. Entropy has been widely used in the literature as an indicator of variability and generalization capacity of process data [3]. Higher entropy reflects a greater ability to reproduce the variability found in real processes, which is desirable for realistic simulation. Specifically, since execution times are continuous variables, we approximated their entropy by discretizing the observed simulated values into bins $b_1, \ldots, b_k$. The optimal number of bins is based on the Sturges' rule, which computes the number of bins as $k = \lceil \log_2(n) + 1 \rceil$, where n is the number of observed samples. Then the entropy of a probability distribution p^a of execution time of activity a is computed as $H(p^a) = -\sum_{i=1}^{k} p_i^a \log(p_i^a)$, where p_i^a expresses the probability of a value being in the bin b_i. The average execution time entropy is then obtained as $\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} H(p^a)$, with $\mathcal{A}$ the set of possible activities.

Table 4 reports the average execution time entropy values, with methods grouped into *Probabilistic* and *Deterministic* categories. We also include DBPS-det, a deterministic variant of our method that replaces probabilistic decision trees with deterministic ones. As expected, all deterministic approaches exhibit substantially lower entropy, confirming their limited ability to capture temporal variability. This statement is particularly confirmed if we compare the entropy values of our approach DBPS with DBPS-det: the simple replacement in the latter of probabilistic decision trees with deterministic ones causes a significant drop in the entropy values, further pointing out that the improvement is certainly related to being probabilistic. Our approach (DBPS) achieves higher entropy on average and across all case studies, except for CVS. The CVS dataset warrants special attention. As noted by its authors, it is synthetically generated, likely without emphasizing behavioral variability. Consequently, it exhibits limited behavior variability, which explains the relatively lower entropy observed in this case.

These findings underscore the effectiveness of our method in generating behaviorally accurate, realistic, and diverse simulations while maintaining full interpretability. We consistently outperform the white-box Simod [9], not only in overall simulation accuracy but also in faithfully modeling decision rules. Moreover, our method achieves competitive accuracy when compared to black-box state-of-the-art approaches, while offering greater realism by capturing a higher degree of behavioral variability, as demonstrated by the entropy analysis.

5 Conclusion

The simulation of business processes that involve human resources and/or services is based on a model that describes how activities are executed, along with a characterization of how the perspectives on data, resource, and time are simulated. This characterization refers to new process-instance arrival times, activity durations, resource assignment, data-dependent probabilistic guards, etc.

While several approaches exist for discovering business process simulation models and characterizing their run-time behavior, most rely on deep learning and/or deterministic models. Deep learning models are typically black-boxes and do not expose their decision logic in an interpretable, analytical form. This limits their configurability and hinders the ability to perform meaningful what-if analyses. Deterministic models, in turn, fail to capture the inherent stochasticity and uncertainty of real-world processes.

This paper proposes a novel approach that uses probabilistic decision trees for run-time characterization. These decision models are both white-box, allowing for easy interpretation and configurability, and stochastic, enabling the simulation of natural process variability.

Our technique has been compared with those existing related techniques that are based on black-box and/or deterministic models as well as with the only technique from the state of the art that is based on stochastic, white-box models. Five different processes were used in the evaluation, and the results outline that our method enables running business process simulations that are more realistic than and generalize more than these existing techniques.

We acknowledge that the current evaluation is largely quantitative on the ability of decision trees to adequately characterize the run-time characterization of business simulation. As future work, we aim to assess the human usability and interpretability of stochastic decision trees to conduct *what-if* analyses. So we plan to investigate the trade-off between usability and interpretability, on the one hand, and decision tree depth, on the other hand: we hypothesize that deeper trees, while potentially more accurate, may hinder users' ability to understand and effectively utilize them for *what-if* analyses. We plan to conduct user studies with process experts using consolidated techniques for human-computer interaction and cognitive psychology.

Acknowledgments. F. Vinci is financially supported by MUR (PNRR) and University of Padua. M. de Leoni is supported by the European Union – Next Generation EU under the National Recovery and Resilience Plan (NRRP), Mission 4 Component 2 Investment 1.1 - Call PRIN 2022 PNRR No. 1409 of September 14, 2022 of Italian Ministry of University and Research; Project P20222XM58 (subject area: SH) *Emergency medicine 4.0: an integrated data-driven approach to improve emergency department performances*.

References

1. Aalst, W.: Process Modeling and Analysis. In: Aalst, W. (ed.) Process Mining, pp. 55–88. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49851-4_3
2. Alaee, A.J., Weidlich, M., Senderovich, A.: Data-driven decision support for business processes: causal reasoning and discovery. In: Marrella, A., Resinas, M., Jans, M., Rosemann, M. (eds.) BPM 2024. LNBIP, vol. 526, pp. 90–106. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70418-5_6
3. Back, C.O., Debois, S., Slaats, T.: Entropy as a measure of log variability. J. Data Semant. **8**, 129–156 (2019)
4. Bartsch, C., Mevius, M., Oberweis, A.: Simulation of it service processes with petri-nets. In: Service-Oriented Computing – ICSOC 2008 Workshops, pp. 53–65 (2009)

5. Camargo, M., Báron, D., Dumas, M., González-Rojas, O.: Learning business process simulation models: a hybrid process mining and deep learning approach. Inf. Syst. **117**, 102248 (2023)
6. Camargo, M., Dumas, M., González-Rojas, O.: Automated discovery of business process simulation models from event logs. Decis. Support Syst. **134**, 113284 (2020)
7. Camargo, M., Dumas, M., Rojas, O.G.: Discovering generative models from event logs: data-driven simulation vs deep learning. PeerJ Comput. Sci. **7** (2021)
8. Chapela-Campa, D., Benchekroun, I., Baron, O., Dumas, M., Krass, D., Senderovich, A.: A framework for measuring the quality of business process simulation models. Inf. Syst. **127**, 102447 (2025)
9. Chapela-Campa, D., López-Pintado, O., Suvorau, I., Dumas, M.: Simod: automated discovery of business process simulation models. SoftwareX **30**, 102157 (2025)
10. Dumas, M., La Rosa, M., Mendling, J., Reijers, H.A.: Fundamentals of Business Process Management, 2nd edn. Springer, Cham (2018)
11. Henkel, M., Zdravkovic, J., Johannesson, P.: Service-based processes: design for business and technology. In: Proceedings of the 2nd International Conference on Service Oriented Computing, ICSOC 2004, pp. 21–29 (2004)
12. Kirchdorfer, L., Blümel, R., Kampik, T., Van der Aa, H., Stuckenschmidt, H.: AgentSimulator: an agent-based approach for data-driven business process simulation. In: 2024 6th International Conference on Process Mining (ICPM), pp. 97–104 (2024)
13. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs - a constructive approach. In: Colom, J.-M., Desel, J. (eds.) PETRI NETS 2013. LNCS, vol. 7927, pp. 311–329. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38697-8_17
14. de Leoni, M., Vinci, F., Leemans, S.J.J., Mannhardt, F.: Investigating the influence of data-aware process states on activity probabilities in simulation models: does accuracy improve? In: Di Francescomarino, C., Burattin, A., Janiesch, C., Sadiq, S. (eds.) BPM 2023. LNCS, vol. 14159, pp. 129–145. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-41620-0_8
15. López-Pintado, O., Dumas, M.: Discovery and simulation of business processes with probabilistic resource availability calendars. In: 2023 5th International Conference on Process Mining (ICPM), pp. 1–8 (2023)
16. López-Pintado, O., Dumas, M., Berx, J.: Discovery, simulation, and optimization of business processes with differentiated resources. Inf. Syst. **120**, 102289 (2024)
17. Meneghello, F., Di Francescomarino, C., Ghidini, C., Ronzani, M.: Runtime integration of machine learning and simulation for business processes: time and decision mining predictions. Inf. Syst. **128**, 102472 (2025)
18. Molnar, C.: Interpretable Machine Learning, 3rd edn. (2025)
19. Pourbafrani, M., van der Aalst, W.M.P.: Interactive process improvement using simulation of enriched process trees. In: Hacid, H., et al. (eds.) ICSOC 2021. LNCS, vol. 13236, pp. 61–76. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-14135-5_5
20. Rozinat, A., Mans, R., Song, M., van der Aalst, W.: Discovering simulation models. Inf. Syst. **34**(3), 305–327 (2009)
21. Wang, J., Lu, C., Yu, Y., Cao, B., Fang, K., Fan, J.: HiGPP: a history-informed graph-based process predictor for next activity. In: Service-Oriented Computing, pp. 337–353 (2025)

MALL: A Mamba-Based Autoencoder Enhanced by an LLM for Multi-Perspective Business Process Anomaly Detection

Zhihao Pei[1] and Guiling Li[1,2(✉)]

[1] School of Computer Science, China University of Geosciences, Wuhan, China
zhihaopei@cug.edu.cn
[2] Hubei Key Laboratory of Intelligent Geo-Information Processing, China University of Geosciences, Wuhan, China
guiling@cug.edu.cn

Abstract. Effective anomaly detection in business process event logs is critical for maintaining operational integrity and ensuring process compliance. If left unchecked, anomalies from system malfunctions or unexpected interventions can cause significant financial losses and undermine subsequent process analysis. However, existing methods for multi-perspective business process anomaly detection often struggle to capture the rich semantics of event logs. Moreover, current attempts to leverage Large Language Models (LLMs) primarily focus on control flow, overlooking other important perspectives. In this paper, we introduce MALL, a Mamba-based Autoencoder enhanced by an LLM for multi-perspective business process anomaly detection. MALL leverages an LLM to create semantically rich embeddings, while using a Mamba-based architecture as its sequential backbone. Notably, its key innovations lie in a bidirectional Mamba-based encoder that captures full contextual information, and a specialized decoder where the control flow perspective guides the modeling of other attributes for more nuanced detection. Our comprehensive experiments on both synthetic and real-world datasets demonstrate that MALL significantly outperforms existing methods.

Keywords: LLM · Mamba · Anomaly detection · Process mining

1 Introduction

In the digital era, companies widely adopt Business Process Management (BPM) to enhance efficiency, but the occurrence of anomalies can significantly disrupt the expected process, making their detection crucial [8]. Numerous studies have been conducted on business process anomaly detection (AD). Among them, deep learning methods have gained significant attention [7,15]. However, these methods treat attribute values as discrete IDs and overlook semantic information in event logs, limiting their generalization in unsupervised anomaly detection.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 353–361, 2026.
https://doi.org/10.1007/978-981-95-5015-9_25

The rapid advances in large language models (LLMs) have made it increasingly feasible to capture the semantic information embedded in event logs. However, existing attempts to leverage LLMs for business process anomaly detection [1,2,4] primarily focus on control flow, overlooking other important perspectives. Moreover, these methods are typically designed for general-purpose scenarios and fail to account for the unique characteristics of individual event logs, often relying solely on common-sense knowledge to detect anomalies.

To address these challenges, we propose a **M**amba-based **A**utoencoder enhanced by an **LL**M for multi-perspective business process anomaly detection, named MALL. MALL leverages a pre-trained LLM to encode rich semantic information from business processes, enhancing the model's robustness in unsupervised anomaly detection scenarios. To capture sequential dependencies, we adopt Mamba [3]—a state space model (SSM)-based architecture—as the backbone. Within the encoder, we introduce a bidirectional Mamba layer that processes input sequences in both forward and backward directions, enabling the capture of both past and future contexts for more expressive representations. Furthermore, recognizing the influence of control flow on other perspectives, we explicitly incorporate the control flow perspective into the reconstruction of attribute values in the decoder. Experiments on synthetic and real-world datasets show that MALL outperforms existing methods.

2 Related Work

Deep Unsupervised Business Process Anomaly Detection. Most existing studies use autoencoders and detect anomalies via trace reconstruction errors. DAEs are applied in [13], while VAEs are used in [12]. To capture sequential patterns, some works integrate sequence encoders into autoencoders, such as RNNs [5,6,14,15], GNNs [6,9] and Transformers [7]. Existing unsupervised methods often treat attribute values as discrete IDs, ignoring their semantic meaning (e.g., in activity names). Additionally, RNNs suffer from vanishing gradients, while GNNs and Transformers are computationally expensive. In contrast, our MALL leverages LLMs for semantic understanding and employs the Mamba architecture to efficiently capture long-term dependencies.

LLMs for Business Process Anomaly Detection. Several studies have begun exploring the use of LLMs for anomaly detection in business processes. For instance, Guan et al. [4] fine-tune LLaMA 2 to detect semantic deviations, while Derakhshan et al. [2] apply GPT-4o to identify rework anomalies. Caspary et al. [1] represent traces as activity pairs and use BERT to check the semantic validity of activity orders. However, these methods focus only on control flow and neglect other perspectives. In contrast, MALL integrates multiple perspectives and leverages LLMs' semantic understanding to capture log-specific patterns more effectively.

3 Methodology

To formally ground our approach, we first consider a collection of attributes $\mathcal{A} = \{a_1, a_2, \ldots, a_A\}$, comprising A distinct attributes. For any given attribute $a \in \mathcal{A}$, there exists a domain $\mathcal{V}_a$ encompassing all permissible values for that attribute. Each **event** e is characterized by an ordered tuple $e = [v_{a_1}, v_{a_2}, \ldots, v_{a_A}]$, where each $v_{a_i} \in \mathcal{V}_{a_i}$ is an attainable value for the corresponding attribute a_i. A sequence of events then forms an event **trace**, denoted as t. The **event log**, denoted as C, consists of a collection of traces, with the total number of traces represented as $|C|$. Furthermore, all possible values for an attribute a are mapped to unique positive integers (ID) via a map function f: $\mathcal{V}_a \to \mathbb{N}$. For an event e within trace t, $t_{e,a}$ denotes the unique ID associated with attribute a of event e.

We artificially add a start and an end event to the trace. The start event, represented by the symbol '▶', is appended to the original sequence, while the end event, represented by '■', is appended. For the augmented trace, assuming the original trace length is $E - 1$, we add a start event at index 0 and an end event at index E. The attribute values for these events are consistently assigned: for any attribute $a \in \mathcal{A}$, $t_{0,a}$ = '▶' and $t_{E,a}$ = '■'. By convention, we designate the first attribute of each event as '*Activity*', which serves as the control flow.

3.1 Model Architecture

Extraction of LLM Embeddings. We generate the LLM embedding matrix for attribute a, denoted as $E_{\text{llm}}^{(a)} \in \mathbb{R}^{|\mathcal{V}_a| \times d_{\text{llm}}}$, by feeding every textual attribute value from the vocabulary $\mathcal{V}_a$ into an LLM, where d_{llm} denotes the hidden size of the LLM. The j-th row of $E_{\text{llm}}^{(a)}$ corresponds to the embedding produced by the LLM for the j-th textual attribute value in $\mathcal{V}_a$. Once $E_{\text{llm}}^{(a)}$ is obtained—used to initialize the semantic embedding layer shown in Fig. 1—the LLM is no longer involved in subsequent model training or inference, thereby reducing computational overhead and simplifying the overall architecture.

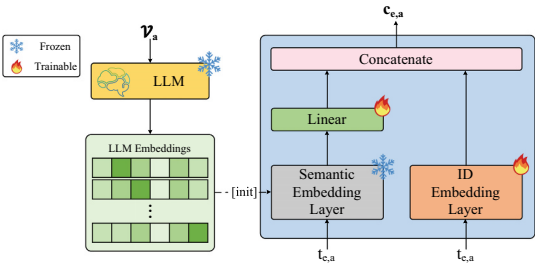
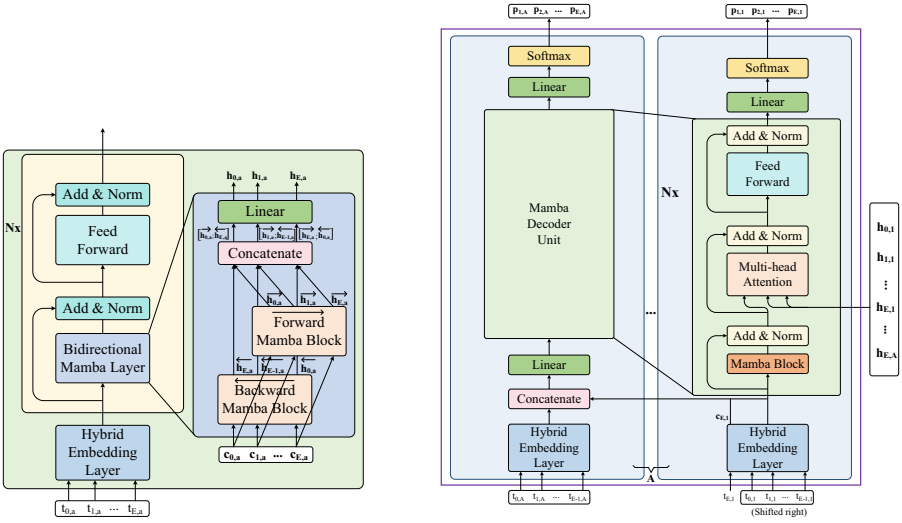


Fig. 1. Hybrid embedding layer.

Hybrid Embedding Layer. Our *hybrid embedding layer*, illustrated in Fig. 1, is designed to generate an embedding that captures both the semantic meaning and intrinsic statistical patterns of each attribute value $t_{e,a}$. The *semantic embedding* for $t_{e,a}$ is generated by passing it through an LLM-initialized embedding layer, which is frozen to preserve semantics, followed by a linear projection that reduces the dimension from d_{llm} to $\frac{d}{2}$. The *ID embedding* is obtained by feeding $t_{e,a}$ into a separate ID embedding layer. This embedding is trainable. Both the semantic and ID embeddings have a dimensionality of $\frac{d}{2}$. Finally, the hybrid embedding $\mathbf{c}_{e,a} \in \mathbb{R}^d$ is formed by concatenating the semantic and ID embeddings.

Encoder. Given that a trace with multiple attributes can be viewed as a collection of interdependent sequences, the encoder is designed as a configuration of A parallel sequence encoders, one for each attribute $a \in \mathcal{A}$. The sequence encoder for each attribute a, illustrated in Fig. 2a, begins by converting the input sequence $\langle t_{0,a}, t_{1,a}, \cdots, t_{E,a} \rangle$ into a sequence of hybrid embeddings $\langle \mathbf{c}_{0,a}, \mathbf{c}_{1,a}, \cdots, \mathbf{c}_{E,a} \rangle$. This embedding sequence is then processed by a stack of N identical processing blocks designed for deep contextual modeling. Each block is composed of a bidirectional Mamba layer and a Feed-Forward Network (FFN), with both sub-layers followed by an Add & Norm operation. Within the Mamba layer, forward states ($\overrightarrow{\mathbf{h}_{i,a}}$) and backward states ($\overleftarrow{\mathbf{h}_{i,a}}$) are generated and combined via reverse-aligned concatenation and a linear projection. The FFN then provides further non-linear refinement. The final output sequence from the stack, denoted as $\langle \mathbf{h}_{0,a}, \mathbf{h}_{1,a}, \ldots, \mathbf{h}_{E,a} \rangle$, serves as the contextualized representation.



(a) The encoder for each attribute (b) The decoder

Fig. 2. The architectures of the Encoder (left) and Decoder (right).

Decoder. Our decoder employs a parallel architecture, as depicted in Fig. 2b, to separately handle the two primary attribute types found in business processes: control flow (*Activity*) and other attributes. This design uses dedicated structures for each category, with the overall goal of generating an interpretable probability distribution $\mathbf{p}_{e,a}$ for each attribute a within each event e. We first introduce the Mamba decoder unit, which serves as the core component of our decoder. Depicted in the light green area of Fig. 2b, its architecture is composed of three sequential sub-layers, each with residual connections and layer normalization. First, a Mamba block captures auto-regressive dependencies. Next, a multi-head cross-attention module injects global context, sourcing its Key and Value pairs from the encoder outputs $\langle \mathbf{h}_{0,1}, \ldots, \mathbf{h}_{E,A} \rangle$. Finally, an FFN enhances the context-enriched representations. The decoder for *control flow (Activity)*, as shown on the right side of Fig. 2b, processes a right-shifted input sequence $\langle t_{0,1}, \ldots, t_{E-1,1} \rangle$ by first generating hybrid embeddings $\langle \mathbf{c}_{0,1}, \ldots, \mathbf{c}_{E-1,1} \rangle$, and then passing them through a stack of N Mamba decoder units and a final prediction head to produce the output probability distributions $\langle \mathbf{p}_{1,1}, \ldots, \mathbf{p}_{E,1} \rangle$. The decoder for *each non-activity attribute* $a \in \mathcal{A}$, as shown on the left side of Fig. 2b, processes an input sequence $\langle t_{0,a}, \ldots, t_{E-1,a} \rangle$ by first creating hybrid embeddings, then enriching them by concatenating with corresponding activity embeddings (e.g., $[\mathbf{c}_{e,1}; \mathbf{c}_{e-1,a}]$), projecting the resulting $2d$-dimensional vectors to dimension d, and finally passing them through a stack of N Mamba decoder units and a prediction head to yield the final probability distributions $\langle \mathbf{p}_{1,a}, \ldots, \mathbf{p}_{E,a} \rangle$.

3.2 Training and Anomaly Detection

The training procedure involves minimizing the total cross-entropy loss over the dataset C. The following equation defines the loss function $\text{Loss}(C, P) = -\sum_{t \in C} \sum_{e=1}^{E} \sum_{a=1}^{A} \log {}^{t}\mathbf{p}_{e,a}[t_{e,a}]$, where ${}^{t}\mathbf{p}_{e,a}[t_{e,a}]$ represents the probability of the attribute a associated with event e in trace t. It's important to note that the summation over events begins at $e = 1$, as the virtual start event (i.e., $\forall a \in \mathcal{A}, t_{0,a} =$'▶') serves as the initial input but is not a target for reconstruction.

Based on the predictive output, we define an anomaly score $S_{t,e,a}$ to formally measure the anomalousness of each observed attribute value. We define the anomaly score as $S_{t,e,a} = \text{Sigmoid}(\sum_{p_i \in \mathbf{p}_{e,a}} p_i \log p_i - \log {}^{t}\mathbf{p}_{e,a}[t_{e,a}])$.

4 Experiments

Our experiments follow an unsupervised setup, with no clean data or domain knowledge. The source code is available at https://github.com/GODPG/MALL.

4.1 Experimental Settings

i) Datasets. For the synthetic logs, we utilize six distinct process models: *Small*, *Large*, *Huge*, *Wide*, *P2P*, and *Paper*. For our real-life logs, we select several

widely recognized datasets: *Open (Open problems)*, *BPIC15*, *BPIC17*, *Intern (International Declarations)*, *Receipt*, *Sepsis*. Artificial anomalies were injected into the logs at various rates; implementation details are available on GitHub.

ii) Benchmark Methods. We benchmark our model against several state-of-the-art approaches, including distance-based methods like W2V-LOF [10] and VAE-OCSVM [11]; autoencoder-based methods like GAE [9], DAE [13], VAE [12], LAE [12], GRASPED [5], BINetv2 [14,15] and BINetv3 [14].

iii) Implementation Details. In our experiments, all models were trained for 20 epochs with a batch size of 64 using the Adam optimizer with a learning rate of 0.0002 and betas (β_1, β_2) of (0.5, 0.999). Our 4-layer encoder and 2-layer decoder architecture utilizes the pre-trained `all-mpnet-base-v2` model for semantic embeddings. Key Mamba parameters were a state dimension of 16, a convolution kernel of 4, and an expansion factor of 2. The hidden dimension d was set to 48 for synthetic logs, but for real-life logs, it was dynamically adjusted to 16, 64, or 160 based on whether the maximum attribute cardinality, denoted as $C_{\max}$, was below 40, between 40 and 300, or above 300, respectively.

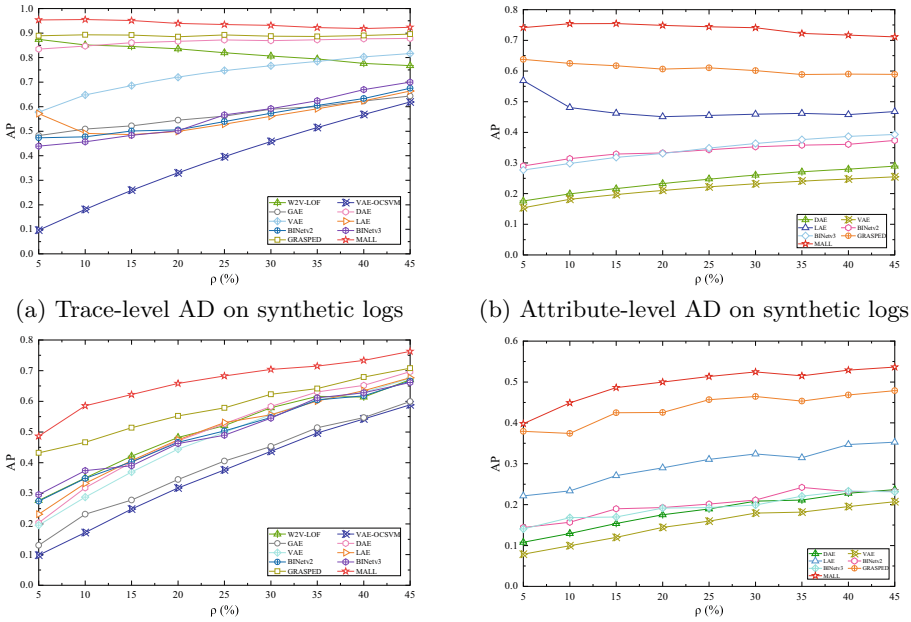
iv) Metrics. To evaluate model performance, we employ the F-score.

Table 1. F-score over synthetic and real-life logs.

Logs / Methods	Small		Large		Huge		Wide		P2P		Paper	
	T	A	T	A	T	A	T	A	T	A	T	A
W2V-LOF	0.829	-	0.833	-	0.804	-	0.808	-	0.796	-	0.846	-
VAE-OCSVM	0.382	-	0.383	-	0.378	-	0.379	-	0.379	-	0.381	-
GAE	0.526	-	0.593	-	0.475	-	0.641	-	0.614	-	0.535	-
DAE	0.910	0.261	0.836	0.216	0.804	0.216	0.876	0.255	0.867	0.257	0.892	0.244
VAE	0.781	0.211	0.749	0.205	0.625	0.206	0.725	0.246	0.667	0.226	0.820	0.200
LAE	0.596	0.510	0.511	0.430	0.501	0.394	0.534	0.447	0.586	0.529	0.618	0.533
BINetv2	0.563	0.362	0.541	0.329	0.552	0.342	0.527	0.323	0.581	0.346	0.557	0.335
BINetv3	0.578	0.365	0.564	0.340	0.538	0.339	0.554	0.344	0.557	0.336	0.564	0.338
GRASPED	0.921	0.593	0.883	0.646	0.852	0.636	0.839	0.552	0.910	0.582	0.934	0.635
MALL	**0.959**	**0.776**	**0.934**	**0.734**	**0.923**	**0.697**	**0.891**	**0.696**	**0.947**	**0.745**	**0.966**	**0.775**
	Open		BPIC15		BPIC17		Intern		Receipt		Sepsis	
W2V-LOF	0.392	-	0.411	-	0.475	-	0.615	-	0.694	-	0.433	-
VAE-OCSVM	0.271	-	0.389	-	0.383	-	0.385	-	0.377	-	0.380	-
GAE	0.343	-	0.389	-	0.383	-	0.420	-	0.413	-	0.387	-
DAE	0.444	0.183	0.402	0.113	0.414	0.234	0.586	0.209	0.648	0.170	0.495	0.184
VAE	0.443	0.191	0.400	0.101	0.404	0.135	0.488	0.154	0.615	0.147	0.491	0.181
LAE	0.463	0.211	0.408	0.244	0.386	0.169	0.702	0.447	0.485	0.385	0.519	0.320
BINetv2	0.282	0.087	0.392	0.023	0.667	0.370	0.775	0.521	0.441	0.141	0.396	0.060
BINetv3	0.298	0.076	0.391	0.018	0.665	0.366	0.769	0.482	0.447	0.157	0.400	0.070
GRASPED	0.431	0.379	0.471	0.391	0.735	0.533	0.748	0.511	0.624	0.479	0.454	0.324
MALL	**0.563**	**0.410**	**0.525**	**0.437**	**0.801**	**0.543**	**0.813**	**0.583**	**0.720**	**0.576**	**0.545**	**0.420**

4.2 Overall Results

Table 1 shows anomaly detection results on synthetic (top part) and real-life (bottom part) logs. MALL consistently achieves the highest F-score at trace and attribute levels, thanks to the combination of LLM-based semantic representations and Mamba's ability to capture long-range dependencies.



(a) Trace-level AD on synthetic logs

(b) Attribute-level AD on synthetic logs

(c) Trace-level AD on real-life logs

(d) Attribute-level AD on real-life logs

Fig. 3. F-score under different anomaly ratios (ρ).

4.3 Impact of Anomaly Ratio

Figure 3 illustrates the impact of anomaly ratio, with MALL consistently outperforming all other methods. While its performance slightly decreases with higher anomaly ratios in synthetic logs, MALL maintains the highest F-scores overall, demonstrating both robustness and scalability.

4.4 Ablation Study

An ablation study (Table 2) evaluates the full MALL model against two variants: without semantic embedding (w/o Semantic) and without ID embedding (w/o ID). By introducing semantic embeddings, MALL shows only limited improvement on synthetic logs, primarily due to their simple attribute values

Table 2. Ablation study. The best results are shown in bold.

Methods \ Logs	Synthetic Logs		Real-life Logs	
	Trace-level	Attribute-level	Trace-level	Attribute-level
w/o Semantic	0.927	0.712	0.640	0.460
w/o ID	0.934	0.721	0.615	0.422
MALL	**0.937**	**0.737**	**0.661**	**0.495**

(e.g., "A", "B"). In contrast, on real-life logs with meaningful attribute values, semantic embeddings lead to larger gains, highlighting their effectiveness in capturing semantic information. The inferior performance of the w/o ID variant demonstrates that relying solely on semantic embeddings is suboptimal. The semantic embedding contributes a semantic understanding of business process, while the ID embedding captures in-domain, process-specific patterns. Their combination proves essential for robust and effective anomaly detection.

5 Conclusion

In this paper, we propose MALL, a Mamba-based autoencoder enhanced by an LLM for multi-perspective business process anomaly detection. MALL harnesses the deep semantic understanding of LLM and the robust capabilities of the Mamba architecture in efficiently handling long sequential data. Extensive experiments conducted on both synthetic and real-world datasets demonstrate that MALL achieves state-of-the-art performance.

Acknowledgments. The work is supported by Open Research Project of the Hubei Key Laboratory of Intelligent Geo-Information Processing (No. KLIGIP-2023-B02) and Open Research Fund of Teaching Laboratory, China University of Geosciences, Wuhan, China (No. SKJ2024206).

References

1. Caspary, J., Rebmann, A., van der Aa, H.: Does this make sense? Machine learning-based detection of semantic anomalies in business processes. In: International Conference on Business Process Management, pp. 163–179. Springer, Cham (2023)
2. Derakhshan, M., Ceravolo, P., Mohammadi, F.: Leveraging GPT-4o efficiency for detecting rework anomaly in business processes. arXiv preprint arXiv:2502.06918 (2025)
3. Gu, A., Dao, T.: Mamba: linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752 (2023)
4. Guan, W., Cao, J., Gao, J., Zhao, H., Qian, S.: Dabl: detecting semantic anomalies in business processes using large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, pp. 11735–11744 (2025)

5. Guan, W., Cao, J., Gu, Y., Qian, S.: Grasped: a GRU-AE network based multi-perspective business process anomaly detection model. IEEE Trans. Serv. Comput. **16**(5), 3412–3424 (2023)
6. Guan, W., Cao, J., Gu, Y., Qian, S.: Gama: a multi-graph-based anomaly detection framework for business processes via graph neural networks. Inf. Syst. 102405 (2024)
7. Guan, W., Cao, J., Yao, Y., Gu, Y., Qian, S.: Comb: interconnected transformers-based autoencoder for multi-perspective business process anomaly detection. In: 2024 IEEE International Conference on Web Services (ICWS), pp. 1115–1124. IEEE (2024)
8. Guan, W., Cao, J., Zhao, H., Gu, Y., Qian, S.: Survey and benchmark of anomaly detection in business processes. IEEE Trans. Knowl. Data Eng. (2024)
9. Huo, S., Völzer, H., Reddy, P., Agarwal, P., Isahagian, V., Muthusamy, V.: Graph autoencoders for business process anomaly detection. In: International Conference on Business Process Management, pp. 417–433. Springer, Cham (2021)
10. Junior, S.B., Ceravolo, P., Damiani, E., Omori, N.J., Tavares, G.M.: Anomaly detection on event logs with a scarcity of labels. In: 2020 2nd International Conference on Process Mining (ICPM), pp. 161–168. IEEE (2020)
11. Krajsic, P., Franczyk, B.: Variational autoencoder for anomaly detection in event data in online process mining. In: ICEIS (1), pp. 567–574 (2021)
12. Nguyen, H.T.C., Lee, S., Kim, J., Ko, J., Comuzzi, M.: Autoencoders for improving quality of process event logs. Expert Syst. Appl. **131**, 132–147 (2019)
13. Nolle, T., Luettgen, S., Seeliger, A., Mühlhäuser, M.: Analyzing business process anomalies using autoencoders. Mach. Learn. **107**(11), 1875–1893 (2018). https://doi.org/10.1007/s10994-018-5702-8
14. Nolle, T., Luettgen, S., Seeliger, A., Mühlhäuser, M.: Binet: multi-perspective business process anomaly classification. Inf. Syst. **103**, 101458 (2022)
15. Nolle, T., Seeliger, A., Mühlhäuser, M.: Binet: multivariate business process anomaly detection using deep learning. In: International Conference on Business Process Management, pp. 271–287. Springer, Cham (2018)

Quantum Service Computing

AI-Driven Resource Optimization of Quantum Service Computing

Javier Romero-Álvarez, Jaime Alvarado-Valiente, Antonio Jesus Sanchez-Gil(✉), Enrique Moguel, and Jose Garcia-Alonso

Quercus Software Engineering Group, Universidad de Extremadura, Cáceres, Spain
{jromero,jaimeav,ajsanchezg,enrique,jgaralo}@unex.es

Abstract. Quantum computing is an emerging technology that is increasingly accessed through cloud service platforms. However, the growing demand for quantum computing is constrained by the limited availability of resources, resulting in long waiting times for users and high execution costs, as well as under-utilization of quantum resources. To address this under-utilization, previous research has demonstrated that quantum task scheduling can effectively optimize the usage of quantum cloud platforms, combining multiple quantum circuits into a single execution task. This paper proposes an optimization method, *QCRAFT AI Scheduler*, to schedule quantum circuits making use of Artificial Intelligence algorithms. Specifically, this proposal is based on a Deep Learning model, which is capable of predicting the optimal scheduling of task queues, in order to use the maximum number of qubits of quantum computers. The validation results show that the proposal achieves 97.28% resource utilization in the scheduling iterations and reduces the average execution cost by 95.06%.

Keywords: Quantum Computing · Quantum Computing as a Service · Quantum Software Engineering · Optimization · Deep Learning

1 Introduction

Quantum Computing as a Service (QCaaS) is currently the main model to provide quantum computing capabilities to users [1]. Companies such as IBM, Amazon, Microsoft, and Google are making significant investments in quantum technologies and offer access to quantum computing through their cloud platforms [2]. Although the number of available quantum computers is increasing, as is their capacity in terms of number of qubits, the demand for these resources is also growing. Moreover, the high costs associated with the development and maintenance of quantum computers limit their availability to corporations or institutions. This, combined with the sequential execution they use, where tasks are executed one by one, leads to slow access times and inefficiencies in execution. leads to slow access times and inefficiencies in execution.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 365–379, 2026.
https://doi.org/10.1007/978-981-95-5015-9_26

To execute a task on a quantum computer, users or developers must submit their tasks to be executed via the API of the selected cloud provider. These tasks enter a queue, with wait times ranging from seconds to several hours or even days [3]. This, combined with the high cost of the executions, for example, at IBM they reach $96 per minute of execution[1], means that quantum computing is not accessible to everyone.

Additionally, we can highlight the under-utilization of resources. Despite long queue times, studies show that the average quantum task uses only 10.5 qubits [4]. Considering that there are currently machines with a capacity of 433 qubits and that in the coming years the roadmap of IBM sets the goal of reaching over 1000 qubits, this means that there is a great under-utilization of resources.

Just as classical computing uses techniques to optimize resource usage in multiprocessor systems, where we can find parallelization techniques for execution [5], in quantum computing there are works that address this problem. This is the case of previous work by the authors, where parallelization concepts used in High Performance Computing (HPC) are adopted to carry out the scheduling of different quantum circuits in just one. This scheduling process implies combining multiple quantum circuits into a single execution task submitted to the quantum cloud service. This is achieved by merging the individual circuits, when compatible in terms of qubit requirements and scheduling policy, into a combined circuit that can be executed in a shared context, mainly up to the maximum number of qubits of the processor. This technique, presented as *QCRAFT Scheduler* [6], proposes various scheduling policies to reduce queue lengths and execution costs.

However, the proposed scheduling policies have gaps that often lead to the under-utilization of the capacity of cloud services, highlighting the need for further optimization strategies. This raises the following research question: *is it possible to achieve maximum utilization of quantum hardware by scheduling quantum tasks?* To address this question a new specialized service-aware scheduling mechanisms that take into account the constraints of quantum computers is proposed, the *QCRAFT AI Scheduler*. Specifically it takes the maximum capacity of the selected machine in order to schedule different tasks. This problem is posed as the knapsack problem, where the different elements to be selected are the circuits available in the queue, and the capacity is the number of qubits of the quantum machine to be used. In this way, the problem could be addressed using a variety of different techniques. Among them, Deep Learning models offer stable resolution times once the model has been trained in the face of a growing input set, while classical algorithms such as Dynamic Programming have an increasing and unpredictable computational cost [7,8]. Therefore, we propose to use a Deep Learning model for this purpose.

To clarify our approach, the following sections are structured as follows. Section 2 expands on the motivation of this research and provides an overview of relevant previous studies. Section 3 presents the proposal for the *QCRAFT AI Scheduler*. Following this, Sect. 4 defines the implementation of the Deep Learning model used. Section 6 presents the threat of validity and Sect. 5 validates

[1] https://www.ibm.com/quantum/pricing.

the proposal, analyzing the results obtained after the execution of a test suite of algorithms. Finally, Sect. 7 presents the conclusions drawn from our research and some future works.

2 Background

The field of Quantum Software Engineering is gaining prominence as quantum computing continues to advance [9]. It applies established practices from classical software engineering to the development of quantum applications, leveraging the distinct properties of quantum mechanics to tackle computational problems with improved efficiency [10,11].

An important concept in Quantum Software Engineering is Hybrid Classical-Quantum computing, which combines classical and quantum resources to maximize performance and enable scalable solutions [12]. This integration takes advantage of both paradigms and opens the way for innovative applications across various domains, e.g., optimization, cryptography, cybersecurity. One of the hybrid applications is what is known as quantum services, where concepts from the well-known Service-Oriented Computing are adopted to enable the use of quantum computing through services [10].

As quantum technologies become more widely adopted, efficient resource utilization poses a growing challenge [4]. The execution of quantum circuits is often constrained by long delays [13], and due to the limited availability of quantum processors, users can face significant queueing times, from a few seconds to several hours, which leads to significant delays and reduces system throughput [14]. Furthermore, the current infrastructure is not optimized for resource allocation, an issue that is expected to persist while quantum hardware remains limited [15].

While some research addresses task reduction via scheduling, and others leverage Deep Learning to improve circuit compactness [16], these efforts primarily operate at the gate level and focus on single-circuit transformations. In contrast, our approach aligns with the quantum services paradigm, emphasizing the intelligent scheduling of quantum circuit tasks originating from multiple service consumers. Rather than simplifying internal circuit structures, our focus is on global scheduling strategies that optimize the combined execution flow of quantum service requests, improving resource efficiency at the infrastructure level. Other contributions have expanded the scope of workload optimization in quantum computing. Das et al. [17] investigate multiprogramming by partitioning qubit resources to allow concurrent executions, while Liu et al. [18] propose QuCloud+, a system for efficient mapping and allocation of qubits across applications. Ohkura et al. [19], Seitz et al. [20] explore workload parallelization through circuit assignment and circuit cutting strategies, and Giuseppe et al. [21] propose a distribution of shots on different machines. These solutions focus on improving throughput in high-performance environments by decomposing or distributing workloads. In contrast, our work aims to coordinate circuit submissions from diverse service endpoints, optimizing their collective scheduling without relying on circuit slicing techniques.

Furthermore, AI has been increasingly incorporated to tackle key challenges in quantum computing. Wang and Tang [22] analyze how machine learning models can enhance quantum error correction by increasing adaptability in NISQ devices. Alexeev et al. [23] explore AI integration throughout the quantum software stack, from circuit synthesis to hardware control. More recently, advanced techniques in Reinforcement Learning and Deep Learning have demonstrated promising results in quantum job scheduling. Luo et al. [24] present an RL-based approach for adaptive scheduling in quantum clouds, achieving reduced latency and improved fidelity across varying workloads. Nguyen et al. [25] introduce DRLQ, a method based on Deep Q-Networks and Rainbow architecture, which outperforms traditional heuristics by reducing execution time. Complementarily, Giortamis et al. [26] introduce QOS, a quantum operating system that integrates compilation, scheduling, and multiprogramming.

These works illustrate the potential of AI-driven scheduling to improve quantum execution pipelines. However, our approach differs by focusing on Deep Learning-based models for queue-aware scheduling and the combination of quantum circuit tasks. It aims to optimize execution plans proactively rather than reactively adjusting job order or splitting circuits. Earlier work by the authors [6,27] demonstrated the feasibility of using scheduling policies to reduce both execution cost and task volume. This paper extends that vision by introducing AI as a key part of maximizing the use of available resources wherever possible and intelligent quantum service scheduling across constrained computing resources.

3 Proposal

This section describes the *QCRAFT AI Scheduler*, an AI-driven system designed for quantum circuit scheduling, specifically developed to manage the scheduling process, as illustrated in Fig. 1. The architecture is centered on two main components: the *AI Scheduler* and the *Unscheduler*.

The AI Scheduler component is responsible for analyzing the quantum circuits waiting in the task queue and selecting the optimal combination of circuits to maximize the usage of available qubits on the target quantum computer.

Once the combined circuit has been executed on the selected quantum device, chosen by the developer at the service request, the Unscheduler component processes the results. It splits the output corresponding to each original circuit and assigns them accordingly. These individual results are then stored in a centralized database, from which developers can later retrieve them.

Developers interact with the system by sending their circuit submissions via REST requests to the Scheduler service endpoint. These submissions are automatically queued for scheduling. This architecture offers a flexible and accessible interface that supports the entire lifecycle of quantum circuit execution, from submission and optimal scheduling to result retrieval, making it especially suitable for environments with multiple users and shared quantum resources [27].

In this context, the aim is to achieve the best possible combination among all queued circuits. To this end, the concept of priority has been introduced: the

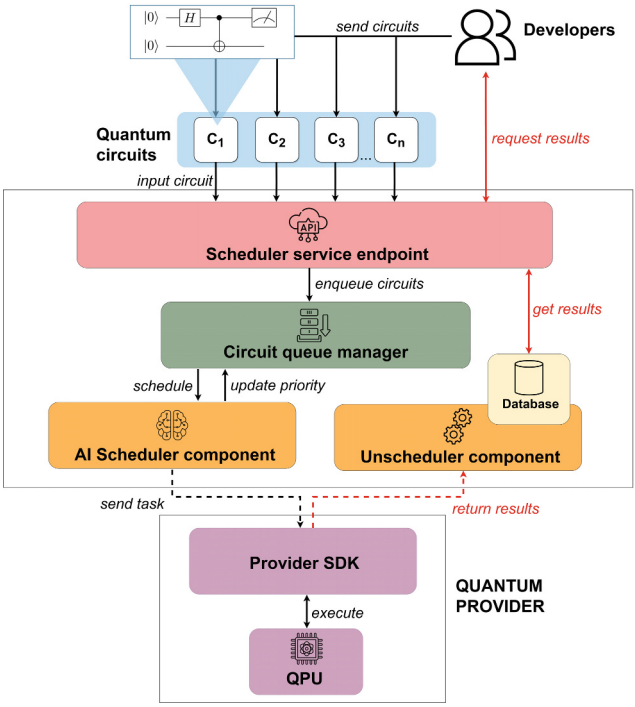


Fig. 1. *QCRAFT AI Scheduler* Architecture.

higher the priority, the greater the probability that a circuit will be selected for scheduling. This mechanism is designed to prevent situations where a circuit, due to its characteristics, could remain indefinitely in the queue. If a circuit exceeds a maximum threshold of scheduling iterations without being selected, the AI Scheduler is designed to mandatorily schedule it in the next iteration. This threshold is defined by the provider that incorporates the AI Scheduler and can be modified at any time.

The AI scheduling process and the functionality of each architectural component are described in detail below. The implementation code and the training data for replication are publicly available in the Zenodo repository[2].

3.1 AI Scheduler Component

The AI Scheduler component is the core module responsible for selecting the optimal combination of circuits available in the queue to maximize the utilization of qubits available in the machine.

To achieve this, it operates directly with two different components: the *Scheduler service endpoint* and the *Circuit queue manager*. Then, it selects the optimal

[2] https://doi.org/10.5281/zenodo.15363453.

combination of circuits for a given capacity. The selection process considers the number of qubits required by each circuit and their current priorities. These two components, which are interconnected to enable the AI Scheduling process, are explained below:

- **Scheduler service endpoint**: It is the interface through which users submit their quantum circuits to the Scheduler. Interacts with the circuit queue to receive, track, and manage circuit submissions. Each time it receives a new execution request, it retrieves the circuit's data (such as the number of qubits used), initializes its priority, assigns a unique identifier, and returns it to the user before enqueueing the circuit.
- **Circuit queue manager**: It receives the circuits submitted by users through the Scheduler service endpoint and stores them until the moment they are scheduled.

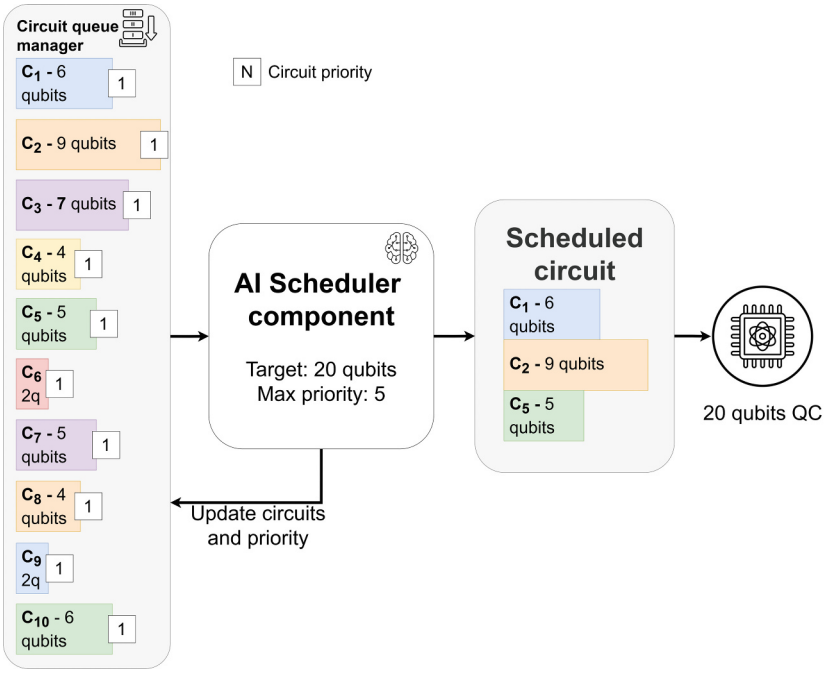


Fig. 2. Scheduling circuits.

Once the circuits to be scheduled are selected, the scheduled circuit is generated, and the selected circuits are removed from the queue. Then, the priority of the circuits that were not scheduled is increased by one unit, and the scheduled circuit is submitted for execution on the quantum computer. This process

is illustrated with an example in Fig. 2 to facilitate understanding of how the AI Scheduler component works.

The selection of the circuits to be scheduled, optimized for the maximum capacity of the machine, is performed as follows.

First, the priority level of each element is evaluated, allowing adjustment of their processing and descending order by priority level. If any circuit in the queue exceeds the defined priority threshold, that circuit must be scheduled in the current scheduling cycle. In the case that there are many circuits exceeding the threshold, and that these cannot fit in a single scheduling, the first ones will be scheduled and the remaining ones will wait for the next cycle. Subsequently, various combinations are generated and analyzed in order to find the one that maximizes the utilization of the available resources, that is, the total number of qubits in the machine.

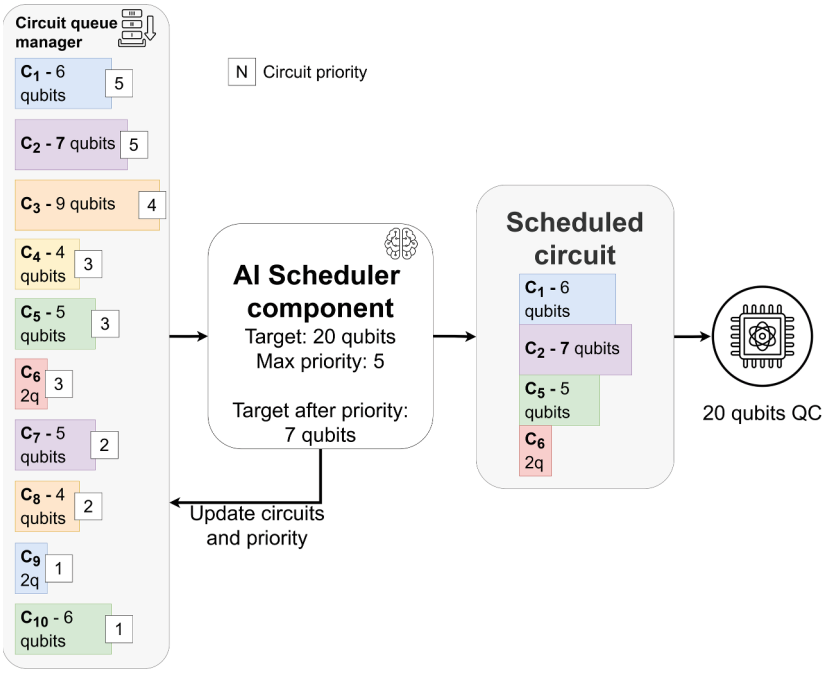


Fig. 3. Scheduling priority circuits.

In the aforementioned case, where at least one circuit in the queue has reached the maximum priority threshold defined in the scheduler, those circuits are scheduled prior to the generation of the circuit combinations. For example, if circuits C1 and C2 require 6 and 7 qubits, respectively and have maximum priority, they are scheduled first, and then the remaining combinations are optimized to maximize the use of the available qubits. In this example, if a 20-qubit quantum

computer is used, after scheduling the two high-priority circuits, the combinations would be generated to optimize the remaining 7 available qubits. This process is depicted in Fig. 3.

3.2 Unscheduler Component

The results obtained from the execution of the scheduled circuit on the quantum computer are a combination of the individual results from each circuit included in the schedule. The Unscheduler component is responsible for separating the obtained results and assigning each circuit in the schedule to its corresponding individual result.

To achieve this, it keeps track of the qubits used by each circuit in the schedule, allowing it to separate the results accordingly. For example, if two circuits, using 3 and 4 qubits, respectively, were scheduled, the first 3 bits of the results would correspond to the first circuit, and the following 4 bits would correspond to the second.

This component also includes a database in which the results of each circuit are stored. It is connected to the *Scheduler service endpoint*, allowing users to retrieve the results via a REST request, indicating the unique identifier generated for their circuits.

4 Deep Learning Model

To train the Deep Learning model used in the *QCRAFT AI Scheduler*, a simulated task execution environment was created to generate representative scheduling data. These data are used to train a Deep Learning model capable of predicting the optimal organization of task queues, thereby maximizing resource utilization.

For the training process, a synthetic dataset was generated to simulate the qubit usage of quantum circuits of varying sizes. The dataset is processed in each of the model training epochs with a classical Dynamic Programming algorithm approach of knapsack, in order to generate different combinations that reach the maximum number of qubits. Each entry in the dataset contains the following information:

- **Number of qubits:** The number of qubits used by the circuit.
- **Priority level:** A numerical value that increases with each iteration in which the circuit is not executed. Once a predefined threshold has been reached, the circuit must be executed in the following iteration.

The trained model consists of a combination of layers that use the Binary Cross Entropy loss function [28], along with the Adam optimizer [29] for parameter updates. The model assigns a probability value, *prob*, to each circuit in the queue, which indicates the probability that a given circuit should be scheduled for the next iteration.

In addition, two complementary functions are computed based on the model output.

On the one hand, it uses a Penalty Factor function to reduce the influence of high-priority elements when there are many of them in the queue. Without this penalty, the optimizer might consistently select only high-priority elements, leaving no room for circuits with lower priority. The function is defined as follows, where *num_high_priority* represents the number of maximum-priority elements in the queue, and *Lambda_penalty* is a tuning coefficient. In this context, the higher the priority, the greater the penalty factor applied.

$$\text{penalty_factor} = 1 + \lambda_{\text{penalty}} \cdot \text{num_high_priority} \tag{1}$$

On the other hand, the model also uses a Profit function, which incorporates the previously calculated Penalty Factor for each element in the queue. This function assigns a score, expressed as a percentage, to each circuit, allowing a more effective and balanced selection process. The function takes the following parameters:

- **Qubits:** The number of qubits used by the circuit.
- **Alpha:** A hyperparameter that amplifies the effect of the probability estimated by the model. The higher the value, the more weight the prediction will have. In this case, after performing different tests with different alpha values, the used value has been 1, in order to give maximum importance to the predictions.
- **Prob:** The probability predicted by the neural network for each element in the queue.
- **Count:** The number of iterations in which the item has remained in the queue, that is, the priority level of the element.
- **Penalty_factor:** The penalty coefficient associated with the element in order to ensure that not only circuits with high priority are selected in the queue, calculated as previously described. This factor increases linearly with the number of high priority circuits.

This equation can be represented as follows:

$$\text{profit} = \text{qubits} \times (1 + \alpha \cdot \text{prob}) \times \left(\frac{\text{count}}{\text{penalty_factor}} \right) \tag{2}$$

These values are then taken into account by the scheduling algorithm, based on the knapsack algorithm, in such a way that it selects the circuits best positioned by the model.

5 Validation

To evaluate the effectiveness of the *QCRAFT AI Scheduler*, a validation scenario has been designed to maximize the qubit usage while addressing the previously identified limitations. The tests were conducted using the IBM Brisbane

quantum computer, which has a capacity of 127 qubits. Consequently, the AI Scheduler was configured to schedule circuits in such a way that this capacity would be reached whenever possible with the circuits available in the queue.

The designed scenario consisted of a total of 300 circuits randomly selected from a set of widely known quantum algorithms, including: Kickback, Traveling Salesman Problem (TSP), Simon, Shor's Factoring, Grover's Search, Quantum Fourier Transform (QFT), Phase Estimation, Bernstein-Vazirani, Adders, and the Deutsch-Jozsa algorithm. Circuit implementations of these algorithms have been obtained from the MQT Bench [30], a widely recognized benchmark in the quantum community.

These circuits were submitted simultaneously to the AI Scheduler via REST requests, allowing the AI component to work with the full set to generate the best possible combinations. At the same time and for comparative purposes, the same circuits were compared with the time policy of the original *QCRAFT AI Scheduler*, which had previously achieved the highest utilization of qubits, to evaluate the improvement introduced by the current proposal. This time policy does not use any optimization algorithm like the AI Scheduler, rather it implements a variation of the FIFO (First In First Out) method, called AL-FIFO (At Least First In First Out). This means that, at least, the circuit in the first position of the queue will be included in the scheduling. However, when it comes to reaching the maximum number of qubits available in the machine, this can become a problem. There may be a combination of other circuits in the queue that reach the maximum and, if the first circuit is included, that optimal combination might no longer be possible.

The results, in terms of the number of qubits used in each iteration of scheduling, are presented in Fig. 4.

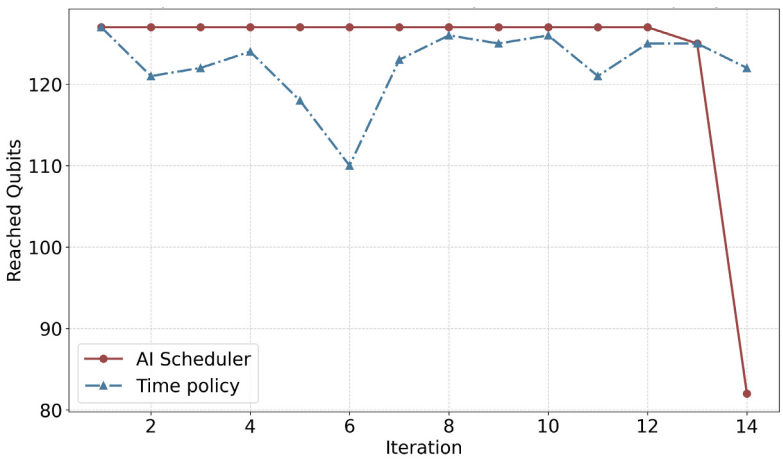


Fig. 4. Temporal evolution of qubits reached per iteration for each policy.

The qubits achieved with the AI Scheduler are shown in red, while those of the time policy are shown in blue. The results show that the AI Scheduler reaches the maximum qubit capacity of the machine in 12 of 14 scheduling iterations, while the time policy achieves this only once.

A noteworthy observation is that, in the final scheduling iteration, the AI Scheduler fills fewer qubits compared to the time policy. This is due to the fact that it scheduled a higher number of circuits in previous iterations, resulting in a smaller number of remaining circuits in the queue by the end of the process.

Another comparison was performed regarding cost reduction. In this case, the evaluation also included the cost of executing the circuits individually (green lines), in order to measure the impact of the AI Scheduler (red lines) in reducing execution costs compared to the time policy (blue lines). The results of this analysis are presented in Fig. 5.

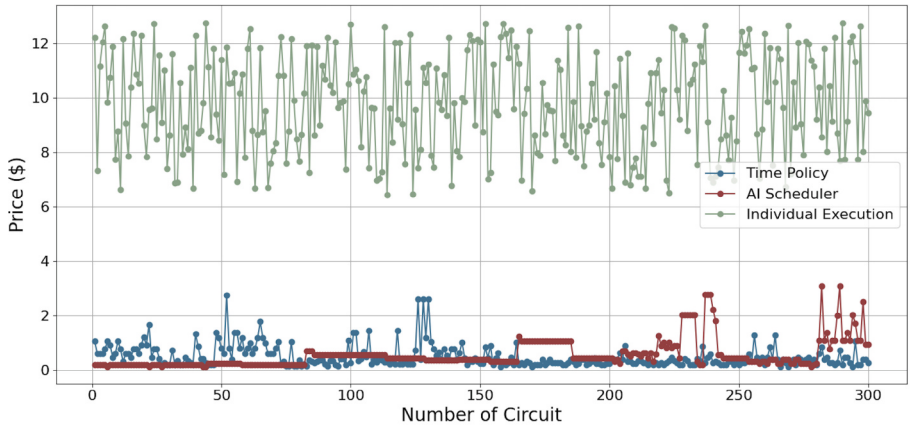


Fig. 5. Comparison of circuit prices between different techniques. (Color figure online)

It can be observed that the obtained costs using the AI Scheduler are more stable across circuits compared to those obtained with the time policy. This is due to the consistent achievement of the maximum capacity of the machine, which ensures a lower cost ratio at all times. We can also observe that there are schedules in which the time policy achieves a greater number of qubits than the AI Scheduler. This is because there are more circuits available in the queue for scheduling, unlike the AI, which uses a greater number of circuits in the first ones.

In general terms, the time policy achieves a 94.21% cost reduction compared to individual execution, whereas the AI Scheduler reaches a cost reduction of 95.06%.

6 Threats to Validity

While validating our approach, we identified several limitations that may affect the generalizability and realism of the results.

To ensure realistic validation, we considered simulating a real task queue from a commercial quantum computer. Both IBM and AWS provide APIs and SDKs that allow access to data related to the current state of their quantum computers, such as whether they are online or offline, the calibration of the processor's qubits, and even the number of tasks currently in the execution queue. By monitoring this last data point, the average number of tasks entering the queue per minute could be calculated, as well as an estimate of the number of tasks being executed. However, there is no access to data regarding the characteristics of the tasks in the queue, which are essential for simulating them, such as the number of qubits used by each circuit.

Taking this into account, the task queue of the IBM Brisbane machine was monitored over a period of two weeks, which we considered sufficient to capture representative queue activity and would allow us to see the potential of the proposal. During this period, a machine downtime of approximately 24 h occurred; however, task queues remained active, allowing users to continue to submit tasks for execution. From this monitoring period, an average of 3 tasks were received per minute in the queues and an average number of 2 tasks executed per minute were obtained. This supposes that, at any time, there are still tasks in the queue waiting to be executed.

As an alternative, we decided to generate a sufficiently large set of circuits to ensure that this maximum capacity could be reached without issue. This may pose a weakness when validating the proposal, as real execution data from the machines is not being taken into account.

7 Conclusion

Managing quantum services in cloud computing environments poses unique challenges that cannot be completely solved with traditional scheduling mechanisms, such as growing queues of tasks, topological limitations of devices, simultaneous demands from multiple users, and cost constraints. The work presented addresses the need for new specialized and service-aware scheduling mechanisms capable of responding to the high demand for quantum resources and achieving maximum utilization of hardware capacity. Therefore, the *QCRAFT AI Scheduler* has been developed that incorporates a Deep Learning-based model to improve decision making during the scheduling process.

The proposed AI-based approach offers stable resolution times once trained, even in the face of increasing input sets, as occurs when the number of queued tasks increases significantly. In this way, it promotes a closer view of task scheduling, considering cost and scalability.

The main goal is to achieve higher levels of efficiency by fully leveraging the maximum capacity of the machine. Specifically, one of the most relevant

results is the significant reduction in execution costs, with an average reduction of 95.06%, representing a 0.8% improvement over the best previous policy of the original *QCRAFT Scheduler*. While a 0.8% gain may appear modest, its effect becomes highly significant in scenarios where execution costs are elevated. In such contexts, even small percentage improvements translate into considerable financial savings, making this result particularly valuable for large-scale or cost-sensitive quantum service operations.

In addition, the *QCRAFT AI Scheduler* achieves a near complete usage of the capacity of the quantum computer in 12 out of 14 scheduling iterations, which means a resource utilization of 97.28%. This not only improves overall efficiency but also suggests even greater benefits as quantum computers increase the number of qubits.

A key aspect of this work is the choice of a learning-based approach over heuristic or greedy scheduling strategies. While heuristics have the advantage of avoiding training time and can be simpler to implement, they often fail to adapt to dynamic and heterogeneous workloads, leading to suboptimal qubit utilization in complex service scenarios. In contrast, Deep Learning models can capture hidden patterns in task arrival rates, circuit requirements, and device topologies, enabling them to generalize across diverse scheduling contexts. Once trained, the model provides near-constant inference times, making it competitive with heuristics in runtime but superior in terms of solution quality and scalability.

In conclusion, this work demonstrates the potential of AI as a key tool in quantum task planning, enabling more efficient resource utilization, a considerable reduction in operational costs, and an improvement in the quality of service offered to users of QCaaS platforms. As these systems scale, the ability to manage multiple tasks in an efficient and automated manner will become indispensable.

Acknowledgements. This work has been partially funded by the European Union "Next GenerationEU/PRTR", by the Ministry of Science, Innovation and Universities (TED 2021-130913 B-I00, RED2022-134148-T, and PDC2022-133465-I00). It is also supported by QSERV project (PID2021-1240454OB-C31) funded by the Spanish Ministry of Science and Innovation and ERDF; by European Union under the Agreement—101083667 of the Project "TECH4E -Tech4effiencyEDlH" Call: DIGITAL-2021-EDlH-01 supported by the European Commission through the Digital Europe Program; by the Regional Ministry of Education, Science and Vocational Training of the Regional Government of Extremadura (GR24099). Grant PRE2022-102070, funded by MCIN/AEI/10.13039/501100011033 and by FSE+.

References

1. Rahaman, M., Islam, M.M.: An overview on quantum computing as a service (QCaaS): probability or possibility. Int. J. Math. Sci. Comput. (IJMSC) **2**(1), 16–22 (2016). https://doi.org/10.5815/ijmsc.2016.01.02
2. Hassija, V., et al.: Present landscape of quantum computing. IET Quantum Commun. **1**(2), 42–48 (2020). https://doi.org/10.1049/iet-qtc.2020.0027

3. Alvarado-Valiente, J., Romero-Álvarez, J., Moguel, E., García-Alonso, J., Murillo, J.M.: Technological diversity of quantum computing providers: a comparative study and a proposal for API Gateway integration. Softw. Qual. J. 1–21 (2023). https://doi.org/10.1007/s11219-023-09633-5
4. Ichikawa, T., et al.: A comprehensive survey on quantum computer usage: how many qubits are employed for what purposes? arXiv:2307.16130 (2023)
5. Chickerur, S.: Introductory chapter: high performance parallel computing. In: High Performance Parallel Computing, IntechOpen (2019). https://doi.org/10.5772/intechopen.84193
6. Alvarado-Valiente, J., Romero-Álvarez, J., Casco-Seco, J., Moguel, E., Garcia-Alonso, J., Murillo, J.M.: Circuit scheduling policies on current QPUs: QCRAFT scheduler. In: Gaaloul, W., Sheng, M., Yu, Q., Yangui, S. (eds.) ICSOC 2024. LNCS, vol. 15405, pp. 195–209. Springer, Singapore (2025). https://doi.org/10.1007/978-981-96-0808-9_15
7. Ahmad, A.: Optimizing the traveling salesman problem using machine learning and predictive algorithms. J. Comput. Sci. Appl. Eng. (JOSAPEN) **3**(1), 1–5 (2025)
8. Bengio, Y., Lodi, A., Prouvost, A.: Machine learning for combinatorial optimization: a methodological tour d'horizon. Eur. J. Oper. Res. **290**(2), 405–421 (2021)
9. Murillo, J.M., et al.: Quantum software engineering: roadmap and challenges ahead. ACM Trans. Softw. Eng. Methodol. (2025). https://doi.org/10.1145/3712002
10. Moguel, E., Rojo, J., Valencia, D., Berrocal, J., Garcia-Alonso, J., Murillo, J.M.: Quantum service-oriented computing: current landscape and challenges. Softw. Qual. J. **30**(4), 983–1002 (2022). https://doi.org/10.1007/s11219-022-09589-y
11. Zhao, J.: Quantum software engineering: landscapes and horizons, arXiv preprint arXiv:2007.07047 (2020)
12. McCaskey, A., Dumitrescu, E., Liakh, D., Humble, T.: Hybrid programming for near-term quantum computing systems (2018). https://doi.org/10.1109/ICRC.2018.8638598
13. Ravi, G.S., Smith, K.N., Gokhale, P., Chong, F.T.: Quantum computing in the cloud: analyzing job and machine characteristics (2021). https://doi.org/10.48550/arXiv.2203.13121
14. Alvarado-Valiente, J., Romero-Álvarez, J., Moguel, E., García-Alonso, J., Murillo, J.M.: Orchestration for quantum services: the power of load balancing across multiple service providers. Sci. Comput. Program. **237**, 103139 (2024). https://doi.org/10.1016/j.scico.2024.103139
15. Nguyen, H.T., Usman, M., Buyya, R.: iQuantum: A Case for Modeling and Simulation of Quantum Computing Environments (2023). https://doi.org/10.1109/QSW59989.2023.00013
16. Fösel, T., Niu, M.Y., Marquardt, F., Li, L.: Quantum circuit optimization with deep reinforcement learning, arXiv preprint arXiv:2103.07585 (2021)
17. Das, P., Tannu, S.S., Nair, P.J., Qureshi, M.: A case for multi-programming quantum computers. In: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, pp. 291–303 (2019). https://doi.org/10.1145/3352460.3358287
18. Liu, L., Dou, X.: Qucloud+: a holistic qubit mapping scheme for single/multi-programming on 2D/3D nisq quantum computers. ACM Trans. Archit. Code Optim. **21**(1), 1–27 (2024). https://doi.org/10.1145/3631525
19. Ohkura, Y., Satoh, T., et al.: Simultaneous execution of quantum circuits on current and near-future nisq systems. IEEE Trans. Quantum Eng. **3**, 1–10 (2022). https://doi.org/10.1109/TQE.2022.3164716

20. Seitz, P., et al., Scim milq: an hpc quantum scheduler, arXiv preprint arXiv:2404.03512 (2024)
21. Bisicchia, G., García-Alonso, J., Murillo, J.M., Brogi, A.: Distributing quantum computations, by shots. In: Monti, F., Rinderle-Ma, S., Ruiz Cortés, A., Zheng, Z., Mecella, M. (eds.) ICSOC 2023. LNCS, vol. 14419, pp. 363–377. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-48421-6_25
22. Wang, Z., Tang, H.: Artificial intelligence for quantum error correction: a comprehensive review. arXiv preprint arXiv:2412.20380 (2024)
23. Alexeev, Y., et al.: Artificial intelligence for quantum computing. arXiv preprint arXiv:2411.09131 (2024)
24. Luo, W., Zhao, J., Zhan, T., Guan, Q.: Adaptive job scheduling in quantum clouds using reinforcement learning. arXiv preprint arXiv:2506.10889 (2025)
25. Nguyen, H.T., Usman, M., Buyya, R.: DRLQ: a deep reinforcement learning-based task placement for quantum cloud computing. In: 2024 IEEE 17th International Conference on Cloud Computing (CLOUD), pp. 475–481. IEEE (2024). https://doi.org/10.1109/CLOUD62652.2024.00060
26. Giortamis, E., Romão, F., Tornow, N., Bhatotia, P.: QoS: a quantum operating system (2024). https://doi.org/10.48550/arXiv.2406.19120
27. Romero-Álvarez, J., Alvarado-Valiente, J., Casco-Seco, J., Moguel, E., Garcia-Alonso, J., Murillo, J.M.: A noise validation for quantum circuit scheduling through a service-oriented architecture. Int. J. Softw. Eng. Knowl. Eng. 1–16 (2024). https://doi.org/10.1142/S0218194024410018
28. Mao, A., Mohri, M., Zhong, Y.: Cross-entropy loss functions: theoretical analysis and applications. In: Proceedings of the 40th International Conference on Machine Learning, ICML 2023. JMLR.org (2023). https://doi.org/10.5555/3618408.3619400
29. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)
30. Quetschlich, N., Burgholzer, L., Wille, R.: MQT bench: benchmarking software and design automation tools for quantum computing (2023). QuantumMQT Bench is https://www.cda.cit.tum.de/mqtbench/

DeepQUBO: Quantum-Optimized Route Planning for Carpooling Service

Zifeng Liu[1], Yuzhuo Zhao[1], Xiaofeng Gao[1(✉)], and Kun Tian[2]

[1] Shanghai Key Laboratory of Scalable Computing and Systems, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
{liuzifeng,a23333}@sjtu.edu.cn, gao-xf@cs.sjtu.edu.cn

[2] College of Engineering, Computer Science and Construction Management, California State University Chico, Chico, CA, USA
ktian@csuchico.edu

Abstract. Optimizing route planning for Mobility-as-a-Service platforms, such as carpooling, involves solving combinatorial problems under precedence and multi-candidate constraints. Traditional methods struggle to balance real-time requirements and solution quality due to computational complexity. We introduce **DeepQUBO**, a quantum-compatible framework that reformulates the *Precedence-Constrained Generalized Traveling Salesman Problem* (GTSP-PC) into a sparse *Quadratic Unconstrained Binary Optimization* (QUBO) model. The framework employs auxiliary variables that decouple complex constraints, maintaining optimality while reducing QUBO elements by 51% for connections and 76% for constraints relative to classical encodings. Experiments on Chengdu road networks and a *Coherent Ising Machine* (CIM) demonstrate 338× faster convergence than conventional approaches. Frustration analysis confirms DeepQUBO's enhanced suitability for quantum optimization, achieving lower constraint-frustration ratios across benchmark datasets. Our work bridges service orchestration with quantum-ready optimization, offering practical value for large-scale transportation systems.

Keywords: Quantum Optimization · Carpooling Service · QUBO Formulation

1 Introduction

Motivation. Quantum computing exhibits inherent advantages in service-oriented domains such as passenger travel [1,2] and task scheduling [3]. The reason is that quantum computing uses physical laws to efficiently solve binary optimization problems and find the global optimum [4]. A common approach involves formulating practical service challenges as planning problems and subsequently translating them into the *Quadratic Unconstrained Binary Optimization* (QUBO) format [5]. QUBO represents a form of binary optimization that serves as the standard paradigm for quantum optimization.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 380–395, 2026.
https://doi.org/10.1007/978-981-95-5015-9_27

However, **how to convert a service planning problem into a quantum-computing-friendly QUBO formulation** (Fig. 1) is often ignored by most research [6]. Existing quantum computing solvers—including *Quantum Annealing Computer* (QAC) [7], *noisy intermediate-scale quantum* (NISQ) devices [8], and *Coherent Ising Machine* (CIM) [9]—cannot solve QUBO problems of arbitrary size or precision, as their solution quality and scale are often limited by the underlying physical hardware. In this paper, we use a real-world carpooling problem with multiple candidate pickup and drop-off points [10], which involves various complex constraints, to explore QUBO formulation suitable for quantum computing. The DeepQUBO approach we establish for multi-candidate carpooling problems can also be applied to many other variants of the TSP.

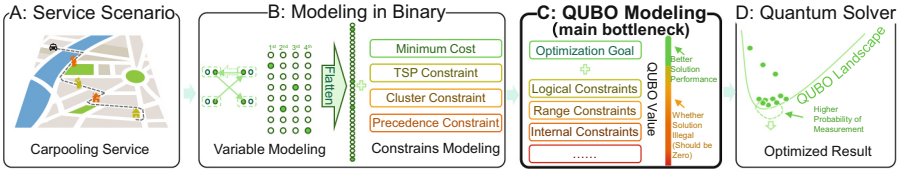


Fig. 1. Schematic of Quantum Optimization Construction Methods, Using Carpooling Services as a Case.

Contribution. The multi-candidate carpooling problem is modeled as the *precedence- constrained generalized Traveling salesman problem* (GTSP-PC). The presence of TSP, precedence, and multi-candidate point constraints complicates finding an efficient QUBO formulation for GTSP-PC. DeepQUBO, a novel QUBO formulation designed for GTSP-PC, abstracts the TSP and multi-candidate constraints using auxiliary inference variables.

Theoretically, we prove DeepQUBO preserves the optimal solutions of the original GTSP-PC model. Our formulation also achieves greater sparsity than classical QUBO encodings, with over 51% fewer connection elements and nearly 76% fewer constraint elements in typical carpooling cases. This enhanced sparsity demonstrates DeepQUBO's compatibility with quantum computing hardware.

Experimentally, we employed real-world Chengdu road network data to validate DeepQUBO's performance on a practical CIM platform. The proposed method demonstrates significant computational advantages, achieving 338× faster convergence compared to conventional approaches. Additionally, frustration element analysis conducted on both real-world Chengdu road network and *GTSP instances* [11] benchmark datasets indicates that DeepQUBO exhibits enhanced suitability for quantum optimization frameworks.

Outline. Section 2 establishes theoretical foundations for quantum optimization systems (Sect. 2.1) and formulates the carpooling scenario as a precedence-constrained Generalized Traveling Salesman Problem (Sect. 2.2). Section 3

presents the novel DeepQUBO formulation, contrasting it with Classical QUBO (Sect. 3.1). Section 4 provides theoretical guarantees and complexity analysis. Section 5 details experimental methodology, including frustration analysis and physical CIM implementation (Sect. 5.2). Section 6 contextualizes contributions within existing TSP literature.

2 Preliminaries

In this section, we first introduce preliminaries for quantum optimization systems applied to service optimization (Sect. 2.1). Subsequently, a complex real-world carpooling scenario is formulated as a *precedence-constrained Generalized Traveling Salesman Problem* (GTSP-PC) (Sect. 2.2).

2.1 Preliminaries on Quantum Optimization

Quantum optimization strategies show promise in fields like transportation [12], flight scheduling [13], and workforce planning [3], as they align with binary optimization frameworks. Using superposition, quantum computers solve n-variable problems (space 2^n) in polynomial time.

Quantum Optimization for Service. Service optimization via quantum computing typically follows Fig. 1: model the scenario in binary variables with objectives and constraints, then formulate it as a *Quadratic Unconstrained Binary Optimization* (QUBO) problem for quantum processors [8]. Quantum systems can explore all 2^n states in constant time and naturally encode constraints by linear superposition. A key challenge remains in translating real-world problems into effective QUBO formulations.

Essentially, this method (Fig. 1D) treats a QUBO like a custom-shaped bowl representing an energy landscape. Quantum effects naturally make computational systems "roll" solutions down to the absolute bottom (ground state), finding the optimal answer easily. Creating a good QUBO is like designing the bowl's shape so its low point matches desired real-world solution.

Quadratic Unconstrained Binary Optimization (QUBO). QUBO represents one of the most common and versatile modeling frameworks for optimization in quantum computing, as defined in Definition 1. "Quadratic" indicates the objective function involves terms at most up to second degree (often due to nearest-neighbor interactions), while "Unconstrained" signifies it inherently imposes hard binary variable constraints without additional restrictions. This optimization form exhibits a direct equivalence to physical Hamiltonians (energy functions describing quantum systems).

Definition 1 (Quadratic Unconstrained Binary Optimization (QUBO)). *The quadratic unconstrained binary optimization problem is an NP-hard combinatorial optimization problem defined over n binary*

variables. Given a real-valued upper-triangular matrix $Q \in \mathbb{R}^{n \times n}$*, the objective is to find a binary vector* $\boldsymbol{x} \in \{0,1\}^n$ *that minimizes the quadratic function:*

$$H(\boldsymbol{x}) = \boldsymbol{x}^\top Q \boldsymbol{x} = \underbrace{\sum_{i=1}^{n} Q_{ii} x_i}_{\textit{linear terms}} + \underbrace{\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Q_{ij} x_i x_j}_{\textit{quadratic terms}}$$

where:

(i) The solution space consists of 2^n *possible binary configurations.*
(ii) Diagonal entries Q_{ii} *are coefficients for linear terms.*
(iii) Off-diagonal entries Q_{ij} $(i < j)$ *encode pairwise interactions.*

QUBO-Ising Equivalence. The Ising model formulates the same optimization problem over spin variables $\boldsymbol{s} \in \{-1, +1\}^n$ as $E(\boldsymbol{s}) = \sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j$, where h_i are local fields and J_{ij} are coupling strengths. QUBO and Ising formulations are equivalent via $x_i = \frac{1+s_i}{2}$, enabling the same problem to be solved across different platforms.

Quantum Optimization Solvers. Several approaches optimize Ising models using different physical implementations. The *Coherent Ising Machine* (CIM) [9] is a quantum photonic device using quantum measurement and optical amplification, exploiting noise without entanglement. *Quantum Annealing Computer* (QAC) [7] employs quantum tunneling in superconducting qubits. The *Quantum Approximate Optimization Algorithm* (QAOA) [5] runs variational circuits on gate-model quantum hardware. In contrast, the *Simulated Bifurcation Algorithm* (SBA) [14] is a classical algorithm simulating bifurcation dynamics. Crucially, while CIM, QA, and QAOA are quantum paradigms with varying noise susceptibility, SBA runs deterministically on classical computers. Notably, CIM's measurement-feedback architecture offers coherence advantages over qubit-based systems, though scaling remains challenging for all quantum methods.

2.2 Preliminaries on Carpooling Service Modeling

Carpooling Service. Carpooling is a core component of MaaS platforms, where a single vehicle is assigned to transport multiple passengers with overlapping travel intentions [10]. As shown in Fig. 2A, a single vehicle is assigned to serve multiple passengers, each of whom specifies a cluster of candidate pick-up locations and drop-off locations. The system must jointly determine which pick-up point to select for each passenger and in what order to visit all selected pick-up and drop-off points, such that every passenger is picked up before being dropped off and the total travel cost is minimized. This leads to a routing problem that involves both location selection and precedence-constrained sequencing.

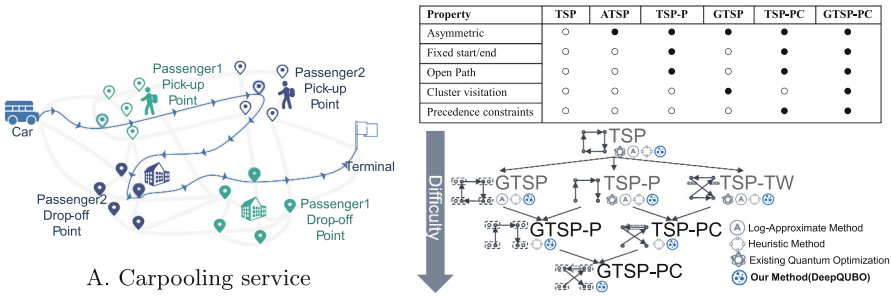


Fig. 2. Visualizations of the Carpooling Scenario and its Formalization as a Generalized Precedence-Constrained Routing Problem: (a) Illustration of multiple candidate pick-up and drop-off points for each passenger in the carpooling service. (b) Schematic overview of the TSP family, showing constraint structures and how GTSP-PC generalizes multiple TSP variants.

TSP Family. The *Traveling Salesman Problem* (TSP) seeks the shortest route visiting each location exactly once before returning to the start. Numerous variants address practical constraints: the *Asymmetric TSP* (ATSP) accounts for direction-dependent travel costs, suited to one-way streets or variable traffic; the *Precedence-Constrained TSP* (TSP-PC) enforces visit orderings; and the *Generalized TSP* (GTSP) [15] groups nodes into clusters, requiring exactly one visit per cluster to allow spatial flexibility. A precedence-constrained GTSP (GTSP-PC) further models paired operations common in passenger and freight transport. In addition, the *GTSP-Path* (GTSP-P) fixes both the starting and ending vertices while still following the cluster-visiting rule of GTSP, making it suitable for scenarios such as point-to-point delivery or tasks that begin and end at predetermined locations.

In our setting, the carpooling service problem naturally leads to a unified and more expressive variant: the *Precedence-Constrained Generalized Traveling Salesman Problem* (GTSP-PC), which simultaneously accounts for both clustered candidate points and ordered visit constraints. This formulation generalizes the above variants (Fig. 2B) and is particularly well-suited to modeling real-world service planning tasks such as carpooling, where spatial and logical constraints must be handled jointly.

GTSP-PC Definition. We formalize the carpooling routing problem as the *Precedence-Constrained Generalized Traveling Salesman Problem* (GTSP-PC), illustrated in Fig. 3A. This formulation extends classical TSP variants by integrating cluster-based location selection with inter-cluster precedence constraints. The formal definition is provided in Definition 2.

Definition 2 (Precedence-Constrained Generalized Traveling Salesman Problem (GTSP-PC)). *Let* $G = (V, E)$ *be a complete graph with*

vertex set V and edge set E, where each edge $(u, v) \in E$ has a non-negative weight defined by $d : E \to \mathbb{R}_{\geq 0}$. The vertex set V is partitioned into N_C disjoint clusters $\mathcal{C} = \{C_1, C_2, \ldots, C_{N_C}\}$. A set of precedence pairs is given as $P = \{(p_i^s, p_i^t) \mid 1 \leq i \leq m\}$, where each pair (p_i^s, p_i^t) enforces the constraint that any vertex in cluster $C_{p_i^s}$ must be visited before any vertex in cluster $C_{p_i^t}$. Let $C_s \in \mathcal{C}$ and $C_t \in \mathcal{C}$ denote the designated starting and terminating clusters, respectively. The GTSP-PC seeks a minimum-weight path that begins at a vertex $v_s \in C_s$, ends at a vertex $v_t \in C_t$, visits exactly one vertex from each cluster in $\mathcal{C}$, and satisfies all precedence constraints $(C_{p_i^s}, C_{p_i^t}) \in P$.

3 DeepQUBO for GTSP-PC

In this section, we construct QUBO formulations for GTSP-PC suitable for quantum computing, beginning with Classical QUBO and enhancing it to DeepQUBO. The key improvement is introducing auxiliary variables that decouple constraints through deep structural decomposition, creating a layered architecture where cluster-level variables separate node-level routing from precedence relationships, transforming complex entangled optimization problems for quantum solvers.

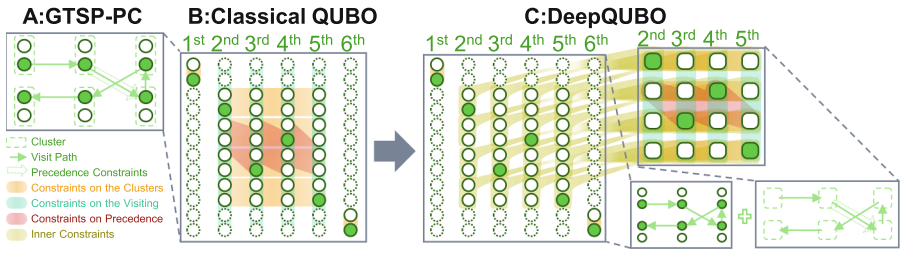


Fig. 3. Schematic Diagram of GTSP-PC and Its Related Classical QUBO and DeepQUBO Representations.

3.1 Classical QUBO Construction

We can construct the Classical QUBO formulation for GTSP-PC, denoted as $H_{\text{Classical QUBO}}$:

$$H_{\text{Classical QUBO}} = H_{\text{TC}} + A(H_{\text{CV}} + H_{\text{CC}} + H_{\text{PC}}),$$

where H_{TC} represents the Traveling Cost, H_{CV} denotes the Constraints on the Visiting, H_{PC} indicates the Precedence Constraint, and A is a large constant. Generally, A is set to the sum of absolute coefficients in H_{TC}. A larger A ensures better constraint satisfaction, while a smaller A facilitates optimization. In Classical QUBO, the variable x_i^j is a $\{0, 1\}$ QUBO variable which, as shown in

Fig. 3(B), represents whether vertex v_i is visited at the j-th step (1) or not visited (0). Formally, we define:

$$x_i^j = \begin{cases} 1 & \text{if vertex } v_i \text{ is visited at step } j \\ 0 & \text{otherwise} \end{cases}$$

Traveling Cost. H_{TC} represents the QUBO formulation of the total traveling cost, as shown in Sect. 3.1:

$$H_{\mathrm{TC}} = \sum_{k=1}^{N_C-1} \sum_{(i,j)\in\mathcal{V}_k\times\mathcal{V}_{k+1}} d(i,j) x_i^k x_j^{k+1},$$

where $\mathcal{V}_k$ denotes the set of accessible vertices at step k. For example, at the first step ($k = 1$), only vertices from cluster C_s can visit vertices not in C_s or C_t. Since $x_i^k x_j^{k+1} = x_i^k \wedge x_j^{k+1} \in \{0, 1\}$, H_{TC} accumulates the distance $d(i, j)$ only when vertices v_i and v_j are adjacent in the path, thus correctly representing the total traveling cost.

Constraints on the Visiting. H_{CV} represents the QUBO formulation of visiting constraints:

$$H_{\mathrm{CV}} = \sum_{k=1}^{N_C} \left(\sum_{i\in\mathcal{V}_k} x_i^k - 1 \right)^2,$$

where $\mathcal{V}_k$ denotes the set of accessible vertices at step k. For the first step ($k = 1$) and last step ($k = N_C$), $\mathcal{V}_k$ contains vertices from clusters C_s and C_t respectively, while for intermediate steps, it includes all vertices except those in C_s and C_t.

Constraints on the Clusters. H_{CC} represents the QUBO formulation of cluster visiting constraints:

$$H_{\mathrm{CC}} = \sum_{i\neq s,t} \sum_{j\in\{j|v_j\in C_i\}} \left(\sum_{k=2}^{N_C-1} x_j^k - 1 \right)^2,$$

ensuring that exactly one vertex from each intermediate cluster is visited. Reason is same as the form of H_{CV}.

Precedence Constraint. The QUBO formulation H_{PC} encodes the precedence constraints defined in set P, where for each pair $(p_i^s, p_i^t) \in P$, vertices cluster p_i^s must be visited before those in cluster p_i^t. Let x_m^j denote a binary variable indicating whether vertex v_m is visited at step j. The formulation is:

$$H_{\mathrm{PC}} = \sum_{(p_i^s,p_i^t)\in P} \sum_{j=2}^{N_C-2} \sum_{k=j+1}^{N_C-1} \sum_{v_m\in C_{p_i^t}} \sum_{v_l\in C_{p_i^s}} x_m^j x_l^k$$

This penalty term accumulates when any vertex from p_i^t (step j) is visited before a vertex from p_i^s (step k), where $j < k$, thus enforcing the required precedence relationships through minimization.

3.2 DeepQUBO Construction

As shown in Fig. 3(C), GTSP-PC can be decomposed into two parts: 1) a partially visited TSP problem with respect to vertices, and 2) a TSP-PC problem with respect to clusters. If these two parts can be separated independently, the QUBO construction would become simplified. DeepQUBO is defined as follows:

$$H_{\text{DeepQUBO}} = H_{\text{TC}} + A(H_{\text{CV-C}} + H_{\text{CC-C}} + H_{\text{IC-C}} + H_{\text{PC-C}}),$$

where H_{TC} and H_{CV} remain the same as defined in Sect. 3.1. To implement DeepQUBO, we introduce an auxiliary variable matrix denoted as $x^k_{(i)}$:

$$x^j_{(i)} = \begin{cases} 1 & \text{if cluster } c_i \text{ is visited at step } j, \\ 0 & \text{otherwise.} \end{cases}$$

Constraints on the Visiting of Clusters in DeepQUBO. $H_{\text{CV-C}}$ represents the QUBO formulation of visiting constraints:

$$H_{\text{CV-C}} = \sum_{i \neq s,t} \left(\sum_{j=2}^{N_C - 1} x^j_{(i)} - 1 \right)^2 .$$

This construction follows the same principle as H_{CV}, but applies the constraints at the cluster level rather than the vertex level.

Constraints on the Clusters in DeepQUBO. $H_{\text{CC-C}}$ represents the QUBO formulation of visiting constraints:

$$H_{\text{CC-C}} = \sum_{i \neq s,t} \left(\sum_{j=2}^{N_C - 1} x^j_{(i)} - 1 \right)^2 + \sum_{k \in \{s,t\}} \left(\sum_{i \in C_k} x_i^{f(k)} - 1 \right)^2 ,$$

where $f(k)$ equals 1 when $k = s$ and N_C when $k = t$. This formulation ensures two key constraints: first, each intermediate cluster (except start and end clusters) must be visited exactly once; second, exactly one vertex must be selected from the start cluster and from the end cluster.

Inner Constraints in DeepQUBO. $H_{\text{IC-C}}$ represents the QUBO formulation of visiting constraints:

$$H_{\text{IC-C}} = \sum_{i \in \{i | i \neq s, i \neq t\}} \sum_{j=2}^{N_C - 1} \left(\sum_{k \in \{v_k \in C_i\}} x^j_k - x^j_{(i)} \right)^2 .$$

This equation enforces consistency between vertex visits (x^j_i) and their corresponding cluster variable ($x^j_{(i)}$).

Precedence Constraint in DeepQUBO. The precedence constraints in DeepQUBO are simplified by utilizing the auxiliary cluster variables, resulting in

$$H_{\text{PC-C}} = \sum_{(p_i^s, p_i^t) \in P} \sum_{j=2}^{N_C-2} \sum_{k=j+1}^{N_C-1} x^j_{(p_i^t)} x^k_{(p_i^s)}.$$

This formulation enforces precedence constraints at the cluster level using auxiliary variables $x^j_{(p_i^t)}$ and $x^k_{(p_i^s)}$. The penalty term is minimized when cluster p_i^s precedes cluster p_i^t in the visiting sequence, i.e., $k > j$ for all valid steps.

3.3 Analytical Comparison of DeepQUBO

The estimation of **non-zero elements** and **non-zero constraint elements** in QUBO provides metrics for assessing implementation difficulty on quantum hardware. Non-zero elements represent mapping complexity, while constraint elements (multiplied by coefficient A) determine optimization precision requirements. These two metrics primarily determine the feasibility of solving QUBO problems on actual quantum devices.

Table 1. Analytical Comparison of Element Counts between Classical QUBO and DeepQUBO Formulations

Classical QUBO			DeepQUBO			Red%[a]
Type	Number of Elements	Nom.[b]	Type	Number of Elements	Nom.[b]	
H_{TC}	$\lvert V\rvert(\lvert V\rvert-1)N_C$	1800	H_{TC}	$\lvert V\rvert(\lvert V\rvert-1)N_C$	1800	0%
H_{CV}*	$N_C\lvert V\rvert^2$	1600	$H_{\text{CV-C}}$	$(N_C-2)^3$	125	92%
H_{CC}*	$\sum_{i=2}^{N_C-1}(N_C\lvert C_i\rvert)^2$	1650	$H_{\text{CC-C}}$	$(N_C-2)^3$	125	92%
H_{PC}	$\sum_{i=1}^{\lvert P\rvert}\frac{(N_C-3)(N_C-2)}{2}\lvert C_{p_i^s}\rvert\lvert C_{p_i^t}\rvert$	300	$H_{\text{PC-C}}$	$\lvert P\rvert\frac{(N_C-3)(N_C-2)}{2}$	12	96%
-	-	-	$H_{\text{IC-C}}$	$N_C\sum_{i=1}^{N_C}(\lvert C_i\rvert+1)^2$	576	-
Total QUBO Elements		**5350**	**Total QUBO Elements**		**2638**	**51%**
Total QUBO Constraint Elements		**3550**	**Total QUBO Constraint Elements**		**838**	**76%**

[a] **Red%**: Reduction in Percentage.
[b] **Nom%**: Typical two-passenger carpooling scenario.
[*] Simplified element count, excluding the influence of the starting and terminating clusters, maintains the same order of magnitude.

As shown in Table 1, we estimated the number of non-zero elements in Classical QUBO and DeepQUBO for different terms. For a GTSP-PC with an average of $\bar{C}$ elements per cluster, the visiting constraints ratio $|H_{\text{CV}}|/|H_{\text{CV-C}}| \approx (|V|/N_C)^2$; for cluster constraints, $|H_{\text{CC}}|/|H_{\text{CC-C}}| \approx |\bar{C}|^2$; and for precedence constraints, $|H_{\text{PC}}|/|H_{\text{PC-C}}| \approx |\bar{C}|^2$. DeepQUBO only requires adding $|H_{\text{IC-C}}| = N_C\sum_{i=1}^{N_C}(|C_i|+1)^2$ variables to maintain the original constraints. Therefore, DeepQUBO demonstrates significant advantages for problems with large $|\bar{C}|$.

For a typical two-passenger carpooling scenario (**Nom.**) with $N_C = 6$, assuming each cluster has 5 candidate vertices ($|C_i| = 5$) and two precedence constraints ($|P| = 2$), the total non-zero elements decreased by 51%, while non-zero constraint elements reduced by 76%, making the problem considerably more tractable.

4 Proof of DeepQUBO Optimal Solution for GTSP-PC

Theorem 1 (Equivalence of DeepQUBO and GTSP-PC). *Optimal solutions of DeepQUBO and GTSP-PC are in one-to-one correspondence.*

Proof. Let $A = |V|\, N_C \max_{i,j} d(i,j)$. Consider $H_{\text{DeepQUBO}} = H_{\text{TC}} + A(H_{\text{CV}} + H_{\text{CV_C}} + H_{\text{CC_C}} + H_{\text{PC_C}})$. We prove this in three steps:

Step 1. At the minimum of H_{DeepQUBO}, all terms with coefficient A must vanish. Suppose otherwise, then any non-zero term would contribute at least A to H_{DeepQUBO}, which exceeds the maximum possible reduction in H_{TC} (bounded by $|V|\, N_C \max_{i,j} d(i,j)$).

Step 2. The vanishing of constraint terms implies:

(i) $\sum_{k \in C_i} x_k^j = x_{(i)}^j, \quad \forall i \notin \{s,t\}, j \in [2, N_C - 1]$

(ii) $\sum_{j \notin \{s,t\}} x_{(j)}^i = 1, \quad \forall i \in [2, N_C - 1]$

(iii) $\sum_{k=2}^{N_C - 1} x_j^k = 1, \quad \forall j \in C_i, i \notin \{s,t\}$

Step 3. These constraints collectively ensure:

- From (4): Cluster and vertex visits are consistent
- From (4): Exactly one cluster is visited at each step
- From (4): Each cluster (except s, t) is visited exactly once
- From $H_{\text{PC_C}} = 0$: All precedence relationships are satisfied

Therefore, proof in Theorem 1, any solution minimizing H_{DeepQUBO} must satisfy all GTSP-PC constraints. Since H_{TC} is identical in both formulations, the optimal solution of DeepQUBO corresponds to the optimal solution of GTSP-PC.

4.1 QUBO Matrix Reduction via Variable Filtering

We propose a systematic filtering technique to reduce QUBO complexity by exploiting predetermined variable assignments (Definition 3). Let $\mathcal{X} = \{x_1, x_2, \ldots, x_n\}$ denote the complete variable set and $\mathcal{D} \subset \mathcal{X}$ represent the subset of deterministic variables with known assignments.

Definition 3 (Variable Filtering Operations). *For any deterministic variable $x_i \in \mathcal{D}$, we define the filtering transformation $\mathcal{F} : \mathbb{R}^{n \times n} \to \mathbb{R}^{(n-1)\times(n-1)}$ as:*

$$\mathcal{F}(Q)_{jk} = \begin{cases} Q_{jk} & \text{if } x_i = 0 \text{ and } j,k \neq i \\ Q_{jk} + Q_{ji} & \text{if } x_i = 1 \text{ and } j = k \neq i \\ Q_{jk} & \text{if } x_i = 1 \text{ and } j \neq k, j,k \neq i \end{cases}$$

The mathematical foundation relies on the Boolean algebra properties of QUBO variables. For $x_i = 0$, the elimination follows from $x_i \wedge x_j = 0 \wedge x_j = 0$ for any $x_j \in \mathcal{X}$. For $x_i = 1$, we utilize the identity $x_i \wedge x_j = 1 \wedge x_j = x_j$, transforming quadratic terms $Q_{ij}x_ix_j$ into linear terms $Q_{ij}x_j$.

Theorem 2 (Optimality Preservation). *Let $H(\mathcal{X}) = \sum_{i,j} Q_{ij}x_ix_j$ be the original QUBO and $H'(\mathcal{X}')$ be the filtered formulation where $\mathcal{X}' = \mathcal{X} \setminus \mathcal{D}$. Then* $\arg\min H(\mathcal{X}) = \arg\min H'(\mathcal{X}') \cup \mathcal{D}$.

As shown in Theorem 2, this approach achieves computational efficiency by reducing the problem dimension from $|\mathcal{X}|$ to $|\mathcal{X}'| = |\mathcal{X}| - |\mathcal{D}|$ qubits while preserving solution optimality. The technique is particularly advantageous for path-constrained problems where boundary conditions naturally determine variable assignments, enabling significant matrix sparsification without compromising solution quality.

5 Experimental Settings

We utilize a Chengdu transportation network dataset with geographically clustered pickup and dropoff locations to generate GTSP-PC instances simulating ridesharing scenarios. The *GTSP instances* [11] benchmark suite provides additional challenging instances with diverse topological structures.

We conduct a two-phase evaluation: (1) frustration analysis comparing Classical QUBO and DeepQUBO across problem distributions, and (2) physical CIM implementation demonstrating 338× convergence improvement for DeepQUBO.

5.1 Frustration Analysis

We employ frustration analysis to quantify QUBO complexity and quantum hardware compatibility. Since frustration analysis requires Ising variables $s_i \in \{-1, +1\}$, we transform the QUBO formulation using the standard mapping $x_i = \frac{1+s_i}{2}$, yielding the equivalent Ising Hamiltonian:

$$H_{\text{Ising}} = \sum_{i<j} J_{ij}s_is_j + \sum_i h_is_i + \text{const}$$

where J_{ij} and h_i are derived from the original QUBO coefficients Q_{ij}.

Let $G = (V, E)$ represent the resulting Ising graph where vertices V correspond to spin variables $s_i \in \{-1, +1\}$ and edges E encode spin-spin interactions $J_{ij}s_is_j$. Cycle Frustration is a usual way to analyse the QUBO, which is defined in Definition 4.

Definition 4 (Cycle Frustration). *For a cycle C in the Ising graph, frustration occurs when $\prod_{(v_i,v_{i+1})\in C} sgn(J_{ij}) = -1$, indicating conflicting spin interactions that cannot be simultaneously satisfied in the ground state.*

Our experimental evaluation employs 9 metrics based on a unified frustration counting framework. For any graph element $\xi \in \{V, E, \mathcal{C}\}$ (vertex, edge, or cycle set), we define:

$$\begin{aligned} \mathrm{F}\xi &= \text{Number of frustrated elements in } \xi \\ \mathrm{NF}\xi &= \text{Number of non-frustrated elements in } \xi \\ \mathrm{RF}\xi &= \frac{\mathrm{F}\xi}{\mathrm{F}\xi + \mathrm{NF}\xi} \quad \text{(Frustration ratio)} \end{aligned}$$

Quantum Solvability Correlation. Higher frustration ratios (RFV, RFE, RFC) hinder quantum annealing due to energy landscape complexity, coherence demands, and embedding overhead.

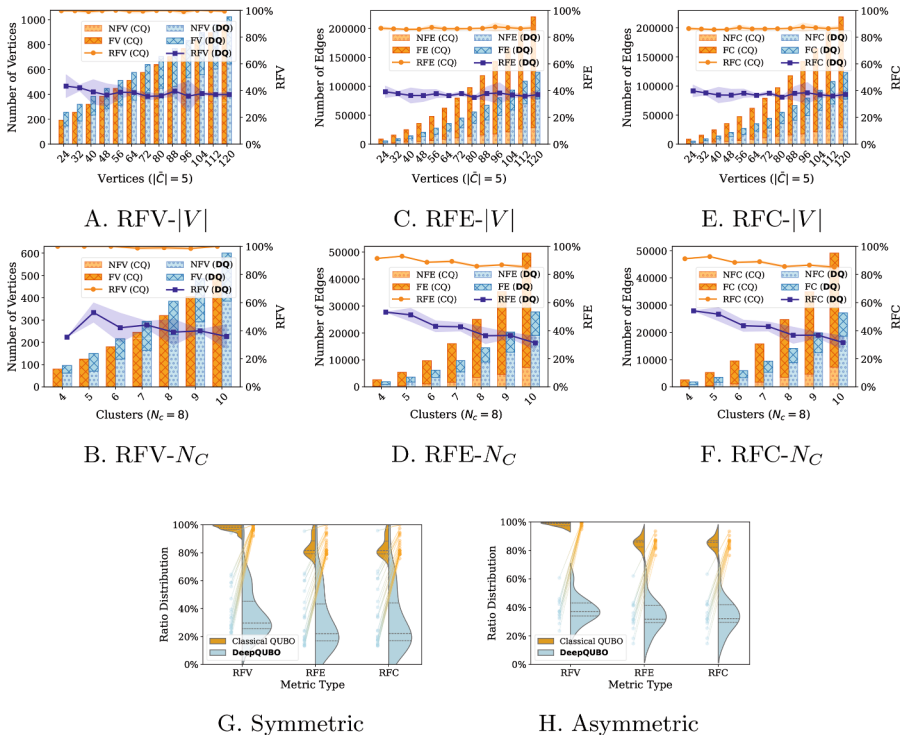


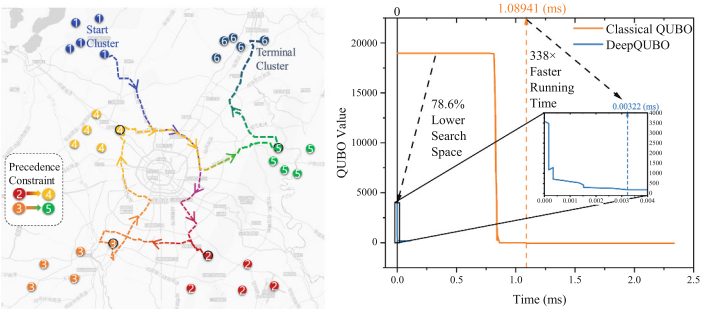
Fig. 4. A–F: Comparative analysis of QUBO frustration metrics across varying problem scales (vertex and frustration ratio evaluation) for Chengdu ridesharing scenarios. G–H: Distribution of three types of frustration under different GTSP cases for two QUBO configurations; connections represent frustration ratios for the same graph in both QUBOs.

Quantitative Analysis. We analyze the number of vertices, edges, and cycles in QUBO matrices. As shown in Figs. 4A and 4B, DeepQUBO increases vertex count by less than 15% across different $|V|$ and N_C values. However, as demonstrated in Figs. 4C to 4F, DeepQUBO achieves approximately 50% reduction in both edge and cycle counts compared to Classical QUBO. This makes DeepQUBO more advantageous for practical quantum computing deployment.

Frustration Ratio Analysis. Examining Fig. 4, we find that Classical QUBO exhibits over 97% frustrated vertices and 80% frustrated edges/cycles, indicating high problem difficulty. In contrast, our DeepQUBO formulation maintains frustration ratios around 40%. Furthermore, as shown in Fig. 4B, 4D and 4F, frustration ratios decrease as cluster size $|\bar{C}|$ increases. In Figs. 4G and 4H, we analyze QUBO constructions with *GTSP instances* [11] and demonstrate that DeepQUBO outperforms Classical QUBO across all three frustration metrics.

5.2 Quantum Computing Experiments Using Physical CIM

As demonstrated in Fig. 5B, we implement a typical two-passenger Chengdu carpooling problem (Fig. 5A) and solve it using physical CIM hardware. The DeepQUBO formulation successfully obtains the optimal solution, while Classical QUBO fails to find a feasible solution. Furthermore, DeepQUBO achieves a **338× convergence speedup** and **78.6% reduction in search space** compared to the classical approach. These results demonstrate DeepQUBO's significant advantages in practical problem solving.



A. Experiment Demo B. CIM Evolution

Fig. 5. Evolution of QUBO expectation values over time in CIM hardware experiments using Classical QUBO and DeepQUBO.

6 Related Work

Quantum Optimization and QUBO Formulation. Quantum computing has demonstrated potential in service-oriented domains including transportation [2] and task scheduling [3]. Various quantum paradigms have emerged, including QAC [7], QAOA [5], and CIM [9]. Recent QUBO construction advances address specific domains: graph problems, Bayesian networks [16], and sparse formulations [17]. Tools like PyQUBO [18] facilitate problem mapping, while specialized hardware approaches [19] address implementation challenges. However, most research overlooks quantum-computing-friendly QUBO construction that maintains optimality while addressing hardware constraints [6].

TSP Variants and Carpooling Optimization. The TSP family encompasses variants addressing real-world constraints, from classical approximations to modern stochastic approaches and constrained variants like TSP-P [20]. Carpooling services in MaaS platforms present unique challenges combining spatial flexibility with temporal constraints [10], naturally leading to GTSP-PC formulation [15]. Current quantum approaches remain limited: quantum annealing addresses only simplified TSP-TW [21], while hybrid methods face scalability barriers. No existing QUBO formulation adequately handles GTSP-PC's intricate constraints, motivating novel quantum-compatible frameworks.

7 Summary

We optimize carpooling scheduling via quantum computing by modeling it as a *Precedence-Constrained Generalized Traveling Salesman Problem* (GTSP-PC) and introducing DeepQUBO, a novel QUBO formulation with hierarchical constraints implemented through auxiliary cluster variables. This design reduces constraint-related non-zero matrix elements by 76% compared to classical formulations, while preserving theoretical equivalence to GTSP-PC. Experiments on quantum-inspired Coherent Ising Machines show up to 338× faster convergence relative to existing QUBO strategies. Future extensions will target dynamic mobility scenarios and performance benchmarking on emerging quantum hardware. The results highlight that combining domain-specific problem reformulation with quantum-aware constraint engineering can substantially advance practical quantum optimization for intelligent transportation systems.

Acknowledgments. This work was supported by the National Key R&D Program of China [2024YFF0617700], the National Natural Science Foundation of China [U23A20309, 62272302, 62372296].

The authors would like to thank Mrs Zeyun Li, Mr. Yuhong Liu, Mr. Jiachun Bao, Mr. Xiang Li, Mr. Zhi Han, Mr. Haoming Zhang, and Mrs Jiayi Yu for their contributions on the early versions of this paper.

References

1. Stollenwerk, T., et al.: Quantum annealing applied to de-conflicting optimal trajectories for air traffic management. IEEE Trans. Intell. Transp. Syst. (ITSM) **21**(1), 285–297 (2020)
2. Azad, U., Behera, B.K., Ahmed, E.A., Panigrahi, P.K., Farouk, A.: Solving vehicle routing problem using quantum approximate optimization algorithm. IEEE Trans. Intell. Transp. Syst. (ITSM) **24**(7), 7564–7573 (2023)
3. Ruan, Y., Yuan, Z., Xue, X., Liu, Z.: Quantum approximate optimization for combinatorial problems with constraints. Inf. Sci. **619**, 98–125 (2023)
4. Rajak, A., Suzuki, S., Dutta, A., Chakrabarti, B.K.: Quantum annealing: an overview. Philos. Trans. R. Soc. A Math. Phys. Eng. Sci. **381**(2241), 20210417 (2023)
5. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm (2014). arXiv preprint arXiv:1411.4028
6. Pichugina, O., Tan, Y., Beck, C.: Deriving compact qubo models via multilevel constraint transformation (2024). arXiv preprint arXiv:2404.03610
7. Hauke, P., Katzgraber, H.G., Lechner, W., Nishimori, H., Oliver, W.D.: Perspectives of quantum annealing: methods and implementations. Rep. Prog. Phys. **83**(5), 054401 (2020)
8. Preskill, J.: Quantum computing in the NISQ era and beyond. Quantum **2**, 79 (2018)
9. Honjo, T., et al.: 100,000-spin coherent ising machine. Sci. Adv. **7**(40), eabh0952 (2021)
10. Zhang, J., Huang, X., Liu, Z., Gao, X., Chen, G.: Multi-candidate carpooling routing problem and its approximation algorithms. In: Wu, W., Guo, J. (eds.) Combinatorial Optimization and Applications (COCOA), pp. 380–391. Springer, Cham (2024)
11. Fischetti, M., González, J.J.S., Toth, P.: A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. Oper. Res. **45**(3), 378–394 (1997)
12. Borowski, M., et al.: New hybrid quantum annealing algorithms for solving vehicle routing problem. In: Krzhizhanovskaya, V.V., et al. (eds.) ICCS 2020. LNCS, vol. 12142, pp. 546–561. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-50433-5_42
13. Stollenwerk, T., Lobe, E., Jung, M.: Flight gate assignment with a quantum annealer. In: Feld, S., Linnhoff-Popien, C. (eds.) QTOP 2019. LNCS, vol. 11413, pp. 99–110. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-14082-3_9
14. Goto, H., Tatsumura, K., Dixon, A.R.: Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems. Sci. Adv. **5**(4), eaav2372 (2019)
15. Garg, N., Konjevod, G., Ravi, R.: A polylogarithmic approximation algorithm for the group Steiner tree problem. J. Algorithms **37**(1), 66–84 (2000)
16. Shikuri, Y.: Decomposed quadratization: efficient qubo formulation for learning Bayesian network. In: AAAI Conference on Artificial Intelligence (AAAI), vol. 39, pp. 11345–11352 (2025)
17. Luo, J., Shu, Y., Ha, Y.: RSQC: recursive sparse qubo construction for quantum annealing machines. IEEE Trans. Comput. (TOC) **74**(6), 2114–2128 (2025)
18. Zaman, M., Tanahashi, K., Tanaka, S.: Pyqubo: Python library for mapping combinatorial optimization problems to qubo form. IEEE Trans. Comput. (TOC) **71**(4), 838–850 (2022)

19. Pu, T., Chen, C., Zeng, L., Liu, S., Sun, R., Fan, C.: Solving combinatorial optimization problem over graph through qubo transformation and deep reinforcement learning. In: IEEE International Conference on Data Mining (ICDM), pp. 390–399 (2024)
20. Zenklusen, R.: A 1.5-approximation for path tsp. In: ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 1539–1549 (2019)
21. Papalitsas, C., Andronikos, T., Giannakis, K., Theocharopoulou, G., Fanarioti, S.: A qubo model for the traveling salesman problem with time windows. Algorithms **12**(11) (2019)

Industrial Papers

IoT and LLM Supported Digital Twin Platform: A Case Study on a Net Zero Oriented Building

Jiaqi Li, Xingchen Hu, and Jun Shen(✉)

University of Wollongong, Wollongong, NSW 2500, Australia
xh739@uowmail.edu.au, jshen@uow.edu.au

Abstract. Digital twins have become widely adopted in the building sector for operational monitoring and management; however, existing solutions rely heavily on users' knowledge for decision-making and lack effective human-computer interaction mechanisms, limiting user engagement and actionable feedback. This paper introduces a human-computer interaction-enhanced digital twin visualization framework integrated with large language models (LLMs), designed to optimize occupants' energy use by providing real-time insights into indoor environmental conditions and enhancing user engagement and actionable understanding through intuitive data exploration. By combining Building Information Modeling (BIM), Internet of Things (IoT) sensor networks, and LLMs, the framework enables users to interact with a 3D model of the building, monitor real-time data (e.g., temperature collected from edge devices), and receive environment adjustment suggestions to support decision-making. Compared to the conventional visualization platforms, the framework adds an additional interaction-feedback layer which combines predefined environmental threshold rules with a lightweight LLM to provide real-time prompts and context-aware suggestions to users. The 3D visualization environment is implemented using the Unity engine, enabling immersive interaction with building models and sensor data in real time. A locally hosted, lightweight LLM is embedded into the interface to support offline reasoning and privacy-preserving feedback generation. The system simulates the ability to detect and interpret abnormal building conditions and provide context-relevant responses, offering preliminary decision-support capabilities within the digital twin framework. Experimental results show that this system improves data comprehensibility, responsiveness, and user engagement, encouraging more sustainable occupant behavior. The integration of digital twins with lightweight interactive intelligence highlights a promising step toward adaptive and user-centered building operations. This paper also introduces our real implementation of the edge intelligence supported platform for an energy saving and net zero oriented building in collaboration with industry partners.

Keywords: Digital Twins · Building Information Modeling · IoT and Edge · Large Language Models · Net Zero

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 399–414, 2026.
https://doi.org/10.1007/978-981-95-5015-9_28

1 Introduction

Globalization and rapid advances in information and communication technologies have brought both unprecedented opportunities and multifaceted challenges to urban development. In response, smart city will be essential to both seize these opportunities and overcome respective new challenges where digital twins might be one of the most important technologies for such smart urbanization movement [11].

As a key component of smart cities, the building sector plays a central role in energy consumption and emissions. That's why Net Zero Energy Buildings (NZEBs) has become a common goal for both smart cities and smart buildings. To this end, the concept of Net Zero Oriented Buildings has emerged, referring to buildings being designed and operated with the goal of gradually approaching NZEBs performance through intelligent systems and informed decision-making [20].

Digital twins are virtual representations of physical assets that enable real-time monitoring, analysis, and optimization. Combined with Building Information Modeling (BIM) and Internet of Things (IoT) sensors, digital twins offers a strong foundation for real-time building monitoring and control. However, current digital twins often struggle with understanding building data at a meaningful level and rely heavily on users' domain knowledge and experience for decision-making. This dependence on manual interpretation makes it challenging for users to extract practical insights from complex building information and make informed decisions efficiently. Recent advancements in artificial intelligence (AI), especially in large language models (LLMs) offer new opportunities for overcoming these limitations. LLMs enable semantic reasoning, natural language interaction, and multi-modal interpretation of data, making them promising tools for bridging the gap between building systems and occupant decision-making. Data collected from edge devices and services now has a better potential to be transformed into meaningful knowledge or wisdom more accessible by human or other AI agents.

In this paper, we propose an interactive digital twin framework integrates BIM, IoT, and a fine-tuned lightweight LLM to enhance human-building interaction and support energy-efficient decision-making. The framework enables users to explore a real-time 3D model of the building, monitor environmental data, and receive clear, context-aware suggestions through a dual interaction system that uses both rule-based responses and LLM-generated natural language guidance. Using a real-world building project as the case, the system demonstrates improved data interpretability, user engagement, and sustainability awareness without compromising data privacy. By adding intelligent reasoning to digital twin systems, this framework represents a practical approach toward smart, responsive, and user-focused operations for energy-efficient buildings.

The rest of the paper is organised as follows. Section 2 presents the related work and points out the current research gaps. Section 3 outlines the proposed digital twin framework and implementation methodology, detailing its multi-layer architecture and data processing pipeline. Section 4 provides a case study

to introduce the experimental setup, system functionalities, and integration of the LLM-based assistant within the digital twin platform. Section 5 discusses our results and evaluations. Finally, Sect. 6 concludes the paper and discusses potential directions for future research.

2 Related Works

2.1 IoT in Buildings Energy Management

IoT serves as the foundation of digital twins, as environmental monitoring and energy consumption in buildings rely on IoT and edge devices for real-time data collection. In recent years, IoT-based energy services have become deeply integrated into various aspects of daily life. Abusafia [3] established an effective IoT-based framework for providing energy services. And their applications in building energy management have also reached a significant level of maturity. For example, Islam et al. [12] integrated the Proximal Policy Optimization (PPO) algorithm with IoT to optimize BEMS for more effective control of carbon emissions. At the same time, case studies [4,15] from different countries have also demonstrated the broad applicability of IoT in supporting net-zero buildings.

2.2 Digital Twins for Net Zero Oriented Buildings

A basic digital twin model can be established through the integration of BIM, IoT, simulation, and control logic, enabling real-time monitoring and supporting simulation and optimization toward net-zero energy performance [18]. Research by Kaewunruen [14] has demonstrated the effectiveness of digital twins for NZEB management by a literature review, and the case study in the UK shows it can achieve a 23-year return period. With the growing maturity of research and applications in NZEBs [6,16], there has been more academic focus toward net-zero oriented buildings, reflecting the aspiration to align both future developments and existing building stock with the goal of NZEB [8]. For instance, Shen et al. [19] proposed a conceptual framework that integrates BIM and digital twins by an ontology-based method to support whole-life-cycle decision-making for net-zero-carbon buildings. Wang et al. [21] proposed an integrated energy management approach for smart building microgrids by combining a modified differential evolution algorithm with random forest regression, leveraging digital twins to optimize renewable energy utilization under multi-objective constraints related to cost and environmental impact for achieving net-zero emissions in smart buildings. Although the development of digital twins has accelerated the research on net-zero oriented buildings, achieving the goal of net-zero still heavily relies on decision-making by users. The advent of AI provides stronger support for evidence-based decisions and enhances decision-making efficiency, offering new solutions for realizing intelligent net-zero oriented buildings [13].

2.3 LLMs for Building Intelligence

In recent years, the continuous evolution of smart building concepts has paid more attention to AI. For example, Ma et al. [17] integrated BIM, Building Automation Systems (BAS), IoT, AI, and mixed reality technologies to propose and implement a multi-floor digital twin architecture for 3D interactive building operations and maintenance. However, such approaches still fall in providing decision support at the semantic level for end users. Although some Building Energy Management System (BEMS) have begun to integrate digital twin with AI such as natural language processing methods [10], the process of constructing domain knowledge bases remains complex and poses significant challenges for real-time knowledge updating. However, LLMs offer a promising solution to these challenges. As a result, the exploration of how LLMs can be leveraged to enhance building intelligence has emerged as an important research direction in this field. In other areas of building intelligence, LLMs and digital twin integrated methods for fault detection and diagnosis (FDD) have already achieved in operation and maintenance applications [22]. In contrast, existing studies on the application of LLMs in building energy intelligence are mainly focused on the integration with BIM or IoT [7,9]. Inspired by the research from Zhang [23] and Arslan [5], directly applying large language models (LLMs) to the analysis and interpretation of building energy data presents a promising direction for future research. Based on such motivations, introducing LLMs into the digital twin of buildings as a dedicated layer to support decision-making might empower it to gain a more detailed understanding of building-specific information, thereby facilitating deeper involvement in decision processes compared to their approach.

3 Proposed Method

To address the limitations of conventional digital twin platforms, particularly their dependence on user interpretation and the absence of actionable feedback, this paper propose a structured method that integrates physical sensing, real-time visualization, and intelligent human-computer interaction. The proposed framework integrates BIM, IoT sensor networks, and LLMs into a unified digital twin system for the net zero oriented residential buildings.

3.1 Digital Twin Structural Framework

The proposed digital twin framework is organized into a multi-layer architecture that enables seamless integration of physical sensing, virtual representation, and human-computer interaction. Each layer plays a distinct role in the system, collectively supporting the flow of data from real-world sensing to interactive visualization and user engagement.

As shown in Fig. 1, the framework consists of three conceptual layers: the Physical Asset Layer, the Digital Twin Layer, and the improved Service Layer. These layers form a processing pipeline from data collection to intelligent feedback presentation.

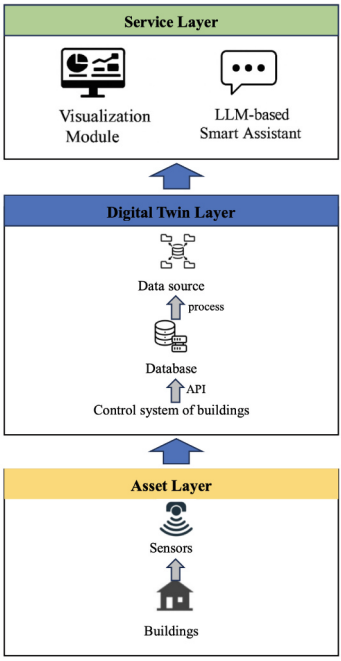


Fig. 1. Multi-layer architecture of the digital twin platform for the residential building.

Physical Asset Layer. This layer represents the target physical building environment and its associated sensing infrastructure. A network of IoT sensors is deployed throughout the building to continuously monitor environmental parameters, including temperature, humidity, illuminance, and occupancy, among many others. These sensors are connected to a local control system, which aggregates sensor data and transmits it to a centralized database via specific API interfaces.

Digital Twin Layer. This layer acts as the core data engine of the platform. It retrieves raw sensor data from the backend database and applies preprocessing procedures unto them. The processed data serves as the synchronized data source for the visualization environment, ensuring consistent and reliable input for digital representation.

Service Layer. The improved service layer constitutes the user-facing interface of the digital twin system, integrating both 3D visualization and intelligent feedback to enhance environmental awareness and occupant engagement. Unlike conventional service layers that primarily focus on data presentation, the proposed implementation introduces a significantly improved version of the service layer. This upgraded design presents the BIM-derived building model alongside

environmental data in a unified and interactive environment. It includes modules for real-time display of sensor values, temporal trend visualization with selectable aggregation ranges, and an interactive floor plan that presents room-specific environmental information. What sets this service layer apart is the integration of a lightweight LLM-powered assistant, which extends the platform's capabilities beyond visualization. By continuously evaluating environmental conditions against predefined threshold rules, the system triggers natural language prompts that simulate intelligent responses. These prompts are generated based on context-aware descriptions and are delivered within a dedicated dialogue panel, providing occupants with actionable suggestions in a human-readable form. This represents a shift from traditional passive interfaces to an interactive, feedback-driven paradigm. By embedding both real-time visualization and LLM-based reasoning within the same interface, the enhanced service layer supports a more adaptive, proactive, and interpretable user experience.

3.2 The Development of Digital Twin Platform Combined with IoT and LLM

To support real-time environmental awareness and intelligent user interaction, the proposed digital twin platform integrates sensing, visualization, and feedback into a unified workflow. The system is organized into three functional modules: data acquisition and preprocessing, data visualization, and a lightweight LLM-based feedback mechanism. The following subsections outline the implementation strategy for each module.

Data Acquisition and Preprocessing. The platform features a data acquisition mechanism designed to capture real-time environmental conditions from the physical space. To ensure the collected data is suitable for downstream processing and interpretation, a preprocessing routine is applied to address common irregularities such as missing values, noise, and temporal misalignment. The resulting data stream is structured for continuous synchronization across all functional layers of the system.

Data Visualization. Building on the preprocessed data, the platform provides a visualization module that presents environmental information through intuitive graphical interfaces. Users can examine real-time values, analyze historical trends, and gain zone-specific insights. This visual interface serves as the primary touch point for user engagement and supports intuitive navigation across different environmental contexts.

Feedback Mechanism Based on LLM. To enhance interactivity and provide actionable insights, the platform introduces a lightweight feedback mechanism powered by LLM. This module continuously evaluates environmental parameters and, based on predefined threshold rules, generates context-aware natural language suggestions. By translating observations into user-friendly prompts, the

platform facilitates informed decision-making and bridges the gap between raw data and behavioral response.

4 Case Study

The introduced case is an award winning net-zero residential demonstration house located in New South Wales, Australia, and it's exterior is illustrated in Fig. 2. Originally designed as an energy-efficient retrofit, the building integrates a range of net zero oriented green technologies including rooftop solar photovoltaic systems, a high-efficiency heating, ventilation, and air conditioning (HVAC) system, and an intelligent environmental monitoring infrastructure. These features make it a representative prototype for exploring digital twin applications in residential contexts.

The house offers a well-defined and sensor-rich environment that supports the continuous collection of environmental data across multiple functional zones. Its relatively compact yet multi-room structure allows for clear spatial segmentation, making it particularly well-suited for testing visualization and feedback mechanisms. As such, this net-zero building could serve as an ideal reference model for evaluating the core functionalities of the proposed digital twin platform.

Fig. 2. Exterior view of the studied net-zero residential building.

4.1 Data Acquisition and Preprocessing

The digital twin platform operates on real-time environmental data continuously streamed from the monitoring infrastructure of the net-zero residential building. The system continuously receives indoor environmental parameters, such as temperature, humidity, illuminance, and occupancy, collected across multiple rooms.

These data streams reflect the actual operating conditions of the residential environment and serve as the dynamic input layer for downstream visualization and reasoning components. The modular nature of the building and the availability of spatially distributed measurements ensure a reliable basis for spatial-temporal analysis.

To ensure reliable and consistent downstream processing, the incoming data flow undergoes a dedicated preprocessing routine. This includes timestamp normalization to ensure temporal alignment across variables, interpolation to resolve occasional missing values, and smoothing techniques to eliminate irregularities caused by transmission noise or transient anomalies. These operations are performed in real time, allowing the system to maintain a stable and coherent understanding of environmental dynamics as they evolve.

Through this preprocessing pipeline, the platform achieves continuous synchronization between the physical environment and its digital representation. The resulting data stream serves as a high-fidelity input for both visualization and intelligent feedback modules, ensuring that subsequent user-facing components reflect current and accurate building conditions.

4.2 Data Visualization Platform Development

To provide a spatially explicit representation of indoor environmental conditions, a digital twin visualization platform was developed based on a 3D building model of this net-zero residential building. The platform leverages a BIM-derived model and real-time data integration to simulate the actual environmental status of the building in an immersive, interactive interface. The Unity platform was selected as the development environment due to its flexibility in 3D rendering, support for real-time updates, and compatibility with external data streams.

The digital twin platform's user interface is structured to support spatial contextualization and multi-dimensional data interpretation through a cohesive visual layout. As depicted in Fig. 3, the interface adopts a dual-panel design: the left region is dedicated to the interactive 3D building model, while the right region presents layered environmental data modules. This model, created in Autodesk Revit, was imported into Unity and optimized for real-time rendering. Users can interact with the model using standard operations such as rotation, zooming, and panning, allowing them to explore the building layout and focus on specific areas of interest. On the right side of the interface, environmental information is displayed through three integrated modules. The first module presents real-time data, including temperature, humidity, illuminance, occupancy, HVAC operational mode, and setpoint temperature. The second module supports historical trend analysis by allowing users to select time ranges via a drop-down menu. A corresponding line chart then visualizes indoor temperature changes over the selected period, alongside system control targets. The third module features an interactive 2D floor plan, enabling users to click on specific zones to view room-level environmental values.

To maintain consistency across all components, a centralized time control mechanism was implemented. This controller ensures that all visual elements,

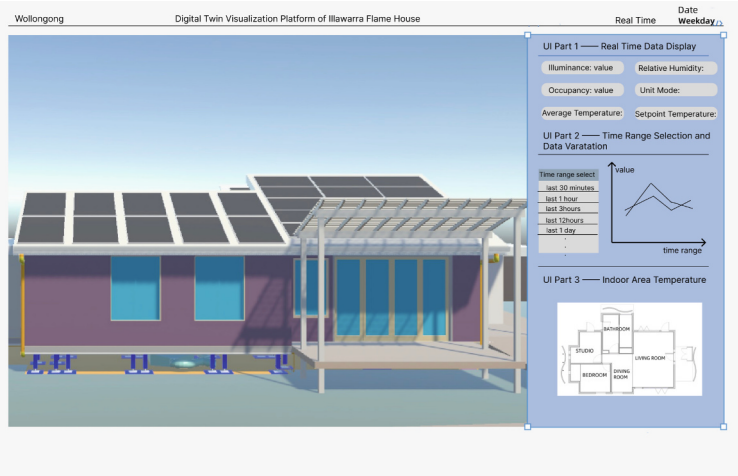


Fig. 3. Layout of the proposed digital twin visualization platform.

including textual indicators, trend charts, and spatial highlights—remain temporally synchronized, providing a coherent and unified view of indoor environmental conditions.

This visualization environment acts as the main interface through which users engage with the system. It helps them observe and understand changes in indoor conditions more easily by turning raw data into intuitive visual displays. By presenting environmental information in a clear and spatially organized way, the platform improves user awareness and prepares the system for integrating intelligent feedback features in the next layer.

4.3 LLM-Based Assistant Mechanism

To address the limitations of conventional digital twin systems, particularly in their dependence on passive data visualization and the lack of actionable user feedback, this paper introduces a lightweight interaction-feedback layer built upon the existing visualization platform. This additional layer is designed to enhance human–computer interaction by delivering context-aware recommendations in natural language, thereby enabling users to make informed decisions in response to evolving indoor environmental conditions.

At the core of this feedback mechanism lies a hybrid reasoning strategy that combines rule-based triggers with the interpretive capabilities of a locally deployed large language model (LLM). Instead of simply relaying raw sensor values, the system monitors environmental dynamics such as elevated indoor temperature, high humidity levels, or HVAC inactivity during periods of occupancy, and transforms these observations into structured prompts. The underlying lightweight LLM then processes these prompts to generate adaptive, user-facing suggestions. This approach represents a conceptual shift from passive

monitoring to proactive guidance, embedding a primary yet effective form of cognitive reasoning into the digital twin architecture. The overall processing pipeline of this interaction-feedback mechanism is illustrated in Fig. 4, which outlines the sequential flow from real-time environmental input through LLM-based reasoning to user-facing feedback.

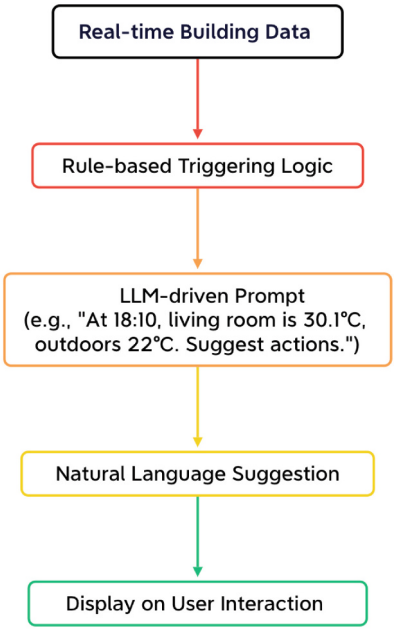


Fig. 4. Workflow of LLM-based Feedback Generation.

To support this architecture, the system employs Ollama [2], an open-source runtime environment optimized for hosting LLMs on local devices. In this implementation, a Mistral-based model [1] is used to strike a balance between inference quality and computational efficiency. Crucially, all model execution occurs offline, ensuring that sensitive building data such as occupancy status, temperature readings, and HVAC operation is securely maintained within the local network. This setup not only enhances data privacy and system responsiveness but also removes dependency on Internet connectivity.

The effectiveness of the assistant's recommendations depends largely on the quality and granularity of contextual data it receives. To that end, it is granted structured access to the set of real-time environmental variables extracted from the digital twin platform. These include average temperature, setpoint temperature, humidity, illuminance, occupancy status, HVAC operation mode, and room-specific temperature. All relevant values are dynamically synchronized and formatted into semantically structured summaries, enabling the assistant to maintain a coherent representation of the building's current state.

To further enhance contextual awareness and facilitate comparative reasoning, the assistant is additionally provided with outdoor environmental data such as ambient temperature and humidity. These values are retrieved from simulated real-time weather feeds. By bridging the indoor-outdoor environmental boundary, the assistant can provide feedback that is sensitive to both internal discomfort and external conditions, such as suggesting natural ventilation when the outside air is cooler and drier.

The logic for triggering suggestions is governed by a set of domain-informed rules that reflect thresholds for comfort, efficiency, and safety. For example, if the indoor temperature in an occupied room exceeds 28 °C while the HVAC system remains off, the system flags this condition as warranting user intervention. Similarly, high humidity during extended occupancy, a large discrepancy between average and setpoint temperatures, or prolonged inactivity of HVAC components may also trigger a response. These rules are continuously evaluated in real time based on the synchronized data stream from the digital twin model.

Upon detecting a trigger condition, the system aggregates key environmental variables, including temporal context, occupancy information, and both system and climate parameters, into a structured, semantically enriched prompt. This prompt encapsulates the situational context with sufficient specificity to support meaningful language generation by the LLM. For example:

"At 17:15, the living room is occupied. The current temperature is 30.1 °C, *the humidity is 68%, and the HVAC system is off. The setpoint temperature is 24* °C, *and the outdoor temperature is 22* °C. *Please provide suggestions for improving indoor comfort."*

This prompt is then processed by the locally hosted Mistral model via the Ollama framework, which returns a natural-language response such as:

"It appears the living room is experiencing elevated heat and humidity while occupied. Consider turning on the air conditioning or opening a window to improve comfort."

By combining structured detection with generative reasoning, the assistant delivers feedback that is both data-driven and user-friendly. Compared to conventional alerts or rigid automation systems, this hybrid approach allows for more flexible, personalized, and transparent communication. This enables the digital twin platform to go beyond merely visualizing environmental states by actively guiding user behavior in a proactive and interpretable manner, which represents a significant advancement toward intelligent interaction between humans and buildings.

4.4 Results

The final implementation of the digital twin platform delivers a cohesive and interactive user interface that supports real-time environmental monitoring, building model exploration, and contextual interpretation. As shown in Fig. 5, the layout is organized into two primary sections: a 3D building model on the left and a data visualization panel on the right. A top banner provides essen-

tial context, including the platform title, building information, and the current system time.

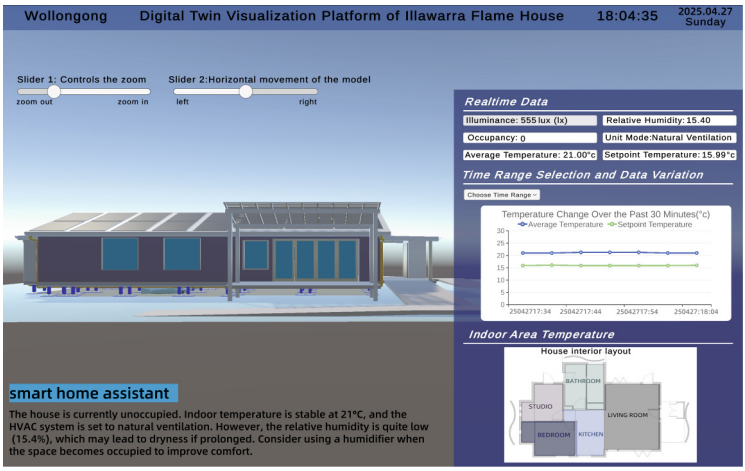


Fig. 5. Interface layout of the digital twin visualization platform for the studied net-zero residential building.

The left panel features a 3D model of the net-zero residential building. This model supports interactive operations such as rotation, zooming, and panning, allowing users to explore the building layout. Two sliders are positioned above the building model: one adjusts the zoom level (zoom in or out), while the other controls horizontal translation. For rotation, users can directly click and drag the model to change the viewing angle, enabling smooth and intuitive spatial interaction.

On the right, the data visualization panel includes three functional modules:

Real-Time Environmental Display. This module presents key environmental parameters, including average temperature, humidity, illuminance, occupancy status, HVAC operational mode, and setpoint temperature. The data is continuously updated, providing users with immediate insight into indoor conditions.

Trend Analysis Panel. Users can select from a range of predefined time intervals, including 30 min, 1 h, 1 day, up to 90 days, by using a drop-down menu. Based on the selected interval, the platform renders a line chart comparing indoor temperature trends with HVAC setpoint values. For shorter timeframes, raw sensor data is visualized; for longer durations, averaged values are computed to convey broader trends. Tooltips are displayed when hovering over specific data points, offering detailed numeric values for deeper analysis.

Interactive Floor Plan. This component allows users to click on specific rooms, such as the studio, living room, or kitchen, to view the real-time temperature in that zone. When a room is selected, a pop-up overlay displays the

corresponding temperature value in real time. This interaction bridges the 3D spatial model and sensor data, offering an intuitive way to access zone-specific information.

In the lower-left corner of the interface, a dedicated icon represents the LLM-based smart assistant. When users click on this icon, the system generates a context-aware textual response based on the current environmental conditions of the building. This feedback appears within a dialog box overlay, offering real-time suggestions. The assistant is seamlessly integrated into the visualization environment, ensuring that interaction remains non-intrusive while enhancing the platform's ability to provide intelligent, natural-language support.

As illustrated in Fig. 5, when the assistant is activated under the environmental conditions shown, including unoccupied status, an indoor temperature of 21 °C, natural ventilation mode, and low relative humidity (15.4%), it produces the following suggestion: *"The house is currently unoccupied. Indoor temperature is stable at 21* °C, *and the HVAC system is set to natural ventilation. However, the relative humidity is quite low (15.4%), which may lead to dryness if prolonged. Consider using a humidifier when the space becomes occupied to improve comfort."*

5 Discussion and Evaluation

Although formal user testing and quantitative evaluation have not yet been fully conducted subject to institutional approval for this very novel design, the proposed digital twin platform introduces key innovations that distinguish it from existing approaches. Most prior works focus on static dashboards or rule-based automation, whereas our system embeds a lightweight, locally hosted LLM to deliver context-aware feedback in natural language. This enables more interactive, interpretable, and user-centered engagement, marking a conceptual shift in the service layer from passive visualization to proactive semantic guidance. Unlike approaches that rely on predefined ontologies or complex knowledge bases, the proposed platform utilizes real-time data and threshold-based triggers to generate adaptive suggestions, eliminating the need for cloud access or in-depth domain modeling. This design supports greater flexibility, offline deployment, and enhanced user interpretability, particularly in residential settings. These design philosophy behind the work also aligns with the broad framework of edge intelligence.

Evaluation of the framework can be based on usability metrics like average response time, average log probability, interaction frequency, time required for user decision-making. Additionally, interaction complexity and behavioral intention surveys may help determine how the interactive LLM layer enhances interpretability, improve mutual understanding between human and platform, and also promotes sustainable user behaviors.

6 Conclusion and Future Work

This paper presents the design and implementation of an interactive digital twin platform for residential buildings, with the net-zero oriented building serving as the experimental foundation. By integrating BIM-based 3D visualization, real-time environmental sensing, and intelligent feedback mechanisms, the platform advances beyond conventional monitoring systems toward a more proactive and human-centered interaction model.

A key contribution of this work is the introduction of a lightweight interaction-feedback layer, powered by a locally deployed LLM. Through a hybrid reasoning mechanism—combining rule-based triggers with generative natural language responses—the system provides context-aware suggestions based on real-time dynamic indoor and outdoor conditions. It not only enhances the interpretability of environmental data but also introduces a preliminary form of decision support into the digital twin architecture. The final implementation demonstrates the feasibility of embedding natural language feedback into real-time visualization environments without sacrificing data privacy. By maintaining local model execution using the Ollama framework and enabling prompt generation based on semantically structured inputs, the assistant offers a meaningful step toward intelligent human-building interaction.

Despite these contributions, several limitations remain. The current feedback logic is reactive, relying on static, manually defined rules, and lacks learning or predictive capabilities. The assistant does not yet adapt to occupant preferences, behavioral patterns, or inter-room dynamics. Additionally, the system currently operates on simulated or prototype-level data; real-world deployment may introduce challenges in sensor reliability, calibration, and exception handling. Lastly, the user interface remains limited to text-based interaction, which constrains the modes of user engagement.

Future work will focus on augmenting the system with predictive models for occupant behavior and thermal comfort estimation, as well as reinforcement learning agents for long-term energy optimization. Broader deployment across multiple buildings or user groups will also be considered to evaluate scalability, usability, and user engagement. Finally, we will aim to enhance the interaction experience by supporting multi-modal feedback delivery and cross-device access, moving toward a truly intelligent and interactive digital twin assistant for net zero oriented building or even sustainable building operations.

References

1. Mistralai/mistral-7b. Mistral AI (2025). https://mistral.ai/news/announcing-mistral-7b
2. Ollama/ollama. Ollama (2025). https://ollama.com
3. Abusafia, A., Bouguettaya, A., Lakhdari, A.: Maximizing consumer satisfaction of IoT energy services. In: Service-Oriented Computing, ICSOC 2022. LNCS, vol. 13740, pp. 395–412. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20984-0_28

4. Alam, M., Graze, W., Graze, T., Graze, I.: As-built performance of net-zero energy, emissions, and cost buildings: a real-life case study in Melbourne, Australia. Buildings **14**(11), 3614 (2024). https://doi.org/10.3390/buildings14113614
5. Arslan, M., Mahdjoubi, L., Munawar, S.: Driving sustainable energy transitions with a multi-source RAG-LLM system. Energy Build. **324**, 114827 (2024). https://doi.org/10.1016/j.enbuild.2024.114827
6. Chang, Y.T., Kuo, W.L., Wu, T.H., Hsieh, S.H.: A digital twin platform based on building information models and smart sensors for a net zero energy, circular and WELL campus- a case study of national taiwan university. In: Advances in Information Technology in Civil and Building Engineering, ICCCBE 2024. LNCE, vol. 628, pp. 209–220. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-84208-5_17
7. De Vito, G., Palomba, F., Ferrucci, F.: The role of Large Language Models in addressing IoT challenges: a systematic literature review. Future Gener. Comput. Syst. **171**, 107829 (2025). https://doi.org/10.1016/j.future.2025.107829
8. Elghaish, F., Matarneh, S., Hosseini, M.R., Tezel, A., Mahamadu, A.M., Taghikhah, F.: Predictive digital twin technologies for achieving net zero carbon emissions: a critical review and future research agenda. Smart Sustain. Built Environ. (2024). https://doi.org/10.1108/sasbe-03-2024-0096
9. Forth, K., Borrmann, A.: Semantic enrichment for BIM-based building energy performance simulations using semantic textual similarity and fine-tuning multilingual LLM. J. Build. Eng. **95**, 110312 (2024). https://doi.org/10.1016/j.jobe.2024.110312
10. Fortuna, C., Hanžel, V., Bertalanič, B.: Natural language interaction with a household electricity knowledge-based digital twin (2024). https://doi.org/10.48550/arXiv.2406.06566
11. Hu, X., Shen, J., Ma, Z.: Prefabricated buildings for smart cities: a literature survey. In: 2024 Twelfth International Conference on Advanced Cloud and Big Data (CBD), pp. 302–307. IEEE, Brisbane, Australia (2024). https://doi.org/10.1109/CBD65573.2024.00061
12. Islam, F., Ahmed, I., Mihet-Popa, L.: Development and testing of an IoT platform with smart algorithms for building energy management systems. Energy Build. **344**, 115970 (2025). https://doi.org/10.1016/j.enbuild.2025.115970
13. Jiang, G., Ma, Z., Zhang, L., Chen, J.: EPlus-LLM: a large language model-based computing platform for automated building energy modeling. Appl. Energy **367**, 123431 (2024). https://doi.org/10.1016/j.apenergy.2024.123431
14. Kaewunruen, S., Rungskunroch, P., Welsh, J.: A digital-twin evaluation of net zero energy building for existing buildings. Sustainability **11**(1), 159 (2018). https://doi.org/10.3390/su11010159
15. Liang, X., Chen, K., Chen, S., Zhu, X., Jin, X., Du, Z.: IoT-based intelligent energy management system for optimal planning of HVAC devices in net-zero emissions PV-battery building considering demand compliance. Energy Convers. Manage. **292**, 117369 (2023). https://doi.org/10.1016/j.enconman.2023.117369
16. Liu, Z., Li, M., Ji, W.: Development and application of a digital twin model for Net zero energy building operation and maintenance utilizing BIM-IoT integration. Energy Build. **328**, 115170 (2025). https://doi.org/10.1016/j.enbuild.2024.115170
17. Ma, T., et al.: Digital twin for 3D interactive building operations: integrating BIM, IoT-enabled building automation systems, AI, and mixed reality. Autom. Constr. **176**, 106277 (2025). https://doi.org/10.1016/j.autcon.2025.106277

18. Papadonikolaki, E., Anumba, C.: How can digital twins support the net zero vision? In: Advances in Information Technology in Civil and Building Engineering, ICCCBE 2022. LNCE, vol. 358, pp. 81–97. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-32515-1_7
19. Shen, K., Ding, L., Wang, C.: Development of a framework to support whole-life-cycle net-zero-carbon buildings through integration of building information modelling and digital twins. Buildings **12**(10), 1747 (2022). https://doi.org/10.3390/buildings12101747
20. Tiagarajan, T., Go, Y.I.: Integration of BIPV design and energy efficient technologies for low energy building in meeting net zero target. e-Prime Adv. Electr. Eng. Electron. Energy **8**, 100554 (2024). https://doi.org/10.1016/j.prime.2024.100554
21. Wang, Q., Yin, Y., Chen, Y., Liu, Y.: Carbon peak management strategies for achieving net-zero emissions in smart buildings: advances and modeling in digital twin. Sustain. Energy Technol. Assess. **64**, 103661 (2024). https://doi.org/10.1016/j.seta.2024.103661
22. Yoon, S., Song, J., Li, J.: Ontology-enabled AI agent-driven intelligent digital twins for building operations and maintenance. J. Build. Eng. **108**, 112802 (2025). https://doi.org/10.1016/j.jobe.2025.112802
23. Zhang, L., Fu, X., Li, Y., Chen, J.: Large language model-based agent Schema and library for automated building energy analysis and modeling. Autom. Constr. **176**, 106244 (2025). https://doi.org/10.1016/j.autcon.2025.106244

Edge-Native Lightweight Model Design and Scheduling for Vehicle Localization Services

Jiaqi Chen and Zhicheng Cai(✉)

Nanjing University of Science and Technology, Nanjing 210094, China
{chenjiaqi,caizhicheng}@njust.edu.cn

Abstract. In unmanned weighbridge Edge services, determining whether vehicles are correctly positioned on weighbridges from video streams efficiently at the Edge is a critical challenge. Traditional monocular deep models for vehicle localization are not lightweight and accurate enough for deploying on Edge devices. Meanwhile, existing inference scheduling methods often neglect multi-stream concurrency and effective collaboration among Edge devices. Therefore, we propose a monocular indirect vehicle localization method based on a lightweight keypoint detection model, optimized for NPUs through customized neural operators, together with a Multi-stream video Distributed Inference and Scheduling (MDIS) framework that integrates frame stitching, adaptive sampling, and weight-based load balancing. The proposals are evaluated on a real Edge device-based cluster, and experimental results show that our approach reduces parameters by 33.46%, computational cost (FLOPs) by 38.63%, and improves the inference speed on Edge devices by approximately 33% without decreasing detection accuracy. MDIS is able to obtain the lowest latency and fail rates. The codes are available at https://github.com/JiaQiChen-Lucas/vehicle_localization.

Keywords: Edge service · Lightweight model · Keypoint detection

1 Introduction

Deploying video-based intelligent models in centralized Cloud systems requires transmitting video streams to remote data centers, incurring high latency and bandwidth costs. With the integration of NPUs, Edge devices can locally host deep learning models [1], reducing response time. One of the main challenges in implementing an unmanned Edge-based weighbridge service is to design lightweight yet accurate location models. Moreover, as multiple video streams must be processed simultaneously, Edge clusters are needed, where efficient scheduling of heterogeneous and distributed inference tasks is another challenge. In this paper, we propose an indirect object localization approach that integrates a lightweight keypoint detection model (Wheel-YOLOv8-pose) tailored for Edge NPUs, along with MDIS for collaborative inference task scheduling on Edge clusters.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 415–423, 2026.
https://doi.org/10.1007/978-981-95-5015-9_29

2 Related Work

Current scheduling approaches for Edge services largely emphasize mobile user task offloading [5,8]. However, in weighbridge systems, cameras are fixed, and inference services are deployed on the local Edge cluster. Thus, efficient cross-node task scheduling becomes the central challenge. In existing methods, Round-Robin [6], Earliest Finish Time first (EFT), Least Response Time first (LRT) [6], Least Queuing Length first (LQL) [13], Least CPU and Memory Utilization first (LCMU) [10] are usually applied heuristics which are lightweight and efficient. Furthermore, many studies focus on resource requirement prediction models to improve scheduling performance [9], but most of them are not for cross-device scheduling. Most of the existing works use a single metric about network latency or computing resources, but using these metrics comprehensively is able to improve performance [10]. Existing localization methods, such as LiDAR [12], typically involve heavy computational demands, limiting their applicability on Edge devices. Although monocular-based methods are lightweight than most LiDAR and stereo vision-based methods, they are still not computation efficient on NPU-based Edge devices. Model lightweighting techniques include pruning [2], quantization [7], knowledge distillation [4], and efficient architecture design [3]. However, no light-weight indirect location models currently exist.

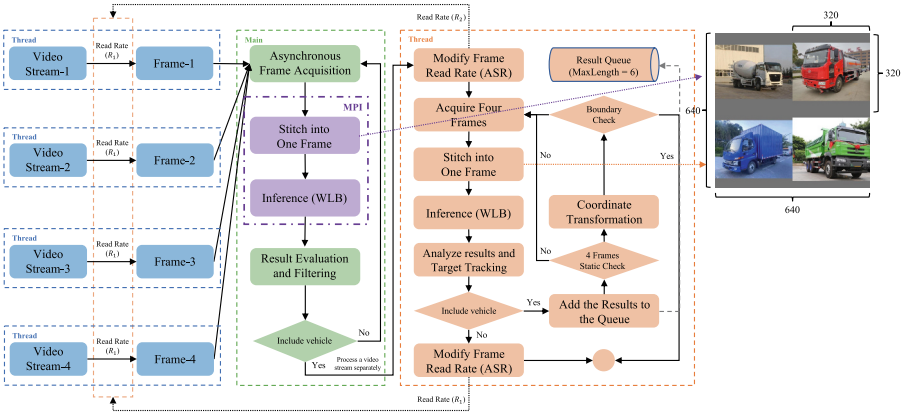


Fig. 1. Structure of MDIS.

3 Multi-stream Edge Inference Scheduling

To reduce NPU workload, we propose an Adaptive Sampling Rate (ASR) method that lowers the sampling rate when no vehicles are present. A Multi-stream Parallel Inference (MPI) strategy stitches frames from multiple streams into one, reducing inference overhead. Generated tasks are then distributed across devices using a Weighted Load Balancing (WLB) scheme. The overall MDIS framework is illustrated in Fig. 1.

3.1 Adaptive Rate Based Frame Sampling Method

The sampling service first captures frames from four video streams at a base rate R_1 using FFmpeg in the main thread. These frames are stitched by MPI into a single frame and sent to the inference service for vehicle detection. If a stream contains targets, the service spawns worker threads for fine-grained analysis at a higher rate R_2, still using four-frame stitching. Detection outputs are parsed individually, and tracking results are maintained in a FIFO queue Q (maximum length six). If no targets appear in four consecutive frames, the worker thread is deallocated and the stream reverts to R_1. For stationary target detection, a centroid stability criterion is applied: if the bounding box center remains stable across four consecutive results in Q, the object is classified as stationary, and boundary intrusion detection is triggered. If stability fails or intrusion occurs, another round of frame stitching and analysis is performed. This process repeats until the object leaves the monitored area or decision conditions are met.

3.2 Multi-stream Frame Stitching Based Parallel Inference Method

In MPI, each frame is resized to 320×320, and four frames from different streams are stitched into a 640×640 composite image to form a single inference task. After inference, results are parsed to recover detections for each original frame. This approach enables four frames to be processed in one pass, greatly reducing computational cost.

3.3 Weight Based Load Balancing

In WLB, each device reports its resource metrics to Nacos. The scheduler assigns tasks based on a weighted score for device d_i, calculated as follows to integrate latency, queue length, and resource status.

$$\begin{aligned} Score_i = {} & (100 - U_c^i) \times W_c + (100 - U_m^i) \times W_m + (100 - T^i) \times W_T \\ & + (100 - U_n^i) \times W_n - R^i \times W_R - \sum_{j=1}^{k} [(Q_j^i \times W_Q^j) + (N_j^i \times W_N^j)] \end{aligned} \tag{1}$$

where U_c^i, U_m^i, T^i, U_n^i, R^i, Q_j^i and N_j^i denote the device's CPU utilization, memory usage, temperature, NPU utilization, average response latency, queue length and number of active tasks (forwarded from current node) for the j^{th} task type. The corresponding weights W_c, W_m, W_T, W_n, W_R, W_Q^j, and W_N^j balance the influence of each metric.

4 Lightweight Vehicle Localization Model

This paper introduces a keypoint-based indirect vehicle localization approach that delivers high-precision positioning from monocular images in weighbridge settings and enables real-time inference on Edge devices like the OrangePi 5B.

4.1 Lightweight YOLOv8n-Pose for Key Point Detection

We propose C2REPA, as shown in Fig. 2, a lightweight feature extraction module optimized for Edge NPUs. Unlike the C2f structure that uses inefficient Split, Concat, and Add operations, C2REPA employs a parallel multi-channel RepConv [3]. During training, the RepConv module comprises a 3×3 convolution, a 1×1 convolution, and a residual connection, which collaboratively enhance multi-scale feature learning. At inference, these branches are merged into a single 3×3 convolution, enhancing efficiency while preserving accuracy. Additionally, we design REPAPoseHead, which incorporates a simplified RepConv to recover multi-scale detection capability affected by lightweighting. The full model integrating both components is named Wheel-YOLOv8n-pose.

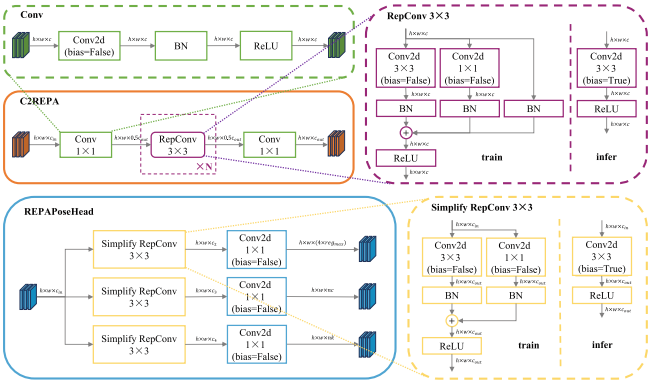


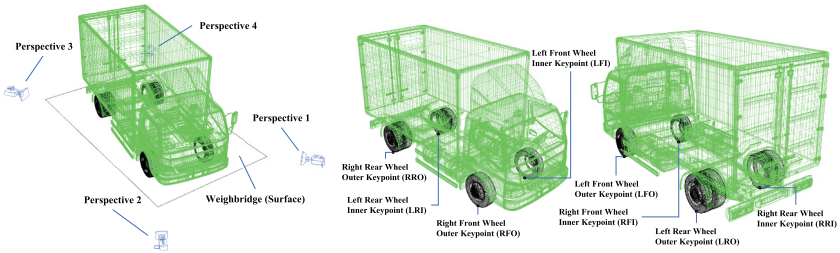
Fig. 2. Structure of C2REPA and REPAPoseHead Modules.

4.2 Monocular Based Vehicle Position Generation Method

Due to the high chassis of engineering vehicles, a suitable camera angle can be selected to capture three of the four wheels, as shown in Fig. 3(a). The camera angle must provide a complete view of the empty weighbridge surface. To ensure robust performance across multiple perspectives, eight wheel keypoints are defined-representing the inner and outer contact points of the front and rear wheels (see Fig. 3(b)). With the camera fixed after deployment, the four corners of the weighbridge can be pre-labeled in the image. Using these coordinates and the actual weighbridge dimensions, the homography matrix H is obtained via projective transformation. The projection of a keypoint is then computed as

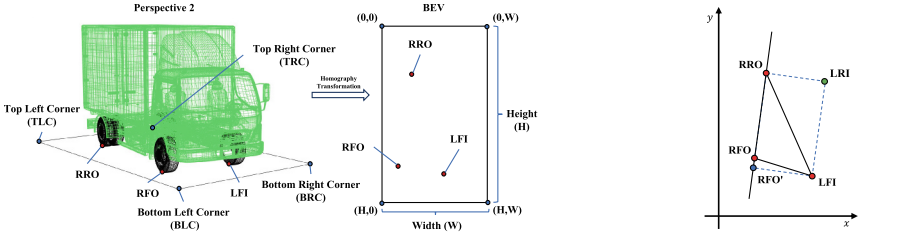
$$[x', y', z']^T = H \cdot [x, y, 1]^T \tag{2}$$

where (x, y) and $(\frac{x'}{z'}, \frac{y'}{z'})$ are the camera and transformed coordinates, respectively. Accordingly, keypoints RFO, RRO, and LFI (see Fig. 4(a)) are thus projected to BEV via Eq. 2.



(a) Camera Placement (b) Vehicle Wheel Keypoints

Fig. 3. Schematic of Camera Placement and Vehicle Wheel Keypoints.



(a) Keypoints Transformation Process (b) Coordinate Correction

Fig. 4. Wheel Keypoints Processing: From Transformation to Correction.

After calculating the coordinates of the wheel keypoints in the BEV, geometric error correction is applied. We model the four wheel-ground contact points as rectangle vertices. The three detected points are first adjusted to form a right-angled triangle, allowing the fourth vertex to be derived geometrically, as shown in Fig. 4(b). The corrected coordinates of point RFO and point LRI can be calculated as follows

$$RFO' = RRO + \frac{(LFI - RRO) \cdot (RFO - RRO)}{|RFO - RRO|^2} \cdot (RFO - RRO) \tag{3}$$

$$LRI = RRO + LFI - RFO' \tag{4}$$

5 Experimental Results

The experiment uses an Edge cluster comprising two OrangePi 5B (RK3588S, 4 GB RAM, 6 TOPS NPU) and one OrangePi AIPro (Ascend310, 8GB RAM, 8 TOPS NPU). The model was trained on 11,538 images using an NVIDIA 4080 Super GPU with SGD ($\mathrm{lr} = 0.01$, $\mathrm{batch} = 64$, $\mathrm{epochs} = 800$), where SiLU was

Table 1. Ablation Experiments and inference time comparison

Model	Activation	Module	Layers	Parameters (M)	GFLOPs	PCK_w	OKS_w	$mAP@50_{Box}$	Inference (ms)
YOLOv8n-pose	SiLU	None	187	3.120	8.49	0.9198	0.8778	0.9784	36 ± 1
YOLOv8n-pose	ReLU	None	187	3.120	8.49	0.9546	0.9000	0.9932	27 ± 1
YOLOv8n-pose	ReLU	C2REPA	175	2.313	6.53	0.9196	0.8632	0.9842	20 ± 1
YOLOv8n-pose	ReLU	REPAPoseHead	170	2.883	7.17	0.9694	0.9214	0.9931	25 ± 1
YOLOv8n-pose	ReLU	C2REPA + REPAPoseHead	158	2.076	5.21	0.9533	0.9057	0.9937	18 ± 1
YOLO11n-pose	ReLU	None	257	2.697	6.72	0.9576	0.9180	0.9910	50 ± 1
YOLOv5n+Hourglass	ReLU	None	1031	1080.124	241	0.7054	0.6230	0.9638	177 ± 5

replaced by ReLU for faster Edge inference. For deployment, models were converted to RKNN/OM format using hybrid quantization (int8 for well-performing layers, fp16 for others).

5.1 Ablation Experiments and Inference Time Comparison

Model performance was assessed using three metrics including weighted Percentage of Correct Keypoints (PCK_w), weighted Object Keypoint Similarity (OKS_w), and $mAP@50_{Box}$. The final score used to select model is

$$score = PCK_{weighted} \cdot 0.5 + OKS_{weighted} \cdot 0.4 + mAP@50_{Box} \cdot 0.1 \quad (5)$$

Ablation experiments (Table 1) show that Wheel-YOLOv8n-pose (YOLOv8n with C2REPA and REPAPoseHead) reduces parameters by 33.46% and FLOPs by 38.63%, achieving 33% faster inference (excluding pre/postprocessing) while preserving accuracy. Compared with YOLO11n-pose and YOLOv5n+Hourglass [11], it demonstrates clear advantages in both accuracy and speed.

5.2 Accuracy of Vehicle Indirect Localization

To evaluate the accuracy of wheel position prediction, a test set with 70 images of concrete mixer trucks, tankers, and trucks is used. Evaluation metrics include Normalized Center Error (NCE), Angle Error (AE), and Intersection over Union (IoU). NCE represents the normalized offset between the ground-truth and predicted vehicle centers. AE measures orientation discrepancy between ground-truth and predicted rectangles. IoU measures the interaction between two rectangles, which is the ratio of the intersection area to the union area of the predicted rectangle and the ground-truth rectangle. Overall, our localization method exhibits good accuracy with $NCE = 2.2\%$, $AE = 1.34°$ and $IoU = 78\%$.

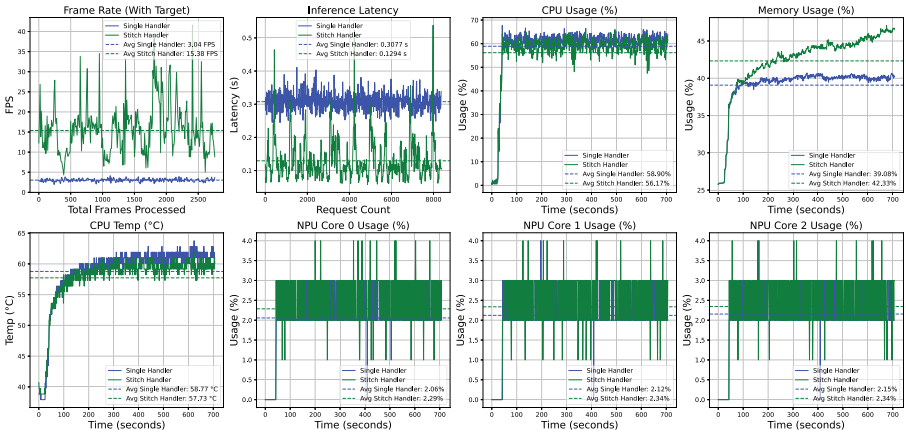


Fig. 5. Keypoint Detection Based on Wheel-YOLOv8n-pose.

5.3 Effectiveness of Parallel Video Stream Inference

To evaluate the effectiveness of MPI with frame stitching, we compared it against a baseline (Single Handler) which uses a separate thread for processing each video stream (totally four threads). Both methods employed three parallel inference threads bound to separate NPU cores. As shown in Fig. 5, our method enhances sampling rate, latency, and CPU efficiency by decreasing task arrival rates through frame stitching, though it increases memory consumption.

5.4 Comparison of Different Scheduling Strategies

To evaluate WLB's effectiveness, we jointly tested vehicle localization and image segmentation models (YOLOv8s-Seg and YOLOv8m-Seg). Ablation studies results (Table 2) compare four experimental groups with different metric weight combinations. Methods relying solely on latency [6] (Exp3), queuing length [13] (Exp4), or resource utilization (CPU, memory, or NPU) [10] (Exp2) exhibit higher response times and failure rates than our comprehensive approach (Exp1). This is because resource-only strategies ignore queuing conditions, while latency or queuing-based methods overlook resource competition on Edge devices. WLB is also compared to a round-robin baseline [6]. Figure 6 shows that our strategy reduces average response times across all three tasks.

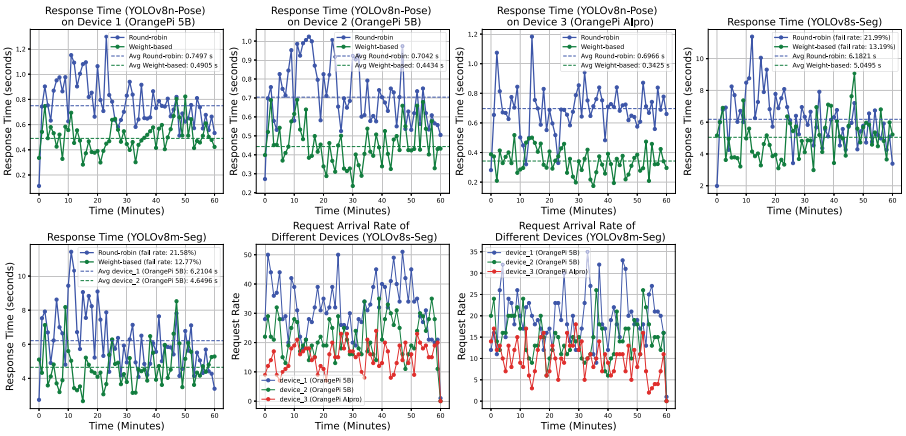


Fig. 6. Response Time of Different Scheduling Strategies.

Table 2. Ablation Study on Weight-based Scheduling Strategy Performance

Exp	Device	Model	W_{cpu}	W_{mem}	W_{npu}	W_T	W_R	W_Q	W_{task}	Response(s)	Fail Rate(%)
Exp.1	OrangePi 5B	Wheel-YOLOv8n-pose	0.7	0.4	0.02	0.01	0.8	3	1	0.4283	-
	OrangePi AIpro		0.6	0.4	0.01	0.01	0.8	3	1		
	OrangePi 5B	YOLOv8s-seg	0.7	0.4	0.02	0.01	0.3	9	2	**5.0495**	**13.19**
	OrangePi AIpro		0.6	0.4	0.01	0.01	0.3	9	2		
	OrangePi 5B	YOLOv8m-seg	0.7	0.4	0.02	0.01	0.1	12	4	**4.6496**	**12.77**
	OrangePi AIpro		0.6	0.4	0.01	0.01	0.1	12	4		
Exp.2	OrangePi 5B	Wheel-YOLOv8n-pose	0.7	0.4	0.02	0.01	-	-	-	0.7093	-
	OrangePi AIpro		0.6	0.4	0.01	0.01	-	-	-		
	OrangePi 5B	YOLOv8s-seg	0.7	0.4	0.02	0.01	-	-	-	6.3004	33.63
	OrangePi AIpro		0.6	0.4	0.01	0.01	-	-	-		
	OrangePi 5B	YOLOv8m-seg	0.7	0.4	0.02	0.01	-	-	-	5.8068	29.16
	OrangePi AIpro		0.6	0.4	0.01	0.01	-	-	-		
Exp.3	OrangePi 5B	Wheel-YOLOv8n-pose	-	-	-	-	0.8	-	-	0.3730	-
	OrangePi AIpro		-	-	-	-	0.8	-	-		
	OrangePi 5B	YOLOv8s-seg	-	-	-	-	0.3	-	-	5.5800	16.91
	OrangePi AIpro		-	-	-	-	0.3	-	-		
	OrangePi 5B	YOLOv8m-seg	-	-	-	-	0.1	-	-	5.0425	15.50
	OrangePi AIpro		-	-	-	-	0.1	-	-		
Exp.4	OrangePi 5B	Wheel-YOLOv8n-pose	-	-	-	-	-	3	-	**0.3272**	-
	OrangePi AIpro		-	-	-	-	-	3	-		
	OrangePi 5B	YOLOv8s-seg	-	-	-	-	-	9	-	5.7745	15.33
	OrangePi AIpro		-	-	-	-	-	9	-		
	OrangePi 5B	YOLOv8m-seg	-	-	-	-	-	12	-	5.4357	14.20
	OrangePi AIpro		-	-	-	-	-	12	-		

6 Conclusion

We propose an integrated solution for weighbridge unmanned Edge services, comprising a monocular-based lightweight model (Wheel-YOLOv8-pose) optimized for NPUs and a Multi-stream video Distributed Inference and Scheduling

framework (MDIS). Experimental results demonstrate that our model maintains detection accuracy while significantly improving inference speed, and the proposed framework effectively reduces service response times.

References

1. Alqahtani, D.K., Cheema, M.A., Toosi, A.N.: Benchmarking deep learning models for object detection on edge computing devices. In: International Conference on Service-Oriented Computing, pp. 142–150. Springer, Cham (2024)
2. Cheng, H., Zhang, M., Shi, J.Q.: A survey on deep neural network pruning: taxonomy, comparison, analysis, and recommendations. IEEE Trans. Pattern Anal. Mach. Intell. **46**(12), 10558–10578 (2024)
3. Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., Sun, J.: RepVGG: making VGG-style convnets great again. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13733–13742 (2021)
4. Gou, J., Yu, B., Maybank, S.J., Tao, D.: Knowledge distillation: a survey. Int. J. Comput. Vis. **129**(6), 1789–1819 (2021)
5. Hao, Y., Yang, S., Li, F., Zhang, Y., Wang, S., Ren, X.: Edgetimer: adaptive multi-timescale scheduling in mobile edge computing with deep reinforcement learning. In: IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, pp. 671–680 (2024)
6. Oo, T., Ko, Y.B.: Application-aware task scheduling in heterogeneous edge cloud. In: 2019 International Conference on Information and Communication Technology Convergence (ICTC), pp. 1316–1320 (2019)
7. Rokh, B., Azarpeyvand, A., Khanteymoori, A.: A comprehensive survey on model quantization for deep neural networks in image classification. ACM Trans. Intell. Syst. Technol. **14**(6), 1–50 (2023)
8. Tang, B., Yu, N., Han, F., Gong, J., Ge, Y.: Collaborative service placement and request scheduling in mobile edge networks. In: 2023 19th International Conference on Mobility, Sensing and Networking (MSN), pp. 730–735 (2023)
9. Tian, S., Xu, K., Xiang, S., Dai, X., Xiao, Z., Zeng, L.: Task offloading and resource scheduling in mobile edge-cloud computing based on edge competition and task prediction. IEEE Trans. Serv. Comput. 1–14 (2025)
10. Wang, Z., Zhang, X., Yang, L.: EIS: edge information-aware scheduler for containerized IoT applications. In: 2023 IEEE International Conference on Edge Computing and Communications (EDGE), pp. 280–289 (2023)
11. Xu, B., Liu, C.: Keypoint detection-based and multi-deep learning model integrated method for identifying vehicle axle load spatial-temporal distribution. Adv. Eng. Inform. **62**, 102688 (2024)
12. Xu, X., et al.: A review of multi-sensor fusion slam systems based on 3D lidar. Remote Sens. **14**(12), 2835 (2022)
13. Zhang, M., Cao, J., Yang, L., Zhang, L., Sahni, Y., Jiang, S.: Ents: an edge-native task scheduling system for collaborative edge computing. In: 2022 IEEE/ACM 7th Symposium on Edge Computing (SEC), pp. 149–161 (2022)

Author Index

A
Adams, Jan Niklas 287
Alonso, Juan C. 83
Alvarado-Valiente, Jaime 365
Anh, Nguyen Quoc 225
Aslan, Haci Ismail 149
Atwi, Hassan 306

B
Benzin, Janik-Vasily 175
Bouguettaya, Athman 122, 191
Brito, Eduardo 159

C
Cai, Zhicheng 415
Cao, Jian 138
Castillo, Fernando 159
Cavero, Francisco Javier 83
Chen, Jiaqi 415
Chen, Pengyu 92
Chen, Wei 92
Chen, Wenlong 321
Chen, Yuhan 35

D
de Leoni, Massimiliano 337
Deng, Yufeng 138
Di Ciccio, Claudio 268
Di, Kai 216

G
Gan, Shanyu 35
Gao, Xiaofeng 380
García, José María 19
Garcia-Alonso, Jose 365
García-Fernández, Alejandro 51
Geng, Wangyouqi 321
Goretti, Valerio 268
Guo, Taolin 207

H
Haou, Jie 67
Haque, Syed Muhammad Mahmudul 149
Heiss, Jonathan 159
Hu, Xianghui 216
Hu, Xingchen 399
Hua, Qin 138

J
Jiang, Yichuan 216
Jin, Yumeng 321
Jmaiel, Mohamed 3

K
Kao, Odej 149
Kirrane, Sabrina 268
Kumara, Indika 253

L
Lahyani, Imene 3
Li, Guiling 353
Li, Jiaqi 399
Li, Pan 216
Li, Siyu 321
Lin, Leilei 321
Lindenberg, Pierre 253
Liu, Hang 216
Liu, Hao 67
Liu, Peng 35
Liu, Zifeng 380
Lokuruge, Thilina 191
Luo, Fuchang 207
Luo, Jiayin 107

M
Ma, Yuxin 107
Mangler, Jürgen 175
Moguel, Enrique 365
Molino-Peña, Elena 19

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
M. Aiello et al. (Eds.): ICSOC 2025, LNCS 16321, pp. 425–426, 2026.
https://doi.org/10.1007/978-981-95-5015-9

N
Nghiem, Pham Thanh 225

O
Owotogbe, Joshua 253

P
Parejo, José Antonio 51, 83
Park, Gyunam 287, 337
Pautasso, Cesare 306
Pei, Zhihao 353
Peng, Jun 237
Peng, Shunshun 207
Pullonen-Raudvere, Pille 159

Q
Qian, Shiyou 138

R
Rinderle-Ma, Stefanie 175
Rizvi, Syeda Amna 122
Romero-Álvarez, Javier 365
Ruiz-Cortés, Antonio 19, 51, 83

S
Sanchez-Gil, Antonio Jesus 365
Sanou, Dia Jean Cédric 3
Schuster, Daniel 287
Sfaxi, Henda 3
Shen, Jun 399
Son, Ha Xuan 225

T
Tamburri, Damian Andrew 253
Tian, Kun 380
Tran-Truong, Phat T. 225
Truong, Hong-Linh 237
Tuan, Le Thanh 225

V
van den Heuvel, Willem-Jan 253
van der Aalst, Wil M. P. 337
Vinci, Francesco 337

W
Wan, Haipeng 321
Wang, Hongbing 207
Wang, Lie 35
Wang, Ran 67
Wang, Zhenyu 216
Wei, Jun 92
Werner, Sebastian 159
Witzke, Joel 149
Wu, Guoquan 92
Wu, Haonan 207
Wu, Qiang 67
Wu, Quanwang 207

X
Xu, Yiyun 67
Xue, Guangtao 138

Y
Yang, Danping 35
Yang, Dingyu 138
Yang, Mengmeng 207
Yangui, Sami 3
Yin, Jianwei 107

Z
Zhang, Kaixuan 138
Zhang, Liang 237
Zhang, Yunkang 67
Zhao, Xinkui 107
Zhao, Yuzhuo 380
Zhou, Wei 107
Zhu, Jingwei 237
Zhuang, Xinran 216